Машинное обучение

Проектирование систем от идеи до реализации



Валерий Бабушкин
Арсений Кравченко

MANNING

Machine Learning System Design

With end-to-end examples

Valerii Babushkin, Arseny Kravchenko



MANNING
Shelter Island

Машинное обучение

Проектирование систем от идеи до реализации

Валерий Бабушкин, Арсений Кравченко

ПИТЕР®

Санкт-Петербург • Москва • Минск

2026

Валерий Бабушкин, Арсений Кравченко

Машинное обучение. Проектирование систем от идеи до реализации

Перевела с английского В. Щербакова

ББК 32.973.2-018 +32.813
УДК 004.41+004.85

Бабушкин Валерий, Кравченко Арсений

Б12 Машинное обучение. Проектирование систем от идеи до реализации. — СПб.: Питер, 2026. — 416 с.: ил. — (Серия «Библиотека пользователя»).

ISBN 978-5-4461-4373-3

Проектирование и развертывание системы машинного обучения — это сложный многоэтапный процесс, требующий разнообразных компетенций и участия специалистов разных ролей. Независимо от того, интегрируете ли вы ML в существующий проект или проектируете ML-систему с нуля, вам необходимо ориентироваться в огромных датасетах и потоках, определять требования к тестированию и развертыванию, а также учитывать сложности внедрения ML-моделей в продакшен. Это практическое руководство по полному циклу разработки ML-проектов. Вы шаг за шагом пройдете весь фреймворк: проектирование, внедрение, релиз и сопровождение ML-системы. Чек-листы с необходимыми требованиями и примеры из реальной практики помогут развернуть и оптимизировать вашу собственную ML-систему. Вам особенно понравятся авторские байки и советы из личного опыта авторов, а также рекомендации по подготовке к System design интервью.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

ББК 32.973.2-018 +32.813
УДК 004.41+004.85

Права на издание получены по соглашению с Manning Publications. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

ISBN 9781633438750 англ.

ISBN 978-5-4461-4373-3

Authorized translation of the English edition © 2024 Manning Publications. This translation is published and sold by permission of Manning Publications, the owner of all rights to publish and sell the same.
© Перевод на русский язык ООО «Прогресс книга», 2025
© Издание на русском языке, оформление ООО «Прогресс книга», 2025
© Серия «Библиотека пользователя», 2025

Изготовлено в России. Изготовитель: ООО «Прогресс книга». Место нахождения и фактический адрес: 194044, Россия, г. Санкт-Петербург, Б. Сампсониевский пр., д. 29А, пом. 52. Тел.: +78127037373.

Дата изготовления: 02.2026. Наименование: книжная продукция. Срок годности: не ограничен.

Налоговая льгота — общероссийский классификатор продукции ОК 034-2014, 58.11.12 — Книги печатные профессиональные, технические и научные.

Импортер в Беларусь: ООО «ПИТЕР М», 220020, РБ, г. Минск, ул. Тимирязева, д. 121/3, к. 214, тел./факс: 208 80 01.

Подписано в печать 04.12.25. Формат 70×100/16. Бумага офсетная. Усл. п. л. 33,540. Тираж 500. Заказ 0000.

Краткое содержание

Часть 1. Подготовка к работе

Глава 1. Основы проектирования систем машинного обучения 24

Глава 2. В чем проблема? 40

Глава 3. Предварительное изучение 57

Глава 4. Дизайн-документ 77

Часть 2. Первые шаги

Глава 5. Функции потерь и метрики 96

Глава 6. Подготовка датасета 125

Глава 7. Схемы валидации 154

Глава 8. Базовая модель 178

Часть 3. Промежуточные этапы

Глава 9. Анализ ошибок 198

Глава 10. Пайплайны обучения 233

Глава 11. Признаки и генерация признаков 254

Глава 12. Измерение и представление результатов 290

Часть 4. Интеграция и масштабирование

Глава 13. Интеграция 322

Глава 14. Мониторинг и надежность 343

Глава 15. Сервинг и оптимизация инференса 377

Глава 16. Владение и сопровождение 399

Оглавление

От издательства **14**
О научном редакторе русского издания 14

Предисловие **15**

Благодарности **16**

О книге **18**
Для кого эта книга? 18
Структура книги 19
Форум liveBook 20

Об авторах **21**

Иллюстрация на обложке **22**

Часть 1. Подготовка к работе

Глава 1. Основы проектирования систем машинного обучения **24**
1.1. Что такое проектирование ML-систем? 25
1.1.1. Почему проектирование ML-систем так важно 30
1.1.2. Истоки проектирования ML-систем 30
1.2. Структура книги 32
1.3. Где могут быть полезны принципы проектирования ML-систем 35
Итоги 39

Глава 2. В чем проблема? **40**
2.1. Пространство проблем и пространство решений 41
2.2. Поиск проблемы 45
2.2.1. Как прийти к решению проблемы с помощью ML-системы 49

2.3. Риски, ограничения и возможные последствия ... 50
2.4. Цена ошибки ... 53
Итоги ... 55

Глава 3. Предварительное изучение ... 57
3.1. Какие проблемы способны вдохновлять? ... 58
3.2. Создать или купить — технологии с открытым исходным кодом или коммерческие ... 60
3.2.1. Создать или купить ... 60
3.2.2. Технологии с открытым исходным кодом или коммерческие решения ... 63
3.3. Декомпозиция проблемы ... 63
3.4. Выбор подходящего уровня инновации ... 68
3.4.1. Как найти эффективное решение? ... 70
3.4.2. Работа в пространстве решений: пример из практики ... 72
Итоги ... 75

Глава 4. Дизайн-документ ... 77
4.1. Распространенные мифы о дизайн-документе ... 78
4.1.1. Миф № 1. Дизайн-документы полезны в крупных компаниях, но не в стартапах ... 78
4.1.2. Миф № 2. Дизайн-документы нужны только в сложных проектах ... 79
4.1.3. Миф № 3. Дизайн-документ пишется по шаблону ... 79
4.1.4. Миф № 4. Каждый дизайн-документ должен описывать развернутую систему ... 80
4.2. Цели и антицели ... 81
4.3. Структура дизайн-документа ... 83
4.4. Ревью дизайн-документа ... 88
4.4.1. Пример ревью дизайн-документа ... 90
4.5. Дизайн-документ — это живой организм ... 92
Итоги ... 94

Часть 2. Первые шаги

Глава 5. Функции потерь и метрики ... 96
5.1. Функции потерь ... 97
5.1.1. Выбор функции потерь для моделей глубокого обучения ... 100

5.2. Метрики ... 103
5.2.1. Метрики согласованности ... 112
5.2.2. Офлайн- и онлайн-метрики, прокси-метрики и иерархия метрик ... 114
5.3. Дизайн-документ: добавляем функции потерь и метрики ... 117
5.3.1. Метрики и функции потерь для Supermegaretail ... 118
5.3.2. Метрики и функции потерь для PhotoStock Inc. ... 122
5.3.3. Вывод ... 124
Итоги ... 124

Глава 6. Подготовка датасета ... 125
6.1. Источники данных ... 126
6.2. Подготовка датасета ... 129
6.2.1. ETL ... 129
6.2.2. Фильтрация ... 130
6.2.3. Генерация признаков ... 131
6.2.4. Разметка ... 131
6.3. Данные и метаданные ... 136
6.4. Сколько нужно данных? ... 138
6.5. Проблема курицы и яйца ... 141
6.6. Критерий устойчивого пайплайна данных ... 143
6.7. Дизайн-документ: датасет ... 146
6.7.1. Датасет для Supermegaretail ... 147
6.7.2. Датасет для PhotoStock Inc. ... 150
Итоги ... 152

Глава 7. Схемы валидации ... 154
7.1. Достоверная оценка ... 155
7.2. Стандартные схемы ... 156
7.2.1. Отложенная выборка ... 156
7.2.2. Кросс-валидация ... 157
7.2.3. Выбор значения K ... 158
7.2.4. Валидация временных рядов ... 159
7.3. Нестандартные схемы ... 162
7.3.1. Вложенная валидация ... 163
7.3.2. Адверсариальная валидация ... 164
7.3.3. Оценка использования утечек данных ... 165

7.4. Процедура раздельного обновления ... 166
7.5. Дизайн-документ: выбор схем валидации ... 173
7.5.1. Схемы валидации для Supermegaretail ... 173
7.5.2. Схемы валидации для PhotoStock Inc. ... 176
Итоги ... 177

Глава 8. Базовая модель ... 178

8.1. Что такое базовая модель ... 179
8.2. Константные базовые модели ... 182
8.2.1. Зачем нужны константные базовые модели? ... 184
8.3. Базовые модели и базовые признаки ... 185
8.4. Базовые модели глубокого обучения ... 188
8.5. Сравнение базовых моделей ... 189
8.6. Дизайн-документ: базовые модели ... 192
8.6.1. Базовые модели для Supermegaretail ... 192
8.6.2. Базовые модели для компании PhotoStock Inc. ... 194
Итоги ... 195

Часть 3. Промежуточные этапы

Глава 9. Анализ ошибок ... 198

9.1. Анализ кривой обучения ... 199
9.1.1. Переобучение и недообучение ... 200
9.1.2. Кривая потерь ... 201
9.1.3. Анализ кривых потерь ... 202
9.1.4. Кривая обучения по модели ... 206
9.1.5. Кривая обучения по объему выборки ... 207
9.1.6. Двойной спуск ... 207
9.2. Анализ остатков ... 209
9.2.1. Цели анализа остатков ... 210
9.2.2. Предположения о модели ... 211
9.2.3. Распределение остатков ... 215
9.2.4. Справедливость остатков ... 217
9.2.5. Недооценка и переоценка ... 219
9.2.6. Кривые эластичности ... 219
9.3. Поиск закономерностей в остатках ... 222
9.3.1. Анализ наихудших/наилучших случаев ... 222
9.3.2. Адверсариальная валидация ... 223

9.3.3. Разнообразие группового анализа 224
9.3.4. Анализ крайних случаев 225
9.4. Дизайн-документ: анализ ошибок 226
9.4.1. Анализ ошибок для Supermegaretail 226
9.4.2. Анализ ошибок для компании PhotoStock Inc. 230
Итоги 232

Глава 10. Пайплайны обучения 233
10.1. Что такое пайплайн обучения? 233
10.1.1. Пайплайн обучения vs пайплайн инференса 235
10.2. Инструменты и платформы 238
10.3. Масштабируемость 241
10.4. Конфигурируемость 242
10.5. Тестирование 245
10.5.1. Тестирование на основе свойств 247
10.6. Дизайн-документ: пайплайны обучения 249
10.6.1. Пайплайн обучения для Supermegaretail 249
10.6.2. Пайплайн обучения для PhotoStock Inc. 251
Итоги 252

Глава 11. Признаки и генерация признаков 254
11.1. Что такое генерация признаков? 255
11.1.1. Критерии хороших и плохих признаков 256
11.1.2. Основы генерации признаков 258
11.1.3. Прогнозы модели как признак 260
11.2. Анализ важности признаков 261
11.2.1. Классификация методов 263
11.2.2. Компромисс между точностью и интерпретируемостью 265
11.2.3. Важность признаков в глубоком обучении 266
11.3. Отбор признаков 269
11.3.1. Генерация признаков vs отбор признаков 269
11.3.2. Цели и возможные недостатки 270
11.3.3. Методы отбора признаков 272
11.4. Хранилище признаков 276
11.4.1. Хранилище признаков: плюсы и минусы 277
11.4.2. Желательные свойства хранилища признаков 280
11.4.3. Каталог признаков 284

11.5. Дизайн-документ: генерация признаков 284
11.5.1. Признаки для Supermegaretail 284
11.5.2. Признаки для PhotoStock Inc 287
Итоги 289

Глава 12. Измерение и представление результатов 290
12.1. Измерение результатов 291
12.1.1. Эффективность модели 291
12.1.2. Переход к бизнес-метрикам 292
12.1.3. Среда моделирования 293
12.1.4. Оценка человеком 298
12.2. A/B-тестирование 298
12.2.1. Планирование эксперимента 299
12.2.2. Стратегия разделения 301
12.2.3. Выбор метрик 303
12.2.4. Статистические критерии 303
12.2.5. Моделирование экспериментов 304
12.2.6. Когда проведение A/B-тестирования невозможно 306
12.3. Представление результатов 306
12.3.1. Контрольные и вспомогательные метрики 307
12.3.2. Мониторинг прироста 307
12.3.3. Когда завершать эксперимент 308
12.3.4. Что указывать в отчете 309
12.3.5. Отчетный документ 310
12.4. Дизайн-документ: измерение и отчетность 311
12.4.1. Измерение и отчетность для Supermegaretail 311
12.4.2. Измерение и отчетность для компании PhotoStock Inc. 313
Итоги 320

Часть 4. Интеграция и масштабирование

Глава 13. Интеграция 322
13.1. Проектирование API 323
13.1.1. Работа с API 327
13.2. Релизный цикл 329
13.3. Эксплуатация системы 333
13.3.1. Технические связи 333
13.3.2. Нетехнические связи 334

13.4. Переопределения и фоллбэк-решения ... 334
13.5. Дизайн-документ: интеграция ... 336
13.5.1. Интеграция для Supermegaretail ... 336
13.5.2. Интеграция для PhotoStock Inc. ... 339
Итоги ... 342

Глава 14. Мониторинг и надежность ... 343
14.1. Зачем нужен мониторинг ... 344
14.1.1. Входные данные ... 345
14.1.2. Модель ... 345
14.1.3. Выход модели ... 346
14.1.4. Постобработка / Принятие решений ... 347
14.2. Работоспособность программного обеспечения ... 348
14.3. Качество и целостность данных ... 350
14.3.1. Проблемы обработки ... 350
14.3.2. Повреждение источника данных ... 351
14.3.3. Каскадные/вышестоящие модели ... 352
14.3.4. Изменение схемы данных ... 353
14.3.5. Перекос между обучением и эксплуатацией ... 353
14.3.6. Как осуществлять мониторинг и реагировать на проблемы ... 354
14.4. Качество и релевантность модели ... 358
14.4.1. Дрейф данных ... 361
14.4.2. Дрейф концепции ... 361
14.4.3. Как осуществлять мониторинг ... 363
14.4.4. Как реагировать ... 365
14.5. Дизайн-документ: мониторинг ... 371
14.5.1. Мониторинг для Supermegaretail ... 371
14.5.2. Мониторинг для PhotoStock Inc. ... 374
Итоги ... 376

Глава 15. Сервинг и оптимизация инференса ... 377
15.1. Сервинг и инференс: сложности ... 378
15.2. Компромиссы и паттерны ... 381
15.2.1. Компромиссы ... 381
15.2.2. Паттерны ... 383

15.3. Инструменты и фреймворки 385
15.3.1. Выбор фреймворка 385
15.3.2. Бессерверный инференс 388
15.4. Оптимизация пайплайнов инференса 390
15.4.1. Начинаем с профилирования 391
15.4.2. Минимальная оптимизация — наилучшая оптимизация 393
15.5. Дизайн-документ: сервинг и инференс 394
15.5.1. Сервинг и инференс для Supermegaretail 394
15.5.2. Сервинг и инференс для PhotoStock Inc. 395
Итоги 398

Глава 16. Владение и сопровождение 399
16.1. Ответственность 400
16.2. Фактор автобуса 405
16.2.1. Чем опасна чрезмерная эффективность? 406
16.2.2. Чем опасна чрезмерная избыточность? 407
16.2.3. Когда и как учитывать фактор автобуса 407
16.3. Документация 408
16.4. Сложность 410
16.5. Сопровождение и владение: Supermegaretail и PhotoStock Inc 414
Итоги 415

От издательства

Мы выражаем огромную благодарность компании КРОК за помощь в работе над русскоязычным изданием книги и вклад в повышение качества переводной литературы.

Ваши замечания, предложения, вопросы отправляйте по адресу comp@piter.com (издательство «Питер», компьютерная редакция).

Мы будем рады узнать ваше мнение!

На веб-сайте издательства www.piter.com вы найдете подробную информацию о наших книгах.

О научном редакторе русского издания

Мария Силинская — старший консультант по трансформации бизнеса в компании КРОК, сертифицированный скрам-мастер PSM II. Более 16 лет опыта в ИТ. Занималась разработкой тренажерных систем на базе реалистичного моделирования событий в виртуальной среде. Участвовала в создании систем для государственного и коммерческого сектора, включая проекты по внедрению платформ для обработки больших объемов неструктурированной информации в режиме реального времени. Вела семинары и лабораторные работы по нейронным сетям в НИЯУ МИФИ.

Предисловие

В начале карьеры наши рабочие задачи — Арсения в онлайн-маркетинге и Валерия в хемометрике — напрямую не были связаны с машинным обучением (ML). Тем не менее именно применяемые нами математические инструменты, такие как регрессионные модели и метод главных компонент, пробудили в нас страсть к извлечению максимальной пользы из данных. Мы оба начали свой путь в начале 2010-х: Валерий впоследствии занимал руководящие должности в области data science в таких компаниях, как Facebook, Alibaba, Blockchain.com и BP, а Арсений оттачивал инженерные навыки в deep-tech[1]-стартапах на разных стадиях развития.

До начала совместной работы над этой книгой нас связывало лишь одно — участие в соревнованиях по ML, когда мы совершенствовали свои навыки на платформе Kaggle. Валерий получил статус Grandmaster и ранее входил в тридцатку лучших участников по всему миру. Арсений — Kaggle Master с широким опытом в соревновательном ML. Оба автора активно делятся своими знаниями, выступая на конференциях и мероприятиях, посвященных ML.

Наша совместная работа сочетает практический опыт Арсения по созданию и оптимизации ML-систем с видением и управленческим подходом Валерия в сфере data-driven-решений (решений, управляемых данными). От обработки видеопотока в режиме реального времени и оптимизации ритейла до анализа финансовых транзакций — мы постарались обобщить все накопленные нами знания и сформулировать ключевые принципы построения рабочих систем ML.

Несмотря на различия в карьерах, мы оба пришли к пониманию существующего разрыва между знанием алгоритмов ML и умением эффективно применять их в реальных задачах. Мы не раз наблюдали, как талантливые специалисты испытывают трудности в объединении разрозненных знаний в единую логичную систему. Именно это и вдохновило нас на написание книги.

---

[1] DeepTech (от англ. deep technologies — «глубокие технологии») — компании, чаще стартапы, цель которых — предложить глобальные технологические решения, требующие долгих научных исследований и больших инвестиций. — *Примеч. ред.*

Благодарности

Эта книга не появилась бы на свет без активной поддержки наших друзей и коллег.

Мы выражаем искреннюю благодарность Богдану Печенкину за его вклад в подготовку черновых глав, а также Игорю Котенкову, Рустему Фейзханову, Евгению Макарову и Адаму Елдарову за рецензирование книги в процессе ее написания.

Особую признательность мы выражаем Семену Козлову и Сэму Вайссу (Sam Weiss), чей подход к решению проблем послужил вдохновением для многих разделов этой книги, Сергею Форису — за вклад в редактуру, Татьяне Путиловой — за создание визуальных материалов, одновременно эстетичных и информативных.

Благодарим нашего научного редактора Иоанниса Атсониоса (Ioannis Atsonios), обладающего опытом работы в науке, консалтинге и IT-индустрии, в том числе в создании концепций data-продуктов, разработке прототипов, вводе решений в продакшен, а также в области научных исследований и разработок в ML, в частности, систем персонализации.

Мы выражаем признательность всем нашим рецензентам: Алексею Агаркову, Антониосу Цалтасу (Antonios Tsaltas), Ариджиту Дасгупте (Arijit Dasgupta), Крейгу Хендерсону (Craig Henderson), Динешу Гханте (Dinesh Ghanta), Флэолю Фредерику (Flayol Frédéric), Джорджу Онофрею (George Onofrei), Константину Клякхандлеру (Konstantin Kliakhandler), Лакшминарянану А. С. (Lakshminarayanan A. S.), Лауре Ускагеги (Laura Uzcategui), Лусиану М. Сасу (Lucian M. Sasu), Максиму Волгину, Микаэлю Дотрею (Mikael Dautrey), Майку Райту (Mike Wright), Мирерфану Гейби (Mirerfan Gheibi), Никосу Канакарасу (Nikos Kanakaris), Нинославу Черкецу (Ninoslav Cerkez), Одиссеасу Пентакалсо (Odysseas Pentakalso), Оливеру Кортену (Oliver Korten), Прашанту Каушику (Prashant Kowshik), Роберту Диане (Robert Diana), Шрираму Мачарле (Sriram Macharla), Стивену Джону Варнетту (Stephen John Warnett), Стипе Чувало (Stipe Cuvalo), Ватсалу Десаи (Vatsal Desai), Вишну Раму Венкатараману (Vishnu Ram Venkataraman) и Уильяму Джамиру Силве (William Jamir Silva).

Спасибо за ваши замечания и ценные идеи, которые помогли улучшить книгу и придать ей четкость.

Благодарим также первых читателей черновика и студентов наших курсов по проектированию ML-систем — ваши вопросы и предложения сыграли важную роль в создании этой книги.

Мы искренне благодарим наши семьи и друзей, чье терпение и понимание позволили нам довести проект до конца. Мы признательны вам за вашу неизменную поддержку, без которой эта книга не появилась бы на свет.

И наконец, огромная благодарность команде издательства Manning за возможность опубликовать нашу книгу и за поддержку на каждом этапе ее создания.

О книге

«Машинное обучение. Проектирование систем от идеи до реализации» — это подробное пошаговое руководство, призванное помочь вам на всех этапах разработки ML-системы: от сбора информации и выполнения предварительных шагов до разработки, запуска в эксплуатацию и последующего сопровождения.

Как следует из названия, книга посвящена именно проектированию ML-систем. Мы не сосредоточиваемся на какой-то одной технологии, а предлагаем универсальный фреймворк для решения задач, связанных с созданием, сопровождением и совершенствованием ML-систем различного масштаба и сложности.

В условиях стремительного роста значимости машинного обучения и искусственного интеллекта (ИИ) уже существует множество книг и курсов, посвященных алгоритмам, различным предметным областям и отдельным аспектам проектирования. Однако они не дают целостного представления о данном процессе, что ведет к проблеме, с которой Арсений и Валерий сталкивались во многих компаниях: способные инженеры успешно разрабатывают отдельные компоненты, которые, к сожалению, не складываются в надежную и целостную систему. Наша книга, среди прочего, стремится восполнить данный пробел.

Эта книга не для начинающих. Мы предполагаем, что читатель уже знаком с основами ML (может понять учебник уровня бакалавриата) и обладает прикладными навыками программирования (имеет опыт решения реальных задач вне учебной среды).

Для кого эта книга?

Мы надеемся, что наша книга пригодится:

- мидл-разработчикам — для совершенствования навыков создания и сопровождения устойчивых ML-систем без каких-либо критических упущений;
- техническим менеджерам и сеньор-разработчикам — для восполнения пробелов в знаниях и формирования более широкого взгляда на проектирование ML-систем;

- тем, кто делает свои первые шаги в прикладном ML, — чтобы получить структурированные рекомендации перед началом разработки своей первой ML-системы.

Структура книги

Книга представляет собой чек-лист или практическое руководство с парой «баек у костра» из нашего собственного опыта. Ее можно прочитать целиком или же обращаться к ней по мере необходимости при работе над конкретным аспектом ML-системы. При этом мы старались избежать сухого академического стиля, характерного для типичных учебников или курсов по классическому ML и глубокому обучению (deep learning).

Мы разделили книгу на четыре основные части, соответствующие жизненному циклу любой системы:

- исследование;
- создание базовой версии;
- улучшение;
- сопровождение.

Главы 1–8 посвящены ранним этапам проектирования ML-систем. В главах 1–4 мы акцентируем внимание на общем понимании проблемы, которую предстоит решить системе, и определяем действия, которые необходимо выполнить до начала ее разработки. Данный этап редко включает написание кода и в основном фокусируется на создании небольших прототипов или доказательствах концепций.

Главы 5–8 углубляются в технические детали начальных этапов работы. Данный период требует проведения исследований и взаимодействия с коллегами, что крайне важно для понимания проблемы, построения карты возможных решений и согласования ожиданий с другими участниками проекта. Если сравнивать ML-систему с человеческим телом, то это этап формирования ее скелета.

Главы 9–12 посвящены промежуточным этапам жизненного цикла системы, когда действия инженеров, как правило, резко меняются: исследований и обсуждений становится меньше, а на первый план выходит практическая работа по реализации и улучшению системы. Мы сосредоточимся на вопросах создания устойчивой, точной и надежной системы. Продолжая метафору с телом, здесь система наращивает «мышцы».

Заключительная часть, включающая главы 13–16, посвящена интеграции и масштабированию. Для неподготовленного наблюдателя может показаться, что система уже готова к запуску, но такое впечатление обманчиво. Существует еще множество моментов (в основном инженерных), которые необходимо учесть, прежде чем система будет успешно запущена в эксплуатацию. В мире ПО сбой

редко становится непоправимой катастрофой, как, например, происходит в строительстве, но этого все равно хочется избежать. На данном этапе вы узнаете, как сделать свою систему надежной, сопровождаемой и устойчивой к будущим изменениям. И если вы еще не устали от метафор, связанных с человеческим организмом, то можно сказать, что здесь система обретает свой «разум», ведь неуправляемая сила нередко приводит к негативным последствиям.

Форум liveBook

Приобретая книгу *«Машинное обучение. Проектирование систем от идеи до реализации»*, вы также получаете бесплатный доступ к платформе для онлайн-чтения liveBook издательства Manning (на английском языке). Эксклюзивные возможности liveBook позволяют оставлять комментарии как к книге в целом, так и к отдельным ее разделам или абзацам. Можно легко делать заметки для себя, задавать технические вопросы и отвечать на них, а также получать помощь от авторов и других пользователей. Чтобы получить доступ к форуму, посетите страницу https://livebook.manning.com/book/machine-learning-system-design/discussion. Информацию о форумах Manning и правилах поведения на них см. на https://livebook.manning.com/discussion.

В рамках своих обязательств перед читателями издательство Manning предоставляет ресурс для содержательного общения читателей и авторов. Эти обязательства не подразумевают конкретную степень участия авторов, которое остается добровольным (и неоплачиваемым). Задавайте авторам хорошие вопросы, чтобы им было интересно участвовать в диалоге! Форум и архивы обсуждений доступны на сайте Manning, пока книга продолжает издаваться.

Об авторах



Арсений Кравченко — опытный ML-инженер с солидным послужным списком в создании и оптимизации надежных ML-систем для самых различных отраслей. В настоящее время Member of Technical Staff в Databricks. Как Kaggle Master он известен своим опытом в соревновательном машинном обучении и любит делиться своими познаниями с другими. Арсений часто выступает в роли консультанта по вопросам ИИ и ML для венчурных фондов и стартапов на ранней стадии развития.



Валерий Бабушкин — выдающийся специалист data science с богатым опытом работы в IT. В настоящее время он занимает позицию Senior Principal[1] в BP, где отвечает за реализацию data-driven-стратегий. До прихода в BP Валерий занимал ключевые должности в таких технологических гигантах, как Facebook, Alibaba и Blockchain.com. Помимо профессиональных достижений, Валерий — Kaggle Grandmaster и ранее входил в топ-30 участников по всему миру, что подтверждает его высокий уровень навыков и компетентности в области data science.

[1] Примерный перевод — «старший принципал-разработчик», но в статьях в интернете его называют «старший главный начальник». — *Примеч. ред.*

Иллюстрация на обложке

Иллюстрация на обложке книги «Машинное обучение. Проектирование систем от идеи до реализации» под названием «Les Pauvres» («Бедняки») взята из книги Луи Курмера, изданной в 1841 году. Каждая иллюстрация этой книги тщательно прорисована и раскрашена от руки.

В прежние времена по одежде человека можно было легко определить, где он живет и какова его профессия или положение в обществе. Издательство Manning приветствует изобретательность и инициативность IT-индустрии и в знак этого размещает на обложках изображения, которые демонстрируют богатое разнообразие региональных культур, запечатленное на старинных рисунках.

Часть 1

Подготовка к работе

Эта часть книги посвящена начальным этапам проектирования систем машинного обучения. В главе 1 представлено наше видение проектирования ML-систем, а также областей, в которых могут пригодиться его принципы. В главе 2 подробно описано пространство проблем и пространство решений, а также подчеркивается абсолютный приоритет первого над вторым. Там же рассказывается об определении проблемы, рисках, ограничениях и цене ошибки. Глава 3 анализирует дилемму «создать или купить», отвечает на вопрос, может ли проблема стать источником вдохновения, и предлагает наш взгляд на декомпозицию проблем. Глава 4 посвящена дизайн-документу как основе любой ML-системы.

1 Основы проектирования систем машинного обучения

В этой главе

- ✓ Что такое проектирование систем машинного обучения (ML system design), почему сложно дать определение этому процессу и где вы с ним можете впервые столкнуться
- ✓ Кто, по нашему мнению, получит наибольшую пользу от прочтения этой книги, какую информацию мы собираемся вам дать и как она будет структурирована
- ✓ Какие принципы проектирования ML-систем могут быть полезны и когда их лучше всего применять

Проектирование систем машинного обучения (ML system design) — это относительно новый термин, который часто вызывает замешательство. Бывает сложно сформулировать четкий круг обязанностей, стоящих за данным понятием, не говоря уже о попытках подобрать правильное название для соответствующей роли или должности. Эту работу могут выполнять как ML-инженеры, так и программисты-разработчики или даже специалисты data science в зависимости от обязанностей и функций роли, при этом уровень эффективности каждого из них будет разным.

Хотя все эти позиции существуют, мы считаем, что для того, чтобы стать опытным специалистом в проектировании ML-систем, необходимо объединить знания и навыки каждой из этих областей. И хотя некоторые моменты, рассматриваемые

в этой книге, специфичны для ML, другие же будут знакомы и читателям, разрабатывающим программные системы без ML (такая информация представлена в главах 2, 13 и 16). Это связано с тем, что, несмотря на свою новизну, проектирование ML-систем по-прежнему основывается на классических принципах разработки программного обеспечения.

Но прежде чем углубляться в детали, необходимо понять, что такое проектирование систем машинного обучения в целом. В этой вводной главе мы предложим наше определение этого понятия и подкрепим его примерами как из личного опыта, так и опыта наших коллег. Мы также рассмотрим навыки идеального специалиста в этой области и поделимся примерами из практики, демонстрирующими, почему последовательный и согласованный подход к проектированию ML-систем позволяет значительно сэкономить время в долгосрочной перспективе и при этом способствует достижению краткосрочных бизнес-результатов, что особенно важно для завоевания доверия коллег к этому новому методу работы с ранних этапов.

1.1. Что такое проектирование ML-систем?

Это понятие может быть вам знакомо, если вы когда-либо проходили собеседования в Deep Tech или Big Tech компаниях на должность ML-инженера или ML-менеджера. Понятие «Deep Tech» обычно относится к стартапам или R&D-отделам (Research and Development, отдел исследований и разработок) в крупных корпорациях, которые работают с самыми передовыми технологиями. Термин «Big Tech», в свою очередь, характеризует крупнейшие и наиболее влиятельные технологические компании в мире, известные своим высоким уровнем требований к сотрудникам и развитой инженерной культурой. Авторы этой книги имеют солидный опыт работы в сфере Deep Tech, поэтому, загоревшись идеей написать эту книгу, мы были уверены, что определение понятия, вынесенного в заголовок данного раздела, всем хорошо известно и не требует каких-либо дополнительных разъяснений.

Однако после обсуждения концепции книги с несколькими специалистами мы заметили, что интерпретация термина вызывает ряд разногласий. Возможно, это связано с тем, что в индустрии давно сложился четкий перечень должностей, позволяющий кандидатам достаточно точно понимать, какие обязанности и функции предполагает та или иная роль. Такие должности, как инженер-программист (software engineer), инженер-исследователь (research engineer), инженер по машинному обучению (ML engineer) и др., подразумевают выполнение определенного набора задач, описанных в учебниках и четко определенных в описании вакансий.

Отсюда возникает вопрос: существует ли отдельная должность, напрямую связанная с проектированием ML-систем? В настоящее время не существует позиции, полностью охватывающей весь объем работы, который мы будем

рассматривать в этой книге. Но если вы все же встретите специалиста, выполняющего все перечисленные задачи, его должность, скорее всего, будет обозначена как «специалист data science».

Пытаясь разобраться в причинах такой путаницы, мы пообщались с несколькими специалистами, работающими в сфере data science, и пришли к выводу, что эта роль подразумевает широкий и довольно размытый круг обязанностей. Поэтому если спросить десять специалистов data science, работающих в десяти разных компаниях, чем они занимаются, можно получить десять совершенно разных ответов:

- создание сводных таблиц в Excel;
- настройка распределенного кластера объемом в 10 Пбайт;
- разработка системы компьютерного зрения в режиме реального времени;
- развертывание множества чат-ботов;
- визуализация данных в Tableau/Metabase/Looker/PowerBI;
- написание SQL-скриптов;
- проведение A/B-тестирования;
- разработка рекомендательных систем;
- взаимодействие со стейкхолдерами;
- ответы на вопросы высшего руководства.

Как видно из этого списка, короткое и лаконичное название «специалист data science» объединяет в себе довольно разнородный набор задач. Фактически эта должность стала универсальным термином для всех видов деятельности, выходящих за рамки традиционных ролей инженера данных, ML-инженера и инженера-исследователя.

Однако в случае проектирования ML-систем ситуация прямо противоположная: существует нечто, пока не имеющее общепринятого названия, но при этом обладающее четко определенным набором функций и обязанностей. В данном случае наша задача заключается в том, чтобы систематизировать этот набор и представить в виде организованной структуры взаимосвязанных процессов.

В последующих главах мы предложим свой взгляд на проектирование ML-систем и даже представим несколько нестандартных идей и решений. Однако прежде чем углубляться в детали, рассмотрим само определение:

> Проектирование ML-систем — это сложный, многоэтапный процесс проектирования, внедрения и *сопровождения* систем на основе машинного обучения. Он включает в себя сочетание методов и навыков из различных областей, таких как машинное обучение, разработка программного обеспечения, проджект-менеджмент, продакт-менеджмент и управление.

Рисунок 1.1 иллюстрирует это определение.



Рис. 1.1. Навыки, необходимые для успешного проектирования ML-систем

Мы выделили слово *«сопровождение»* курсивом, так как считаем, что проектирование ML-систем не заканчивается на этапе релиза. Помимо обеспечения точности прогнозов и эффективного принятия решений, ваша система должна быть масштабируемой и достаточно гибкой, чтобы легко адаптироваться к изменяющимся бизнес-условиям или другим факторам — как внутренним, так и внешним. Таким образом, сразу после запуска вашей ML-системы в эксплуатацию ее сопровождение и тонкая настройка обеспечат ее эффективность в долгосрочной перспективе, что особенно важно в условиях ограниченного бюджета или небольших вычислительных мощностей.

Однако у тех, кто ознакомился с аннотацией книги или просмотрел ее содержание, возникли вопросы не только относительно самого термина *«проектирование систем машинного обучения»*. Мы также получили множество вопросов, касающихся различных аспектов книги, и ниже приведены наиболее интересные из них:

- «Специалист data science, ML-инженер и инженер-программист — это разные роли. Почему вы их объединяете?»
- «Мне непонятно, зачем книга о ML-системах затрагивает такие темы, как сбор данных и отчетность, ведь именно это отличает классическое машинное обучение от data science».
- «Я удивлен, почему в оглавлении нет упоминания MLOps — распространенного в индустрии термина, включающего в себя многие описанные вами компоненты (воспроизводимость, тестирование, пайплайны и т. д.)».

Для нас эти вопросы стали еще одним подтверждением существования путаницы между ML и data science, а также между ML-инженерами и специалистами data science. У нас есть своя точка зрения относительно этого вопроса, но сначала давайте попробуем прояснить нашу позицию.

Работая в Deep Tech компаниях, мы привыкли называть специалистов, занимающихся ML, *ML-инженерами*. Однако граница между ML-инженером

и инженером-программистом становится все более размытой, особенно с учетом того, что некоторые ведущие специалисты в данной области называют машинное обучение *«программным обеспечением 2.0»* (https://mng.bz/yoNe). В то же время должность *специалиста data science* чаще всего ассоциируется с продуктовыми аналитиками, работающими с метриками, инсайтами и подобными задачами. (Отметим, что мы опираемся на наш опыт работы в Deep Tech компаниях, но поскольку подобные компании нанимают тысячи высококвалифицированных специалистов, тем самым формируя стандарты для всей индустрии, мы рассматриваем их подход в качестве эталона.)

Когда кандидаты претендуют на вакансию ML-инженера, им, как правило, приходится проходить весь стандартный процесс отбора для инженеров-программистов, включающий дополнительные этапы. Ключевой из подобных этапов — проектирование ML-системы. Он нужен для того, чтобы оценить уровень навыков кандидата, его компетентность, а также способность анализировать сложные системы и декомпозировать их на взаимосвязанные блоки задач. Это непростое испытание, поскольку у соискателя есть всего 40–45 минут, чтобы представить свой проект системы, случайно выбранной экспертом, проводящим собеседование.

Элиезер Юдковский (Eliezer Yudkowsky), писатель и философ в области ИИ, писал: «Самая опасная привычка мышления, прививаемая еще в школе, состоит в том, что даже если вы чего-то не поняли, то все равно должны это повторить, как попугай» (https://mng.bz/r1Zy). Данное утверждение хорошо применимо к процессу технических собеседований в некоторых компаниях: интервьюер дает задачу и ожидает, что соискатель просто повторит заранее известный ответ. После того как интервьюируемый получает работу и затем сам становится интервьюером, эта вредная практика закрепляется и компания продолжает нанимать людей с идеально заученными фрагментарными знаниями из разных областей. Нет никакой гарантии, что эти люди понимают общую картину, и мы сами столкнулись с этим, когда проводили собеседования.

Мы проводили собеседования и нанимали ML-инженеров для различных компаний. Среди них были как новички, так и опытные специалисты и сильные инженеры-программисты, решившие перейти в ML. И у всех тех, кто не прошел собеседование, наблюдалась одна общая черта — в процессе проектирования ML-системы они зацикливались на деталях, не видя общей картины.

Для нас такое поведение указывало на несоответствие ожиданий: будучи молодыми менеджерами по найму, мы были уверены, что человек, знающий все алгоритмы, инструменты и шаблоны, автоматически будет нам подходить. Однако со временем мы поняли, что иногда люди просто не могут объединить все свои знания в какую-то цельную картину.

Кроме того, разработка системы в реальных условиях кардинально отличается от ее обсуждения на собеседовании. Можно заучить ответы на десятки популярных вопросов по проектированию ML-систем («Как бы вы спроектировали систему

рекомендаций вакансий для сайта наподобие LinkedIn?») и все равно не справиться с аналогичной задачей в реальной работе.

Но давайте на время оставим тему собеседований. Специалистов по ML нанимают не просто так — компании нуждаются в них для создания, сопровождения, эксплуатации и улучшения систем, а не ради написания кода или закрытия тикетов в Jira. Бизнесу требуются надежные ML-системы для достижения определенных целей и решения конкретных задач.

Разработка ML-систем требует широкого набора навыков. Говоря кратко, человек, ответственный за это, должен быть способен ответить на три вопроса:

1. Что мы создаем?
2. Какова цель системы?
3. Как она должна быть построена?

На практике это требует сочетания навыков из нескольких областей: частично продакт-менеджмента — чтобы понимать основную цель и уметь донести ее до коллег и стейкхолдеров, частично ML-исследователя — для усиления системы и, конечно, создания прочного ПО-фундамента, чтобы продукт был удобным, сопровождаемым и надежным. Эксперт по проектированию ML-систем должен мыслить масштабно и, при необходимости, детально разбираться в отдельных аспектах.

Мало кто обладает всеми этими навыками на должном уровне. Однако сегодня создается множество ML-систем, а значит, их кто-то проектирует. Исходя из нашего опыта, проектирование ML-систем обычно поручается либо сильному специалисту в ML (потому что это ML), либо опытному инженеру-программисту (потому что это система). Они выполняют поставленные перед ними задачи, но зачастую сталкиваются с трудностями в тех областях, где не являются экспертами.

Подводя итог, можно сказать, что вся путаница вокруг проектирования ML-систем чаще всего возникает у кандидатов с недостаточным опытом, а также у менеджеров по найму или рекрутеров, которые ищут универсального специалиста. Однако если посмотреть на ситуацию с точки зрения руководителя или эксперта, картина становится гораздо шире. Они понимают, что специалистов в этой области нанимают для создания, сопровождения и улучшения ML-систем, и результат их работы с этими системами становится главным критерием их карьерного роста.

Мы считаем, что эксперт в области проектирования систем машинного обучения сочетает навыки специалиста по data science и инженера-программиста с опытом работы в академической сфере ML. Сотрудники, занимающиеся проектированием ML-систем, могут иметь различный профессиональный бэкграунд — разработка программного обеспечения, прикладное ML, академическое ML, исследование данных и т. д. Мы надеемся, что наш практический опыт

в сочетании с небольшими теоретическими вставками поможет таким специалистам устранить пробелы в знаниях, систематизировать их и почувствовать себя увереннее в тех сферах, где им не хватает опыта.

1.1.1. Почему проектирование ML-систем так важно

Хотя MLOps предоставляет набор инструментов для создания и сопровождения ML-систем, их проектирование можно рассматривать как фундаментальную архитектурную схему, на которую всегда можно опереться и которая обеспечивает необходимую масштабируемость и гибкость (глубокое понимание строительных блоков системы и их взаимосвязей помогает выявлять возможные проблемы и эффективно их решать). Однако самое главное — она задает фреймворк, связывающий всю систему воедино.

Некоторые проекты достаточно просты и не требуют такого скрупулезного подхода. Проведем аналогию со строительством: скорее всего, сарай можно построить и без предварительного плана. Но если речь идет о доме или небоскребе, обойтись без заранее подготовленного детализированного проекта уже невозможно. Проектирование ML-систем — это архитектурный подход к инженерии ML-систем, объединяющий опыт сотен специалистов, работавших в десятках компаний над множеством различных проектов.

1.1.2. Истоки проектирования ML-систем

Разработка сложных программных систем всегда была непростой задачей, и компании были вынуждены как-то структурировать этот процесс. В основе управления сложностью лежит принцип абстракции — сначала создаются низкоуровневые модули с инкапсулированной в них сложностью, затем они используются как «черные ящики» при построении более высокоуровневых компонентов, и т. д.

Такой метод работал, но имел уязвимое место — кто-то должен был определить структуру всех этих компонентов (какие компоненты являются высокоуровневыми, какова их внутренняя организация, какие существуют уровни детализации до самого низкого уровня реализации). Наиболее ответственные решения принимали архитекторы ПО — опытные инженеры, работавшие с множеством различных систем.

Такой подход традиционно ассоциируется с методологией Waterfall («водопадный подход») и парадигмой Big Design Upfront («глобальное проектирование прежде всего»). Другими словами, он предполагает, что разработка ПО начинается постепенно и проходит тщательный анализ и документирование еще до написания первой строки кода. Данный метод зарекомендовал себя как весьма надежный, но в то же время инерционный и бюрократический. В условиях быстро развивающегося мира проект может потерять актуальность еще до своего завершения.

Противники такого медленного, но основательного подхода — сторонники гибкой методологии разработки программного обеспечения. Авторы *Agile-манифеста* (Manifesto for Agile Software Development) (https://agilemanifesto.org/) сформулировали четыре ключевые ценности:

- Люди и взаимодействие важнее процессов и инструментов.
- Рабочий продукт важнее подробной документации.
- Сотрудничество с заказчиком важнее согласования условий контракта.
- Готовность к изменениям важнее следования первоначальному плану.

Другими словами, сторонники данного метода вполне обоснованно утверждают, что многие программные системы не могут быть эффективны, если их создатели пытаются заранее спланировать и задокументировать абсолютно всё. Конечно, в некоторых случаях такая бюрократия оправданна, например, при разработке ПО для управления медицинскими приборами или самолетами. Однако большинство программистов работают над другими типами приложений — офисным ПО, развлекательными сервисами, веб-сайтами и мобильными приложениями. Именно поэтому роль архитектора ПО стала ассоциироваться с чем-то медленным и устаревшим — полной противоположностью быстрым хакерам, которые меняют мир, не дожидаясь утверждения технической спецификации всей иерархией архитекторов, менеджеров и других экспертов. Такой гибкий подход приобрел популярность после успеха компаний Кремниевой долины и тысяч других стартапов. Даже такие крупные компании, как Meta, стараются придерживаться подобной культуры — их корпоративный девиз гласит: «Действуй быстро, ломай все» (Move fast and break things).

Подведем итог этому небольшому историческому обзору: в какой-то момент индустрия столкнулась с двумя крайностями в разработке ПО — от строго регулируемого, управляемого архитекторами процесса до хаотичного, анархического стиля «к черту иерархию». И как это часто бывает, постепенно два подхода начали перемешиваться. И теперь более традиционные компании стремятся к максимальной гибкости, а анархичные стартапы со временем взрослеют, внедряя процессы и вводя отдельные роли.

Такая смесь привела к консенсусу, который на данный момент установился в технологических компаниях: вместо того чтобы передавать все решения отдельным специалистам, таким как, например, архитекторы ПО, ответственность за проектирование возлагается на рядовых инженеров-разработчиков. Они не только пишут код, но и проектируют системы, над которыми работают. Однако такая свобода не устранила изначальную потребность в четких архитектурных решениях — кто-то все равно должен принимать окончательные решения о структуре системы и нести ответственность за ее проектирование. Конечно, каждый инженер может в той или иной степени участвовать в этом процессе, но важно, чтобы кто-то видел всю картину целиком.

Навыки реализации низкоуровневых компонентов системы не равны навыкам проектирования правильной архитектуры. Именно поэтому в Deep Tech компаниях собеседования обычно состоят из двух частей: проверки навыков написания эффективного кода (этап, посвященный алгоритмам) и проектирования систем. Ожидается, что инженеры будут совмещать обе эти роли. Разделение между ними может быть разным: обычно джуниор-разработчики лишь следуют дизайн-документам, тогда как сеньоры становятся их авторами или активными участниками.

Так что общий консенсус таков: хороший инженер-разработчик способен делать все — от работы над низкоуровневой реализацией до принятия высокоуровневых архитектурных решений.

Все, что мы до сих пор говорили о проектировании систем, применимо к любому программному обеспечению — пока что мы не затрагивали ничего, связанного непосредственно с машинным обучением. Однако не каждый, кто умеет проектировать программные системы, способен успешно спроектировать ML-систему, поскольку это особый подкласс систем. При их проектировании специалисту необходимо учитывать множество аспектов, не имеющих значения в традиционной разработке ПО. В этой книге мы сосредоточимся именно на них, поэтому тем, кого интересуют более общие вопросы проектирования систем, лучше обратиться к другой литературе.

1.2. Структура книги

Существует множество книг, посвященных проектированию программных систем, но вот литература по проектированию ML-систем встречается довольно редко. Мы решили внести свой вклад в эту область и восполнить разрыв между спросом и предложением. Наша цель — поделиться своими знаниями и опытом и собрать воедино все разрозненные сведения.

Эта книга представляет собой всестороннее практическое руководство по построению сложных и полноценно функционирующих ML-систем в различных сферах, независимо от размера компании, в которой вы работаете. В это руководство входят:

- общий обзор, включающий основные принципы структурирования, описание всех компонентов, из которых состоят такие системы, а также подводные камни, с которыми вы можете столкнуться;
- набор низкоуровневых чек-листов с перечнем инструментов, которые могут пригодиться на каждом этапе, и кратким пояснением их значимости.

Структура книги напоминает некий чек-лист или техническое руководство, дополненное историями из нашего собственного опыта. Ее можно сразу прочитать от начала до конца или же использовать в качестве справочника в процессе работы над конкретным аспектом ML-системы.

Каждая глава, в свою очередь, представляет собой высокоуровневый чек-лист, обязательный для любой ML-системы. При этом не все пункты обязательны для каждого проекта, но о каждом из них необходимо помнить и учитывать в процессе разработки.

Кроме того, в каждой главе рассматривается, почему и в каких ситуациях тот или иной аспект имеет критическое значение. Вы также найдете систематизированное описание доступных методов и инструментов, подходящих для решения поставленных задач (а не просто перечисление модных терминов). И хотя наше описание не претендует на исчерпывающую полноту, мы уверены, что читатели смогут сопоставить предложенные примеры со своим профессиональным опытом и сделать соответствующие выводы. При этом мы стараемся не превращать книгу в типичный учебник или курс по классическому машинному обучению или глубокому обучению.

Мы пришли в область ML из разных профессиональных сфер и поэтому отлично дополняем друг друга. В совокупности у нас более 20 лет опыта работы над проектами ML в самых разных ролях, компаниях и средах — от молодых стартапов до международных корпораций с многомиллиардными оборотами. Иногда мы работали как индивидуальные контрибьюторы, иногда же больше сосредоточивались на развитии команды и наставничестве молодых инженеров. Нам довелось стать свидетелями (и участниками) как успешных запусков, так и неудач, крупных поглощений и массовых сокращений. Конечно, мы также часто обсуждали успехи и провалы ML-проектов с друзьями и коллегами.

Однако, несмотря на различия между нашими карьерами, в одном мы абсолютно согласны — ML-проекты почти никогда не терпят неудачу из-за того, что их участники не умеют применять алгоритмы. Причины провалов могут быть совершенно другими: неправильно поставленная или вовсе ненужная задача, небрежная работа с данными, нерентабельное и неспособное к масштабированию решение — список можно продолжать бесконечно.

Существует одна распространенная закономерность, из-за которой нам придется повторять некоторые истории в разных частях книги: узкий специалист глубоко погружается в свою область — возможно, осваивает смежные сферы, но все равно не видит полной картины. В результате упускаются важные нюансы, что приводит к провалу проекта, срыву сроков и превышению бюджета.

В то время как большинство книг по машинному обучению предлагают «правильные» ответы, наша главная цель совершенно иная. Мы хотим научить вас задавать правильные вопросы — себе, своей команде, пользователям, стейкхолдерам — кому угодно. Каждый специалист в технологической отрасли накапливает огромное количество ценной информации, но не всегда способен связать ее воедино. Именно на этом этапе вовремя заданные вопросы помогают структурировать имеющиеся знания.

Мы разделили книгу на четыре основные части, чтобы ее структура соответствовала жизненному циклу любой системы: исследование, создание базовой версии, улучшение и сопровождение.

Первые две части посвящены ранним этапам проектирования систем машинного обучения. В первой части мы сосредоточимся на общем понимании проблемы, которую должна решить система, и определим шаги, которые необходимо выполнить до начала разработки. Данный этап редко связан с написанием кода и чаще фокусируется на создании небольших прототипов или доказательствах концепций. Вторая часть погружается в технические детали начального этапа работы. Этот этап требует значительного объема исследований, которые критически важны для понимания проблемы, определения возможных решений и согласования ожиданий между участниками проекта. Если сравнивать ML-систему с человеческим телом, то на этом этапе формируется ее скелет.

Третья часть посвящена промежуточным этапам жизненного цикла системы, когда работа инженеров зачастую кардинально меняется — становится меньше исследований и обсуждений и больше практической работы по реализации и улучшению системы. Здесь мы сосредоточимся на вопросах, касающихся того, как сделать систему надежной, точной и устойчивой. Продолжая метафору с человеческим телом, на данном этапе система наращивает мышцы.

Заключительная часть посвящена интеграции и развитию системы. Для неопытного наблюдателя может показаться, что система уже полностью готова к работе, но это впечатление обманчиво. Существует множество (в основном инженерных) аспектов, которые необходимо учесть, чтобы система успешно запустилась в эксплуатацию. В мире ПО сбой системы редко становится катастрофой, как, например, происходит в строительстве, но этого все равно хочется избежать. На этом этапе вы узнаете, как сделать систему надежной, удобной в сопровождении и готовой к будущим изменениям. Если вы еще не устали от метафор, связанных с человеческим организмом, то можно сказать, что здесь система обретает разум, ведь необузданная сила без контроля может привести только к разрушению.

В целом первые главы содержат более общую информацию, которая, тем не менее, крайне важна для правильной постановки проблемы и формирования базовых принципов построения эффективной ML-системы. Однако по мере изучения книги материал будет становиться все более сложным и подробным, включая практические примеры и упражнения. Начиная со следующей главы, мы представим два вымышленных кейса, радикально отличающихся друг от друга, которые будут сопровождать нас на протяжении всей книги. Обе ситуации требуют разработки ML-системы для решения ряда задач, и обе будут развиваться по мере изучения материала.

В каждой части книги мы всегда отдаем предпочтение интуитивному пониманию, а не охвату как можно большего объема материала. Существует множество аспектов построения ML-систем, и каждый из них заслуживает отдельной книги.

Однако мы не планируем писать отдельное издание о сборе и подготовке данных, еще одно — о генерации признаков и еще одно — о метриках. Вместо этого мы описываем верхушку айсберга и анализируем общую структуру ландшафта, подкрепляя наши идеи и аргументы ссылками на значимые научные работы. Это позволяет читателям как ознакомиться с примерами более высокого уровня, так и дополнить предложенный каркас собственными знаниями. Кроме того, мы не стремимся объяснять технические детали, связанные с конкретными библиотеками или вычислительными движками. В некоторых главах мы упомянем ключевые примеры, но исключительно в иллюстративных целях для демонстрации более высокоуровневых абстракций.

Реальные системы всегда сложнее, чем примеры, которые можно встретить в блогах, на конференциях и, конечно же, на собеседованиях. Во всех этих сценариях обычно обсуждают высокоуровневые абстракции, но на практике именно детали играют ключевую роль. Вот почему мы считаем, что интуитивное понимание методов решения проблем так важно: успешный разработчик ML-систем должен не только знать готовый «рецепт» из учебника и воспроизводить его, но и уметь адаптироваться к специфике компании, что порой может полностью изменить правила игры.

Мы надеемся, что эта книга будет полезна:

- тем, кто готовится к собеседованию на позицию ML-инженера или ML-менеджера;
- разработчикам программного обеспечения, техническим менеджерам и специалистам в области ML, работающим с существующими системами и желающим их понять или улучшить;
- тем, кто планирует разработать собственную ML-систему или уже создал ее и хочет убедиться, что не упустил ничего критически важного.

В силу изложенной выше философии, эта книга не предназначена для начинающих. Мы ожидаем, что наши читатели уже знакомы с основами машинного обучения (например, способны понять учебник по ML для студентов бакалавриата) и обладают опытом прикладного программирования (например, сталкивались с реальными задачами разработки за пределами учебных курсов). В противном случае лучше сначала изучить базовый материал.

1.3. Где могут быть полезны принципы проектирования ML-систем

Как уже упоминалось ранее, применение данных принципов имеет решающее значение при построении системы, достаточно сложной, чтобы иметь множество возможных сценариев отказа. Проигнорировав эти принципы, можно создать систему «на глиняных ногах», то есть такую, которая на данный момент работает,

но при этом недостаточно устойчива, чтобы выдержать возможные изменения. Причины таких изменений могут быть сугубо техническими (что, если объем данных увеличится в 10 раз?), связанными с продуктом (как адаптироваться к новым пользовательским сценариям?), продиктованными бизнесом (что, если систему предстоит интегрировать в сторонний программный стек после поглощения компании?), юридическими (что, если государство введет новые нормы регулирования персональных данных?) или носить иной характер. Последние годы лишь подтвердили — предугадать каждый возможный риск невозможно.

Еще более важно уметь улучшать систему. Как будет подробно рассмотрено в последующих главах, создание систем с нуля — довольно редкое явление. Люди, далекие от индустрии, могут полагать, что разработчики ПО большую часть времени пишут код, но на практике, как вам хорошо известно, гораздо больше времени уходит на *чтение* кода. То же справедливо и для систем: обычно усилия концентрируются не на создании новых решений, а на улучшении и сопровождении уже существующих (что требует глубокого понимания внутреннего устройства системы).

Разница между улучшением и сопровождением может показаться размытой. Поэтому для ясности под *улучшением* (improvement) мы будем подразумевать добавление нового функционала или существенные изменения существующего, тогда как *сопровождение* (maintenance) — это обеспечение работоспособности существующего функционала в условиях постоянно изменяющейся среды (новые пользователи, новые датасеты, развитие инфраструктуры и т. д.).

Некоторые принципы, представленные в этой книге, в первую очередь ориентированы на улучшение ML-систем. Они помогают выявить слабые места, точки роста системы, а иногда даже открывают возможности для новых способов ее применения.

Часть принципов относится скорее к сопровождению ML-систем. Печальная истина состоит в том, что зачастую сопровождением систем занимаются команды, которые не участвовали в их создании. Это палка о двух концах — команда, создающая систему, должна помнить о некоторых принципах, чтобы облегчить жизнь своим «последователям», а команда сопровождения должна понимать эти принципы, чтобы в разумные сроки разобраться в логике системы и найти подходящие решения, позволяющие поддерживать ее работоспособность в течение длительного времени.

Можно с уверенностью сказать, что практически 100 % проектов в области машинного обучения, у которых не было грамотно составленного дизайн-документа, потерпели неудачу. В то же время подавляющее большинство тщательно спланированных систем достигли успеха. Дизайн-документ вовсе не обязательно должен быть сложным и состоять из множества страниц, зачастую достаточно нескольких страниц со сжатой, но полезной информацией. В таком случае документ выполняет две ключевые функции: он не только помогает правильно расставить приоритеты в рамках проекта, но и позволяет понять, нужен ли этот

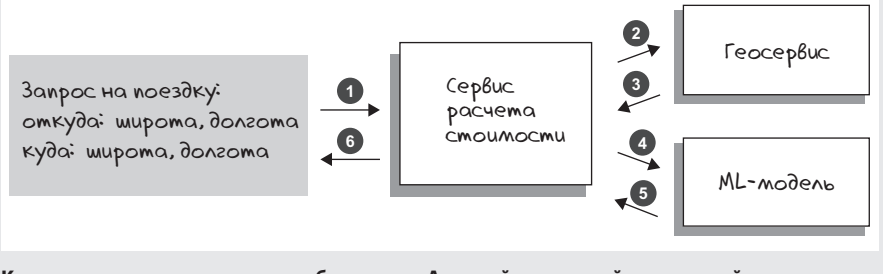
проект вообще, отвлекая ваше внимание от основной идеи (например, когда вы чрезмерно зациклены на самом проекте) и помогая взглянуть на ситуацию шире. Подробности приведены в главе 4.

Поработав в разных компаниях, мы с уверенностью можем утверждать: наличие структурированной документации, описывающей все аспекты функционирования вашей системы, в разы ускоряет любые процессы — от онбординга новых сотрудников до внедрения ключевых изменений. Вместо того чтобы искать единственного «хранителя знаний», в голове которого сосредоточена вся информация (причем без гарантий точности), вы просто обращаетесь к нужному документу в библиотеке.

Байка от Арсения

Давным-давно я работал в сервисе заказа такси. Одним из амбициозных проектов компании было создание системы для расчета стоимости поездки. Обычная модель ценообразования представляла собой классическую схему, используемую в «олдскульных» такси: стоимость = X * время + Y * расстояние. Компании было необходимо научиться оценивать стоимость поездки до ее фактического начала, чтобы информировать как водителя, так и пассажира.

Изначально задача казалась предельно ясной. Все, что нам нужно было сделать, — обучить простую модель, использующую геолокацию из картографического сервиса, и обернуть ее в микросервис. Проект казался настолько элементарным, что я даже не стал писать дизайн-документ.

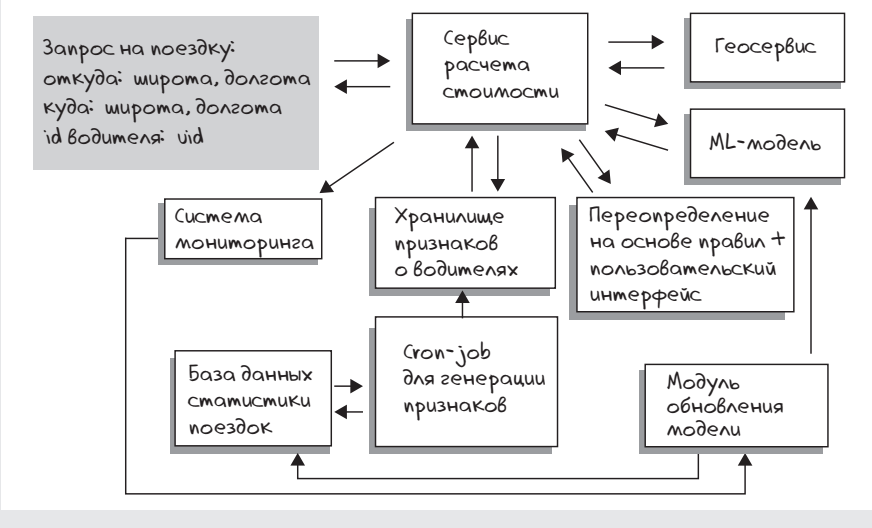


Как изначально представлял себе систему Арсений — простой пошаговый алгоритм

На практике же возникло множество подводных камней (большинство из них мы рассмотрим в следующих главах):

- Геолокации оказалось недостаточно для точной оценки, а более сложные метрики требовали развитой инфраструктуры (так называемого *хранилища признаков*, хотя в то время такой термин и подход еще не были популярны). Этому будет посвящена глава 11.
- По мере усложнения модели ее прогнозы становились менее надежными (определенное количество результатов оказывались выбросами, то есть значениями, которые были слишком высокими или слишком низкими).

- Ошибки распределялись неравномерно, поэтому модель была предвзятой. Мы затронем эту тему в главе 9.
- Руководство время от времени хотело вручную изменять расчеты стоимости поездки, используя акции или эвристики. Данная тема обсуждается в главе 13.
- Было потрачено слишком много времени на построение модели, которая на самом деле не решала поставленную задачу. Мы возвращаемся к этому вопросу в главе 2.
- Проблема в целом была связана с неравномерным распределением ресурсов и, следовательно, требовала тщательного мониторинга. Подробнее мы рассмотрим эту тему в главе 14.
- Инфраструктура не была готова к предполагаемому сценарию, что привело к неприемлемым задержкам в часы пик. Поговорим об этом в главе 15.
- Некоторые другие команды не знали о том, что разрабатывается такая система, это привело к несовместимостям в API. Данная тема рассматривается в главе 16.



Как система выглядела после нескольких итераций

В итоге система так и не была развернута — до того, как все проблемы были устранены, ситуация на рынке существенно изменилась и потребность в исходной системе исчезла. Хотя изначальная идея была хорошей (некоторые конкуренты реализовали похожие решения), мои коллеги и я так и не смогли воплотить ее должным образом: были упущены ключевые аспекты — как технические, так и продуктовые, — и всплыли они лишь на поздних стадиях проекта, когда любые изменения были слишком затратными. При этом если бы эти аспекты были учтены на более ранних этапах, их решение оказалось бы элементарным. Если бы кто-то из нас — я, мой руководитель или члены команды — прочитал такую книгу, как эта, мы, возможно, избежали бы провала.

Тем не менее на несколько историй провала всегда находится своя история успеха. Следующий случай менее драматичен и может даже показаться скучным, но для баланса его все же стоит упомянуть. В свое время, когда Валерий работал в российском технологическом гиганте «Яндекс», компания приобрела стартап, занимавшийся предоставлением рекомендаций в реальном времени. При слиянии компаний требуется время для настройки взаимодействия между новыми и существующими подразделениями, онбординга новых сотрудников, синхронизации бизнес-процессов и т. д. Однако Валерий был поражен тем, насколько гладко и беспроблемно произошло интегрирование нового бизнеса в масштабную корпорацию. Причина заключалась в грамотно составленном дизайн-документе, благодаря которому такой переход стал возможным.

Подводя итог: мы искренне убеждены, что подготовка дизайн-документа — с постановкой правильных вопросов бизнесу и формулировкой корректных целей — является ключом к успеху при разработке ML-системы. А в ряде случаев это даже может стать основанием для отказа от проекта на ранней стадии, что само по себе является положительным результатом, учитывая, сколько времени, сил и денег можно сэкономить, если отказаться от неудачной идеи. Данному этапу проекта мы посвятим как минимум три главы ввиду его критической важности.

Итоги

- Несмотря на относительную новизну термина, проектирование ML-систем опирается на классические основы разработки программного обеспечения и включает в себя уже существующие знания из смежных дисциплин. В этой книге мы постараемся преобразовать эту базу знаний в набор рабочих алгоритмов.
- В то время как MLOps можно рассматривать как набор инструментов для создания и сопровождения ML-систем, проектирование ML-систем — это фреймворк, который объединяет все воедино.
- Для успешного проектирования ML-систем необходимо одинаково хорошо разбираться в таких областях, как машинное обучение, разработка ПО, проджект-менеджмент, продакт-менеджмент и управление людьми.
- Прежде чем приступить к проектированию ML-системы, вы должны четко понимать, что именно вы создаете, с какой целью и каким образом ваша система будет реализована.
- Основой успешно спроектированной ML-системы является согласованный подход, продуманный план работы и список предварительных действий, которые организуют вашу деятельность и сэкономят время в долгосрочной перспективе.

2 *В чем проблема?*

В этой главе

- ✓ Пространство проблем и пространство решений — что первично?
- ✓ Определение проблемы как самый важный шаг
- ✓ Определение рисков и ограничений
- ✓ Цена ошибки

Для успешного проектирования ML-системы необходимо хорошо разбираться в нескольких областях одновременно, включая проджект-менеджмент, машинное обучение и глубокое обучение, управление людьми, продакт-менеджмент и разработку программного обеспечения. Однако если максимально упросить данную систему знаний, даже самые сложные и продвинутые решения в проектировании ML-систем будут опираться на тот же фреймворк и те же фундаментальные принципы, что и любой другой программный продукт.

Накопленные за последние годы знания предоставляют беспрецедентную свободу в выборе подхода к созданию ML-системы. Но какими бы изощренными ни были выбранные инструменты, они являются лишь средствами реализации.

Каковы бизнес-цели? Каков бюджет? Насколько гибкие сроки? Покроет ли потенциальный результат все издержки и принесет ли прибыль? Это лишь некоторые из тех вопросов, что необходимо задать себе до определения масштаба вашего ML-проекта.

Но прежде чем приступить к ответу на эти вопросы, необходимо выполнить одно важнейшее действие, которое заложит основу для успешного проектирования ML-системы, — *четко сформулировать проблему, которую решит (или поможет решить) ваша система*[1]. На первый взгляд это может показаться банальной и очевидной деталью, особенно для опытных инженеров, однако, как показывает наш собственный опыт, пропуск этого шага на предварительном этапе работы чреват серьезными последствиями. Особенно когда становится ясно, что некоторые проблемы в принципе невозможно решить на должном уровне — либо из-за текущего состояния доступных технологий, либо из-за алеаторной неопределенности плохо сформулированной проблемы. В первом случае решение возможно в будущем (например, современный уровень генерации текста был недостижим для ML-инженера начала 2010-х годов), во втором случае — ее не следует решать вообще (например, невозможно создать алгоритм, который сможет обыграть рулетку в казино).

В этой главе мы рассмотрим важность понимания проблемы до разработки ее решения, а также все риски и ограничения, с которыми можно столкнуться при ее формулировке. Кроме того, мы затронем возможные последствия некорректной формулировки проблемы.

2.1. Пространство проблем и пространство решений

> Если единственный инструмент, который у вас есть, — это молоток, то вы будете на любую проблему смотреть как на гвоздь.
>
> *Абрахам Маслоу, американский психолог*

Представьте себе ситуацию: начальник приходит к инженеру с идеей превосходной новой функции (знакомо, правда?). Добавим в наш пример конкретики. Стив работает ML-инженером в развивающейся SaaS-компании. Его начальница Линда только что вернулась со встречи с Джеком, вице-президентом по продажам, и он поделился проблемой, с которой сталкивается его команда: слишком много лидов (потенциальных клиентов) и слишком мало менеджеров, чтобы их обработать. Джек интересуется, может ли ML-команда разработать решение на базе ИИ, которое будет автоматически ранжировать лиды — от самых перспективных до наименее прибыльных — по ожидаемой выгоде для компании. Это позволило бы команде по продажам сначала работать с наиболее прибыльными

[1] Обычно принято говорить «задача ML», соответственно, «пространство задач». Но автор Валерий Бабушкин в своих выступлениях на русском языке использует именно слово «проблема», поэтому в переводе так и оставлено. — *Примеч. ред.*

клиентами, а остальные лиды обрабатывать по возможности. На бумаге выглядит идеально. Кажется, решение будет проще, чем дважды два!

Стив, молодой, но старательный специалист, сразу же начинает задавать множество вопросов по данному проекту. Каковы сроки? Насколько объемен датасет с текущими лидами, на основе которого необходимо построить ML-модель? Какое максимальное время дается на оценку одного лида? Какова ожидаемая точность? Какие данные имеются по каждому лиду? Насколько быстро должна работать система? Что именно подразумевается под «перспективным лидом»? В какую систему продаж будет интегрировано решение? В итоге Стив узнает следующее:

- Датасет на данный момент довольно небольшой (компания — молодой стартап).
- Джек хочет, чтобы инструмент интегрировался с существующей системой управления взаимоотношениями с клиентами (CRM), что позволит избежать дополнительных расходов на новое ПО и не тратить ресурсы на переобучение команды.
- К счастью, жестких ограничений по времени обработки нет, это означает, что надежный API в режиме реального времени не требуется.
- Сроки стандартные — «чем скорее, тем лучше».

Итак, Стив возвращается на свое рабочее место и начинает определять масштаб проекта. «Выглядит просто, поэтому можем сформулировать как проблему ранжирования или классификации, выделить признаки, обучить модель, предоставить API, интегрировать и развернуть — и дело сделано». Однако Стива беспокоят два вопроса:

- Какой метод лучше всего подойдет для решения проблемы классификации такого типа?
- Как интегрировать код на Python с CRM, которую использует команда Джека?

Через три часа его браузер уже полон вкладок с методами few-shot-классификации и документацией по API CRM. Стив хочет сообщить коллегам точные сроки выполнения проекта, но сделать это будет сложно из-за одной ключевой ошибки, которая может дорого ему обойтись, — в процессе своих размышлений он сосредоточился на пространстве решений, а не на пространстве проблем.

С точки зрения Стива, полученной информации было более чем достаточно для выбора подходящего решения, хотя на деле это была лишь вершина айсберга. Остальной контекст можно было бы получить только с помощью множества уточняющих вопросов всем участникам проекта.

Что такое пространство проблем и пространство решений? Это два подхода к исследованию проблемы, охватывающие разные ее аспекты. И хотя оба подхода крайне важны, первым всегда должно идти пространство проблем (рис. 2.1).



Рис. 2.1. Опытный инженер всегда начинает с пространства проблем, задавая уточняющие вопросы

Пространство проблем зачастую определяется с помощью целой серии вопросов «что?» и «почему?». Существует даже популярная методика под названием «Пять почему», где рекомендуется задавать несколько последовательных вопросов «почему?», чтобы добраться до самой сути анализируемой проблемы. Приведем несколько примеров таких вопросов:

- Почему нам необходимо разработать это решение?
- Какую проблему оно решает?
- Почему возникает эта проблема?
- Какие альтернативы нам известны?
- Почему мы хотим реализовать это решение с учетом заданных ограничений (метрик, задержек, количества обучающих выборок)?

После проведения такого анализа у вас должно сложиться понимание того, что именно нужно сделать и зачем.

Вопрос «что?», в свою очередь, связан с пониманием пользовательских и функциональных характеристик (рис. 2.2). Например: «Инструмент, который присваивает клиентским лидам оценку вероятности заключения сделки; оценка должна быть присвоена до того, как менеджеры по продажам начнут планировать работу на еженедельной встрече в понедельник».

В некоторых компаниях постановкой таких вопросов занимается исключительно продакт-менеджер. Однако анализ пространства проблем без участия инженеров не слишком продуктивен, поскольку корректное понимание проблемы существенно влияет на конечный результат.



Рис. 2.2. Вопросы, которые необходимо задать перед началом проекта, и ключевое различие между ними

Пространство решений, напротив, гораздо меньше связано с самой проблемой и потребностями пользователей — здесь внимание уделяется реализации. В рамках этого этапа обсуждаются фреймворки и интерфейсы, рассматривается внутренняя архитектура решений, оцениваются технические риски. Однако реализация не должна начинаться до тех пор, пока не будет достигнуто целостное понимание проблемы.

Досконального понимание проблемы до перехода к ее технической реализации позволяет рассмотреть различные обходные пути, некоторые из них могут значительно сократить масштаб проекта. Возможно, существует какой-то сторонний плагин для CRM, специально разработанный для решения этой проблемы. Возможно, цена ошибки в ML-модели в данном случае не так критична, несмотря на первоначальные утверждения Джека (заказчики часто начинают с утверждений, что им нужна точность почти 100 %). Возможно, данные показывают, что 95 % неактуальных лидов можно отфильтровать с помощью простых эвристических правил. Все эти предположения выходят за рамки первоначального запроса, но могут стать важными составляющими контекста. И именно раскрытие этого контекста даст вам реальное понимание проблемы.

Существует две причины, по которым глава начинается с истории Стива. Во-первых, это типичная ситуация, и скорее всего, вы с ней сталкивались. Во-вторых, такой подход применим к любому сценарию, будь то разработка новой системы, модификация уже существующего решения или прохождение собеседования в IT-компании. В-третьих, и это главное, масштаб и последствия такого подхода могут быть весьма серьезными:

- Стиву придется переписать значительную часть финальной системы.
- Линде придется пользоваться частичным решением и вручную компенсировать отсутствующий функционал.
- От решения могут вообще отказаться.

Итак, в любом сценарии требуется предварительно и глубоко понять исходную проблему.

2.2. Поиск проблемы

> Компании, занимающиеся проектированием систем (в широком смысле этого слова), вынуждены разрабатывать системы, повторяющие структуры коммуникации внутри этих компаний.
>
> *Мелвин Э. Конвей*

Некоторые компании с устаревшей корпоративной культурой до сих пор придерживаются подхода, при котором рядовым инженерам предлагается сосредоточиться исключительно на низкоуровневой реализации (просто написании кода), оставляя проектирование (включая понимание и декомпозицию проблемы) архитекторам и системным аналитикам. По нашему опыту, с ростом требований к гибкости такая культура стремительно уходит в прошлое, уступая место более горизонтальным структурам, где анализ проблемы все чаще доверяется индивидуальным контрибьюторам.

Это означает, что инженеру не обязательно быть глубоким экспертом в предметной области (она может быть слишком сложной для специалиста без определенного опыта). Причина проста: невозможно изучить все нюансы фондовой биржи или контроля качества на производстве в перерыве между встречами, код-ревью и обучением нейросетей. Но общее понимание проблемы абсолютно необходимо для проектирования ML-системы.

Мы рекомендуем составлять описание проблемы по структуре перевернутой пирамиды, в основании которой находится высокоуровневое понимание, а в верхней части — детали. Это распространенный и эффективный подход «сверху вниз», который поможет вам собрать как можно больше общей информации, определить, какие данные наиболее ценны для проекта, а затем, с помощью цепочки наводящих вопросов, углубиться в специфику проблемы (рис. 2.3).

На самом верхнем уровне вы получите общее понимание проблемы. Это тот уровень, который будет понятен любому топ-менеджеру компании — тому, кто не слишком заинтересован в деталях ML-алгоритмов или архитектуры ПО. Например:

- Наше мобильное приложение подвергается атакам мошенников, наносящих вред легальным пользователям.
- Наша ценовая модель показала крайне низкую рентабельность по ряду товаров, одновременно оказавшись абсолютно неконкурентоспособной в других категориях.
- Пользователи жалуются, что для работы с нашим ПО требуется слишком много ручной настройки. Все должно работать *«автомагически»*.
- У нашего приложения слишком низкая вовлеченность пользователей.



Рис. 2.3. Схема перевернутой пирамиды — рекомендуемый нами подход для сбора данных, необходимых для успешного запуска проекта

Наличие подобного утверждения в самом начале работы открывает множество возможностей для дальнейшего изучения. Просто попробуйте задать вопросы к каждому слову в предложении, чтобы убедиться, что вы можете объяснить проблему десятилетнему ребенку. Кто такие мошенники? Как они атакуют? Откуда мы узнали о завышенных ценах? Что больше всего беспокоит наших пользователей? На что тратится больше всего времени? Как мы измеряем вовлеченность пользователей? Как рекомендации связаны с этой метрикой? Задавайте эти вопросы себе и своим коллегам до тех пор, пока не будете готовы построить следующий блок пирамиды, расширяющий исходное утверждение.

Второй блок пирамиды требует более конкретных, продуманных вопросов. Один из эффективных приемов на данном этапе — поиск источника ответов предыдущего уровня. Как мы определили, что какое-то поведение является мошенническим? Какие именно ручные настройки приходится выполнять пользователям? Как в данный момент соотносятся показатели вовлеченности пользователей и эффективность рекомендательной системы?

Другой полезный прием — поиск противоречий в ответах. Люди склонны группировать объекты по сходству и различать их по отличиям. Может существовать множество похожих пользователей: одних следует считать спамерами и блокировать, других же не стоит, хотя их поведение частично совпадает с действиями за пределами проблемной области. Для стороннего наблюдателя одна и та же наценка на похожие товары может быть приемлемой или нет — но каковы критерии? Инженеру в данном случае не нужно формулировать все критерии разделения в описании задачи (он не строит дерево решений), но это отличная возможность уловить наиболее важные сигналы и получить инсайты. Все это можно резюмировать следующим образом: понять, чего хотят люди, — важно; понять, что им действительно нужно, — критически необходимо.

Обязательно привлекайте к процессу всех заинтересованных лиц. В проекте участвует не только ваш руководитель или продакт-менеджер, скорее всего, у вас будет несколько стейкхолдеров (важно понимать, кто из них отвечает за бюджет, а кто будет утверждать конкретные компоненты системы). Рекомендуем пообщаться с экспертами на разных уровнях, чтобы взглянуть на проект как со стратегической, так и с тактической точки зрения. Руководитель более высокого уровня хорошо понимает цели конкретной инициативы, тогда как отдельные специалисты, которые в данный момент вручную компенсируют отсутствие системы, владеют всеми тонкостями и деталями, которые могут существенно повлиять на архитектуру решения.

Как только вы поймете, что способны объяснить проблему простыми словами, можно подвести итог. Мы рекомендуем письменно зафиксировать ваше понимание проблемы. Обычно это всего лишь несколько абзацев текста, но именно они впоследствии станут фундаментом вашего дизайн-документа. Пока что не нужно концентрировать на нем слишком большое внимание; вы находитесь лишь на первом (хотя и очень важном) этапе работы.

Степень значимости данного этапа зависит от вашей компании или контекста. Иногда проблему легко понять, но ее решение чрезвычайно сложно — типичная ситуация на зрелых конкурентных рынках. На другом конце спектра — стартапы, которые стремятся изменить существующий рынок; в таких случаях изначальное понимание вопроса редко бывает точным. Один из авторов работал в компании, где до 50 % времени, отведенного на проект, тратилось на формулировку целей и сбор релевантного контекста. Как только контекст становился понятным, инженерная работа по машинному обучению шла гладко и последовательно.

Байка от Валерия

В 2019 году я работал в IT-компании, решившей развивать свой отдел электронной коммерции и «выйти в офлайн». Идея нового проекта под названием «Super Bill» («Суперчек») была следующей:

1. С помощью приложения Super Bill пользователи могли сравнивать цены в разных продуктовых магазинах, загружать свои чеки, чтобы получать кэшбэк, и совершать покупки в магазине через приложение.
2. Бренды могли использовать приложение для предоставления кэшбэка за покупку конкретных товаров, рекламировать свои товары в качестве заменителей или рекомендованных товаров.

Проблема заключалась в том, что один и тот же товар мог иметь разные названия (указанные в чеке) в разных торговых сетях. «Mars маленький», «М. батончик маленький», «Mars батончик маленький» и т. д. — различные написания одной и той же товарной позиции, и их нужно было сопоставить с одним и тем же товаром — батончиком Mars.

Изначально идея заключалась в том, чтобы обучить глубоко структурированную семантическую модель и реализовать сопоставление в стиле поисковой системы, где

название из чека играло бы роль запроса, а уникальный складской идентификатор товара (SKU) — роль документа.

Мне не понравилось это решение, так как оно казалось слишком сложным и громоздким для данной задачи. Оно требовало сбора данных, их разметки, обучения модели, оценки качества и т. д. Поэтому я решил подумать над возможной альтернативой. Если мне все равно нужно размечать данные, я сделаю это только один раз для каждого названия в чеке в пределах конкретного магазина. Следовательно, для уже размеченных семплов ML-модель вовсе не требуется, и такие случаи могут возникать довольно часто. Учитывая, что частотность товаров подчиняется закону Ципфа, нам нужно размечать наиболее популярные пары «название — SKU», которые составляют небольшую часть от всех уникальных пар, но значительную часть от общего объема, подлежащего разметке (согласно этому эмпирическому закону, при сортировке списка измеренных значений по убыванию значение *n*-го элемента приблизительно обратно пропорционально *n*).

Оставшуюся часть можно разметить силами сотрудников, имеющих доступ ко всей базе данных SKU, однако, скорее всего, мы не захотим делиться этой базой с платформой краудсорсинга для ручной разметки. В лучшем случае мы можем предоставить список кандидатов из этой базы и проверить, какой из них (если такой имеется) соответствует искомому товару.

Что можно сделать в таком случае? Мы можем попробовать спрогнозировать или извлечь название бренда и определить категорию семпла в чеках. Сужение списка кандидатов — это относительно простая задача классификации или поиска на основе расстояний, так как у нас ограниченное число категорий и брендов. Для постобработки можно использовать очень простые методы, например расстояние Левенштейна. Как только у нас есть список кандидатов, мы можем отправить их вместе с семплом из чека на разметку через краудсорсинговую платформу. Как часто нужно это делать? Ответ — только один раз для каждой пары «SKU — торговая сеть». Это делает решение гораздо проще и быстрее по сравнению с изначальной идеей. В конце концов мы не строили поисковый движок для миллиардов запросов в день, а разрабатывали ограниченную систему сопоставления.

Нам удалось внедрить это решение в эксплуатацию менее чем за три недели — это невероятно быстро. Все, что потребовалось, — это понять, какую проблему мы хотим решить, и собрать соответствующий контекст.

P. S. Позднее, когда мы создавали аналогичную систему для другой Deep Tech компании, имеющей десятки миллионов товарных единиц, мы заменили последний этап постобработки на модель семантического сходства на основе глубокого обучения, чтобы создать более интеллектуальную систему. Это стало возможным благодаря тому, что архитектура системы изначально была легко адаптируемой и модульной.

Как только формулировка проблемы становится достаточно четкой, приходит время задуматься о том, что мы, как инженеры по машинному обучению, можем с ней сделать.

2.2.1. Как прийти к решению проблемы с помощью ML-системы

Неопытные или просто торопливые инженеры зачастую пытаются подогнать проблему под уже известные ML-алгоритмы — обучение с учителем или без учителя, проблема классификации или регрессии. Однако это не лучшее начало. Для стороннего наблюдателя модель машинного обучения напоминает некий волшебный оракул — универсальную машину, способную ответить на любой правильно сформулированный вопрос. Ваша задача как ML-инженера — этого волшебства достичь, то есть построить «оракул» с помощью ML-алгоритмов. Но сначала необходимо правильно сформулировать вопрос — и научить этому пользователей. Если отбросить все метафоры, то мы должны из бизнес-проблемы вывести проблему программного/ML характера.

Некоторые вопросы могут показаться довольно прямолинейными:

- В случае с мошенничеством мы хотим, чтобы оракул как можно более оперативно помечал пользователя как мошенника — в идеале даже до того, как тот что-либо совершил. *Это звучит как своего рода классификация.*
- В случае модели ценообразования мы хотим понять, сколько клиент готов заплатить за товар, не отказавшись от сервиса в пользу конкурента (если речь идет о краткосрочной задаче) или не подумав: «Вот это они цену задрали, больше не буду пользоваться этим приложением» (в рамках долгосрочной перспективы бренда). *Это напоминает типичный пример регрессии.*
- Что касается системы рекомендаций, то нам необходимо ответить на вопрос: что мы можем предложить клиенту, чтобы он остался доволен сервисом? *Это похоже на проблему ранжирования.*

Даже сравнив ML-систему с магическим оракулом, нам все равно придется вносить множество корректировок, влияющих на потенциальный ответ. В этой книге мы будем периодически уделять внимание подобным деталям и оговоркам, однако здесь важно подчеркнуть следующее: не всегда существует один простой ответ на поставленный вопрос, и ваша система должна учитывать это заранее.

В приведенном примере с ценообразованием у нас могут быть самые разные цели — от максимизации прибыли в настоящий момент времени до устойчивого роста компании в долгосрочной перспективе. Хорошо спроектированная ML-система должна быть способна адаптироваться к каждой из них. В следующих главах мы подробно рассмотрим технические аспекты реализации такой адаптивности.

Многие специалисты в области машинного обучения, включая известного эксперта в сфере ИИ, профессора Стэнфордского университета и основателя Landing AI Эндрю Ына (Andrew Ng), рекомендуют строить систему таким образом, чтобы она отвечала так же, как это сделал бы специалист в какой-либо области. Такой подход применим во многих сферах (один из ярких примеров — здравоохранение) и дает нам начальное понимание, возможно ли решить ту или иную проблему с помощью ИИ. Однако у такого подхода есть и недостатки: существуют проблемы, в которых машины превосходят человека. Такие ситуации

обычно возникают в сферах, где данные представлены в виде журналов событий (например, отражающих поведение пользователей), а не размечены вручную, что характерно для индустрии рекламных технологий и финансов. Поэтому ориентация на более «человеческую» производительность не всегда разумна.

И только после того, как вопрос четко сформулирован, имеет смысл переходить к аппроксимации и разрабатывать способную к выполнению данной задачи модель, причем не обязательно одну — иногда эффективно использовать пайплайн из нескольких моделей или алгоритмов. В следующей главе мы подробно рассмотрим процесс декомпозиции проблемы.

2.3. Риски, ограничения и возможные последствия

Представьте, что вы создали систему обнаружения мошенничества: она оценивает действия пользователей и предотвращает подозрительные действия путем блокировки потенциально опасных аккаунтов. Это крайне полезный инструмент — с момента его запуска ни один мошенник не проник в систему, и команда клиентской поддержки довольна. Однако недавно маркетинговый отдел запустил масштабную рекламную кампанию, и ваша «идеальная» система начала блокировать значительную часть новых пользователей на основании источника трафика (он неизвестен, а значит, по логике алгоритма, — подозрителен). Отрицательные последствия для маркетинга могут значительно перевешивать выгоду от выявления мошенников.

Возможно, такой пример покажется вам банальным и не заслуживающим особого внимания, однако на деле подобные ситуации регулярно происходят в компаниях с несогласованной работой команд — это как раз один из тех рисков, что необходимо предусмотреть при проектировании системы. Не стоит думать: «Мы — команда профессионалов, с нами такого не случится». Трезвое осмысление рисков имеет ключевое значение, поскольку существует высокая вероятность того, что потенциальные риски выйдут за рамки проектной команды или одного отдела.

С большой силой приходит большая ответственность — эта крылатая фраза как нельзя лучше подходит к программному обеспечению на базе ML. Машинное обучение действительно обладает огромной мощностью, но помимо силы, у него есть еще одно важное и потенциально опасное свойство — непрозрачность для большинства наблюдателей, особенно если устройство модели довольно сложное. Поэтому профессиональные системные архитекторы должны осознавать все потенциальные риски и ограничения.

Базовая теория разработки ПО вводит понятие функциональных и нефункциональных требований. Вкратце, функциональные требования описывают работу новой функции или системы, ее ценность и пользовательский сценарий, в то время как нефункциональные требования касаются таких аспектов, как производительность, безопасность, транспортабельность и т. д. Иными словами, функциональные требования определяют, что именно мы проектируем, а нефункциональные — как это будет работать. Таким образом, когда мы говорим о потенциальных рисках и ограничениях, мы, по сути, обсуждаем нефункциональные требования.

Основой любой защитной стратегии является модель рисков. Проще говоря, это ответ на вопрос «От чего мы защищаемся?». Каковы наихудшие возможные сценарии и чего следует избегать? Ответы вроде «ошибочный прогноз модели» неинформативны. Необходимо детальное понимание, согласованное со всеми возможными стейкхолдерами.

Байка от Валерия

Однажды я разрабатывал алгоритм динамического ценообразования для одной Big Tech компании. Это была неплохая система, способная оптимизировать выручку, маржу или трафик с учетом ограничений для двух последних. Она могла работать на уровне отдельных пользователей, но при наличии атомарности на уровне пользователя была возможность агрегировать данные на любой желаемый уровень. Система быстро адаптировалась к изменениям в поведении пользователей и могла в реальном времени генерировать ценовые предложения. В ней был удачно реализован баланс между исследованием и эксплуатацией, учитывался фактор неопределенности, а обучение и инференс занимали совсем немного времени. Я даже подумывал написать статью об этом и придумал ей название: «Двойные байесовские универсальные контекстные бандиты для динамического ценообразования». На схеме ниже показан алгоритмический дизайн системы — выглядит неплохо, правда?



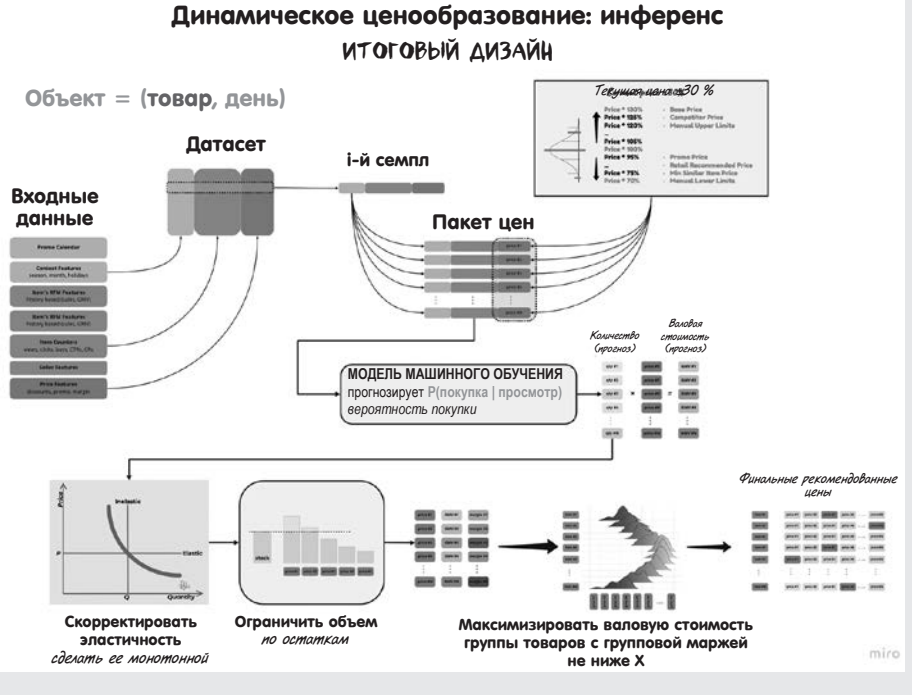
Первоначальный дизайн системы динамического ценообразования

Однако возможные риски, существующие ограничения и нежелательные последствия радикально изменили итоговый дизайн.

Риски: оказалось, что нельзя дискриминировать пользователей, устанавливая разные цены на один и тот же товар. Разумеется, можно дифференцировать цены по регионам, но если речь идет об одном и том же городе и одном и том же товаре, продаваемом онлайн, цена должна быть одинаковой. Технически мы могли бы проводить такую дифференциацию, но это привело бы к судебным искам от пользователей, а следовательно, к многомиллионным финансовым потерям и урону репутации компании. К счастью, мы были к этому в какой-то мере готовы, так как могли агрегировать данные на любом нужном уровне. Еще помните об атомарности на уровне пользователя?

Ограничения: вторая беда пришла со стороны бэкенда. Оказалось, что цену можно менять не чаще одного раза в 6 часов (и это в лучшем случае!), и, следовательно, мы не могли изменять цены в реальном времени. Это вынудило меня радикально упростить систему, сохранив, тем не менее, некоторую способность к адаптации — об этом речь пойдет в следующей истории.

Следовательно, именно ограничения и риски сформировали итоговый дизайн системы, что меня совершенно не расстроило, поскольку сам процесс его создания приносил мне удовольствие и я был готов к любым изменениям (см. следующую схему).



Итоговый дизайн системы динамического ценообразования

Но что, если бы это было мое opus magnum[1]? Что бы я почувствовал тогда? Мораль проста: необходимо как можно раньше выявлять все возможные риски и ограничения, иначе можно оказаться в ситуации, когда вам придется отказаться от всей проделанной работы.

Понимание рисков и ограничений повлияет на множество будущих решений, поэтому мы вернемся к этому позже, в главах, посвященных датасетам, метрикам, отчетности и стратегическим резервам. Но прежде чем перейти к этому, хотим привести несколько примеров, демонстрирующих, как учет (или игнорирование) значимых данных может повлиять на постановку целей.

2.4. Цена ошибки

Говоря о цене ошибки, стоит привести цитату Стива Макконнелла (Steve McConnell), который в своей книге «Code Complete»[2] точно формулирует разницу между устойчивостью и корректностью на примерах создания рентгеновского аппарата и видеоигры:

> *Как нам показали примеры с видеоигрой и рентгеновской установкой, выбор подходящего метода обработки ошибки зависит от приложения, в котором эта ошибка происходит. Кроме того, обработка ошибок в общем случае может стремиться либо к большей корректности, либо к большей устойчивости кода. Разработчики привыкли применять эти термины неформально, но, строго говоря, эти термины находятся на разных концах шкалы. Корректность предполагает, что нельзя возвращать неточный результат; лучше не вернуть ничего, чем неточное значение. Устойчивость требует всегда пытаться сделать что-то, что позволит программе продолжить работу, даже если это приведет к частично неверным результатам.*
>
> ...
>
> *Приложения, требовательные к безопасности, часто предпочитают корректность устойчивости. Лучше не вернуть никакого результата, чем неправильный результат. Радиационная машина — хороший пример применения такого принципа. В потребительских приложениях устойчивость, напротив, предпочтительнее корректности. Какой-то результат всегда лучше, чем прекращение работы. Текстовый редактор, которым я пользуюсь, временами показывает последнюю на экране строку лишь частично. Хочу ли я, чтобы при обнаружении этой ситуации редактор завершал выполнение?*

---

[1] Opus magnum (лат. «великая работа») — лучшее творение, созданное кем-либо. — *Примеч. пер.*

[2] Макконнелл С. «Совершенный код».

Данная концепция даже более актуальна для ML-систем, поскольку они, как правило, остаются непрозрачными как для разработчиков, так и для конечных пользователей. Набор операторов *if* и *while* гораздо проще держать в уме, чем длинные цепочки матричных умножений в современных глубоких нейронных сетях.

Представьте, что вы разрабатываете развлекательное приложение, например AR-маску для Snap или TikTok. В худшем случае добавленный эффект будет выглядеть некрасиво в кадре — это не представляет серьезного риска, и поэтому подход, ориентированный на устойчивость, вполне оправдан. Противоположный случай — ML-система для медицинской или транспортной сферы. Хотели бы вы, чтобы беспилотный автомобиль продолжал движение вперед, даже если он не уверен, есть ли поблизости пешеход? Конечно же нет. Именно поэтому в таких случаях следует ориентироваться на корректность.

Мы подробнее рассмотрим баланс между данными понятиями и его практические аспекты в третьей части книги. Пока же следует отметить, что понимание цены ошибок — один из ключевых моментов при сборе информации на этапе до проектирования. По сути, это количественная формализация концепции рисков: сначала мы определяем, что может пойти не так и чего мы хотим избежать, а затем стараемся присвоить этим рискам численные параметры. Численные оценки могут сильно различаться в зависимости от проблемы и не обязательно должны быть точными на данном этапе, но они крайне важны для понимания общей картины.

По нашему опыту, люди чаще склонны сосредоточивать свое внимание на положительных сценариях, тогда как на практике именно негативные исходы требуют большего внимания. Логика здесь проста: у любой системы обычно есть один (или несколько) положительный сценарий и множество потенциальных точек отказа, относящихся к негативным сценариям. Конечно, вероятность каждой отдельной ошибки, как правило, намного ниже вероятности успешного исхода, но это не всегда так, если мы оцениваем ожидаемые значения. Представьте торговую систему, которая зарабатывает несколько центов в 99 % сделок, но с вероятностью 0.1 % теряет весь капитал. Или, чтобы сделать пример еще более утрированным, представьте диагностическую медицинскую систему, которая экономит по 3 минуты от каждого приема, но при этом пропускает серьезное, но излечимое заболевание у каждого тысячного пациента.

Некоторые ошибки, однако, могут быть безвредными или даже приводить к положительным результатам. В 2018 году Арсений работал в компании, разрабатывающей приложение дополненной реальности — виртуальную примерку обуви. Приложение позволяло пользователю увидеть, как пара обуви будет выглядеть на его ногах, до покупки. Одна из первых версий приложения содержала недообученную модель, отвечающую за обнаружение и отслеживание стоп. В результате обувь часто отображалась не только на человеческих

ногах, но и на лапах домашних животных и даже на игрушках. Многим ранним пользователям приложения это показалось забавным, так что цена такой ошибки была незначительной. Однако со временем этот эффект исчез, после того как производительность модели была улучшена для более типичных пользовательских сценариев.

Оценивая стоимость ошибки, следует также помнить о возможности вторичных последствий. Например, ваша система защиты от мошенников может сегодня заблокировать слишком много добросовестных пользователей, и завтра они начнут рассказывать другим о вашем приложении («Никогда им не пользуйтесь — меня забанили ни за что»), и это может подорвать ваш потенциал роста. Ваша система рекомендаций выдает нерелевантные предложения, и в итоге вы обучаете новую модель на логах редких кликов по таким неудачным рекомендациям, попадая тем самым в замкнутый цикл отрицательной обратной связи.

Классическим примером цены ошибки является кредитный скоринг (оценка кредитного риска) — типичная задача, встречающаяся практически в любом банке. Прежде чем кредитная заявка будет одобрена или отклонена, она, как правило, обрабатывается системой на основе ML, которая выдает значение кредитного риска. Это значение может быть бинарным (1/0 при заданном пороге) или непрерывным в диапазоне от 0 до 1.

Очевидно, что цена выдачи кредита потенциальному заемщику, который не сможет его вернуть, и отказа в том же кредите клиенту, который вернул бы его вовремя, — это совершенно разные вещи. Сколько клиентов должны вернуть кредит, чтобы покрыть убыток от одного невыплаченного? Следует ли учитывать количество людей или количество выданных кредитов? Следует ли ориентироваться на общую сумму выданных средств? Ожидаем ли мы, что это соотношение будет постоянным во времени? Ответы на все эти вопросы и учет этой информации существенно увеличивают шансы проекта на успех.

Что это означает для специалиста, разрабатывающего ML-систему? Выявление картины рисков помогает понять, каких проблем следует избегать. Некоторые ошибки практически безвредны, некоторые могут сильно повлиять на бизнес, а некоторые представляют угрозу для жизни. Корректное понимание цены ошибки применительно к разрабатываемой системе критически важно для последующих этапов, поскольку оно формирует требования к надежности и сбору данных, помогает выбрать более подходящие метрики и может повлиять на другие аспекты проектирования.

Итоги

- Пространство проблем всегда должно определяться раньше пространства решений. В противном случае велика вероятность возникновения негативных последствий на более поздних этапах проекта.

- При сборе исходной информации от стейкхолдеров и участников проекта начинайте с более широкой картины с возможностью при необходимости углубиться в детали.
- При выборе из множества возможных решений на базе ML исследуйте их ограничения и оценивайте риски, которые эти ограничения могут за собой повлечь.
- Всегда оценивайте потенциальную цену ошибки. Если ошибка возможна, проанализируйте ее побочные эффекты: некоторые из них могут оказаться даже положительными.

3 Предварительное изучение

В этой главе

- ✓ Применение кейсов из других сфер к вашей проблеме
- ✓ Ответ на вопрос «Создать или купить?» при выборе подходящего решения
- ✓ Декомпозиция проблемы
- ✓ Выбор подходящего уровня инновации

В главе 2 мы выяснили, что определение проблемы — это отправная точка в разработке успешной системы машинного обучения. Чем точнее и подробнее вы опишете проблему, тем легче будет создать продукт, эффективно достигающий необходимых бизнес-целей.

Теперь мы перейдем к ключевым моментам следующего важного этапа проектирования комплексной и эффективной ML-системы — пространству решений. В этой главе мы поговорим о поиске решений, которые ранее уже смогли справиться с аналогичными задачами, о сложном выборе между самостоятельной разработкой компонентов и использованием сторонних решений, о корректном подходе к декомпозиции проблемы, а также о выборе оптимального уровня инновации в зависимости от ключевых целей будущей системы.

3.1. Какие проблемы способны вдохновлять?

> Если я видел дальше других, то потому, что стоял на плечах гигантов.
>
> *Исаак Ньютон*

Представьте, что вы работаете в сервисе по заказу такси, например Uber или Lyft, и сталкиваетесь со следующей распространенной схемой мошенничества: водитель начинает работать в компании, но позже передает свой аккаунт другому человеку, который не имеет права управлять транспортным средством (возможно, у него даже нет действительного водительского удостоверения). Ваша задача — выполнить повторную идентификацию личности водителя, сопоставив фотографию из документа, загруженного при регистрации, с селфи, сделанным водителем из автомобиля, и убедиться, что на обоих изображениях — один и тот же человек. При этом есть определенные нефункциональные требования: в целях защиты конфиденциальности желательно не загружать фотографию водителя с его устройства на сервер. Еще один важный момент — проверка должна быть быстрой и устойчивой к различным атакам со стороны злоумышленников (которые бывают весьма изобретательны!).

Подытожим кейс:

- Задача основана на распознавании лиц. Следовательно, как архитектор системы, вы должны быть знакомы с этой предметной областью.
- Решение должно быть ориентировано на мобильные устройства. Значит, вам понадобятся знания в области ML на мобильных устройствах.
- Решение должно быть устойчивым к попыткам мошенничества (нечестный водитель может попытаться показать на камеру фотографию настоящего водителя вместо собственного лица). Следовательно, понадобится опыт определения «живости» изображения.

Подобные задачи широко распространены в индустрии, однако редко выполняются одновременно в рамках одного решения. Приведем несколько примеров. Крупные системы видеонаблюдения (например, используемые в аэропортах) применяют технологии распознавания лиц, но они, как правило, не ограничены в вычислительных ресурсах, и проверка проводится не на мобильных устройствах. Многие развлекательные приложения, напротив, выполняют инференс непосредственно на смартфоне, и их разработчики умеют запускать модели в условиях ограниченных ресурсов. Определение «живости» изображения обычно используется в биометрических системах для аутентификации (самый распространенный пример — FaceID на iPhone).

Ничто не заменит реального опыта, поэтому если вам посчастливилось в своей карьере успешно справиться со всеми тремя проблемами, вы сможете выполнить

и эту. Если же нет, мы рекомендуем сначала уделить время изучению кейсов из различных сфер, потому что широкий кругозор — ваш лучший союзник. За один год вы вряд ли поработаете с десятками разных ML-систем, запущенных в эксплуатацию, но изучить такое количество кейсов вполне реально — и это может компенсировать недостаток опыта.

При проектировании системы полезно учитывать уже существующие аналогичные решения и использовать их в качестве ориентира. Вам вовсе не обязательно в точности копировать определенные подходы, но они могут послужить источником вдохновения. Мы также рекомендуем не пренебрегать историями неудач — они могут подсказать, чего следует избегать. Данный подход частично перекликается с понятием антицелей, о котором мы поговорим в главе 4.

Как это часто бывает в мире ПО, существует как минимум два уровня сходства: предметный и технический (рис. 3.1).

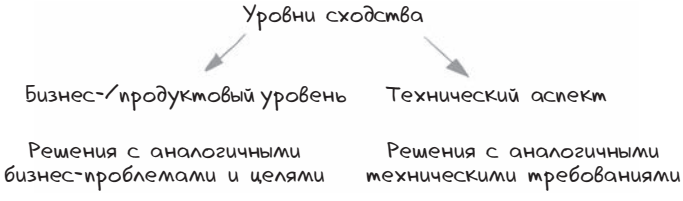


Рис. 3.1. Оба уровня одинаково важны при поиске решений, которые помогут вам построить систему

Первый связан с поиском систем, максимально близких по бизнес-проблематике; второй — с нахождением решений, схожих по техническим требованиям (например, платформа, задержка, модель данных, объем и т. д.).

Байка от Арсения

Я работал над задачей сегментации изображений в компании, занимающейся оптимизацией производственных процессов. Моя задача состояла в нахождении конкретных компонентов на изображениях, полученных со сборочной линии. Особенность заключалась в необходимой точности — субпиксельной. Иными словами, система должна была выдавать крайне детализированные результаты при поиске крайне мелких объектов.

Сегментация изображений для производственных данных — это нетипичная задача, и ее нельзя просто загуглить и взять первое попавшееся решение из интернета. Однако сегментация с высокой точностью востребована в других областях, например в анализе медицинских изображений и редактировании фото/видео, где она называется матированием изображения (image matting).

Если вы когда-либо пытались сменить фон в приложении Zoom, то наверняка замечали артефакты вокруг волос — это как раз связано с тем, что используемый алгоритм далек от идеала (скорее всего, он оптимизирован по вычислительной эффективности, а не по мелкозернистой точности). Сегментация волос — класси-

ческий пример задачи матирования: отделить волосы от фона сложно, и требуются специальные подходы, такие как отказ от понижения разрешения изображения, когда это возможно, а также использование «мягких меток» — пикселей, одновременно отнесенных и к переднему, и к заднему плану.

Исходя из этого я изучил передовые методы матирования и применил их к производственным данным, что в конечном итоге позволило мне значительно снизить вероятность ошибки на тестах.

Мы также рекомендуем задать себе вопрос: почему в кейсах, которые вы считаете релевантными вашей проблеме, были приняты те или иные решения? Подобные размышления полезны для формирования и дальнейшего применения интуиции в процессе проектирования сложных ML-систем, включая способность выбирать, стоит ли создавать свое решение с нуля или же воспользоваться готовыми предложениями.

3.2. Создать или купить — технологии с открытым исходным кодом или коммерческие

Представьте, что вы работаете в Slack — мессенджере с поддержкой аудио- и видеозвонков. В нем реализована функция распознавания речи в аудиозвонках почти в реальном времени.

Однако Slack изначально задумывался как мессенджер с приоритетом на текстовое общение, и, вероятно, доля пользователей, использующих голосовые звонки, существенно меньше. А текстовые субтитры применяются еще реже, поскольку функция по умолчанию отключена, а ее использование ограниченно. При этом требования к точности распознавания речи высоки, ведь функция будет бесполезной или даже вредной, если качество распознавания не будет соответствовать ожидаемому уровню.

Потребность в качественной, но не ключевой функциональности может побудить вашу продуктовую команду изучить рынок в поисках готовых решений. При этом нельзя игнорировать масштаб компании: до пика пандемии в 2019 году у Slack было 12 миллионов активных пользователей в день. Сейчас, по заявленным данным, это число снизилось до 10 миллионов, но оно все еще впечатляет. Это означает, что использование сторонней технологии от внешнего поставщика может оказаться слишком затратным, и запуск разработки внутреннего, идеально адаптированного решения может быть более предпочтителен. Какой же путь выбрать?

3.2.1. Создать или купить

В сложных системах, включая ML-системы, часто существует дилемма — создать самим или купить? Если проблема, которую надо решить, достаточно распространена, то скорее всего, найдется поставщик, предлагающий готовое решение.

Давайте рассмотрим основные моменты, которые необходимо учитывать при ответе на вопрос, вынесенный в заголовок подраздела.

Является ли проблема частью ключевой бизнес-функции? Распространенная практика — сосредоточиться на ключевых конкурентных преимуществах и использовать сторонние сервисы для типовых задач, таких как инфраструктура. Пятнадцать лет назад у большинства компаний были собственные системные администраторы, управлявшие масштабными серверами в дата-центрах; сегодня же большинство арендует виртуальные машины у облачных провайдеров. Это пример использования стороннего сервиса для критически важной инфраструктуры, которая, тем не менее, не играет решающей роли в борьбе за рынок. Хотя существуют исключения — компании, где инфраструктура серверов имеет ключевое значение (например, высокочастотный трейдинг, рекламные технологии или облачный гейминг); в таких сферах активно инвестируют в собственные R&D (исследования и разработку).

Многие компании используют сторонние сервисы для таких задач ML, как машинный перевод, распознавание речи, борьба с мошенничеством и др. Сравнение селфи водителей с фотографией из водительского удостоверения — пример задачи, которую можно делегировать внешнему поставщику.

Другая сторона вопроса — экономическая. Предположим, существует поставщик, предлагающий решение вашей проблемы, и его сервис демонстрирует достаточно хорошие метрики, но при этом его цена вам не подходит. А может, ваша компания нанимает сотрудников из регионов с низкой стоимостью жизни (и, соответственно, более низким уровнем зарплат), и поэтому разработка собственной системы с нуля окажется дешевле, чем использование стороннего решения. Если поставщик предлагает разумную цену для стартапа из Калифорнии с венчурным финансированием, это еще не означает, что та же цена будет устраивать компанию, основанную без внешних инвестиций в Восточной Европе или Азии.

Вы можете перейти на решение с открытым исходным кодом, однако выбор между таким вариантом и коммерческим продуктом может быть неоднозначным. Нельзя сказать, что стоимость решения с открытым кодом равна нулю, поскольку его сопровождение зачастую сопряжено со скрытыми затратами, связанными с инфраструктурными задачами и возможными проблемами, требующими вашего вмешательства. С другой стороны, использование коммерческого решения позволяет делегировать большинство таких проблем поставщику. Это подразумевает необходимость предварительной оценки потенциальных затрат до того, как вы остановитесь на каком-либо варианте.

Существует еще один момент, который редко обсуждается, но при этом имеет большое значение в данном вопросе, — карьеризм. Не каждое решение принимается исключительно в интересах бизнеса, и чем крупнее компания, тем чаще встречается такая практика. Некоторые сотрудники могут быть заинтересованы в разработке собственного решения, а не в покупке готового, чтобы реализовать проект с высоким «эффектом воздействия» и, таким образом, обосновать свое

продвижение по карьерной лестнице или пополнить резюме впечатляющим достижением. Разумеется, мы не поддерживаем такой подход, но поскольку подобные случаи нередки, мы не могли не упомянуть их.

В целом дилемма «создать или купить» сводится к нескольким ключевым факторам, определяющим контекст вашей работы. Приобретение готового решения позволяет сэкономить время на разработке (что особенно актуально для стартапов со сжатыми сроками релиза) и избежать необходимости нанимать дополнительных специалистов, которые могут быть незаменимы на этапе внедрения, но для которых впоследствии сложно будет найти задачи. Да, вы получаете проверенную временем платформу. Однако вы будете зависеть от графика поставщика при выпуске обновлений и патчей. Разработка собственного решения дает вам полный контроль над функционалом, масштабируемостью и графиком релизов, а также возможность оперативно устранять критические ошибки без привязки к внешнему поставщику. Но такой контроль сопровождается повышенными затратами: потребуется внутренняя техническая поддержка, а также сильная команда опытных разработчиков.

Мы рекомендуем выбор «купить», если:

- Вам важен быстрый выход на рынок.
- У вас нет отдельной команды для разработки и сопровождения решения.
- На данное решение существует высокий спрос со стороны множества компаний из разных отраслей.

Обязательно выбирайте стабильную платформу с проверенной репутацией на рынке. Мы рекомендуем выбор «создать», если:

- У вас есть достаточно времени на реализацию решения.
- Вам важнее масштабируемость и гибкие обновления по мере необходимости, а не строгое следование календарю релизов.
- Вы можете позволить себе внутреннюю поддержку.
- Вам требуется не просто «достаточно хорошее», а передовое решение.
- У вас есть множество унаследованных (легаси) систем, что требует бесшовной интеграции.
- Вы работаете с чувствительными данными и не можете позволить себе полагаться на внешнюю информационную безопасность или ограничены нормативными требованиями, запрещающими передачу данных третьим сторонам.

Существует также чрезвычайно важный бюджетный аспект, который нельзя игнорировать, но при этом его невозможно отнести ни к одной из ранее рассмотренных категорий. Это связано с тем, что бюджет может повлиять на решение в любую сторону. Вы можете выбрать вариант покупки, если разработка собственного решения может привести к перерасходу. С другой стороны, вариант разработки будет предпочтительным, если ни одно из готовых решений не

укладывается в бюджет. Какой бы ни была ваша ситуация, бюджет — это ключевой элемент, который всегда необходимо учитывать.

Вернемся к исходному примеру со Slack. Одним из способов разрешить данную дилемму может быть сначала выбор стороннего поставщика, проверка востребованности функционала у пользователей, определение основных сценариев использования и затем уже запуск собственной разработки на основе полученной информации.

Напоминание: нам неизвестно, как именно в итоге была реализована данная функция. Мы лишь описываем подход, который применили бы сами.

Со временем тенденции в вопросе «Создать или купить?» меняются. Например, по крайней мере девять из десяти задач в области обработки естественного языка, которые в 2010-х годах требовали бы создания полностью кастомизированного решения, в 2020-х решаются с помощью одного вызова API большой языковой модели (large language model, LLM), что делает самостоятельную разработку таких решений гораздо менее эффективной.

3.2.2. Технологии с открытым исходным кодом или коммерческие решения

У вас может возникнуть и другой вопрос — использовать ли технологию с открытым исходным кодом или корпоративное проприетарное платное решение. На каком-то этапе вам нужно будет решить, какая база данных будет использоваться для хранения данных или какой сервер инференса предпочтительнее. Чтобы ответить на эти вопросы, важно обладать глубоким пониманием нефункциональных требований (например, необходимое время безотказной работы, допустимая задержка, устойчивость к нагрузке и т. п.). Для первоначальной оценки логика следующая: если вы уверены, что вам не потребуется срочная помощь экспертов, безопасным выбором будет решение с открытым исходным кодом. Противоположный случай — когда вы строите высоконагруженную, критически важную для бизнеса систему, тогда имеет смысл «стоять на плечах гигантов», то есть использовать решения конкретных проверенных поставщиков. Есть и смешанные сценарии — можно приобрести корпоративную поддержку для решений с открытым кодом, порой это весьма удачный компромисс.

Стоит отметить, что описанные выше принципы применимы не только к машинному обучению, — те же соображения актуальны и при проектировании «обычного» программного обеспечения, не связанного с ML.

3.3. Декомпозиция проблемы

Одним из наиболее полезных инструментов в арсенале любого инженера-программиста является подход под названием «разделяй и властвуй», который прекрасно применим в ML как на уровне реализации низкоуровневых алгоритмов,

так и при проектировании высокоуровневых систем. Это первое, что можно сделать при столкновении со сложной проблемой, поначалу кажущейся неразрешимой.

Типичным примером декомпозиции проблемы является проектирование поисковой системы. Пользователь может ввести любую произвольную комбинацию слов, в том числе такие, которые ранее ни разу не запрашивались (около 15 % поисковых запросов в Google являются новыми), и при этом получить релевантный результат всего за несколько сотен миллисекунд.

На высоком уровне поисковая система эффективно выполняет одну задачу — она быстро предоставляет релевантные результаты из базы данных. Но давайте сосредоточимся на двух ее свойствах — релевантности и быстродействии. Будет ли проще быстро получить относительно релевантный результат? Мы считаем, что да, — достаточно отказаться от сложных алгоритмов ранжирования и заменить их простым алгоритмом «документ содержит некоторые из запрашиваемых слов». Сканирование всей базы данных по такому условию вполне выполнимо. Будет ли проще найти релевантные результаты среди небольшой выборки документов — тысяч, а не миллиардов? Конечно, ведь на меньшем объеме можно применить более сложные ML-алгоритмы и большие, пусть и медленные, модели.

Вы наверняка уже догадались, к чему мы ведем: пришло время объединить все эти шаги и построить двухэтапную систему. Первый этап — это быстрый отбор кандидатов, а второй — более сложное ранжирование среди ранее отобранных документов. Такой подход используется в большинстве поисковых систем на протяжении десятилетий.

Можно пойти дальше и вместо одной итерации фильтрации кандидатов применить каскадную структуру. Так, на основании языка запроса и местоположения пользователя документы на других языках могут быть исключены еще до этапа отбора кандидатов, что уменьшит количество документов, подлежащих дальнейшей обработке (рис. 3.2).

Аналогичные системы широко применяются в области компьютерного зрения: сначала используется модель глубокого обучения, а затем уже идет этап постобработки, отвечающий за формирование окончательного ответа. То же самое наблюдаем и в случае с текстами и другими полуструктурированными данными: сначала извлекаются структурированные данные, а затем они уже обрабатываются более узкоспециализированными моделями.

Можно выделить шесть причин провести декомпозицию:

- *Сложность вычислений* — декомпозиция применяется для снижения объема необходимых вычислений (как показывает пример с поисковой системой).
- *Несовершенство алгоритма* — следующий шаг корректирует ошибку, допущенную на предыдущем. Читатели, хорошо знакомые с теорией ML, могут провести параллель с алгоритмами бустинга.

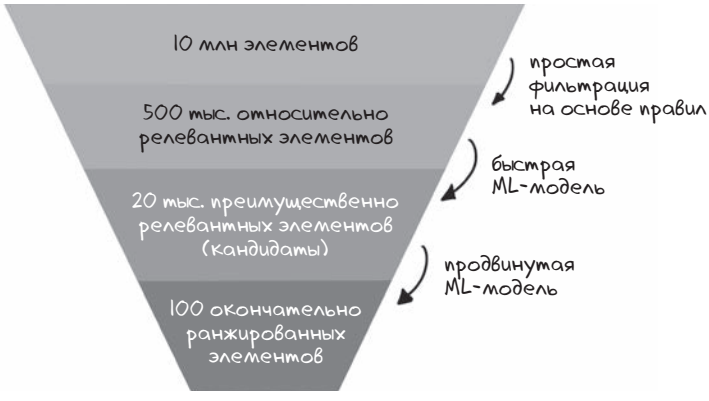


Рис. 3.2. Процесс декомпозиции проблемы

- *Использование сильных сторон алгоритма и избегание слабых* — например, когда требуется подсчитать количество объектов на изображении. Можно обучить сверточную нейронную сеть для решения проблемы регрессии, но классические сверточные нейросети по своей архитектуре не совсем подходят для таких задач (например, пулинговые слои часто теряют информацию подобного рода — рис. 3.3). В качестве альтернативного подхода можно использовать модель, обнаруживающую объекты на изображении, и классический алгоритм компьютерного зрения поверх нее для подсчета контуров, выделенных на предыдущем этапе. Модель, ориентированная только на детекцию, способна к лучшей обобщаемости, чем сквозная регрессионная модель, за счет встроенного индуктивного смещения, а шаг постобработки при этом детерминирован и точен.

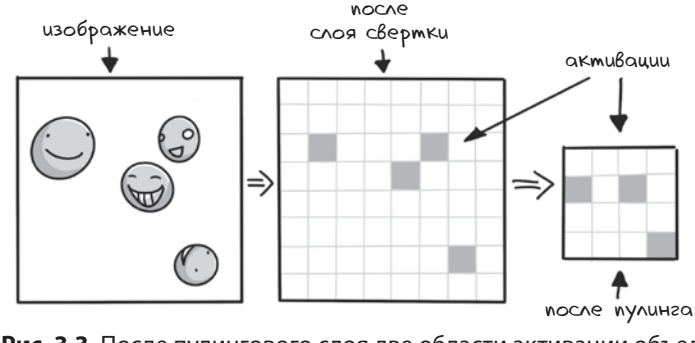


Рис. 3.3. После пулингового слоя две области активации объединились, в результате чего была утеряна информация, критически важная для подсчета объектов

- *Потребность в объединении данных* — решения на базе ML не могут напрямую извлекать данные из других источников, поэтому распространенной практикой является запуск модели, получение дополнительных данных на основе результата и последующая обработка информации. Примером может

послужить недавний интерес к приложениям на базе больших языковых моделей (LLM): множество специализированных LLM-решений реализуются с помощью генерации, дополненной поиском, что, по сути, заключается в извлечении релевантных данных из векторной базы и предоставлении этих данных модели в промте.

- *Обработка крайних случаев* — решения на базе ML могут давать сбои, в таком случае декомпозиция позволяет выявлять проблемы на раннем этапе. Например, простая модель (или просто набор условных проверок) может оценивать входные данные и выдавать ошибку, если есть вероятность, что они некорректны.
- *Применение различных моделей или логики к разным подмножествам данных* — возможно, модель работает хорошо для широкой категории пользователей, но плохо распространяется на всю аудиторию. Отсюда вытекает идея направления пользователей к разным моделям или ветвям системы на основе какого-либо признака (например, использование отдельных моделей для разных географических регионов). Более подробно мы рассмотрим это в главе 14.

Конечно, это неполный список, но он выделяет шесть наиболее очевидных причин для декомпозиции, с которыми мы сами сталкивались в работе.

Иногда на ранних этапах пайплайны проектируются непоследовательно, и позже возникает идея добавить еще один шаг в качестве усовершенствования. Это не лучший подход: последовательное добавление новых компонентов для устранения проблем, выявленных на предыдущих этапах, может привести к неустойчивому дизайну, подверженному ошибкам и трудному в сопровождении, так как он не следует единой концепции. С другой стороны, вполне допустимо на этапе проектирования и первых реализаций оставлять так называемые заглушки («в дальнейшем здесь будет отбор кандидатов моделью, но пока используем случайную выборку в качестве доказательства концепции»).

Принципы проектирования ML-систем в настоящее время находятся под влиянием последних тенденций в данной области. Ранее было распространено построение пайплайнов, состоящих из множества небольших последовательно выполняемых компонентов. Однако с появлением и ростом популярности моделей глубокого обучения тенденция сместилась в сторону использования единой модели, охватывающей весь процесс от начала до конца. Такой подход позволяет выявлять потенциально более сложные зависимости в данных, поскольку он не связан предположениями и ограничениями, присущими ручному проектированию, требует меньшего объема предметных знаний и снижает вероятность накопления ошибок между этапами обработки.

Обработка речи — хороший пример того, как данный подход изменил архитектуру систем. До появления сквозных моделей системы преобразования текста в речь (text-to-speech, TTS) обычно включали два основных компонента: один обрабатывал текстовый ввод и преобразовывал его в лингвистические элементы,

такие как фонемы, ударения и интонационные шаблоны, а другой — синтезировал речь на основе набора правил или статистической модели, сопоставляя лингвистическую информацию со звуковыми волнами.

Байка от Арсения

Я работал в компании, занимающейся дополненной реальностью, и разрабатывал решение для виртуальной примерки. Один из продуктов позволял примерить обувь — это было приложение, которое определяло положение стопы на видео и накладывало выбранную обувь. Для реализации требовалось объединить множество алгоритмов, включая алгоритм окклюзии, который определял, какая часть обуви должна быть видимой и отображенной, а какая скрыта объектами в кадре.

Эта часть решения доставила множество проблем перед первым релизом; у команды не было идей, как реализовать ее корректно и надежно. В какой-то момент технический директор компании взял инициативу на себя и предложил собственный алгоритм, который решал проблему в большинстве случаев. У этого алгоритма было множество недостатков: он был не особо быстрым, плохо обобщался, был непонятным остальной команде и т. д. Но у него было одно большое преимущество, перевешивающее все остальное, — он работал!



Пример решения на основе ML, при котором обувь отображается только в тех областях, где в кадре могла бы быть реальная обувь

Алгоритм нашего директора стал частью раннего дизайна и реализации и оказался ценным компонентом первых версий продукта. Позже команда собрала совершенно иной подход, который в основном устранял недостатки предыдущего. Благодаря правильно спроектированной архитектуре значительные изменения в системе не потребовались — просто один шаг в пайплайне был заменен на более продвинутый, что улучшило пользовательский опыт в последующих версиях.

Сквозные модели TTS, напротив, не опираются на языковую форму. Вместо этого они напрямую преобразуют текст в звуковую волну с помощью одной нейронной сети. Хотя сквозные модели показали высокую эффективность, они не обладают встроенными знаниями и во многих приложениях требуют подключения к базам данных.

В последнее время большие языковые модели (LLM), такие как GPT-4, продемонстрировали впечатляющие возможности в zero-shot, то есть способность отвечать на вопросы без дополнительного ввода или обучения. Однако такие модели являются ресурсоемкими и могут генерировать недостоверную информацию (так называемые галлюцинации — представление ложных данных как правдивых. Больше информации об этом можно найти в «Survey of Hallucination in Natural Language Generation» (https://arxiv.org/abs/2202.03629). Кроме того, знания, которыми обладают модели, не поддаются модификации.

В настоящее время ведутся активные исследования, направленные на объединение преимуществ LLM с возможностью использования внешних источников информации. Например, Bing AI и плагины ChatGPT (https://openai.com/blog/chatgpt-plugins) используют дополнительные онлайн-источники, аналогично тому, как люди пользуются поисковыми системами. Разработка Galactica (https://galactica.org/) от Meta AI стала одной из первых реализаций концепции токена с памятью (working memory token), позволяющего модели генерировать фрагмент кода на Python, который затем может быть выполнен интерпретатором для получения точного ответа. Такая идея в дальнейшем получила развитие в Toolformer (https://arxiv.org/abs/2302.04761v1) — модели, специально обученной использовать различные сторонние API. Аналогичные концепции реализуются в быстро развивающемся фреймворке с открытым исходным кодом LangChain (https://python.langchain.com/api_reference). И хотя данные подходы пока не получили широкого применения в промышленных системах, они способны изменить сам принцип декомпозиции ML-систем.

В зависимости от сложности и степени новизны, ML-системы могут предполагать различные уровни инновационности. Некоторые конкурентные области требуют значительных инвестиций в научные исследования, тогда как в других можно использовать самые базовые решения. Поэтому давайте разберемся, как определить необходимый уровень инновации для вашей системы.

3.4. Выбор подходящего уровня инновации

Задайте любому стейкхолдеру, участвующему в создании ML-системы, простой вопрос: насколько хорошим (то есть точным) должен быть итоговый продукт? Скорее всего, вам ответят что-то вроде: «идеальным», «стопроцентным» или «настолько хорошим, насколько это возможно». Но давайте попробуем разобраться, что скрывается за этими простыми, но неоднозначными формулировками.

Ответ «настолько хорошим, насколько это возможно» на самом деле означает «насколько позволяют прочие ограничения». Самые очевидные из них — это

время и бюджет. Хотели бы они получить идеальную ML-систему через 10 лет? Скорее всего, нет. Отдадут ли они предпочтение системе, которая «достаточно хороша» и готова к концу следующего квартала? Почти наверняка — да.

Мы подробнее остановимся на различиях между «достаточно хорошо» и «идеально» позже, в главе 5. На самых ранних этапах проектирования точное значение метрик еще не столь критично. Важно общее понимание.

Исходя из нашего опыта в создании, сопровождении и улучшении ML-систем различных масштабов и с разными целями, мы выделили три категории систем относительно требуемого уровня совершенства. Терминология может различаться, но, на наш взгляд, именно такие описания подходят лучше всего:

- минимально жизнеспособная ML-система;
- стандартная ML-система;
- ML-система высшего класса.

Минимально жизнеспособная система — это некое спартанское решение, сделанное, так скажем, с помощью изоленты и клея. Ожидания от такой системы сводятся к «в основном работает», при этом сторонний наблюдатель может столкнуться с различными сценариями сбоев. Подобные системы представляют собой базовые решения или прототипы, инноваций от них не ожидают.

Стандартная ML-система уже поднимает планку. Многие успешно функционирующие ML-системы порой даже не соответствуют и этому уровню, но все равно приносят пользу бизнесу. Поэтому можно утверждать, что достижение такого уровня требует значительных исследований и инноваций.

И наконец, существует категория «высшего класса». Некоторые системы окажутся бесполезны, если не будут превосходить своих конкурентов, — такое часто бывает в сверхконкурентных областях, таких как трейдинг, рекламные технологии или какие-то глобальные продукты вроде поисковых систем. Лишь небольшое улучшение точности уже может обернуться для них миллионами долларов прибыли или убытков, поэтому такие ML-системы проектируются с целью добиться максимально возможного результата.

Зачем мы вообще говорим об инновациях? Связь между пространством проблем и пространством решений напрямую зависит от уровня инноваций, установленного для проекта. У минимально жизнеспособных систем их ровно ноль — мы просто используем самое простое и быстрое из доступных решений и двигаемся дальше. На другом же конце спектра — постоянные инновации, когда система никогда не бывает полностью готова, а команда не перестает искать новые улучшения для следующего релиза.

Отнести ту или иную проблему к одной из трех категорий непросто, но крайне важно. Тут есть один ключевой фактор, который нельзя игнорировать: необходимый уровень инновации никогда не бывает постоянным. Во многих случаях,

особенно в стартапах — системы изначально создаются минималистичными, но с возможностью последующего улучшения, что логично: сначала компания оценивает, действительно ли данный функционал востребован клиентами (или внутренними пользователями), и только затем улучшает систему на основе обратной связи. Если выпущенная функция уникальна на рынке, даже ее базовая реализация уже способна принести значительную ценность, что заставит конкурентов улучшать свои собственные продукты. Это перемещает исходную систему из первой категории во вторую или даже приближает ее к числу передовых технологий. Многие стартапы сталкиваются с трудностями на этапе такого перехода, и поэтому случаи проектирования системы, которая затем от прототипа эволюционирует до технологии мирового уровня (в чем и заключается инженерное искусство), крайне редки. Упрощенный подход — создание системы, которую впоследствии можно переработать, сохранив как можно больше реализованных компонентов, — к этому уже стоит стремиться.

3.4.1. Как найти эффективное решение?

Зная требуемый уровень инноваций и имея общее представление о структуре системы, можно начать поиск идей для ее реализации. На момент написания этой главы существовало пять популярных источников информации, с которыми стоит ознакомиться.

arXiv

arXiv (https://arxiv.org/) — это сайт, на котором размещаются научные статьи, в основном связанные с технологиями, инженерией и математикой. Именно математике, computer science и их поддисциплинам посвящена значительная доля из более чем 2 млн опубликованных на данном ресурсе работ.

arXiv позволит вам ознакомиться с академической стороной вашей проблемы. Помимо чтения всего, что можно найти по вашим ключевым словам, мы также рекомендуем использовать механизм ссылок и цитирований: найдя релевантную статью, не забудьте заглянуть как в более старые публикации, упомянутые в ней, так и в новые работы, цитирующие ее. arXiv — это особая экосистема: существуют расширения для браузеров и дополнительные сайты, облегчающие поиск. Для начала поищите обзорные статьи (в их названии часто присутствует слово «survey»), ведь в них содержится тщательно отобранная и структурированная информация по вашей теме.

Сама по себе платформа arXiv как источник информации может показаться немного «сырой» — прочитать все новые статьи практически невозможно, а механизм поиска по современным меркам выглядит довольно примитивно. Однако существует множество инструментов, облегчающих навигацию по сайту. Мы рекомендуем https://arxivxplorer.com/ — современный поисковик по аннотациям статей, хотя вполне возможно, что к моменту выхода книги появится другой

удобный инструмент (например, ранее наиболее популярным был сервис https://arxiv-sanity-lite.com/).

Papers with Code

Как несложно догадаться, сайт Papers with Code[1] (https://paperswithcode.com/) представляет собой подборку научных статьей, посвященных машинному обучению и сопровождаемых кодом. Статьи сгруппированы по тематикам и по возможности ранжированы по результатам.

Там вы можете найти проблему, аналогичную вашей, и посмотреть топ статей, ее решающих, их метрики, основную информацию (например, требуются ли тому или иному подходу дополнительные данные) и, что особенно важно, ссылки на открытые реализации. Этот сайт — настоящая находка для тех, кто предпочитает репозитории формальному академическому стилю изложения.

GitHub

Раз уж зашла речь о коде, нельзя не упомянуть GitHub (https://github.com). Это самая популярная платформа для размещения проектов с открытым исходным кодом, где найдется репозиторий на любой случай. Однако масштаб данной платформы — это и ее преимущество, и недостаток: если вы ищете на сайте что-то нестандартное, легче найти иголку в стоге сена.

GitHub не специализируется исключительно на машинном обучении, но при этом подавляющее большинство ML-проектов размещено именно там.

Hugging Face

Хаб моделей Hugging Face (https://huggingface.co/models) — один из крупнейших ресурсов, на котором размещено огромное количество моделей и датасетов. На момент написания главы в хабе находилось более 560 000 общедоступных ML-моделей. Категории и система тегов на сайте работают довольно точно, а значительная часть моделей сопровождается небольшими интерактивными демоверсиями, демонстрирующими их возможности.

Компания Hugging Face начинала свою деятельность с фокусом на обработку естественного языка (NLP), и ее хаб стал основной площадкой для размещения моделей, ориентированных на NLP. Мы рекомендуем использовать данную платформу для поиска исследовательских моделей, если ваша проблема связана с обработкой текста.

Kaggle

Kaggle (https://kaggle.com) — наиболее популярная платформа для проведения соревнований по ML. Различные компании используют ее для запуска

[1] Не доступен в прошлом виде, это часть https://huggingface.co/papers/trending с похожими функциями.

челленджей, чтобы привлечь лучших специалистов в области ML со всего мира, предлагая денежные призы и, конечно же, престиж и уважение. В ходе соревнований участники делятся своими идеями и фрагментами кода, связанными с решением поставленной задачи. А по завершении победители и лидеры, как правило, раскрывают все «секреты» своих подходов. Помимо соревнований, Kaggle также служит площадкой для размещения датасетов, поэтому там можно найти открытый датасет, связанный с вашей проблемой.

Kaggle представляет собой ресурс, отличный от остальных по нескольким причинам. Если соревнование организовано плохо, проблема может оказаться некорректно сформулированной, поэтому, вместо того чтобы решать реальную проблему, участники ищут лазейки, например утечки данных. Кроме того, полученные в итоге решения зачастую оказываются неприменимыми на практике: модели могут быть чрезмерно громоздкими, так как требования к задержке при их создании игнорируются, а фрагменты кода редко бывают чистыми — участники ориентируются на быструю итерацию, а не на долгосрочное сопровождение.

Тем не менее, несмотря на все указанные недостатки, форумы Kaggle могут стать ценным источником информации по вашей проблеме, включая как научные статьи, так и нестандартные подходы, которые со временем могут закрепиться в общей практике. Также стоит упомянуть сайты, собирающие лучшие решения с Kaggle, например https://farid.one/kaggle-solutions/.

Стоит подчеркнуть, что на данном этапе вам пока не требуется выбирать конкретные решения на основе проведенного исследования — оно должно лишь дать вам более полное представление о проблеме, что поможет в дальнейшем принять более обоснованное решение.

3.4.2. Работа в пространстве решений: пример из практики

Повторим основные моменты предыдущего раздела на примере одного конкретного случая. Представим, что вы устроились в компанию, занимающуюся стоковыми фотографиями. По сути, это маркетплейс: фотографы регистрируются на платформе и загружают свои работы, а клиенты, которым нужны конкретные изображения для иллюстраций (редакторы, дизайнеры, специалисты по рекламе), покупают права на эти фото. Компания зарабатывает на комиссии с продаж. Она крайне заинтересована в создании эффективной поисковой системы на своем сайте.

С одной стороны, база изображений огромна — миллионы снимков. Когда клиенты ищут фотографии, они, как правило, заинтересованы в чем-то очень конкретном, что трудно найти с помощью простых категорий или других подобных элементов навигации. Именно поэтому вас и наняли — создать эффективный инструмент поиска, способный находить наиболее релевантные изображения по текстовым запросам клиентов. Как же понять, как устроено пространство решений для данной задачи?

Первым возникает вопрос: создавать свое решение или покупать готовое? Вы предполагаете, что компании такого масштаба, как ваша, чаще разрабатывают собственные решения, но это не всегда так. Стоит провести небольшое исследование. Многие поставщики — как крупные корпорации, так и стартапы — предлагают в качестве услуги свои поисковые движки. Однако при анализе их решений, скорее всего, окажется, что большинство из них вам не подходят — вашей компании нужен поисковик изображений по текстовому запросу, а это не самый распространенный сценарий. Тем не менее могут найтись некоторые поставщики, предлагающие что-то подходящее, поэтому стоит иметь их в виду.

Но сначала рассмотрим схожие проблемы, которые решают другие компании:

- Конечно, существуют и другие фотостоки, и некоторые из них могли создать качественные поисковые движки. Маловероятно, что они раскроют вам какую-то информацию об их реализации (хотя можно попробовать найти блоги или доклады с конференций), но вероятность все же есть.
- Существуют универсальные поисковые системы, такие как Google и Bing. Очевидно, что необходимая вам система имеет совершенно другой масштаб — вы работаете с миллионами изображений, а они — с миллиардами. Здесь может возникнуть вопрос: «Как мне повторить такую махину, если моя команда в N раз меньше?» Разумеется, вряд ли кто-то может конкурировать с такими гигантами, но вам это и не нужно: ваша цель — найти решения, которые подходят под конкретные задачи вашего проекта, не написав при этом ни одной лишней строчки кода.
- В противоположность предыдущему пункту, существуют ориентированные на конечного пользователя проекты, помогающие классифицировать личные фотоархивы. Они рассчитаны скорее на тысячи изображений, чем на миллионы. Но плюс тут в том, что у многих из этих решений открытый исходный код, а значит, можно изучить его и почерпнуть некоторые полезные идеи.
- Наконец, существуют маркетплейсы нецифровых товаров, например, по продаже одежды, мебели и т. д. Некоторые из них — такие гиганты, как Amazon, а другие — нишевые и даже меньше вашей компании. Их бизнес во многом зависит от качества поиска, но при этом товары представлены не только изображениями, но и множеством дополнительных атрибутов (например, текстовое описание или информация о продавце). Такие поисковые системы используют больше данных об объектах, а не только визуальные. В области ML подобные системы называют мультимодальными.

Поисковые системы — один из наиболее распространенных инструментов в сфере информационного поиска. Специалисты в данной области были одними из первых, кто начал применять методы ML, но при этом не ограничивались только лишь ими. Для тех, кто чувствует себя неуверенно в этой теме, полезно освежить знания или ознакомиться с основами, начав, например, с Википедии.

После получения общей картины можно уже перейти к более углубленному изучению извлечения изображений.

При изучении доступной информации по созданию поисковых систем можно заметить повторяющийся паттерн декомпозиции: как и в других поисковых системах, в вашем случае нет необходимости ранжировать все документы. Уже на начальном этапе пользователь может указать конкретные требования: например, чтобы фотография была представлена в формате RAW (а не в сжатом JPEG), имела ширину не менее 5000 пикселей и стоила не более $50. Такие фильтры позволяют сузить выбор с миллионов до десятков тысяч кандидатов еще до того, как мы начнем анализировать семантику изображений и запроса. Такая оптимизация может оказаться крайне полезной и даже лечь в основу архитектуры будущей системы.

Еще один момент, который вы, скорее всего, обнаружите, заключается в том, что на деле большинство поисковых систем выполняют, по сути, одну задачу. Они рассчитывают релевантность между пользовательским запросом и потенциально подходящими объектами (документами), а затем ранжируют эти объекты на основе этого показателя (рис. 3.4):

```
релевантность = f(закодированный текст(запрос), закодированное
изображение(объект))
```

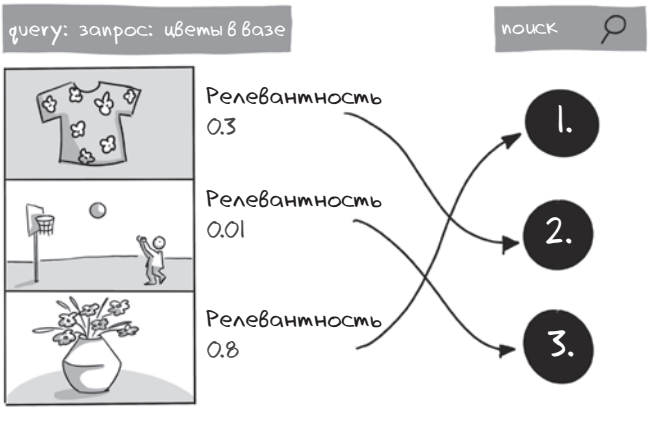


Рис. 3.4. Поисковая система ранжирует изображения по степени релевантности к запросу

В контексте нашего сценария это порождает множество вопросов:

- К какому семейству алгоритмов может принадлежать функция f?
- Как закодировать запрос (текст) и объект (изображение)?
- Как измерить релевантность?
- Как включить обратную связь от пользователей в систему?

Каждый из этих вопросов обширен и заслуживает отдельной книги (или, по крайней мере, нескольких глав), поэтому для краткости мы лишь укажем, что все эти вопросы следует держать в уме, когда вы перейдете к анализу источников информации, упомянутых ранее, — от arXiv до Kaggle.

Следующий вопрос — уровень инновационности, к которой вы стремитесь. Здесь можно выделить несколько моментов:

- У компании уже есть базовая технология поиска. Она устарела и часто выдает нерелевантные результаты, но это все же лучше, чем ничего.
- Бизнес может значительно выиграть от качественного поиска. В настоящее время многие пользователи не могут найти нужный снимок из-за плохих результатов поиска и поэтому покидают сайт, переходя к конкурентам. Качество поиска блокирует и другие инициативы в компании: какой смысл запускать масштабную рекламную кампанию, если новые пользователи с большой вероятностью уйдут, не найдя того, что им нужно?
- Бюджет сильно ограничен. В то же время после первого успешного кейса по данному проекту высока вероятность, что появится дополнительное финансирование для новых инициатив в области R&D.

Первый пункт явно указывает, что простейший, минимально жизнеспособный продукт здесь не подойдет, поскольку таковой уже существует. Однако ограниченный бюджет подразумевает, что стремиться к передовому решению с самого начала тоже не стоит. Следовательно, необходимо разработать надежную систему с ограниченным бюджетом и возможностью дальнейшего улучшения.

В этой главе мы рассмотрели наиболее важные элементы подготовки к написанию дизайн-документа. Теперь вы знаете, как декомпозировать проблему, какие внешние и внутренние факторы будут влиять на ваш ответ на вопрос «создать или купить?», какие онлайн-источники могут быть наиболее полезны и как определить необходимый уровень инновационности вашего решения.

Все эти знания станут основой не только для написания дизайн-документа по вашей системе, но и для понимания того, нужна ли вам вообще ML-система. Последнее утверждение может вас заинтриговать или даже возмутить, поэтому мы постараемся подробнее пояснить его (как и многие другие моменты) в следующей главе.

Итоги

- Ищите те решения на рынке, которые могут полностью или частично удовлетворить ваши потребности. Если таковые найдены, задайте себе вопрос, какие части системы могут использовать эти решения.
- Составьте общее представление о системе. Попробуйте изобразить ее в виде трех-пяти блоков, которые будут понятны даже неспециалисту.

- Ответ на вопрос «создать или купить?» может стать решающим фактором при выборе между созданием собственных компонентов и использованием сторонних решений. Чтобы сделать этот выбор правильно, учитывайте различные внутренние и внешние ограничения, включая доступное время и ресурсы всех задействованных команд.
- Ознакомьтесь со списком потенциальных факторов, указывающих на необходимость декомпозиции. Если проблема, которую вы решаете, соответствует хотя бы одному из них, скорее всего, ее тоже стоит декомпозировать.
- Может возникнуть сильное желание создать передовой продукт, тем не менее сначала следует задать себе вопрос, сколько средств вы готовы вложить в систему ради ее инновационности и окупится ли это.
- Не забывайте изучать популярные онлайн-ресурсы в поиске кейсов, которые можно использовать при создании собственного решения.

4 Дизайн-документ

В этой главе

- ✓ Наиболее распространенные мифы о дизайн-документе
- ✓ Определение антицелей для фокусировки на целях
- ✓ Создание черновика дизайн-документа на основе имеющейся информации
- ✓ Ревью дизайн-документа
- ✓ Эволюция дизайн-документа

Как только вы определили проблему, которую должна решать система, составили список стейкхолдеров и получили общее представление о том, какие технологии и решения наилучшим образом подойдут для продукта (как это описано в главе 3), наступает момент подготовки дизайн-документа.

Здесь стоит отметить, что на раннем этапе создания ML-системы не существует какого-либо строго установленного порядка действий. Вы можете начать подготовку дизайн-документа сразу после постановки проблемы и целей (особенно если вы работаете в стартапе, где скорость вывода продукта на рынок зачастую важнее следования установленным процессам). Но поскольку мы хотели, чтобы после прочтения этой книги у вас сформировался некий чек-лист этапов, все действия в ней представлены в традиционной последовательности.

Как однажды сказал менеджер одного из авторов: ни один алгоритм рекомендаций не справится с покупателем, у которого есть список покупок. У таких людей всегда есть цель и план ее достижения, а поэтому их ничто не остановит.

Если задуматься, написание кода — это всего лишь предоставление конкретного набора инструкций для достижения определенной цели. В этом смысле дизайн-документ — это метаалгоритм, направленный на достижение конкретной цели со множеством подалгоритмов. Тем не менее дизайн-документ часто критикуется и считается пережитком бюрократического прошлого, используемым по привычке.

В этой главе мы рассмотрим наиболее распространенные мифы о дизайн-документе. Мы введем и определим понятие «антицели» — дополнительные ориентиры, направляющие вас к целям проекта, а также начнем практическую часть книги и представим два примера дизайн-документа, основанных на вполне реальных сценариях.

4.1. Распространенные мифы о дизайн-документе

Со временем дизайн-документ оброс множеством ложных представлений и неверных интерпретаций, которые могут помешать вам создать по-настоящему хорошо структурированный документ, соответствующий потребностям проекта. Далее мы рассмотрим наиболее распространенные заблуждения и объясним, почему не стоит придавать им большого значения.

4.1.1. Миф № 1. Дизайн-документы полезны в крупных компаниях, но не в стартапах

Здесь возможно весьма справедливое замечание: выделять время и силы на подготовку дизайн-документов могут только крупные компании. Конечно, зрелым организациям действительно приходится инвестировать больше времени и ресурсов в написание дизайн-документов по сравнению со стартапом с десятком сотрудников. Однако это не означает, что небольшим компаниям вовсе не стоит писать такие документы: как гласит известное высказывание, «план — ничто, планирование — все». Преимущество написания дизайн-документа заключается в том, что он позволяет выявить «слепые зоны» — как с точки зрения продукта, так и с технической стороны. Это поможет вам избежать множества проблем в среднесрочной перспективе, особенно если вы исключите все нерелевантные данные. Для последнего мы рекомендуем применять метод, который мы называем «антицели» и которому далее посвятим отдельный раздел.

Когда книга находилась в стадии раннего доступа, мы неоднократно слышали от наших первых читателей один и тот же комментарий: «Да, отлично, но

в стартапах все работает не так». И хотя мы согласны с тем, что в стартапах иная скорость вывода продукта на рынок, мы по-прежнему придерживаемся мнения, что этап проектирования необходим всем. Действительно, основатели и первые разработчики проекта могут с легкостью встретиться за чашкой кофе и обсудить все необходимые детали, тогда как крупной корпорации на аналогичный объем задач потребуется шесть месяцев. Мы также согласны с тем, что написание официальных документов может быть неэффективным, но мы и не призываем к этому. Достаточно простой заметки с кратким описанием, которая обеспечит единое понимание проекта всеми его участниками. Игнорирование успешных практик разработки ПО и ML-систем допустимо, если ваша цель — выиграть хакатон, но такой стиль практически неприменим в долгосрочной перспективе.

4.1.2. Миф № 2. Дизайн-документы нужны только в сложных проектах

В этом утверждении есть доля истины, если рассматривать дизайн-документ в его классическом виде: как масштабную, трудоемкую работу, охватывающую каждую деталь конечного продукта — от общего понимания до анализа рисков при развертывании. В таком случае составление дизайн-документа может занять больше времени, чем сам проект!

Как правило, данный аргумент исходит либо от людей с недостаточно гибким мышлением, либо от убежденных противников проектной документации, готовых использовать любой довод в свою пользу.

Практика же показывает, что даже в небольших проектах грамотно структурированный дизайн-документ позволяет на ранних этапах выявить потенциальные риски, служит справочным материалом при дальнейшем развитии проекта и, что особенно важно, позволяет избежать неконтролируемого расширения объема задач, когда каждый стейкхолдер стремится добавить «еще одну маленькую функцию».

Даже простым инициативам может быть полезен дизайн-документ, конечно, если уровень его детализации будет соответствовать масштабу задачи.

4.1.3. Миф № 3. Дизайн-документ пишется по шаблону

Многие компании, особенно крупные и хорошо структурированные, придерживаются рекомендованных шаблонов, и в масштабах их деятельности это вполне оправданно. Однако мы не советуем воспринимать шаблоны дизайн-документов как нечто обязательное. По нашему опыту, шаблон не должен ставиться превыше содержания. Дизайн-документы зачастую содержат слишком много целей одновременно, в результате чего становятся перегруженными и только отпугивают людей от их составления и изучения. Поэтому мы рекомендуем придерживаться

минималистичной базовой структуры, дополняя ее там, где это необходимо, в зависимости от специфики системы и самого проекта.

На первый взгляд процесс написания дизайн-документа может показаться простым и последовательным. На практике же с самого начала вы столкнетесь со множеством факторов, которые будут вам мешать и отбрасывать всю работу на несколько шагов назад, если своевременно эти факторы не учесть.

Помните: ваша задача — не создать драфт документа и убедить всех в его безупречности. Наоборот — вам нужно выявить как можно больше его слабых мест (в том числе привлекая к их поиску стейкхолдеров), чтобы в результате — после ряда итераций — у вас сформировался документ, позволяющий приступить к разработке ML-системы.

4.1.4. Миф № 4. Каждый дизайн-документ должен описывать развернутую систему

Если вы инженер и вам нужно создать некий механизм, вы начинаете с разработки чертежа. Другие инженеры затем его изучат и дадут обратную связь, которая, скорее всего, приведет к каким-то корректировкам — и так далее, пока ваш проект не будет готов к воплощению в жизнь.

Тот же принцип применим и к проектированию ML-систем. ML-система — это сложный механизм, состоящий из взаимосвязанных компонентов и требующий тщательной подготовки, в процессе которой вы составляете и утверждаете дизайн-документ. Тем не менее случается и так, что качественно написанный дизайн-документ приводит к отказу от реализации ML-проекта.

Возможно, звучит абсурдно, но представьте, что перед вами стоит выбор между двумя опциями:

- Потратить 6 месяцев на разработку моделей, признаков, функций потерь и датасетов, только чтобы в итоге проект оказался отложен в долгий ящик (где и заканчивают свой жизненный цикл большинство ML-проектов).
- Потратить 2–4 недели на попытку описать:
  - Зачем мы вообще занимаемся этим проектом?
  - Как именно мы собираемся его реализовывать?
  - Все ли у нас есть для этого?
  - Можно ли обойтись менее эффективным, но и менее трудозатратным решением?
  - Достижим ли желаемый результат?

Осознание того, что 90 % результата можно получить с помощью двух условных операторов if, может быть неприятным, но это все же гораздо предпочтительнее первого варианта.

4.2. Цели и антицели

Одна из задач дизайн-документа — снизить неопределенность в решаемой проблеме путем расстановки ключевых ориентиров. До составления документа уровень понимания как самой проблемы, так и ее возможного решения среди участников довольно низкий и несогласованный. Один из приемов, способных помочь в такой ситуации, — использование «антицелей»: обратных формулировок целей, которые позволяют сузить как пространство проблем, так и пространство решений.

Каждую часть дизайн-документа можно рассматривать как ответ на множество вопросов: каковы цели потенциальной системы, каковы ключевые критерии успеха, на какие технические аспекты следует обратить внимание, как решается конкретная подзадача и т. д. Типичная ошибка новичков — игнорировать возможные компромиссы и перечислять бесконечное количество целей для системы: например, она должна делать X, Y и Z, быть производительной, точной, легко сопровождаемой, дешевой в разработке и интуитивно понятной. Очевидно, невозможно уместить все положительные качества в одной системе, поэтому здесь нужен подход, позволяющий сбалансировать требования.

Определение антицелей позволяет вычеркнуть те аспекты системы, которые нас не особенно волнуют, и, напротив, акцентировать внимание на действительно важных. Допустим, мы разрабатываем систему для внутреннего использования, которая будет выводить отчеты для высшего руководства и аналитиков. Можно допустить, что время обработки данных некритично — важно лишь, чтобы отчеты были готовы, например, к утру. В таком случае «время обработки» — это первая антицель, и нет никакого смысла тратить усилия на оптимизацию этого параметра. Или, к примеру, вы создаете систему рекомендаций для бутика — вряд ли вам потребуется поддержка миллионов товаров, если ассортимент магазина насчитывает всего несколько сотен наименований (рис. 4.1). Это означает, что избыточная производительность здесь нецелесообразна и не является ключевой целью.

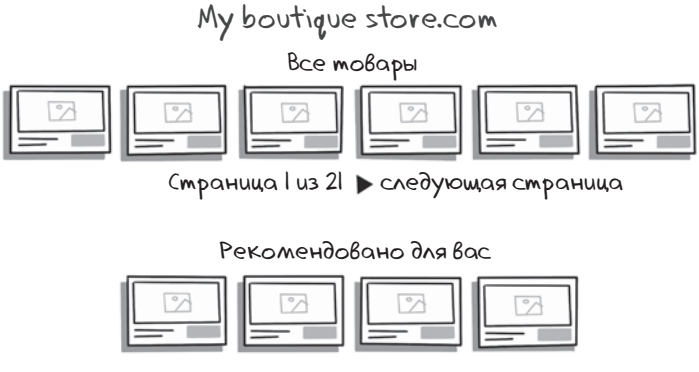


Рис. 4.1. Магазину с менее чем 1000 товаров и низким трафиком не стоит ориентироваться на масштабируемость при создании системы рекомендаций, так как практически любое техническое решение справится с подобной нагрузкой

Антицели помогают нам сосредоточиться только на важных аспектах и отбросить те, что не участвуют в достижении основной цели системы.

Следующий пример демонстрирует, как могут выглядеть списки целей и антицелей для системы рекомендаций бутика:

- Цели:
  - повышение числа переходов от просмотра товара к добавлению в корзину;
  - разнообразие рекомендаций для пользователей;
  - низкая задержка для пользователей.
- Антицели:
  - масштабируемость по числу обрабатываемых товаров;
  - масштабируемость по числу одновременных пользователей;
  - поддержка новых категорий товаров.

Аналогичный подход применим и к другим разделам дизайн-документа. Если вы сформировали некое представление об архитектуре, а затем поняли, что в ней есть критическая ошибка, стоит упомянуть об этом в документе, приведя «контрпример»: как не надо делать. Представьте, что при проектировании масштабируемой системы вы изначально планировали использовать облачную инфраструктуру, но затем узнали, что ваш крупнейший потенциальный клиент накладывает строгие ограничения на использование внешнего оборудования из соображений конфиденциальности. В этом случае в дизайн-документ достаточно добавить одну фразу, например: «Облачное решение X могло бы быть неплохим вариантом для хранения данных, но неприменимо из-за ограничений на использование облака у клиента Y». Такая формулировка задает определенные рамки и может натолкнуть остальных разработчиков на другие идеи реализации: «Если решение X технически нас устраивает, есть ли аналоги с открытым кодом, которые мы можем развернуть на собственных серверах?»

Антицели не должны рассматриваться как основной источник информации в дизайн-документе, но они могут восполнить некие пробелы в понимании проекта, постепенно становясь неотъемлемой частью структуры документа.

Сомнительные цели и их влияние на конечный результат

Поделимся с вами двумя историями, иллюстрирующими, как нечеткая постановка цели может повлиять на разработку ML-системы.

В 2016 году Валерий работал в отделе взыскания долгов одного крупного банка, руководство которого приняло решение внедрить в свою работу ML и перейти от жестких правил и интуитивных решений к алгоритмической поддержке. Одной из первых задач Валерия было создание модели, которая определяла бы, к какому клиенту обратиться в первую очередь, чтобы максимизировать результат, — то есть к тому клиенту, которого можно побудить к действию с помощью каких-либо стиму-

лов (обещания оплаты, отмены комиссии, скидки). Существующий процесс включал много ручной работы и давал коэффициент конверсии около 50 %. Новый же процесс, включавший достаточно простую нелинейную модель на наборе из примерно 100 признаков, за два месяца показал поразительные результаты — 80 % конверсии, тогда как старый процесс по-прежнему давал 50 %.

Команда была довольна и с радостью представила результаты старшему вице-президенту компании, которая по окончании презентации спросила: «А чем эти клиенты отличаются? Какова их мотивация?». Ответить на такой вопрос в 2016 году при наличии нелинейной модели со 100 признаками было непросто, не говоря уже о том, что то, *что делает человек*, и то, *почему он это делает,* — это две совершенно разные вещи. Например, изначальная цель старшего вице-президента — понять «*почему*», а цель бизнеса — понять «*кто*». Соответственно, команда должна была проектировать систему и модель иначе, чтобы иметь возможность ответить на оба вопроса, даже если это будет менее эффективно. Таким образом, плохо (или некорректно) сформулированная цель изначально отбросила всю работу команды на три месяца назад.

Второй пример касается алгоритма ценообразования, о котором мы говорили в главе 2. Изначально наша цель заключалась в максимизации валовой стоимости при сохранении маржи на заданном уровне.

В какой-то момент модель нашла гениальный способ достичь этой цели. В каталоге товаров был бумбокс, который модель начала продавать по цене ниже закупочной. В результате за 24 часа было продано больше колонок, чем за предыдущие 90 дней. Надо отдать ей должное: формально задача была выполнена — даже при отрицательной марже все укладывалось в допустимые рамки.

Однако это было совершенно не то, чего мы хотели. Правильно сформулированная цель могла бы звучать так: «Увеличение выручки при сохранении маржи, задействуя X % категорий и Y % артикулов в них, с каннибализацией не выше Z». Да, выручка выросла, маржа осталась в рамках, но в итоге все бросились покупать только одну модель товара. Остальные колонки не покупались вовсе.

Нам повезло, что это был тестовый запуск на ограниченном количестве товаров с динамическим ценообразованием. Это позволило увидеть, что изначальная цель была плохо сформулирована и необходимо выработать более продуманный подход. К счастью, архитектура была модульной и легко поддавалась корректировке.

4.3. Структура дизайн-документа

В этом разделе мы могли бы сосредоточиться на теоретических сведениях о содержании и структуре классического дизайн-документа, но на практике дизайн-документ, разрабатываемый для ML-системы, вряд ли будет опираться на подходы, применяемые в традиционной разработке ПО. Более того, структура такого документа может варьироваться от компании к компании, поэтому мы не видим смысла подробно останавливаться на вопросах оформления. Вместо этого мы рекомендуем уделить больше внимания тому, какие разделы необходимо включить в него. Кроме того, наша цель — доказать вам, что дизайн-документ — это неотъемлемый элемент проектирования ML-системы. Именно по этой причине,

начиная с данного раздела и далее на протяжении всей книги, в конце каждой главы будет представлен большой практический блок с фрагментом дизайн-документа, отражающим основную идею главы. Это важный компонент книги, который будет идти рука об руку с теорией и байками, демонстрируя применение практических решений к реальным проблемам.

Далее мы представим вашему вниманию два вымышленных кейса, каждый из которых имеет собственную специфику, проблемы и контекст. Эти два кейса лягут в основу двух разных дизайн-документов, которые мы будем постепенно развивать от главы к главе, придавая им все большую глубину и сложность. В итоге у нас на руках окажутся два полноценных документа.

В этом разделе мы начинаем составлять дизайн-документ одного из проектов — так, как это могло бы быть в реальной жизни. Для этого мы придумали вымышленную компанию Supermegaretail — розничную сеть, запускающую проект по прогнозированию спроса.

Раздел 4.4 содержит краткий пример того, как может выглядеть первая глава дизайн-документа. Мы включим в него только основные темы — иначе все это не уместилось бы в одной книге.

ПРИМЕЧАНИЕ Текст *курсивом* в дизайн-документе — это наши поясняющие комментарии, они не является частью самого документа.

ДИЗАЙН-ДОКУМЕНТ: SUPERMEGARETAIL

ОПРЕДЕЛЕНИЕ ПРОБЛЕМЫ

I. ИСТОКИ

Supermegaretail — это розничная сеть, насчитывающая тысячи магазинов в разных странах и регионах. Приобрести там можно самые разные товары: продукты питания, товары первой необходимости, средства личной гигиены, спортивные пищевые добавки и многое другое.

Для реализации этих товаров Supermegaretail должна заранее закупить или произвести продукцию и доставить ее в конкретные магазины. Количество закупаемых товаров — ключевая величина, которую необходимо определить, и здесь возможны различные сценарии.

Для упрощения расчетов предположим, что Supermegaretail закупила 1000 единиц товара A для конкретного магазина.

1. Supermegaretail закупила 1000 единиц и продала 999 до следующей поставки. Это оптимальная ситуация. Остаток в 0.1 % указывает на близость к максимальной выручке и марже.
2. Supermegaretail закупила 1000 единиц и продала 100 до следующей поставки. По очевидным причинам это крайне нежелательная ситуация.

Supermegaretail стремится продать как можно больше закупленных единиц без дефицита товара. Чем больше разрыв между закупленным и проданным, тем выше убытки сети.

3. Supermegaretail закупила 1000 единиц и продала все 1000. Это тоже не лучший исход, так как мы не знаем, сколько бы единиц купили клиенты, если бы имелась такая возможность, — 1001, 2000 или даже 10 000. Дефицит товаров искажает наше понимание реального спроса. Более того, это перенаправляет покупателей к конкурентам Supermegaretail, у которых необходимый товар есть в наличии.

Дополнительным ограничением является наличие большого количества скоропортящихся продуктов, которые не могут долго храниться на полках магазинов, — они либо продаются, либо утилизируются.

Цель проекта — максимально сократить разрыв между поставленным и проданным товаром, при этом не допуская его дефицита, с соблюдением определенного уровня сервиса, который будет описан далее. Для этого мы планируем прогнозировать спрос на конкретный товар в конкретном магазине в определенный период времени с использованием ML-системы.

II. АКТУАЛЬНОСТЬ И ПРИЧИНЫ

В этом разделе мы раскрываем актуальность рассматриваемой проблемы, подтверждая ее результатами предварительного анализа данных.

A. Анализ текущего процесса

Как на данный момент организованы процессы заказа, доставки и продажи товаров в Supermegaretail?

Для Supermegaretail возможное описание выглядит следующим образом:

1. Горизонт планирования для заключения сделки с производителями товаров:
   - Годовой контракт с возможностью продления на 90 дней в течение первых 9 месяцев.
2. Дополнительная скидка при увеличении объема заказа:
   - Предоставляется скидка 2 % за каждые дополнительные $20 млн.
3. Количество распределительных центров, выполняющих роль логистических узлов между производителями и магазинами:
   - По всей стране расположено 47 распределительных центров — точек присутствия и единиц агрегации для прогнозирования.
4. Периодичность поставок между распределительными центрами и магазинами:
   - Обычно грузовик доставляет товар из распределительного центра в магазин каждые 2 дня.

5. Наличие или отсутствие складов при магазинах:
   - В большинстве магазинов склады отсутствуют. Тем не менее зона разгрузки эффективно используется для хранения выгруженных товаров в течение 2–3 дней.
6. Кто и на каком этапе принимает решение, что и куда поставлять:
   - План поставок составляется в распределительном центре. Менеджер магазина может его скорректировать.
7. Горизонт прогнозирования:
   - Основной горизонт прогнозирования — неделя и месяц. Однако при работе с производителями товаров требуется прогноз на 1 год.
8. Владельцы бизнес-процесса:
   - отдел логистики;
   - отдел закупок;
   - оперативный отдел (менеджеры магазинов).

B. Каков объем потерь Supermegaretail из-за расхождений между прогнозируемым и фактическим спросом?

Рассчитать убытки от лишних запасов и просроченных товаров гораздо легче, чем оценить потери из-за их дефицита (отсутствия в наличии), что можно сделать либо с помощью серии A/B-тестов, либо на основе экспертного мнения, — причем второй вариант обычно намного быстрее и дешевле.

Общая сумма потерь может быть примерно рассчитана как сумма двух указанных компонентов, в результате чего мы получим размер потенциальной выгоды от идеального (но недостижимого) решения.

Первоначальный расчет показал, что потери за последний год составили около $800 млн.

Начиная со следующего раздела дизайн-документа (в рамках данной главы), мы приводим лишь примерный список вопросов, чтобы документ не получился слишком объемным. Ответы на эти вопросы помогут принять решение о дальнейших действиях, которые мы рассмотрим в последующих главах по мере прохождения каждого этапа создания системы.

C. Прочие причины

- Могут ли другие команды использовать наше решение, что сделает его разработку более полезной и значимой?
- Вполне вероятно, в дальнейшем мы сможем продавать свою систему прогнозирования спроса другим розничным компаниям (разумеется, не прямым конкурентам).

III. ПРЕДЫДУЩИЕ РАБОТЫ

В этом разделе указывается, является ли данная проблема совершенно новой или по ней уже предпринимались какие-то действия. Обычно это перечень

вопросов, на которые необходимо ответить, чтобы избежать лишней работы или повторения прошлых ошибок.

- Что, если в Supermegaretail знают об этой проблеме и уже внедрили какой-то подход к прогнозированию спроса? У компании есть магазины в разных регионах, поэтому, вероятно, ее система уже достаточно эффективна. Как компания осуществляет прогнозирование?
  - «Скользящее окно»?
  - Экспертная комиссия?
  - Эмпирическое правило + ускоренная доставка?
  - Имеются ли какие-либо ограничения, которые необходимо учитывать, например минимальный или максимальный объем заказа?
- Можно ли быстро усовершенствовать уже существующее решение или требуется полностью новое?
- Что, если текущая система прогнозирования Supermegaretail хорошо работает для одних товарных категорий, но совершенно не подходит для других? Другими словами, можем ли мы применить гибридный подход — хотя бы на начальном этапе — и начать с наименее успешных категорий, в которых наблюдается наибольший разрыв между прогнозом и фактическими продажами?
- Если наше решение случайно нарушит текущие процессы, это некритично. Мы тестируем его только на проблемных категориях и не затрагиваем те, где все работает хорошо.
- Иными словами, нам необходимо провести обширный и свежий предварительный анализ текущего решения.

IV. ПРОЧИЕ ВОПРОСЫ И РИСКИ

- Имеется ли у нас необходимая инфраструктура или ее нужно создавать с нуля?
- Если мы выберем слишком сложное решение, оно может повести себя непредсказуемо. Какие механизмы контроля и защиты необходимо внедрить, чтобы избежать проблем? Есть ли у нас резервный план на случай, если что-то сломается?
- Насколько мы уверены в том, что сможем значительно повысить качество и сократить объем ручной работы? Действительно ли мы способны выполнить эту задачу?
- Какова цена ошибки? Для ситуаций дефицита товара и его избытка цена ошибки, скорее всего, будет разной.
- Сможем ли мы справиться с возросшей нагрузкой в ситуации нехватки какого-либо товара?
- Как часто должно проводиться прогнозирование и насколько точными должны быть его результаты?

Как можно заметить, даже краткий обзор проблемы и анализ существующих данных может легко перерасти в документ объемом 10 страниц. Такой драфт позволит вам решить, стоит ли продолжать работу или, возможно, лучше остановиться сейчас и не реализовывать сложное решение на основе ML.

Следующий раздел главы не менее важен — он содержит практический пример того, как проводить ревью дизайн-документа. Если вы новичок в проектировании ML-систем, то скорее всего, вы еще не достигли того этапа карьеры, когда станете обладать достаточным опытом и авторитетом, чтобы участвовать в подобных рабочих процессах. Тем не менее однажды вы окажетесь в роли ревьюера дизайн-документа — это лишь вопрос времени, и лучше к этому подготовиться. Далее вы найдете несколько практических рекомендаций, как правильно проводить ревью.

4.4. Ревью дизайн-документа

Audi alteram partem —
Выслушай и другую сторону.

Латинская пословица

Мы ни разу не видели такой драфт дизайн-документа, который можно было бы сразу взять и реализовать. Но довольно неплохие нам все же встречались — и для первой итерации этого вполне достаточно.

Обосновать подобное умозаключение довольно легко. Разработка сложных систем требует участия большого числа специалистов с разным опытом и навыками. Как автор дизайн-документа вы должны сделать его удобным и понятным для всех вовлеченных сторон. Если ваш документ будет состоять из глав и подразделов, то любой эксперт сразу сможет понять, с чего ему начать. В противном случае вы получите типичную реакцию большинства людей при виде документа объемом более 10 страниц — они просто закроют его и забудут.

Переходим к первым двум ключевым моментам — дизайн-документ должен быть доступен и виден как можно большему числу участников проекта, а также удобен для навигации.

Как только люди получают какой-либо материал, они неизбежно начинают его критиковать и предлагать свои улучшения. Как автор вы должны поощрять такое поведение. В конце концов, каковы ваши шансы с первой попытки прийти к лучшему и наиболее подходящему решению?

Всегда узнавайте причины для каждого предложения или изменения, так как ревьюеры могли высказать их из самых разных соображений:

- Ревьюеры ранее уже использовали этот инструмент и поэтому считают его универсальным для любых задач.

- В существующей инфраструктуре имеется ряд ограничений. Например, мы не можем обеспечить поддержку в реальном времени, но можем выполнять пакетные задания каждые 60 секунд. Повлияет ли это на процесс?
- В нашей команде есть специалисты, способные поддерживать технологию A, но нет никого, кто умеет работать с технологией B. Следовательно, разумнее перейти от упомянутой в проекте технологии B к технологии A.
- Ревьюеры хотят продемонстрировать свои знания какой-либо технологии и познакомить с ней остальных.
- Ревьюеры видят иной способ решения поставленной задачи и предлагают альтернативу.

Старайтесь понять мотивацию каждого замечания и запрашивайте дополнительную информацию до тех пор, пока полностью не разберетесь в причинах критики. Поверьте, комментарий вроде «выглядит неплохо» — всегда самый бесполезный. Лучше найдите часть, которая вызывает у вас больше всего сомнений, и узнайте мнение ревьюера о ней, выразив свои опасения. Также полезно составить список вопросов и затруднений, чтобы сфокусировать на них внимание ревьюеров и упростить предоставление обратной связи.

Распространенная ошибка при написании дизайн-документов — чрезмерная обобщенность. Это серьезный недостаток, часто возникающий из-за того, что один человек просто не обладает достаточным контекстом, чтобы заполнить все пробелы. Задача автора документа — упростить процесс внесения правок другими: например, выделить проблемные участки, где не хватает информации, и призвать ревьюеров восполнить недостающие фрагменты.

Мы уже поговорили о том, как создать дизайн-документ и чего ожидать от ревьюеров. Однако поскольку название раздела — «Ревью дизайн-документа», давайте попробуем перевернуть наши рекомендации и рассмотреть их с точки зрения ревьюера:

- Ознакомьтесь с документом и пройдитесь по его структуре. В каких разделах вы лучше разбираетесь?
- Если структуры нет, посмотрите, есть ли в конце документа какие-либо вопросы или пункты, требующие внимания.
- Если их нет — попросите автора документа их добавить.
- При добавлении комментария постарайтесь для себя ответить на вопрос: каков мой вклад и чего хочу этим достичь?
- Подумайте дважды, прежде чем написать что-то вроде «выглядит неплохо». Вы действительно так считаете или просто хотите сэкономить время и довериться мнению других? Если второе — возможно, лучше вообще не комментировать.

4.4.1. Пример ревью дизайн-документа

В качестве второго примера дизайн-документа мы выбрали кейс компании стоковых фотографий, о которой говорилось в главе 3, — назовем ее PhotoStock Inc. Нам было поручено создать современный инструмент поиска, способный находить наиболее релевантные изображения по текстовым запросам пользователей, обеспечивая высокую производительность и отображая наиболее подходящие изображения из имеющегося ассортимента.

Бизнес построен по типу маркетплейса: фотографы регистрируются на платформе и загружают свои снимки, а клиенты, нуждающиеся в конкретных изображениях для иллюстративных целей (редакторы, дизайнеры, рекламные специалисты), приобретают права на использование этих фотографий. Платформа зарабатывает на комиссии с продаж, а поэтому крайне заинтересована в разработке эффективной поисковой системы на сайте.

Рассмотрим фрагмент «сырого» и недоработанного дизайн-документа, составленного на основе материала из предыдущих глав, и прокомментируем его так, как будто мы проводим ревью. В этот раз текст, выделенный *курсивом*, представляет собой комментарии ревьюеров.

ДИЗАЙН-ДОКУМЕНТ: PHOTOSTOCK INC.

I. ПРОБЛЕМА

Девяносто процентов пользователей PhotoStock Inc. ищут изображения через поисковую строку на сайте. Это делает поиск ключевым компонентом пользовательского опыта.

В настоящее время поисковая система основана на нечетком алгоритме поиска, реализованном с помощью Elasticsearch, индекс которого автоматически обновляется каждую неделю в ночь с понедельника на вторник. Мы полагаем, что алгоритм плохо обрабатывает синонимы. Кроме того, пользователи могут применять дополнительные фильтры из заранее заданных пресетов, которые вручную формируются продуктовой командой.

Многие пользователи недовольны качеством поиска, что подтверждается результатами опроса клиентов и анализа кликстрима. Лишь малая часть поисковых сессий заканчивается покупкой.

Ревьюер: Сколько пользователей конкретно? Пожалуйста, добавьте ссылки на отчеты и дашборды для полноты контекста.

Ревьюер: Конверсия из поиска в покупку зависит от множества переменных, и релевантность поисковой выдачи — лишь один из факторов. Предлагаю декомпозировать проблему, чтобы более точно оценить прибыль, упущенную из-за некачественного поиска.

Ревьюер: Пожалуйста, предоставьте больше информации о текущем поисковом решении — неясно, как оно работает и как взаимодействует с другими системами. Какие основные типы отказов?

Ревьюер: Как мы оцениваем удовлетворенность пользователей? Пожалуйста, приведите конкретные критерии.

II. ЦЕЛИ

Увеличить конверсию из поиска в покупку на 100 %.

Ревьюер: Почему именно на 100 %? Есть ли причины выбора такого показателя?

Ревьюер: Как уже упоминалось ранее, воронка «поиск — покупка» определяется не только качеством поиска. Нужно сузить цели.

Ревьюер: Есть ли какие-то нетехнические требования, например, по задержке отклика?

Ревьюер: Как мы на данный момент измеряем конверсию? Как мы впоследствии докажем, что ее рост был достигнут в результате именно этой инициативы?

Ревьюер: Каковы антицели, чтобы мы не тратили время на незначимые моменты?

III. РИСКИ

Мы можем потерять множество постоянных клиентов, если они не смогут продолжать пользоваться привычными сценариями.

Если будет выпущено нестабильное ПО, мы можем потерять значительный источник дохода.

Ревьюер: Интересное замечание о сценариях. Есть ли примеры того, как пользователи уже адаптировались к недостаткам текущей поисковой системы?

Ревьюер: С сине-зеленым развертыванием и A/B-тестами мы можем внедрять новую систему постепенно, это позволит снизить риски.

IV. ИСТОЧНИКИ

- [Ссылка на YourPowerfulSearch — корпоративную поисковую систему для маркетплейсов]
- [Ссылка на научную статью команды Bing по релевантности поиска]
- [Ссылка на дашборд Google Analytics с различными метриками, связанными с поиском на PhotoStock]

Ревьюер: Пожалуйста, добавьте больше внутренних материалов, связанных с поиском, например дашборд BI PhotoStock и исследования UX.

Ревьюер: YourPowerfulSearch — не единственное подходящее решение на рынке. Какие есть альтернативы? То же касается и статьи от Bing.

Ревьюер: Как мы будем оценивать влияние релевантности поиска на коммерческие метрики? Это существенно повлияет на возможный бюджет.

Можно заметить некоторые закономерности в комментариях, например:

- сразу же задаются все важные вопросы;
- указываются недостающие элементы — в утвердительной или вопросительной форме.

Ранняя обратная связь еще на этапе рассмотрения проекта может в дальнейшем сэкономить вам много времени. Но помните, что вопросы должны инициировать и поддерживать здоровую дискуссию, способствуя поиску лучших решений, а не быть агрессивными или токсичными.

4.5. Дизайн-документ — это живой организм

Изначально этот раздел планировался как миф № 5, но мы посчитали, что данный пункт достаточно важен, чтобы рассмотреть его отдельно.

Почему же можно без страха и сомнений редактировать дизайн-документ на любом этапе проекта? Ответ прост: дизайн-документ — это настоящий живой организм.

Обычно эволюция дизайн-документов выглядит следующим образом:

1. Первая итерация
2. Обратная связь от коллег
3. Переработка 60 % документа
4. Обратная связь от коллег
5. Переработка 30 % документа
6. Обратная связь от коллег
7. Переработка 10 % документа
8. Начало внедрения системы
9. (Через 3 месяца) Обратная связь после реального использования
10. Переработка 30 % документа

При таком развитии событий становится понятно, что закончить дизайн-документ можно разве что только после завершения внедрения всей системы — но и то не факт.

Как только система будет внедрена, практика выявит ее слабые места, которые придется устранять; или продакт-менеджеры решат, что нужны новые функции, и систему придется расширять; или государство примет новый закон, который необходимо будет учитывать; или произойдет миграция инфраструктуры, появится новый сценарий использования — продолжать можно бесконечно. Чтобы внести все эти изменения, разработчикам нужно понимать, как устроена система,

для чего они и знакомятся с дизайн-документом. Кроме того, к тому моменту может появиться новый паттерн или технология, идеально подходящие для данной системы.

И даже если этого не произойдет, новые функции и рефакторинг все равно должны быть отражены в дизайн-документе — именно в этом и заключается идея его эволюции.

Поэтому работа над дизайн-документом никогда не останавливается. Документ «живет» до тех пор, пока существует описываемый им сервис. Даже если вы покинете компанию, другим придется подхватить эстафету, если они не хотят столкнуться с полностью неработоспособной системой.

Необходимость переписать значительную часть дизайн-документа может расстроить, но в долгосрочной перспективе это обязательно принесет пользу. Работая со сложными системами, важно помнить о правиле «проектируй дважды» — это означает признать, что первый вариант проекта, скорее всего, неидеален, и спроектировать систему еще раз, но уже с использованием принципиально отличного подхода. Как показывает практика, это позволяет выявить как скрытые проблемы, так и новые возможности. Здесь хочется привести цитату из книги «A Philosophy of Software Design» Джона Аустерхаута (John Outerhout) (отличная книга, которую мы рекомендуем прочитать, чтобы прочувствовать всю суть качественного проектирования):

> *Я заметил, что принцип «проектируй дважды» бывает сложно принять по-настоящему умным людям. По мере взросления они замечают, что их самая первая идея по решению любой проблемы оказывается наилучшей, а потому нет нужды рассматривать второй или третий вариант. Это приводит к формированию вредных рабочих привычек. Со временем люди получают повышение и сталкиваются со все более сложными задачами, и однажды каждый из них достигает той точки, когда первой поверхностной идеи оказывается недостаточно, ведь если вы хотите достичь действительно хороших результатов, нужно рассматривать и второй вариант, а возможно, и третий, — вне зависимости от ваших умственных способностей. Проектирование крупных программных систем относится именно к той категории задач, где невозможно с первой попытки все сделать правильно.*

Хорошая архитектура (и, соответственно, хорошо составленный дизайн-документ) должна понижать сложность системы, будь то сложность в понимании, создании, модификации или сопровождении. Если с самого начала ясно, что система будет сложной, то дополнительное время, потраченное на ее упрощение с помощью нескольких итераций проектирования, — полезная инвестиция.

Это всегда раздражает тех программистов, кто любит больше писать код, чем размышлять о нем: «Да ладно вам, мало того что вы говорите, что сначала надо писать документацию, а не код, а теперь вы еще хотите, чтобы мы делали это по

нескольку раз?» Что ж, нет смысл выполнять множество итераций, если вы больше не получаете никакой новой информации, а иногда вы не можете улучшить дизайн-документ без написания доказательства концепции (proof of concept). Однако принцип «проектируй дважды» оказывается разумным компромиссом между гибкостью и надежностью.

Итоги

- В то время как правильно сформулированные цели являются для системы ориентиром, антицели указывают на области, которые следует избегать. Четко обозначьте их и не забывайте.
- Хорошо структурированный дизайн-документ поможет понять, нужна ли вам вообще ML-система.
- В ревью дизайн-документа должны принимать участие все стейкхолдеры.
- Получив комментарий «выглядит неплохо», обязательно свяжитесь с коллегой повторно и попросите предоставить более конкретную обратную связь. Неопределенность на начальной стадии работы почти наверняка приведет к необходимости вносить какие-то изменения позже.
- Если вы запрашиваете ревью вашего дизайн-документа, поощряйте участников критиковать текущие идеи и предлагать альтернативы.
- Пригласите ревьюеров с разным опытом и профессиональными навыками, чтобы получить разностороннюю обратную связь.
- В ревью дизайн-документа старайтесь задавать уточняющие вопросы, чтобы выявить его слабые стороны или ненужные элементы.
- Не бойтесь многократных итераций, поскольку ни один черновик не может обойтись без правок.
- Помните: дизайн-документ — это живой организм, который будет продолжать развиваться даже после запуска системы.

Часть 2

Первые шаги

В этой части мы более детально рассмотрим технические аспекты работы над проектом на его ранних этапах. Глава 5 посвящена выбору подходящих метрик и функций потерь для вашей ML-системы, определению и использованию прокси-метрик, а также применению их иерархии. В главе 6 мы поговорим о работе с датасетами — от выбора оптимальных источников данных и обработки исходных данных до определения характеристик устойчивого пайплайна и принятия решения о необходимом объеме данных для достижения моделью наилучших результатов. Глава 7 рассматривает как стандартные, так и нетривиальные схемы валидации, описывает процедуру раздельного обновления и дает обзор схем валидации как части дизайн-документа. В главе 8 вы узнаете больше о различных типах базовых моделей, начиная с константных — самых ранних, простых, но при этом весьма эффективных версий моделей — и заканчивая базовыми моделями, базовыми признаками, а также решениями на основе глубокого обучения.

5 Функции потерь и метрики

В этой главе

- ✓ Выбор подходящих метрик и функций потерь для ML-системы
- ✓ Определение и использование прокси-метрик
- ✓ Применение иерархии метрик

Предыдущая глава была посвящена созданию дизайн-документа для системы машинного обучения. Мы выяснили, что данный документ подвержен постоянным изменениям, а потому все вносимые в него правки не только неизбежны, но и необходимы.

К сожалению, ML-система не способна решить проблему напрямую, она может лишь попытаться приблизиться к ее решению, оптимизируя определенную задачу. И для этого ее необходимо настраивать, направлять и контролировать.

Управлять ML-системой можно с помощью функции потерь используемого в ней алгоритма, которая «поощряет» либо «наказывает» за уменьшение или увеличение определенных ошибок. Функция потерь применяется для обучения модели и, как правило, должна быть дифференцируемой; следовательно, выбор доступных функций потерь ограничен. Поэтому для оценки качества модели используются метрики. И хотя любая функция потерь может быть взята в качестве метрики (например, корень средней квадратичной ошибки, или RMSE, часто применяется как метрика, хотя это не всегда лучшее решение), далеко не каждая метрика может быть использована в качестве функции потерь.

В этой главе мы обсудим, как выбрать наиболее подходящие метрики и функции потерь, а также научимся грамотно проводить исследование и обосновывать свой выбор на этапе проектирования.

5.1. Функции потерь

Функция потерь, также известная как *целевая функция* или *функция стоимости*, определяет, как модель воспринимает окружающий мир и взаимосвязи между зависимыми и независимыми переменными: на что она обращает больше внимания, чего избегает и что считает допустимым. Таким образом, выбор функции потерь может существенно повлиять на общую производительность модели, даже если все остальное — признаки, целевая переменная, архитектура модели, размер датасета — остается неизменным. Смена функции потерь может полностью изменить поведение всей системы.

Выбор подходящей функции потерь (то есть способа, с помощью которого модель будет учиться на своих ошибках) — одно из важнейших решений при проектировании ML-системы. Вспоминая известный анекдот, можно с уверенностью сказать, что оптимизация среднего значения при расчете средней зарплаты посетителей бара имеет смысл до тех пор, пока в него не заходит Билл Гейтс (https://mng.bz/M1w8).

К сожалению, не каждая функция может использоваться в качестве функции потерь. Как правило, функция потерь должна обладать двумя свойствами:

- глобальная непрерывность (изменения в прогнозах должны приводить к изменениям в значениях потерь);
- дифференцируемость (возможность вычисления градиента для использования в алгоритмах оптимизации на основе градиентного спуска). Здесь существует одно исключение: в особых случаях возможны методы оптимизации без использования градиента, но на практике их стараются избегать, так как градиентные методы обычно сходятся гораздо лучше.

И хотя оба этих свойства важны для любой функции потерь, необходимо выбрать ту, что наилучшим образом соответствует вашей конкретной задаче и максимально приближает модель к конечной цели системы.

Здесь на помощь приходят расширенные функции потерь, предлагающие дополнительные способы улучшения модели. В отличие от манипуляций с признаками или самой моделью, они, как правило, не влияют на время выполнения. Это означает, что все изменения касаются только пайплайна обучения. А возможность изолировать изменения в небольшой части системы всегда будет плюсом. Тем не менее на практике мы часто наблюдаем, как ML-инженеры (особенно новички) выбирают конкретную функцию потерь только потому, что привыкли применять ее в похожих задачах. Наиболее распространенный пример — задачи регрессии,

в которых функции потерь среднеквадратичной ошибки (mean squared error, MSE) или средней абсолютной ошибки (mean absolute error, MAE) не просто выбираются чаще всего, но для некоторых становятся *единственным возможным вариантом*.

Несмотря на то что выбор подходящей функции (или функций) потерь может значительно повысить производительность модели, это все же не гарантирует успеха. Нам приходилось работать с ML-инженерами (с солидным академическим бэкграундом и научной степенью), которые пытались решить все возникавшие у них проблемы с помощью всего лишь одной элегантной функции потерь. Конечно, такой подход подчеркивает важность выбора правильной функции, но он все же далек от идеала. Хороший архитектор ML-системы всегда использует множество инструментов, а не зацикливается на каком-то одном. Таким образом, в качестве вывода можно сформулировать следующий принцип: если ваша система создается в исследовательских целях, высока вероятность того, что придется потратить изрядное время на поиск или разработку нетривиальной функции потерь.

Пару лет назад Валерий вместе со стажером работали над созданием модели, прогнозирующей объем торгов криптовалютами. Как всегда, он сначала попросил стажера подготовить дизайн-документ, и, как оказалось, не зря — стажер не стал включать в свой документ раздел, посвященный функции потерь, а просто перечислил несколько метрик, с помощью которых он планировал оценивать производительность системы, без каких-либо обоснований.

Почему такой подход неприемлем? Чтобы проиллюстрировать это, давайте рассмотрим немного упрощенную ситуацию с двумя наиболее часто используемыми для задач регрессии функциями потерь — MSE и MAE.

Представим, что у нас есть вектор целевых значений Y = [100, 100, 100, 100, 100, 100, 100, 100, 100, 1000] и вектор независимых переменных X, одинаковый для всех семплов.

Если мы обучим модель с использованием MSE в качестве функции потерь, она выдаст следующий вектор прогнозов:

```
Y_hat= [190, 190, 190, 190, 190, 190, 190, 190, 190, 190]
```

Если мы обучим модель с использованием MAE в качестве функции потерь, она выдаст:

```
Y_hat = [100, 100, 100, 100, 100, 100, 100, 100, 100, 100]
```

> **ПРИМЕЧАНИЕ** Это гипотетическая ситуация, с помощью которой мы хотим проиллюстрировать и упрощенно изложить нашу идею. Мы могли бы сгенерировать синтетические данные — признаки, целевые значения и модели, — но для легкости восприятия будем использовать только указанные ранее числа.

Если рассчитать MSE и MAE для модели, обученной с функцией потерь RMSE (root mean square error — среднеквадратичная ошибка), мы получим следующие

значения: MSE = 72 900, MAE = 162, при этом среднее значение остатков будет равно 0, а медианное — –90 (рис. 5.1).

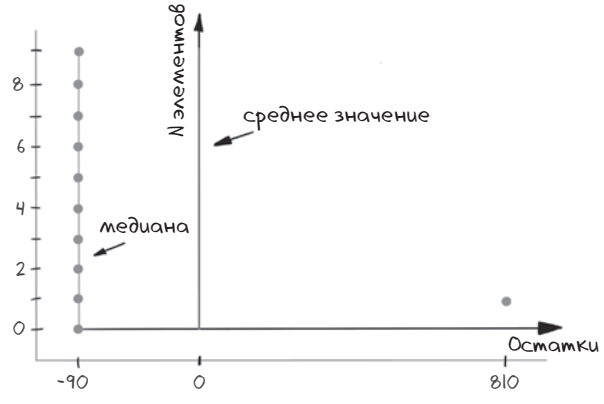


Рис. 5.1. Остатки после оптимизации среднего значения

Если рассчитать MSE и MAE для модели, обученной с функцией потерь MAE, результат будет таким: MSE = 81 000, MAE = 90, при этом среднее значение остатков составит 90, а медианное — 0 (рис. 5.2).

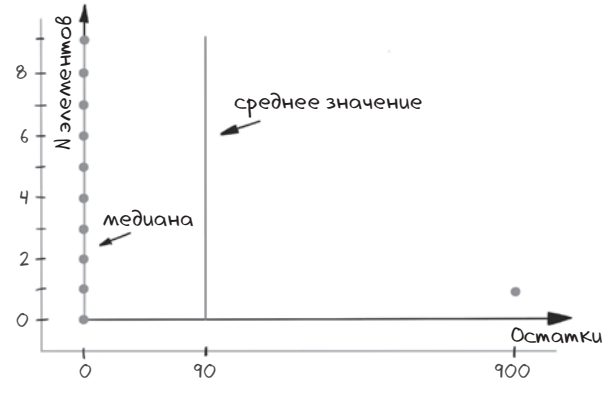


Рис. 5.2. Остатки после оптимизации медианы

Неудивительно, что модель, оптимизированная по MSE, демонстрирует лучшие результаты по MSE: поскольку MSE минимизирует среднее значение, соответствующие средние остатки оказываются лучше. С другой стороны, модель, оптимизированная по MAE, показывает лучшие значения MAE: так как MAE стремится минимизировать медиану, медианные остатки оказываются более точными. Что это в итоге означает? Какая функция потерь лучше? Все зависит от конкретного применения.

Предположим, мы оптимизируем навигационную систему для самолетов, и ошибка более 850 означает, что самолет вырулит за пределы взлетно-посадочной

полосы и потерпит крушение. В таком случае не стоит использовать MAE. Конечно, можно сказать, что в девяти из десяти случаев результат будет идеальным, и всего лишь в одном случае самолет не взлетит, но это недопустимо. Нам необходимо любой ценой избегать выбросов и штрафовать их, а значит, использовать MSE или даже более строгие функции потерь.

А теперь представим, что мы оптимизируем показатели ликвидности, необходимой для криптовалютной биржи на каждый торговый день. *Ликвидность* — это способность криптовалюты быть конвертированной в наличные средства или другие криптовалюты без потери стоимости, и она крайне важна для функционирования любой биржи. Высокая ликвидность означает активный и устойчивый рынок, позволяющий своим участникам быстро заключать сделки по адекватной цене. Однако чрезмерная ликвидность говорит о том, что выделенные ресурсы используются неэффективно. При таких условиях избыточного денежного резерва стоит избегать в девяти из десяти случаев. На данную ситуацию можно взглянуть и с другой стороны: модель, обученная на MSE, допустила избыток ресурсов в 810 единиц, а также их дефицит — тоже в 810 единиц, тогда как модель, оптимизированная на MAE, точно распределила ресурсы в девяти из десяти случаев и только в одном случае допустила дефицит ликвидности в 900 единиц. Видим, что система уже лучше оценивает наши потребности (при учете, что дефицит менее чем в 9 раз хуже, чем избыток).

Нетрудно заметить, что хотя мы использовали MSE и MAE при обучении моделей, при их оценке мы применили разные критерии. Для навигационной системы мы подсчитывали количество случаев, когда отклонение между фактическим и прогнозируемым значением превышало 850. Для задачи оптимизации ликвидности — число точных прогнозов или суммарное значение неточных оценок. Это иллюстрирует важную мысль: обучение модели для оптимизации конкретной функции потерь и оценка производительности модели — это две разные задачи. Об этом подробнее пойдет речь в разделе 5.2, посвященном метрикам. А пока давайте рассмотрим нюансы, связанные с выбором функций потерь для моделей глубокого обучения.

5.1.1. Выбор функции потерь для моделей глубокого обучения

В системах, основанных на глубоком обучении, особенно при обработке текста, изображений или аудиоданных, выбор подходящей функции потерь становится еще более важным.

Грамотно подобранная функция потерь может решить множество проблем, возникающих на этапе обучения модели, особенно в случае сложной архитектуры или специфичной предметной области. Например, функция потерь перекрестной энтропии — классическое решение для проблем классификации. Однако одна из связанных с ней проблем — это несбалансированность классов. Если какой-либо класс значительно доминирует, модель, оптимизированная

на перекрестную энтропию, может столкнуться с явлением, известным как *коллапс модели*, когда независимо от входных данных она всегда прогнозирует наиболее популярный класс. Существует множество решений этой проблемы (например, сокращение выборки (андерсемплинг), расширение выборки (оверсемплинг), индивидуальные веса классов и т. д.), но все они требуют ручной настройки и не всегда надежны. Данной проблемой заинтересовались исследователи, задумавшие создать функцию потерь, способную ее решить. Наиболее значимым результатом является публикация Лин (Lin) и соавторов — «Focal Loss for Dense Object Detection» (https://arxiv.org/abs/1708.02002). Их функция потерь сегодня по праву занимает достойное место среди прочих инструментов, помогающих работать с несбалансированными данными.

Focal Loss (рис. 5.3) — это функция потерь перекрестной энтропии с динамическим масштабированием, при котором масштабирующий коэффициент стремится к нулю по мере роста уверенности модели в правильном классе. Данный коэффициент позволяет автоматически снижать вклад простых примеров в функцию потерь и быстрее сосредоточивать внимание модели на сложных примерах (подробнее: https://paperswithcode.com/method/focal-loss).

Изначально данная функция была предложена для задачи обнаружения объектов в компьютерном зрении, но позднее подход распространился и на другие области, включая те, что не связаны с изображениями, например обработку аудио и естественного языка. Самое необычное известное нам применение Focal Loss описано в работе «Can Natural Language Processing Help Differentiate Inflammatory Intestinal Diseases in China?»[1] (https://mng.bz/aV9X), что лишь доказывает универсальность данной функции.

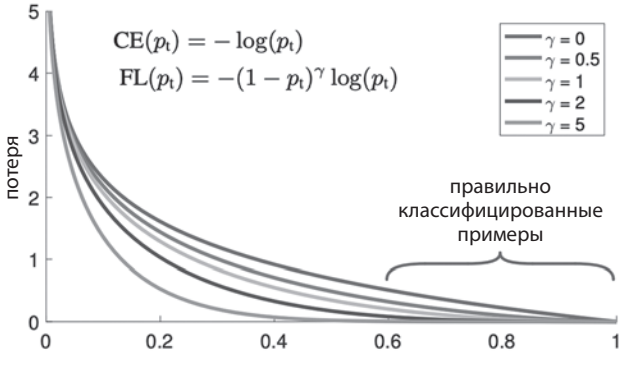


Рис. 5.3. Функция Focal Loss работает с неправильно классифицированными примерами, одновременно снижая относительное значение потерь для правильно классифицированных примеров (источник: Лин и др.)

[1] «Может ли обработка естественного языка помочь в дифференциальной диагностике воспалительных заболеваний кишечника в Китае?» — *Примеч. пер.*

В некоторых случаях удачным решением может стать объединение нескольких функций потерь в одной модели. Необходимость в таком подходе возникает при решении сложных задач, зачастую мультимодальных и нередко связанных с несколькими параллельными датасетами. Мы не будем вдаваться в подробности использования комбинированных функций потерь, поскольку это требует значительных академических познаний, но все же приведем несколько примеров связанных с данным вопросом публикаций:

1. «Authentic Volumetric Avatars from a Phone Scan»[1] (Cao и др.) (https://dl.acm.org/doi/abs/10.1145/3528223.3530143). Авторы объединили три группы функций потерь (сегментация, реконструкция и перцептивные потери). Генеративные модели компьютерного зрения зачастую нуждаются в использовании комбинированных функций потерь.
2. «Highly Accurate Protein Structure Prediction with AlphaFold»[2] (Jumper и др.) (https://www.nature.com/articles/s41586-021-03819-2). Известная модель AlphaFold 2 со впечатляющей точностью прогнозирует трехмерные структуры белков на основе их генетической последовательности. Это значительный прорыв в мире биотехнологий, и в этой модели используется несколько вспомогательных функций потерь. Например, маскированное языковое моделирование (masked language modeling) — подход, вероятно, вдохновленный функцией потерь, применяемой в архитектурах типа BERT, представляет собой популярное семейство моделей обработки естественного языка (NLP).
3. «GrokNet: Unified Computer Vision Model Trunk and Embeddings for Commerce»[3] (Bell и др.) (https://mng.bz/Xxr6). Это один из самых ярких примеров комбинированных функций потерь, которые мы только знаем. Авторы стремились создать единую модель для решения множества задач, поэтому использовали 7 товарных датасетов и 83 функции потерь (80 категориальных и 3 на основе эмбеддингов)!

В целом множественные функции потерь, как правило, применяются либо для ускорения сходимости модели, либо для решения нескольких задач настройки в рамках одной модели.

Функции потерь позволяют задать параметры точности и эффективности модели, а также минимизировать ошибки на этапе обучения. Метрики, в свою очередь, применяются для оценки производительности модели в рамках определенного набора параметров.

---

[1] «Достоверные объемные аватары на основе сканирования с телефона». — *Примеч. пер.*

[2] «Высокоточное предсказание структуры белка с помощью AlphaFold». — *Примеч. пер.*

[3] «GrokNet: унифицированная основа и векторные представления модели компьютерного зрения для коммерции». — *Примеч. пер.*

5.2. Метрики

Функция потерь, которую мы оптимизируем, и метрика, с помощью которой оцениваем производительность модели, могут существенно различаться. Напомним, что конечной целью системы прогнозирования спроса для компании Supermegaretail, описанной в главе 4, было сократить разрыв между количеством поставленных и проданных товаров, сделав его как можно меньшим и при этом избежав ситуации отсутствия товара на складе. Если визуально представить данную цепочку, она будет выглядеть, как показано на рис. 5.4.

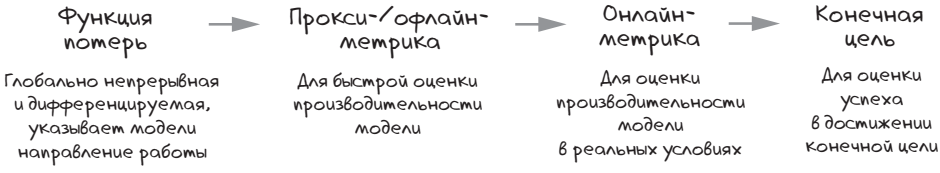


Рис. 5.4. Универсальный пайплайн для системы прогнозирования спроса, идеально подходящий Supermegaretail

Мы уже поняли, что выбор подходящей функции потерь крайне важен, но что насчет метрик? Разве нельзя просто выбрать стандартные метрики, протестировать несколько моделей, выбрать лучшую, развернуть ее и оценить успех с помощью A/B-тестирования?

К сожалению, нет. Выбор подходящего набора метрик должен быть столь же тщательно проработан, как и выбор функций потерь. Более того, если число наиболее распространенных функций потерь ограниченно, то метрики всегда можно адаптировать под конкретный бизнес. Неверно выбранная метрика, в свою очередь, может привести к ошибочной оптимизации, когда мы обучаем модель на оптимизацию нерелевантных значений, что в итоге приводит к низкой производительности. Чтобы исправить ошибку, потребуется откатиться на несколько шагов назад в разработке модели, что повлечет за собой существенные потери времени и ресурсов. Однако даже правильно выбранная метрика не гарантирует успеха проекта.

Байка от Валерия

Некоторое время назад я разрабатывал ML-систему для банка, который хотел как-то решить проблему неплатежеспособных заемщиков. Разрабатываемая нами система преследовала две основные цели:

- снизить количество просроченных платежей;
- повысить отклик клиентов.

В качестве метрики мы выбрали коэффициент конверсии клиентов из неплательщиков в плательщиков.

Первым делом мы внедрили систему обещанных платежей, которая работала следующим образом. Допустим, мистеру Смиту звонят из банка: «Мистер Смит, вы не оплатили кредит вовремя. Можем ли мы рассчитывать на то, что вы внесете нужную сумму в течение трех дней?» — «О, конечно, конечно, я заплачу», — отвечает мистер Смит. Сотрудники банка кладут трубку и ставят галочку в поле «обещал заплатить». Но затем мистер Смит нарушает свое обещание и ничего не платит.

На момент начала нашей работы коэффициент конверсии составлял 0.5, то есть подобная ситуация происходила в половине случаев. Это не так уж плохо, но явно далеко от идеала.

Учитывая отношение людей к подобным звонкам из банков и их желание как можно скорее положить трубку, пустые обещания — довольно распространенное явление. Но дело в том, что это палка о двух концах. С одной стороны, клиенту неприятно разговаривать с банком, особенно если инициатором звонка был не он. С другой стороны, банку тоже неинтересно вести бессмысленные разговоры, тратя ресурсы на содержание колл-центра.

В качестве решения мы построили систему, которая прогнозировала вероятность того, что клиент действительно заплатит. Мы заменили звонки сотрудника текстовыми сообщениями, что избавило от необходимости звонить клиентам и убеждать их оплатить кредит. Система также должна была прогнозировать поведение клиента.

На этапе валидации система показала коэффициент конверсии 0.9 — почти в два раза выше, чем при обзвонах! Однако спустя две недели работы конверсия резко упала до 0.35, а до отчета вице-президенту оставалась всего неделя.

Очевидно, что-то пошло не так, и нам нужно было выяснить, в чем дело. Мы изучили, как ранее работала эта метрика, и все оказалось довольно просто: если клиент обещал оплатить долг в определенный день месяца, но не делал этого в течение трех дней, он считался должником. Почему именно три дня? Потому что задержка между фактической операцией и поступлением информации о ней в базу данных банка составляла как раз три дня.

Допустим, вы должны внести следующий платеж по кредиту до конца дня 1 марта. Вечером вы идете в банк и оплачиваете нужную сумму. 2 марта система проверяет базу данных и видит, что платеж не был зафиксирован (что неудивительно — информация поступит только 4 марта). «Похоже, у нас просрочка», — решает система и отправляет вам SMS, потому что согласно собранным данным вероятность того, что вы заплатите после получения сообщения, составляет 90 %! Позже 2 марта вы получаете сообщение от банка с просьбой оплатить кредит. «Наверное, у них что-то сбилось. Скажу, что уже заплатил», — думаете вы и начинаете заполнять форму в ответном сообщении. Проблема в том, что форма не позволяет указать дату оплаты раньше текущей даты. Вы можете выбрать только 2 марта или позже. Но вы уже оплатили 1 марта. Что делать? Вы указываете, что оплатили 2 марта, и отправляете форму. Через три дня система проверяет список неплательщиков, открывает ваш профиль и видит, что вы обещали оплатить 2 марта, но не сделали этого в течение трех дней с момента обещания.

Когда мы перенастроили систему, коэффициент конверсии почти вернулся к изначальному значению, достигнув 0,8, но данная проблема показала, как достижение определенных метрик может оказаться затруднено из-за сбоев в общей логике поведения системы.

На первый взгляд выбор правильной метрики кажется довольно простым — выбираем ту, что ближе всего к конечной цели. Однако, как покажет следующая байка, сделать это очень даже непросто. Можно попытаться найти нужную метрику самостоятельно или же воспользоваться чьей-то помощью. Ниже приведены несколько вариантов, которые стоит рассмотреть:

- Если вам повезло и у вас есть иерархия метрик (мы обсудим это далее), используйте ее, чтобы прийти к нужной метрике.
- В некоторых компаниях есть специальный отдел, работающий с метриками, — если у вас такой есть, обязательно обратитесь туда.
- Если ничего из вышеуказанного не подходит, привлеките продакт-менеджеров и специалистов data science для разработки наиболее подходящей метрики.
- Если решаемая вами задача похожа на ту, с которой вы уже сталкивались и которую удалось решить, вполне логично перенести метрики из одного проекта в другой, внеся необходимые коррективы.
- Если у вас есть команда, занимающаяся A/B-тестированием, она, как правило, может помочь вам выбрать или создать метрику.

Если у вас нет возможности воспользоваться перечисленными выше ресурсами, можно сделать следующее:

- Обратитесь к разделу с целями в дизайн-документе и соотнесите с ним свои действия (важно освежить в памяти цели проекта, а не полагаться на то, как хорошо вы их запомнили). Понимание целей поможет определить, какие метрики приблизят вас к их достижению, или, по крайней мере, отсеять заведомо неподходящие.
- Попробуйте разложить финальную цель на составляющие, создав карту, аналогичную иерархии метрик (см. раздел 5.2.2). Скорее всего, для построения корректной карты целей потребуется несколько этапов, но такое упражнение поможет вам разбить большую цель на несколько более мелких компонентов, каждый из которых будет иметь свою метрику. Наличие множества мелких элементов упростит сборку единого целого.
- Подберите наилучшие метрики, описывающие успех на каждом этапе.
- Если что-то трудно измерить напрямую, замените это прокси-метриками (см. раздел 5.2.2). Прокси-метрики позволят вам собрать всю необходимую информацию до того, как система будет запущена в эксплуатацию.
- Используя карту, выберите метрику, которая либо отражает самый важный этап работы, либо наиболее полно обобщает всю карту.

В следующей байке мы рассмотрим типичную задачу бинарной классификации.

Байка от Валерия

Недавно я беседовал с одним своим другом по поводу оценки моделей для обнаружения мошенничества. Такие модели, как правило, решают задачу бинарной классификации, где 0 — это отсутствие мошенничества, а 1 — мошенническая операция.

Идеальной метрики тут нет — все зависит от конечной цели. Однако когда речь идет о моделях обнаружения мошенничества, мы обычно хотим поддерживать определенное соотношение между мошенническими и легитимными транзакциями. Если количество транзакций увеличится в 10 раз, допустимо, чтобы количество мошенничеств тоже выросло в 10 раз, но не в 20 и не в 30. Другими словами, нам нужна вероятностная модель.

Кроме того, мошенничество относится к задачам с несбалансированными классами, причем такой дисбаланс нестабилен во времени. Сегодня соотношение может быть 1 : 100 (всплеск мошеннической активности), завтра — 1 : 1000 (обычный день), а послезавтра — 1 : 10 000 (мошенники решили отдохнуть).

Наиболее распространенный набор метрик для таких моделей — это точность (precision) и полнота (recall), хоть это и не идеальное сочетание.

Проблема точности заключается в том, что при ее расчете учитываются оба класса:

$$\text{Precision} = \frac{\text{TP}}{(\text{TP}+\text{FP})}.$$

Представим, что у нас есть модель, которая с вероятностью 95 % правильно распознает мошенничество (число истинноположительных срабатываний — true positive, TP) и с вероятностью 5 % ошибочно помечает легитимные транзакции как мошеннические (число ложноположительных срабатываний — false positive, FP).

Рассмотрим три сценария, в которых P — количество положительных примеров, а N — количество отрицательных:

- $P = 10\,000, N = 10\,000, \ \text{Precision} = 0.95 \times \frac{10\,000}{0.95 \times 10\,000 + 0.05 \times 10\,000} = 0.95.$
- $P = 100\,000, N = 10\,000, \ \text{Precision} = 0.95 \times \frac{100\,000}{0.95 \times 100\,000 + 0.05 \times 10\,000} = 0.99947.$
- $P = 1000, N = 10\,000, \ \text{Precision} = 0.95 \times \frac{1000}{0.95 \times 1000 + 0.05 \times 10\,000} = 0.65.$

Как видим, баланс классов значительно влияет на метрику, даже если больше ничего не изменилось.

Теперь давайте посмотрим на метрику полноты (здесь TPR — true positive rate, доля истинноположительных срабатываний):

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN}) = \text{TP} / \text{P} = \text{TPR}$$

и применим ее к тем же трем сценариям:

- P = 10 000, N = 10 000, $\text{Recal} = 0.95 \times \frac{10\,000}{10\,000} = 0.95.$
- P = 100 000, N = 10 000, $\text{Recal} = 0.95 \times \frac{100\,000}{100\,000} = 0.95.$
- P = 1000, N = 10 000, $\text{Recal} = 0.95 \times \frac{1000}{1000} = 0.95.$

В данном случае баланс классов никак не повлиял на значение метрики.

Существует также такая метрика, как специфичность (specificity), которую можно использовать вместо точности (precision):

$$\text{Specificity} = \text{TN/N} = \text{TNR} = 1 - \text{FPR}$$

$$\text{FPR} = \frac{\text{FP}}{\text{N}} = \frac{\text{FP}}{\text{FP} + \text{TN}}.$$

(TNR — true negative rate — доля истинноотрицательных, FPR — false positive rate — доля ложноположительных срабатываний)

Рассмотрим те же три примера:

- P = 10 000, N = 10 000, $\text{Specificity} = 0.95 \times \frac{10\,000}{10\,000} = 0.95.$
- P = 100 000, N = 10 000, $\text{Specificity} = 0.95 \times \frac{10\,000}{10\,000} = 0.95.$
- P = 1000, N = 10 000, $\text{Specificity} = 0.95 \times \frac{10\,000}{10\,000} = 0.95.$

Значения полноты и специфичности не изменяются при дисбалансе классов, поскольку эти метрики нечувствительны к соотношению классов.

Изначально мой друг создал блокнот в Google Colab (https://mng.bz/5Ov8), чтобы доказать мне, что я не прав. Следующий код иллюстрирует его ход мыслей:

```
import numpy as np
def gen_labels_preds(fraud, genuine, fraud_predicted,
                     correct_fraud_predicted):
    labels = np.concatenate([np.repeat(True, fraud),
                             np.repeat(False, genuine)])
    preds = np.concatenate([
        np.repeat(True, correct_fraud_predicted),  # TP
        np.repeat(False, fraud - correct_fraud_predicted),  # FP
        np.repeat(True, fraud_predicted - correct_fraud_predicted),  # FN
        np.repeat(False, genuine - (fraud_predicted -
                  correct_fraud_predicted))  # TN
    ])
    return labels, preds
def calculate_metrics(labels, preds):
```

```
    TP = (preds & labels).sum()
    FP = (preds & ~labels).sum()
    TN = (~preds & ~labels).sum()
    FN = (~preds & labels).sum()
    recall = TP / (TP + FN)
    precision = TP / (TP + FP)
    FPR = FP / (FP + TN)
    return recall, precision, FPR
```

Он предложил два варианта модели с данными метриками:

- Модель A имеет 20 ложноположительных срабатываний и обнаруживает 80 % мошеннических транзакций.
- Модель B имеет 920 ложноположительных срабатываний и тоже обнаруживает 80 % мошеннических транзакций.

Затем он протестировал обе модели в трех сценариях с различным количеством транзакций и случаев мошенничества. В первом сценарии количество транзакций составляло 100 000. Всего было 100 случаев мошенничества, так что соотношение классов составило 1:1000:

```
fraud = 100  # сильный дисбаланс классов
genuine = 100000
model_A_FP = 20
model_B_FP = 920

# Модель A
a_total_fraud_predicted = model_A_FP + fraud * 0.8
a_correct_fraud_predicted = fraud * 0.8
a_labels, a_preds = gen_labels_preds(fraud, genuine,
a_total_fraud_predicted, a_correct_fraud_predicted)
a_recall, a_precision, a_FPR = calculate_metrics(a_labels, a_preds)

# Модель B
b_total_fraud_predicted = model_B_FP + fraud * 0.8  # помечает значительно
больше транзакций
b_correct_fraud_predicted = fraud * 0.8
b_labels, b_preds = gen_labels_preds(fraud, genuine,
b_total_fraud_predicted, b_correct_fraud_predicted)
b_recall, b_precision, b_FPR = calculate_metrics(b_labels, b_preds)

print("Показатели эффективности модели A:")
print('TPR:', a_recall)
print("Точность:", a_precision)
print("FPR:", a_FPR)

print("\nПоказатели эффективности модели B:")
print('TPR:', b_recall)
print("Точность:", b_precision)
print("FPR:", b_FPR)

Показатели эффективности модели A:
TPR: 0.8
Точность: 0.8
FPR: 0.0002
```

```
Показатели эффективности модели B:
TPR: 0.8
Точность: 0.08
FPR: 0.0092
```

Во втором сценарии он использовал те же метрики, что и в первом. Количество транзакций составило 100 000. Всего было 10 случаев мошенничества, так что соотношение классов составило 1 : 10 000:

```
fraud = 10  # сильный дисбаланс классов
genuine = 100000

# Модель A
a_total_fraud_predicted = model_A_FP + fraud * 0.8
a_correct_fraud_predicted = fraud * 0.8
a_labels, a_preds = gen_labels_preds(fraud, genuine,
                    a_total_fraud_predicted, a_correct_fraud_predicted)
a_recall, a_precision, a_FPR = calculate_metrics(a_labels, a_preds)

# Модель B
b_total_fraud_predicted = model_B_FP + fraud * 0.8  # помечает гораздо
                                                      больше транзакций
b_correct_fraud_predicted = fraud * 0.8
b_labels, b_preds = gen_labels_preds(fraud, genuine, b_total_fraud_
predicted, b_correct_fraud_predicted)
b_recall, b_precision, b_FPR = calculate_metrics(b_labels, b_preds)

print("Показатели эффективности модели A:")
print('TPR:', a_recall)
print("Точность:", a_precision)
print("FPR:", a_FPR)

print("\nПоказатели эффективности модели B:")
print('TPR:', b_recall)
print("Точность:", b_precision)
print("FPR:", b_FPR)
```

```
Показатели эффективности модели A:
TPR: 0.8
Точность: 0.2857142857142857
FPR: 0.0002

Показатели эффективности модели B:
TPR: 0.8
Точность: 0.008620689655172414
FPR: 0.0092
```

В третьем сценарии он снова использовал те же метрики и 100 000 транзакций. Всего было 1000 случаев мошенничества, так что соотношение классов составило 1 : 100:

```
fraud = 1000  # сильный дисбаланс классов
genuine = 100000

# Модель A
a_total_fraud_predicted = model_A_FP + fraud * 0.8
a_correct_fraud_predicted = fraud * 0.8
```

```
a_labels, a_preds = gen_labels_preds(fraud, genuine,
                    a_total_fraud_predicted, a_correct_fraud_predicted)
a_recall, a_precision, a_FPR = calculate_metrics(a_labels, a_preds)

# Модель B
b_total_fraud_predicted = model_B_FP + fraud * 0.8  # помечает гораздо
                                                      больше транзакций
b_correct_fraud_predicted = fraud * 0.8
b_labels, b_preds = gen_labels_preds(fraud, genuine, b_total_fraud_
predicted, b_correct_fraud_predicted)
b_recall, b_precision, b_FPR = calculate_metrics(b_labels, b_preds)

print("Показатели эффективности модели A:")
print('TPR:', a_recall)
print("Точность:", a_precision)
print("FPR:", a_FPR)

print("\nПоказатели эффективности модели B:")
print('TPR:', b_recall)
print("Точность:", b_precision)
print("FPR:", b_FPR)

Показатели эффективности модели A:
TPR: 0.8
Точность: 0.975609756097561
FPR: 0.0002

Показатели эффективности модели B:
TPR: 0.8
Точность: 0.46511627906976744
FPR: 0.0092
```

Модель A показывает лучшие результаты как по метрике площади под характеристической кривой зависимости истинных и ложных положительных срабатываний (ROC AUC), так и по метрике площади под кривой точности-полноты (PR AUC). Модель B — неудачная модель, но при этом она демонстрирует хороший уровень ложноположительных срабатываний (FPR) — 0.0092. Однако если бы такую модель внедрили в эксплуатацию, ее прогнозы оказались бы бесполезными — 920 из 1000 предполагаемых мошенничеств были бы ошибочными. Это видно по метрике точности — у модели B она составляет всего 0.08, и поэтому ее даже не стоит всерьез рассматривать.

В чем здесь ошибка?

Прежде всего модель B имеет FPR 0.0092, что в 46 раз выше, чем у модели A (FPR — 0.0002). Нет понятий «хороший» или «плохой» FPR — все зависит от объема данных. Даже небольшое различие может привести к значительным последствиям. Например, FPR 0.99 — это в 10 раз хуже, чем 0.999 (соотношение 1:100 против 1:1000).

Аналогичную ситуацию видим и в приведенном примере: хотя точность у модели B лишь в 10 раз ниже, FPR у нее — в 46 раз выше, что не очень хорошо.

Как видно из предыдущих расчетов, метрика точности значительно изменяется при смещении баланса классов, даже если сама модель сохраняет прежний уровень производительности. В то же время показатели TPR и FPR остаются неизменными.

Как использовать эту информацию при выборе подходящих метрик?

В одной из компаний, в которой мы работали, нашей задачей было сокращение количества спама и мошеннической активности — при более чем 100 000 000 000 событий в день. Мы задали требование, чтобы специфичность (Specificity = TNR = 1 — FPR) была не ниже 0.999999 (то есть мы допускали один ложноположительный результат на миллион событий), и при этом стремились максимизировать полноту (TPR) при заданной специфичности. Это оказалось более эффективно, чем использование стандартной пары «точность — полнота», особенно с учетом нестабильности исходных данных.

Однако в некоторых случаях, чтобы найти метрику, позволяющую добиться желаемого поведения системы, приходится импровизировать.

Байка от Арсения

Я работал в компании, занимающейся оптимизацией производственных процессов, и должен был улучшить систему обнаружения дефектов. Однако в ходе работы всплыла другая проблема: используемые метрики оказались недостаточно чувствительными. Для тестирования предполагаемого сценария не хватало данных — всего 10–20 дефектных образцов на продукт. Получить больше данных было невозможно: других дефектных единиц просто не существовало благодаря высокому качеству инженерных решений.

Кроме размера датасета была еще одна особенность: клиентов не интересовали промежуточные показатели (например, насколько точно модель оценивает вероятность брака). Их требования были довольно прямолинейны:

- Есть 10 дефектных единиц и N исправных.
- Идеальный вариант — 0 ошибок.
- 1 ложноположительное или 1 ложноотрицательное срабатывание допустимо.
- Все остальное делает систему непригодной к использованию.

Большинство моих попыток улучшить систему в существующих условиях не дали никаких результатов, пока я не решил разработать собственную непрерывную метрику, которая бы использовала внутренние метрики модели и имела разумные пороговые значения. Такая метрика оказалась весьма дискретной:

- «0» означало «идеальная система».
- «1» — «приемлемая».
- «2» — «мусор».

С такой метрикой я начал постепенно улучшать систему, будучи абсолютно уверенным, что все делаю правильно.

После ряда незначительных улучшений система из «мусора» превратилась в «приемлемую», а затем — для многих клиентов — в «идеальную».

Одним из ключевых факторов успеха вашей ML-системы всегда будет ее согласованность. Чтобы ее достичь, существует отдельная категория метрик, о которой мы поговорим в следующем разделе.

5.2.1. Метрики согласованности

В прикладном ML часто требуется, чтобы модель демонстрировала согласованность вывода при незначительных изменениях входных данных. Данное свойство, известное в различных дисциплинах как согласованность (consistency), устойчивость (robustness), стабильность (stability) или гладкость (smoothness), может быть формально определено как требование инвариантности модели к определенным преобразованиям — разность между выходными значениями модели для исходного и немного измененного ввода должна стремиться к нулю. Иными словами, это свойство можно математически выразить следующим образом:

$$f(x) - f(x + \text{eps}) \to 0,$$

где f обозначает модель, x — исходный ввод, а eps — какое-то возмущение. Метрики согласованности редко обсуждаются в академических кругах, но при этом играют важную роль на практике, где даже небольшие изменения входных данных могут существенно повлиять на вывод модели с точки зрения конечного продукта.

Существуют разные типы возмущений. Например, для модели компьютерного зрения небольшое изменение освещенности не должно приводить к изменению вывода модели. Аналогично, модель для анализа тональности текста не должна быть чувствительна к замене слов синонимами. О подобных возмущениях и инвариантах мы поговорим более подробно в главе 10, посвященной тестированию ML-систем.

Существует и другое, схожее по смыслу свойство: если модель проходит повторное обучение (например, с добавлением новых данных или при использовании других начальных условий), мы ожидаем, что она будет выдавать такие же или близкие результаты при тех же входных данных. Например, для системы обнаружения мошенничества недопустимо, чтобы один и тот же пользователь сегодня считался мошенником, завтра — уже нет, а через неделю — снова мошенником.

$$f_1 = \text{train}(\text{dataset_version_N});$$

$$f_2 = \text{train}(\text{dataset_version_N} + 1);$$

$$f_1(x) - f_2(x) \to 0.$$

Изменение выходных данных модели со временем может нарушить работу смежных систем или повлиять на конечных пользователей, особенно если речь идет о регулярных обновлениях модели (обычное явление для большинства ML-систем). Пользователи крайне редко положительно воспринимают

непредсказуемые изменения в инструментах или среде, с которой они взаимодействуют.

Такие свойства могут быть не менее важны, чем базовые метрики модели, например точность, поскольку они формируют ожидания пользователей. Как уже упоминалось в предыдущих главах, если модель нельзя считать надежной, ее практическая ценность снижается. Следовательно, нам нужны специфические метрики для оценки такого поведения.

Хорошая новость: мы сформулировали данные свойства, так что главный оставшийся вопрос — оценка подходящего типа шума или возмущения для вышеупомянутых формул: какие инварианты следует учитывать и как со временем могут меняться условия?

При наличии данных характеристик можно использовать стандартные метрики оценки согласованности. Например, в случае поисковой системы (Photostock Inc.) нам важно, чтобы релевантность документа по конкретному запросу не изменялась в разных релизах системы. В таком случае метрикой согласованности может выступать изменение позиции документа в ранжировании по паре «запрос — документ» в течение определенного периода времени и по множеству документов и запросов. Разумеется, чем меньше дисперсия, тем лучше для системы. Однако также нельзя забывать о некорректных сценариях: например, простейшая модель, возвращающая постоянный результат, даст минимальную дисперсию, но это не тот тип согласованности, к которому мы стремимся.

Согласованность является важным свойством ML (рис. 5.5). Если это актуально и для вашей системы, стоит ввести метрику, отражающую реакцию системы на изменения входных данных, обучающей выборки или параметров обучения.

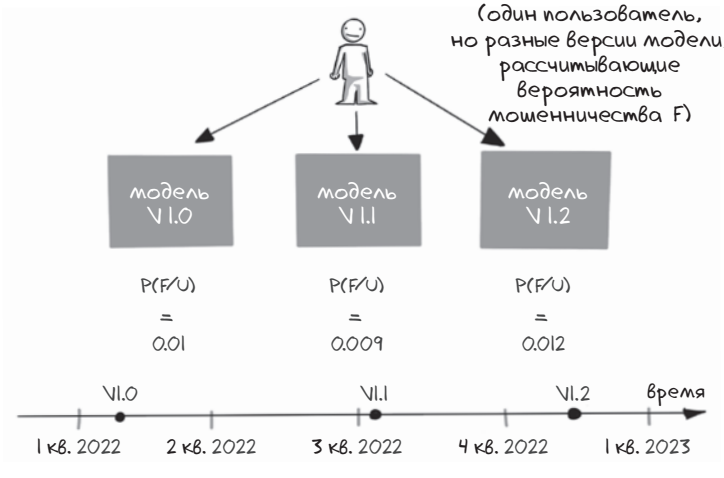


Рис. 5.5. Новые версии модели демонстрируют достаточно стабильные результаты при оценке вероятности (P) того, что пользователь (U) является мошенником (F)

В итоге у вас будет единая система метрик, основанная на четкой иерархии офлайн- и онлайн-метрик.

5.2.2. Офлайн- и онлайн-метрики, прокси-метрики и иерархия метрик

Настройка и улучшение метрик — важный этап в построении эффективной ML-системы. Но даже это не является нашей конечной целью; мы копнем еще глубже. Ведь когда нам было нужно сократить спам и мошенническую активность, целью было не просто достичь максимального значения полноты при заданной специфичности, а улучшить пользовательский опыт путем уменьшения числа спам-сообщений и повышения безопасности за счет обнаружения мошенничества.

В случае Supermegaretail целью было сокращение убытков, вызванных дефицитом или переизбытком товаров. Это можно выразить в денежном эквиваленте, но не через такие метрики, как средняя абсолютная ошибка (MAE), среднеквадратичная ошибка (MSE), взвешенная средняя абсолютная процентная ошибка (wMAPE), взвешенная абсолютная процентная ошибка (WAPE) и др.

Иными словами, метрика, которая была использована для оценки модели на этапах обучения, тестирования и валидации, редко совпадает с итоговыми метриками (табл. 5.1).

Таблица 5.1. Примеры офлайн- и онлайн-метрик

Офлайн-метрики	Онлайн-метрики
Полнота при заданной специфичности для распознавания спам-сообщений	Количество жалоб пользователей на спам-сообщения
Квантили 1.5, 25, 50, 75, 95 и 99	Стоимость просроченных товаров, общий объем продаж
Средний обратный ранг, нормализованное дисконтированное кумулятивное усиление (NDCG)	Коэффициент кликабельности по результатам поисковой выдачи

Перечисленный выше набор называют *офлайн-метриками* (или *автономными*), потому что их можно применить и рассчитать без ввода модели в эксплуатацию. В противоположность им, некоторые метрики, как правило, являющиеся целевыми, могут быть рассчитаны только после внедрения системы и использования ее результатов в бизнесе (*онлайн-метрики*, или *оперативные*). И хотя иногда офлайн- и онлайн-метрики могут совпадать, мы все равно должны оценивать их по-разному. Наиболее распространенный способ оценки онлайн-метрик (изменения/улучшения) — это A/B-тестирование.

Необходимость офлайн-метрик объясняется просто — ими можно пользоваться до ввода системы в эксплуатацию. Такой подход быстр, воспроизводим и не

требует затратного внедрения модели. Офлайн-метрики должны обладать одним важным свойством — уметь прогнозировать онлайн-метрики. Другими словами, рост или падение значений офлайн-метрик должно либо сильно коррелировать, либо быть пропорционально изменению онлайн-метрик. Таким образом, офлайн-метрики играют роль прокси-метрик для онлайн-метрик и могут эффективно использоваться для их прогнозирования.

Но если мы можем найти офлайн-метрики, сильно коррелирующие с нашими онлайн-метриками, сохранив при этом транзитивность улучшений, мы можем применить тот же подход и к офлайн-метрикам. Рассмотрим это на примере.

Представим, что мы разрабатываем систему рекомендаций для сайта интернет-магазина. Наша конечная цель — увеличить валовую стоимость товара (gross merchandise value, GMV; это метрика, измеряющая общий объем продаж за определенный период). К сожалению, как уже упоминалось ранее, мы не можем ее измерить до ввода системы в эксплуатацию и проведения A/B-тестов. Поэтому мы лишь предполагаем, что увеличение числа покупок приведет к росту GMV. Для достижения этой цели мы хотим повысить коэффициент конверсии (CR), предлагая пользователям товары, которые они могут купить (при условии, что это увеличит общее число покупок).

В среднем 3 % предложений приводят к клику, и 3 % из этих кликов заканчиваются покупкой: 3 % умножить на 3 % означает, что, если мы покажем 10 000 предложений, только 9 из них приведут к покупке. Это влечет за собой два неблагоприятных и взаимосвязанных последствия:

- малый объем данных положительного класса (покупки), сильный дисбаланс классов;
- увеличение продолжительности A/B-тестов.

Например, для A/B-тестов с соотношением успешных попыток 9/10 000 потребуется в 100 раз больше данных, чем для соотношения 90/10 000 (квадратичная зависимость между минимальным обнаруживаемым эффектом и объемом выборки — подробнее об этом далее).

Для решения этой проблемы мы можем использовать прокси-метрику — коэффициент кликабельности (CTR), принимая во внимание следующие моменты:

- Без клика не может быть и покупки, поэтому мы можем ожидать положительную корреляцию между CTR и CR и даже рассчитать ее.
- Количество кликов в 33.3 раза превышает количество покупок, а это означает, что мы получим в 33.3 раза больше обучающих данных для положительного класса системы, и A/B-тесты станут в 1111 (33.(3)^2) раз быстрее. (Точнее говоря, мы можем ожидать изменения дисперсии, так как

$\text{var} = p \times (1 - p)$, соответственно, при $p = \frac{9}{10000}$ var = 0.000899, а при $p = \frac{3}{100}$ var = 0.0291, что означает, это в целом мы увеличим скорость сходимости в $\frac{1111}{\frac{0.0291}{0.000899}} \approx 33.33$ раза.)

Использование CTR вместо CR позволяет нам проводить итерации быстрее и с большей чувствительностью как офлайн (оценка метрик и функции потерь становится проще благодаря большему объему данных по интересующему классу), так и онлайн (по крайней мере частично — через A/B-тестирование).

Это можно представить следующей цепочкой:

- CTR → CR → (общее количество покупок) → GMV.

Построим следующую иерархию метрик:

1. Основная корпоративная метрика — это выручка.
2. Общая выручка (GMV) складывается из выручки по различным продуктам, включая тот, за который мы отвечаем.
3. Выручка по нашему продукту зависит от:
   - средней стоимости покупки;
   - частоты покупок;
   - числа пользователей (эти показатели взаимосвязаны и влияют друг на друга).
4. Частота покупок зависит от CR.
5. Коэффициент конверсии зависит от CTR.

Иерархия метрик (рис. 5.6) помогает подобрать подходящие прокси-метрики. И хотя ее построение выходит за рамки проектирования ML-системы, полезно иметь такую иерархию и опираться на нее в процессе разработки. Такая общая основа позволяет обосновать свои решения и снижает риск неудачи.

Иерархия метрик особенно важна, когда система достигает такой степени зрелости, при которой отдельные метрики могут противоречить друг другу. Один наш знакомый рассказал небольшую историю о разработке системы рекомендаций: вариант, показавший более высокий уровень вовлеченности внутренних пользователей (им больше понравились новые рекомендации по сравнению с предыдущими), оказался значительно менее прибыльным при применении на более широкой аудитории.

Иерархия метрик и концепция прокси-метрик тесно связаны с многокомпонентными функциями потерь, о которых мы говорили ранее. Например, при построении системы рекомендаций для Supermegaretail можно адаптировать

специальную функцию потерь, учитывающую различные уровни активности пользователей (клики, покупки, общее количество приобретенных товаров) и балансирующую разные метрики.

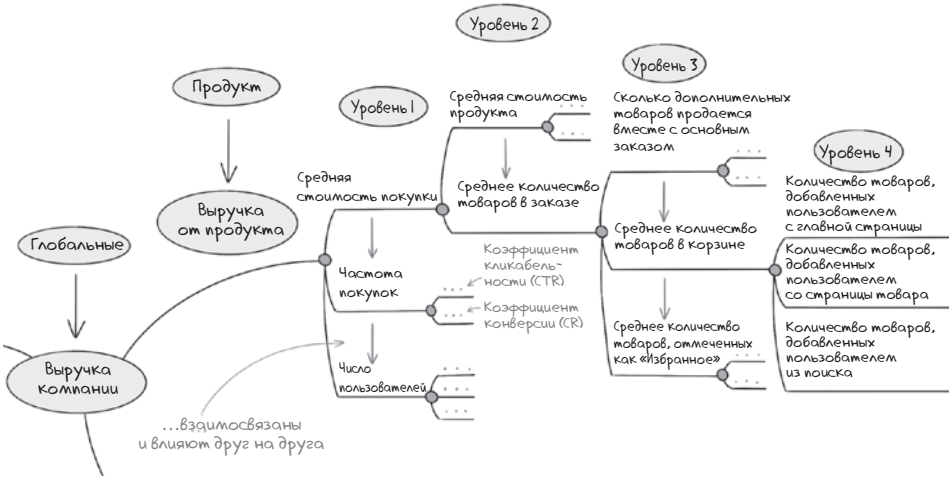


Рис. 5.6. Иерархия метрик

Байка от Арсения

Однажды я работал над совершенно новой функциональностью, основанной на компьютерном зрении. Предложенное решение было разбито на компоненты, и каждый компонент и подкласс компонентов был тщательно аннотирован метриками. Из-за новизны функции метрики были кастомными — в основном это были соотношения различных возможных исходов. Мы разрабатывали иерархию метрик совместно с продакт-менеджером. После нескольких экспериментов, направленных на улучшение одной из метрик, у меня появилось чувство, что метрика несбалансирована. Чтобы это проверить, я провел контрольный эксперимент: заменил прогноз модели случайным шумом, сгенерированным по определенным параметрам. Удивительно, но случайная модель показала идеальный результат! Изначально метрика была спроектирована с приоритетом полноты над точностью, но такой дисбаланс оказался нежелательным, поэтому мы вскоре переработали метрику.

5.3. Дизайн-документ: добавляем функции потерь и метрики

Начиная с главы 4, мы работаем над дизайн-документами для двух вымышленных кейсов — Supermegaretail и PhotoStock Inc. В данном разделе продолжим развитие решений на основе ML для каждого кейса и рассмотрим выбор подходящих функций потерь и метрик. Начнем с Supermegaretail, а затем перейдем к PhotoStock Inc.

5.3.1. Метрики и функции потерь для Supermegaretail

Давайте освежим в памяти задачу кейса Supermegaretail. Нам требовалось сократить разрыв между поставленными и проданными товарами, сделав его как можно меньшим, при этом избегая ситуации дефицита на складе и соблюдая определенное соглашение об уровне сервиса (SLA), которое будет описано далее.

ДИЗАЙН-ДОКУМЕНТ: SUPERMEGARETAIL

II. МЕТРИКИ И ФУНКЦИИ ПОТЕРЬ

1. МЕТРИКИ

Прежде чем самостоятельно выбирать метрику, стоит предварительно провести исследование. К счастью, существует множество научных работ, посвященных данной проблеме, среди которых выделяется «Evaluating Predictive Count Data Distributions in Retail Sales Forecasting» Стивена Колассы (Stephen Kolassa) (https://mng.bz/eVl9).

Вспомним цель проекта: сократить разрыв между поставленными и проданными товарами, сделав его как можно меньшим, при этом избегая дефицита на складе при соблюдении заданного SLA. Для этого нам нужно спрогнозировать спрос на конкретный товар в конкретном магазине за определенный период времени с использованием ML-системы.

В данном контексте аннотация упомянутой статьи подходит идеально:

> *Значительное увеличение вычислительных мощностей и новые архитектуры баз данных позволяют хранить и обрабатывать данные со всё большей детализацией, в результате чего временны́е ряды счетных данных содержат всё меньшие значения количества событий. Такие ряды больше не могут обрабатываться с помощью приближенных методов, предназначенных для непрерывных распределений вероятностей. Более того, недостаточно рассчитывать только точечные прогнозы: необходимо прогнозировать полные (дискретные) прогнозные распределения, особенно в задачах прогнозирования цепочек поставок и управления запасами, а также в других процессах планирования.*

(Под «счетными данными» понимаются временны́е ряды, значения которых являются целыми числами. Это критически важно для задачи прогнозирования цепочек поставок, с которой мы имеем дело, поскольку большинство товаров продаются поштучно.) Мы можем кратко привести основное содержание статьи (см. далее пункты A–G) и выбрать наиболее подходящие метрики для достижения нашей цели.

A. Метрики, основанные на абсолютных ошибках

MAE (средняя абсолютная ошибка) оптимизирует медиану; взвешенная средняя абсолютная процентная ошибка (wMAPE) — это MAE, деленная на среднее значение выборки вне обучающего набора, а средняя абсолютная

масштабированная ошибка (MASE) получается путем деления MAE (полученной на основе прогнозов реальной модели) на MAE обучающего набора с помощью метода случайного блуждания.

Оптимизация по медиане мало чем отличается от оптимизации по среднему значению в случае симметричного прогнозного распределения. Однако прогнозные распределения, характерные для счетных данных с низким числом событий, как правило, несимметричны, и это различие стоит принимать во внимание, поскольку оно приводит к смещенным прогнозам.

B. Процентные ошибки

Средняя абсолютная процентная ошибка (MAPE) не определена в случае, если хотя бы одно из будущих фактических значений равно нулю, поэтому она абсолютно непригодна для счетных данных.

Симметричная MAPE представляет собой «симметризованную» версию MAPE, которая определена при условии, что точечные прогнозы и фактические значения не равны нулю одновременно на всех будущих временны́х точках. Однако если в какой-либо временной точке фактическое значение равно нулю, вклад этой точки в метрику будет равен 2, независимо от прогнозируемого значения, что делает ее непригодной для счетных данных.

C. Метрики, основанные на квадратичных ошибках

Минимизация квадратичной ошибки приводит к несмещенному точечному прогнозу. Однако среднеквадратичная ошибка (MSE) не подходит для товаров с прерывистым периодическим спросом, поскольку она чувствительна к крупным ошибкам прогнозирования. Та же проблема актуальна и для непрерывных счетных данных.

D. Относительные ошибки

К ним относятся медианная относительная абсолютная ошибка и геометрическое среднее относительных абсолютных ошибок.

В контексте прогнозирования счетных данных эти метрики обладают двумя основными недостатками:

- Относительные ошибки обычно сравнивают абсолютные ошибки. В этом смысле они схожи с метриками, основанными на MAE, которые были упомянуты ранее.
- При анализе по периодам простые базовые бенчмарки, такие как модель случайного блуждания, могут давать нулевую ошибку прогнозирования, в результате чего относительная ошибка за этот период будет неопределенной из-за деления на ноль.

E. Ошибки, основанные на относительных показателях

Курентзес (Kourentzes) не так давно предложил две новые метрики ошибок для прерывистого спроса: MSR (mean square rate) и MAR (mean absolute

rate), оценивающие то, насколько точечный прогноз прерывистого спроса адекватно отражает средний спрос на протяжении увеличивающегося временного интервала. Это интересное предложение, однако одной из особенностей этих метрик является то, что они неявно придают больший вес краткосрочным прогнозам по сравнению со средне- и долгосрочными. Можно утверждать, что именно это и требуется при прогнозировании, однако при этом логично предположить, что такое взвешивание должно быть явным — путем использования соответствующей схемы весов при усреднении по будущим временным периодам.

F. Масштабированные ошибки

Петропулос (Petropoulos) и Курентзес предложили масштабированную версию MSE — sMSE, которая представляет собой среднее значение квадратов ошибок, масштабированных с использованием квадрата среднего фактического значения на прогнозном горизонте. Метрика sMSE хороша, за исключением случаев, когда все фактические значения равны нулю. Она минимизируется математическим ожиданием распределения спроса f и, благодаря масштабированию, может быть использована для сравнения различных временных рядов. Кроме того (снова благодаря масштабированию), она менее чувствительна к крупным ошибкам прогнозирования, чем MSE. В частности, она более устойчива к серьезным занижениям прогноза, хотя по-прежнему чувствительна к крупным завышениям прогноза.

G. Функционалы и функции потерь

Другой способ анализа — сосредоточить внимание на точечных прогнозах, являющихся функционалами прогнозного распределения. Можно утверждать, что продавец ориентируется на определенный уровень сервиса (например, 95 %), и, следовательно, его интересует соответствующий квантиль прогнозного распределения. Такой подход можно реализовать с помощью соответствующих функций потерь или оценочных правил, при этом он тесно связан с концепцией прогнозирования как части системы управления запасами. С этой точки зрения квантильные прогнозы используются как входные данные для стандартных стратегий управления запасами, а качество прогнозов оценивается на основе общего объема товарных запасов с течением времени и его соотношения с потерями от дефицита.

И хотя авторы не позиционировали данный метод как наилучшее решение, а лишь предложили в качестве альтернативы, последний абзац статьи выглядит многообещающе. Прогнозирование различных квантилей для соблюдения SLA обосновано с точки зрения бизнеса; важно, чтобы функция потерь совпадала с оцениваемой метрикой. Таким образом, выбор квантильных метрик для квантилей 1.5, 25, 50, 75, 95 и 99 полностью обоснован. Более того, если необходимо уделить большее внимание конкретной SKU, группе товаров или кластеру, квантильные метрики позволяют учитывать вес объекта/группы (например, цену товара).

1.2. ВЫБРАННЫЕ МЕТРИКИ

Квантильные метрики для квантилей 1.5, 25, 50, 75, 95 и 99 как в базовом виде, так и с весами, равными цене SKU, и дополнительным штрафом за недопрогноз или перепрогноз (если это сочтено необходимым) рассчитываются как точечные оценки с 95 % доверительными интервалами (с использованием бутстрэпа или кросс-валидации). Кроме того, данную метрику можно преобразовать, представив ее не как абсолютное значение, а как абсолютную процентную ошибку для заданного квантиля. Следует учитывать все соображения, изложенные в статье Петропулоса и Курентзеса относительно процентных ошибок. В итоге серия экспериментов поможет определить окончательную форму метрики. Скорее всего, будут использованы оба варианта, так как целесообразно оценивать как абсолютные значения в деньгах/штуках, так и процентную ошибку.

Онлайн-метрики, представляющие интерес в ходе A/B-тестирования:

- выручка — ожидается увеличение;
- уровень запасов — ожидается снижение или стабильные показатели;
- маржа — ожидается увеличение.

$$\frac{\sum_{i=1}^{N}(\alpha - I(t_i \le a_i))(t_i - a_i)w_i}{\sum_{i=1}^{N} w_i}.$$

- Alpha — коэффициент, используемый в функциях потерь на основе квантилей;
- W — веса;
- I — индикаторная функция;
- A — выход модели;
- T — метка.

2. ФУНКЦИИ ПОТЕРЬ

При совпадении метрик и функций потерь выбор последних становится очевидным. Мы обучим шесть моделей, используя квантильную функцию потерь для квантилей 1.5, 25, 50, 75, 95 и 99, что приведет к созданию шести различных моделей, каждая будет работать по соответствующему квантилю прогнозного распределения.

В качестве второго направления экспериментов дополнительно будет рассмотрена функция потерь Твиди. Распределения Твиди представляют собой семейство вероятностных распределений, включающее исключительно непрерывные распределения (нормальное, гамма, обратное распределение Гаусса), дискретное масштабированное распределение Пуассона, а также класс составных распределений Пуассон-гамма, которые имеют положительную массу в нуле, но в остальном являются непрерывными. Эти свойства делают функцию потерь Твиди подходящей для работы со счетными данными.

5.3.2. Метрики и функции потерь для PhotoStock Inc.

Далее рассмотрим кейс PhotoStock Inc., в котором нам понадобится совершенно другой набор функций потерь и метрик, что объясняется спецификой бизнеса и решаемой задачи. В данном случае нам необходимо разработать современную поисковую систему, способную находить наиболее релевантные изображения на основе текстовых запросов пользователей, обеспечивая при этом высокую производительность и выдавая наиболее актуальные изображения из базы.

ДИЗАЙН-ДОКУМЕНТ: PHOTOSTOCK INC.

II. МЕТРИКИ И ФУНКЦИИ ПОТЕРЬ

1. Метрики

При выборе метрик для новой поисковой системы PhotoStock необходимо учитывать ожидаемое поведение системы, включая следующее:

- Пользователи кликают по ссылкам в результатах поиска, при этом верхние результаты получают больше кликов. Это можно отразить в метрике CTR (коэффициент кликабельности), которая показывает, сколько пользователей кликнули по результатам поиска.
- Пользователи приобретают изображения, найденные через поиск. Это можно отразить в метрике CR (коэффициент конверсии), которая показывает, сколько кликов привело к покупке.
- Пользователям предлагаются разнообразные варианты на странице результатов поиска (SERP, search engine result page). Здесь нет готовых решений, поскольку у нас нет четкого понимания разнообразия; это нужно обсудить с UX-командой. В качестве базовой метрики можно использовать количество различных категорий изображений, представленных на SERP. В дальнейшем стоит изучить опыт других компаний, например статью Airbnb «Learning to Rank Diversely at Airbnb» (https://arxiv.org/abs/2210.07774).
- Результаты поиска соответствуют ожиданиям. Это можно определить с помощью метрики экспертной оценки, показывающей, сколько пользователей считают результаты поиска релевантными.

CTR и CR — это онлайн-метрики, то есть они могут быть измерены только при реальной работе системы. Разнообразие — это неконтролируемая офлайн-метрика, а значит, она не требует дополнительных данных и может рассчитываться регулярно без каких-либо затрат. Экспертная оценка, напротив, является контролируемой офлайн-метрикой, для нее необходимы дополнительные данные (экспертные оценки), на сбор которых требуется время и ресурсы.

Для введения офлайн-прокси-метрик CTR и CR можно использовать классические метрики для проблем ранжирования, такие как средний обратный ранг (mean reciprocal rank, MRR) и нормализованный дисконтированный кумулятивный выигрыш (normalized discounted cumulative gain, NDCG). MRR — это метрика, которая вычисляет среднее значение обратных рангов

для заданного набора результатов и отражает среднее значение обратного ранга первого релевантного результата. NDCG — метрика, вычисляющая среднее значение дисконтированных кумулятивных выигрышей (DCG) для заданного набора результатов и отражающая сумму оценок релевантности первых N результатов, деленную на идеальное значение DCG. В свою очередь, DCG — это сумма оценок релевантности первых N результатов, упорядоченных по убыванию релевантности.

Как MRR, так и NDCG требуют наличия списка релевантных результатов для каждого запроса с целью расчета метрик. Мы можем использовать один и тот же список как для MRR, так и для NDCG, однако его необходимо составить с помощью краудсорсинга, чтобы он отражал те результаты, которые будут видеть пользователи. И хотя MRR — это подходящая офлайн-метрика для CTR, для CR он может оказаться не самой удачной прокси-метрикой, поскольку список релевантных результатов, полученный с помощью краудсорсинга, не отражает реальные данные о покупках. Поэтому для точного измерения CR следует использовать реальные данные. Тем не менее в первой версии системы мы, скорее всего, сможем отслеживать CR только в онлайн-режиме с помощью A/B-тестов и поэтапного внедрения.

Итак, как можно классифицировать метрики:

- быстрые офлайн-метрики: MRR, NDCG, разнообразие;
- медленные офлайн-метрики: экспертная оценка;
- онлайн-метрики: CTR, CR.

2. Функции потерь

Для применения функций потерь при обучении поисковой системы важно учитывать доступные данные и желаемые результаты. В данном случае основными аспектами, которые мы хотим оптимизировать, являются клики, покупки и разнообразие.

Для улучшений, связанных с кликами и покупками, можно использовать функцию потерь на основе бинарной кросс-энтропии. Однако важно учитывать, что данные по кликам и покупкам могут быть несбалансированными, то есть в выборке может быть значительно больше примеров одного класса по сравнению с другим. В таких случаях можно использовать функцию потерь, устойчивую к дисбалансу классов, например focal loss или другие специализированные функции.

Focal loss — это функция потерь, продемонстрированная в статье «Focal Loss for Dense Object Detection» (https://arxiv.org/abs/1708.02002v2). Она представляет собой обобщение функции бинарной кросс-энтропии, широко используемой в задачах классификации. Ключевое отличие focal loss от стандартной бинарной кросс-энтропии заключается в том, что она снижает вес простых примеров — тех, которые в большинстве случаев классифицируются правильно. Это полезно в случаях дисбаланса классов, поскольку помогает модели сосредоточиться на сложных примерах, которые обычно оказывают большее

влияние на общее качество модели, а потому данная функция актуальна и для поисковой системы PhotoStock.

Что касается разнообразия, можно добавить дополнительную функцию потерь, штрафующую за схожесть результатов. Один из возможных способов реализации этого — использование энтропии распределения по категориям среди результатов поиска. Однако данный подход не всегда может быть реализован, поэтому потери, связанные с разнообразием, опциональны.

Таким образом, итоговая функция потерь может быть записана следующим образом (click — клики, purchase — покупки, diversity — разнообразие):

$$L = \alpha \times L_{\text{click}} + \beta \times L_{\text{purchase}} + \gamma \times L_{\text{diversity}}.$$

Альфа, бета и гамма представлены в виде гиперпараметров, управляющих относительной важностью трех компонентов. Данные гиперпараметры можно настроить, чтобы найти оптимальный баланс между тремя аспектами.

5.3.3. Вывод

Примеры двух дизайн-документов демонстрируют, насколько важно правильно выбрать метрики и функции потерь. Как и любой другой ключевой элемент при создании ML-системы, метрики и функции потерь должны соответствовать целям проекта. И если вы чувствуете, что требуется больше времени для определения подходящих параметров, — обязательно выделите на это несколько дней, чтобы потом не пришлось снова к этому возвращаться и переделывать бóльшую часть работы.

Следующая глава будет посвящена сбору данных, датасетам, различиям между данными и метаданными, а также построению качественного пайплайна данных.

Итоги

- Не стоит использовать проверенные временем функции потерь только потому, что они сработали в предыдущих проектах.
- Функция потерь должна быть глобально непрерывной и дифференцируемой.
- Выбор функции потерь — важный этап, особенно в системах, основанных на глубоком обучении.
- Метрики согласованности стоит использовать в случаях, когда небольшие изменения входных данных могут сильно повлиять на вывод модели с точки зрения конечного продукта.
- Офлайн-метрики можно применять до ввода проекта в эксплуатацию, и они могут выступать в роли прокси-метрик для онлайн-показателей.
- Обязательно имейте под рукой иерархию метрик — она будет полезна при проектировании системы.

6 Подготовка датасета

В этой главе

- ✓ Источники данных
- ✓ Преобразование необработанных данных в датасеты
- ✓ Различия между данными и метаданными
- ✓ Определение достаточного объема данных
- ✓ Решение проблемы холодного старта
- ✓ Критерии устойчивого пайплайна данных

В предыдущих главах мы рассмотрели ключевые этапы подготовки к созданию системы машинного обучения — от анализа пространства проблем и пространства решений до оценки рисков и выбора подходящих функций потерь и метрик. Теперь пришло время поговорить о том, без чего ваша ML-система просто не сможет работать, — о датасетах. Датасеты можно сравнить со всеми неотъемлемыми составляющими нашей повседневной жизни: как машине нужен бензин, чтобы ехать, а человеку — питательный завтрак, чтобы зарядиться энергией перед рабочим днем, так и ML-системе нужен датасет для ее полноценного функционирования.

Есть одна старая поговорка о недвижимости: три самых важных фактора — это местоположение, местоположение и еще раз местоположение. А если бы нам нужно было выбрать только три наиболее важных аспекта при построении

ML-системы, это были бы данные, данные и еще раз данные. Другая распространенная цитата из мира компьютерных наук — «мусор на входе — мусор на выходе» — остается непреложной истиной.

В этой главе мы разберем особенности работы с датасетами — от поиска и обработки источников данных до правильной подготовки датасета и построения пайплайнов данных. В завершение главы мы рассмотрим, как датасеты отражаются в дизайн-документе, на примерах компаний Supermegaretail и PhotoStock Inc.

6.1. Источники данных

В качестве источников данных для вашего датасета можно использовать буквально что угодно. Доступность и качество этих источников будут зависеть от вашей рабочей среды, технологического наследия компании и множества других факторов. То, какие источники стоит рассматривать в первую очередь, определяется в основном целями вашей ML-системы. Ниже приведены наиболее популярные источники данных или их категории с примерами из практики:

- *Глобальная деятельность* — это довольно широкая категория источников данных, включающая любую деятельность в рамках какой-либо отрасли, которая регулярно фиксируется. Например, в сфере биржевой торговли трейдеры по всему миру совершают сделки, результаты которых (цены и объемы) впоследствии становятся доступными для других участников рынка.
- *Физические процессы* — это глобальные изменения, происходящие на планете в реальном времени. Их можно отслеживать на разных уровнях — от спутниковых снимков до микродатчиков, размещенных на сельскохозяйственных угодьях.
- *Внешние базы данных* — ряд компаний специализируются на сборе данных в конкретных предметных областях, используя собственные методы и технологии. К таким источникам стоит обращаться, если их данные соответствуют вашим задачам и требованиям.
- *Локальные бизнес-процессы* — здесь мы переходим от глобального к локальному. Сам бизнес в процессе своего развития может генерировать значительные объемы данных. Например, если вы работаете в сфере электронной коммерции, история покупок клиентов — ваш основной источник данных.
- *Разметка, выполняемая специальной командой*, — ваша компания может нанять отдельную команду экспертов для разметки данных под конкретную задачу.
- *Разметка, выполняемая конечными пользователями системы*, — схожий подход, при котором компания предоставляет пользователям интерфейс, позволяющий им вводить данные, которые затем будут использованы в вашей ML-системе.

- *Искусственно сгенерированные датасеты* — это данные, создаваемые научными модуляторами, воссозданной средой или другими синтетическими источниками. Объекты, созданные с помощью генеративного ИИ (например, генераторов изображений или больших языковых моделей), также относятся к этой категории.

Некоторые источники данных уникальны, и доступ к ним может обеспечить значительное конкурентное преимущество. Многие Big Tech-компании, такие как Google и Meta, добились успеха в первую очередь благодаря ценным данным о поведении пользователей, используемым для таргетированной рекламы. С другой стороны, существуют датасеты, доступ к которым получить легко: информация либо бесплатна и доступна для скачивания, либо может быть создана за умеренную плату (многие датасеты, предлагаемые поставщиками данных, стоят относительно недорого). Низкая цена вовсе не говорит о низком качестве — все зависит от того, какие данные вам нужны. Может оказаться, что именно этот бесплатный источник идеально подходит для вашей ML-системы. Кроме того, доступ к некоторым данным может быть ограничен в определенных регионах или с точки зрения закона находиться в так называемой серой зоне. Зрелые компании, как правило, придерживаются законодательства (и мы настоятельно рекомендуем делать то же самое!), но молодые стартапы с менталитетом YOLO[1] склонны иногда игнорировать эти ограничения.

Некоторые датасеты приобретают свою ценность только после обогащения или аннотирования/разметки. *Аннотирование* означает добавление в необработанный датасет соответствующих меток или, другими словами, создание тесно связанного датасета и объединение его с новым. Это распространенная практика — объединять уникальный проприетарный датасет с открытым источником данных, в результате получая гораздо более ценный датасет.

Компании в сфере рекламных технологий, упомянутые ранее, также могут выигрывать от объединения датасетов. Рассмотрим классический пример — компания занимается разработкой игры с системой монетизации free-to-play, что предполагает наличие большого количества игроков, из которых лишь некоторые совершают покупки. Список и количество платящих пользователей держатся в секрете и доступна только разработчику. В то же время партнерская рекламная сеть компании располагает миллионами пользовательских профилей, полученных на основе поведенческих данных. Объединение этих датасетов (рис. 6.1) создаст отличную маркетинговую возможность таргетировать новую рекламу на потенциальных пользователей, схожих с теми, кто уже совершал покупки. Подобные обмены данными повышают эффективность онлайн-маркетинга и являются одной из движущих сил современного интернета.

[1] YOLO — you only live once — «ты живешь только один раз». — *Примеч. пер.*

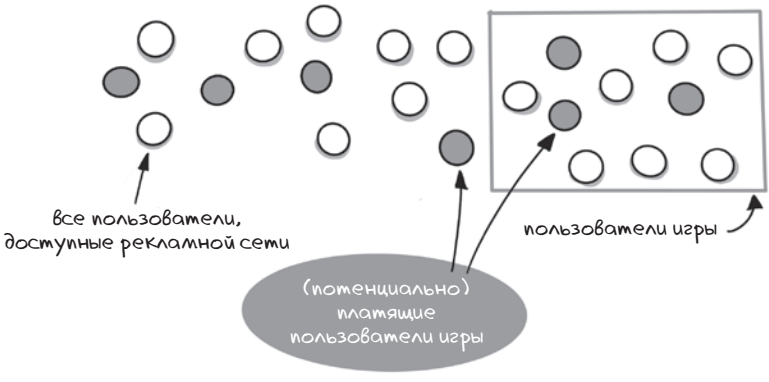


Рис. 6.1. Объединение источников данных двух совершенно разных бизнесов может принести пользу обоим

Когда мы говорим об объединении датасетов, это не всегда означает прямое соединение схожих наборов данных (например, SQL join двух таблиц). Важное понятие здесь — мультимодальность, то есть пересечение различных модальностей внутри одного датасета. Проще говоря, модальность — это тип информации, которую мы получаем. Для человека характерно мультимодальное восприятие мира (мы слышим звуки, видим цвета, ощущаем прикосновения и т. д.). В литературе, посвященной ML, мультимодальные датасеты — такие, которые объединяют разные типы данных для описания одной задачи. Например, изображения товара и его текстовое описание. Объединение датасетов различного происхождения и модальностей может принести впечатляющие результаты.

Продолжая тему объединения источников данных: Арсений однажды выполнял роль консультанта при запуске стартапа. Компания работала в сфере агротехнологий, помогая фермерам и смежным предприятиям повышать эффективность работы, и ее основное конкурентное преимущество заключалось в использовании датасетов. Механизм работы был следующим. Один из источников данных был открытым — спутниковые изображения поверхности Земли, доступные благодаря таким инициативам, как Landsat от NASA и Copernicus от ESA (с помощью которых можно получить огромное количество изображений сельскохозяйственных земель). Однако одних лишь изображений было недостаточно для повышения эффективности, так как они не содержали информации, описывающей земли. Основная проблема заключалась в том, что большинство аграрных компаний далеки от мира технологий, и не существует единого надежного источника данных о выращиваемых культурах, урожайности и других параметрах. Такие данные почти не оцифрованы, но тем не менее представляют высокую ценность для бизнеса — они могут использоваться для оптимизации расхода удобрений, прогнозирования цен на продовольственные товары и других целей. В итоге команда реализовала смарт-методы сбора такой информации и объединения

ее с обширной базой изображений. ML-система, построенная на полученных датасетах, позволила компании стремительно развиваться.

Определение источников данных и их взаимосвязей — это первый шаг в работе с данными для системы. Однако необработанные данные будут бесполезны, пока мы не сделаем их доступными для системы и модели ML, не отфильтруем и не обработаем их.

6.2. Подготовка датасета

Опытные инженеры, работающие с данными, знают, что в подавляющем большинстве случаев данные в их исходном виде слишком «сырые» для эффективного использования. Отсюда и название — *«сырые данные»*, то есть хаотично собранная, неорганизованная масса информации. Сырые датасеты редко бывают в таком состоянии, чтобы их можно было сразу использовать. Необходимо сначала их «приготовить», а уже потом вводить в вашу ML-систему.

Мы собрали список техник, которые вы можете использовать для подготовки датасета, каждая из которых представлена в отдельном подразделе. Этот список не имеет строгой последовательности, и порядок действий может варьироваться в зависимости от вашей предметной области, поэтому не существует какого-то одного универсального ответа. В некоторых случаях невозможно отфильтровать данные до их разметки, в других — фильтрация может проводиться несколько раз. Давайте кратко рассмотрим эти техники.

6.2.1. ETL

ETL расшифровывается как «extract, transform, load» (извлечение, преобразование, загрузка) и представляет собой этап подготовки данных, на котором происходит получение информации из внешнего источника и преобразование ее структуры в соответствии с вашими потребностями. В качестве упрощенного примера: вы можете получить JSON-файлы из стороннего API и сохранить их в локальное хранилище в формате CSV или Parquet.

ПРИМЕЧАНИЕ Основная цель ETL — решить проблему доступности данных.

Доступность данных на этом этапе означает следующее:

- Данные можно легко и эффективно извлекать для процесса обучения модели (например, если целевой датасет является результатом взаимодействия нескольких источников, необходимо понять, как сделать его доступным с помощью одного клика, командой или вызовом).
- Данные будут доступны как на этапе обучения, так и на этапе выполнения. Нас пока не волнует, насколько эффективно происходит извлечение данных

для инференса (мы обсудим это позже, в главе 11), но мы должны гарантировать, что те же источники данных будут доступны и для инференса.

ПРИМЕЧАНИЕ Проектирование эффективного ETL-процесса — это своего рода искусство, поскольку оно требует знания различных хранилищ данных и инструментов их обработки. В нашей книге мы лишь поверхностно касаемся этой темы и рекомендуем обратиться к другим источникам для более глубокого изучения.

Важный вопрос, который возникает на данном этапе: «Стоит ли уже сейчас задумываться о структуре и хранилище данных?» Ответ на этот вопрос зависит от следующих моментов:

- Иногда датасеты небольшие, и вероятность их внезапного увеличения в несколько раз крайне мала. Это означает, что вы можете выбрать практически любое хранилище (например, то, которое уже активно используется в вашей компании, или то, с которым вы привыкли работать). Вы удивитесь, насколько часто неопытные ML-инженеры склонны чрезмерно усложнять ситуацию, разрабатывая распределенное многокластерное хранилище для статического табличного датасета с тысячами строк и десятками столбцов.
- Иногда с самого начала очевидно, что ваш датасет будет огромным и будет быстро расти, поэтому при проектировании модели данных следует уделить этому особое внимание. Мы не являемся экспертами в данной сфере, а поэтому рекомендуем ознакомиться с другими книгами, если этот вопрос актуален для вашей системы. Нам больше всего нравится «Designing Data-Intensive Applications»[1] Мартина Клеппмана (Martin Kleppman).

6.2.2. Фильтрация

Ни один источник данных не бывает совершенно идеальным. Под *идеальностью* мы понимаем чистоту, согласованность и релевантность вашей задаче. В небольшой истории в начале этой главы мы упомянули API, предоставляющие спутниковые изображения, но компании были нужны только те из них, на которых не было слишком много облаков и которые отображали сельскохозяйственные районы; в противном случае хранение нерелевантных данных значительно увеличило бы издержки. Это означало, что большая часть предварительной работы по отбору подходящих спутниковых снимков должна была быть выполнена на самых первых этапах проекта.

Фильтрация данных — это операция, строго зависящая от предметной области. В одних случаях ее можно полностью автоматизировать с использованием набора правил или статистических методов; в других же требуется участие человека. Практика показывает, что итоговый подход, как правило, находится

[1] Клеппман М. «Проектирование высоконагруженных приложений». СПб., издательство «Питер».

где-то посередине между этими двумя крайностями. Наилучший результат дает комбинация человеческого вклада и автоматических процессов. Распространен следующий алгоритм: просматривается подмножество данных (случайным образом или на основе предварительных наблюдений или обратной связи), выявляются закономерности, которые затем отражаются в коде для расширения охвата, после чего анализируется уже более узкое подмножество.

Отсутствие фильтрации данных приводит к значительному шуму в датасете и, как следствие, к снижению производительности всей системы. Однако чрезмерно строгая фильтрация также может оказать негативное влияние — она способна исказить распределение данных, что ухудшит работу модели на реальных данных.

6.2.3. Генерация признаков

Генерация признаков означает преобразование представления данных таким образом, чтобы оно стало максимально ценным для ML-алгоритма. Мы подробно рассмотрим эту тему позже, когда речь пойдет о промежуточных этапах работы, поскольку на ранних стадиях проектирования ML-систем ей редко уделяется достаточное внимание. На текущем этапе мы, как правило, сосредоточиваемся на том, как получить исходные данные для построения базовой модели (baseline), что требует определенного уровня абстракции.

Иногда признаки создаются не вручную, а с помощью более сложной модели; в отличие от «обычных» признаков, они могут представлять собой векторы, не интерпретируемые человеком. В таких случаях точнее использовать термин *«представления»*, хотя в каком-то смысле это одно и то же — входные данные для ML-модели.

Например, в крупной компании, ориентированной на ML, может существовать основная команда, создающая модель, генерирующую наилучшие представления пользователей, товаров или чего-либо другого. Эта модель не решает бизнес-задачи напрямую, но прикладные команды могут использовать ее для создания представлений под свои конкретные нужды. Подобный подход распространен при работе с изображениями, видео, текстами или аудио.

6.2.4. Разметка

Сам по себе датасет зачастую не представляет особой ценности, но добавление дополнительных аннотаций, известных в сфере машинного обучения как *метки*, может кардинально изменить ситуацию. Определение того, какие именно метки следует использовать, крайне важно, так как это влияет на множество последующих решений.

Представим, что вы разрабатываете медицинскую систему, помогающую рентгенологам анализировать снимки пациентов. Это одно из самых известных

применений ML в медицине, которое по-прежнему крайне сложно правильно спроектировать. Казалось бы, задача проста: врач смотрит на снимок и определяет наличие злокачественного образования, так что вам нужно, чтобы врачи разметили датасет.

Существует множество способов такой разметки (рис. 6.2), в том числе:

- *Бинарная классификация* — есть ли на снимке злокачественное образование?
- *Многоклассовая классификация* — какой тип злокачественного образования на снимке, если таковое имеется?
- *Детекция* — какая область снимка содержит злокачественное образование, если таковое имеется?
- *Сегментация* — какие именно пиксели изображения представляют злокачественное образование, если таковое имеется?

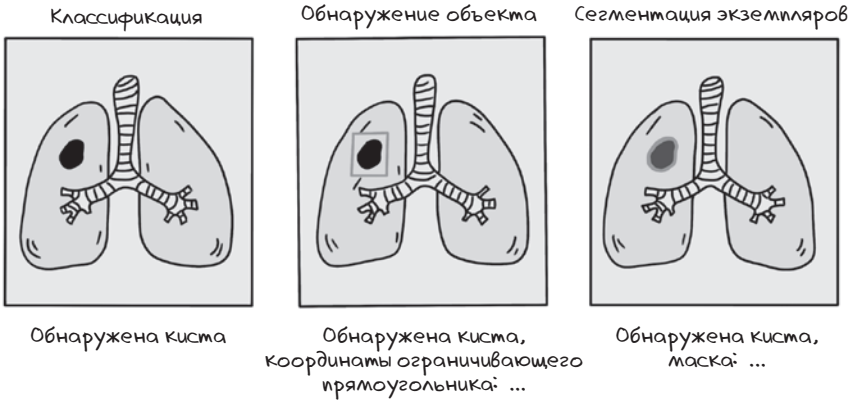


Рис. 6.2. Различные подходы к разметке данных

Эти способы разметки данных требуют разных инструментов аннотирования и различной квалификации от команды разметчиков. Кроме того, они различаются по временным затратам. Метод разметки влияет на выбор модели для решения проблемы и, что особенно важно, определяет ограничения продукта. Принять правильное решение на данном этапе невозможно без выполнения шагов, описанных в главе 2, и сбора достаточной информации о целях и требованиях продукта.

В некоторых случаях разметка данных с нужным уровнем детализации оказывается практически невозможной. Например, на это может потребоваться слишком много времени, недостаточно специалистов для охвата необходимого объема данных либо инструменты для разметки еще не готовы. В таких случаях стоит рассмотреть использование методов слабоконтролируемого обучения (weakly supervised learning), которые допускают использование неточных или

частичных меток. Если вы не знакомы с этой стороной машинного обучения, рекомендуем прочитать «A Brief Introduction to Weakly Supervised Learning» (https://mng.bz/75My), а дополнительные материалы можно найти в подборке «Awesome-Weak-Supervision» (https://mng.bz/mRp2).

Эффективная разметка данных требует выбора подходящей платформы, грамотной декомпозиции задач, четких и понятных инструкций, удобных пользовательских интерфейсов, методов контроля качества, обзора способов агрегирования и анализа стоимости. Кроме того, необходимо соблюдать лучшие практики при создании инструкций и интерфейсов, настройке различных типов шаблонов, обучении и проверке исполнителей, а также построении пайплайнов для оценки процесса разметки.

Когда данные необходимо разметить вручную, существуют два основных подхода: внутренняя команда разметки или использование сторонних краудсорсинговых сервисов. Первый подход обеспечивает более контролируемый и стабильный процесс разметки, в то время как второй позволяет ускорить процесс, зачастую с меньшими затратами. Выбор между ними зависит от сложности разметки: если она требует специализированных знаний или навыков, стоит сформировать собственную команду опытных специалистов; если же нет, то можно использовать и сторонний сервис.

Существует множество краудсорсинговых платформ, наиболее популярной, скорее всего, является Amazon Mechanical Turk. Поскольку в этой книге мы делаем акцент на широте охвата, а не на глубине, мы не будем подробно рассматривать особенности разных платформ. Вместо этого сосредоточимся на общих характеристиках краудсорсинговой разметки:

- Разметчики склонны к ошибкам даже в самых простых заданиях, например при бинарной классификации обычных объектов. Большинство разметчиков работают одновременно с несколькими заказчиками, поэтому они вряд ли запомнят все нюансы ваших инструкций. Следовательно, резметчики должны быть взаимозаменяемыми, а инструкции — максимально простыми и однозначными.
- Некоторые разметчики более внимательны, чем другие; вы можете замотивировать их уделять больше времени вашим задачам, чем остальным проектам. Другие могут быть менее внимательными или даже работать недобросовестно, например пользоваться ботами, генерирующими непроверенные метки, вместо того чтобы полагаться на собственное суждение. Крайне важно уметь отличать первых от вторых. Наиболее распространенный способ это сделать — включить в процесс разметки тесты. Возьмите ту часть датасета, метки которой вам точно известны, и используйте ее в заданиях на разметку, чтобы установить точность каждого конкретного разметчика. Также можно вести логирование определенных аспектов процесса разметки для получения более полной картины. Например, если метки генерируются слишком быстро,

это может быть сигналом о том, что исполнитель использует бот или другой запрещенный инструмент автоматизации.

- Задание на разметку должно быть сформулировано таким образом, чтобы минимизировать вариативность, вызванную участием краудсорс-работников. Рассмотрим следующую ситуацию: компания разрабатывает чат-бота и нанимает разметчиков для оценки его ответов (вопросы и несколько возможных вариантов ответа). Наиболее очевидный способ оценки — попросить разметчиков присвоить каждому ответу числовую оценку (например, от 1 до 5). Однако такие метки могут быть непоследовательными, поэтому лучше ввести несколько критериев оценки, чтобы каждый ответ оценивался отдельно: по фактической корректности, стилю высказывания и т. д. Такие метки будут более последовательными, хотя все еще далекими от идеала. Еще одно возможное улучшение — использование парного сравнения. Вместо присваивания числовой оценки каждому ответу разметчики выполняют тройное сравнение на основе критериев (версия A лучше / версия B лучше / A и B равны). Такие метки легче собирать, и ожидаемая вариативность между разметчиками будет ниже.

В XVIII веке французский математик и философ маркиз де Кондорсе (Marquis de Condorcet) сформулировал теорему, применяемую в политической науке. Де Кондорсе высказал следующую идею: если группе необходимо принять бинарное решение на основе большинства голосов и каждый член группы принимает правильное решение с вероятностью $p > 0.5$, то вероятность правильного коллективного решения возрастает с увеличением числа участников, асимптотически приближаясь к единице. Это стало обоснованием коллективного принятия решений в ранний период Французской революции — и та же логика применима к ML-системам!

Представим, что у нас есть три исполнителя, размечающих один и тот же объект для задачи бинарной классификации. Задача сложная, поэтому каждый из них прав только в 70 % случаев. Тогда можно ожидать, что результат выбора большинства из трех разметчиков будет правильным в 0.7 * 0.7 * 0.7 + 0.3 * 0.7 * 0.7 + + 0.7 * 0.3 * 0.7 + 0.7 * 0.7 * 0.3 = 78.4 % случаев. Неплохая точность!

Однако к этим числам стоит относиться с долей скепсиса. Теорема де Кондорсе предполагает выполнение ряда условий, которые здесь не соблюдаются: разметчики не являются полностью независимыми, и источники их ошибок могут быть взаимосвязаны (например, если семплы шумные и трудны для восприятия). Тем не менее группа разметчиков дает более точные результаты по сравнению с одним исполнителем, что может быть использовано для улучшения качества разметки.

В целях снижения затрат данную идею можно немного преобразовать. Один из известных нам вариантов алгоритма выглядит следующим образом:

- Объект размечается двумя разметчиками.
- Если они согласны, метка принимается.

- Если нет — добавляются еще три голоса, и итоговая метка определяется по большинству голосов.

Таким образом, больше голосов используется для сложных, неоднозначных семплов, в то время как для простых случаев требуется меньше голосов.

Данная идея проста, но весьма эффективна. Однако если разметка данных является ключевым элементом вашей проблемы и вы хотите вложить в нее дополнительные усилия, существует множество научных исследований, посвященных улучшению дизайна разметки. Мы порекомендуем «A Survey on Task Assignment in Crowdsourcing» (https://arxiv.org/abs/2111.08501) и обучающий курс от краудсорсинговой платформы Toloka.ai (https://toloka.ai/events/tutorial-wsdm/).

Постоянство и взаимное согласие разметчиков можно и нужно измерять. Специалисты в области ML зачастую стремятся использовать стандартные метрики для оценки данных показателей, однако те, кто имеет опыт работы со статистикой, знакомы с концепцией, известной как *межоценочная надежность* (interrater reliability), и соответствующим ей набором метрик (в числе которых — каппа Коэна, пи Скотта, альфа Криппендорфа и др.). Данные метрики позволяют обеспечить надежность работы команды разметки, отфильтровать недобросовестные метки (тем самым значительно повышая общую эффективность системы) и в отдельных случаях предоставляют верхнюю границу возможной производительности модели (вспомним упомянутый ранее в главе 3 стандартный уровень производительности).

Для эффективной работы команды разметки необходим соответствующий инструментарий, который может повысить эффективность и улучшить качество меток. Если вы используете сторонний ресурс, он, скорее всего, уже будет включать инструментарий для наиболее распространенных задач; при работе с собственной командой потребуется настроить или разработать такие инструменты самостоятельно. Выбор между готовым решением и созданием собственного сильно зависит от конкретной предметной области, однако в целом действует следующее правило: чем более распространена ваша проблема, тем выше вероятность найти современный и качественный инструментарий для команды разметки. Например, существует множество программных решений для разметки изображений в рамках задач детектирования, но если вам потребуется разметить видео с помощью 3D-полигональной сетки, скорее всего, придется разработать собственное решение.

Последние достижения в области крупных базовых моделей стали поворотным моментом и в процессе разметки — они могут использоваться для получения начальных меток во few-shot-сценариях с точностью, превышающей результаты разметчиков-неэкспертов. Один из многочисленных инструментов, применяемых для разметки с помощью LLM, — это Refuel Docs (https://docs.refuel.ai/), ориентированный на работу с текстовыми данными. Однако базовые модели могут использоваться и в других сферах, например в универсальных моделях

сегментации, таких как Segment Anything (https://segment-anything.com/), или в мультимодальных решениях вроде LLaVA (https://llava-vl.github.io/).

Существует множество алгоритмов, позволяющих упростить процесс разметки данных в рамках построения системы, однако в этой книге мы не будем подробно их рассматривать, так как выбор сильно зависит от конкретной задачи. Тем не менее мы хотим подчеркнуть, что вложенные в процесс разметки данных и соответствующие инструменты усилия зачастую определяют успех вашей ML-системы.

6.3. Данные и метаданные

Хотя именно датасеты используются для построения ML-системы, над ними существует дополнительный уровень информации — назовем его *метаданными*. Метаданные — это информация, описывающая определенные характеристики ваших датасетов. Например:

- *временны́е атрибуты* — включают информацию о том, когда произошло событие, когда оно было обработано и когда сохранено;
- *атрибуты источника* — указывают, собираются ли данные из нескольких источников;
- *атрибуты пользователя* — если пользователь каким-либо образом участвует в генерации датасета (а это не редкость), его метаданные становятся частью связанных с ним семплов;
- *версии* — если данные уже обработаны, полезно знать, какая версия программного обеспечения использовалась при обработке.

Метаданные играют ключевую роль в управлении потоками данных и обеспечении их согласованности. Вернемся к предыдущему примеру с медицинским приложением. Представим, что компания наняла 10 специалистов-медиков с целью разметки данных для вашей системы. После более чем года работы они наконец разметили большой датасет. Внезапно приходит электронное письмо: один из разметчиков оказался мошенником, его диплом — подделка, а лицензия приостановлена. Такая ситуация недопустима, поскольку значительная часть данных становится ненадежной, что ставит под угрозу весь датасет. Единственно разумное решение в данном случае — немедленно прекратить использование его меток в модели. И хотя такой пример может показаться немного утрированным, в вашей системе со временем обязательно возникнут десятки менее критичных, но все же значимых проблем. Проблемы с источниками данных, регуляторные вмешательства и попытки интерпретировать ошибки модели заставят вас обратиться к метаданным.

Как мы уже обсуждали ранее, несмотря на то что существуют разные способы «приготовления» датасетов, в процессе работы вы наверняка выявите какие-то новые аспекты проблемы и, соответственно, будете корректировать структуру

датасета, внося в него определенные изменения, которые и найдут свое отражение в метаданных.

Другим источником изменений может стать тестирование и исправление ошибок. Допустим, вы работаете в агрегаторе такси и решаете задачу оценки времени, необходимого для того, чтобы доехать из точки A в точку B. Вы начинаете с исторических данных компании, которые предварительно обрабатываются, — информация извлекается из логов и сохраняется в выбранную вами базу данных, в результате чего вы получаете следующую таблицу: `latitude_from`, `longitude_from`, `datetime_start`, `datetime_finish`, `distance`, `data_source`.

В какой-то момент компания поглощает своего крупного конкурента, и вы решаете включить его данные в ваш ETL-процесс. Новый инженер в команде начинает писать код, подключаются новые источники, в ваш датасет добавляются новые точки данных с той же схемой — и неожиданно производительность модели падает. Что случилось?

Ответ — ваша компания измеряла расстояние в километрах, а команда приобретенной компании — в милях. Инженер, отвечавший за интеграцию, знал об этом и реализовал поддержку миль для нового источника, но по ошибке включил ее и для старого. Таким образом, новые данные корректны для нового источника, но некорректны для старого. Если бы у вас были метаданные, например `preprocessing_function_version`, вы могли бы легко найти семплы, затронутые изменением, но без них вам придется собирать датасет заново.

Ошибки такого рода — не единственный случай, когда необходимо знать дату появления семпла данных. Некоторые ML-системы могут влиять на источники данных, создавая феномен, известный как *«петля обратной связи»*. Самый распространенный пример — системы рекомендаций. Допустим, маркетплейс продает множество товаров, и на сайте есть блок «Вас также может заинтересовать». Выбрав пойти по простому пути, компания может просто отображать там самые популярные товары. Но когда такой подход заменяется системой на основе ML, появляются новые товары, а старые лидеры теряют популярность в пользу новых, рекомендованных. При недостаточно продуманном дизайне такая система может позволить товару сомнительного качества занять лидирующее положение и удерживать его в течение длительного времени. Сохранив информацию о моменте генерации семпла и о том, какие версии связанных систем были активны в тот момент, мы сможем предотвратить петлю обратной связи.

Еще один важный сценарий, связанный с метаданными, — *стратификация*, то есть процесс выборки данных, формирующий распределение различных подгрупп. Например, представим, что компания начала свою деятельность в стране X, собрав большое количество данных о клиентах из этой страны. Позже компания вышла на новый рынок — в страну Y — и поставила цель обеспечить одинаково высокий уровень обслуживания, включая точность ML-моделей, для клиентов

обеих стран. Для этого необходимо, чтобы данные о клиентах из X и Y были сбалансированно представлены в обучающих и тестовых выборках.

Стратификация имеет решающее значение для корректной валидации и борьбы с алгоритмическими искажениями, и обе эти темы будут подробно рассмотрены в следующих главах.

6.4. Сколько нужно данных?

После всего, что было сказано о важности датасетов, у неопытного специалиста в области ML может сложиться впечатление, что достаточно просто построить пайплайн данных, который будет непрерывно поставлять множество семплов, и на этом можно остановиться. Что ж, тут нельзя однозначно ответить, прав он или нет.

Прежде всего, не все семплы одинаково полезны. Увеличение объема датасета оправданно только в том случае, если новые объекты позволяют модели обучаться чему-то новому и актуальному в рамках поставленной задачи.

Добавление семплов, очень похожих на уже содержащиеся в датасете, не имеет никакого смысла; то же самое касается и крайне зашумленных данных. Именно поэтому существует популярное направление исследований, посвященное отбору наиболее полезных для расширения датасета семплов, — *активное обучение*. Лежащая в его основе идея заключается в том, чтобы пополнять датасет теми семплами, на которых модель ошибалась (подобные сигналы обрабатываются людьми) или демонстрировала наименьшую уверенность в своем прогнозе. Представители научного сообщества разработали десятки методов активного обучения, подробнее с ними можно ознакомиться в различных обзорах, например: «A Comparative Survey of Deep Active Learning» Сюэина Чжана (Xueying Zhan) и др. (https://arxiv.org/abs/2203.13450).

Еще один момент, связанный с размером датасета, — это *семплирование*. И хотя в большинстве случаев основной вопрос: «Как собрать больше данных?», иногда возникает обратная задача: «Какую часть данных стоит использовать, чтобы повысить эффективность?» Это, как правило, актуально, когда датасет не требует ручной разметки и создается автоматически по определенному процессу, например, кликстрим на популярных B2C-веб-сервисах (поисковые системы, маркетплейсы).

Семплирование особенно эффективно, когда датасет не только объемен, но и склонен к дисбалансу или содержит множество дубликатов. В подобном случае можно применить различные стратегии, и наиболее распространенной из них является стратификация, при которой данные делятся на группы (на основе ключевых признаков или алгоритмически определенных кластеров) и объем данных для каждой группы ограничивается. При этом распределение объемов не обязательно должно быть равномерным: например, если для вашей задачи

важен временной компонент, то скорее всего, более свежие данные будут для вас более актуальны, чем старые.

Кроме того, следует учитывать *уровень шума* данных. Ранее в сфере ML существовал устойчивый консенсус: лучше иметь несколько чистых семплов, чем множество зашумленных. Однако в последнее время ситуация изменилась: прогресс в создании больших моделей, таких как GPT-3 и CLIP, показал, что при определенном масштабе ручная фильтрация становится практически невозможной (обработка датасета из 500 миллиардов токенов текста или десятков миллионов изображений обойдется слишком дорого). Зато использование большого объема слабо или автоматически размеченных данных приносит хорошие результаты, и для некоторых задач масштабные, пусть и несовершенные, датасеты оказываются более предпочтительными, чем небольшие, отобранные вручную.

> **ПРИМЕЧАНИЕ** Обучающие данные могут быть немного зашумленными, но в валидационных и тестовых датасетах это совершенно недопустимо. Более подробно эта тема будет рассмотрена в главе 7.

Можно ожидать, что производительность модели будет улучшаться асимптотически по квадратному корню от объема датасета, независимо от используемой метрики. Такая оценка довольно приблизительна и не обязательно будет точно соответствовать вашей проблеме, но зато она может дать вам некоторое представление о системе (рис. 6.3).

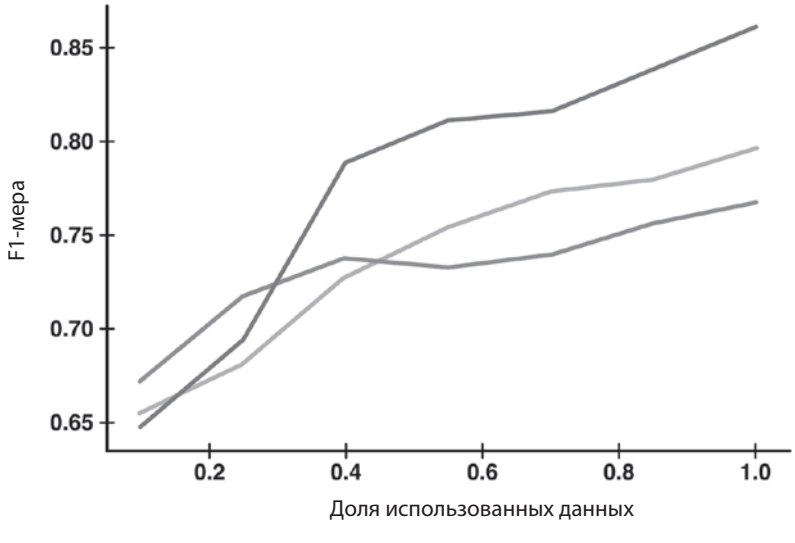


Рис. 6.3. График, демонстрирующий улучшение метрик модели по мере увеличения объема датасета. Каждая линия отражает динамику целевой метрики по отдельной задаче при увеличении объема данных

Однако не каждую модель можно улучшить добавлением новых данных из-за их неопределенности. Неопределенность может возникать по двум основным причинам:

- недостаток знаний или информации (эпистемическая неопределенность);
- неопределенность самих данных (алеаторная неопределенность).

Алеаторная неопределенность возникает, как правило, из-за сложной структуры данных, например перекрывающихся классов или аддитивного шума. Ключевая особенность такой неопределенности заключается в том, что сколько бы дополнительных обучающих данных вы ни добавили, снизить ее не получится.

С другой стороны, эпистемическая неопределенность возникает, когда модель сталкивается со входными данными, расположенными в области, слабо представленной в обучающем датасете либо вообще находящейся за его пределами (рис. 6.4).

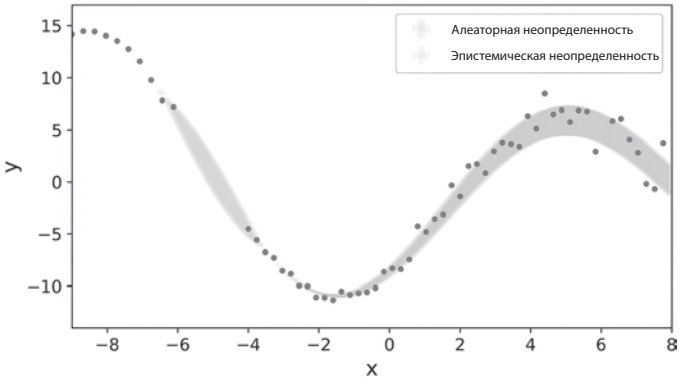


Рис. 6.4. Схематичное представление основных различий между алеаторной и эпистемической неопределенностью (источник: «Explainable Uncertainty Quantifications for Deep Learning-Based Molecular Property Prediction» ЧуИ Янга (Chul Yang) и ИПей Ли (YiPei Li) (https://mng.bz/n0ZV)

Когда вы соберете определенный объем данных и построите пайплайн обучения (подробнее об этом см. в главе 10), у вас появится возможность принимать обоснованные решения о том, сколько еще данных нужно и насколько экономически эффективно их получение. Это особенно важно при работе с дорогостоящими данными (например, размеченными высококвалифицированными специалистами). На высоком уровне алгоритм выглядит следующим образом:

- Разделите датасет на бакеты таким образом, чтобы размеры всех бакетов были приблизительно одинаковыми, а сходство между семплами из разных бакетов — максимальным. Таким образом, каждый пользовательский семпл попадает в один и тот же бакет, определяемый, например, по ID пользователя (что возвращает нас к уже упомянутой ранее стратификации).

- По возможности зафиксируйте вычислительный бюджет. Например, независимо от объема датасета модель будет обучаться на N батчах.
- Обучайте модель на разных подмножествах данных — от совсем небольшой части датасета до его полного объема. При этом убедитесь, что подмножества наращиваются накопительно (например, если семпл X использовался при обучении на 10 % подмножества, он должен быть включен и в 20 %, и в 30 %, и т. д.).
- Рассчитайте ключевые метрики для каждой обученной модели и постройте график: по горизонтальной оси — размер датасета, по вертикальной — соответствующая метрика.
- Попробуйте предположить, сколько еще данных потребуется, чтобы улучшить метрику на дополнительный 1 %.

Точность такой оценки невелика, и она не учитывает системные факторы (например, концептуальные сдвиги, о которых пойдет речь в главе 14), но даже приблизительная оценка помогает решить, сколько вам стоит инвестировать в пайплайн разметки.

6.5. Проблема курицы и яйца

Одна из самых сложных задач, с которой вы можете столкнуться при работе с датасетами, — это проблема холодного старта (также известная как проблема курицы и яйца), когда для построения системы требуются данные, но сами данные станут доступны только после запуска этой системы. Настоящий замкнутый круг!

Особенно часто такое происходит в стартапах или компаниях, запускающих продукты на новых рынках или в новых вертикалях. И как это часто бывает в ML, единственным решением становится аппроксимация. Поскольку у нас нет данных, идеально соответствующих проблеме, необходимо найти максимально им близкие. Определение подходящих данных зависит от конкретной проблемы, поэтому рассмотрим несколько примеров.

Допустим, у нас есть компания, занимающаяся безопасностью сотрудников и разрабатывающая продукты для мониторинга соблюдения техники безопасности в различных рабочих условиях. Новый продукт должен определять, у всех ли работников на территории завода надеты каски. После внедрения системы клиенты поделятся с вами данными о ее использовании, но до этого момента камеры клиентов вам недоступны.

Что же в таком случае делать?

- *Подход 1.* Используем компьютерную графику: берем набор 3D-моделей людей и рендерим их на фоне заводских интерьеров — как с касками, так и без.

- *Подход 2*. Обратимся к общедоступным источникам: ищем релевантные изображения в открытых источниках (от Google Images до фотобанков), скачиваем или покупаем их.
- *Подход 2.5 (гибридный)*. Комбинируем подходы: берем реальные изображения людей с завода и дорисовываем/рендерим на них каски.
- *Подход 3*. Немного побудем актерами: всей командой покупаем строительные каски, отправляемся на заброшенный завод и проводим фотосессию!
- *Подход 4*. Пойдем по пути наименьшего сопротивления: находим открытый датасет с фотографиями людей в похожих касках, но сделанных на улице, на строительной площадке и т. д., — не совсем то, что нужно, но достаточно близко.
- *Подход 5*. Построим простейшую базовую модель без настоящего ML и предложим ее клиенту. Например, система может распознавать голову человека с помощью готового детектора лиц, затем обрезать изображение по лицу и с помощью простейшего алгоритма определять наличие каски (например, искать яркое замкнутое пятно над головой).
- *Подход 6*. Вместо базовой модели интегрируем готовое решение от крупного поставщика, если таковое имеется. Такой подход особенно эффективен, если юридически вам разрешено повторно использовать метки поставщика. Кроме того, проксирование вывода поставщика может дать вам исходные неразмеченные данные, что тоже полезно.

Данные примеры не всегда разумны и применимы конкретно к вашей задаче, но они иллюстрируют возможные способы решения проблемы:

- генерация синтетических данных;
- использование доступных схожих данных;
- создание данных вручную;
- адаптация данных из похожей задачи;
- использование простейшей базовой модели или стороннего решения для запуска процесса.

Следует отметить, что не каждый сценарий применим к любой проблеме. Очевидно, что вы не сможете построить медицинскую систему для выявления рака легких, используя изображения мозга, и базовая модель в роли медицинского консультанта совершенно неуместна. Но обучение на снимках из других клиник, использующих то же самое оборудование, — один из возможных вариантов: несмотря на различия в калибровке сканеров, в совокупности их данные довольно универсальны (модель, обученная на данных из клиник A, B и C, скорее всего, будет применима и в клинике D). Парсинг данных с других сайтов редко допустим в уважаемых компаниях, но в стартапах — довольно распространенный подход. Сгенерированные изображения, как правило, редко подходят, так

как они недостаточно реалистичны (проблемы с освещением, тенями и т. д.). Однако при должной степени упорства можно добиться их реалистичности. В статье «MobilePose: Real-Time Pose Estimation for Unseen Objects with Weak Shape Supervision» (Hou и др., 2020) (https://arxiv.org/abs/2003.03522) исследователи рендерили объекты поверх записей дополненной реальности (с точно рассчитанным освещением), благодаря чему изображения стали гораздо более реалистичными и, следовательно, более ценными для модели.

В каждом из рассмотренных сценариев важно помнить одно: имеющиеся данные не представляют собой точный образец того распределения, с которым система будет работать в реальности. Поэтому результаты валидации следует воспринимать с осторожностью и стремиться как можно скорее заменить прокси-датасеты более реалистичными данными. Мы подробнее поговорим об этом далее, при обсуждении критериев устойчивого пайплайна данных.

6.6. Критерий устойчивого пайплайна данных

Сбор и предварительная обработка данных должны соответствовать трем важным критериям:

- воспроизводимость (reproducibility);
- согласованность (consistency);
- доступность (availability).

Рассмотрим каждый критерий отдельно.

Воспроизводимость означает, что вы имеете возможность при необходимости воссоздать датасет с нуля. Не должно быть некого «золотого» файла с данными, созданного одним-единственным гениальным инженером с применением черной магии. Вместо этого должен существовать программный инструмент (будь то простой скрипт, более сложный пайплайн или целая система), позволяющий заново сформировать все файлы на основе прежнего источника данных. Этот инструмент должен быть настолько хорошо задокументирован и протестирован, чтобы любой участник проекта мог его запустить.

Аргументация в пользу принципа воспроизводимости схожа с подходом «инфраструктура как код» (IaS, infrastructure-as-code), распространенным в DevOps-среде. Да, создать начальный датасет вручную проще, но в долгосрочной перспективе такое решение немасштабируемо и подвержено ошибкам. При добавлении новых данных можно забыть запустить этап предварительной обработки, который приведет к несогласованности данных — трудноуловимой проблеме, способной существенно повлиять на производительность всей системы.

Согласованность — это ключевой момент в построении ML-системы. Нередки случаи, когда метки заданы нестрого и провести четкую границу между классами

невозможно. В таких ситуациях разметчики (эксперты, размечающие данные) могут расходиться во мнениях.

Показательный пример из практики — классификация транзакций по кредитным картам. Ожидается, что клиенты будут присваивать каждой транзакции метку, описывающую категорию расхода. Первоначальная таксономия меток включает следующие категории: «еда и напитки», «бары и винодельни», «кофейни». Возникает вопрос: какая метка лучше всего подходит для оплаты кофе и бутерброда, заказанных в винном баре? Частичное решение в таком случае — создание четкого протокола разрешения спорных ситуаций (например: «если тип продавца и тип товара конфликтуют, приоритет имеет первый по порядку»). Такой подход существенно упрощает как обучение модели, так и валидацию.

Согласованность касается не только разметки. Все аспекты данных должны быть согласованы — их источник, способы предварительной обработки, применяемые фильтры и т. д. Обеспечить это относительно просто, если система разрабатывается в небольшой компании, и значительно сложнее — в крупной международной корпорации. В таких случаях стоит использовать официальный перечень определений. Если возникает подозрение, что ключевой термин, используемый в пайплайне данных, может быть неправильно истолкован, его необходимо зафиксировать в дизайн-документе — это поможет всей команде оставаться на одной волне.

Еще один аспект согласованности — способ сбора данных на этапе построения системы и формат входных данных, поступающих в систему при ее использовании. Это распространенная проблема, которая в ML называется несовпадением распределений (distribution mismatch) и влияет на производительность и справедливость модели. Такая проблема, как правило, проявляется только после реального запуска системы, заранее ее выявить трудно.

Ранее мы уже упоминали феномен под названием «петля обратной связи» — когда модель влияет на источник данных и тем самым нарушает согласованность. Мы также описывали ситуации, когда данные становятся доступны только после запуска системы, — и это также может привести к рассогласованию. Подобные проблемы остаются одними из самых сложных аспектов прикладного ML, которые нельзя игнорировать при проектировании системы.

Согласованность данных — это вопрос, требующий внимания на всех этапах разработки ML-системы — от первых набросков до долгосрочного сопровождения. Мы вернемся к этой теме в главе 14, когда речь пойдет о мониторинге и обнаружении дрейфа (там даже будет короткая, но довольно интересная байка от одного из наших друзей).

Наконец, последнее, но не менее важное свойство — доступность. Данный критерий на самом деле объединяет в себе два момента — доступность для системы и доступность для инженеров. Первый аспект можно также назвать *надежностью* (reliability): архитектор системы должен быть осторожен с ненадежными

источниками данных, иначе все может пойти наперекосяк. Например, представим систему, которая зависит от стороннего API, обогащающего поток входящих данных. Все шло хорошо, пока компания, предоставляющая API, не потеряла своего главного инженера по надежности, вследствие чего перестала поддерживать прежнее состояние инфраструктуры. Если ваша система критически зависит от этого источника, их проблемы становятся вашими.

Конечно, это не означает, что нельзя использовать сторонние API. Как уже упоминалось ранее, внешние решения могут быть полезны — и не только от гигантов вроде Amazon, Google или Microsoft. Но доступность данных — крайне важный фактор, и поэтому вам необходимо учитывать все связанные с этим риски. Конечно, допустима относительная ненадежность некоторых сервисов от поставщиков (например, сбой в экспериментальном визуализаторе вряд ли станет большой проблемой), но не в случае с источниками данных.

Тот же самый принцип применим и к внутренним системам (например, тем, над которыми работают другие команды). Для внешних и внутренних систем применяются разные механизмы контроля: в первом случае это соглашения об уровне сервиса (SLA), во втором — зрелость организационной структуры. При этом в небольших компаниях зачастую легче установить контакт с командой, поддерживающей нужную систему, и тем самым эффективно снизить риски.

Стоит также отметить, что проблемы, связанные с доступностью данных, не всегда обусловлены исключительно техническими причинами. Даже при корректной разработке и сопровождении всех систем могут возникать сложности более высокого уровня, например, связанные с требованиями информационной безопасности и соблюдением правил конфиденциальности (правовых норм, ограничивающих использование персональных данных, или решений генерального директора ключевого клиента о том, что их данные больше не могут покидать их инфраструктуру).

Доступность для инженеров иногда недооценивают. Например, можно услышать что-то вроде: «Да ладно, наши инженеры сами достанут нужные данные, даже если это непросто, — они же профессионалы и умеют справляться с техническими сложностями». И скорее всего, это правда: конечно, наши читатели и их коллеги — отличные специалисты, но нельзя совершенно игнорировать временны́е ограничения.

Представьте, что инженер пошел на обед с коллегой из другой команды, они разговорились о рабочих вопросах, и за чашкой кофе родилась новая гипотеза, связанная с интересующей их системой. Если доступ к данным открыт, можно оперативно принять обоснованное решение, просто загрузив датасет, собрав статистику и, возможно, проведя какие-то базовые эксперименты. Если гипотеза подтверждается уже на первом этапе, она может получить более высокий приоритет, и на нее будут выделены дополнительные ресурсы. Кто знает — возможно, это приведет к значительному улучшению в будущем.

С другой стороны, если получение данных и принятие решений на их основе занимает много времени, гипотеза, скорее всего, будет проигнорирована («Да, любопытно, но у меня и так много задач, а на поиск нужных данных уйдет целая вечность!»). Чтобы избежать подобных ситуаций, мы рекомендуем выделить часть ресурсов на создание инструментов, которые сделают датасеты более доступными для специалистов, работающих с ними. Конечно, доступность важна для любых инструментов, повышающих продуктивность, но ставки особенно высоки, когда речь идет о данных. Как мы говорили в самом начале главы, трудно переоценить значение качественных данных для ML-систем, и потому улучшение взаимодействия с ними — это долгосрочная и оправданная инвестиция.

На данный момент рассмотренной информации о согласованности и доступности достаточно. Мы вернемся к этим двум критериям пайплайна позже, в главе 10. А сейчас пришло время перейти к практической части главы — дизайн-документу.

6.7. Дизайн-документ: датасет

В ходе анализа данных естественным образом возникают дополнительные вопросы, на которые необходимо ответить в дизайн-документе. Ниже приведен перечень вопросов, которые мы рекомендуем задать себе на данном этапе:

- ETL:
  - Каковы источники данных?
  - Как мы будем представлять и хранить данные для нашей системы?
- Фильтрация:
  - Каковы критерии хороших и плохих семплов?
  - Какие крайние/граничные случаи мы ожидаем? Как мы будем с ними справляться?
  - Будем ли мы фильтровать данные автоматически или настроим процесс ручной проверки?
- Генерация признаков:
  - Как вычисляются признаки?
  - Как формируются представления?
- Разметка:
  - Какие метки нам нужны?
  - Каков источник меток?

Как часто бывает, ответы на эти вопросы могут породить новые. Обдумайте и их. Время, потраченное на все вопросы, касающиеся данных, всегда окупается при проектировании ML-систем.

6.7.1. Датасет для Supermegaretail

Теперь вернемся к подготовке дизайн-документа. На этот раз мы оформим раздел, посвященный датасетам для наших воображаемых компаний. Как обычно, начнем с Supermegaretail.

ДИЗАЙН-ДОКУМЕНТ: SUPERMEGARETAIL

III. ДАТАСЕТ

Атомарной единицей датасета является бандл (дата, товар, магазин), а целевая переменная, которую мы хотим спрогнозировать, — это количество проданных единиц товара.

1. Источники данных

Существует множество источников данных, которые мы можем использовать для достижения поставленной цели.

ВНУТРЕННИЕ ИСТОЧНИКИ

- *История покупок (то есть история транзакций)*, собираемая по всей сети магазинов Supermegaretail и сохраняемая в централизованной базе данных. Это наш основной источник — количество продаж, потраченная сумма, примененные скидки, ID транзакции и т. д.
- *История складских остатков* — второй по важности источник для нашей задачи, поскольку так мы определим, сколько единиц каждого товара может быть продано в конкретном магазине. Данные позволяют оценить, сколько товаров было доступно для продажи в начале каждого дня и сколько из них было списано по истечении срока годности.
- Метаданные по *каждому товару, магазину и транзакции*.
- Календарь запланированных *промоакций*. Важный фактор, существенно влияющий на будущие продажи, который обязательно должен учитываться.

ВНЕШНИЕ ИСТОЧНИКИ: ДАННЫЕ, СОБИРАЕМЫЕ ВРУЧНУЮ

- *Мониторинг цен*. Цены и другая информация о товарах, собираемая у конкурентов. Эти данные вручную собираются ежедневно в выбранных магазинах разных конкурентов. Сбор может осуществляться как внутренней командой, так и сторонней организацией (аутсорс). Возможен также гибридный подход. Каждый товар должен содержать глобальный идентификатор (штрихкод), что позволяет легко сопоставлять собранные данные с нашими товарами. Знание средних цен конкурентов и динамики их изменения позволит нам понять ситуацию на рынке.

ВНЕШНИЕ ИСТОЧНИКИ: ПРИОБРЕТЕННЫЕ ДАННЫЕ

- *История погодных условий и прогноз*, покупаемые у метеорологической службы. Погода — важный фактор, напрямую влияющий на поведение потребителей.

- Оценка *потока клиентов* возле наших магазинов (от телеком-провайдеров).
- *Глобальные рыночные индикаторы*.

ДАННЫЕ МОБИЛЬНОГО ПРИЛОЖЕНИЯ И САЙТА (ОПЦИОНАЛЬНО)

- У Supermegaretail есть *служба доставки* (даже если она приносит менее 5 % выручки). Мы собираем *дополнительные данные* о конкретных продажах по конкретным локациям. Иногда эта информация может стать ценным показателем.
- Мобильные и веб-сервисы также собирают информацию о действиях пользователей, включая просмотры, клики и добавления в корзину, — все это тоже поможет спрогнозировать офлайн-продажи.

2. Разметка данных

Поскольку мы решаем задачу прогнозирования спроса, нам не требуется дополнительная разметка данных — все извлекается напрямую из истории транзакций.

3. Доступные метаданные

Мы прогнозируем спрос на уровне SKU в каждом магазине, используя три ключевых элемента: товары, магазины и транзакции.

ТОВАРЫ

- ID товара и штрихкод.
- Коды категорий разных уровней (1, 2, 3). Иерархия категорий может быть использована для определения степени сходства между товарами на основе ряда правил. Также используется дополнительная информация — бренд, производитель и ценовая группа.
- Срок хранения, определяющий степень критичности избытка данного товара.
- Дата добавления товара в ассортиментную матрицу сети.
- Габариты и вес товара.

МАГАЗИНЫ

- ID магазина.
- Местоположение (координаты) с возможностью интеграции сторонних источников — мы можем использовать эти данные для добавления информации о погоде, потоке людей, расстоянии до ключевых объектов, а также других параметров, таких как город, регион и связанный логистический центр.
- Ближайшие магазины конкурентов (с указанием их ID и расстояний до них).

- Размер магазина и его формат. Данные параметры определяют, какие товары и в каком количестве (по уникальным позициям) будут представлены в ассортименте конкретного магазина.
- Даты, когда магазин открыт и закрыт.

ТРАНЗАКЦИИ

- *Временна́я метка*. Позволяет дополнить датасет информацией, например, о праздничных днях.
- *ID клиента* (если применялась карта лояльности). Несмотря на то что базовая единица датасета — (товар, магазин), бандл (клиент, товар) может быть использован в отдельном пайплайне для расчета эмбеддингов товаров через агрегацию транзакций в матрицу «пользователь–товар» с последующей факторизацией. Такие эмбеддинги содержат паттерны покупательского поведения.
- ID магазина и ID товара.

4. Доступная история

Прогнозирование спроса — не новая задача для Supermegaretail. Критически важные процессы ETL уже реализованы. Supermegaretail собирает данные уже более трех лет.

Данная информация крайне важна для обучения модели прогнозирования: она позволяет выявить закономерности, уловить сезонные колебания продаж, оценить тренды и т. п. История по метаданным товаров и магазинов также доступна. Данные о погоде (получаемые из внешних источников) охватывают весь необходимый нам период в прошлом.

История складских остатков и проведения промоакций также собиралась.

Информация о ценах конкурентов мониторится последние два года.

5. Проблемы качества данных

Данные по транзакциям, остаткам и промоакциям могут содержать пропущенные или дублированные значения, поэтому перед агрегацией необходимо провести дополнительную фильтрацию или предобработку.

Приобретаемые внешние данные уже прошли предварительную очистку и контроль качества перед передачей, однако все равно требуется провести ряд проверок.

Данные о ценах конкурентов покрывают около 25 % ассортимента SKU и имеют пропуски.

6. Итоговый etl-пайплайн

Общая схема выглядит следующим образом:

1. Данные транзакций агрегируются на ежедневной основе.

2. Новая агрегированная порция данных добавляется в таблицу агрегатов транзакций.
3. (Опционально.) Мы перезаписываем не только данные за последний день, но и за предыдущие 2–3 дня, чтобы устранить возможные искажения (дубликаты, неполные данные, повторяющиеся записи и т. д.).
4. Мы объединяем остальные внутренние и внешние источники данных по дате, ID товара или ID магазина.
5. Наконец, мы рассчитываем признаки на основе объединенного датасета.

Дополнительно можно реализовать отдельный пайплайн для расчета эмбеддингов товаров, как описано в разделе 3.

6.7.2. Датасет для PhotoStock Inc.

Теперь перейдем к дизайн-документу компании PhotoStock Inc.

ДИЗАЙН-ДОКУМЕНТ: PHOTOSTOCK INC.

III. ДАТАСЕТ

1. Датасет и источники

Одним из потенциальных источников данных, которые можно использовать для построения поисковой системы PhotoStock Inc., являются данные, связанные с каждой фотографией в фотостоке. Они включают в себя теги, метки и описания изображений, что предоставляет ценную информацию о содержании фотографий. Датасет также должен содержать URL-адреса самих изображений и URL-адреса используемых миниатюр. Предлагаем назвать его «Датасет описаний».

Другим потенциальным источником данных являются поисковые запросы, которые пользователи вводили на платформе PhotoStock Inc. Эти запросы дают представление о том, какие фотографии ищут пользователи, и могут быть использованы для разработки ML-моделей. В сочетании с кликами пользователей они дают представление о релевантности изображений. Расширенная версия данного датасета может включать дополнительную информацию о сессиях: сколько времени пользователь провел на странице с изображением и была ли затем совершена покупка. Предлагаем назвать этот датасет «Датасет кликов».

Дополнительно можно нанять разметчиков для оценки релевантности пар «запрос–изображение». Разметчикам будет предоставлен набор поисковых запросов и выборка изображений из библиотеки PhotoStock Inc., и они должны будут присвоить каждой паре оценку релевантности на основе того, насколько изображение соответствует содержанию запроса. Первоначально пары запро-

сов и изображений будут выбираться случайным образом из пула прошлых пользовательских запросов и доступных фотографий в библиотеке, однако впоследствии этот процесс может быть улучшен с использованием методов активного обучения и разметки тех пар, в которых модель менее уверена или чаще ошибается. Предлагаем назвать этот датасет «Датасет меток».

Также можно рассмотреть использование ряда открытых текстово-визуальных датасетов (например, COCO — Microsoft Common Objects in Context) для начального этапа обучения модели. Однако, учитывая наличие истории пользовательских взаимодействий, разумнее использовать собственные данные, в то время как открытые датасеты будут использоваться лишь косвенно — для раннего обучения моделей.

Датасеты описаний и кликов формируются естественным образом в результате основной бизнес-деятельности PhotoStock Inc., поэтому на данном этапе они не требуют серьезного внимания, за исключением построения корректного ETL-процесса и системы хранения. Для датасета меток потребуются отдельные средства для найма команды разметчиков или привлечение стороннего сервиса.

2. Метаданные, фильтрация и подвыборка

Ожидается, что датасет описаний будет достаточно чистым, поскольку описания фотографий модерируются командой по контролю качества контента, а их качество уже является ключевой составляющей нашего продукта. Датасет кликов, напротив, будет зашумленным из-за различий в пользовательском поведении и ботов-скреперов, поэтому необходимо будет разработать способы обработки этих проблем. Расширенная версия датасета, содержащая данные о покупках, потенциально будет менее зашумленной, но значительно меньшей по объему, учитывая низкий коэффициент конверсии от клика к покупке. Учитывая это, мы ожидаем, что датасеты меток и кликов с покупками будут более ценными для валидации, в то время как датасеты описаний и кликов без покупок, несмотря на возможный шум, будут более пригодны для обучения.

Для устранения проблем с качеством данных при фильтрации необходимо уделить внимание метаданным. Датасет описаний должен содержать информацию о продавце и датах изменений, а датасет кликов — информацию о пользователях, продавце изображения, по которому был совершен клик, а также дополнительную информацию о сессии поиска. На данном этапе нет необходимости хранить все данные о пользователях и продавцах, это может быть реализовано на следующих этапах генерации признаков, сейчас мы должны знать соответствующие `user_id`, `seller_id`, `session_id` и `purchase_id`, чтобы в дальнейшем иметь возможность объединить эти данные.

Учитывая объем поисковых запросов и покупок на платформе PhotoStock Inc., маловероятно, что нам потребуется дополнительная подвыборка для решения инженерных задач, — датасеты можно обрабатывать целиком. Однако может возникнуть необходимость в выполнении подвыборки для балансировки

классов — изображений, на которых кликают, значительно больше, чем тех, которые покупают, а изображений, отображаемых в результатах поиска, больше, чем тех, на которых кликают, и т. д.

3. ETL и подготовка данных

Обучающие данные должны извлекаться на регулярной (пока что ежедневной) основе в пакетном режиме. Мы предлагаем использовать фреймворк Flyte для оркестрации задач, поскольку он уже используется для других пакетных задач в PhotoStock Inc.

На данном этапе не требуется никаких сложных решений — достаточно собирать данные из рабочих баз данных и сохранять их отдельно в формате файлов Parquet для упрощенного чтения. Пока что нет необходимости в сложной предварительной обработке данных.

4. Разметка

На данный момент платформа для ручной разметки не определена, также остается открытым вопрос, следует ли использовать внутреннюю команду (например, команду поддержки или модераторов) или привлечь сторонний сервис. Этот вопрос необходимо обсудить совместно с менеджерами соответствующих команд. Также следует запросить расчет стоимости у некоторых сторонних сервисов, чтобы сравнить стоимость внутренней разметки с внешней.

Нам необходимо размечать пары (запрос, изображение), распределяя их по трем классам:

- релевантно;
- нерелевантно;
- не могу оценить.

Возможно, в будущем потребуется разделить класс «Релевантно» на «Абсолютно релевантно» и «Частично релевантно» для большей детализации, то же самое и с классом «Нерелевантно». Что касается класса «Не могу оценить», разметчики должны указывать причину, по которой они не могут провести разметку, — первые итерации процесса дадут нам дополнительную информацию о датасете.

Итоги

- Не ограничивайтесь только одним источником данных. Определите, достаточно ли у вас внутренних источников или есть необходимость выйти за пределы вашей компании.
- Доступ к уникальным датасетам может дать значительное конкурентное преимущество, но и открытые датасеты могут быть полезны, если их правильно подготовить.

- При работе с датасетами не пренебрегайте метаданными — они критически важны для построения потоков данных и обеспечения их согласованности.
- Помните, что не существует строгой последовательности техник при подготовке датасета. Итоговый порядок операций будет зависеть от бизнес-целей вашей системы, области применения и прочих факторов.
- Избегайте ситуаций, когда датасет заполняется семплами, аналогичными уже имеющимся. Вместо этого включайте в него те данные, на которых модель ранее ошибалась.
- Убедитесь, что у вас есть данные, которые можно дать системе перед ее запуском.
- Помните, что пайплайн данных должен соответствовать трем критериям: воспроизводимость, согласованность и доступность.

7 Схемы валидации

В этой главе

- ✓ Достоверная оценка модели
- ✓ Стандартные схемы валидации
- ✓ Нетривиальные схемы валидации
- ✓ Процедура раздельного обновления
- ✓ Схемы валидации в дизайн-документе

Построение правдивого и достоверного процесса оценки модели критически важно для системы машинного обучения, и в этой главе мы рассмотрим процесс создания правильной схемы валидации для получения точных оценок производительности системы. Мы проанализируем наиболее распространенные схемы валидации, а также обсудим, как выбрать подходящую вам схему в зависимости от решаемой проблемы и какие факторы необходимо учитывать при проектировании процесса оценки в реальных условиях.

Правильная процедура валидации должна показывать, какими знаниями мы должны обладать и какими знаниями можно пренебречь при реальной работе системы. Данный вопрос частично связан с проблемой переобучения (overfitting) или генерализации, о которых мы подробно поговорим в главе 9.

Процедура валидации обеспечивает надежную и достоверную оценку производительности системы, в идеале — с некоторыми теоретическими гарантиями. Например, можно гарантировать, что реальное значение метрик будет находиться

в диапазоне между нижней и верхней границами доверительного интервала в 95 случаях из 100 (что мы обсудим чуть позже в байке от Валерия). Кроме того, правильная валидация помогает выявить и предотвратить утечки данных, переобучение и расхождение между производительностью в офлайн- и онлайн-режимах.

Основная цель валидации — это оценка производительности. Мы используем данный процесс для оценки прогностической способности модели на незнакомых ей данных, и предпочтение обычно отдается той схеме, которая обладает наибольшей надежностью и устойчивостью (то есть минимальным смещением и дисперсией).

Мы можем использовать оценку производительности в различных целях, например, в оптимизации гиперпараметров, выборе архитектуры, алгоритма или признаков. В некоторой степени это похоже на A/B-тестирование, где схема с меньшей дисперсией обеспечивает большую чувствительность, что мы также рассмотрим в этой главе.

7.1. Достоверная оценка

Для валидации системы нам необходимо построить стабильный и надежный пайплайн, который будет давать воспроизводимые результаты (рис. 7.1). Все стандартные рекомендации, которые вы наверняка найдете в литературе по теме, сводятся к следующему: необходимо разделить все данные на обучающую, валидационную и тестовую части. Обучающая часть используется для обучения модели, валидационная часть предназначена для оценки производительности в процессе обучения, а тестовая — для расчета итоговых метрик. Такой подход хорошо знаком тем, кто принимал участие в соревнованиях по ML (например, на платформе Kaggle) или работал в академической среде. В то же время в прикладном ML существуют тонкие, но важные отличия, которые мы рассмотрим далее.

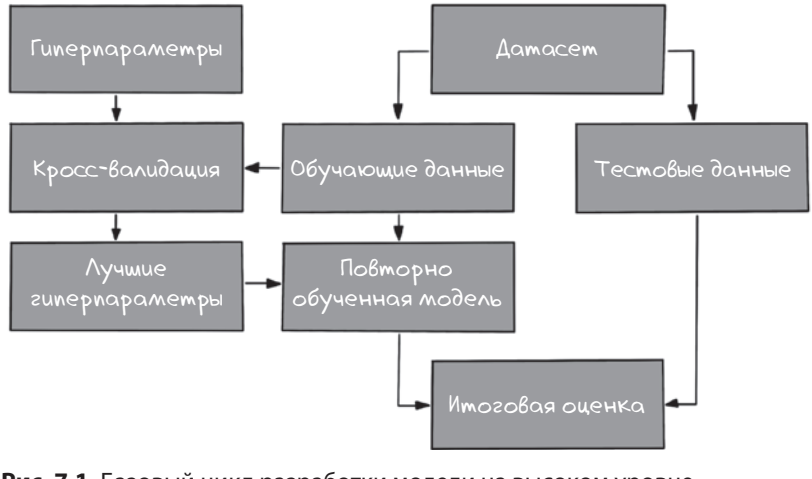


Рис. 7.1. Базовый цикл разработки модели на высоком уровне

На что стоит обратить внимание:

- Простое разделение данных на обучающую, валидационную и тестовую части предполагает, что все три датасета имеют одно и то же распределение, и так это останется и в будущем. Данное правило требует постоянных проверок, и если оно нарушается, то никакой гарантии будущей производительности системы нет.
- Для оценки производительности модели необходимо неоднократно использовать валидационную выборку. Переоценка производительности на основе валидационной выборки приводит к смещению и переобучению относительно этой выборки. Оптимизация гиперпараметров, отбор признаков или выбор модели — это все часть процесса обучения. Следовательно, можно предположить, что то же самое можно сказать и о тестовой выборке.

Именно поэтому использование одного и того же разделения для оценки производительности и для поиска оптимальных гиперпараметров или других настроек приводит к смещенным/переобученным и нестабильным результатам. По этой причине валидацию необходимо воспринимать не как разовую операцию в начале проекта, а как непрерывный процесс, который повторяется каждый раз при изменении контекста системы (например, при появлении новых источников данных, новых признаков, возможных петель обратной связи, вызванных использованием модели, и т. д.).

Мы никогда не можем на сто процентов знать, что произойдет в будущем, поэтому должны быть готовы к неожиданностям.

7.2. Стандартные схемы

Как показывает практика, при выборе схемы валидации для ML-системы не нужно изобретать велосипед. Большинство стандартных схем — это проверенные временем и хорошо зарекомендовавшие себя решения, среди которых просто нужно выбрать ту, что соответствует требованиям вашего проекта. Мы кратко рассмотрим наиболее распространенные схемы далее.

Классические схемы валидации представлены во всеми любимой библиотеке ML для Python — Scikit-learn, и, если вы сомневаетесь в своих знаниях, стоит с ней ознакомиться. Информация доступна по ссылке https://mng.bz/aV6B.

7.2.1. Отложенная выборка

Начнем с разделения датасета на две или более части. Наиболее распространенная рекомендация, которую вы найдете практически в каждой книге, посвященной ML, — это разделение данных на обучающую, валидационную и тестовую части, как мы уже упоминали ранее.

При таком подходе данные делятся на три выборки (случайным образом или на основе определенного критерия или стратификации) в различных пропорциях, например, 60/20/20 (рис. 7.2). Процентное соотношение может варьироваться в зависимости от числа семплов и метрик (объема данных, дисперсии метрик, требований к чувствительности, надежности и устойчивости). Логично, что чем больше датасет, тем меньшая его доля выделяется на валидацию и тестирование, поэтому обучающая выборка растет быстрее. Тестовая выборка (то есть внешняя валидация) используется только в итоговой оценке модели и никогда не должна использоваться для каких-то других целей, тогда как валидационная выборка (внутренняя валидация) используется в первую очередь для сравнения моделей или настройки гиперпараметров.

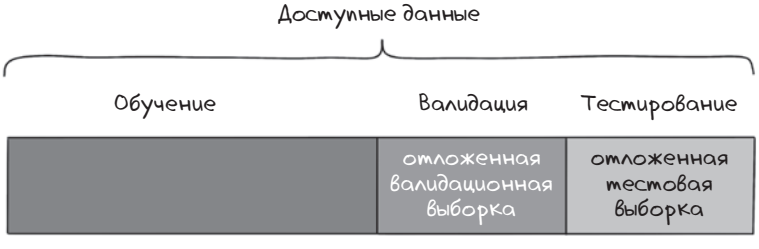


Рис. 7.2. Стандартное разбиение данных «по инструкции»

7.2.2. Кросс-валидация

Отложенная валидация хорошо подходит для затратных с точки зрения вычислений моделей, например моделей глубокого обучения, ее легко реализовать, и она практически не увеличивает время обучения.

Но стоит помнить, что мы берем только лишь одну случайную подвыборку всех данных. Мы не используем повторно все доступные данные, что может привести к смещенной оценке или недоиспользованию доступной информации. Что еще хуже, мы получаем всего одно число, которое не позволяет понять распределение оценок.

Универсальным решением подобной проблемы в статистике является бутстрэп-процедура. В случае валидации это многократное случайное разделение на обучающие и валидационные выборки, то есть обучение и оценка модели на каждой итерации. Поскольку обучение модели занимает много времени, а нам нужно скорее проводить настройку параметров и эксперименты, возникает вопрос: как это сделать более эффективно?

Можно использовать похожую, но упрощенную процедуру семплирования, называемую *кросс-валидацией*. В таком случае мы делим данные на K блоков — «фолдов» (обычно пять), а затем последовательно исключаем каждый из них, обучая модель на оставшихся $K - 1$ фолдах, и оцениваем производительность на

исключенном фолде. Таким образом, мы получаем *K* оценок и можем вычислить их среднее значение и отклонение. В результате вместо одного у нас целых пять чисел, что дает более репрезентативную картину (рис. 7.3).

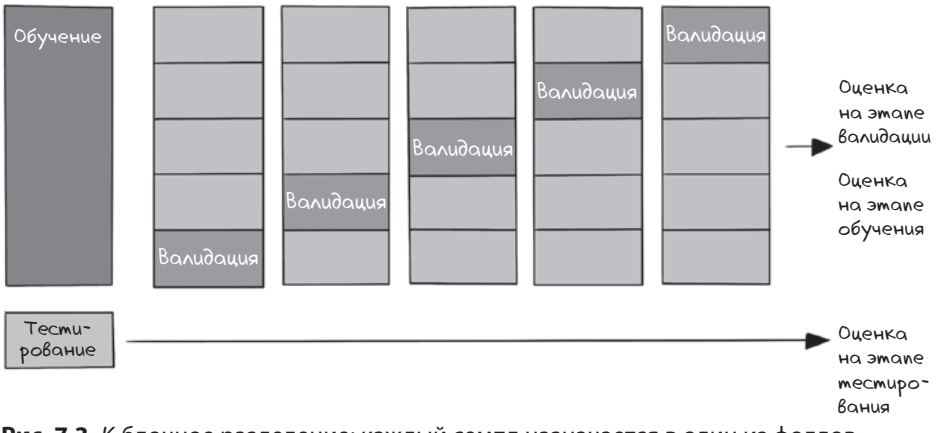


Рис. 7.3. *K*-блочное разделение: каждый семпл назначается в один из фолдов, каждый фолд используется для валидации лишь единожды, в остальных раундах он используется для обучения

Существует несколько вариантов кросс-валидации, включая:

- *стратифицированную* (необходимо сохранять баланс классов);
- *повторную* (делим данные на *K* фолдов *N* раз, так что каждый объект участвует в оценке *N* раз);
- *групповую* (при схожести объектов внутри группы нужно избежать утечки, поэтому вся группа должна полностью входить либо в обучающую выборку, либо в валидационную).

Предположим, мы прогнозируем скорость потока нефти на сотнях скважин. Скважины сгруппированы по местоположению: соседние скважины добывают нефть из одного и того же месторождения, поэтому их производительность взаимосвязана. В таком случае разумно выбрать групповую K-блочную кросс-валидацию. Поиск правильного критерия для группировки семплов при их распределении по фолдам является одним из ключевых решений для валидации в целом, и ошибки на этом этапе существенно влияют на результат.

7.2.3. Выбор значения K

Единственный оставшийся вопрос — какое количество фолдов выбрать. Определение *K* зависит от трех переменных — смещения, дисперсии и времени вычислений. Правило гласит: использовать $K = 5$, это обеспечит хороший баланс между смещением и дисперсией.

Крайний случай для K — это кросс-валидация с поочередным исключением (leave-one-out), когда каждый фолд содержит всего один семпл, следовательно, K равно общему числу семплов в датасете. Такая схема показывает худшие результаты по времени вычислений и дисперсии, но зато наилучшие по смещению.

В статье Рона Кохави (Ron Kohavi) 1995 года под названием «A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection» (https://mng.bz/4pn5) приводятся следующие рекомендации:

- Увеличение числа фолдов снижает смещение и улучшает оценку производительности.
- В то же время дисперсия растет вместе с числом фолдов из-за уменьшения количества семплов в каждом валидационном фолде (оценки становятся слишком шумными). При допущении постоянного смещения чувствительность схемы валидации определяется дисперсией.
- Использование повторной кросс-валидации с $K = 2$ или $K = 3$, повторенной 10–20 раз, позволяет сравнить модели, однако для оптимизации смещения повторная K-блочная кросс-валидация бесполезна, поскольку оценки между разными повторами уже имеют согласованное смещение.
- С увеличением объема датасета требуется меньшее количество фолдов. Чем больше данных в каждом фолде, тем он более репрезентативен.
- Для более простых моделей (например, при работе с базовыми решениями) и хорошо подготовленных датасетов увеличение числа фолдов приводит к снижению как смещения, так и дисперсии.

Важно помнить, что высокая чувствительность схемы валидации (то есть низкая дисперсия) имеет значение только тогда, когда изменения в производительности модели, которые мы пытаемся уловить, незначительны.

7.2.4. Валидация временных рядов

При работе с данными, чувствительными ко времени, нельзя отбирать семплы случайным образом. Продажи товаров в соседние дни связаны между собой. Аналогично, недавние действия пользователей могут помочь спрогнозировать их будущее поведение. Однако мы не можем предсказывать прошлое, основываясь на данных из будущего. Во временны́х рядах распределение закономерностей по датасету неравномерно, поэтому для них необходимо использовать другие схемы валидации. Как же тогда оценивать модель?

Схемы валидации временных рядов схожи с валидацией на отложенной выборке и кросс-валидацией, но при этом с неслучайным разделением по временной метке. Рекомендации по выбору количества фолдов и их размера при скользящей кросс-валидации аналогичны обычной K-блочной валидации.

При проведении валидации временных рядов необходимо учитывать дополнительные степени свободы, которые она предоставляет. В статье «Evaluating Time Series Forecasting Models» (Cerqueira и др.) (https://arxiv.org/pdf/1905.11744.pdf) подробно рассматриваются следующие моменты:

- *Размер окна*. Размер тестовой выборки должен отражать, насколько далеко вперед осуществляется прогноз и как долго модель будет эксплуатироваться без повторного обучения.
- *Размер обучающей выборки*. Существует два подхода к объему данных для обучения: использовать всю доступную историю или же ограничить обучение одним-двумя недавними периодами (неделями, месяцами или годами, в зависимости от сезонности), отбрасывая более старые нерелевантные данные.
- *Сезонность*. В данных могут прослеживаться определенные паттерны, зависящие от циклов дней, недель, месяцев, кварталов или лет. Необходимо определять размеры обучающих и тестовых выборок таким образом, чтобы захватывать эти закономерности. Например, для учета годовой сезонности обучающая выборка должна включать как минимум 2 года истории. Другой пример — недельная сезонность в тестовой выборке: чтобы минимизировать дисперсию между фолдами, каждый из них должен содержать одни и те же дни недели (то есть полную неделю в каждом фолде).
- *Интервал*. Между обучающей и тестовой выборками может существовать интервал, у которого две цели: во-первых, подготовить модель к задержке в получении новых данных (что ведет к задержке в признаках), а во-вторых, сделать обучающие и тестовые данные менее взаимосвязанными, тем самым минимизируя риск утечки данных. Например, можно пропустить 2–3 дня между обучающей и тестовой выборками в обоих примерах, приведенных выше.

И хотя валидация временных рядов является одним из наиболее чувствительных к дефектам методов валидации, полагаться исключительно на правило «не использовать данные из будущего при обучении» было бы недальновидно. Следование этому принципу позволяет избежать 95 % типичных ошибок, но, тем не менее, существуют случаи, когда его необходимо нарушить. Например, ML, применяемое к финансовым данным (скажем, к временным рядам фондового рынка), известно своими строгими требованиями к точности валидации. В то же время некоторые эксперты в этой области отмечают, что базовая схема валидации временных рядов, показанная на рис. 7.4, может привести к переобучению из-за ограниченного объема подмножеств данных. Подробнее об этом см. главу 12 «Backtesting Through Cross-Validation» книги Маркоса Лопеса де Прадо (Marcos Lopez de Prado) «Advances in Financial Machine Learning»[1]. Другая

[1] Прадо М. Л. «Машинное обучение: алгоритмы для бизнеса». СПб., издательство «Питер». Глава 12 называется «Бэктестирование через кросс-валидацию».

причина нарушить это правило — необходимость оценить, как модель будет работать в аномальных сценариях. Для этого модель можно обучить на данных за 2017–2019 годы и 2021–2023 годы, а затем протестировать ее на данных периода COVID 2020 года. Такое разделение не может служить основной схемой валидации и предоставляет лишь вспомогательную информацию.

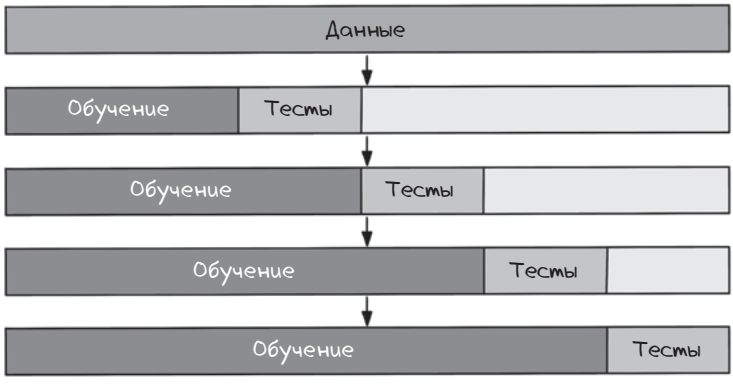


Рис. 7.4. Стандартное разделение по времени. Тестовая выборка всегда следует за обучающей, поэтому обучающие выборки относятся к «прошлому», а тестовые — к «будущему»

В некоторых случаях стоит использовать комбинацию различных схем. В приведенном ранее примере прогноза скорости потока можно объединить групповую *K*-блочную кросс-валидацию с валидацией временных рядов:

```
import numpy as np
from sklearn.model_selection import GroupKFold
```

```
import numpy as np
from sklearn.model_selection import GroupKFold
from sklearn.exceptions import NotFittedError

def grouped_time_series_kfold(model, X, y, groups, n_folds=5,
n_repeats=10, seed=0):
    scores = []
    np.random.seed(seed)
    unique_groups = np.unique(groups)

    for i in range(n_repeats):
        gkf = GroupKFold(n_splits=n_folds)
        shuffled_groups = np.random.permutation(unique_groups)

        for train_group_idx, test_group_idx in gkf.split(X, y,
        groups=shuffled_groups):
            train_groups = shuffled_groups[train_group_idx]
            test_groups = shuffled_groups[test_group_idx]

            # Найти минимальные и максимальные индексы для обучающих
            # и тестовых групп
```

```
        train_indices = np.where(np.isin(groups, train_groups))[0]
        test_indices = np.where(np.isin(groups, test_groups))[0]
        train_end = np.min(test_indices)
        # Обеспечить хронологический порядок
        train_mask = np.isin(groups, train_groups) &
        (np.arange(len(groups)) < train_end)
        test_mask = np.isin(groups, test_groups)

        model.fit(X[train_mask], y[train_mask])
        score = model.score(X[test_mask], y[test_mask])
        scores.append(score)

return np.array(scores)
```

Байка от Валерия

Когда я работал над системой динамического ценообразования в крупном интернет-магазине, перед нами стояла задача построить модель прогноза продаж, которая прогнозировала бы объемы продаж на неделю вперед, а затем обрабатывала свои прогнозы и определяла оптимальные цены.

Изначально для валидации мы использовали данные за предыдущую неделю. По мере появления новых ежедневных данных валидационная неделя сдвигалась на один день вперед. Однако мы заметили, что метрики качества демонстрировали значительные колебания ото дня ко дню, из-за чего становилось сложнее понять, как меняется качество модели на фоне периодического добавления и корректировки признаков, а также изменений в постобработке прогнозов.

Мы хотели разобраться в причинах таких колебаний и вскоре выяснили, что ассортимент товаров менялся на 15 % от недели к неделе и на 40 % от месяца к месяцу. Кроме того, динамика продаж отдельных товаров оказалась крайне неоднородной (например, сегодня продано 10 единиц товара, а в следующие два дня — ни одной). В результате мы наблюдали изменения метрик, вызванные ежедневными обновлениями валидационного набора, а не реальными изменениями качества модели.

Чтобы справиться с этой проблемой, мы внедрили подход «отложенного сдвига» в валидации. Вместо ежедневного обновления валидационного набора мы обновляли его раз в месяц, при этом продолжая использовать данные за одну неделю работы. Это обеспечило относительную свежесть данных (не старше одного месяца) при сохранении фиксированного валидационного набора на протяжении всего месяца. В результате сравнение моделей стало более конструктивным, а метрики качества — гораздо менее шумными.

7.3. Нестандартные схемы

Мы рассмотрели стандартные схемы валидации, подходящие для большинства задач ML. Однако иногда их может быть недостаточно для того, чтобы отразить реальную разницу между видимыми и невидимыми данными, даже если вы используете не одну схему, а какую-то их комбинацию (например, валидацию временных рядов в сочетании с групповой K-блочной кросс-валидацией). Как известно,

ненадлежащая валидация приводит к утечке данных и, как следствие, к слишком оптимистичной оценке производительности модели (если не к случайной!).

В таких ситуациях стоит выбирать более нестандартные процессы. Рассмотрим некоторые из них.

7.3.1. Вложенная валидация

Вложенная (nested) валидация используется в случаях, когда необходимо провести оптимизацию гиперпараметров (или любой другой процесс выбора модели) в ходе обучения. Нельзя просто использовать исключенный фолд или отложенную выборку, которые потребуются нам для финальной оценки качества выбранного набора параметров. Доступ к оценке на тестовых данных во время настройки параметров — это прямой путь к переобучению.

Вместо этого применяется схема «разделение внутри разделения» (fold-in-fold). В каждое «внешнее» разделение обучающих данных мы добавляем «внутреннее» разделение для первоначальной настройки параметров. Затем модель обучается на всех доступных фолдах с выбранными гиперпараметрами и прогнозирует данные, которые не были использованы при настройке. Таким образом, мы получаем два уровня валидации, каждый из которых может иметь свои специфические характеристики (например, мы можем предпочесть, чтобы внутренний уровень имел более низкую дисперсию, а внешний уровень — более низкое смещение). Данный метод можно применять не только к кросс-валидации, но и к валидации временных рядов и порядковому разделению на обучающую и тестовую выборки (или к комбинациям этих схем) (рис. 7.5 и 7.6).

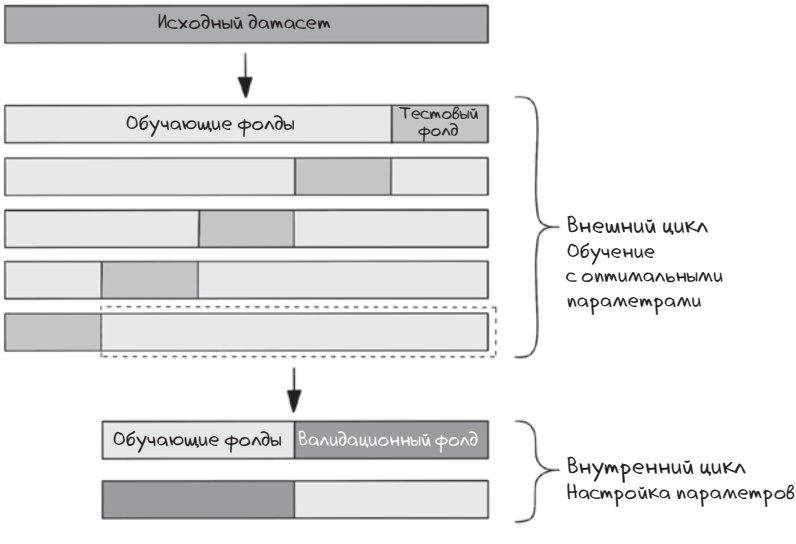


Рис. 7.5. Пример вложенной кросс-валидации

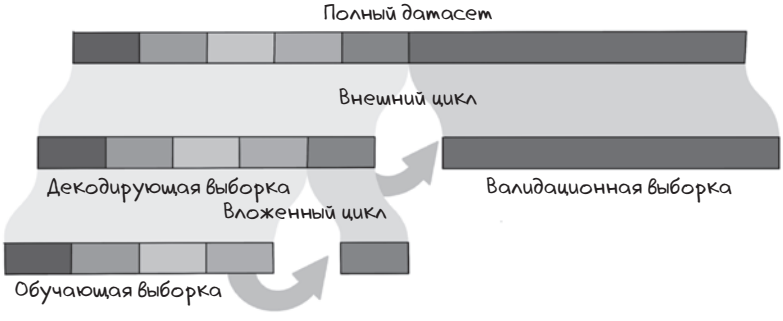


Рис. 7.6. Пример вложенной валидации с комбинацией схем — отложенная выборка во внешнем цикле и *K*-блочная кросс-валидация во внутреннем

7.3.2. Адверсариальная валидация

Вместо использования случайной выборки данных, как это делается в случае с отложенной выборкой, можно пойти другим путем. Существует техника, называемая адверсариальной валидацией (adversarial validation), особенно популярная на таких платформах соревнований по ML, как, например, Kaggle. Она подразумевает использование ML-модели для более эффективной валидации другой ML-модели.

Адверсариальная валидация количественно оценивает, различаются ли два заданных датасета (как размеченные, так и неразмеченные). Если это так, то с ее помощью можно получить количественную оценку на уровне отдельных семплов, что в дальнейшем позволит создавать любое количество взаимопредставительных датасетов, обеспечивая идеальный инструмент для оценки. При этом вы можете использовать и неразмеченные данные, что является еще одним преимуществом этой схемы.

Алгоритм достаточно прост:

1. Объединяем интересующие нас датасеты (отбрасывая целевую переменную, если она есть) и присваиваем опорному датасету (тому, который мы хотим представить) метку 1, а остальным — 0.
2. Обучаем вспомогательную модель на объединенном датасете для решения задачи бинарной классификации (метки 0 и 1).
3. Если датасеты репрезентативны по отношению друг к другу и происходят из одного распределения, можно ожидать, что площадь под кривой характеристик (ROC AUC) будет близка к 0.5. Если же они разделимы (например, ROC AUC больше 0.6), тогда выход модели можно использовать в качестве меры близости.

Обратите внимание, что хотя данный прием давно используется в соревнованиях по ML (первое его упоминание было в 2016 году (http://fastml.com/

adversarial-validation-part-one/)), он не становился предметом научных исследований вплоть до 2020 года, пока не был упомянут в статье Пэна (Pan) и др. «Adversarial Validation Approach to Concept Drift Problem in User Targeting Automation Systems at Uber» (https://arxiv.org/abs/2004.03045).

Такой способ разделения данных имеет широкое применение. При проверке сходства размеченных и неразмеченных датасетов не следует забывать о двух вопросах: насколько различаются их распределения? Какие признаки лучше всего прогнозируют это различие? Анализ модели, построенной с помощью адверсариальной валидации, поможет ответить на эти вопросы. Мы же вновь вернемся к данной технике в главе 9.

7.3.3. Оценка использования утечек данных

Интересную технику валидации можно найти в статье DeepMind «Improving Language Models by Retrieving from Trillions of Tokens» (2021) (https://arxiv.org/abs/2112.04426), в которой описана генеративная модель, обученная предсказывать следующее слово.

Авторы статьи улучшают языковую модель за счет условной генерации на основе контекста, извлеченного из большого корпуса текстов, исходя из его сходства с предыдущими токенами. Такая система запоминает весь датасет и выполняет поиск ближайших соседей, чтобы найти фрагменты текста в истории, релевантные недавним предложениям. Но что, если предложения, которые мы пытаемся продолжить, почти идентичны тем, что модель уже видела на этапе обучения? В таком случае высока вероятность столкнуться с утечкой данных из датасета.

Авторы статьи учли эту проблему и предложили любопытную процедуру оценки. Они разработали специальную метрику для количественной оценки использования утечек.

Основная идея заключается в следующем:

1. Разделить датасет на обучающую и валидационную части, как при стандартной валидации с отложенной выборкой.
2. Разбить обе части на фрагменты («чанки») фиксированной длины.
3. Для каждого чанка из валидационной выборки найти N ближайших соседей в обучающей выборке на основе эмбеддингов фрагментов (детали преобразования чанков в эмбеддинг-пространство можно найти в статье).
4. Вычислить долю схожих токенов между двумя чанками (используется метрика, аналогичная индексу Жаккара), в результате чего мы получим значение от 0 (чанки абсолютно разные) до 1 (чанки идентичны).
5. Если значение превышает заданный порог, исключить этот чанк из обучающей выборки.

Такой подход заставляет модель извлекать полезную информацию из похожих текстов и перефразировать ее, а не просто копировать. Данная процедура работает с любой современной языковой моделью, а поэтому она — отличный пример нетривиальной техники, позволяющей минимизировать утечку данных и повысить репрезентативность датасета. Четкое понимание того, как будет применяться модель, поможет вам разработать собственную схему валидации в случаях, когда стандартных методов недостаточно.

7.4. Процедура раздельного обновления

> Мы тратим столько же времени на тестовые данные, сколько и на обучающие.
>
> *Андрей Карпатый*

Независимо от того, какую схему мы используем, скорее всего, мы применяем ее к динамически изменяющемуся датасету. Периодически мы получаем новые данные, которые могут отличаться по распределению и содержать новые закономерности. Как часто следует обновлять тестовый набор, чтобы оценка оставалась актуальной?

При разработке процедуры раздельного обновления (split update) для новых данных перед нами могут стоять по меньшей мере две цели. Во-первых, мы хотим, чтобы наша тестовая выборка была репрезентативна для новых паттернов, следовательно, процесс валидации должен быть адаптивным.

Во-вторых, мы хотим отслеживать динамику оценки: как модель менялась с течением времени с учетом всех обновлений архитектуры и признаков? Для этого оценки должны быть устойчивыми.

Чаще всего встречаются следующие варианты обновлений (рис. 7.7):

- *Фиксированный сдвиг*. При работе с данными, которые сильно зависят от времени и новизны, нас не интересует оценка качества модели на данных, собранных год назад или ранее, из-за резкого изменения целевого распределения. Вместо этого мы хотели бы использовать для валидации только свежие данные.

  Например, в качестве валидационной выборки можно взять последние две недели (начиная с последнего завершенного дня) и ежедневно обновлять эту выборку, одновременно повторно обучая модель, используемую для оценки.

- *Фиксированное соотношение*. При работе с изображениями или текстами новые метки данных мы получаем нерегулярно. В отличие от первого случая, здесь может отсутствовать сильная зависимость от новизны данных,

то есть новые данные могут быть не более важными, чем старые. Обычно мы расширяем датасет сразу после получения новой порции размеченных данных.

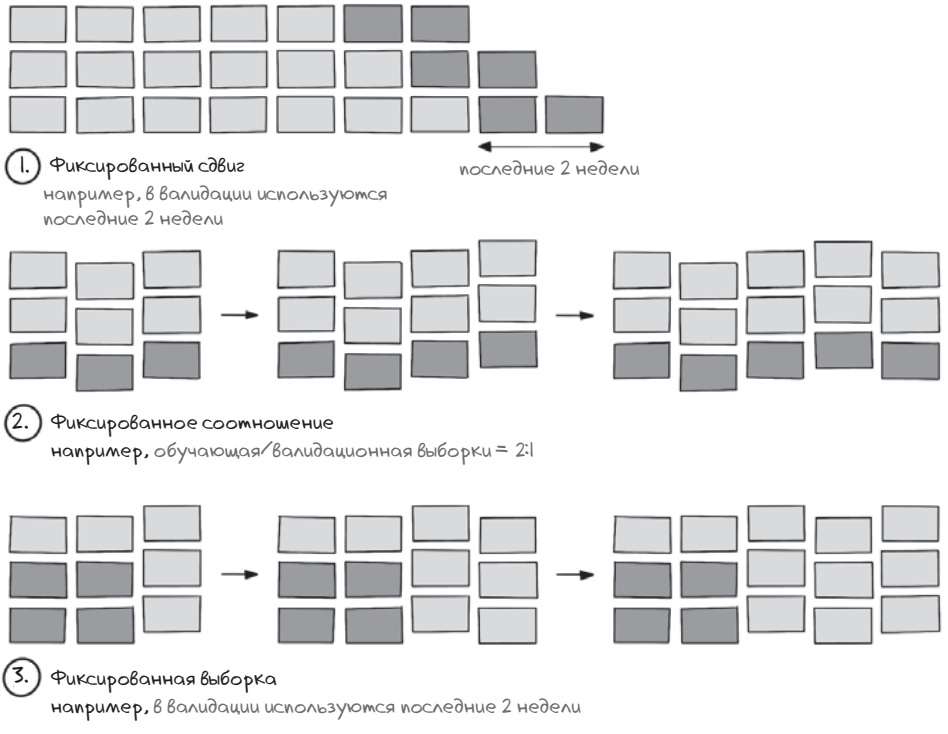


Рис. 7.7. Распространенные варианты обновления обучающих/валидационных выборок. Светлые блоки данных используются для обучения, темные — для валидации

Если мы включаем новые размеченные данные только в обучающую выборку, метрики модели растут за счет увеличения объема данных. Если же мы включаем их только в валидационную выборку, модель может не справляться с новыми закономерностями. Оптимальное решение — сохранить неизменным соотношение размеров обучающего и валидационного датасетов, чтобы новые данные распределялись соответствующим образом.

- *Фиксированная выборка*. Иногда вместо сбалансированной выборки всех доступных в текущий момент данных мы решаем оценивать качество модели по неизменной «золотой выборке», используемой как бенчмарк. Такой подход гарантирует, что модели будут сопоставимы между собой по любой метрике, даже если между их разработкой прошло длительное время.

«Эталонная» выборка может быть создана до начала моделирования или подобрана вручную таким образом, чтобы включать в себя всевозможные сложные случаи и образцы ответов. При этом для стабильного сравнения моделей она не должна обновляться. Если мы в дальнейшем решим расширить ее, то должны будем рассматривать ее как совершенно новую эталонную выборку.

Помните: валидацию необходимо проводить по всему пайплайну, включая датасет, а инференс тестовой выборки должен быть таким же, как в продакшене. Если мы хотим корректно сравнивать модели между собой, то нужно каким-то образом сохранять предыдущие версии как датасетов, так и моделей. Для этого могут пригодиться инструменты контроля версий данных и моделей (например, DVC, Git LFS или Deep Lake).

Если вам кажется, что приведенные здесь варианты не подходят под ваш конкретный случай, стоит подробнее изучить литературу, посвященную динамическим (нестационарным) потокам данных и дрейфу концепций, чтобы получить целостное представление о соответствующих теориях (например, «Scarcity of Labels in Non-Stationary Data Streams: A Survey» (https://mng.bz/gAXE)). Мы затронем проблему дрейфа концепций позже, в главе 11, — как одну из основных причин, почему построение надежной схемы валидации не такая уж и простая задача.

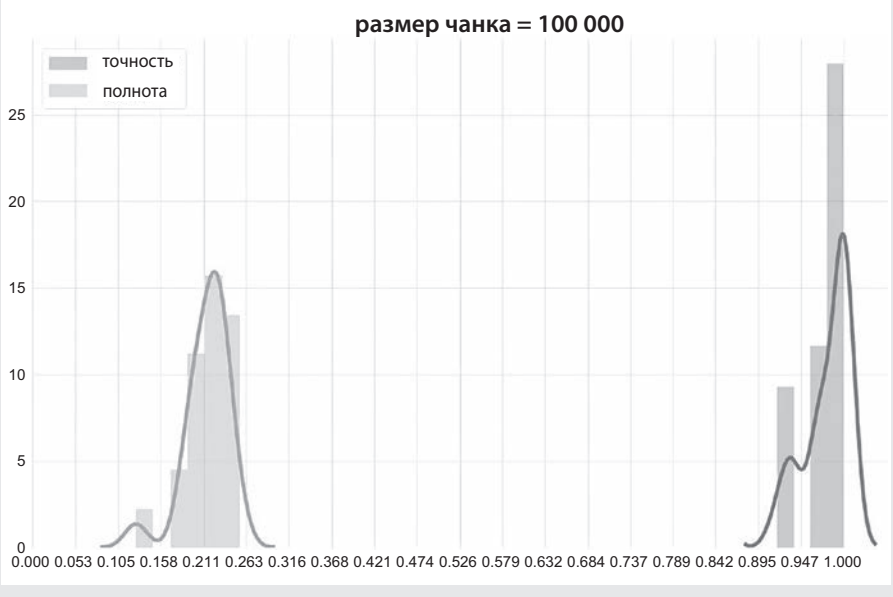
Байка от Валерия

В одной Big Tech компании я занимался обучением нескольких ML-моделей на внутренней платформе для выявления спамеров, мошенников, парсеров и других злоумышленников. Однако платформа выдавала только точечные оценки при проверке качества моделей на валидационной выборке. Это оказалось проблемой, поскольку офлайн-оценка зачастую сильно отличалась от производительности моделей в онлайне, что приводило либо к большому количеству ошибочно заблокированных пользователей, либо к ложным ожиданиям.

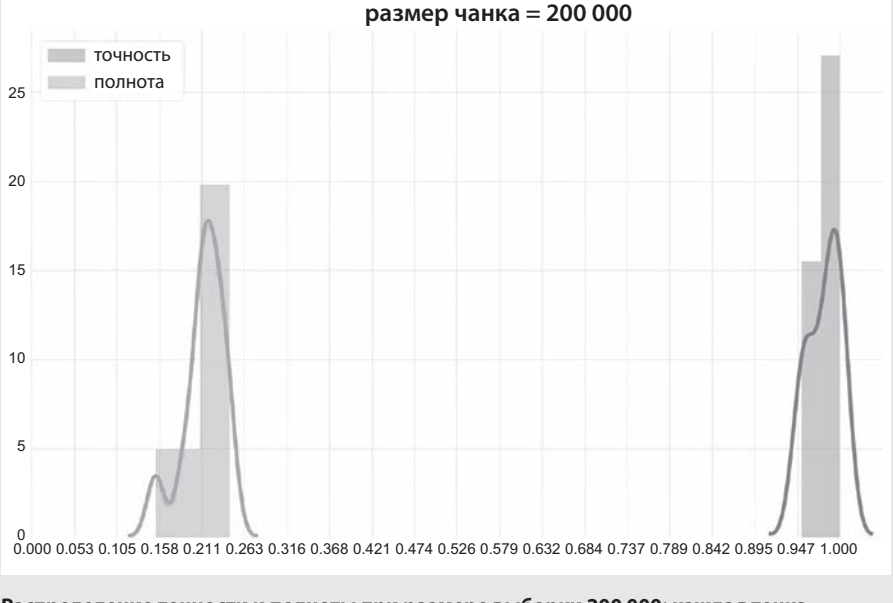
В качестве иллюстрации проблемы точечной оценки рассмотрим пример с подбрасыванием монеты.

Если подбросить обычную монету 100 раз, можно подсчитать количество выпадений орла. Это будет наша точечная оценка. Если повторить этот эксперимент, получится уже другое число. Если же вместо этого сказать, что в 95 из 100 случаев мы ожидаем, что результат будет находиться в диапазоне от 40 до 60, это будет наш доверительный интервал. Нижняя граница доверительного интервала будет 40, это означает, что в 95 % случаев мы ожидаем увидеть не менее 40 выпадений орла.

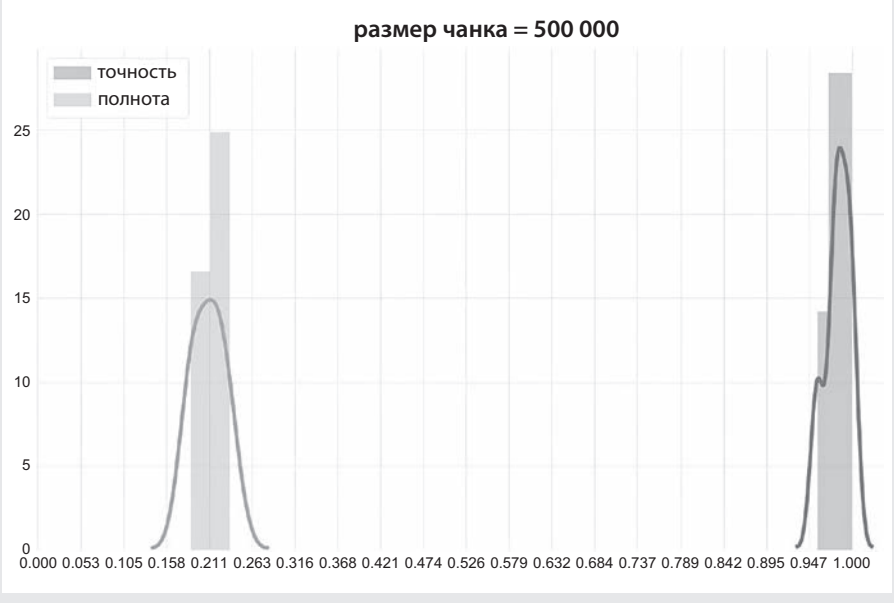
Точечная оценка ненадежна, поскольку она не учитывает постоянно присутствующую неопределенность, что легко продемонстрировать графически. Следующие графики показывают вариацию двух метрик — точности и полноты — при использовании одного и того же порога, ML-классификатора и валидационных данных, сгенерированных одним и тем же распределением на офлайн-данных.



Распределение точности (precision) и полноты (recall) при размере выборки 100 000; каждая точка представляет собой независимый датасет



Распределение точности и полноты при размере выборки 200 000; каждая точка представляет собой независимый датасет



Распределение точности и полноты при размере выборки 500 000; каждая точка представляет собой независимый датасет

Неудивительно, что при сравнении точечных офлайн-оценок с онлайн-результатами они почти всегда существенно различались. Даже в рамках офлайн-оценки дисперсия значений была огромной, несмотря на использование валидационных данных объемом 500 000. Такая ситуация свидетельствует о недостаточной надежности системы.

При разделении тестовых данных на чанки легко продемонстрировать наличие неопределенности для метрик вроде точности, полноты и др. Однако существуют и более эффективные способы это сделать. Намного лучше работает случайная выборка с возвращением, иначе говоря, бутстрэппинг. К сожалению, бутстрэппинг требует крайне много ресурсов: для каждой итерации (а их требуется от 10 000 до 100 000) необходимо делать выборку из мультиномиального распределения длиной N (размер которой может достигать тысяч или миллионов) и делать это N раз.

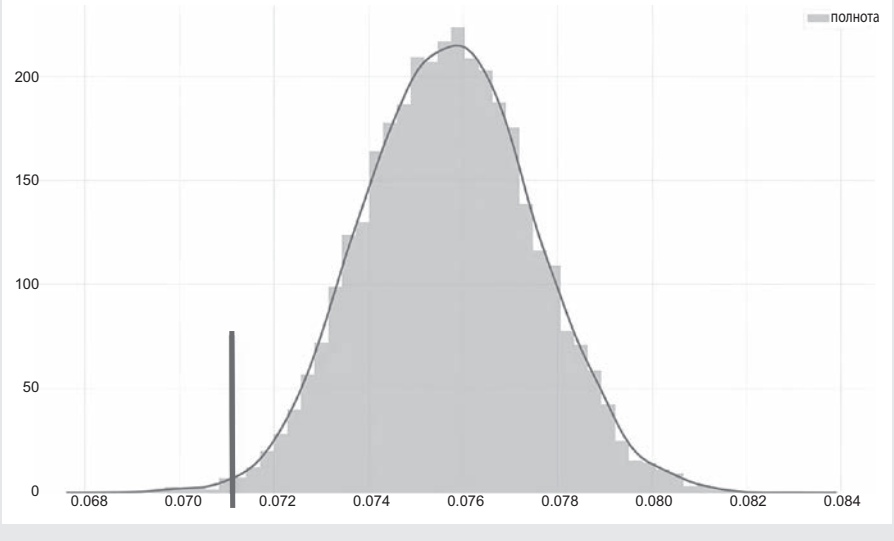
Это оказалось серьезной проблемой. С одной стороны, я не мог использовать уже существующее решение для оценки, поскольку оно было недостаточно надежным. С другой стороны, интеграция бутстрэппинга в каждый этап валидации сделала бы время выполнения даже одной итерации обучения слишком долгим.

Решение я нашел в математике. Если рассматривать каждую выборку отдельно и запускать бутстрэппинг параллельно, можно перейти от мультиномиальной выборки к биномиальной (n,1/n) и независимо отбирать каждое наблюдение для каждой итерации бутстрэппинга. При больших N ($N >> 100$) выборка из распределения Пуассона

с параметром λ = 1 приближает нас к биномиальному распределению (n, 1/n), другими словами: биноминальное распределение (n,1/n) ~ распределение Пуассона (1) при N >> 100 (подробнее можно прочитать здесь: https://mng.bz/OmyR).

В распределении Пуассона (1) не существует N, что делает его полностью независимым от размера данных, поэтому его легко проводить параллельно. Это значительно увеличивает скорость выполнения (примерно в 100–1000 раз быстрее — в моем случае с использованием ряда дополнительных оптимизаций).

Имея распределение для интересующей нас метрики, можем выбрать доверительную границу. На следующем графике показана нижняя 99 %-ная доверительная граница. В среднем в 99 случаях из 100 значение полноты будет не ниже 0.071.



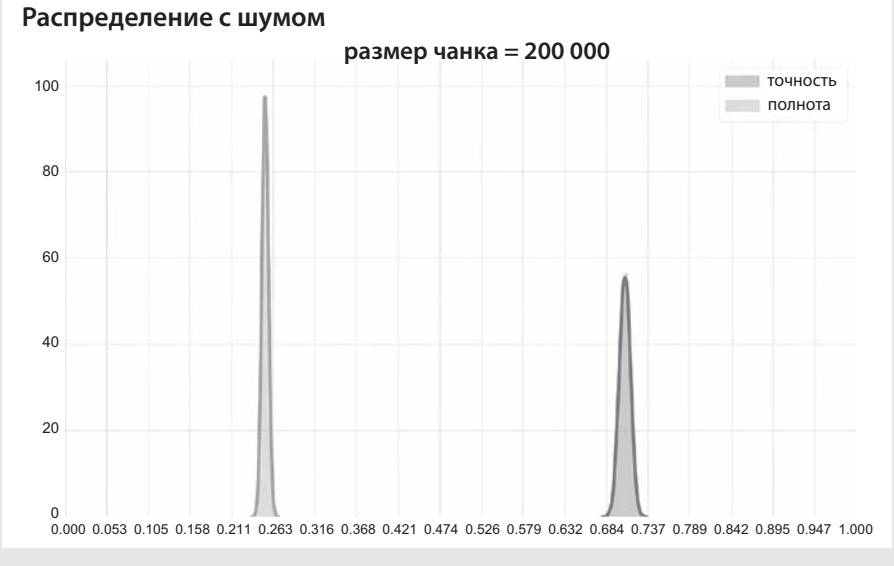
Распределение полноты; каждая точка соответствует бутстрэпированному исходному датасету, красная линия представляет собой нижнюю 99%-ную доверительную границу

Есть еще один важный момент, который стоит учитывать. Некоторые метрики, включая точность и полноту, зависят от выбранного порога для их расчета. Приведенные ниже рисунки демонстрируют распределения точности и полноты с небольшим шумом и без него (среднее распределение = 0, а стандартное отклонение = –0.0125) применительно к выборкам.

Можно заметить, что результаты существенно различаются: с шумом полнота увеличивается, а точность снижается. Графики показывают, что в данной ситуации граница принятия решения узкая и ненадежная. Добавление шума в качестве гиперпараметра помогает более уверенно оценивать доверительные интервалы распределения и повышать надежность границы принятия решений.



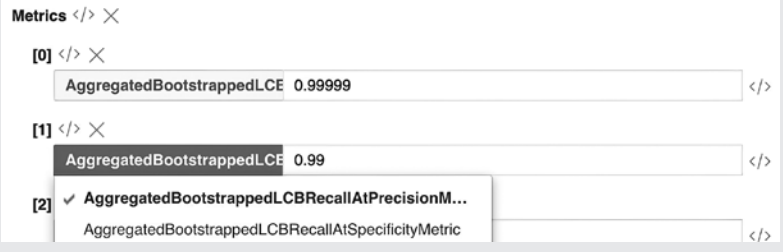
Распределение точности и полноты при размере выборки 200 000; каждая точка соответствует бутстрэпированному исходному датасету, шум не добавлялся



Распределение точности и полноты при размере выборки 200 000; каждая точка соответствует бутстрэпированному исходному датасету, шум добавлен

Оценивать полноту при заданной точности/специфичности сама по себе идея не новая, однако в сочетании с бутстрэпированием по Пуассону и добавлением шума

она позволила создать новые метрики: бутстрэпированная нижняя доверительная граница полноты при заданной точности и бутстрэпированная нижняя доверительная граница полноты при заданной специфичности. Данные метрики обеспечивают гарантированную (с заданным уровнем доверия), надежную и устойчивую оценку производительности ML-модели.



Метрики, встроенные в нативную платформу ML

7.5. Дизайн-документ: выбор схем валидации

Теперь давайте дополним наши дизайн-документы для компаний Supermegaretail и PhotoStock Inc. новыми блоками, содержащими информацию о выбранных схемах валидации.

7.5.1. Схемы валидации для Supermegaretail

Начнем с Supermegaretail.

ДИЗАЙН-ДОКУМЕНТ: SUPERMEGARETAIL

IV. СХЕМА ВАЛИДАЦИИ

1. Требования

Что необходимо учитывать при разработке процесса оценки?

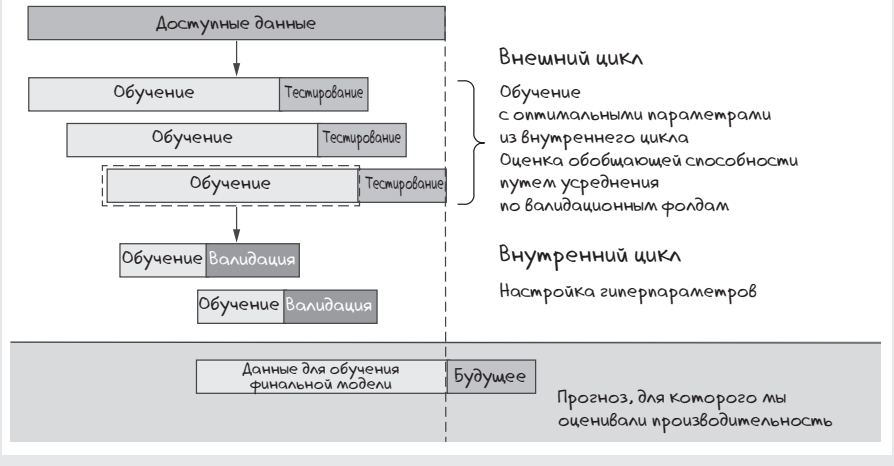
- Новые данные поступают ежедневно.
- Данные могут приходить с задержкой до 48 часов.
- Новые метки (количество проданных единиц) поступают вместе с новыми данными.
- Последние данные, скорее всего, более релевантны для прогнозирования.
- Матрица ассортимента меняется на 15 % каждый месяц.
- Данные подвержены сезонности (недельные/годовые циклы).

Несмотря на то что данные естественным образом делятся на категории, это не имеет значения при выборе схемы валидации.

2. Инференс

После выбора модели (в рамках процесса оптимизации гиперпараметров) мы обучаем ее на данных за последние 2 года и прогнозируем спрос на ближайшие 4 недели. Процесс воспроизводится как во внутренней, так и во внешней валидации.

Важно отметить, что между обучающей и валидационной выборками должен быть интервал в 3 дня, чтобы учесть возможные задержки в поступлении данных. Это, в свою очередь, влияет на то, какие признаки мы можем или не можем использовать при построении модели.



3. Внутренние и внешние циклы

Мы используем два уровня валидации. Внешний цикл предназначен для финальной оценки производительности модели, а внутренний — для оптимизации гиперпараметров.

Во-первых, для внешнего цикла, учитывая работу с временными рядами, очевидным становится выбор скользящей кросс-валидации. Мы устанавливаем $K = 5$ для обучения пяти моделей с оптимальными параметрами. Поскольку прогноз делается на 4 недели вперед, размер окна валидации составляет 28 дней для всех разделений. Между выборками предусмотрен интервал в 3 дня, шаг составляет 7 дней.

Пример внешнего цикла:

- Первый внешний фолд:
  - Данные для тестирования: с 10.10.2022 по 06.11.2022 (4 недели).
  - Данные для обучения: с 07.10.2020 по 06.10.2022 (2 года).
- Второй внешний фолд:
  - Данные для тестирования: с 03.10.2022 по 30.10.2022.
  - Данные для обучения: с 29.09.2020 по 28.09.2022.

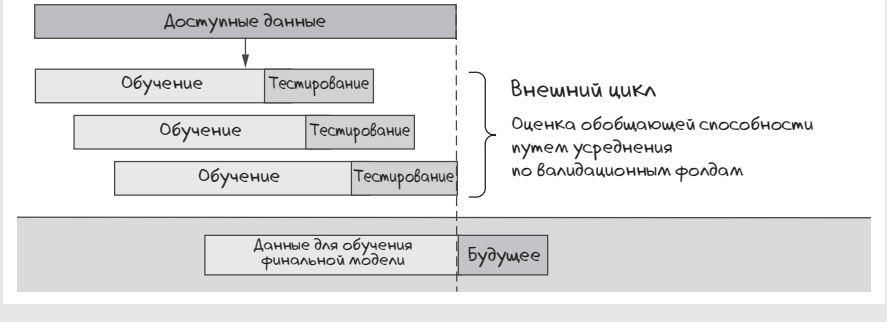
- Пятый внешний фолд:
  - Данные для тестирования: с 12.09.2022 по 09.10.2022.
  - Данные для обучения: с 09.09.2020 по 08.09.2022.

Во-вторых, для внутреннего цикла, внутри каждой «обучающей выборки» внешней валидации мы выполняем дополнительную скользящую кросс-валидацию с разделением на три части. Каждая обучающая выборка внутреннего цикла включает историю за 2 года для учета как годовой, так и недельной сезонности. Внутренний цикл используется для настройки гиперпараметров или отбора признаков.

Пример внутреннего цикла:

- Второй фолд внешнего цикла:
  - Обучающая выборка для второго внешнего фолда: с 03.10.2020 по 02.10.2022.
- Первый внутренний фолд:
  - Данные для тестирования: с 05.09.2022 по 02.10.2022 (4 недели).
  - Данные для обучения: с 02.09.2020 по 01.09.2022 (2 года).
- Второй внутренний фолд:
  - Данные для тестирования: с 29.08.2022 по 25.09.2022.
  - Данные для обучения: с 26.08.2020 по 25.08.2022.
- Третий внутренний фолд:
  - Данные для тестирования: с 22.08.2022 по 18.09.2022.
  - Данные для обучения: с 19.08.2020 по 18.08.2022.

Если для модели настройка гиперпараметров не требуется, внутренний цикл можно пропустить.



4. Частота обновления

Мы обновляем разделения еженедельно вместе с получением новых данных и меток (таким образом, каждая валидационная выборка всегда охватывает целую неделю). Это помогает фиксировать локальные изменения и тренды в производительности модели.

Кроме того, у нас есть отдельная отложенная выборка, которая используется в качестве бенчмарка («золотая выборка»). Мы обновляем ее каждые 3 месяца. Это помогает отслеживать долгосрочные улучшения нашей системы.

7.5.2. Схемы валидации для PhotoStock Inc.

Теперь рассмотрим схемы валидации для PhotoStock Inc.

ДИЗАЙН-ДОКУМЕНТ: PHOTOSTOCK INC.

IV. СХЕМА ВАЛИДАЦИИ

Поисковый запрос является основным объектом валидации. При планировании стратегии валидации для поисковой системы PhotoStock Inc. необходимо учитывать четыре основных момента:

- Выборки для валидации и тестирования должны быть репрезентативными относительно данных с продакшен, то есть отражать реальные пользовательские запросы.
- Выборки для валидации и тестирования должны быть разнообразными, то есть охватывать как можно более широкий спектр тем и контекстов.
- Запросы одного и того же пользователя должны встречаться только в одной из выборок — обучающей, валидационной или тестовой, — но не в нескольких одновременно, чтобы избежать утечки данных.
- Дубликаты запросов должны быть удалены из датасета для предотвращения утечки данных.

Таким образом, мы предлагаем следующую стратегию разделения данных:

1. Сгруппировать запросы по пользователям — каждому запросу назначается один пользователь. Если другой пользователь имеет такой же запрос, он игнорируется.
2. Случайным образом разделить пользователей на обучающую, валидационную и тестовую выборки в фиксированном соотношении (точное соотношение еще неизвестно, можно начать с 90/5/5).
3. Назначать нового пользователя к определенной выборке единожды и больше не изменять это назначение.

Случайное разделение пользователей должно компенсировать возможную несбалансированность распределения данных. Например, можно предположить наличие сезонности в поисковых запросах (пользователи в выходные — любители, а в будние дни — профессионалы), а также наличие дрейфа распределения со временем (появляются новые темы, старые исчезают). Случайное разделение должно справляться с данными вопросами, однако потребуется дополнительный анализ для подтверждения гипотезы.

Назначать выборки пользователям мы предлагаем с помощью детерминированного подхода к группировке: пользователи распределяются по группам на основе хеша их `user_id`, после чего каждой группе назначается определенная выборка. Такой подход универсален, так как позволяет в будущем изменять пропорции разделения. Например, если потребуется увеличить размер валидационной выборки, можно будет просто перераспределить дополнительные группы из обучающей выборки в валидационную.

Пример подхода с использованием группировки:

```
def assign_bucket(user_id):
    _hash = sha1(user_id.encode()).hexdigest()
    return int(_hash, 16) % n_buckets

def assign_split(user_id):
    bucket = assign_bucket(user_id)
    if bucket < n_buckets * train_ratio:
        return 'train'
    elif bucket < n_buckets * (train_ratio + val_ratio):
        return 'val'
    else:
        return 'test'
```

На начальном этапе проекта мы не планируем добавлять дополнительные подвыборки (например, «золотую выборку»), хотя такая возможность в будущем не исключается.

Итоги

- Используйте схемы валидации для точной оценки прогностической способности вашей модели.
- Старайтесь избегать многократного использования одной и той же валидационной выборки для оценки и поиска оптимальных гиперпараметров, поскольку это может привести к смещенным/переобученным и нестабильным результатам.
- Старайтесь проектировать схему валидации таким образом, чтобы она отражала реальное применение модели на практике.
- При выборе необходимого числа *K*-фолдов валидации учитывайте три переменные: смещение, дисперсию и время вычислений.
- Для этого установите различия между видимыми и невидимыми данными (существуют ли группы, классы, временные или другие важные характеристики, которые следует учитывать).
- При необходимости используйте нестандартную схему валидации для решения вашей проблемы.
- Помните, что разные схемы валидации могут хорошо сочетаться между собой.

8 Базовая модель

В этой главе

- ✓ Что такое базовая модель
- ✓ Константные базовые модели
- ✓ Базовые модели и базовые признаки
- ✓ Разнообразие базовых моделей в глубоком обучении
- ✓ Сравнение базовых моделей

> Все должно быть изложено так просто, как только возможно, но не проще.
>
> *Альберт Эйнштейн*

Важнейшей частью нашей будущей системы машинного обучения, или ее основным компонентом, является модель, построенная с использованием методов ML. В определенном смысле это действительно так, и может даже показаться, что вот оно, именно этому следует посвятить все свое время, усилия и творческую энергию.

Но на практике это оказывается ловушкой, куда попадает большинство ML-проектов и где они застревают, так и не дойдя до ввода в эксплуатацию. ML-модель вовсе не обязательно является самым важным элементом в контексте всей системы и ее дизайн-документа. Можно потратить огромное количество времени, усилий и, что важнее всего, денег на создание крутой, современной и сложной

модели ИИ, которая в итоге не принесет никакой пользы ни пользователям, ни компании. Модель среднего качества, запущенная в эксплуатацию, зачастую лучше, чем просто превосходная модель, но существующая лишь на бумаге.

Одним из первых вариантов названия этой книги было *«Проектирование систем машинного обучения, которые действительно работают»*, что соответствует главной цели любого ML-проекта — построить систему, которая будет работать. Только после этого, когда она уже начнет приносить прибыль, мы сможем постепенно ее улучшать, при необходимости делая ее сложнее. В этой главе мы рассмотрим понятие базовой модели — первого шага к созданию работающей системы. Мы обсудим, зачем нужны базовые модели, и пройдем весь путь от константных базовых моделей до сложных специализированных моделей, а также рассмотрим различные варианты их построения на основе ряда признаков.

8.1. Что такое базовая модель

Базовая модель (baseline) — это самая простая (но работающая!) версия модели, набора признаков или любого другого компонента системы. Это минимально жизнеспособный продукт (MVP) в сфере ML, с самого начала приносящий ценность без какого-либо усложнения. Давайте подробнее рассмотрим аналогию с MVP и выделим ключевые цели, которые в равной степени применимы к обоим понятиям:

- *Снижение максимального риска при минимальных затратах времени, денег и усилий*. В самом начале жизненного цикла продукта еще неясно, нужен ли он рынку, какие у него будут сценарии использования, что будет происходить в экономике и т. д. Все эти риски характерны и для ML. Таким образом, базовая модель (или MVP) — это самый простой способ протестировать продукт.
- *Получение ранней обратной связи*. Это принцип быстрого выявления неудачи, но в масштабах всего продукта. Если вся идея ML-системы ошибочна, это будет понятно уже на раннем этапе. В таком случае у нас еще есть возможность пересмотреть план, переписать дизайн-документ с учетом новых знаний и начать заново.
- *Ранняя ценность*. Каждая компания стремится получать прибыль, при этом удовлетворяя потребности своих клиентов. Если мы можем принести пользу клиентам уже с базовой моделью, а затем постепенно ее улучшать, обеспечивая при этом получение прибыли, — почему бы этого не сделать? От этого выиграют все.

В этом и заключается основное сходство между базовой моделью и MVP. Однако существуют также три цели, характерные исключительно для базовых моделей:

- *Плейсхолдер, позволяющий убедиться, что все компоненты работают корректно*. Базовые модели можно сравнить с дымовыми тестами. Как писали Кем Кейнер (Cem Kaner), Джеймс Бах (James Bach) и Брет Петтикорд (Brett

Pettichord) в своей книге «Lessons Learned in Software Testing»[1], понятие «дымовой тест» пришло из производства электронных устройств. Вы вставляете новую плату, подаете питание и смотрите, не пошел ли от нее дым. Если пошел — питание стоит отключить. Конец теста.

Сначала нужно проверить, работает ли система в принципе, а уже затем — правильно ли она это делает. Чтобы «скомпилировать» всю систему, не требуется мощная ML-модель. Необходимо нечто, что прогнозирует значения в требуемом формате, опционально на основе каких-то входных данных. Почему бы не использовать для этого самый простой возможный вариант?

- *Объект для сравнения*. Окупятся ли в будущем ваши вложения? Базовая модель — это своеобразная точка отсчета, начало координат — то, с чем затем сравниваются новые модели.

  Производительность модели — далеко не единственный критерий сравнения. Важно также учитывать затраты, интерпретируемость, удобство сопровождения и т. д. Мы подробнее поговорим об этом в разделе 8.5.

- *Резервный ответ*. В отличие от MVP, который после выхода следующей версии чаще всего полностью заменяется, базовая модель никогда не уходит в прошлое. Лучше хранить ее в системе параллельно с основной, более сложной моделью. Если основная модель выходит из строя или ошибается, система может переключиться на базовую модель и использовать ее ответ.

Каковы преимущества хорошо подобранной базовой модели? Простота сама по себе несет множество плюсов: она устойчива, менее склонна к непредсказуемому поведению и переобучению (из-за меньшей степени свободы), ее легко построить и сопровождать, она не предъявляет высоких требований к вычислительным ресурсам. Следовательно, такие модели легко масштабируются. Дополнительным бонусом является то, что для коллег, не работающих с ML, простые модели легче для восприятия, и им проще понять, как работает система. Это может повысить доверие к продукту, что особенно важно, когда ставки высоки. И тем не менее простота — это не цель, а ценное свойство.

Если представить, что ML-система — это конструктор Lego, то базовая модель — это способ как можно быстрее собрать все блоки. При этом мы настоятельно рекомендуем проектировать систему как можно более модульной (то есть «ортогональной»). Это упростит последующие обновления, включая переход к более сложным моделям и признакам (начальная архитектура не должна ограничивать будущие изменения системы). Изначально система должна быть простой и гибкой, а не примитивной и ограниченной — с базовой моделью в основе.

Тем не менее, несмотря на все преимущества, которые дают базовые модели при минимальных затратах, на практике их используют гораздо реже, чем следовало

[1] Кейнер К., Бах Дж., Петтикорд Б. «Тестирование программного обеспечения: контекстно ориентированный подход». СПб., издательство «Питер».

бы. Причина заключается в том, что, к сожалению, сложность лучше продается. Есть отличная статья Юджина Яна (Eugene Yan), с которой мы рекомендуем ознакомиться. Она называется «Simplicity Is an Advantage But Sadly Complexity Sells Better»[1] (https://eugeneyan.com/writing/simplicity/) и хорошо объясняет основные причины, по которым многие предпочитают сложность простоте:

- Сложность создает видимость усилий.
- Сложность создает видимость мастерства.
- Сложность создает видимость инноваций.
- Сложность создает видимость богатства функционала.

Это приводит к предвзятости в пользу сложности — когда мы неоправданно придаем больше значения и отдаем предпочтение сложным идеям и системам вместо простых решений.

Разумеется, базовые модели — это не универсальное решение всех проблем. Существуют ситуации, когда базовые модели либо не нужны, либо вообще неприменимы:

- *Точность критически важна*. Во многих задачах ошибка в несколько процентов может остаться незамеченной. Но если мы не можем позволить себе снижение качества, например, в медицинских приложениях для обнаружения рака или при работе с автономными транспортными средствами, базовая модель в качестве страховки нам не подходит. В таких случаях более надежным решением будет переключение на ручное управление.
- *Высокая степень уверенности*. Мы четко понимаем, что нужно пользователю (например, на основе опыта конкурентов), или уже имеем собственный опыт реализации аналогичных систем. В таком случае нет необходимости заново изобретать велосипед и тратить время на поэтапные итерации, если у нас уже есть проверенный план, который можно просто воспроизвести.
- *Мы переделываем уже работающую систему.* Допустим, у нас есть поисковая система, основанная на архитектуре DSSM (Deep Semantic Similarity Model), и весь пайплайн уже реализован и протестирован. Когда система стабильно приносит пользу клиентам, приходит время ее оптимизации с точки зрения скорости и точности, например, с переходом на модель, основанную на архитектуре трансформера. В таком случае нет смысла заново строить базовую модель, потому что старая версия, по сути, ею и является.

Тем не менее, даже если в ряде случаев сложность на раннем этапе может быть оправданна, она не должна становиться универсальным решением. Сложность порождает ненужные трудности и способствует развитию синдрома «изобретено не нами», при котором разработчики предпочитают создавать все с нуля, а не

[1] «Простота — это преимущество, но, к сожалению, сложность лучше продается».

использовать уже существующие решения, даже если это позволит сэкономить время и ресурсы. Это ведет к снижению эффективности и зачастую к худшим результатам.

Именно поэтому мы считаем, что построение базового решения — это первый шаг создания вашей системы, а уже потом вы будете по мере необходимости ее улучшать.

8.2. Константные базовые модели

Базовую модель можно сравнить со строительством моста: иногда не нужна ни команда инженеров, ни огромные бюджеты, ни чертежи, ни годы работы. Достаточно устойчиво лежащего бревна. Базовая модель — это то самое бревно, которое позволяет соединить компоненты и решить поставленную задачу в минимальном масштабе, — временное, примитивное, простое в реализации решение (рис. 8.1).

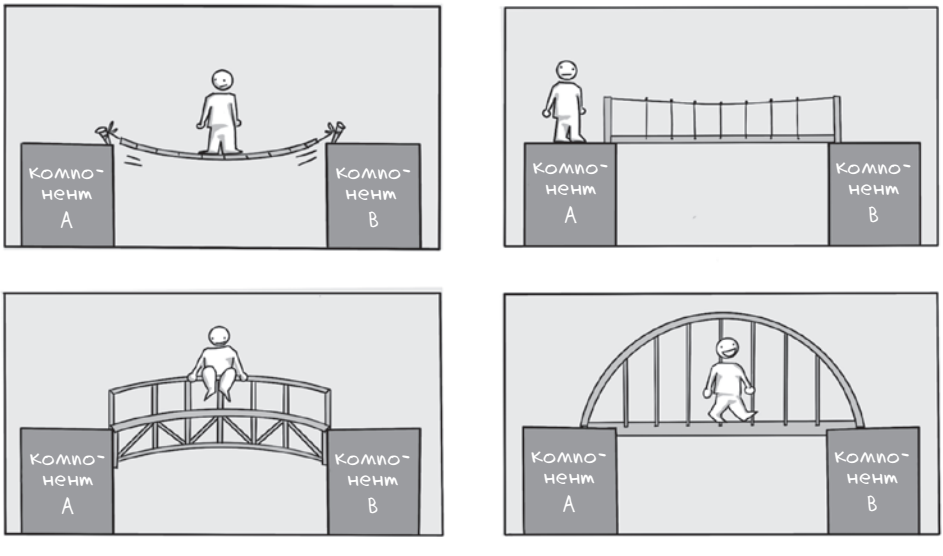


Рис. 8.1. Прежде чем строить сложную модель, начните с простейшей базовой модели — фундамента вашей будущей ML-системы

Прежде чем углубиться в детали, мы хотим донести простую мысль: сначала вы создаете простую, работоспособную ML-систему, а затем ее улучшаете. Представьте себе весь спектр возможных решений разного уровня сложности. Выберите подходящую стартовую точку, исходя из соотношения усилий и точности, и двигайтесь вперед. Не тратьте на моделирование много времени без необходимости.

Опираясь на эту аналогию, начнем с самых элементарных решений, так сказать, моста в виде бревна. При поиске подходящей базовой модели люди часто задают себе вопросы вроде: «Какая самая простая ML-модель может решить эту проблему?» или «С какой модели лучше начать?». Но это неверно. Мы считаем, лучше начать с вопроса: «А нужно ли вообще использовать ML для решения этой задачи?»

Иногда ML вовсе не требуется, или, по крайней мере, не нужно создавать свое решение — можно воспользоваться сторонним сервисом. Такой подход мы уже обсуждали в главе 3 (раздел 3.2).

Но допустим, мы решили разрабатывать собственную модель. Хорошее моделирование начинается с отсутствия модели: с попытки «хакнуть» заданную метрику, выбрав самое тривиальное и ленивое решение из пространства решений. Это первый шаг. Можно утверждать, что константная базовая модель сама по себе уже является моделью. В таком случае все зависимости и взаимодействия задаются одной постоянной (константой).

Чтобы стало понятно, о чем идет речь, рассмотрим несколько знакомых примеров:

- Для задач регрессии константной базовой моделью может быть прогнозирование среднего или медианного значения (в задачах прогнозирования временных рядов можно использовать оба варианта по последнему дню/неделе/месяцу/году) либо последнее доступное значение (например, для соответствующего пользователя или объекта). Это также может быть произвольная константа, выбранная с целью максимизации метрики.
- Для задач классификации это может быть прогнозирование на основе наиболее вероятного класса (например, в задаче выявления мошенничества можно предположить, что мошенничества нет вообще) либо прогнозирование вероятности положительного класса.
- Для задач ранжирования это может быть случайный порядок документов или сортировка на основе нерелевантного числового признака, например ID документа, или по простой эвристике вроде «количество ключевых слов запроса, содержащихся в описании элемента».

В некотором смысле константная базовая модель — это как первый член ряда Тейлора или как средний предиктор, используемый в качестве первой базовой модели в градиентном бустинге (рис. 8.2). Они даже не зависят от переменной x, они уже делают (примерно) нечто, относящееся к нашей проблеме, — не больше, но и не меньше.

$$\boxed{f(x)=f(c)}+f'(c)(x-c)+\frac{f''(c)}{2!}(x-c)^2+\ldots$$

Рис. 8.2. Константная базовая модель как первый член ряда Тейлора: простейшая аппроксимация, закладывающая фундамент для более сложных моделей

8.2.1. Зачем нужны константные базовые модели?

Построение константной базовой модели (constant baseline) преследует две цели.

Первая — это бенчмаркинг. Полезно получить базовое значение выбранной метрики для случайного предсказания. Обычная проверка адекватности (sanity check) сравнивает вашу модель с простыми эмпирическими правилами. Действительно, было бы обидно потратить две недели на сложную модель, а затем за пять минут реализовать самую простую базовую модель, которая в итоге покажет лучшие результаты. Звучит абсурдно, но на практике такое случается довольно часто.

Например, Валерию однажды посчастливилось работать с инженером, которая не только замечательный человек, но еще и отличный специалист. Как-то раз она выиграла соревнование ML по прогнозированию временных рядов на производстве, использовав всего лишь константную базовую модель, или, как она сама любит уточнять, ступенчато-константную. Обычно соревнования в ML проходят по стандартной схеме: участникам предоставляется размеченный и неразмеченный датасеты. Их задача — построить модель на размеченном датасете, которая выдаст прогнозы для неразмеченного, максимально приближенные к реальным (известным только организаторам). Представьте себе разочарование остальных участников, которые месяцами создавали десятки признаков и подбирали параметры своих градиентных моделей.

Этот случай говорит нам о том, что вначале всегда стоит пробовать самые простые модели, — так вы с самого начала получите более реалистичное представление о своей метрике и целевых значениях и сможете оценить, чего возможно достичь с имеющимися данными, а чего — нет.

Помимо этого, базовые модели — это ваш запасной план. Если ваша реальная ML-модель не сможет выполнить прогноз — из-за какой-либо ошибки, из-за превышения ограничения по времени ответа, из-за отсутствия истории для расчета признаков (то есть в условиях холодного старта для новых пользователей или новых объектов) или просто «сошла с ума» (что иногда случается), — ваша система должна возвратить хотя бы что-то. В таком случае вам на помощь приходит константная базовая модель.

В то же время в некоторых ситуациях константная базовая модель может оказаться слишком примитивной и не принести никакой пользы, так как даже самая простая модель должна быть немного сложнее — набором эвристик/регулярных выражений или неглубокой (shallow) моделью. Константные базовые модели обычно хорошо работают в простых задачах регрессии/классификации на табличных данных или коротких текстах, но на их основе невозможно построить, скажем, чат-бота или систему распознавания речи.

8.3. Базовые модели и базовые признаки

Давайте продвинемся дальше по шкале сложности и рассмотрим следующий уровень — модели, основанные на правилах (rule-based). И хотя четкую границу между константными моделями и моделями, основанными на правилах, провести не всегда возможно — в большинстве случаев последние можно определить как константные значения в рамках определенной группы, существует и другой, хорошо известный и наглядный пример базовых моделей, основанных на правилах, — решение задач обработки естественного языка (NLP) с использованием одних только регулярных выражений.

Несколько лет назад Арсений работал в компании — агрегаторе такси, где участвовал в разработке сервиса, прогнозирующего время прибытия ближайшей машины к клиенту. Проблема была следующей: если мы завысим оценку времени, клиент может не захотеть ждать и обратиться к другому сервису, а если занизим — клиент будет ждать дольше, чем мы пообещали, что ему явно не понравится.

Коллега Арсения, на то время сеньор-разработчик, подошел к данной задаче как к стандартной регрессионной и начал с моделей вроде «всегда прогнозировать 5 минут» или «если район == “Манхэттен”, вернуть 4». Короче говоря, такие базовые модели было трудно превзойти даже с помощью сложных подходов ML, вторая из них даже использовалась в продакшене в качестве запасного варианта.

Модель вида «если район == X, вернуть Y» — отличный пример базовой модели, основанной на правилах. Мы можем создавать аналогичные модели, используя среднее значение, медиану или моду по какой-либо категории или нескольким категориям — в нашем примере это медиана по локации.

Чем сложнее становится модель, тем больше связей между свойствами объектов и метками она способна находить.

Типичная последовательность базовых моделей в ML выглядит следующим образом: константная модель, модель, основанная на правилах, и затем уже линейная модель (рис. 8.3). Что-то более сложное и специализированное потребуется только в том случае, если этих базовых моделей окажется недостаточно для выполнения задачи.

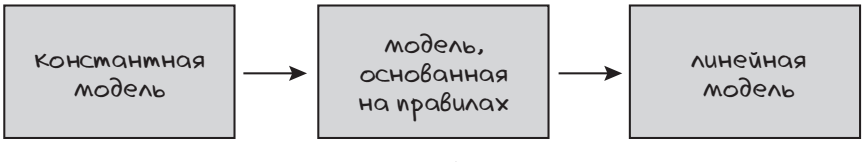


Рис. 8.3. Типичная последовательность базовых моделей на раннем этапе проектирования

Например, при построении системы рекомендаций мы начинаем с константной стратегии отбора, затем пробуем коллаборативную фильтрацию (например, метод чередующихся наименьших квадратов (alternate least squares)), факторизационные машины и, при необходимости, глубокое обучение (например, Deep Structured Semantic Model).

С какой бы задачей вы ни столкнулись, помните о простых подходах, существующих в данной области. Они не всегда уступают по эффективности более сложным методам.

Яркий пример можно найти в статье «Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches» Маурицио Феррари Дакрема (Maurizio Ferrari Dacrema) и др. (https://arxiv.org/abs/1907.06902), получившей награду «Лучшая статья на конференции RecSys» в 2019 году (https://recsys.acm.org/best-papers). Она примечательна тем, что раскрывает поразительную статистику. Исследовательская группа рассмотрела 18 алгоритмов, представленных на ведущих научных конференциях в предшествующие годы. После их анализа оказалось, что только семь можно было воспроизвести с разумными усилиями. И что еще интереснее, шесть из семи алгоритмов уступали по эффективности относительно простым эвристическим методам (например, на основе ближайших соседей или графовых техник). В результате лишь один алгоритм превосходил базовые модели, но даже он не смог опередить хорошо настроенный линейный метод ранжирования, не основанный на нейронных сетях.

Больше примеров можно найти в уже упомянутой ранее статье Юджина Яна (Eugene Yan):

- Древовидные модели (случайный лес, градиентный бустинг) в большинстве случаев превосходят глубокие нейронные сети при работе с табличными данными, особенно при малых/средних объемах датасетов (менее 1 млн записей) (https://arxiv.org/abs/2207.08815).
- Жадные алгоритмы показывают лучшие результаты, чем графовые нейронные сети в задачах комбинаторики на графах (https://arxiv.org/abs/2207.08815).
- Простое усреднение показывает себя не хуже сложных оптимизаторов в задачах многозадачного обучения (https://arxiv.org/abs/2201.04122).
- Скалярное произведение эмбеддингов превосходит нейронную коллаборативную фильтрацию в задачах рекомендации и извлечения элементов (https://arxiv.org/abs/2005.09683).

До этого момента мы говорили исключительно о базовых моделях. Но что насчет признаков? Признаки, по сути, являются частью модели и порой даже важнейшей ее частью. В классическом ML признаки создаются вручную, и их выбор для базовой модели подчиняется тем же принципам: начинать следует с небольшой группы наиболее значимых признаков (обычно тех, что проще всего вычислить). Существует два способа добавления новых признаков:

- генерация новых признаков — сложный и ресурсоемкий процесс, требующий построения новых ETL-пайплайнов;
- генерация признаков, производных от уже существующих.

Последовательность базовых признаков выглядит следующим образом: исходный минимальный набор признаков, различные взаимодействия и счетчики, затем эмбеддинги и только после этого — более сложные признаки (рис. 8.4).

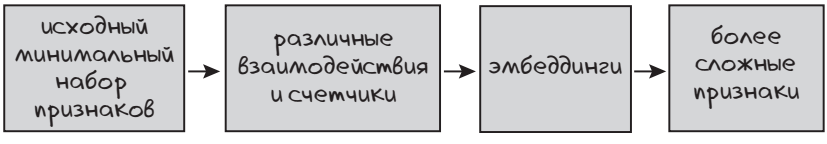


Рис. 8.4. Последовательность базовых признаков: от самых простых к более сложным

Какими свойствами должен обладать хороший стартовый набор признаков? Ответ будет точно таким же, как и в случае с моделями, и мы рассмотрим его далее.

Даже для типичной проблемы, решаемой с помощью ML, можно построить простую базовую модель с использованием неглубоких подходов. Как мы помним, глубокое обучение — это часть обучения представлениям (или признакам), что означает отказ от ручной генерации признаков в пользу их автоматического извлечения нейронной сетью. Однако для некоторых задач, например, для классификации изображений или текстов, можно применить и наивные подходы (основанные на правилах или линейных моделях). Например наивный байесовский классификатор активно применялся в обработке естественного языка до появления архитектур типа BERT. В компьютерном зрении некоторые задачи можно решать, используя гистограммы цветов пикселей (или просто средние/медианные значения!) в качестве признаков для линейной модели. Тем не менее во многих случаях лучше начинать с простой модели глубокого обучения — либо базовой в few-shot-сценарии, либо уже обученной, — поскольку такие модели хорошо зарекомендовали себя в широком спектре задач.

Однажды Арсений разработал задание для кандидатов, в рамках которого им предлагался скрипт, решающий задачу обнаружения аномалий в простом датасете изображений. Скрипт включал два базовых решения: одно на основе нейронной сети, другое — на основе гистограммы цвета. Кандидатам предлагалось улучшить одно из них, чтобы превзойти заданную метрику. Обе базовые реализации показывали примерно одинаковые результаты и были намеренно написаны неэффективно, чтобы было пространство для улучшений. Большинство кандидатов предпочли работать с более сложным решением на основе глубокого обучения, и лишь самые опытные заметили, что требуемого результата можно было достичь, изменив всего одну строку в решении, основанном на гистограмме.

8.4. Базовые модели глубокого обучения

В нестандартных задачах, предполагающих использование глубокого обучения из-за структуры данных (что актуально для большинства задач компьютерного зрения или обработки языка), набор возможных базовых моделей немного отличается. Наиболее распространенные из них — это повторное использование предварительно обученных моделей и обучение/тонкая настройка простейших моделей.

Повторное использование предобученных моделей — распространенная практика, если задача не уникальна и уже существует модель, обученная на аналогичной задаче. Например, если мы хотим обучить модель распознавать породы домашних животных, можно использовать модель, обученную на датасете ImageNet. ImageNet — это набор изображений, охватывающий 1000 классов, из которых более 100 — это породы собак. Таким образом, если наша цель — распознавание пород кошек и собак, мы можем использовать модель, обученную на ImageNet, без необходимости ее повторного обучения. Такая практика широко применяется в задачах общего характера, таких как распознавание речи, обнаружение объектов, классификация текстов, анализ тональности и т. д.

Чуть более продвинутой версией этого подхода является повторное использование признаков (также известных как *эмбеддинги* или *представления*), полученных из предобученной модели, для обучения простой неглубокой модели. Например, можно взять модель, обученную на датасете ImageNet, и использовать ее представления с последнего базового слоя (до финального классификационного слоя) для обучения простой линейной модели, которая затем будет классифицировать изображения по собственному набору классов. Такой подход особенно полезен, когда датасет небольшой, а конечная задача относительно проста (например, классификация), так что обучение крупной модели с нуля вряд ли понадобится. Данный подход относится к трансферному обучению, и мы наблюдали случаи, когда такая базовая модель превосходила по качеству более сложные.

Еще более специализированной версией использования предобученных моделей является применение моделей или сторонних API, способных работать в zero-shot- или few-shot-режимах (то есть выдающих результат без обучения или при наличии лишь нескольких обучающих примеров). Ярким примером такого API является семейство моделей GPT, но вообще API существует великое множество — у всех крупных облачных провайдеров есть длинный список ИИ-решений в области компьютерного зрения, например Amazon Rekognition или Google Cloud Vision AI. Подробное описание таких сервисов выходит за рамки темы данной книги.

Использование стороннего API от крупного вендора в качестве базовой модели дает интересный побочный эффект: он может служить аргументом в переговорах, например, при продаже программного продукта крупному предприятию или при доказательстве состоятельности стартапа перед инвесторами. Потенциальные

заказчики могут не знать, какие метрики считаются хорошими для данной проблемы (см. главу 5), но сравнение вашей технологии с решением от AWS помогает правильно сформулировать контекст. Арсений знает по крайней мере три стартапа, которые использовали данный подход, заявив, что превзошли альтернативы от Amazon, Google и OpenAI. Во всех трех случаях эти заявления были обоснованны, что вполне ожидаемо: крупные вендоры стремятся предлагать универсальные решения, в то время как стартапы могут разрабатывать узкоспециализированные ML-системы, превосходно решающие какую-то одну конкретную задачу.

Наконец, если ни один из этих вариантов вам не подходит, можно попробовать обучить простую модель с нуля. Тут мы рекомендуем отказаться от современных моделей и использовать что-то более простое и проверенное временем. Новые модели зачастую более «капризны» в обучении, тогда как «старички» уже достаточно хорошо изучены и для них существуют надежные рецепты стабильного обучения. Примерами таких моделей являются семейство ResNet для задач компьютерного зрения и семейство BERT для задач обработки естественного языка. Мы считаем, что лучше начинать с моделей, которым как минимум два года, но это не строгое правило — все зависит от уровня инноваций, необходимого для системы (см. главу 3).

Стоит отметить, что существует множество промежуточных вариантов тонкой настройки между «обучением неглубокой модели поверх предобученной» и «обучением модели с нуля». Выбор правильной степени настройки зависит от конкретного случая и может потребовать некоторых экспериментов. Например, при обучении модели классификации текста на основе BERT можно постепенно усложнять процесс обучения:

- обучить только последний слой;
- обучить некоторые блоки, используя методы адаптации, такие как low-rank adaptation (низкоранговая адаптация) (https://arxiv.org/abs/2106.09685);
- обучить некоторые блоки энкодера;
- обучить слои нормализации;
- обучить слой эмбеддингов;
- обучить всю модель;
- обучить всю модель и токенизатор.

Такое разнообразие вариантов приводит к вопросу: как же выбрать подходящую базовую модель?

8.5. Сравнение базовых моделей

Мы рассмотрели различные признаки и базовые модели, начиная с самых простых. Теперь можно перейти к главному вопросу: как понять, когда стоит остановиться в наращивании сложности, и как определить подходящую базовую

модель для своей системы? Для этого необходимо одновременно учитывать несколько факторов, в том числе:

- точность (accuracy)[1];
- трудозатраты (в первую очередь время разработки);
- интерпретируемость;
- время вычислений.

Основополагающим является баланс между точностью (или другой метрикой ML) и необходимыми трудозатратами. Первый компонент уравнения — точность. При переходе от константной модели к модели, основанной на правилах, затем к линейной модели, а также от исходных признаков к их агрегатам и соотношениям вы уже начинаете понимать, насколько подобные изменения улучшают метрики. Насколько чувствительны метрики? Насколько сложно превзойти константную модель? Разумно ли потратить больше времени на попытки повысить точность?

В некотором смысле ML-инженер осуществляет «обратное распространение» — получает «фидбэк» из цикла обучения и обновляет свое понимание проблемы, данных и распределения точности по пространству решений.

Второй компонент — это трудозатраты. Под «трудозатратами» мы в основном подразумеваем время и вычислительные ресурсы. Ни один ML-проект не располагает бесконечным бюджетом и, следовательно, бесконечным временем. Мы учитываем время, необходимое для реализации новой модели (или признака), ее обучения, отладки и тестирования. Также необходимо обратить внимание на все сопутствующие сложности и подводные камни, которые могут возникнуть по пути, особенно связанные с инфраструктурой.

Рассмотрим константную базовую модель. Ее реализация практически не занимает времени, и она дает наименьшую точность. Поэтому на соответствующем графике она будет находиться в точке (0, 0) (рис. 8.5).

Что касается линейной модели, то она требует больше усилий, но скорее всего, обеспечивает и более высокую точность. Соответствующую точку мы найдем правее и выше предыдущей, и т. д. С другой стороны, важно понимать, что по мере того как модель совершенствуется и развивается (а значит, возрастает ее сложность), соотношение затрат и точности начинает снижаться. Ярким

---

[1] Строго говоря, accuracy переводится как «доля верных результатов» (precision — точность, или доля истинно положительных прогнозов среди всех положительных прогнозов, сделанных моделью; accuracy — доля правильных предсказаний, как истинноположительных, так и истинноотрицательных, от общего количества сделанных предсказаний). В контексте этой книги разница не важна, поэтому и precision, и accuracy для простоты изложения переведено как «точность». — *Примеч. науч. ред.*

примером такого падения эффективности является градиентный бустинг, который мы упоминали ранее. Согласно нашим оценкам и опыту, градиентный бустинг требует больше усилий, чем все простые модели, которые вы могли использовать на ранних этапах, вместе взятые, при этом не обеспечивает значительного прироста точности.

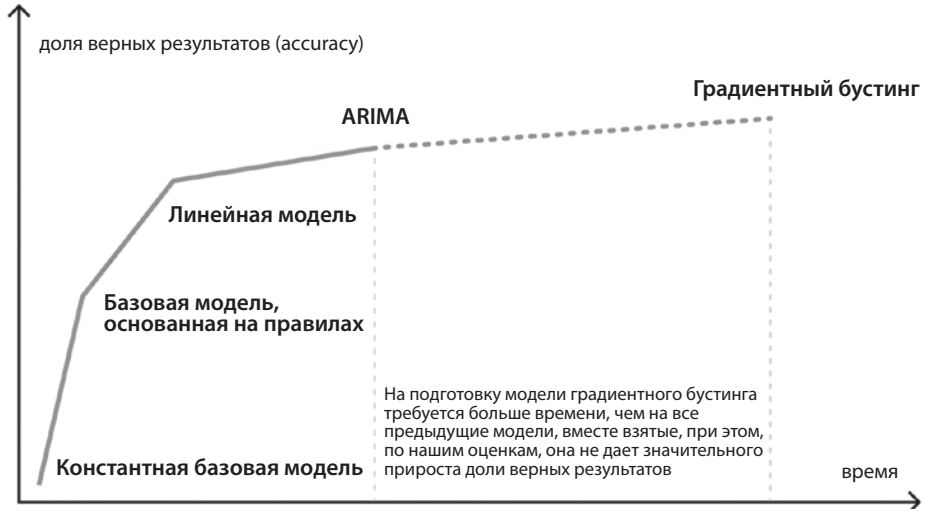


Рис. 8.5. Простые базовые модели легко построить, но итоговые показатели метрик будут хуже (пример для задачи прогнозирования временного ряда)

Мы должны каждый раз оценивать, сколько времени потребуется на то, чтобы попробовать более сложную модель, и какой дополнительный прирост точности она может дать. Как только мы понимаем, что следующий шаг требует слишком больших усилий при почти незаметном улучшении метрики, стоит остановиться. Такой «порог остановки» зависит от конкретной области и проблемы.

Но что, если что-то пойдет не так или потребуется внести дополнительные изменения в модель? Какую модель будет проще отладить или обновить?

- Слева на шкале — линейная регрессия с точным аналитическим решением.
- Справа — глубокая нейронная сеть со сложными пайплайнами обучения и инференса.

Что же выбрать?

Сопровождение, которое мы рассмотрим подробнее в главе 16, включает всю дополнительную работу, необходимую для отладки реализованных признаков или самой модели. Сопровождение можно считать частью дополнительных усилий, которых требует более сложная базовая модель.

Еще одним важным свойством базовой модели является время вычислений. Как время вычисления модели и ее признаков влияет на время отклика? Укладывается ли ваша базовая модель в соглашение об уровне обслуживания (SLA)? Является ли оно естественным пределом пространства решений, особенно при работе с системами реального времени? Но даже при отсутствии требований к работе в реальном времени время вычислений определяет, с какой скоростью мы сможем проводить итерации в будущем.

Наконец, есть интерпретируемость. Этот параметр особенно важен при осуществлении самой первой итерации любой ML-системы, особенно для других членов команды. Когда мы, например, имеем дело с чувствительными или медицинскими данными, это становится уже не только вопросом доверия к модели, но и проблемой безопасности. Общая закономерность следующая: чем проще базовая модель, тем проще объяснить, как она работает.

Мы более подробно рассмотрим эту тему позже, в главе 11.

8.6. Дизайн-документ: базовые модели

Поскольку базовые модели могут быть частью дизайн-документа, мы напишем этот раздел для наших вымышленных компаний — Supermegaretail и PhotoStock Inc.

8.6.1. Базовые модели для Supermegaretail

Начнем с системы прогнозирования. Фактор сезонности будет иметь решающее значение при выборе модели прогнозирования, поэтому мы обязаны учесть это в дизайн-документе.

ДИЗАЙН-ДОКУМЕНТ: SUPERMEGARETAIL

V. БАЗОВОЕ РЕШЕНИЕ

1. Константная базовая модель

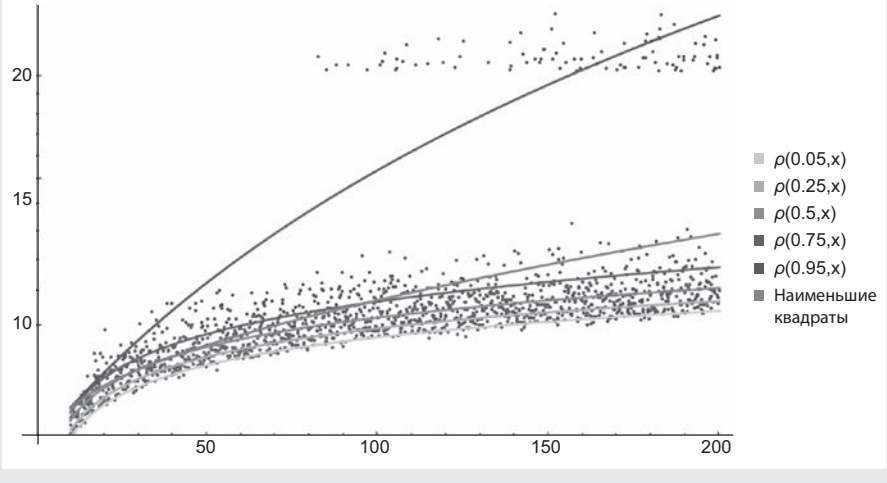
В качестве константной базовой модели для системы прогнозирования спроса компании Supermegaretail мы планируем использовать фактическое значение предыдущего дня для каждого уникального складского идентификатора товара (SKU) в каждом магазине. Учитывая, что данные иногда могут поступать с задержкой, а продажи в продуктовых магазинах подвержены сильной недельной сезонности, мы будем отступать назад не на один день, а на одну полную неделю. В результате наш прогноз для конкретного товара на 8 сентября 2022 года будет равен фактическому значению продаж этого товара за 1 сентября 2022 года.

2. Расширенная константная базовая модель

В главе 5 упоминались квантильные потери по 1.5, 25, 50, 75, 95 и 99 процентилям. Мы можем рассчитать то же самое с помощью нашей базовой модели, используя годовое окно.

3. Линейная базовая модель

Мы будем использовать базовый набор признаков для построения линейной регрессии с функцией потерь по квантилю. Для начала можно использовать только целевые переменные, но с различными лагами и агрегатами, такими как сумма/минимум/максимум/среднее/медиана или соответствующими квантилями за последние 7/14/30/60/90/180 дней, либо с различными скользящими окнами разных размеров. То же самое можно проделать и с другими динамическими данными, помимо данных о продажах, например с ценой, выручкой, средним чеком или количеством уникальных клиентов.



4. Специализированная базовая модель для временных рядов

Авторегрессионная интегрированная скользящая средняя (ARIMA) и сезонная ARIMA (SARIMA) — это авторегрессионные алгоритмы для прогнозирования; последний также учитывает сезонные закономерности.

Обе модели требуют тонкой настройки нескольких гиперпараметров для достижения удовлетворительной точности. Чтобы этого избежать, можно отдать предпочтение современному методу прогнозирования, который уже готов и хорошо работает, — Prophet (https://github.com/facebook/prophet). Преимущества Prophet в том, что он устойчив и не требует серьезной предварительной обработки: выбросы, пропущенные значения, сдвиги и тренды обрабатываются автоматически.

5. Базовые признаки

Какую дополнительную информацию могут использовать некоторые базовые и будущие модели?

Мы задействуем дополнительную статическую информацию о товарах (бренд, категория), магазинах (географические признаки) и контексте (временные признаки, сезонность, день недели) — все это с предварительной обработкой и кодированием, соответствующими выбранной модели.

Признаки, подходящие и для базовой модели, — это счетчики и взаимодействия. Например:

- Разница между текущей и средней ценой (абсолютная и относительная).
- Проникновение: отношение объема продаж товара к объему продаж категории (уровней 1, 2, 3) в скользящих окнах различного размера.
- Количество дней с момента последней покупки.
- Количество уникальных покупателей.

8.6.2. Базовые модели для компании PhotoStock Inc.

Теперь перейдем к компании PhotoStock Inc., для которой мы разрабатываем усовершенствованный поисковый движок, способный предоставлять более точные результаты и в перспективе повышать продажи.

ДИЗАЙН-ДОКУМЕНТ: PHOTOSTOCK INC

V. БАЗОВОЕ РЕШЕНИЕ

Мы предлагаем три подхода к выбору базовой модели для задачи поискового движка в PhotoStock Inc.

1. Базовое решение без ML

В настоящее время компания PhotoStock Inc. использует простое решение без применения ML. Это поисковый движок на основе ключевых слов, использующий базу данных ElasticSearch, поддерживающую нечеткий поиск. Он не требует обучения и уже развернут в продакшене, поэтому является хорошим кандидатом на роль базовой модели.

Однако у него есть два недостатка: он не использует изображения при поиске, только метаданные (например, теги, описания и т. д.) и его не так просто встроить в новую платформу ML для сравнения. Тем не менее он остается полезной базовой моделью, так как позволяет сравнивать производительность ML-модели с производительностью без ML.

2. Базовое решение с ML

Мы также можем использовать простую ML-модель в качестве базовой. Она не будет работать с изображениями, только с метаданными. Такая модель

сможет принимать запросы и метаданные как необработанный ввод, преобразовывать их в признаки с помощью наивного векторизатора TF-IDF (term frequency — inverse document frequency, частота слова — обратная частота документа), а затем использовать простую линейную модель для прогнозирования релевантности. Кроме того, она проста в реализации и обучении, что полезно на ранних этапах отладки и осмысления задачи.

3. Предобученная базовая модель

Наконец, мы можем использовать предобученную модель в качестве базовой. Учитывая специфику проблемы, необходимо решение, способное объединять визуальную и текстовую модальности, и наиболее известным вариантом является CLIP (https://openai.com/research/clip). CLIP был представлен в 2021 году и показал хорошие результаты в различных задачах. Существует несколько его аналогов, которые также можно рассмотреть при необходимости.

CLIP, по сути, представляет собой энкодер изображений и энкодер текста, обученные на задаче прогнозирования того, какие изображения каким текстовым описаниям соответствуют. Он был обучен на огромном датасете и демонстрирует разумную производительность в широком спектре задач. CLIP — это модель с открытым исходным кодом, распространяемая по лицензии MIT, поэтому она может использоваться в коммерческих целях.

Чтобы адаптировать ее под нашу задачу, можно начать с использования ее выходных данных для пары (запрос, изображение) как меры релевантности. Такой подход не использует метаданные и текстовые описания, поэтому его можно либо комбинировать с предыдущим подходом, либо развивать отдельно, используя двухкомпонентную меру релевантности. Например:

$$\text{степень_релевантности} = \text{расстояние}(\text{запрос}, \text{изображение}) + \\ + \text{расстояние}(\text{запрос}, \text{описание})$$

Оба расстояния могут быть вычислены с использованием CLIP — одно с применением как текстового, так и визуального энкодеров, другое — с использованием только текстового энкодера.

На первом этапе можно обойтись без обучения. Что касается последующих шагов, мы можем дообучить модель или ее компоненты на нашем датасете.

Итоги

- Базовые модели представляют собой неотъемлемую часть проектирования ML-систем, поскольку они эффективно решают технические, ML и продуктовые задачи (связывают компоненты, устанавливают метрику для сравнения и помогают понять пользовательский опыт продукта с использованием слабой модели).

- Хотя базовые модели воспринимаются как нечто элементарное, умение определить, с чего начать — как с точки зрения признаков, так и с точки зрения моделей, — часто недооценивается.
- По мере развития проекта стоит придерживаться такой последовательности: константная базовая модель, затем базовая модель, основанная на правилах, затем линейная модель, а уже потом — что-то более сложное.
- Вместо построения сложной модели лучше начать с константной базовой — это сэкономит ресурсы и время, а также покажет, движетесь ли вы в правильном направлении с минимальными затратами.
- Если задача предполагает использование глубокого обучения, наиболее распространенная практика — повторное использование предобученных моделей, обучение/дообучение простейших моделей или обучение простой модели с нуля, если первые два подхода неприменимы.
- При выборе между различными базовыми моделями ключевыми факторами являются точность (accuracy), трудозатраты (преимущественно время разработки), интерпретируемость и время вычислений, при этом особенно важно найти баланс между точностью и трудозатратами.
- По мере усложнения модели соотношение «затраты — точность» неизбежно ухудшается. Особенно это проявляется при переходе к градиентному бустингу, который, как показывает практика, требует больше ресурсов, чем все предыдущие модели, вместе взятые, но при этом не обеспечивает значительного прироста точности.

Часть 3

Промежуточные этапы

Часть 3 посвящена промежуточным этапам проектирования системы. Глава 9 представляет собой анализ кривой обучения, рассматривает понятие остатков и помогает выявить закономерности в ошибках модели. Глава 10 посвящена пайплайнам обучения, описывает инструменты и платформы, которые можно использовать для построения и поддержки пайплайнов обучения, а также охватывает такие темы, как масштабируемость и настраиваемость пайплайнов. В главе 11 речь идет о признаках и их генерации, рассматриваются способы анализа признаков, даются рекомендации по выбору подходящих признаков для ML-модели, а также приводятся преимущества и недостатки использования хранилищ признаков. В главе 12 рассматриваются методы измерения и представления результатов и обсуждаются преимущества A/B-тестов.

9 Анализ ошибок

В этой главе

- ✓ Анализ кривой обучения
- ✓ Анализ остатков
- ✓ Поиск закономерностей в остатках

После подготовки основных строительных блоков системы — первого датасета, метрик, процедуры оценки и базовой модели — можно переходить к этапу итеративных улучшений. Подобно тому как алгоритм обратного распространения ошибки в нейросетях определяет направление наискорейшего снижения функции потерь и передает эту информацию от слоя к слою, анализ ошибок позволяет найти самый быстрый способ улучшить всю систему.

Анализ ошибок служит своеобразным компасом, направляющим итеративные обновления системы. Он помогает понять динамику ошибок во время фазы обучения (анализ кривой обучения) и распределение ошибок после этапа прогнозирования (анализ остатков). Анализируя ошибки, можно выявить общие черты, тренды и закономерности, которые подскажут направления для улучшения вашей системы машинного обучения. В этой главе мы рассмотрим ключевые этапы и типы анализа ошибок и приведем примеры, которые, как мы надеемся, помогут вам лучше понять данный процесс.

Разработчики часто пропускают анализ ошибок при проектировании своих ML-систем, поскольку формально данный шаг не относится к *созданию системы*. Однако уделить этому время все же стоит, поскольку так вы сможете выявить

слабые места системы и определить пути ее улучшения. Поэтому не затронуть данную тему в книге было бы с нашей стороны серьезным упущением.

9.1. Анализ кривой обучения

Анализ кривой обучения оценивает процесс обучения модели путем построения и интерпретации кривой обучения, которая показывает взаимосвязь между качеством обучения модели и объемом использованных для этого данных. Цель анализа кривой обучения — ответить на два ключевых вопроса:

- Сходится ли модель?
- Если да, нет ли недообучения или переобучения?

Если на оба вопроса ответ отрицательный, нет необходимости в остальной части анализа.

Прежде чем перейти к деталям, давайте разберем, что такое кривая обучения. Этот термин пришел из поведенческой психологии, где он используется для отображения прогресса в обучении человека или животного во времени (рис. 9.1). Например, можно проанализировать количество ошибок, совершаемых испытуемым в каждом новом тесте, или изучить, как изменяется время, необходимое мыши для прохождения лабиринта, в зависимости от номера попытки.

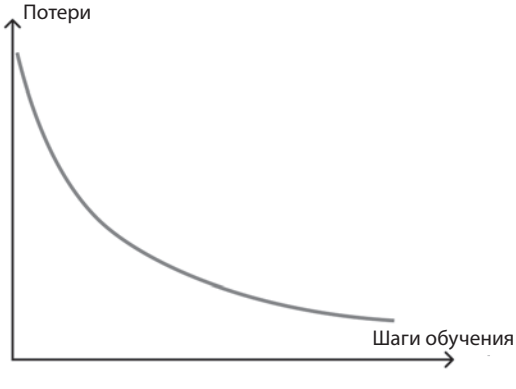


Рис. 9.1. Общий вид кривой обучения

В ML кривая обучения — это, по сути, графическое представление зависимости выбранной метрики от определенного числового параметра, например количества итераций, размера датасета или сложности модели. Кратко рассмотрим все три параметра:

- *Количество итераций*. Этот тип кривой отражает изменение функции потерь или метрики в процессе обучения и позволяет оценить сходимость

модели. В некоторых источниках такие кривые называют кривыми потерь или кривыми сходимости. Примером итераций может служить количество эпох обучения в нейросетях.

- *Сложность модели*. Показывает, как меняется производительность модели в зависимости от изменения ее сложности. По мере увеличения сложности модель, как правило, лучше подстраивается под обучающие данные, но при этом может терять способность к обобщению на новых данных. Примеры параметров сложности модели — глубина дерева решений, количество признаков, количество слоев в нейросети.
- *Размер датасета*. Демонстрирует, как количество обучающих семплов влияет на производительность модели. Такой анализ позволяет решить, стоит ли увеличивать объем данных.

Указанные параметры лежат в основе трех наиболее распространенных типов кривых обучения. Но прежде чем подробнее рассмотреть каждую из них, стоит вспомнить о «святом Граале» машинного обучения — проблеме переобучения и недообучения, также известной как компромисс между смещением и дисперсией (рис. 9.2).

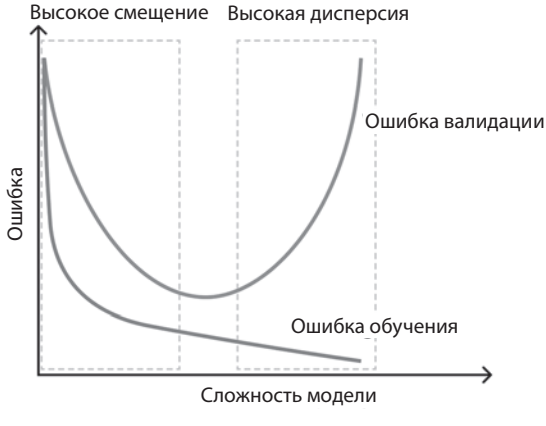


Рис. 9.2. При увеличении числа параметров модели ошибка обучения, как правило, снижается, минимизируя смещение. Дисперсия модели в то же время растет, что приводит к U-образной форме ошибки валидации

9.1.1. Переобучение и недообучение

Переобучение (overfitting) возникает тогда, когда модель показывает отличные результаты на обучающих данных, но плохие — на новых. Обычно это происходит, когда модель слишком хорошо запоминает обучающие данные, становится чрезмерно специализированной и теряет способность к обобщению, концентрируя внимание на незначительных деталях и паттернах, отсутствующих в новых данных.

Недообучение (underfitting), напротив, возникает, когда модель слишком проста и не видит важных зависимостей между признаками и целевой переменной, что приводит к слабым результатам как на обучающих, так и на новых данных.

Обе проблемы тесно связаны с компромиссом между смещением и дисперсией, то есть балансом между сложностью модели и объемом входных данных. Чем выше способность модели извлекать полезные сигналы из данных, тем ниже смещение и выше риск переобучения. Снижение дисперсии, в свою очередь, требует упрощения модели, что приводит к увеличению смещения.

Смещение, или *предвзятость* (bias), — это ошибка, возникающая из-за ограниченной способности модели видеть закономерности в данных. Другими словами, модель смещена в сторону своих упрощенных предположений о данных. Если модель имеет высокое смещение, она *недообучена*.

Дисперсия (variance) — это ошибка, возникающая из-за высокой чувствительности модели к незначительным изменениям в обучающей выборке. Такая модель плохо обобщает на новых данных, поскольку ее параметры сильно зависят от конкретных примеров из обучающей выборки. Зачастую высокая дисперсия является основной причиной *переобучения* (при условии, что остальные компоненты системы работают корректно).

Правильно выстроенный алгоритм обучения должен минимизировать как смещение, так и дисперсию. Однако компромисс между ними наглядно демонстрирует, что снижение одного параметра часто ведет к росту другого. Поэтому наша задача состоит в том, чтобы найти оптимальный баланс между ними. Здесь на помощь приходит анализ кривых обучения.

Следует помнить, что избыточная сложность модели (то есть высокая дисперсия) — не единственная причина переобучения. Возможны и другие сценарии, включая:

- *утечку данных* (использование информации, которая не должна быть доступна на этапе прогнозирования);
- *шумные или чрезмерно детализированные признаки*, заставляющие модель подстраиваться под незначимые паттерны;
- *наличие выбросов*, сильно влияющих на функцию потерь;
- *низкая способность* модели к экстраполяции;
- обучающая и валидационная выборки относятся к *разным распределениям*.

Вне зависимости от конкретной ситуации кривые обучения позволяют эффективно выявлять переобучение и недообучение. Теперь, разобравшись в сути указанных явлений, можем перейти к рассмотрению различных типов кривых и тому, как с их помощью принять оптимальное решение.

9.1.2. Кривая потерь

Кривая потерь (также называемая кривой сходимости, или кривой обучения), построенная на основе итераций обучения, — это первое, что приходит в голову

ML-инженерам при упоминании термина «кривая обучения». Она показывает, насколько алгоритм улучшает свою работу по ходу обучения.

Кривая отображает значение потерь (или какой-либо метрики) по вертикальной оси и количество итераций обучения (или эпох) по горизонтальной оси. В процессе обучения значение потерь должно снижаться, в идеале формируя нисходящий тренд в графике.

В отличие от кривых обучения, где на оси X располагается размер датасета или сложность модели (о них мы поговорим далее), кривая, построенная по итерациям, требует лишь одного прогона обучения, благодаря чему она полезна даже при работе с большими датасетами, когда на один прогон уходят часы, а то и дни. Кривая потерь позволяет контролировать весь процесс обучения. Даже если вы провели всего 10 эпох из запланированных 200, уже можно сделать выводы о том, движется ли значение потерь в ожидаемом направлении или же существуют какие-то проблемы, делающие дальнейшее обучение бессмысленным.

Обязательно отслеживайте кривые потерь, сохраняйте их для всех проведенных экспериментов и анализируйте. Мониторинг кривой потерь на ранних этапах построения пайплайна обучения позволяет получить данные, которые в будущем пригодятся вам для экспериментов, а также повысят воспроизводимость всей системы (мы подробнее рассмотрим эту тему в главе 10).

9.1.3. Анализ кривых потерь

Существует несколько вариантов отклонения кривых потерь от стандартного поведения. Давайте кратко проанализируем каждый из них и рассмотрим, какие выводы можно из них сделать, а также какие шаги предпринять при отладке системы в случае обнаружения подобных отклонений.

Паттерн 1

Этот паттерн демонстрирует расхождение кривой потерь, когда она не стремится к целевому значению, а колеблется (рис. 9.3). Как сделать процесс обучения более стабильным? Вот несколько вариантов:

- Проверьте, есть ли какая-то корреляция между признаками и целевыми значениями и передаются ли семплы и метки модели в правильном порядке.
- Снизьте скорость обучения, чтобы избежать «прыжков» модели по параметрам.
- Уменьшите размер датасета до одного батча (10–100 семплов) и проверьте, способна ли модель переобучиться на них.
- Начните с более простой модели и постепенно увеличивайте ее сложность. Каждый раз проверяйте, превосходит ли она константную базовую модель или модель, основанную на правилах.

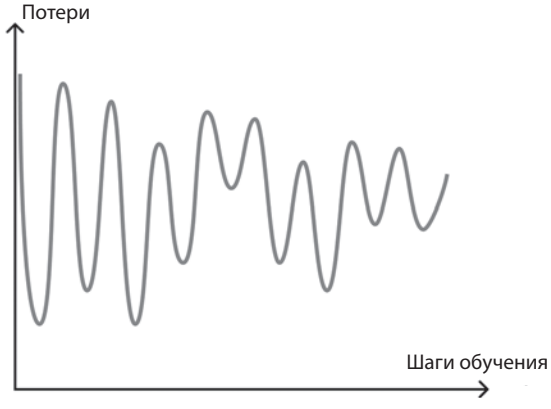


Рис. 9.3. Потери колеблются, что демонстрирует несходимость модели

Паттерн 2

Второй паттерн демонстрирует резкий взлет значений функции потерь или тренд в сторону NaN (нечисло) (рис. 9.4). Такое поведение указывает на вычислительные проблемы, возникающие либо из-за взрыва градиента (в этом случае можно попробовать обрезать градиент, снизить скорость обучения или изменить метод инициализации весов), либо на математические ошибки (например, деление на ноль, логарифм от нуля или отрицательного числа либо наличие NaN в данных — обычно это свидетельствует об ошибке в реализации или отсутствии предварительной обработки данных).

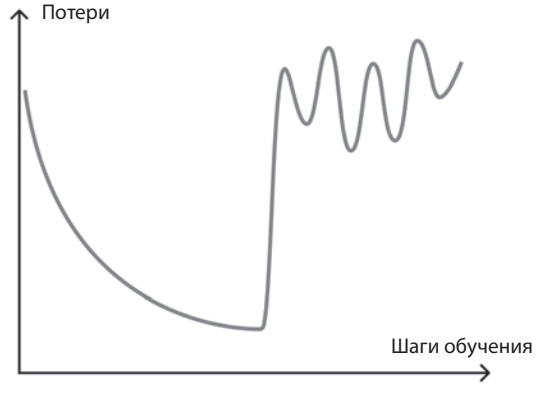


Рис. 9.4. Модель сходилась, но затем что-то пошло не так

Паттерн 3

В третьем паттерне значение функции потерь уменьшается, но метрика ведет себя противоречиво (рис. 9.5). Если модель продолжает улучшаться с точки зрения функции потерь, но метрика остается на месте, это может свидетельствовать о том, что выбранная метрика не подходит для решаемой проблемы или же неправильно реализована. Как правило, это характерно для задач классификации

и других смежных задач, в которых используются метрики с определенным порогом.

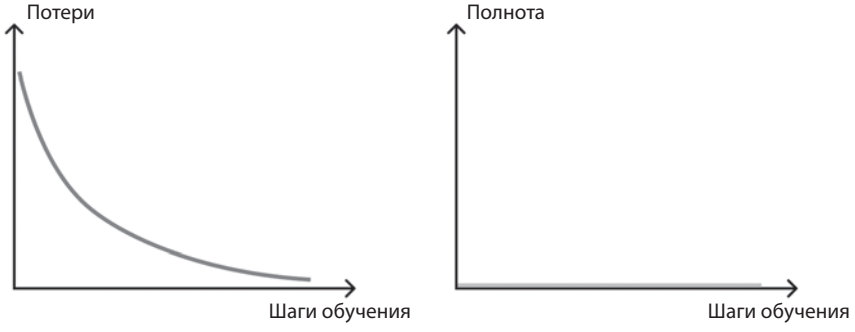


Рис. 9.5. Потери уменьшаются, но значение метрики остается стабильно низким

Байка от Валерия

Когда я работал в компании, занимавшейся разработкой мессенджера, перед нами поставили задачу усовершенствовать существующие системы защиты от спама и мошенничества. Основная сложность заключалась в том, что модель, обученная на заданном датасете, показывала неплохие метрики при офлайн-тестировании, но в продакшене вела себя иначе и совсем не оправдывала ожиданий.

После анализа возможных причин мы обнаружили три основные проблемы:

- Мы использовали некорректные метрики при офлайн-тестировании. Например, точность при заданном уровне полноты — не столь полезная метрика при обнаружении мошенничества, так как она чувствительна к классу (точность). Гораздо лучшие результаты давала специфичность при заданной полноте (см. главу 5). Однако сама по себе полнота — это своего рода кот Шредингера; у нас не было информации обо всех мошеннических случаях (так как мы пропускали часть из них и не знали, сколько именно), поэтому полноту можно было рассчитать лишь на подвыборке известных случаев мошенничества.
- Вторая проблема заключалась в методике оценки производительности, в которой мы основывались на точечных оценках, но в реальности постоянно наблюдались отклонения от конкретных значений. При масштабе 100 миллиардов событий в день даже 0.1 % отклонения означало бы неправильную классификацию 100 миллионов событий — существенно больше, чем мы рассчитывали.
- Третья проблема заключалась в том, что мы оценивали систему как задачу бинарной классификации (спам/не спам), используя логарифмическую функцию потерь. Однако на деле наша цель заключалась в том, чтобы пользователи были довольны, а объем спама был снижен до приемлемого уровня (спам неизбежен, и в допустимых объемах он не особо мешает, но слишком много спама — это проблема), исключая крайние случаи, например 100 000 сообщений от одного абонента в течение одной секунды.

Если первые две проблемы оказались в целом решаемы, то третья была чрезвычайно сложной и требовала правильной иерархии метрик, кастомной функции потерь и длительных экспериментов.

В этом заключается вся суть ML — мы обучаем модель с использованием определенной функции потерь, измеряем ее производительность по какому-то набору метрик в надежде достичь результатов, отличных от предыдущих показателей, и рано или поздно сталкиваемся с ситуаций, при которой одни показатели улучшаются, а другие не реагируют вовсе.

Главный урок, который мы извлекли из этого случая, заключается в следующем: даже если вы провели безупречный анализ ошибок, это все равно не спасет вас от провала, если вы изначально выбрали не те метрики.

Паттерн 4

Этот паттерн демонстрирует сходящуюся кривую обучения с неожиданными значениями функции потерь. Несмотря на то что форма кривой обучения выглядит довольно многообещающе, наблюдаемые значения вызывают недоумение. Чтобы заранее выявить подобные аномалии, рекомендуется провести первичную проверку — запуск простого юнит-теста на одном батче, который проверит, попадает ли значение функции потерь в ожидаемый диапазон. Часто причина подобной проблемы заключается в масштабирующих преобразованиях (например, нормализация изображения или маски при сегментации).

Паттерн 5

Такой паттерн показывает снижение значения функции потерь на обучающей выборке, в то время как значение функции потерь на валидационной выборке растет (рис. 9.6). Это классический пример переобучения, вызванного высокой дисперсией. В подобных случаях необходимо ограничить емкость модели, либо понизив ее сложность, либо усилив регуляризацию.

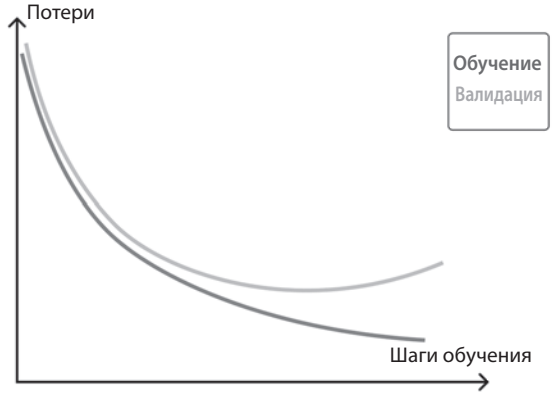


Рис. 9.6. Потери уменьшаются на обучении, но не на валидации, что указывает на возможное переобучение

9.1.4. Кривая обучения по модели

После того как мы убедились, что модель сходится и функция потерь на обучении достигает плато без явного переобучения или недообучения, можно завершить анализ кривой обучения и двигаться дальше, особенно если мы еще находимся на этапе первичного развертывания модели.

Однако если мы сталкиваемся с проблемами переобучения/недообучения или есть время поэкспериментировать с оптимальным размером модели, то понадобится второй тип кривой обучения (рис. 9.7).

1. Сначала выбираем гиперпараметр, отражающий изменяющуюся сложность модели. Это может быть, например, глубина дерева в градиентном бустинге, коэффициент регуляризации, количество признаков или количество слоев в глубокой нейросети.
2. Далее задаем сетку значений для этого гиперпараметра (например, 2, 3, 4, ..., 16 для глубины дерева; 10^{-2}, 10^{-1}, 1, 10, 10^{2}, 10^{3} для коэффициента регуляризации).
3. Обучаем каждую модель до сходимости и фиксируем итоговые значения функции потерь/метрики.

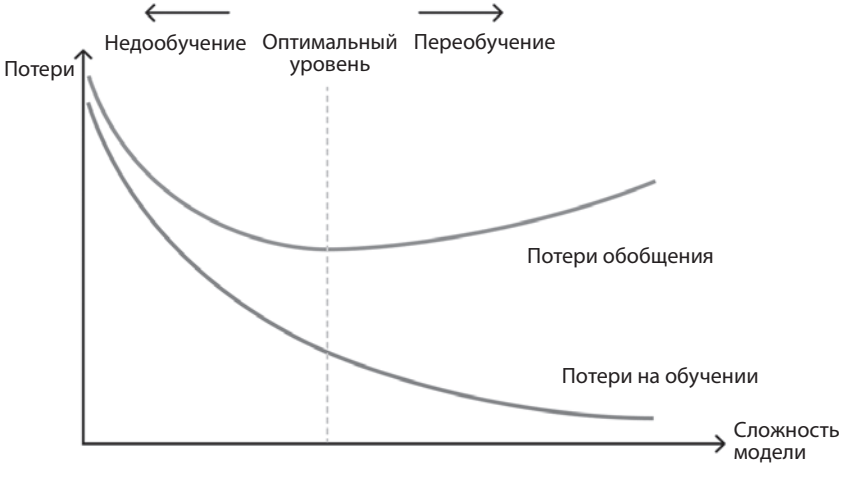


Рис. 9.7. Выбор оптимальной сложности модели на основе кривых обучения

Далее отображаем эти значения на вертикальной оси, а соответствующие значения гиперпараметра — на горизонтальной. Такая кривая обучения позволяет легко определить, какой диапазон сложности модели (определяемый выбранным гиперпараметром) является оптимальным для заданных данных.

9.1.5. Кривая обучения по объему выборки

Наконец, рассмотрим изменение размера датасета. Эта техника подробно описана в разделе 6.4 главы 6. Вкратце: мы сохраняем валидационную выборку неизменной и экспериментируем с различными объемами обучающей выборки — 100, 1000, 10 000 и т. д. Как и в случае с кривой обучения по модели, обучаем модель до сходимости и строим кривые для обучения и валидации.

Если провести экстраполяцию значения метрики на валидации, можно оценить, сколько новых данных потребуется для повышения метрики на 1 %, — и наоборот, если мы ожидаем собрать еще N семплов, то можем спрогнозировать, какое улучшение метрики это даст.

Помимо экстраполяции, кривая обучения по выборке также позволяет выявить переобучение и недообучение. Какие выводы можно сделать при анализе кривых обучения и валидации?

- Если кривые практически совпадают (разрыв между ними мал или отсутствует при максимальном числе семплов), модель уже хорошо обобщает, и добавление новых данных не повысит ее производительность.
- Если кривые почти совпадают, но значения функции потерь остаются высокими, это свидетельствует о высоком смещении (недообучении). В таком случае увеличение размера датасета тоже не поможет. Лучше использовать более сложную модель.
- Если между кривыми большой разрыв, это указывает либо на высокую дисперсию, либо на различия между обучающей и валидационной выборками. В первом случае следует понизить сложность модели или собрать больше данных. Во втором случае стоит пересмотреть метод разделения данных и убедиться, что он адекватно отражает реальные сценарии, с которыми столкнется модель.

Кривая обучения по выборке позволяет понять, ограничивает ли производительность модели объем доступных данных. Знание зависимости метрики от размера датасета помогает решить, что делать дальше — нужно ли собрать больше данных, сосредоточиться на генерации признаков или заняться тонкой настройкой гиперпараметров модели.

9.1.6. Двойной спуск

Компромисс между смещением и дисперсией стабильно работает для классических ML-моделей и глубоких нейросетей умеренного размера. Однако в случае современных сверхпараметризованных глубоких нейронных сетей ситуация становится значительно более запутанной.

Существует явление, известное как *двойной спуск* (double descent), при котором ошибка на тестовой выборке сначала уменьшается, затем возрастает, а после снова снижается. Удивительно, но исследователи обнаружили различные режимы двойного спуска, соответствующие всем трем видам кривых обучения: по эпохам, по размеру модели и по объему выборки.

Пока что остается открытым вопрос, каков механизм, лежащий в основе этого явления, почему оно возникает и означает ли это, что переобучение крупных нейросетей больше не является проблемой. Наиболее распространенная гипотеза, объясняющая двойной спуск, следующая:

- Если мощность модели (число параметров) меньше объема датасета, она стремится лишь аппроксимировать данные. Это приводит к классическому режиму, при котором и проявляется компромисс между смещением и дисперсией. Такую модель называют *недопараметризированной*.
- В условиях интерполяции модель обладает достаточной способностью идеально подгонять данные, достигая нулевого смещения. В пространстве параметров фактически существует только одна такая модель. Если заставить ее подстраиваться даже под слегка зашумленные метки, это разрушит ее глобальную структуру.
- Однако в *сверхпараметризованном* режиме существует множество моделей, способных интерполировать обучающую выборку. Некоторые из них не только хорошо подгоняют данные, но и демонстрируют отличную обобщающую способность на тестовой выборке. Оказывается, стохастический градиентный спуск каким-то образом приводит к таким «хорошим моделям», причины чего пока до конца неизвестны.

Дополнительную информацию можно найти в статье OpenAI «Deep Double Descent» (https://openai.com/research/deep-double-descent).

Современные законы масштабирования крупных нейросетей заставляют переосмыслить наши стратегии моделирования. Явление двойного спуска может удивить тех, кто знаком лишь с классическим компромиссом «смещение–дисперсия».

Тем не менее это явление необходимо учитывать при выборе модели (особенно при работе с крупными сверточными сетями и трансформерами), а также при определении подходов к обучению и отладке.

Мы упоминаем об этом в нашей книге, чтобы подчеркнуть следующую идею: большинство ранее описанных эвристик не являются непреложными законами. Как часто бывает в проектировании ML-модели, они скорее представляют собой некие сигналы — зачастую полезные, но не всегда подходящие для конкретной проблемы.

9.2. Анализ остатков

> Придумывать новые идеи, конечно, важно, но еще важнее — понимать полученные результаты.
>
> *Илья Суцкевер*

После того как мы убедились, что наша ML-модель сошлась и не страдает от недообучения или переобучения, следующим шагом в ее отладке становится проведение анализа остатков (residual analysis). Это предполагает сравнение отдельных прогнозов модели с соответствующими истинными метками. Анализ остатков заключается в вычислении разницы между спрогнозированными и эталонными значениями, называемой *остатками* (рис. 9.8):

$$residual_i = y_pred_i - y_true_i.$$

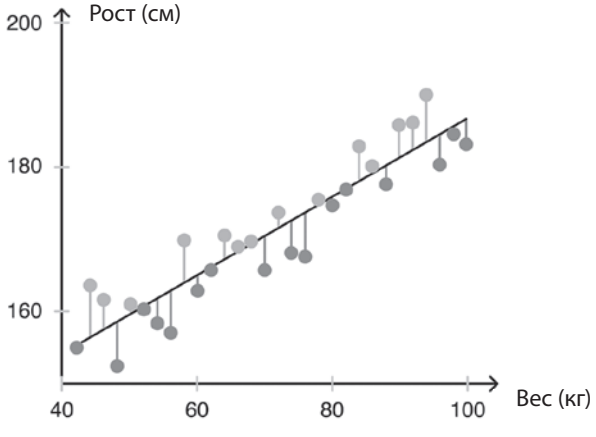


Рис. 9.8. Типичный пример — одиночный регрессор x. Каждая вертикальная линия представляет собой ошибку остатка (e), или просто остаток. Линии, расположенные выше регрессора, имеют отрицательное значение (фактическое значение выше прогноза), а линии ниже — положительное (прогноз выше фактического значения)

Прежде всего, что такое остатки? В узком смысле остатки — это просто разница между спрогнозированными и истинными значениями в задаче регрессии. В более широком понимании остатками можно назвать любые выборочные расхождения или ошибки между прогнозами модели и фактическими значениями.

Таким образом, чтобы согласовать остатки с конкретной функцией потерь, предпочтительнее использовать отдельный член суммы потерь в качестве псевдоостатка, а не просто разность. Такой псевдоостаток может быть квадратом ошибки для среднеквадратичной ошибки, логарифмической ошибкой для среднеквадратичной логарифмической ошибки или значением метки класса, умноженным на прогнозируемую вероятность этого класса для логистической функции ошибки (LogLoss).

Часто остатки ассоциируются исключительно с задачами регрессии и классификации. Но как быть с остатками вне этих задач? Если посмотреть шире, можно найти эквивалентные инструменты практически в любой задаче, связанной с ML.

Например, в контексте поисковых систем истинные метки представляют собой сопоставление поискового запроса с наиболее релевантными документами или товарами. В этом случае, чтобы вычислить остатки, необходимо измерить разницу между ранжированием модели и истинным ранжированием каждого элемента из топ-N списка (рис. 9.9).

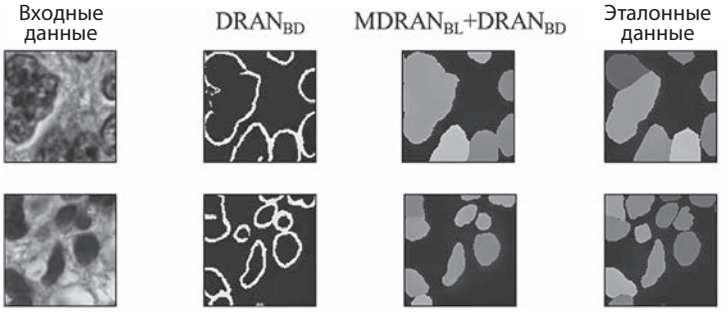


Рис. 9.9. Пример остатков для задачи сегментации изображений (источник изображения: https://arxiv.org/abs/1810.13230)

В задачах сегментации изображений можно вычислить разницу между прогнозируемыми масками и эталонными масками для каждого изображения. Это дает нам двумерные остатки, которые показывают, какие части объекта не покрыты маской или покрыты неверно.

9.2.1. Цели анализа остатков

Анализ остатков помогает выявлять закономерности в ошибках, допускаемых моделью, чтобы можно было определить конкретные направления для улучшения системы. В то время как общая ошибка модели обычно представляется одним числом — функцией потерь или метрикой, — анализ остатков работает наоборот: он исследует «сырые» расхождения между прогнозами и истинными метками, обеспечивая более детальную диагностику работы модели.

Кроме этого, анализ остатков преследует и другие важные цели:

- *Проверка предположений о модели.* Прежде всего он позволяет подвергнуть сомнению базовые допущения, лежащие в основе модели. Соответствуют ли остатки нормальному распределению? Имеют ли прогнозы модели систематическое смещение? Если обнаруживаются существенные отклонения, это может свидетельствовать о необходимости пересмотра подхода или выбора другой модели.

- *Выявление источников изменения метрик.* Общая производительность может улучшаться или оставаться неизменной. Тем не менее мы можем зафиксировать существенные изменения в распределении метрик. Какие семплы демонстрируют различия в распределении остатков для разных моделей? В каком подмножестве данных содержится наибольшее число ошибок? Какие примеры сильнее всего влияют на итоговую метрику?
- *Обеспечение справедливости распределения остатков.* Анализ остатков позволяет оценить, насколько модель справедливо относится к разным семплам и одинаково ли распределяет ошибки между различными когортами. Если выявлены существенные перекосы или диспропорции, можно внести соответствующие корректировки, чтобы модель была беспристрастной и одинаково обрабатывала все примеры.
- *Анализ наихудших и наилучших случаев.* Есть ли общие черты у топ-N примеров с наибольшими остатками? Что можно изменить, чтобы улучшить поведение модели в этих случаях? А что общего у примеров с наименьшими остатками?
- *Исследование крайних случаев.* Как модель работает с пользователями с самой короткой или самой длинной историей либо с самыми короткими/длинными аудиозаписями, текстами и сессиями — в зависимости от решаемой задачи? Как она справляется с объектами с самой низкой/высокой ценой, нулевыми остатками или наибольшей выручкой? Для адекватной оценки всех потенциальных рисков необходимо хорошо понимать бизнес-контекст и природу данных.

Данные вопросы лежат в основе анализа остатков, и поиск ответов на них завершает цикл обратной связи в процессе офлайн-оценки. Чем раньше мы начнем фиксировать сложные примеры (и кривые потерь) при проведении экспериментов, тем лучше. Лучше всего сохранять 10–20 объектов с наибольшими остатками после обучения модели в качестве сопроводительных артефактов. Трудно переоценить важность включения подобных шагов в описание пайплайна обучения в дизайн-документе.

На более поздних этапах проекта данные шаги становятся частью автоматизированных отчетов по каждой обученной модели наряду с мониторингом смещения модели и отчетами о качестве данных. Допустим, метрика улучшилась в большинстве сегментов, но немного снизилась в ключевом сегменте. В зависимости от принятой политики мы можем либо отклонить эту версию модели, либо уделить повышенное внимание данному изменению в последующих итерациях.

9.2.2. Предположения о модели

Какую бы модель мы ни обучали, у нас всегда есть априорные предположения относительно ее прогнозов, смещений и распределения остатков. Проверка этих предположений позволяет убедиться в том, что выбрана подходящая модель,

собрано достаточное количество данных и корректно реализована генерация признаков. Если в процессе проверки выявляется неожиданная закономерность, это может побудить к поиску альтернативных решений.

С точки зрения проектирования важно заранее определить, что именно мы предполагаем об истинности прогнозов модели или, в частности, об остатках, и выразить эти предположения в виде соответствующих юнит-тестов. Это позволит избежать неожиданного поведения модели после дальнейшего развертывания. В следующей главе мы более подробно поговорим о тестировании и его роли в процессе обучения. А пока рассмотрим два разных примера, чтобы понять, как можно использовать предположения.

Пример 1. Предположения в линейной регрессии

Предположим, что мы решаем проблему прогнозирования спроса с помощью простой линейной регрессии. Какие ключевые предположения мы при этом делаем?

- *Линейность*: взаимосвязь между предикторами (x) и целевой переменной (y) является линейной.
- *Строгая экзогенность*: остатки должны быть центрированы относительно нуля.
- *Нормальность*: предполагается, что остатки распределены нормально.
- *Гомоскедастичность*: дисперсия остатков одинакова для любых значений X.
- *Независимость*: ошибки остатков должны быть независимыми.

После обучения модели мы проверяем, выполняются ли эти предположения. Возможные проблемы включают:

- *нелинейность* во взаимосвязи $X - Y$;
- *смещение* в остатках;
- *гетероскедастичность*: переменная дисперсия ошибок;
- наличие точек данных с *чрезмерным влиянием*: выбросы среди прогнозируемых значений (y) или признаков (x).

В целях проверки предположений линейной регрессии мы анализируем распределение остатков. Для этого выстраиваем остатки четырьмя различными способами и создаем так называемые *диагностические графики* (рис. 9.10 и 9.11):

- *Остатки vs. предсказанные значения* (residuals vs fitted). Используется для оценки линейности взаимосвязи. Горизонтальная линия без выраженных паттернов указывает на линейную зависимость, что является благоприятным признаком. Отсутствие различий между сплошной и пунктирной линиями говорит о сильной линейной зависимости.
- *Нормальный квантиль-квантильный график* (Q-Q plot) для проверки нормальности остатков. На оси X откладываются квантили стандартного нор-

мального распределения, на оси Y — квантили стандартизованных остатков (остатки после вычитания среднего значения и деления на стандартное отклонение). Если полученные точки близки к прямой линии (пунктирная линия на графике), то остатки подчиняются нормальному распределению.

- *График масштаба–положения* (scale-location) для оценки однородности дисперсии среди остатков. Горизонтальная линия с равномерно распределенными точками — это признак гомоскедастичности. В нашем примере это условие не выполняется: наблюдается гетероскедастичность, при которой более высокие предсказанные значения сопровождаются большей дисперсией.



Рис. 9.10. Четыре диагностических графика для анализа остатков линейной регрессии в обоих примерах (случай 1: предположения выполняются)



Рис. 9.11. Четыре диагностических графика для анализа остатков линейной регрессии в обоих примерах (случай 2: предположения не выполняются)

- *Остатки vs. рычаг влияния* (residuals vs leverage) для выявления влиятельных наблюдений, то есть экстремальных значений, которые могут существенно повлиять на результаты регрессии при их включении или исключении из анализа. *Рычаг* отражает степень, в которой коэффициенты в регрессионной модели изменились бы при удалении конкретного наблюдения из датасета. Мера, используемая для оценки влияния точек данных, называется расстоянием Кука.

Иногда бывает полезно заставить модель строже придерживаться наших предположений, напрямую внедрив в нее априорные знания.

Пример 2. Карта внимания

Представьте, что вы — владелец банковского приложения и в следующем крупном обновлении вы планируете добавить голосового помощника. Посоветовавшись со своими специалистами, вы нанимаете Стейси — мирового эксперта в области систем синтеза речи (text-to-speech, TTS). Спустя несколько недель работы Стейси представляет первую версию системы.

В задачах синтеза речи существует фундаментальное предположение: порядок символов в тексте должен линейно соответствовать временной последовательности в аудиосигнале. Когда мы читаем текст, вполне естественно предполагать, что позиция текста тесно согласована с тем, что мы слышим. Это предположение отличается от других задач типа «последовательность–последовательность», таких как машинный перевод, где модуль внимания необходим для установления соответствия между словами в языках с разным синтаксисом или порядком токенов, например, в английском и китайском языках.

Для оценки обоснованности данного предположения Стейси использует *карту внимания*, отображающую карту активации между аудиокадрами (ось X) и символами (ось Y). Наблюдая за этой картой на регулярной основе в процессе обучения, Стейси оценивает, насколько она приближается к диагональной матрице.

Чтобы заставить матрицу внимания принять почти диагональный вид, Стейси применяет метод, известный как *направленное внимание* (guided attention). Каждый раз, когда матрица внимания существенно отклоняется от диагонали, за это на нее накладывается штраф с использованием дополнительной функции потерь. Данный прием не только ускоряет процесс обучения, но и направляет модель к осмысленным решениям, с самого начала соответствующим заданному предположению (рис. 9.12).

Отклонение карты внимания от диагональной матрицы и составляет остатки. Остатки умеренного размера допустимы: одни символы люди произносят быстрее других, поэтому график не будет идеально прямолинейным. Однако большие остатки указывают на конкретные звуки или сочетания символов, которые модель не может выучить должным образом.

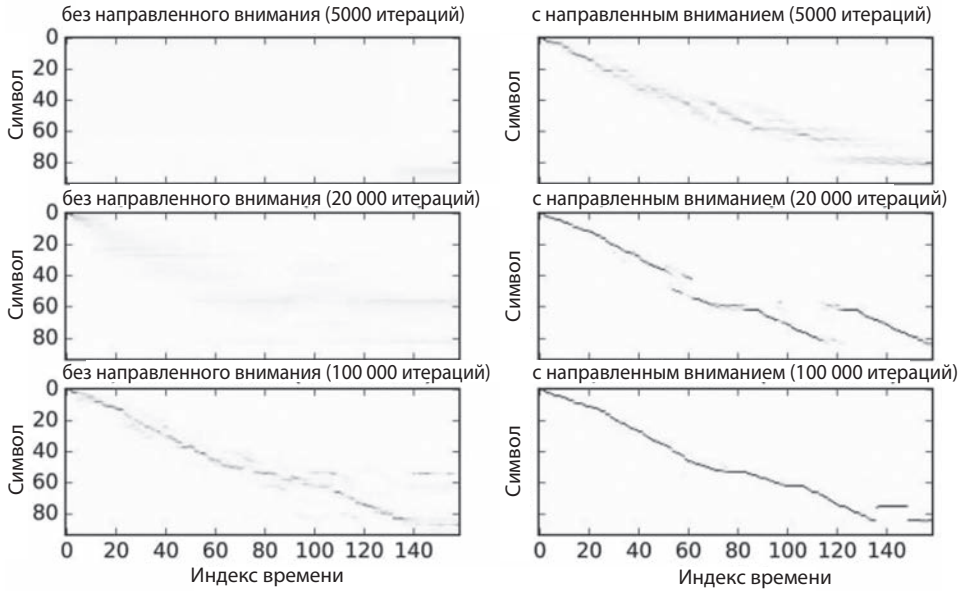


Рис. 9.12. Изменение карты внимания в процессе обучения без использования (*слева*) и с использованием (*справа*) функции потерь направленного внимания

Обладая этой информацией, Стейси может спланировать последующую стратегию сбора данных, чтобы устранить проблемы модели и повысить ее качество.

Узнать больше об архитектуре типичной сети TTS, включая детали построения модуля внимания, можно в статье «Efficiently Trainable Text-to-Speech System Based on Deep Convolutional Networks with Guided Attention» (https://arxiv.org/abs/1710.08969).

9.2.3. Распределение остатков

Если ни одно из предположений о модели не выполняется, это говорит о необходимости пересмотреть процесс обучения, собрать дополнительные данные, добавить новые признаки или изучить альтернативные модели и функции потерь. Но как определить, что именно нужно сделать? Конечно, можно просто действовать наугад, но это все же не лучший вариант.

Рассмотрим, например, ситуацию, когда нарушается предположение о нормальности распределения в линейной регрессии (рис. 9.13–9.15).

В этом примере нам повезло и мы сразу видим, в чем проблема модели:

- *Случай 2*. Похоже, мы не учитываем распределение целевой переменной. Такие величины, как доход, объемы продаж и цены, следуют логарифмически нормальному распределению, тогда как среднеквадратичная ошибка

или средняя абсолютная ошибка, которые минимизируются моделью, нам не подходят (по крайней мере, напрямую). В такой ситуации часто помогает логарифмическое преобразование целевой переменной.

- *Случай 3*. Остатки формируют несколько кластеров. В этом случае преобразование целевой переменной не принесет никакой пользы. Зависимость целевой переменной от признаков немонотонна. Что может помочь: либо использование модели, способной улавливать немонотонные зависимости, либо генерация новых признаков, которые позволят линейной модели свести немонотонные зависимости к монотонным или даже линейным.



Рис. 9.13. В случае 1 наблюдается нормальное распределение остатков, при котором предположение линейности выполняется



Рис. 9.14. В случае 2 наблюдается ненормальное распределение остатков, при котором предположение линейности не выполняется из-за логарифмически нормального распределения целевой переменной (в распределении отчетливо видна асимметрия)



Рис. 9.15. В случае 3 наблюдается ненормальное распределение остатков в линейной регрессии, при котором предположение линейности не выполняется из-за немонотонной зависимости целевой переменной от регрессоров

9.2.4. Справедливость остатков

В ML справедливость (fairness) — это показатель неравенства в данных. Как отдельные семплы влияют на значение функции потерь или метрики? Какой ценой мы повышаем метрику? Вносит ли новая модель больше неравенства в распределение остатков или, наоборот, снижает его? По сути, *справедливость* — это еще один способ описать асимметрию распределения остатков.

Не каждое изменение метрики подразумевает равномерность. Некоторые улучшения распределяются равномерно по всем семплам, а другие — обеспечивают значительный прирост в одной группе и снижение в другой. Концепция справедливости в анализе остатков приводит нас к более целостной процедуре оценки модели, выходящей за рамки единственной числовой метрики.

Чтобы лучше понять, что такое справедливость, рассмотрим рис. 9.16. В этом упрощенном примере у нас есть две модели. Обе снижают среднюю абсолютную ошибку (MAE) на 20 %. Однако мы предпочитаем развернуть первую модель, так как она равномерно снижает абсолютные остатки по всем 10 семплам. В противоположность ей, вторая модель резко улучшает метрику для одной половины выборки и ухудшает — для другой. В этом случае мы вносим неравенство в распределение остатков, поэтому такое распределение называется несправедливым.

Один из способов количественной оценки справедливости — вместо опоры только на визуализацию — это использование коэффициента Джини из экономики (рис. 9.17). Для его вычисления необходимо рассортировать остатки по абсолютному значению, затем вычислить совокупную долю абсолютных значений и поделить ее на совокупную долю числа остатков.

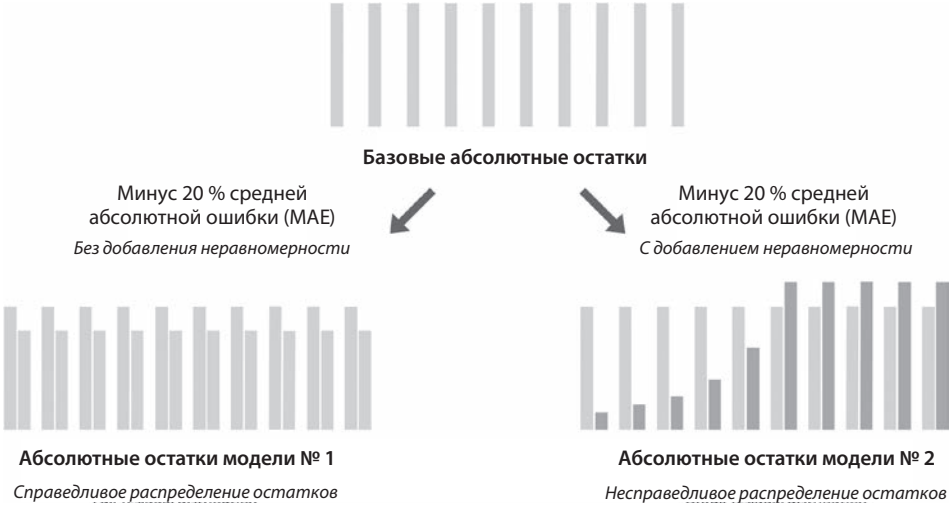


Рис. 9.16. Справедливое и несправедливое распределение остатков



Рис. 9.17. Графическое представление коэффициента Джини: график показывает, что коэффициент равен площади, обозначенной буквой A, деленной на сумму площадей A и B, то есть Джини = A / (A + B). Также это выражение эквивалентно 2A и 1 − 2B, поскольку A + B = 0.5 (так как шкалы по осям варьируются от 0 до 1)

При полной справедливости (Джини = 0.0) почти все остатки вносят одинаковый вклад в общую ошибку. При полном неравенстве (Джини = 1.0) вся ошибка сосредоточена в одном остатке. Это два крайних случая, которые редко встречаются в реальной практике. На деле же значения коэффициента Джини обычно находятся где-то между ними.

Существует две основные причины, почему стоит заботиться о справедливости. Во-первых, мы хотим, чтобы модель показывала высокие результаты для всех пользователей, объектов или других структур, на которых она будет применяться, а не только для части из них. Это также подразумевает

последовательное снижение общего неравенства в остатках на каждой итерации системы.

Во-вторых, наша цель — не только улучшить общие показатели ошибки, но и уменьшить каждый отдельный остаток. Если качество поисковой системы увеличивается на 5 %, но при этом прогнозы ухудшаются на 20 % для определенного сегмента пользователей, это негативно сказывается на их пользовательском опыте. Таким образом, выгода от повышения среднего качества может быть легко нивелирована ростом оттока этих пользователей.

В долгосрочной перспективе мы стремимся к максимально равномерному улучшению по всем стратам. Однако из каждого правила есть исключения, и справедливость не является критически важной для каждого проекта и каждой метрики. Тем не менее мы должны обращать внимание на «хвосты» распределения остатков, поскольку именно они определяют нашу стратегию выбора между улучшением в среднем и улучшением по отдельным семплам.

9.2.5. Недооценка и переоценка

В задачах регрессии остатки часто разделяются по знаку: положительные остатки указывают на переоценку (overprediction) (прогнозируемые значения выше фактических), а отрицательные — на недооценку (underprediction).

В зависимости от решаемой задачи может быть предпочтительна одна или другая склонность модели. Например, если мы разрабатываем систему прогнозирования спроса, упущенная прибыль является менее желательным исходом, чем умеренные излишки. С другой стороны, если мы прогнозируем платежеспособность клиента для банка, то безопаснее недооценить ее, чем переоценить.

Таким образом, цена ошибки часто является асимметричной и указывает, остаткам какого знака и какого масштаба следует уделять наибольшее внимание.

9.2.6. Кривые эластичности

Одним из особых инструментов анализа ошибок в прогнозировании спроса является график эластичности. Это не универсальный инструмент, применимый к любой ML-системе, но поскольку он крайне важен для задач, связанных с ценообразованием, то заслуживает отдельного внимания. Хотя большинство видов криволинейного анализа уже были рассмотрены ранее, этот пример стоит отнести к текущему разделу, так как он представляет собой частный случай анализа остатков.

Одним из основных предположений о модели в прогнозировании спроса является зависимость между ценой и спросом: чем выше цена, тем ниже спрос, и наоборот. Это справедливо практически для всех типов товаров (за исключением особых случаев, таких как товары Веблена и Гиффена, если вы знакомы с основами микроэкономики).

График эластичности представляет собой частный случай более общего концепта, называемого *графиком частичной зависимости*, в котором варьируются некоторые признаки и анализируется, как изменяется прогнозируемый результат. Он используется для интерпретируемости модели (что будет подробно рассмотрено в главе 11).

Существует два способа построения кривой эластичности для нашей модели:

- Использование обучающих данных (с известными фактическими ценами):
  - взять историю продаж конкретного SKU;
  - выполнить прогноз продаж для каждой точки данных (Y);
  - взять историческую цену для каждой точки данных (X);
  - построить зависимость X–Y (цена → прогнозируемые продажи).
- Использование комбинации синтетических и реальных данных:
  - взять последнюю цену для конкретного SKU;
  - умножить эту цену на различные коэффициенты (–20 %, –19 %, –18 %, ..., +19 %, +20 %);
  - пересчитать все признаки, зависящие от цены, и спрогнозировать продажи (Y) для каждой новой строки;
  - построить зависимость X–Y (цена → прогнозируемые продажи).

Как уже упоминалось ранее, в идеале график демонстрирует обратно пропорциональную зависимость: чем выше цена, тем ниже продажи. Однако если график показывает противоположное, является шумным (частично или полностью немонотонным) или содержит какие-либо другие спорные закономерности, это сигнализирует о следующих возможных проблемах:

- У нас недостаточно вариативности цен для данного SKU, чтобы уловить его эластичность (например, слишком короткая история продаж).
- Продажи этого SKU слишком случайны. Например, на них часто влияют промоакции, сезонность или другие внешние факторы.
- Модель по какой-либо причине не может уловить зависимость (так называемый трудный случай).

Чем «лучше» график (чем он более монотонный в отрицательном направлении), тем больше можно полагаться на прогнозы модели для соответствующих SKU. Если же эластичность «плохая», это означает, что прогноз ненадежен, и такие «трудные» SKU следует дополнительно исследовать, а не использовать в модели (рис. 9.18 и 9.19).

Построение графика эластичности не только для прогнозируемых, но и для фактических продаж также может быть полезным. Это помогает понять, проявляет ли данный конкретный уникальный складской идентификатор товара

(SKU) ярко выраженную ценовую эластичность. Если не проявляет, то не стоит ожидать этого и от прогноза спроса.

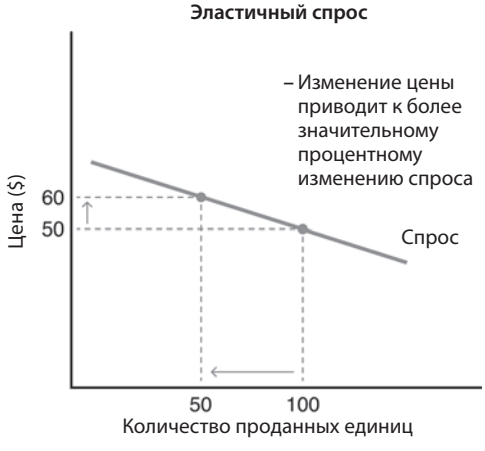


Рис. 9.18. Типичный пример эластичного спроса

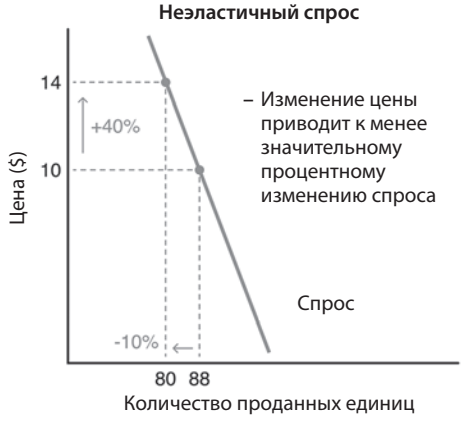


Рис. 9.19. Типичный пример неэластичного спроса

Если вы хотите более подробно ознакомиться с темой ценовой эластичности, рекомендуем прочитать статью «Forecasting with Price Elasticity of Demand» (https://mng.bz/GN5v).

Один наш знакомый недавно рассказал свою байку о том, как он использовал график эластичности на практике. Они работали над проблемой ценообразования, и их решение, по сути, представляло собой «улучшенный» график эластичности. Они построили модель градиентного бустинга, которая прогнозировала количество проданных единиц на основе различных признаков, включая ценовые, и оценивала продажи при разных возможных ценах. Первая версия модели показала неожиданный результат: вместо плавной зависимости

график демонстрировал отчетливую «лестницу» из ступеней. После более глубокого анализа выяснилось, что причина появления таких ступеней заключалась в особенностях использованных признаков: непрерывные переменные были разбиты на категории с низкой кардинальностью (например, всего 256 бакетов для всех возможных цен по всем товарам на маркетплейсе), что ограничивало чувствительность модели. После увеличения числа бакетов модель смогла выявлять более тонкие закономерности, кривая эластичности сгладилась, а производительность всей системы улучшилась.

9.3. Поиск закономерностей в остатках

Теперь, когда мы рассмотрели распределение остатков в целом, пришло время изучить закономерности и тренды в подгруппах остатков. Мы можем подойти к этой задаче с двух сторон (рис. 9.20):

- сгруппировать семплы по их остаткам и проанализировать признаки внутри каждой группы;
- сгруппировать семплы по их признакам и проанализировать остатки внутри каждой группы.

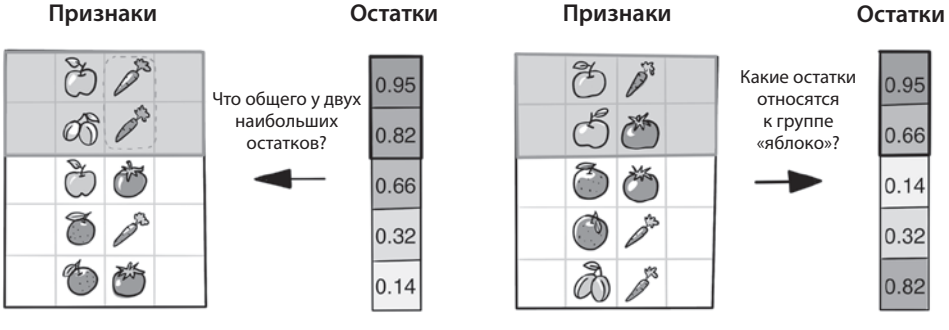


Рис. 9.20. Два подхода к поиску закономерностей в остатках: выбор подмножеств по остаткам или по признакам семплов

Группировка остатков по их значениям включает анализ наихудших и наилучших случаев, а также затрагивает проблемы переоценки и недооценки. Группировка остатков по признакам позволяет провести групповой анализ и анализ крайних случаев.

9.3.1. Анализ наихудших/наилучших случаев

Цель анализа наихудших/наилучших случаев — определить типичные ситуации, в которых модель работает хорошо, а также те случаи, в которых модели доверять не следует. Что общего у таких остатков?

Байка от Валерия

Возвращаясь к случаю, упомянутому в предыдущих главах: однажды мы ввели систему динамического ценообразования на крупном маркетплейсе, которая устанавливала цену на товар на основе прогнозируемого спроса. Со временем мы стали сталкиваться с ситуациями, когда система не справлялась с точным прогнозированием продаж. Мы решили сосредоточиться на 200 товарах с наибольшими остатками (residuals) и вскоре выявили наиболее выраженные проблемы.

- Первая проблема касалась новых товаров, недавно добавленных в ассортиментную матрицу маркетплейса, и известна как *«проблема холодного старта»*. Данные позиции представляли сложность, поскольку отсутствие исторических данных мешало точному прогнозированию. Стало ясно, что полагаться исключительно на нашу модель в таких случаях недостаточно. Вместо этого нам пришлось использовать данные продаж аналогичных товаров в той же категории для получения более надежных прогнозов.
- Вторая проблема возникла в категории электронной техники. Продажи таких товаров были раскиданы во времени, что затрудняло использование прогнозов модели. Осознав это, мы приняли важное решение — исключить эти товары из пилотной версии модели и рассмотреть альтернативные подходы к улучшению качества прогнозов. Нам пришла идея разделить модель по крупным категориям, это должно было позволить лучше учитывать специфику динамики каждой группы.
- Однако самые большие проблемы доставляли маркетинговые активности — крупные распродажи, промокоды и скидки. Модель испытывала затруднения с учетом этих факторов, что приводило к заметной недооценке, выражавшейся в виде крупных отрицательных остатков. Чтобы устранить такое смещение, мы добавили в набор признаков календарь промоакций. Это позволило модели корректировать свои прогнозы с учетом акций, что повысило их точность.

Помимо выявления сфер, в которых модель давала сбои, мы также исследовали остатки, близкие к нулю, чтобы определить границы применимости модели. Такой анализ позволил нам понять, в каких ситуациях можно уверенно полагаться на прогнозы модели, когда ее качество снижается, но все еще допустимо, а когда доверять ее прогнозам не стоит вовсе.

Анализируя такие закономерности в остатках, мы получили подробное представление о сильных и слабых сторонах нашей системы динамического ценообразования, что позволило нам принимать обоснованные решения о масштабировании и правильном использовании модели.

9.3.2. Адверсариальная валидация

Если анализ наихудших случаев вручную не дает никакой новой информации, вам может помочь «модель машинного обучения для анализа модели машинного обучения». В главе 7 мы обсуждали концепцию, известную как адверсариальная валидация (adversarial validation). Она пришла из соревнований по

ML и применяется для проверки различий в распределениях двух датасетов. Обычно мы объединяем обучающую и тестовую выборки, присваивая им метки 0 и 1 соответственно.

Адверсариальная валидация легко адаптируется к анализу остатков: мы задаем 0 для «хороших» семплов и 1 для «плохих» (например, берем N % наибольших остатков во втором случае). Стоит попробовать разные пороговые значения для конкретной выборки.

Далее алгоритм аналогичен: мы обучаем простой классификатор (например, логистическую регрессию) на этих метках и вычисляем площадь под кривой характеристик качества классификации (ROC AUC). Если классы разделимы (площадь под кривой значительно больше 0.5), мы анализируем веса модели, что позволяет понять, какие именно признаки лучше всего отличают наши «худшие» случаи от остальных.

Иногда при анализе наихудших случаев не удается выявить какие-либо четкие закономерности. Это нормально и означает, что мы уже устранили наиболее очевидные проблемы и использовали все возможности для простого улучшения модели.

9.3.3. Разнообразие группового анализа

Групповой анализ позволяет выявить характерные закономерности и тренды в остатках для различных групп, сегментов, классов, когорт или кластеров. Например, в задачах как бинарной, так и многоклассовой классификации эффективный подход — разбить остатки по классам (то есть по целевой переменной), так как это дает возможность проводить отдельный анализ остатков для каждой группы.

Многие типовые приложения, работающие с табличными данными, такие как системы обнаружения мошенничества, опираются на группировку семплов на основе определенных характеристик, например географического положения или источника трафика. Анализ остатков внутри каждого сегмента позволяет выявить типичные смещения, свойственные прогнозам модели. Его результаты подскажут вам, как можно доработать модель — за счет добавления более релевантных семплов или корректировки весов уже существующих признаков.

При работе с данными в формате текста, изображений или видео зачастую изначально отсутствуют явно выраженные когорты или группы. В таких случаях альтернативным решением может стать классификация набора из N остатков вручную и присвоение меток каждому обнаруженному типу ошибки (например, изображения слишком темные или размытые, тексты содержат специфическую лексику). Данный процесс позволяет выявить определенные проблемные кластеры, в которых модель работает хуже всего. Соответственно, он указывает, какие именно типы данных следует собирать в дальнейшем для улучшения системы.

9.3.4. Анализ крайних случаев

Анализ крайних случаев проверяет поведение модели в редких или нестандартных обстоятельствах. Для этого желательно иметь бенчмарк — заранее составленный список всех крайних случаев, позволяющий быстро проверять каждую новую версию модели.

Вот что можно проверять в разных моделях:

- *Модели прогнозирования*: пользователи/товары с короткой или отсутствующей историей, пользователи/товары с крайне большим числом действий/продаж, предельные значения признака X, пользователи/товары с редкими действиями/продажами.
- *Сегментация изображений*: изображения плохого качества, низкого и высокого разрешения, наличие окклюзий и бликов, нестандартные условия освещения, несколько объектов на изображении, отсутствие объектов на изображении.
- *Языковые модели*: самые короткие и самые длинные тексты, шутки, оскорбительные темы, простая арифметика, тексты с опечатками, тексты на разных языках, чрезмерное использование эмодзи.
- *Распознавание речи*: самый короткий и самый длинный аудиофайл, плохое качество звука, аудиозапись без речи, музыка вместо речи, слишком быстрое или медленное произношение, шум на фоне, тихая речь, записи с несколькими говорящими (так называемая коктейльная вечеринка).

Если анализ лучших и худших случаев отвечает на вопрос, с какими данными модель справляется лучше или хуже всего, то анализ крайних случаев и когорт отвечает на вопрос, как модель работает на заранее заданных подвыборках данных.

Байка от Арсения

Когда я работал в компании, занимающейся дополненной реальностью, одной из важных частей системы был детектор ключевых точек, основанный на глубокой нейросети. Его задачей было определять несколько ключевых точек, которые затем использовались для определения координат объекта. Пайплайн обучения с самого начала был реализован с корректной системой диагностики, и поэтому мы еще на раннем этапе обнаружили, что определенные примеры с высоким значением функции потерь имели одну общую черту: на некоторых изображениях присутствовали зеркала или другие отражающие поверхности (даже лужа в дождливый день!), и модель могла перепутать ключевую точку с ее отражением. Поэтому нам потребовались дополнительные функциональные свойства от системы: выбирать реальный объект, запоминать его между кадрами и игнорировать ключевые точки, принадлежащие отражениям. Раннее понимание этой проблемы помогло нам сконструировать решение, снижающее влияние отраженных ключевых точек.



Модель распознает настоящую ногу, а не ее отражение

9.4. Дизайн-документ: анализ ошибок

Мы считаем, что анализ ошибок должен входить в число ключевых компонентов проектирования ML-систем, а поэтому включаем данный этап в оба дизайн-документа.

9.4.1. Анализ ошибок для Supermegaretail

Начнем с Supermegaretail, где мы работаем над системой, которая позволит компании достичь своей основной цели — минимизировать разрыв между доставленными и проданными товарами, при этом избегая ситуации дефицита каких-либо товаров на складе.

ДИЗАЙН-ДОКУМЕНТ: SUPERMEGARETAIL

VI. АНАЛИЗ ОШИБОК

Напомним, у нас имеется шесть функций потерь по квантилям для 1.5, 25, 50, 75, 95 и 99 квантилей целевой переменной и соответствующие шесть моделей для каждого из них. Константная базовая модель оценивает данные квантильные значения для каждого товара на основе продаж за последние *N* дней. Даже у этих базовых моделей наблюдается распределение остатков с определенным смещением, что необходимо учитывать.

Сравнение более сложных моделей (линейных и градиентного бустинга) с этими простыми базовыми моделями позволит понять, правильно ли мы выстраиваем модель и разрабатываем признаки.

1. Анализ кривой обучения

I. АНАЛИЗ СХОДИМОСТИ

Пошаговая кривая обучения, основанная на числе итераций, становится актуальной только на этапе экспериментов с алгоритмом градиентного бустинга. Основные вопросы, на которые следует ответить при ее изучении:

- Сходится ли модель вообще?
- Превосходит ли она базовые метрики (функция потерь по квантилям, средняя абсолютная процентная ошибка и др.)?
- Есть ли признаки недообучения или переобучения?

После подтверждения сходимости модели можно выбрать достаточное количество деревьев по грубой сетке (500–1000–2000–3000–5000) и зафиксировать это значение для последующих экспериментов. Для более простых базовых моделей анализ сходимости не требуется.

II. СЛОЖНОСТЬ МОДЕЛИ

Кривая обучения по модели поможет определить оптимальное число признаков и общую сложность модели.

Предположим, мы фиксируем все гиперпараметры, кроме количества используемых лагов: чем больше лагов мы берем, тем более сложные паттерны и сезонные колебания может уловить модель и тем легче она переобучится на обучающих данных. Должны ли это быть $N - 1$, $N - 2$, $N - 3$ дней? Или $N - 1$, $N - 2$, ..., $N - 30$? Ответить на данный вопрос нам поможет график «размер модели — величина ошибки».

Аналогично можно оптимизировать и размер окон. Например, окна 7/14/21/... более детализированы, чем 30/60/90/... . Подходящий уровень детализации также выбирается с помощью кривой обучения по модели.

Таким же образом на этапе первичной настройки можно регулировать и другие ключевые гиперпараметры модели, например, коэффициент регуляризации.

III. РАЗМЕР ДАТАСЕТА

Нужно ли использовать все доступные данные для обучения модели? За сколько последних месяцев данных будет достаточно и какие из них релевантны? Нужно ли использовать все точки данных (день, магазин, товар) или можно выполнить даунсемпинг (сокращение) выборки на 20 % / 10 % / 5 % без существенного ухудшения метрик?

Здесь нам поможет анализ кривой обучения по выборке, который определяет, сколько примеров необходимо для достижения плато по ошибке на валидационной выборке. Следует также принять важное решение: использовать ли (день, магазин, товар) в качестве единицы выборки или перейти к менее детализированной структуре (неделя, магазин, товар). Последний вариант уменьшает объем вычислений в 7 раз, при этом производительность модели может остаться прежней или даже улучшиться.

Данное решение влияет не только на скорость и производительность сервиса прогнозирования спроса, но и на конечный продукт (систему управления запасами), кардинально изменяя спектр возможных сценариев его использования. Поэтому, несмотря на потенциальные преимущества, данное решение должно быть согласовано с нашими продакт-менеджерами, конечными пользователями (категорийными менеджерами) и другими стейкхолдерами.

2. Анализ остатков

Напомним, что в нашей задаче используется асимметричная функция издержек: излишки товара менее критичны, чем дефицит. В первом случае мы имеем дело с просроченной продукцией, во втором — с упущенной прибылью. Проблема неудовлетворенного спроса — гораздо более нежелательный сценарий, который в долгосрочной перспективе приводит к недовольству клиентов и вероятности их перехода к конкурентам.

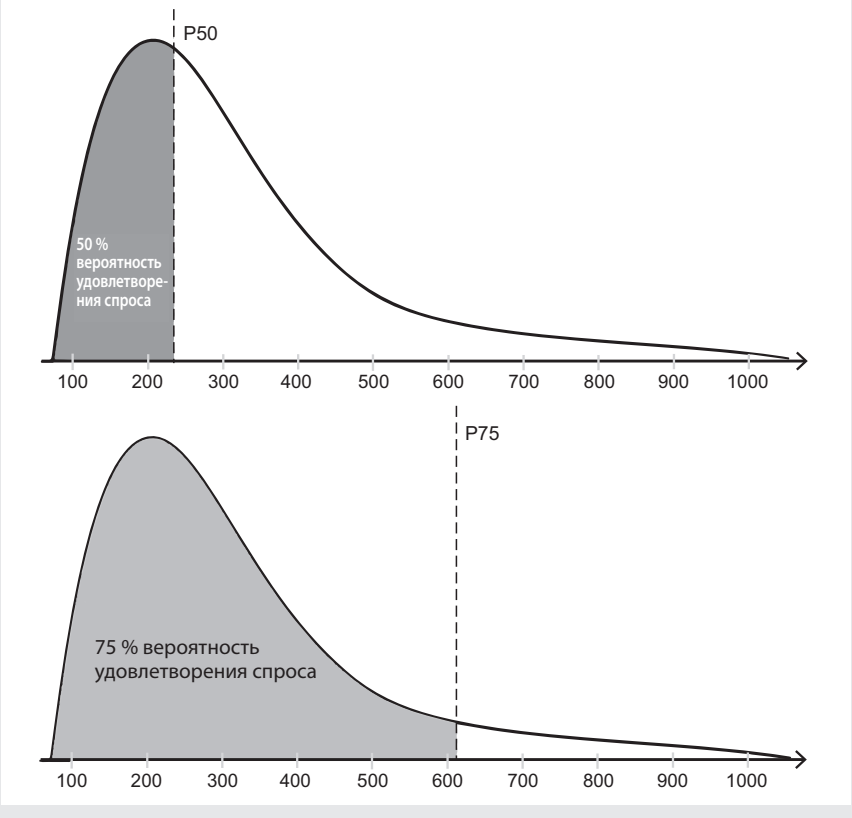
I. РАСПРЕДЕЛЕНИЕ ОСТАТКОВ

Указанная особенность спроса должна служить ориентиром в анализе остатков модели прогнозирования: положительные остатки (перепрогнозирование) предпочтительнее, чем отрицательные (недопрогнозирование). Однако и чрезмерное перепрогнозирование также нежелательно.

Поэтому мы строим график распределения остатков вместе со смещением (простым средним значением необработанных остатков). Ожидается один из следующих возможных сценариев:

- Небольшое положительное смещение указывает на умеренное перепрогнозирование, что является желаемым результатом. Если при этом остатки имеют небольшое разброс (низкую дисперсию), то это — идеальный сценарий.
- Остатки равномерно распределены в отрицательную и положительную сторону — допустимо, но менее предпочтительно по сравнению с предыдущим случаем. Следует подталкивать модель к более оптимистичным прогнозам, чтобы снизить риски упущенной прибыли.
- Наихудший сценарий — перекос в сторону отрицательных остатков. Это означает, что модель имеет тенденцию занижать прогнозы, приводящую к росту недовольства клиентов. Такой случай является серьезным сигналом о неудовлетворительности текущей версии модели.
- Если же перекос имеется, но в сторону положительных остатков, то для Supermegaretail это приемлемая ситуация, но она все же уступает по предпочтительности первому сценарию.

Данные сценарии применимы при оценке несмещенного спроса (для этого используется медианный прогноз). Однако, как уже упоминалось, у нас имеются и модели для других квантилей (1.5 %, 25 %, 75 %, 95 %, 99 %).



Для каждой из этих моделей мы анализируем базовые предположения, лежащие в их основе. Например:

- Верно ли, что 95 % остатков положительные у модели, прогнозирующей 95 % квантиль?
- Верно ли, что 75 % остатков отрицательные у модели, прогнозирующей 25 % квантиль?

II. ЭЛАСТИЧНОСТЬ

Следует проверить гипотезы об эластичности с использованием кривых. Нет уверенности в том, что все товары обладают эластичностью спроса, и поэтому данный вопрос необходимо согласовать со стейкхолдерами.

Если возникают какие-либо проблемы, связанные с эластичностью, есть два пути их решения:

- *Постобработка* (быстрое, простое, ad hoc решение): можно использовать дополнительную модель (например, изотоническую регрессию) для постобработки прогнозов и их калибровки.

- *Улучшение основной модели* (медленное, сложное, универсальное решение): требует дополнительного моделирования, генерации признаков, предобработки данных и т. д. Не существует заранее определенного набора действий, который гарантированно устранит проблему.

III. ЛУЧШИЙ, ХУДШИЙ И КРАЙНИЕ СЛУЧАИ

Каждый раз при развертывании новой версии модели мы автоматически фиксируем ее поведение в лучших, худших и корнер-сценариях и сохраняем лучшие *N* % кейсов в качестве артефактов пайплайна обучения. Ниже приведен черновой список вопросов, на которые необходимо получить ответы:

- Какова ошибка прогноза модели при короткой истории продаж товара? Преобладают положительные или отрицательные остатки?
- Что происходит с ошибками для товаров с высокой или низкой ценой?
- Как ошибка прогноза зависит от выходных, праздников, дней проведения акций?
- Что общего у товаров с почти нулевыми остатками? Обязательна ли для них длинная история продаж? Насколько длинной она должна быть, чтобы обеспечить приемлемую точность? Нужны ли модели дополнительные условия, позволяющие заранее выделять случаи с высокой уверенностью в качестве прогноза?
- Что общего у товаров с наибольшими отрицательными остатками? Мы бы предпочли исключить такие случаи или даже целые категории из групп для A/B-тестирования или пилотных запусков. Также именно на эти товары стоит обращать внимание при последующем улучшении модели.
- Что общего у товаров с наибольшими положительными остатками?

9.4.2. Анализ ошибок для компании PhotoStock Inc.

Теперь вернемся к компании PhotoStock Inc., которой необходим современный инструмент поиска, способный находить релевантные изображения по текстовым запросам пользователей, обеспечивая при этом высокую производительность и выводя на экран наиболее подходящие изображения из текущего ассортимента.

ДИЗАЙН-ДОКУМЕНТ: PHOTOSTOCK INC.

VI. АНАЛИЗ ОШИБОК

Для раннего обнаружения возможных проблем необходимо с самого начала включить в проект инструменты анализа ошибок. В данном разделе документа

мы хотим заранее запланировать ключевые аспекты, на которых в дальнейшем следует сосредоточить внимание.

1. АНАЛИЗ КРИВЫХ ОБУЧЕНИЯ

- Кривые потерь должны использоваться для первичной проверки адекватности модели и дальнейшей настройки важных гиперпараметров, таких как порог ранней остановки, скорость обучения и др.
- Поскольку наша функция потерь является составной (включает несколько компонентов — см. раздел «Метрики и функции потерь»), необходимо обеспечить возможность визуализации кривых потерь по каждому компоненту для корректной настройки их весов.
- Модель должна иметь возможность обучаться на подвыборках данных, чтобы впоследствии строить кривые зависимости качества от размера выборки и оценивать влияние новых данных на общую производительность.
- Параллельно с кривыми потерь следует отслеживать и кривые метрик, чтобы убедиться в их адекватной корреляции.
- Поскольку датасет может быть распределенным, необходимо иметь возможность визуализировать кривые по каждому шарду.

2. АНАЛИЗ ОСТАТКОВ

- Для каждой эпохи обучения необходимо сохранять наиболее интересные семплы — с наибольшими/наименьшими значениями потерь как в целом, так и по каждому компоненту отдельно.
- Для каждого отображаемого семпла должны быть доступны метаданные, чтобы мы фиксировали не только текст запроса и релевантные изображения, но также категорию, теги, географию запроса, язык запроса и другие атрибуты, которые могут появиться в будущем.

После обучения каждой модели-кандидата (достаточно качественной для использования в реальной системе) предлагается следующая процедура:

- Выбрать 100 результатов с высокими потерями/метриками.
- Для каждого из них выдвинуть краткую гипотезу, почему данный семпл оказался аномальным (например: изображение размыто, описание изображения слишком длинное, запрос слишком короткий и т. п.), и сгруппировать примеры по причинам. Такой анализ следует проводить при каждом новом этапе улучшения системы, поскольку он является важным источником информации о модели.

В будущем стоит рассмотреть возможность применения методов интерпретируемости, поскольку на каком-то этапе обязательно возникнут вопросы вроде: «Почему изображение *X* семантически близко к изображению *Y*?». Но пока что данный этап можно отложить.

Итоги

- Не пренебрегайте анализом ошибок при проектировании вашей ML-системы — это поможет выявить ее слабые стороны и определить направления для улучшений.
- Анализ кривых обучения — важнейший шаг в оценке эффективности модели. Если модель не сходится или наблюдается переобучение/недообучение, дальнейший анализ теряет смысл.
- Наличие переобучения или недообучения свидетельствует о возможном дисбалансе между сложностью модели и объемом входных данных.
- В зависимости от того, какой тип кривой потерь вы наблюдаете во время анализа ошибок, необходимо применять соответствующие действия для устранения выявленных проблем.
- Предназначенный для оценки различий между прогнозируемыми и фактическими значениями анализ остатков необходим для проверки предположений о модели, выявления источников изменений метрик, оценки справедливости остатков, проведения анализа крайних случаев и диагностики наилучших/наихудших сценариев.

10 Пайплайны обучения

В этой главе

- ✓ Что такое пайплайн обучения
- ✓ Инструменты и платформы для построения и сопровождения пайплайнов обучения
- ✓ Масштабируемость и конфигурируемость пайплайнов обучения
- ✓ Методы тестирования пайплайнов

Существует верный способ отличить опытного инженера в области машинного обучения от новичка: попросите человека описать процесс обучения в работающей системе одним предложением. Новички, как правило, сосредоточивают все свое внимание на модели, инженеры с некоторым опытом упоминают также этап обработки данных, а вот самые зрелые специалисты описывают весь пайплайн — всю последовательность действий, необходимых для получения обученной ML-модели. В этой главе мы рассмотрим эти стадии и узнаем, как они связаны между собой и как объединяются в единую систему.

10.1. Что такое пайплайн обучения?

Давайте представим себе небольшую сеть пиццерий. Бизнес успешно развивается на местном рынке, но владельцы, понимая, что программное обеспечение «поглощает мир» (выражение взято из статьи Марка Андерссена (Marc

Anderssen)) (https://a16z.com/why-software-is-eating-the-world) и всё переходит в цифровой формат, решают заняться цифровизацией. Еще до пандемии COVID они нанимают инженеров для разработки мобильных приложений, простой системы управления взаимоотношениями с клиентами (CRM) и ряда внутренних IT-систем. Иными словами, компания не обладает масштабами или амбициями какого-то технологического гиганта, но все равно следует основным трендам и знает, что стоит вложиться в ПО сейчас, чтобы получить прибыль позже. Стоит ли говорить, что именно мобильное приложение помогло бизнесу пережить пандемию в 2020 году.

Теперь, когда компания продолжает следовать трендам, а все вокруг только и говорят, что об искусственном интеллекте, неудивительно, что они нанимают Джейн — молодого и перспективного ML-инженера. После онбординга технический директор поручает ей первую задачу: создать ассистента на базе ИИ, который поможет пиццайоло выполнять базовые визуальные оценки, например определять и подсчитывать количество ингредиентов для каждой пиццы на основании сделанного заказа.

Цикл разработки ПО в этой пиццерии ранее не включал ML-системы, поэтому технический менеджер Алекс просит Джейн подготовить модель и небольшой фрагмент кода, показывающий, как ее запускать, — все остальное возьмет на себя внутренняя команда разработчиков.

Теперь перенесемся на несколько месяцев вперед: Джейн собрала небольшой датасет, обучила модель, которая затем показала достойные результаты на начальном тестировании, поэтому команда Алекса смогла ее запустить. Но прямо перед внедрением продакт-менеджер сообщил о добавлении в меню новых рецептов и сказал, что модель должна их также поддерживать. На первый взгляд ничего сложного — просто нужно добавить новые данные, обновить карту меток и повторно обучить модель. Сильно повлиять на график развертывания не должно. Однако вскоре Джейн и Алекс поняли, что на самом деле на это потребуется еще несколько месяцев, даже если новый датасет уже готов. Что же пошло не так? Джейн выполнила все необходимые шаги для обучения модели: вручную проверила датасеты, применила самые разные методы обработки и очистки данных в среде Jupyter Notebook, обучила модель с многократными прерываниями, проверила результаты вместе с шеф-поваром и командой по работе с клиентами, загрузила обученную модель в общее хранилище компании и отправила ссылку Алексу.

ПРИМЕЧАНИЕ Она все сделала правильно, но действовала исключительно по ситуации, не предприняв должных усилий, чтобы обеспечить воспроизводимость всех шагов в рамках единого, прозрачного рабочего процесса.

Этим примером мы хотим показать, что ML — это не только обучение модели, но и построение пайплайна, который позволяет воспроизводимым образом подготавливать модель и все сопутствующие артефакты. В данной главе мы обсудим все стадии пайплайна, способы их организации и подходы к обеспечению их воспроизводимости.

10.1.1. Пайплайн обучения vs пайплайн инференса

В ML термин «пайплайн» используется в самых разных контекстах. Обычно под пайплайном понимается серия упорядоченных шагов и процессов. Каждый шаг — это программа, которая принимает некоторый вход, выполняет действия и выдает результат. Выход одного шага служит входом для следующего. Более формально пайплайн можно описать как ориентированный ациклический граф (directed acyclic graph, DAG) шагов.

Все усложняет то, что сама модель зачастую тоже является пайплайном, но несколько другого рода. Например, простой классификатор на логистической регрессии часто включает этап масштабирования признаков, в результате чего мы получаем пайплайн как минимум из двух шагов. Туда же добавляется базовая генерация признаков (например, one-hot-кодирование категориальных переменных), так что даже самая простая модель уже обладает свойствами пайплайна. Другие модальности, такие как изображения, текст, аудио и т. д., требуют дополнительных этапов предварительной обработки. Например, типичная модель классификации изображений представляет собой пайплайн, включающий чтение изображений, нормализацию, изменение размера и собственно саму модель. В случае обработки естественного языка пайплайн начинается с токенизации текста и т. п. В итоге термин «пайплайн» несет в себе множество значений и может вводить в заблуждение. Чтобы избежать путаницы, мы будем использовать следующие определения:

Пайплайн обучения — это пайплайн, используемый для обучения модели. Он представляет собой DAG шагов, который принимает на входе полный датасет и, при необходимости, дополнительную метаинформацию, а на выходе выдает обученную модель. Это более высокий уровень абстракции по сравнению с самой моделью (рис. 10.1).

Пайплайн инференса — это пайплайн, используемый для запуска модели в эксплуатацию или как часть пайплайна обучения (например, обучение нейросети с помощью градиентного спуска требует многократного инференса, и каждое повторение представляет собой отдельный пайплайн). Это DAG шагов, принимающий на входе сырые данные и выдающий прогнозы. Это более низкоуровневая абстракция по сравнению с пайплайном обучения (рис. 10.2).

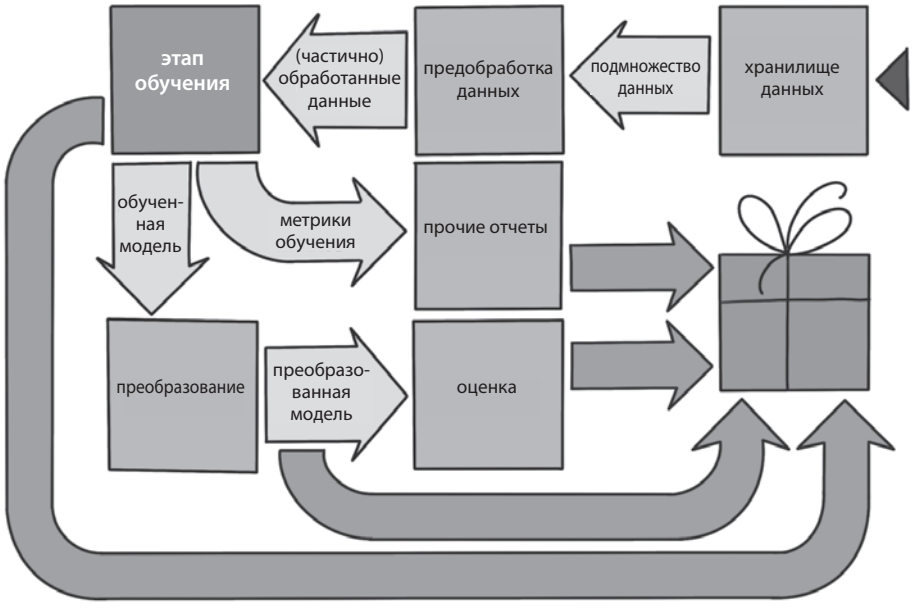


Рис. 10.1. Схема пайплайна обучения

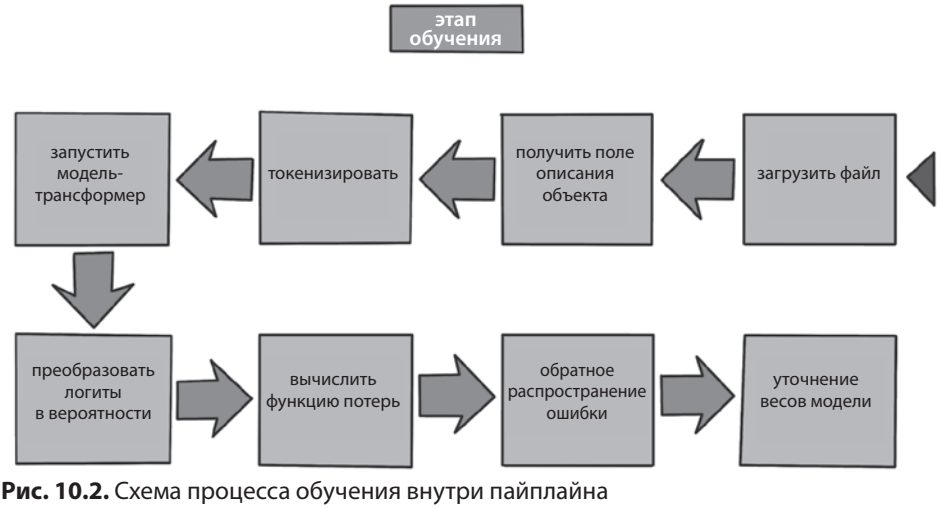


Рис. 10.2. Схема процесса обучения внутри пайплайна

В этой главе мы сосредоточим внимание на пайплайне обучения, а пайплайн инференса будет рассмотрен в главе 15. На верхнем уровне типичный пайплайн обучения включает в себя следующие шаги:

1. Получение данных.
2. Предобработка.

3. Обучение модели.
4. Оценка и тестирование модели.
5. Постобработка.
6. Генерация отчетов.
7. Упаковка артефактов.

Рассмотрим кратко каждый из шагов.

Получение данных — первый шаг пайплайна. Он отвечает за загрузку данных из источников и обеспечение их доступности для последующих шагов. Как уже упоминалось в главе 6, мы не считаем себя экспертами в области инженерии данных, а поэтому не будем подробно рассматривать загрузку и хранение данных.

Предобработка — обычно второй шаг пайплайна. Это весьма обобщенный термин, который может означать разные действия в зависимости от конкретной задачи. В общем смысле под предобработкой понимается набор операций по подготовке данных к обучению модели. И хотя в нашей книге мы разделяем пайплайны обучения и инференса, на практике такое разделение довольно размыто. Например, можно полностью предобработать исходный датасет до обучения модели либо сделать предобработку частью самого инференса. В данном контексте мы говорим о специфической предобработке для обучения. Примером может послужить отбор признаков: он выполняется только перед обучением, а выбранные признаки «замораживаются» для последующих шагов. *Обучение модели* — это ядро пайплайна обучения, шаг, который принимает на входе предобработанные данные и возвращает обученную модель. Как правило, это самый длительный и сложный этап во всем пайплайне (особенно в системах на основе глубокого обучения).

После того как модель обучена, ее можно *оценить и протестировать*. Это разные процессы, но направлены они на получение ответа на один и тот же вопрос: насколько хороша модель? Оценка — это этап вычисления метрик, в то время как тестирование представляет собой набор проверок, подтверждающих, что модель работает так, как ожидается.

Постобработка выполняется после оценки и тестирования. Это совокупность действий, необходимых для подготовки модели к развертыванию. На данном этапе можно преобразовать модель в формат, поддерживаемый целевыми платформами, применить квантизацию после обучения или другие оптимизации (по возможности), подготовить задания для ручной проверки и т. д. Стоит также отметить, что порядок этапов постобработки и оценки можно менять. Например, можно сначала оценить модель, а потом преобразовать ее в целевой формат либо наоборот — сначала выполнить преобразование, а затем оценить модель в формате, используемом при развертывании.

Упаковка артефактов — завершающий этап пайплайна, который отвечает за упаковку модели и других артефактов (например, конфигурационных файлов

с параметрами препроцессора) в формат, используемый при эксплуатации. Цель данного этапа — упростить дальнейшее развертывание и отделить пайплайн обучения от пайплайна развертывания. В идеале результат должен быть максимально независимым от пайплайна обучения. Например, модель экспортируется в универсальный формат, такой как ONNX для бэкенда или CoreML для iOS, все конфигурационные файлы также экспортируются в универсальный формат, например JSON, при этом любые изменения в пайплайне обучения должны минимально влиять на развертывание. В противном случае пайплайн развертывания будет сильно связан с пайплайном обучения и потребует множества изменений при каждом обновлении, что помешает быстрому развитию модели и проведению сопутствующих экспериментов.

Отчеты — особый вид артефактов. Это обобщенный термин, обозначающий множество вещей, включая базовые таблицы с метриками валидации и тестирования, различные виды анализа ошибок (см. главу 9), дополнительные визуализации и другую вспомогательную информацию. И хотя данные артефакты напрямую не используются при развертывании, они крайне важны для признания обучения успешным. Ни один ответственный инженер не выпустит новую модель без хотя бы краткого ознакомления с соответствующими отчетами. Единственное исключение, с которым мы сталкивались, — это вариант сценария AutoML, когда множество новых моделей обучаются автоматически по запросу пользователей. В таком случае ручная проверка не всегда возможна, поэтому инженеры могут просматривать только подозрительные выбросы. К теме релизного цикла мы вернемся в главе 13.

Некоторые из этих артефактов связаны с отслеживанием экспериментов и воспроизводимостью, что особенно важно для проектов с большим количеством контрибьюторов или заинтересованных сторон. Когда исследователь работает один над собственной проблемой, он может отслеживать все идеи и эксперименты в простом блокноте или текстовом файле. Однако если над одной проблемой работает команда исследователей, требуется более структурированный способ отслеживания, сравнения и воспроизведения экспериментов. Централизованное хранилище всех экспериментов — один из инструментов, который может в этом помочь.

10.2. Инструменты и платформы

Инструменты и практики, связанные с пайплайном обучения, а также с пайплайнами инференса, развертывания и сервисами мониторинга, в контексте проектирования ML-систем часто объединяются под единым понятием ML-операций (MLOps). Поскольку MLOps — это относительно новая область, здесь еще не сложились четкие стандарты по платформам и инструментам. Некоторые из них достаточно известны (MLflow, Kubeflow, BentoML, AWS Sagemaker, Google

Vertex AI, Azure ML), другие только начинают набирать популярность, а третьи все еще находятся на ранних этапах разработки.

В нашей книге мы не выделяем какую-то конкретную платформу или инструмент, поэтому не будем рассматривать их детально. С учетом темпов изменений в экосистеме MLOps весьма вероятно, что наше текущее представление о платформах и инструментах устареет к моменту выхода книги. Вместо этого мы лучше сосредоточимся на принципах и практиках, общих для всех платформ и инструментов. В простейших случаях вы можете реализовать полноценный пайплайн обучения, используя универсальные, не специализирующиеся на ML инструменты — например, написав серию Python-скриптов, связанных между собой shell-скриптами. Однако на практике такое встречается довольно редко: как правило, используются специальные инструменты для ML, которые вводят абстракции и упрощают реализацию пайплайна. Большинство инструментов MLOps являются концептуально жесткими, то есть их использование подразумевает определенный способ структурирования кода. В долгосрочной перспективе это повышает согласованность кода пайплайна обучения и упрощает его поддержку в будущем (см. главу 16). Базовые возможности, требуемые от платформы пайплайна обучения, включают:

- *Разрешение зависимостей*: так как пайплайн представляет собой ориентированный ациклический граф (DAG), крайне важно корректно разрешать зависимости между шагами и выполнять их в правильном порядке.
- *Воспроизводимость*: при заданном наборе параметров и версии пайплайна (например, определенной по коммиту в Git) его выполнение должно каждый раз давать один и тот же результат.
- *Интеграция с вычислительными ресурсами* (облачными провайдерами, установками Kubernetes и т. п.): например, пользователи должны иметь возможность запускать задачу на конкретном вычислительном экземпляре (скажем, виртуальной машине с X ядрами CPU и N GPU) либо на группе таких экземпляров.
- *Хранение артефактов*: после выполнения пайплайна обучения его артефакты должны оставаться доступными. Отслеживание экспериментов можно рассматривать как частный случай этой функции.
- *Кэширование промежуточных результатов*: поскольку многие шаги в конвейере являются ресурсоемкими, крайне важно кэшировать промежуточные результаты, чтобы сэкономить ресурсы и время.

Помимо функциональных возможностей, необходимо также упомянуть некоторые нефункциональные требования, предъявляемые к платформам, такие как экономическая эффективность и конфиденциальность данных (особенно в чувствительных областях, таких как здравоохранение или юриспруденция).

Следует подчеркнуть, что все функции не обязательно должны поддерживаться одной и той же платформой. Например, можно использовать универсальную платформу для выполнения пайплайна и отдельный инструмент для отслеживания экспериментов, если готовые решения не удовлетворяют вашим специфическим требованиям. А порой это вопрос оптимизации затрат. Арсений как-то работал в компании, которая использовала гибрид из двух инструментов только потому, что один из них предоставлял множество полезных функций и был удобен для разработчиков, а другой был интегрирован с облачным провайдером, предлагавшим самые дешевые GPU для обучения. В такой ситуации стоило потратить немного времени на интеграцию, чтобы в дальнейшем сэкономить значительные средства на обучении моделей.

Выбор подходящих инструментов определяется масштабом ML-проблемы и инфраструктурой компании. Компании уровня FAANG, как правило, имеют собственные платформы и инструменты ML, способные работать на нужном масштабе, в то время как более мелкие компании чаще отдают предпочтение набору инструментов с открытым исходным кодом и облачным сервисам. У каждого инструмента есть своя стоимость внедрения, поэтому делать выбор стоит исходя из конкретной ML-проблемы. Насколько нам известно, не существует универсального решения, применимого ко всем случаям, в отличие от более зрелых направлений в разработке ПО (рис. 10.3).

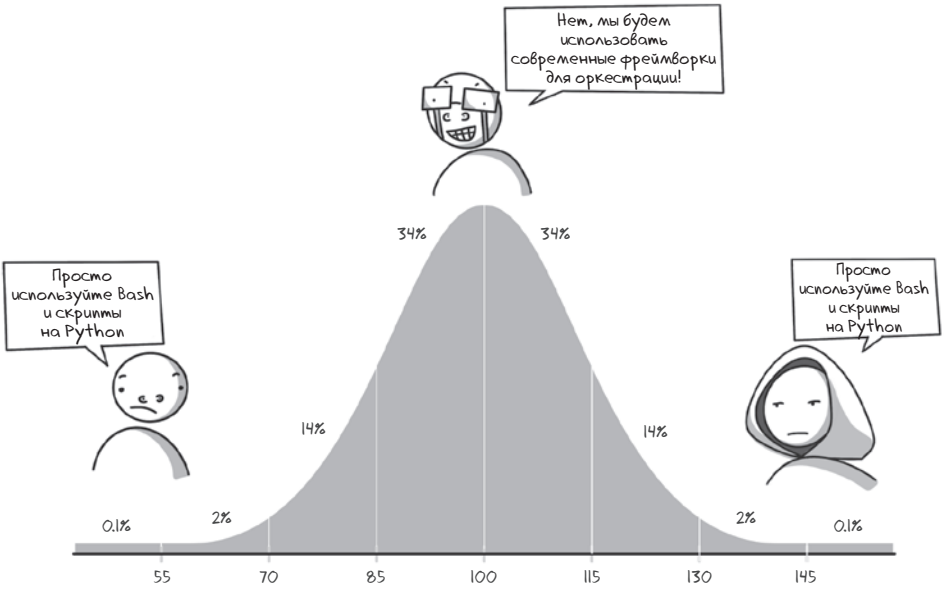


Рис. 10.3. Использовать подходящие фреймворки для пайплайна обучения — это, конечно, хорошо, но иногда достаточно и простых решений

10.3. Масштабируемость

Для некоторых проблем масштабируемость может стать критически важным свойством пайплайна обучения. Если речь идет о датасете, содержащем тысячи семплов практически любого типа, это, как правило, не вызывает серьезных затруднений — с ним справится даже одна машина. Но когда объем данных становится огромным, ситуация меняется. Что можно считать огромным датасетом? Зависит от проблемы и типа данных (например, миллион строк табличных данных — ничто по сравнению с миллионом видеороликов). Но допустим, нам нужно задать критерий: «датасет, который не помещается в оперативную память одной машины» — в таком случае его уже можно отнести к большим.

Текущий размер датасета не стоит путать с предполагаемым будущим объемом данных. Мы можем столкнуться с проблемой холодного старта (см. главу 6), и наличие даже нескольких тысяч семплов уже станет значительным преимуществом на начальном этапе разработки системы. Однако в дальнейшем объем данных может вырасти на несколько порядков. И если вы планируете использовать все доступные данные, то должны быть к этому готовы.

И хотя не существует универсального решения для обучения моделей на больших датасетах, в разработке ПО применяются два классических подхода к масштабированию — вертикальное и горизонтальное (рис. 10.4).



Рис. 10.4. Сравнение вертикального и горизонтального масштабирования

Вертикальное масштабирование означает модернизацию оборудования или замену вычислительного узла, на котором выполняется обучение модели, на более мощный. Главное преимущество такого подхода — его простота: добавление

ресурсов (особенно при использовании облачных вычислений, что встречается довольно часто) крайне удобно. Однако у вертикального масштабирования есть и один серьезный недостаток — его ограниченность. Допустим, вы удвоили или даже учетверили объем оперативной памяти машины и обновили GPU до последнего поколения. Если и этого недостаточно, в рамках вертикального масштабирования сделать уже больше ничего нельзя.

Горизонтальное масштабирование предполагает распределение нагрузки между несколькими машинами. Первый уровень горизонтального масштабирования — это использование нескольких GPU. В таком случае ML-инженер вносит лишь незначительные изменения в код пайплайна, поскольку основная часть работы уже реализована средствами используемого фреймворка. Однако это все еще не настоящее горизонтальное масштабирование, так как речь идет об одной машине. Полноценное горизонтальное масштабирование требует использования нескольких машин с распределением нагрузки между ними. Сегодня данный тип масштабирования также поддерживается рядом фреймворков, но он сложнее и требует больших инженерных усилий при реализации. Примеры таких фреймворков — DeepSpeed от Microsoft, Accelerate от Hugging Face и Horovod, разработанный Uber.

Один из специфичных для ML способов масштабирования — выборка подмножества, или сабсемплинг (subsampling): если датасет слишком большой, лучше сделать подвыборку, чтобы снизить потребность в вычислительных ресурсах. Самый простой способ сабсемплинга применим ко многим проблемам и заключается в удалении дублирующихся семплов. Однако существуют и более агрессивные методы: уменьшение выборки, или даунсемплинг, почти идентичных семплов (например, близких по простой метрике расстояния, такой как, расстояние Левенштейна для строк) и даунсемплинг на основе внутренних идентификаторов (например, не более X образцов на пользователя) или искусственных идентификаторов (например, кластеризация всего датасета с помощью простого метода и сохранение не более X образцов на кластер).

В пайплайнах ML масштабирование часто требует изменения других параметров пайплайна: например, размер батча ограничен объемом памяти GPU, более крупные батчи ускоряют сходимость, а график изменения скорости обучения зависит как от размера батча, так и от ожидаемой динамики сходимости. Поэтому у вас должна быть возможность изменять параметры.

10.4. Конфигурируемость

Когда ML-инженеры проектируют конфигурируемость пайплайна обучения, они оказываются перед выбором из целого спектра с двумя крайностями, обе из которых считаются антипаттернами: недостаточная конфигурация (underconfiguration) и избыточная конфигурация (overconfiguration).

Недостаточная конфигурация означает, что пайплайн недостаточно гибок, и изменить архитектуру модели, датасет, шаги препроцессинга и другие аспекты затруднительно. Значения параметров жестко зашиты в коде, структура сложная и запутанная; тяжело понять, как работает пайплайн, и внести даже простейшие изменения. Это типичная проблема начальных этапов разработки в ML. Пока пайплайн мал и прост, его легко понять и изменить, поэтому специалисты без опыта разработки ПО могут посчитать ненужным внедрение надлежащих абстракций, что со временем приводит к хаотичному наращиванию объема кода без четкой структуры. Такой антипаттерн встречается довольно часто.

Избыточная конфигурация нам тоже не нужна. Типичный пайплайн ML содержит множество гиперпараметров, связанных с обработкой данных, архитектурой модели, генерацией признаков и самим процессом обучения. На практике трудно заранее предсказать все возможные сценарии и параметры, которые могут понадобиться, и поэтому неопытные разработчики пытаются предусмотреть абсолютно все случаи и ввести как можно больше абстракций. В какой-то момент дополнительные уровни абстракции только увеличивают сложность системы. В данном разделе мы будем использовать выражения «параметры пайплайна обучения» и «гиперпараметры модели» как взаимозаменяемые. Напомним, гиперпараметры — это параметры модели, которые не усваиваются в процессе обучения, а задаются пользователем.

В примере слишком сложного кода в листинге ниже можно увидеть, как иерархия подконфигураций усложняет его реализацию.

Листинг 10.1. Многоуровневая иерархия подконфигураций

```
def train():
    ...
    batch_size = 32
    ...
    learning_rate = 3e-4
    ...
    model.train(data, batch_size, loss_fn)
    ...
    # пример кода с недостаточной конфигурацией — все слишком строго задано

class Config(BaseConfig):
    def __init__(self):
        self.data_config = DataConfig()
        self.model_config = ModelConfig()
        self.training_config = TrainingConfig()
        self.inference_config = InferenceConfig()
        self.environment_config = EnvironmentConfig()

class DataConfig(BaseConfig):
    def __init__(self):
        self.train_data_config = TrainDataConfig()
        self.validation_data_config = ValidationDataConfig()
```

```
        self.test_data_config = TestDataConfig()

config = Config(
    data_config=DataConfig(
        train_data_config=TrainDataConfig(
            ...
        ),
        validation_data_config=ValidationDataConfig(
            ...
        ),
        test_data_config=TestDataConfig(
            ...
        ),
    ),
    ...
)
```

Чтобы найти баланс между двумя крайностями, необходимо оценить вероятность изменения различных параметров. Например, практически со стопроцентной вероятностью датасеты будут обновляться, тогда как функции активации внутри модели, скорее всего, останутся неизменными. Поэтому разумно сделать настройки датасета конфигурируемыми, а функции активации — нет. Изменяемые параметры меняются от одного пайплайна к другому, и единственный способ найти баланс — это заранее определить, какие эксперименты вы, вероятно, проведете в ближайшие месяцы. Полезное руководство, которое мы рекомендуем для пайплайнов, основанных на глубоком обучении, было написано командой Google (https://github.com/google-research/tuning_playbook).

После определения подлежащих настройке гиперпараметров важно правильно выбрать стратегию. Когда вычислительные ресурсы ограниченны, предпочтение следует отдавать экспериментам, проводимым вручную. Если же ресурсов достаточно, можно использовать и автоматические методы настройки гиперпараметров, такие как простой случайный поиск или более продвинутые методы, например байесовская оптимизация. Инструменты для настройки гиперпараметров (например, Hyperopt, Optuna и scikit-optimize) могут быть частью ML-платформы и требовать определенных конфигурационных файлов.

По нашему опыту, масштабная настройка гиперпараметров более применима к небольшим датасетам, позволяющим проводить множество экспериментов за разумное количество времени. А если один эксперимент занимает недели, куда более практично положиться на интуицию ML-инженера и выполнить несколько экспериментов вручную. Стоит также отметить, что эксперименты с небольшими датасетами способствуют формированию той самой интуиции, хотя и не все выводы будут применимы к обучению на больших объемах данных.

Важно подобрать правильный способ конфигурирования пайплайна обучения (рис. 10.5).

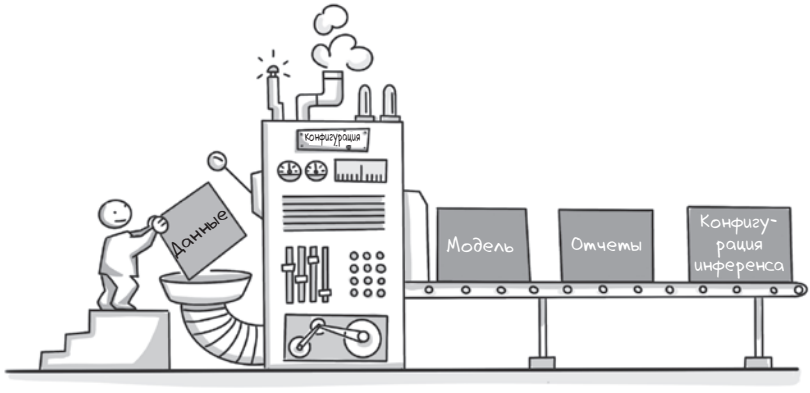


Рис. 10.5. Для оптимальной производительности пайплайн должен быть корректно сконфигурирован

Наиболее распространенным подходом является выделение отдельного файла (часто написанного на специализированном языке, таком как YAML или TOML), содержащего все изменяемые значения. Другой вариант — использовать библиотеки, например Hydra (https://hydra.cc/). Один из антипаттернов, с которым мы сталкивались, — это ситуация, когда конфигурация разбросана по различным файлам пайплайна, причем один и тот же параметр может быть указан в нескольких файлах с разным уровнем приоритета (например, размер батча может считываться из файла X, но если он там не задан — пробуем получить его из файла Y). Это может привести к ошибкам на стадии экспериментов, особенно если их проводят не очень опытные инженеры, не знакомые с устройством конкретного пайплайна.

10.5. Тестирование

Одна из распространенных проблем, с которой мы часто сталкиваемся в ML-пайплайнах, — отсутствие тестов. И это неудивительно, поскольку тестирование ML-пайплайнов — задача не из легких. При разработке обычной программной системы ее можно протестировать, запустив с определенным вводом и проверив результат. Однако выполнение пайплайна обучения может занять дни, и, разумеется, мы не можем запускать его заново после каждого внесенного изменения. Еще одна проблема, как уже упоминалось ранее, заключается в том, что ML-пайплайны часто недостаточно конфигурируемы, что затрудняет их изолированное тестирование. Наконец, учитывая количество возможных гиперпараметров, протестировать все возможные комбинации в разумные сроки практически невозможно. Проще говоря, внедрение тестов в ML-пайплайны — сложная задача. Но она того стоит!

Правильно реализованные тесты выполняют три функции:

- предотвращают ошибки регрессии при внесении изменений;
- ускоряют итерации за счет раннего выявления дефектов;
- улучшают архитектуру пайплайна в целом, поскольку заставляют инженера искать правильный баланс конфигурируемости.

Мы рекомендуем использовать комбинацию высокоуровневых дымовых тестов для всего пайплайна и низкоуровневых юнит-тестов для его наиболее важных компонентов.

Дымовой тест должен проводиться быстро, поэтому его следует запускать на небольшом подмножестве датасета, на малом количестве эпох и, возможно, с урезанной версией модели. Он должен проверить, что пайплайн запускается без ошибок и выдает разумный результат, например убедиться, что значение функции потерь уменьшается на данном тестовом датасете. В листинге ниже приведен упрощенный пример дымового теста для пайплайна обучения.

Листинг 10.2. Пример дымового теста для пайплайна обучения

```
from unittest.mock import patch, Mock
import torch
from training_pipeline import train, get_config

class DummyResnet(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.model = torch.nn.Sequential(
            torch.nn.AdaptiveAvgPool2d(1),
            torch.nn.Conv2d(3, 2048, 1)
        )

    def forward(self, x):
        return self.model(x).squeeze(-1).squeeze(-1)

def test_train_pipeline():
    config = get_config()
    config["dataset_path"] = "/path/to/fixture"
    config["num_epochs"] = 1

    mock = Mock(wraps=DummyResnet)

    with patch('training_pipeline.models.Resnet', mock):
        result = train(config)

    assert mock.call_count == 1
    assert result['train_loss'] < 0.5
    assert result['val_loss'] < 1
```

Дымовые тесты, подобные приведенному выше, значительно повышают скорость итераций, тем самым упрощая проведение экспериментов и отладку. Однако у них есть и обратная сторона. Как и любые интеграционные тесты, они требуют достаточно сложного сопровождения. Это связано с тем, что практически любое значимое изменение в пайплайне может повлиять на код. Низкоуровневые юнит-тесты должны охватывать отдельные компоненты пайплайна. Нередко случается, что таких тестов совсем немного или их нет вовсе, — и в этом нет ничего постыдного. Тем не менее мы рекомендуем покрывать хотя бы наиболее чувствительные компоненты. Примером такого компонента может послужить финальный этап конвертации модели — представьте, что модель обучается в PyTorch, а затем должна быть развернута на iOS (с использованием CoreML) и в бэкенде (с использованием ONNX). В таком случае важно убедиться, что модель конвертируется корректно и что сам процесс конвертации не вносит никаких изменений, то есть выходные данные конвертированной модели должны совпадать с результатами оригинальной.

10.5.1. Тестирование на основе свойств

Другая группа тестов, применимая к обученной модели, относится к тестированию на основе свойств (property-based testing). Тестирование на основе свойств — это подход в тестировании ПО, при котором для функции или системы генерируются случайные входные данные, а затем проверяется, соблюдаются ли определенные свойства или инварианты для этих входных данных. Вместо того чтобы писать конкретные тест-кейсы с заранее заданными входами и ожидаемыми выходами, в рамках данного подхода формулируются общие свойства, которым должна удовлетворять система, и затем автоматически генерируются тестовые случаи для проверки этих свойств.

В контексте ML-проекта тестирование на основе свойств может использоваться для проверки того, что финальная обученная модель ведет себя должным образом и соблюдает заданные свойства. Ниже приведены некоторые примеры свойств, которые можно проверить:

- *Согласованность* (consistency): при подаче одних и тех же входных данных модель должна стабильно выдавать одинаковый результат или прогноз, независимо от количества запусков:

$$f(x) = f(x).$$

- *Монотонность* (monotonicity): в простых ML-моделях результат должен монотонно увеличиваться или уменьшаться в зависимости от некоторых признаков входных данных. Тестирование на основе свойств позволяет проверить, что выход модели соответствует ожидаемому монотонному поведению:

$$f(x) < f(y) \text{ если } x_i < y_i.$$

Монотонность ожидается, например, в моделях прогнозирования цен. Так, цена дома должна расти при увеличении его площади, даже если остальные характеристики остаются неизменными.

- *Инвариантность к преобразованиям* (invariance under transformations): некоторые ML-модели должны быть инвариантны к определенным преобразованиям входных данных, таким как масштабирование или поворот. Тестирование на основе свойств позволит убедиться, что результат модели остается неизменным при заданных преобразованиях входных данных:

$$f(x) = f(g(x)),$$

где g — это ожидаемое преобразование. Это может быть поворот или масштабирование изображений, замена слова на синоним в задачах обработки естественного языка, изменение громкости в задачах обработки звука и т. д.

- *Устойчивость* (robustness): модель должна быть устойчива к незначительным возмущениям во входных данных. С помощью тестирования на основе свойств можно проверить, что выход модели не изменяется существенно при небольших изменениях входных данных:

$$f(x) \approx f(x + \varepsilon).$$

- *Отрицание* (negation): модель должна выдавать противоположный прогноз, если входные данные инвертируются. Тестирование на основе свойств может использоваться для проверки, что результат модели противоположен ожидаемому, когда входные данные подвергаются инверсии. Простейший пример — анализ тональности, где модель обычно должна прогнозировать отрицательную тональность, если слово «люблю» заменено на «ненавижу»:

$$f(x) = -f(-x).$$

Мы рассматривали схожую концепцию в разделе 5.2.1. Разница заключается лишь в том, что в одном случае мы ожидаем некоторую вариативность результатов (и хотим ее измерить), тогда как в другом — мы ожидаем строгую согласованность (и хотим это проверить). Использование ряда семплов в качестве фикстур и написание тестов, основанных на свойствах, — это способ убедиться, что модель ведет себя ожидаемо и остается надежной.

Точный перечень тестов обычно не включается в дизайн-документ, однако мы рекомендуем все же заранее продумать его и упомянуть. Дизайн-документ часто используется как основа для реализации, поэтому полезно в нем зафиксировать и тестирование.

ПРИМЕЧАНИЕ Если вам интересна тема тестирования ML-моделей, рекомендуем ознакомиться с презентацией Арсения: https://arseny.info/reliable_ML.

10.6. Дизайн-документ: пайплайны обучения

Мы продолжаем работу над двумя дизайн-документами для наших вымышленных компаний. Пришло время рассмотреть пайплайны обучения для Supermegaretail и PhotoStock Inc.

10.6.1. Пайплайн обучения для Supermegaretail

Рассмотрим, как может выглядеть потенциальный пайплайн для Supermegaretail.

ДИЗАЙН-ДОКУМЕНТ: SUPERMEGARETAIL

VII. ПАЙПЛАЙН ОБУЧЕНИЯ

1. Обзор

Модель для Supermegaretail занимается прогнозированием спроса на конкретные товары в конкретных магазинах за определенный период времени. Для достижения этой цели требуется пайплайн обучения, который может выполнять предобработку данных, обучение модели и оценку ее производительности. Предполагается, что пайплайн должен быть масштабируемым, легко сопровождаемым и должен обеспечивать возможность проведения экспериментов с различными архитектурами моделей, методами построения признаков и гиперпараметрами.

2. Инструменты

Рекомендуемые инструменты для пайплайна:

- Python в качестве основного языка программирования благодаря его универсальности и богатой экосистеме для обработки данных и ML.
- Spark для параллельных и распределенных вычислений.
- PyTorch для построения моделей глубокого обучения.
- MLflow для отслеживания экспериментов и управления жизненным циклом моделей.
- Docker для контейнеризации и воспроизводимости.
- AWS Sagemaker или Google Cloud AI Platform для обучения и развертывания в облаке.

3. Предобработка данных

Этап предобработки данных должен включать:

- *Очистку данных*: обработку пропущенных значений, удаление дубликатов и исправление ошибочных данных.
- *Генерацию признаков*: создание новых признаков на основе существующих, таких как агрегация данных о продажах, извлечение временных признаков (день недели, месяц и т. д.), а также включение внешних данных (например, праздники, погода, акции).

- *Нормализацию данных*: масштабирование числовых признаков до стандартного диапазона.
- *Разделение на обучающую и валидационную выборки* (train-test split) без временного пересечения между ними для предотвращения утечки данных.

4. Обучение модели

Этап обучения модели должен поддерживать различные архитектуры и конфигурации моделей, включая:

- *Базовые модели* — простые методы прогнозирования, такие как скользящее среднее, экспоненциальное сглаживание и авторегрессионная интегрированная модель скользящего среднего.
- *ML-модели* — деревья решений, случайные леса, градиентный бустинг и метод опорных векторов.
- *Модели глубокого обучения* — рекуррентные нейронные сети, сети с долгой краткосрочной памятью, а также трансформеры (при необходимости).

Также необходимо реализовать механизм подбора гиперпараметров, например перебор по сетке или байесовскую оптимизацию, для нахождения наилучших конфигураций модели.

5. Оценка модели

Производительность модели должна оцениваться с использованием заранее определенных метрик, таких как квантильные метрики для квантилей 1.5, 25, 50, 75, 95 и 99, как в абсолютном виде, так и с весами, равными стоимости SKU. Данные метрики рассчитываются в виде точечных оценок с 95 %-ными доверительными интервалами (с использованием бутстрэппинга или кросс-валидации), а также стандартных метрик, таких как средняя абсолютная ошибка (MAE), среднеквадратичная ошибка (MSE) и корень из среднеквадратичной ошибки (RMSE). Также следует включить настраиваемые метрики, соответствующие бизнес-требованиям Supermegaretail, например стоимость избыточных запасов и дефицита товаров. (См. раздел дизайн-документа «Выбор схем валидации».)

6. Отслеживание экспериментов и управление моделями

Инструмент MLflow позволяет отслеживать и управлять экспериментами, включая:

- параметры модели и гиперпараметры;
- входные данные и методы построения признаков;
- метрики оценки и производительность;
- артефакты модели, такие как веса обученной модели и сериализованные версии.

7. Непрерывная интеграция и развертывание

Пайплайн обучения должен быть интегрирован в существующую инфраструктуру CI/CD Supermegaretail. Это включает настройку автоматического

обучения и оценки модели на регулярной основе, обеспечение использования актуальных данных для обновления модели и развертывание обновленной модели в продакшене с минимальным участием человека.

8. Мониторинг и сопровождение

Необходимо организовать мониторинг производительности модели в продакшене и настроить оповещения при существенных отклонениях от ожидаемой производительности. Это позволит своевременно выявлять проблемы и запускать повторное обучение модели или обновления при необходимости (см. главу 14).

9. Перспективы и эксперименты

Пайплайн обучения должен быть достаточно гибким для поддержки будущих экспериментов, таких как подключение дополнительных источников данных, тестирование новых архитектур моделей и изменение функций потерь с целью оптимизации под конкретные бизнес-цели.

10.6.2. Пайплайн обучения для PhotoStock Inc.

Теперь вернемся к PhotoStock Inc., где нам необходимо построить продвинутую внутреннюю поисковую систему для повышения точности выдачи релевантных результатов.

ДИЗАЙН-ДОКУМЕНТ: PHOTOSTOCK INC.

VII. ПАЙПЛАЙН ОБУЧЕНИЯ

Мультимодальная ранжирующая модель является ключевым компонентом поисковой системы компании PhotoStock Inc., и для нее необходим пайплайн обучения. Как обсуждалось ранее, у нас есть надежная базовая модель на основе предобученной модели CLIP. Тем не менее необходимо дообучить ее на нашем собственном датасете, который представляет собой комбинацию изображений и текстовых описаний. Хотя на начальном этапе датасет будет небольшим, со временем он может значительно увеличиться, поэтому пайплайн должен быть хотя бы частично масштабируемым. Мы предполагаем, что обучение можно начать на одном высокопроизводительном GPU, но в будущем потребуется возможность масштабирования на несколько процессоров в рамках одной машины. Полноценное распределенное обучение пока не планируется.

Мы предлагаем следующий набор инструментов для пайплайна:

- *PyTorch* — основной фреймворк для глубокого обучения, так как он является самым популярным в мире и пользуется доверием сообщества.

- *PyTorch Lightning* — высокоуровневая надстройка над PyTorch, упрощающая обучающий цикл и повышающая воспроизводимость.
- *Flyte* — инструмент управления рабочими процессами, уже применяемый в компании для задач обработки данных, что позволяет использовать часть уже существующего кода.
- *AWS Sagemaker* — платформа для обучения, поскольку AWS уже используется в компании и легко интегрируется с Flyte.
- *Tensorboard* — простой инструмент визуализации метрик обучения.
- *Docker* — инструмент контейнеризации, обеспечивающий переносимость и воспроизводимость пайплайна.

Выходными данными пайплайна будут две модели: текстовый энкодер и энкодер изображений. Обе модели необходимо преобразовать в формат статического графа (ONNX) и сохранить в хранилище S3. Кроме того, необходимо сохранить список параметров обучения, которые будут использоваться при инференсе (генерация промтов, предобработка изображений, функция расстояния). Наконец, каждый запуск должен сопровождаться отчетом с метриками обучения. Все артефакты должны сохраняться в S3 по завершении запуска.

Мы предполагаем эксперименты в следующих направлениях:

- *Что настраивать:* отдельные компоненты, всю модель или их комбинацию с использованием кастомного планировщика обучения.
- *Техники аугментации*: можно применять различные методы аугментации данных как для изображений, так и для текста.
- *Различные функции потерь*: с разными весами для различных компонентов.
- *Различные архитектуры семейства моделей CLIP*: например, энкодеры изображений на основе сверточных сетей или трансформеров. Четких аргументов в пользу превосходства одной архитектуры над другой нет, поэтому необходимы эксперименты с обеими.
- *Способы генерации текстовых промтов для текстового энкодера*: они должны представлять собой комбинацию описания изображения, тегов и других элементов.
- *Методы предобработки изображений*: например, параметры изменения размера, обрезки, добавления отступов.

Итоги

- Помните, что машинное обучение — это не только обучение модели. Одним из ключевых элементов является построение пайплайна, обеспечивающего воспроизводимость модели и других артефактов.
- Хотя разница между пайплайнами обучения и пайплайнами инференса может показаться размытой и трудноуловимой, мы предлагаем следующее определение: пайплайн обучения используется непосредственно для обучения

модели, тогда как пайплайн инференса — для запуска модели в эксплуатацию или как часть обучающего процесса.

- Жизненный цикл типичного пайплайна обучения включает семь последовательных этапов: получение данных, предобработка, обучение, оценка и тестирование модели, постобработка, упаковка артефактов и генерация отчетов.
- В настоящее время не существует устоявшихся стандартов платформ и инструментов для работы с пайплайнами. Однако в ML имеются проверенные временем решения, которые можно адаптировать под вашу систему.
- Рано или поздно перед вами встанет выбор между двумя подходами к масштабированию пайплайна — вертикальным или горизонтальным. Первый проще в реализации и требует меньше усилий, но ограничен максимальными возможностями одной машины. Второй открывает более широкие возможности по увеличению вычислительной мощности.
- Постарайтесь найти баланс в степени конфигурируемости вашего пайплайна. Если вы скатитесь в одну из крайностей (недостаточная или избыточная конфигурация), пайплайн окажется либо негибким к изменениям, либо излишне сложным для решения поставленных задач.
- Не пренебрегайте тестированием пайплайна! Это поможет избежать регрессионных ошибок при внесении изменений, ускорит итерации за счет раннего выявления дефектов и улучшит общий дизайн, поскольку вы будете искать оптимальный баланс между гибкостью и надежностью.

11 Признаки и генерация признаков

В этой главе

- ✓ Итеративный процесс генерации признаков
- ✓ Анализ важности признаков
- ✓ Выбор подходящих признаков для вашей модели
- ✓ Плюсы и минусы хранилищ признаков

Часто говорят, что посредственная модель с отличными признаками превзойдет отличную модель с плохими признаками. На нашем опыте это утверждение более чем справедливо. Признаки являются ключевыми входными данными для системы: они управляют алгоритмами, обеспечивают модель необходимыми паттернами и предоставляют ей данные, которые модель должна изучить, чтобы делать прогнозы. Без хороших признаков модель слепа, глуха и нема.

И хотя роль генерации признаков не столь критична для систем, спроектированных на основе глубокого обучения, ни один ML-специалист никогда не станет игнорировать ее значение. В каком-то смысле подготовка сложных мультимодальных данных для подачи в модель глубокого обучения или даже составление промта для большой языковой модели — это частный случай генерации признаков, и именно поэтому классические техники, например анализ важности признаков, по-прежнему сохраняют свою актуальность.

В этой главе мы рассмотрим искусство и науку построения эффективных признаков. Мы изучим инструменты, которые помогут вам определить наиболее

ценные для системы признаки, инженерные сложности, с которыми можно столкнуться, факторы и компромиссы, которые следует учитывать при выборе оптимального набора признаков, а также способы обеспечения их надежности и устойчивости.

11.1. Что такое генерация признаков?

Генерация признаков (feature engineering) — это итеративный процесс, включающий создание и тестирование новых признаков или преобразование уже существующих с целью повышения производительности модели. Данный процесс требует определенной креативности, знаний и навыков в области инженерии данных для построения новых пайплайнов в системе. Ввиду трудоемкости и итеративной природы на генерацию признаков нередко уходит значительная часть ресурсов, выделенных под моделирование.

Чтобы обеспечить продуктивный и упорядоченный процесс моделирования, следует обязательно сформировать эффективную стратегию генерации признаков еще на этапе проектирования системы. Такой план станет своеобразным компасом, который будет направлять команду при выявлении и построении наиболее значимых признаков, при этом минимизируя риск потратить усилия впустую. Выстроив правильную последовательность итераций, можно избежать потенциальных ловушек и направить все действия на достижение конечной цели.

Генерация признаков в ML схожа с написанием промтов для генеративных моделей, таких как большие языковые модели или генераторы изображений. И признаки, и промты служат входными данными, которые направляют «фокус внимания» модели (в прямом или переносном смысле) на наиболее значимые аспекты.

> **ПРИМЕЧАНИЕ** Создавая корректные признаки и промты, мы даем модели определенную перспективу, контекст или «индуктивное смещение», заставляя ее склоняться к желаемым результатам. Несмотря на различную природу и признаки, и промты преследуют одну и ту же цель — контекстуализировать модель с учетом наших знаний о предметной области и направить ее к нужному результату.

Говоря о мощных моделях глубокого обучения, стоит отметить, что в некоторых сферах, например в обработке аудио и изображений, генерация признаков раньше была весьма сложной задачей. Однако затем произошла революция в области глубокого обучения, вызвавшая восторг у всех ML-специалистов, — вместо бесконечного и ненадежного построения признаков вручную можно было просто обучать модель от начала до конца. И теперь порой встречаются инженеры, которые никогда не занимались генерацией признаков вне учебных проектов! Такая тенденция может навести на мысль, что эту главу можно пропустить. Однако мы считаем, что даже пайплайны, построенные на глубоком обучении, могут выиграть от генерации признаков и сопутствующих техник. Яркий тому пример — история Арсения.

Байка от Арсения

Однажды я работал над ML-системой, в основе которой лежала модель глубокого обучения. Система принимала изображение, обрабатывала его с помощью глубокой модели и затем применяла постобработку без использования ML, чтобы в результате выдать определенное числовое значение. Однако неточности на первом этапе существенно влияли на итоговый вывод, и из-за низкой точности система не была готова к промышленному использованию. Улучшить первый компонент было затруднительно из-за серьезных ограничений по данным (система должна была работать в условиях few-shot, то есть эффективно функционировать при наличии лишь нескольких размеченных семплов на класс). Спасло ситуацию неожиданное решение — простая модель регрессии, уточняющая финальный результат. Благодаря использованию признаков, построенных вручную, модель не требовала большого объема данных. В результате сочетание глубокого обучения для выполнения основной задачи и простой модели на признаках, обеспечивающей индуктивное смещение, наладило работоспособность системы.

11.1.1. Критерии хороших и плохих признаков

Рассмотрим основные характеристики признаков, а также компромиссы, которые следует учитывать:

- *Производительность модели*. Признаки должны соответствовать бизнес-задаче и отражать релевантные аспекты данных, обеспечивая значимые зависимости с целевой переменной. Работая вместе с экспертами в какой-либо предметной области, можно не только сформировать корректный набор признаков, но и создать новые. Анализ важности признаков позволяет точно оценить значение каждого из них и понять, какие признаки действительно приближают нас к цели проекта.
- *Объем исторических данных*. Ограниченный объем исторических данных может привести к пропущенным значениям, что снижает общее качество датасета. Восполнение недостающих данных посредством одноразовой инженерной обработки — неплохое решение, но не всегда осуществимое. Кроме того, нехватка исторических данных мешает моделям прогнозирования выявлять тренды и сезонные колебания в значениях признаков.
- *Компромисс между количеством и качеством признаков*. Хотя наличие большего числа признаков может повысить прогнозирующую способность модели, избыточное количество нерелевантных или дублирующих признаков приведет к переобучению и в конечном счете ухудшит результат. Предпочтительнее, чтобы модель опиралась на небольшой, но проработанный и разнообразный набор, чем пыталась охватить множество сгенерированных и взаимосвязанных признаков.

- *Интерпретируемость и объяснимость*. Это ключевые факторы, которые необходимо учитывать при разработке признаков. Сложные признаки могут повысить производительность модели, но при этом снизить ее интерпретируемость и затруднить объяснение логики прогнозирования. Простые признаки легче интерпретировать, но они охватывают лишь часть данных. Необходимо найти баланс между объяснимостью и эффективностью, который, в свою очередь, зависит от конкретной системы и области применения.
- *Сложность разработки признаков*. Сложные признаки требуют больше времени на реализацию. Они могут зависеть от других признаков и сложных пайплайнов обработки данных либо основываться на новых источниках данных, что затрудняет их реализацию и сопровождение. От инженеров данных требуется больше усилий на построение и поддержку соответствующей инфраструктуры. Поэтому необходимо тщательно взвешивать затраты и потенциальные выгоды от каждого признака и только затем принимать решение, оправданна ли дополнительная сложность.
- *Стоимость признаков и соглашение об уровне обслуживания (SLA)*. Помимо вычислительной сложности отдельных признаков, следует учитывать общее время, необходимое для расчета всех признаков, а также объем оперативной памяти, требуемый для поддержания растущей нагрузки. Это включает время вычисления каждого признака и их взаимозависимости. Взаимодействие признаков определяет порядок их вычисления и возможность параллельной обработки. Например, для систем, работающих в режиме реального времени, признаки, требующие длительных расчетов, могут оказаться неприемлемыми из-за ограничений SLA. Также важно учитывать доступность данных в пайплайнах — как обучения, так и инференса. Если во время инференса невозможно получить все необходимые данные, модель выдаст неточные или неполные прогнозы.
- *Риски плохо спроектированных признаков*. Хрупкие признаки делают хрупкой всю систему. Они могут быть чувствительны к изменениям в данных или в модели, вызывая нестабильное или непредсказуемое поведение. Признаки, зависящие от внешних источников данных или API, могут быть подвержены изменениям или сбоям, что скажется на надежности модели. Чтобы этого избежать, необходимо тщательно тестировать и валидировать признаки перед их интеграцией в модель, а также отслеживать их поведение во времени, чтобы убедиться, что они продолжают приносить бизнес-ценность.

Генерация признаков — непрерывный процесс, поскольку бизнес-цели и распределение данных со временем меняются. Мы должны постоянно оценивать и обновлять набор признаков, чтобы он оставался актуальным и эффективно решал поставленные задачи. При разработке признаков важно отслеживать изменения, внесенные в каждый из них, включая версионирование и взаимозависимости. Это делает систему воспроизводимой и поддерживаемой.

11.1.2. Основы генерации признаков

Держа на вооружении описанные выше критерии и ограничения, можно перейти к рассмотрению наиболее распространенных способов генерации новых признаков.

Наиболее очевидный способ получить новый признак — добавить новый источник данных в пайплайн или использовать столбец, который ранее не был включен в датасет. Источник может быть как внутренним (например, существующая таблица в базе данных), так и внешним (например, данные, купленные у стороннего поставщика). Такие признаки не только доступны, но и ценны для модели. Однако они требуют значительных усилий со стороны инженерии данных и времени на сопровождение и могут создать инфраструктурные проблемы, так как повышение сложности всегда требует больше поддержки.

Помимо использования новых источников, существуют два альтернативных подхода — преобразование существующих признаков либо создание новых признаков на основе комбинации двух или более уже имеющихся.

Преобразование числовых признаков включает масштабирование, нормализацию и применение математических функций (например, использование логарифмов для улучшения асимметрии распределения). Тип модели определяет целесообразность применения того или иного преобразования. Например, часто можно наблюдать отсутствие улучшения метрик градиентного бустинга после применения монотонных преобразований признаков, поскольку основной элемент алгоритма — дерево решений — инвариантен к монотонным преобразованиям входных данных.

При работе с временными рядами часто используются такие преобразования, как лаги (сдвиг значений признака назад во времени для создания новых признаков), агрегаты (вычисление статистических показателей, таких как среднее, максимум или минимум, за заданный временной интервал), а также генерация статистических признаков на основе прошлых данных, таких как стандартное отклонение или дисперсия за различные временные периоды.

Разбиение по квантилям (или *квантилизация*) представляет собой частный случай преобразования — непрерывные признаки преобразуются в категориальные путем группировки значений по дискретным бакетам. Например, такой подход применяет Uber в своей модели DeepETA (https://mng.bz/zn6B) (рис. 11.1).

Такая сеть использует архитектуру трансформеров для прогнозирования предполагаемого времени прибытия машины, обрабатывая для этого разнообразные табличные данные. Данные, включающие непрерывные, категориальные и геопространственные признаки, преобразуются в дискретные токены, которые затем переводятся в обучаемые эмбеддинги, подходящие для трансформеров. Подробнее о DeepETA можно прочитать в статье «DeeprETA: An ETA Post-Processing System at Scale» Синью Ху (Xinyu Hu) и др. (https://arxiv.org/pdf/2206.02127.pdf).

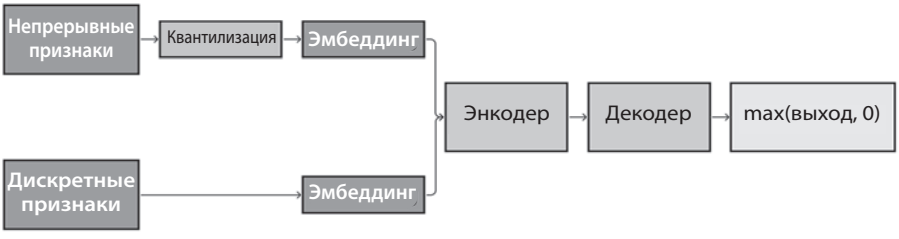


Рис. 11.1. Пайплайн модели DeepETA — пример сочетания базовой генерации признаков и глубокой нейронной сети (источник: https://mng.bz/0MoN)

Категориальные признаки зачастую требуют преобразований, которые могут выполняться с помощью методов, таких как one-hot кодирование, целевое кодирование по среднему значению, порядковое кодирование (этот метод упорядочивает категории на основе их естественного порядка) или хеширование, позволяющее обрабатывать большой объем категориальных данных. Важно отметить, что, несмотря на высокую эффективность, целевое кодирование по среднему значению при некорректной реализации может привести к утечке данных, поскольку для создания новых признаков оно использует информацию из целевой переменной.

Для последовательных данных, например текста, можно применять такие техники, как мешок слов, TF-IDF (term frequency–inverse document frequency) и BM25, чтобы преобразовать данные в формат, пригодный для обработки ML-алгоритмами. Следует также отметить, что данные методы теряют информацию о порядке слов. Такой недостаток частично компенсируется использованием более длинных N-грамм вместо отдельных слов (униграмм). Также можно использовать предварительно обученные языковые модели, такие как BERT, чтобы представлять входные данные в виде эмбеддингов в пространстве низкой размерности, которые затем подаются на вход итоговой модели.

Важно помнить, что почти любые последовательные данные можно представить в виде токенов, не обязательно только тексты. Например, в сфере онлайн-торговли или стриминговых сервисах пользовательская сессия может рассматриваться как последовательность просмотренных страниц товаров или видеороликов. Каждая посещенная страница будет иметь свое обучаемое представление (эмбеддинг). Такие эмбеддинги можно использовать в системе рекомендаций в качестве промтов в задаче предсказания следующей страницы, чтобы определить, какой товар или видео может заинтересовать пользователя.

Если необходимо включить эмбеддинги товаров в табличный датасет, одним из распространенных подходов является анализ расстояний между товарами. Примерами признаков в этом случае могут быть следующие:

- Насколько близки пять ближайших соседей к товару X?
- Какова средняя/минимальная цена пяти ближайших товаров для товара X?
- Какова абсолютная/относительная разница в цене между товарами X и Y?

Несмотря на то что подобные признаки повышают сложность пайплайнов обучения и инференса, они способны значительно улучшить производительность модели.

А что, если нам требуется объединить сигналы от нескольких признаков в один? Когда в датасете имеется несколько признаков, мы можем комбинировать их и создавать более информативные и значимые для модели признаки. Например, вместо того чтобы иметь отдельные признаки для количества кликов и количества покупок, совершенных пользователем на сайте онлайн-магазина, можно объединить их в новый признак — «соотношение покупок и кликов» (purchase-to-click ratio), который станет более точным индикатором намерения пользователя совершить покупку.

В случае агрегатора такси, вместо того чтобы использовать отдельные признаки, такие как «количество поездок» и «общая дистанция», можно объединить их и создать новый признак — «среднее расстояние за поездку», который даст более ценные сведения о поведении водителей и пассажиров.

Также следует учитывать взаимосвязь между существующими признаками. Например, абсолютные продажи продукта за определенный период могут быть недостаточно информативными. Однако сравнение этих продаж с продажами других товаров в той же категории или с продажами за предыдущие периоды может выявить важные закономерности или тренды. Объединение сигналов от нескольких признаков позволяет создавать новые признаки, фиксирующие более сложные взаимосвязи в данных и повышающие эффективность модели.

Техника объединения нескольких признаков называется *взаимодействием признаков* (feature interaction), или *пересечением признаков* (feature cross). Данная техника особенно важна для линейных моделей, поскольку такие признаки могут обеспечить линейную разделимость точек данных.

11.1.3. Прогнозы модели как признак

Как мы уже обсуждали ранее, если один признак зависит от другого, то любые изменения или обновления второго могут потребовать соответствующих изменений или обновлений первого. Это усложняет процесс сопровождения и отладки и со временем повышает общую сложность системы.

Прогнозы модели можно рассматривать как частный случай признака, когда выходные данные одной модели используются в качестве входных данных для другой модели или системы. Такой подход иногда называют *стекингом моделей*. И хотя использование прогнозов моделей в качестве признаков может быть эффективно, оно часто влечет за собой определенные инженерные сложности и риски.

Простейший пример использования прогнозов в качестве признака — это целевое кодирование (https://maxhalford.github.io/blog/target-encoding/), при котором

категориальный признак кодируется средним значением целевой переменной (с определенной степенью регуляризации) и используется в модели как числовой признак. Однако если кодирование опирается на информацию из обучающей выборки, которая недоступна во время инференса, существует риск утечки данных. Это может привести к переобучению и низкому качеству работы модели на новых данных, если не применить более сложные методы валидации, например вложенную кросс-валидацию (см. главу 7).

Другой пример — использование моделей сторонних разработчиков (например, метеопрогнозов в модели прогнозирования спроса). И хотя погодные данные могут быть весьма информативными, существует риск, что у прогноза может отсутствовать необходимая историческая глубина. В таком случае предпочтение следует отдавать прогнозам с достаточной историчностью, а не просто высокой точностью. Кроме того, зависимость от внешних источников данных может ввести дополнительные зависимости и риски, выходящие за рамки работы ML-команды.

Наконец, использование сторонних моделей или моделей с открытым исходным кодом в качестве экстракторов признаков в системах глубокого обучения также может нести определенные риски. Несмотря на то что создаваемые эмбеддинги могут эффективно улавливать полезные паттерны в данных, существует риск дрейфа модели или ее нестабильности, если внешняя модель обновляется без должного контроля версий. Обратная ситуация тоже опасна: при отсутствии обновлений модель может утратить ценность из-за изменений в распределении входных данных. Это может привести к непредсказуемому поведению и резкому снижению качества ML-системы.

Чтобы минимизировать такие риски, необходимо тщательно продумывать пайплайн генерации признаков и внедрять надежные процедуры тестирования и мониторинга (описание тестирования см. в главах 10 и 13, мониторинг рассматривается в главе 14). Сюда относятся кросс-валидация и другие методы предотвращения утечки данных и проверки достоверности внешних источников и моделей, а также процессы отслеживания и актуализации признаков.

11.2. Анализ важности признаков

После того как вы выбрали первоначальный базовый набор признаков для своей модели, необходимо понять, какие из признаков в наибольшей степени влияют на прогнозы модели и предоставляют информацию о том, как именно модель принимает решения и как ее можно улучшить.

ML-модели часто воспринимаются как некие «черные ящики», которые не дают никакого представления о процессе получения прогнозов. Такая непрозрачность может представлять проблему как для инженеров, так и для стейкхолдеров или конечных пользователей, которым важно понимать логику решений, принимаемых моделью.

Для повышения прозрачности моделей мы используем два ключевых понятия: интерпретируемость и объяснимость. Обе концепции служат «демистификации» работы модели.

- *Интерпретируемость* связана с пониманием внутренней механики модели и позволяет выяснить, как и почему модель генерирует свои прогнозы.
- *Объяснимость* описывает возможность объяснить поведение модели на уровне, понятном обычному человеку, даже если внутренняя структура модели сложна или непрозрачна (https://mng.bz/KDvj).

Анализ важности признаков служит инструментом для достижения как интерпретируемости, так и объяснимости, поскольку позволяет точно определить те признаки, которые в наибольшей степени влияют на прогнозы модели. Результаты анализа важности признаков фиксируются в артефактах пайплайна обучения и могут играть роль в процессе верификации модели, по итогам которой принимается решение — развертывать ли новую версию модели или нет (подробности см. в главе 13). Хорошим примером может послужить система, определяющая стоимость поездки в приложении агрегатора такси, как показано на рис. 11.2.

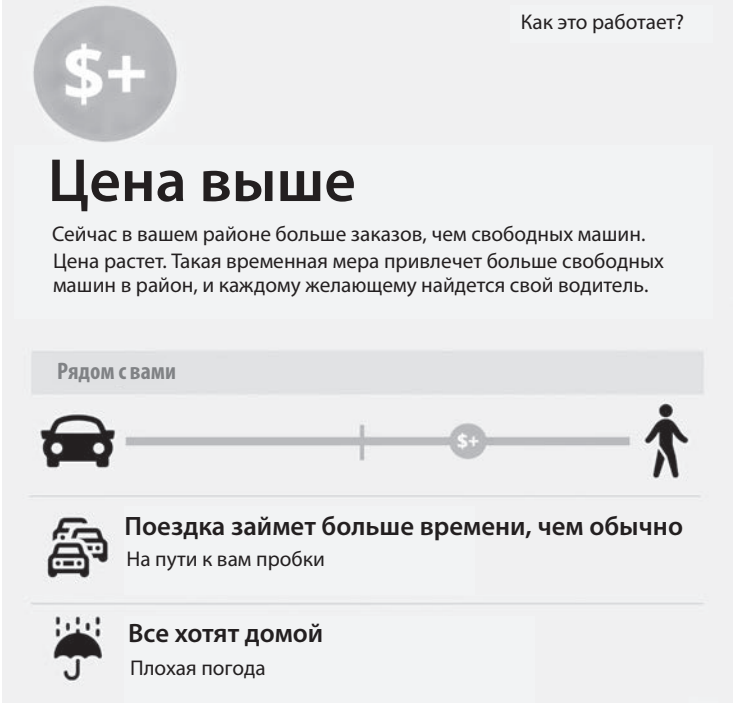


Рис. 11.2. Пример пользовательского интерфейса приложения такси, поясняющего, почему алгоритм динамического ценообразования выбрал именно такую цену в определенном районе и в определенное время суток

Приложение работает со всеми ключевыми признаками и анализирует текущие оперативные данные: плотность движения, погодные условия и т. д. — для расчета конечной стоимости поездки. Однако помимо этого система предоставляет обоснование предложенной цены в удобной и понятной форме. Благодаря этому пользователь получает объяснение, почему поездка, которая обычно стоит меньше, внезапно подорожала.

Кроме того, анализ важности признаков может повысить уровень доверия к ML-системе. Это особенно важно в критически значимых областях, таких как медицина или финансы. И хотя Общий регламент по защите данных (GDPR) (https://gdpr.eu) не требует соблюдения критерия объяснимости, он подразумевает определенную степень прозрачности в процессе автоматизированного принятия решений, что может быть полезно или даже необходимо во многих ML-приложениях (https://mng.bz/9oX7).

Выявление наиболее значимых признаков помогает объяснить, какие переменные лежат в основе прогнозов модели и почему. Эта информация может быть использована для оптимизации соответствующих признаков с целью повышения качества модели, а также для исключения нерелевантных или избыточных признаков с целью повышения эффективности. Кроме того, она может служить ориентиром при отладке модели, например, при выявлении переобучения или оценке полезности недавно добавленных признаков.

11.2.1. Классификация методов

Рассмотрим методы анализа важности признаков и то, как они могут быть использованы для повышения прозрачности и эффективности ML-систем.

Выбор подходящего метода может показаться сложной задачей, но карта доступных подходов укажет правильное направление. Методы можно классифицировать по различным свойствам, таким как тип модели, уровень ее интерпретируемости и тип используемых признаков.

Классическое ML vs глубокое обучение

Методы, применяемые для анализа важности признаков, могут существенно различаться в зависимости от того, используется ли классическое ML или глубокое обучение. Для моделей классического ML, где признаки зачастую выбираются вручную на основе экспертных знаний или статистического анализа, определение важности признаков достаточно прозрачно — можно напрямую проанализировать веса модели и правила принятия решений либо исключить/изменить отдельный признак и оценить его вклад в прогнозы модели.

С другой стороны, в моделях глубокого обучения, которые автоматически извлекают представления признаков из данных, проанализировать важность труднее. Из-за сложных нелинейных преобразований и высокого уровня абстракции невозможно просто посмотреть на параметры модели, чтобы понять важность

признаков. Вместо этого, чтобы понять, что происходит внутри нейросетей, мы полагаемся на более продвинутые подходы, такие как карты значимости, максимизация активации (подробнее см. в статье «Visualizing Deep Convolutional Neural Networks Using Natural Pre-Images» Аравиндха Махендрана (Aravindh Mahendran) и др. (https://arxiv.org/abs/1512.02017) или послойное распределение релевантности (подробнее см. «Layer-wise Relevance Propagation for Neural Networks with Local Renormalization Layers» Александра Биндера (Alexander Binder) и др. (https://arxiv.org/abs/1604.00825), а также на методы, основанные на данных подходах. Разумеется, существуют и другие подходы, однако ни один из них не является универсальным, поскольку проблема объяснимости глубокого обучения еще не решена и остается предметом активных научных исследований.

Специфичные для модели vs независимые от модели методы

Методы, специфичные для модели (model-specific), используют ее структуру и параметры для оценки важности признаков. Например, в моделях, основанных на деревьях, мы можем подсчитать, сколько раз разбиение происходило по конкретному признаку во время обучения, или общий выигрыш, который этот признак дает на всех разбиениях. Аналогично, в линейных моделях мы можем анализировать величину и знак коэффициентов, назначенных каждому признаку.

В свою очередь, методы, не зависящие от модели (model-agnostic), рассматривают ее как «черный ящик». Они основаны на возмущении входных данных и наблюдении за эффектом, который это возмущение оказывает на выход модели. Примеры независимых от модели методов:

- *Перестановочная важность признаков* — измеряет важность каждого признака путем случайной перестановки его значений в датасете и оценки соответствующего снижения производительности модели.
- *Значения SHAP (SHapley Additive exPlanations values)* — оценивает вклад каждого признака в конкретный прогноз путем усреднения по всем возможным комбинациям признаков.

Интерпретация отдельных прогнозов vs интерпретация всей модели

Еще одно важное различие — ориентированы ли методы на отдельные прогнозы или на модель в целом (рис. 11.3). Методы, фокусирующиеся на отдельных прогнозах, оценивают важность признаков для конкретного входа, объясняя, почему модель приняла то или иное решение, тогда как методы, интерпретирующие всю модель, оценивают важность признаков в более общем смысле, описывая поведение модели в целом.

Примеры методов, ориентированных на отдельные прогнозы, включают: LIME (Local Interpretable Model-agnostic Explanations) — см. статью «Why Should I Trust You?» Марко Тулио Рибейро (Marco Tulio Ribeiro) и др. (https://arxiv.org/abs/1602.04938), где упрощенная и более интерпретируемая модель (например, линейная регрессия) используется для аппроксимации границы решений вокруг

конкретного входа, и Anchor Explanations — см. статью «Anchors: High-Precision Model-Agnostic Explanations» Марко Тулио Рибейро и др. (https://mng.bz/j0vr), где выявляется правило, которое «закрепляет» принятое решение, делая его понятным для человека.

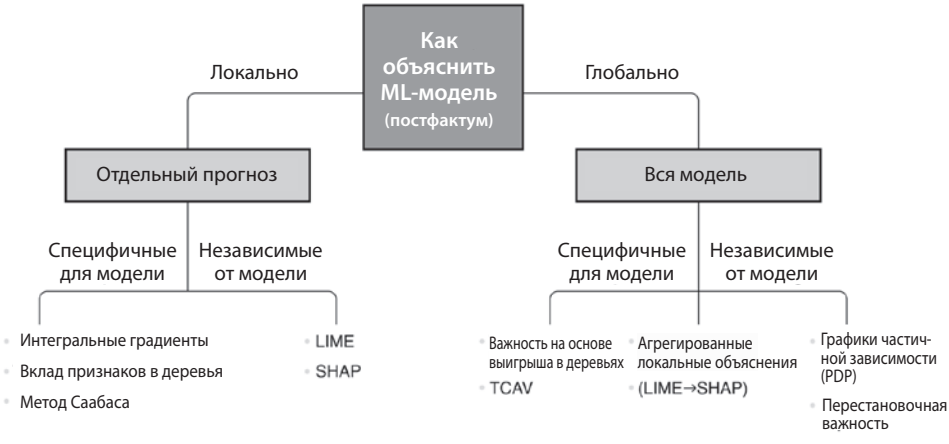


Рис. 11.3. Систематизация методов интерпретации моделей

Зачастую для получения более полной картины мы используем комбинацию методов, чтобы компенсировать ограничения отдельных подходов. Однако важно помнить, что не существует универсального метода, способного дать исчерпывающий ответ на все вопросы, связанные с важностью признаков. Выбор метода должен соответствовать конкретной задаче и контексту.

11.2.2. Компромисс между точностью и интерпретируемостью

Модели с высокой интерпретируемостью могут жертвовать точностью ради прозрачности, и наоборот, модели с высокой точностью часто оказываются менее интерпретируемыми (рис. 11.4). Современные крупные языковые модели, основанные на трансформерах, такие как GPT, — яркий тому пример. Они произвели революцию в области ML, достигнув передовых результатов во множестве задач обработки естественного языка (NLP), однако такие модели крайне сложны и содержат миллиарды параметров, что затрудняет понимание логики их решений.

Компромисс между точностью и интерпретируемостью остается сложной задачей, особенно с появлением новых, еще не изученных архитектур. Выбор метода должен быть адаптирован к конкретной задаче и учитывать такие факторы, как значимость интерпретируемости, сложность модели и желаемый уровень точности.

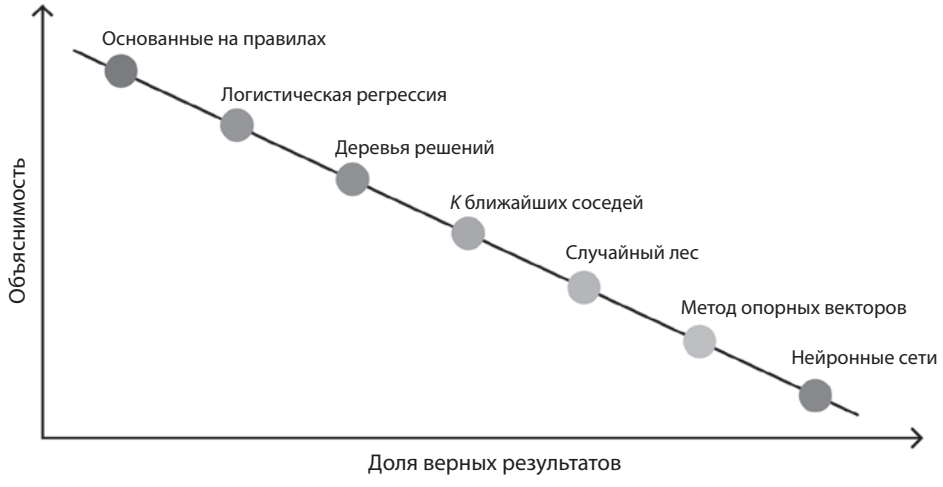


Рис. 11.4. Чем сложнее модель (а значит, и точнее), тем менее объяснимой она становится

11.2.3. Важность признаков в глубоком обучении

Анализ важности признаков для моделей, работающих с табличными данными, — это задача с понятной формулировкой и очевидным решением: у нас есть четко выделенные признаки и хорошо известные инструменты для оценки того, как каждый из них влияет на модель, целевую переменную или итоговую метрику.

Однако в контексте глубокого обучения, особенно при работе с такими типами данных, как изображения, аудио или текст, определение важности признаков становится менее очевидным и более сложным. Модели глубокого обучения по своей природе автоматически обучаются иерархическим представлениям данных, зачастую в высокоабстрактной и нелинейной форме. В таких случаях «признаком» может выступать все что угодно: от отдельного пикселя изображения до отдельного слова или символа в тексте, от конкретной частоты в аудиозаписи до сложных характеристик, таких как расположение объекта на изображении, стиль предложения в тексте или определенный звуковой паттерн в голосовой записи.

Тем не менее все еще возможно получить представление о том, какие элементы исходных данных модель считает важными и на какие паттерны она обращает внимание. Рассмотрим несколько техник анализа важности признаков в глубоком обучении:

- *Значения SHAP.* Подобно классическим признакам, значения SHAP оценивают вклад каждого токена или пикселя в выход модели, предоставляя независимое от модели объяснение того, какие именно части входных данных оказывают наибольшее влияние на модель.

- *Карты значимости.* Карты значимости представляют собой форму локального объяснения, выделяющего области входного изображения, чувствительные к выходу модели. По сути, они рассчитывают градиент выхода модели по входу, создавая тепловую карту, в которой каждый пиксель показывает, насколько его изменение повлияет на результат. Примеры методов построения карт значимости: GT, SalNet, SALICON и SalClassNet. Рисунок 11.5 взят из статьи «Top-Down Saliency Detection Driven by Visual Classification» Франчески Мурабито (Francesca Murabito) и др. (https://arxiv.org/abs/1709.05307), описывающей общие принципы работы карт значимости.

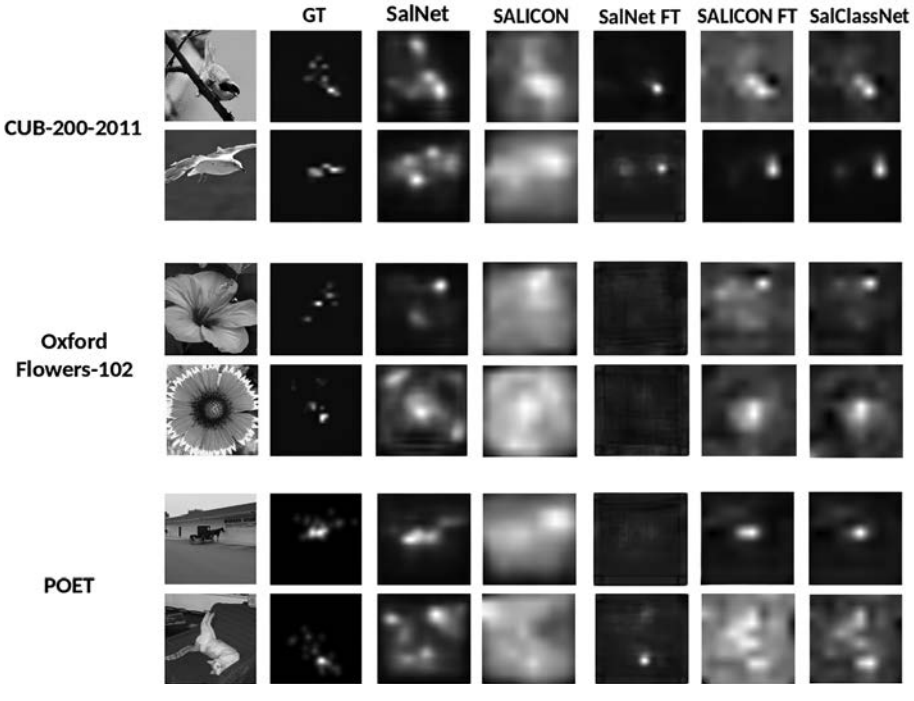


Рис. 11.5. Примеры карт значимости выходных данных, сгенерированные различными методами (источник: https://arxiv.org/abs/1709.05307)

- *Методы на основе возмущений.* Эти методы определяют важность признака, наблюдая за изменением вывода модели при его модификации или удалении. Хорошим примером такого подхода является метод окклюзии, который используется в основном в задачах компьютерного зрения и заключается в систематическом закрытии (маскировке) отдельных участков изображения с последующим анализом изменений в выходе модели. Идея окклюзии впервые была предложена в работе Мэттью Д. Целлера (Matthiew D. Zeller) и др. «Visualizing and Understanding Convolutional Networks» (https://arxiv.org/abs/1311.2901) и активно использовалась в более поздних разработках, таких

как «RISE: Randomized Input Sampling for Explanation of Black-Box Models» Виталия Петсюка (Vitali Petsiuk) и др. (https://arxiv.org/abs/1806.07421). Данный метод позволяет визуализировать, какие области изображения модель считает наиболее релевантными для своих прогнозов.

- *Механизм внимания в трансформерах.* В архитектуре трансформеров механизм внимания является естественным способом оценки важности признаков. Оценки внимания показывают, насколько модель учитывает каждый токен входной последовательности при формировании своего прогноза. Данные веса внимания можно визуализировать и интерпретировать как область фокуса модели, что позволяет понять, как модель (а точнее, конкретная голова внимания) «читает» и интерпретирует входной текст (рис. 11.6).

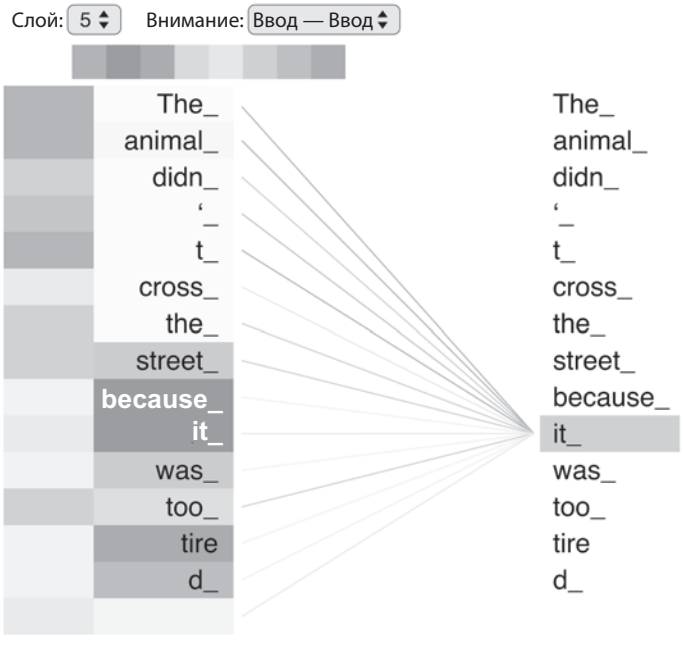


Рис. 11.6. Визуализация головы внимания в архитектуре Encoder-Decoder трансформера (текст: «Животное не перешло дорогу, потому что оно слишком устало», фокус на «it» — «оно»)

Как можно заметить, здесь начинают прослеживаться параллели с классическим ML. Несмотря на то что в моделях глубокого обучения достаточно сложно анализировать важность признаков, существуют методы, позволяющие получить представление о том, как модель принимает решения и какие паттерны являются ключевыми для целевой переменной.

11.3. Отбор признаков

> Совершенство — это не когда нечего больше добавить. Совершенство — это когда нечего больше отнять.
>
> *Антуан де Сент-Экзюпери*

В предыдущих разделах мы рассмотрели искусство генерации признаков и способы преобразования необработанных данных в значимые характеристики. Однако не все признаки одинаково полезны — некоторые из них могут оказаться нерелевантными, избыточными или слишком сложными для их эффективной обработки моделью.

Именно здесь на помощь приходит отбор признаков. Тщательно выбирая наиболее информативные признаки, мы можем повысить производительность и интерпретируемость системы, одновременно снижая ее сложность и время обучения. Мы рассмотрим соответствующие техники, лучшие практики и возможные подводные камни, а также узнаем, как выбрать оптимальные признаки для конкретной проблемы ML.

11.3.1. Генерация признаков vs отбор признаков

Процессы генерации и отбора признаков в ML можно сравнить с садоводством. Подобно садовникам, которые высаживают в землю различные семена, мы создаем множество признаков, исследуем новые источники данных, экспериментируем с различными преобразованиями признаков и генерируем идеи, которые могут улучшить производительность модели.

Однако как не все растения в саду приживаются, так и не все признаки оказываются полезными для модели, и в какой-то момент нам приходится избавляться от мертвых или бесполезных растений (в нашем случае — от нерелевантных или избыточных признаков), чтобы поддерживать здоровый рост других. Этот цикл выращивания и прореживания, добавления и сокращения непрерывен в жизни ML-систем, поскольку мы постоянно совершенствуем и оптимизируем наборы признаков.

Древнегреческий философ Гераклит однажды сказал: «Борьба — отец всего. Гармония мира складывается из противоположностей». Это утверждение справедливо и в ML, где мы достигаем оптимальной производительности, поддерживая баланс между созданием новых признаков и тщательным отбором наиболее информативных из них.

11.3.2. Цели и возможные недостатки

Вы можете спросить: «Хорошо, но зачем вообще так много внимания уделять отбору признаков?» В этом есть определенные преимущества:

- *Повышение точности и снижение переобучения*. Отбор наиболее информативных признаков помогает модели сосредоточиться на важных сигналах. Исключение нерелевантных или избыточных признаков снижает риск переобучения, при котором модель становится слишком сложной и подстраивается под шум в обучающих данных, а не под реальные закономерности.
- *Упрощение интерпретации*. Решение, основанное на 10 значимых признаках, легче интерпретировать и понять, чем решение, опирающееся на 100 признаков, даже если производительность модели в последнем случае выше.
- *Упрощение разработки и отладки*. Если на этапе отбора признаков мы выяснили, что три из пяти источников данных оказались избыточными, мы можем сэкономить значительное количество времени и вычислительных ресурсов, исключив их из пайплайнов обучения и развертывания. Более простой пайплайн данных требует меньше усилий для его сопровождения и устранения сбоев.
- *Уменьшение времени на обучение и сервинг*. Сокращение числа признаков понижает сложность модели, что приводит к более быстрому обучению и меньшим затратам на вычисления.

Для удобства все преимущества отбора признаков собраны в единую схему на рис. 11.7.



Рис. 11.7. Преимущества отбора признаков

В задачах с обработкой в реальном времени критерий скорости зачастую выходит на первый план, даже если ради этого приходится жертвовать точностью

модели для соблюдения соглашений об уровне обслуживания (SLA). Например, в системах распознавания речи, используемых в виртуальных помощниках, пользователи ожидают мгновенной и точной трансформации произнесенных слов в текст. Даже малейшая задержка может испортить пользовательский опыт и создать впечатление низкой эффективности системы.

Идеальная персонализация теряет всякую ценность, если задержка в прогнозе составляет 300 мс, и негативно воспринимается пользователем. Поэтому целесообразным оказывается более легкий подход к персонализации с умеренным качеством, а не модель, собирающая все возможные входные данные от пользователя, но заставляющая его ждать.

Пример из практики

Компания Amazon однажды провела серию A/B-тестов, чтобы выразить данный компромисс в конкретных цифрах: каждая задержка в 100 мс снижает продажи на 1 %. Для компании с годовыми доходами в $500 млрд потеря в 1 % равна 5 млрд — такой убыток она себе позволить, разумеется, не может (https://mng.bz/WVga).

Существуют также потенциальные риски и недостатки отбора признаков помимо необходимости балансировать между вычислительным временем и точностью:

- *Потеря потенциально ценных сигналов*. Удаление признаков может привести к потере важной информации, которая могла бы в будущем улучшить модель. Мы можем упустить перспективные методы предварительной обработки или агрегирования данных и поспешно заключить, что признак не несет полезного сигнала.
- *Непредвиденные взаимодействия*. Удаление определенных признаков может привести к возникновению непредвиденных взаимодействий между оставшимися признаками, что станет причиной неожиданного поведения модели и снижения ее производительности. При выборе признаков крайне важно учитывать их взаимосвязь и возможные взаимодействия.
- *Смещение*. Некоторые признаки могут получать больший вес по сравнению с другими, в результате чего отбор признаков может привести к смещенным прогнозам. Представьте, что мы выбираем только те признаки, которые сильно коррелируют с целевой переменной. В таком случае мы рискуем внести смещение в модель и не учесть признаки, которые хоть и не демонстрируют высокой корреляции, но все же важны для прогнозирования.
- *Риск переобучения*. Подобно оптимизации гиперпараметров, отбор признаков представляет собой процедуру обучения, а любая процедура обучения, использующая целевую переменную, требует правильной схемы валидации (см. главу 7). Если мы используем одни и те же данные и для выбора при-

знаков, и для оценки производительности модели, мы рискуем переобучить модель на тестовых данных, что приведет к чрезмерно оптимистичным оценкам производительности.

Помимо этих проблем регулярное применение отбора признаков добавляет к этапу обучения дополнительные вычислительные затраты, что также следует учитывать, особенно при использовании жадных оберточных методов.

11.3.3. Методы отбора признаков

Существует множество методов отбора признаков, каждый из которых имеет свои достоинства и недостатки (рис. 11.8). Наиболее распространенными подходами являются фильтрационные методы, оберточные методы и встроенные методы. Рассмотрим подробнее каждый из них.

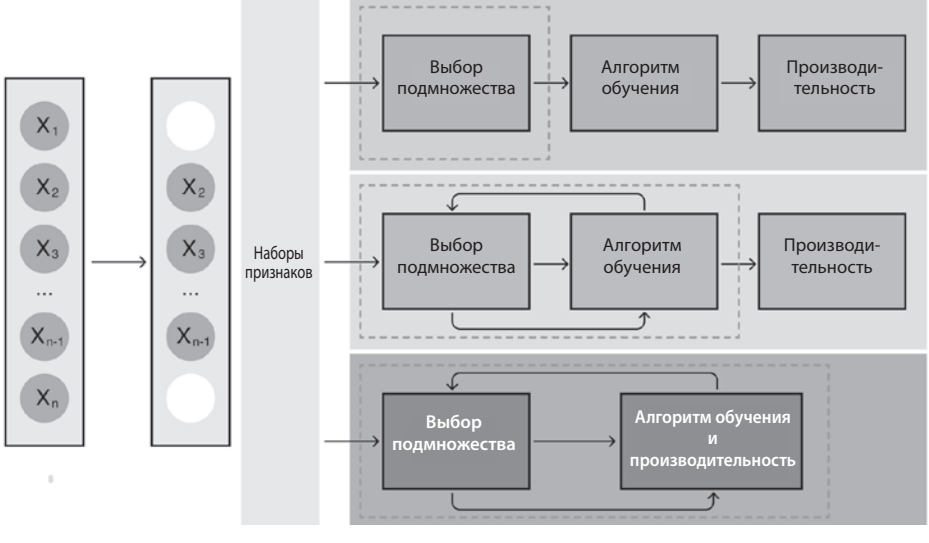


Рис. 11.8. Семейства методов отбора признаков

Фильтрационные методы выполняют отбор признаков независимо от модели, применяя простые правила ранжирования на основе статистических характеристик отдельного признака (унивариантные методы) или корреляции между признаками (мультивариантные методы). Данные методы хорошо масштабируются (в том числе на задачах с высокоразмерными данными) и позволяют быстро выполнить предварительный отбор признаков перед выполнением основной задачи.

Порядок ранжирования признаков в унивариантных фильтрационных методах определяется их внутренними свойствами, такими как дисперсия признака,

его детализация, согласованность, корреляция с целевой переменной и др. После этого мы отбираем верхние *N* признаков в качестве подмножества и либо обучаем модель, либо применяем более сложные, ресурсоемкие методы отбора признаков в качестве второго уровня фильтрации.

В мультивариантных методах признаки анализируются во взаимосвязи друг с другом (например, путем оценки ранговой корреляции или взаимной информации). Если два признака несут схожую информацию, один из них можно исключить без ущерба для производительности модели. Например, показатель взаимодействия признаков (вне зависимости от способа его вычисления) может быть включен в автоматический отчет. При высоком значении данного показателя может быть сгенерировано предупреждение о возможном снижении качества модели еще до начала обучения.

Оберточные методы (wrapper methods) сосредоточиваются на наборе признаков, которые способствуют повышению качества модели на основе выбранной метрики. Свое название они получили потому, что алгоритм обучения буквально «обернут» этими методами. Они также требуют проектирования корректной схемы валидации, вложенной во внешнюю валидацию (о выборе подходящей схемы см. главу 7).

К оберточным методам относятся последовательные и эволюционные алгоритмы. Примеры последовательных алгоритмов включают прямой пошаговый отбор, при котором признаки добавляются по одному, начиная с пустого набора; обратное пошаговое исключение (SBE, sequential backward elimination), при котором признаки удаляются по одному; и их гибриды — промежуточные версии, допускающие повторное включение ранее удаленного признака и наоборот. В эволюционных алгоритмах подмножества признаков выбираются случайным образом. Распространенный пример эволюционного алгоритма — использование варианта дифференциальной эволюции в пространстве бинарных масок признаков, где «1» обозначает включенный признак, а «0» — исключенный.

Основной недостаток таких методов заключается в высоких вычислительных затратах, а также в их склонности к сходимости в локальные оптимумы. Тем не менее они обеспечивают наиболее точную оценку влияния признаков на целевую метрику. Используйте их с осторожностью, особенно если ваше оборудование имеет ограниченные ресурсы.

Во *встроенных методах* (embedded methods) применяется дополнительная «встроенная» модель (которая может относиться или не относиться к тому же классу, что и основная модель), а решения принимаются на основе значимости признаков, определенной этой моделью. Пример — Лассо-регрессия — из-за способности L1-регуляризации уменьшать коэффициенты признаков, не имеющих значения для целевой переменной, до нуля, как показано на рис. 11.9.

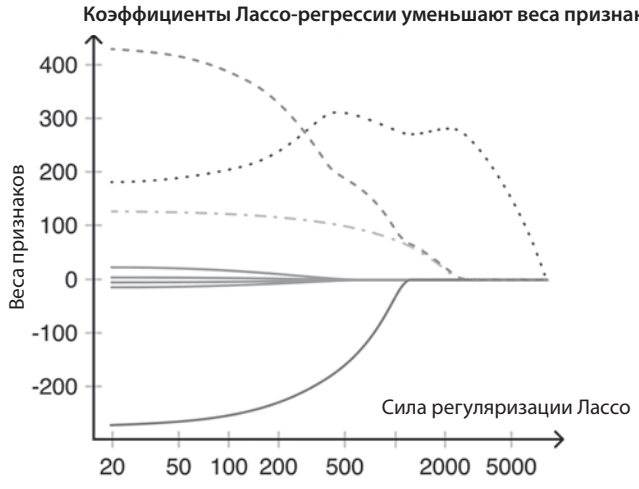


Рис. 11.9. Лассо-регрессия исключает признаки по одному, уменьшая их коэффициенты до нуля по мере роста коэффициента L1-регуляризации

Другим широко используемым алгоритмом отбора признаков является рекурсивное исключение признаков (RFE, recursive feature elimination), которое включает обучение модели и последовательное удаление K наименее значимых признаков на каждом шаге на основе важности признаков встроенной модели.

С точки зрения вычислительной сложности и качества отбора признаков встроенные методы находятся где-то между фильтрационными и оберточными методами.

Для первоначального отбора мы предпочитаем начать с грубых отсечек с целью сокращения числа признаков. Затем, если нужно, выбираем между Лассо или RFE — в зависимости от того, какой из них выдает более осмысленные подмножества. Вычислительно затратные методы могут дмонстрировать лучшую производительность, но быстрые методы зачастую достаточно хороши для первичного прореживания признаков, особенно если есть подозрение, что некоторые признаки совершенно бесполезны.

Существует множество примитивных, но при этом полезных методов, которые могут служить разумной базой для отбора признаков. Например, можно взять признак, перемешать его значения, добавить обратно в исходный датасет и обучить новую модель. Если значимость исходного признака оказывается ниже, чем значимость данного случайного признака (так называемого *теневого признака*), то с высокой вероятностью он не имеет отношения к проблеме. Такой алгоритм можно считать простейшим вариантом оберточного метода.

Байка от Валерия

Один из проектов Валерия был связан с динамическим ценообразованием, и в рамках него использовалась модель, которую требовалось улучшить. Причина была проста — модель не обеспечивала необходимой точности, и после базового анализа ошибок выяснилось, что большинство ошибок происходило по SKU с большим объемом продаж.

Дальнейшее исследование показало, что некоторые признаки, особенно основанные на истории цен, были критически важными, в то время как другие оказывались почти незначительными. Исключив неинформативные признаки с помощью Лассо-регрессии, удалось упростить модель. Сокращение числа позволило применить простые взаимодействия признаков (полиномиальные комбинации оставшихся признаков) благодаря уменьшению общего количества признаков (10^2 — это 100, а 20^2 — уже 400). Сначала это не помогло из-за специфической предварительной обработки, но после ее изменения эффект стал более заметен: входные данные были нормированы в диапазон (0..1), и некоторые взаимодействия превращались в нули, поскольку один из компонентов равнялся нулю независимо от второго, так что 0*0 и 1*0 давали одинаковый результат, хотя на самом деле они разные. Поэтому, адаптировав масштаб до диапазона (1..10), чтобы избежать проблемы умножения на ноль, преобразовав числа в формат float16 для снижения потребления памяти, применив полиномиальные взаимодействия признаков (что стало проще благодаря ограниченному диапазону 1..10), а затем вновь отмасштабировав до диапазона 1..10 и обучив простую гребневую регрессию на новых признаках, удалось снизить ошибку на 30 %.

Стоит отметить, что предыдущие попытки улучшения модели использовали более сложные алгоритмы, такие как градиентный бустинг и нейросети, но генерация признаков оказалась более быстрым и эффективным путем, превзойдя сложные модели.

Байка от Арсения

Однажды Арсений работал над системой, представлявшей собой, по сути, задачу классификации текста: по описанию транзакции (зашумленная, частично усеченная строка, полная аббревиатур) и ее дополнительным атрибутам (например, сумма переведенных денег) системе требовалось классифицировать транзакцию. Модели на базе трансформеров продемонстрировали свою эффективность в обработке текста, а вот с дополнительными атрибутами возникли проблемы.

Итоговое решение основывалось на модели-трансформере типа BERT, с мультикомпонентным промтом на входе. Этот промт содержал как текстовый ввод, так и различные признаки, вручную извлеченные из атрибутов транзакции. Работа с признаками (включая анализ важности и отбор) позволила дополнительно усовершенствовать систему по целевым метрикам даже лучше, чем более распространенные варианты усовершенствования моделей глубокого обучения, такие как предварительное обучение бэкбона или сложные функции потерь.

11.4. Хранилище признаков

Мы подошли к важному паттерну проектирования, объединяющему множество техник, рассмотренных в этой главе, в единый компонент, — *хранилищу признаков*. Оно позволяет командам вычислять, сохранять, агрегировать, тестировать, документировать и отслеживать признаки для своих пайплайнов ML в централизованном хабе.

Представьте, что вы потратили несколько недель на разработку сложных признаков, а затем в случайной беседе с коллегой во время кофе-брейка узнали, что другая команда уже реализовала и протестировала точно такие же признаки. Или, например, к вам обращается коллега за помощью в реализации конкретного признака. Вы знаете, что этот признак у вас уже реализован, но понимаете, что ваш код не задокументирован должным образом и непригоден для повторного использования из-за его зависимости от других компонентов в репозитории. В итоге ваш коллега решает, что проще сделать все заново. Такое несоответствие может привести к тому, что каждый инженер будет реализовывать и вычислять все признаки самостоятельно, и в итоге это обернется неприемлемыми тратами ресурсов компании (рис. 11.10).

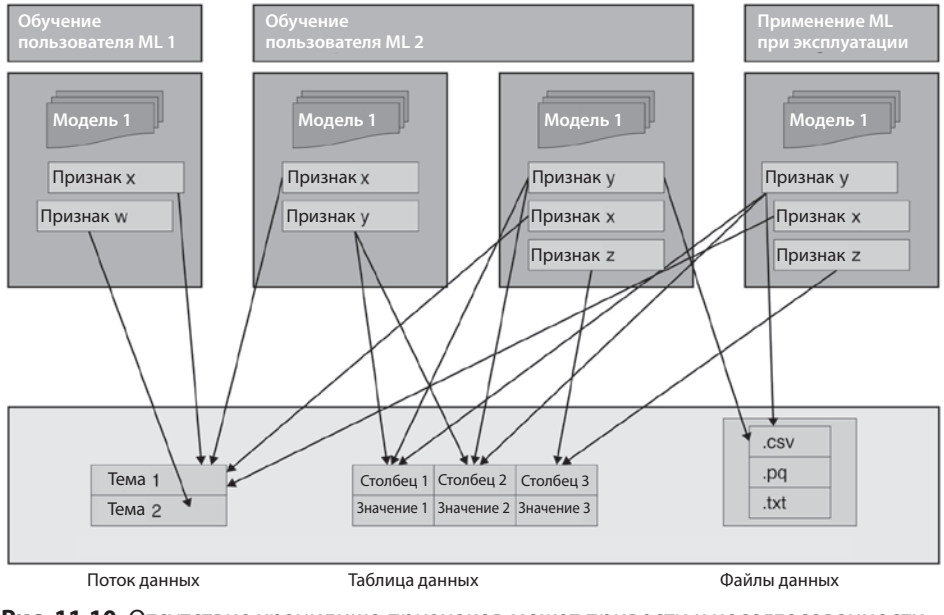


Рис. 11.10. Отсутствие хранилища признаков может привести к несогласованности и чрезмерным трудозатратам

Подобные сценарии иллюстрируют лишь малую часть проблем, которые может решить внедрение хранилища признаков. С ним мы отходим от фрагментированного подхода, когда каждая команда реализует и рассчитывает

признаки независимо от других. Вместо этого мы применим унифицированную систему, позволяющую повторное использование признаков, как показано на рис. 11.11.

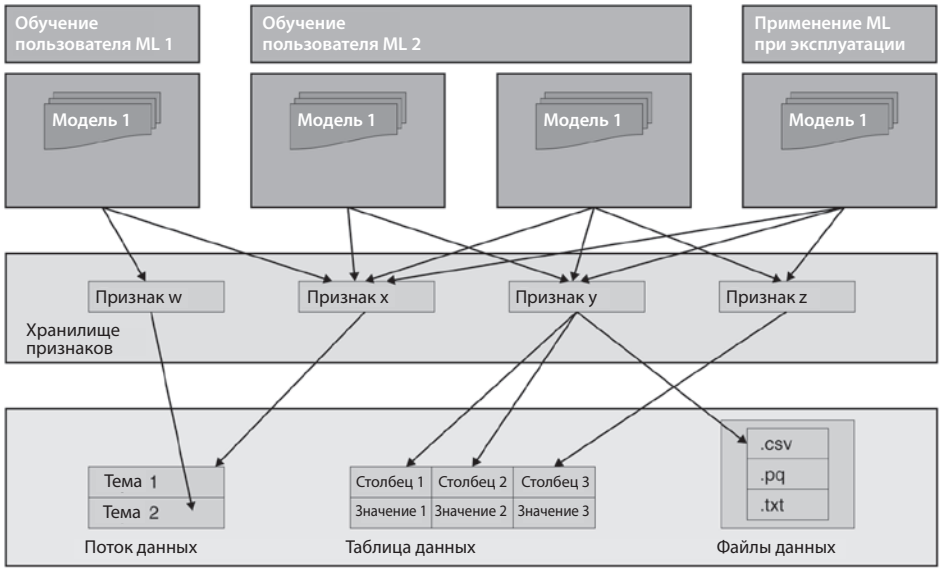


Рис. 11.11. Хранилище признаков делает возможным повторное использование признаков за счет их унифицированной реализации

11.4.1. Хранилище признаков: плюсы и минусы

Проектирование, создание и сопровождение хранилища признаков может представлять определенные сложности, однако преимущества зачастую значительно перевешивают недостатки. Рассмотрим основные достоинства наличия хранилища признаков (на рис. 11.12 показаны те области ML, которые больше остальных выигрывают от наличия хранилищ признаков):

- *Повторное использование и совместная работа.* Хранилище признаков способствует их повторному использованию, позволяя командам делиться признаками и применять их в различных проектах и пайплайнах. Это экономит значительное количество времени и усилий, а также способствует сотрудничеству между командами, поскольку те могут использовать наработки друг друга и развивать уже существующие реализации.
- *Упрощение рабочего процесса.* Вместо того чтобы каждый раз начинать все с нуля, команды могут опираться на базу повторно используемых признаков, что ускоряет процесс разработки. Такой упрощенный рабочий процесс способствует ускорению итераций и экспериментов, что дает больше инсайтов и повышает производительность моделей. Хранилище признаков позволяет

командам сосредоточиться на создании моделей, минимизируя рутинные задачи и предоставляя структурированный фреймворк.

- *Согласованность и стандартизация*. С хранилищем признаков появляется единый и стандартизированный подход к генерации признаков. Это обеспечивает единообразие в вычислении признаков, снижая риск несогласованностей или расхождений между различными моделями, пайплайнами или их этапами. Соблюдение заранее определенных стандартов позволяет командам работать более слаженно и повышает общую стабильность системы.
- *Документирование и прозрачность*. Хранилище признаков обеспечивает качественное документирование (или автодокументирование) признаков, включая их источники данных и методы вычисления. Это повышает прозрачность, упрощая для команд поиск и оценку доступных признаков, а также помогает в устранении неполадок и отладке, поскольку документация содержит ценную информацию о процессе генерации признаков.
- *Масштабируемость и сопровождение*. Правильно спроектированная архитектура хранилища признаков обеспечивает масштабируемость, позволяя обрабатывать большие объемы данных и удовлетворять меняющимся требованиям. Упрощается добавление новых признаков и изменение уже существующих, позволяя командам адаптироваться к меняющимся потребностям без серьезных сбоев. Кроме того, централизованный характер хранилищ облегчает сопровождение и мониторинг признаков, повышая общую надежность ML-пайплайнов.



Рис. 11.12. Проблемы ML с различными потребностями в хранилище признаков

Недостатки использования хранилища признаков очевидны:

- Необходимо время для сбора требований от различных команд, проектирования хранилища признаков, которое удовлетворяет всем потребностям, и его реализации (или интеграции стороннего решения, например Tecton, Feast, Feathr или Databricks Feature Store).
- Снижается гибкость в работе с признаками и одновременно увеличивается зависимость ML-команд друг от друга.
- Высокая стоимость при разработке с нуля. Поэтому не стоит заново изобретать велосипед, а лучше использовать уже готовые решения (например, Tecton).
- Хранилище признаков может не подходить для конкретного проекта.

Рассмотрим подробнее последний недостаток. Не все проблемы ML можно эффективно оптимизировать с помощью хранилища признаков. Типичными задачами, где хранилище признаков полезно, являются задачи с табличными данными (структурированными данными) и несколькими источниками данных с различной детализацией и SLA. Большинство проблем чистого глубокого обучения (обычно с одним-двумя источниками неструктурированных данных, например текстом или изображениями) меньше подходят для хранилищ признаков. Тем не менее, с учетом растущей популярности мультимодальных данных, концепция хранилища признаков стала более универсальной. Представим типичный онлайн-магазин: еще несколько лет назад ML-системы основывались на табличных данных, таких как история покупок и кликов, тогда как сейчас стало обычной практикой включать изображения товаров, их описания и отзывы пользователей. Самый простой способ включить такие данные — извлечь эмбеддинги с помощью предварительно обученной нейросети (например, CLIP для изображений или SentenceTransformers для коротких текстов) и использовать их в качестве признаков. Данный подход замыкает круг: такие признаки можно хранить в хранилище признаков по тем же причинам, что и «классические» признаки, это позволяет экономить время обработки и обеспечивает согласованность системы. В качестве бонуса хранение таких признаков в централизованном хранилище открывает дополнительные сценарии их использования. Например, при хранении в векторной базе данных (такой, как Qdrant или Faiss) можно быстро находить похожие объекты и использовать их в последующих моделях.

Лучший способ начать — проанализировать существующие пайплайны извлечения, трансформации и загрузки данных всех команд. Вот какие вопросы следует задать:

- Какие источники данных использует каждая команда? Сколько их?
- Какие пересечения имеются между признаками и сценариями их использования?

- Какие типы признаков они рассчитывают или хотели бы рассчитывать?
- Каким командам необходим отклик в режиме реального времени («онлайн-признаки»)?

Если мы придем к выводу, что многие команды могут извлечь выгоду из хранилища признаков, — это сигнал к тому, чтобы инвестировать ресурсы в его проектирование и построение.

Здесь мы хотим еще раз подчеркнуть: создание собственного кастомного хранилища признаков — крупный и дорогостоящий проект. Это как раз тот случай, когда стоит рассмотреть возможность использования решения с открытым исходным кодом или продукта от стороннего поставщика. Популярные варианты — Feast, Tecton, Databricks Feature Store и AWS Sagemaker Feature Store.

11.4.2. Желательные свойства хранилища признаков

В этом разделе мы рассмотрим полезные паттерны и свойства при проектировании хранилищ признаков, а также подчеркнем важные моменты, на которые следует обратить внимание. Не все из них обязательно проявятся в вашем конкретном хранилище, но наша цель — подробно рассказать о каждом.

Перекос между чтением и записью

Чтение и запись — две ключевые стороны любой системы управления признаками, поэтому на этапе проектирования необходимо понимать объем нагрузки как по этим операциям, так и по объему данных. Какая задержка при чтении критична? Как часто нужно пересчитывать существующие признаки? Случается, что расчет признака в режиме выполнения быстрее или сравним по времени с извлечением его из хранилища.

Запись данных обычно осуществляется батчами. Нам больше нравится обрабатывать весь датасет одновременно, если возможно, хотя это и считается широко распространенным антипаттерном. Более того, обновление признаков за последние несколько дней позволяет устранить временные ошибки или недоступность на стороне хранилища данных, которые могли произойти незадолго до нашего ежедневного обновления признаков. Стоит отметить, что «обычно» не значит «всегда»: признаки могут добавляться или обновляться по разным расписаниям. Например, одни вычисляются длительной задачей ежедневно, а другие достаточно легкие, чтобы передаваться почти в режиме реального времени.

Критически важным аспектом при чтении признаков обычно является задержка. Мы должны быть уверены, что инфраструктура, которую мы строим для хранилища признаков, соответствует нефункциональным требованиям. Иногда мы можем объединять предварительно вычисленные признаки с признаками,

требующими данных о последних событиях, при операциях чтения, как показано на рис. 11.13.

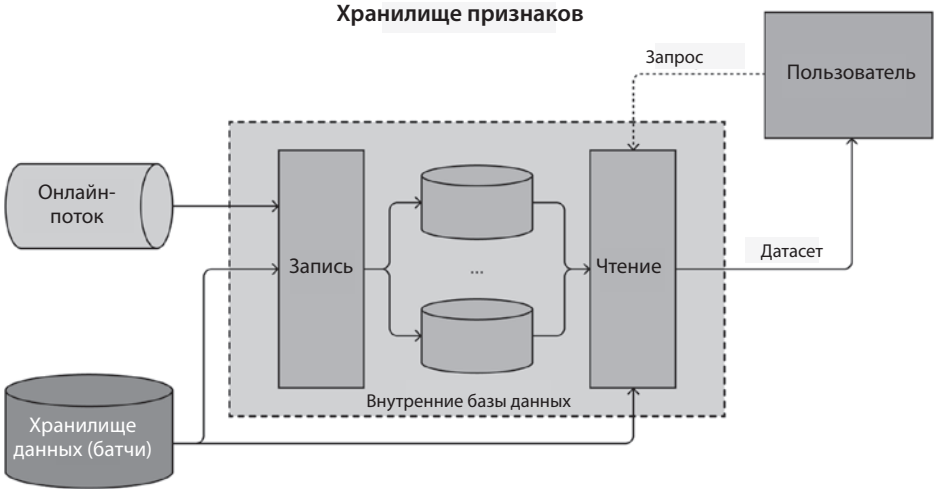


Рис. 11.13. Признаки, рассчитанные в режиме реального времени и в батчах, записываются в хранилище признаков по-разному, но используются одинаково

Предварительный расчет

Принцип DRY (don't repeat yourself — «не повторяйся») в программировании приводит нас к основному правилу любой оптимизации: если можно избежать вычислений — следует это сделать.

В частности, один из наиболее очевидных случаев — это предварительный расчет признаков, который должен выполняться заранее, а не в момент, когда хранилище признаков формирует датасет. Например, подходящее время для обновления признаков — сразу после завершения обработки базы данных за предыдущий день.

Связанная с данным процессом техника оптимизации — разделение вычислений на несколько этапов:

1. Предварительная агрегация исходных данных (например, клики, цены, доход) в суммы по дням/товарам.
2. Агрегация этих сумм в заданные временные окна (например, за 7/14/30/60 дней).

Такой подход позволяет повторно использовать признаки, рассчитанные вчера или несколько месяцев назад (а не выполнять одни и те же расчеты ежедневно), а также объединять вычисления схожих признаков, имеющих частично общую родословную или перекрывающиеся окна агрегации, как показано на рис. 11.14.

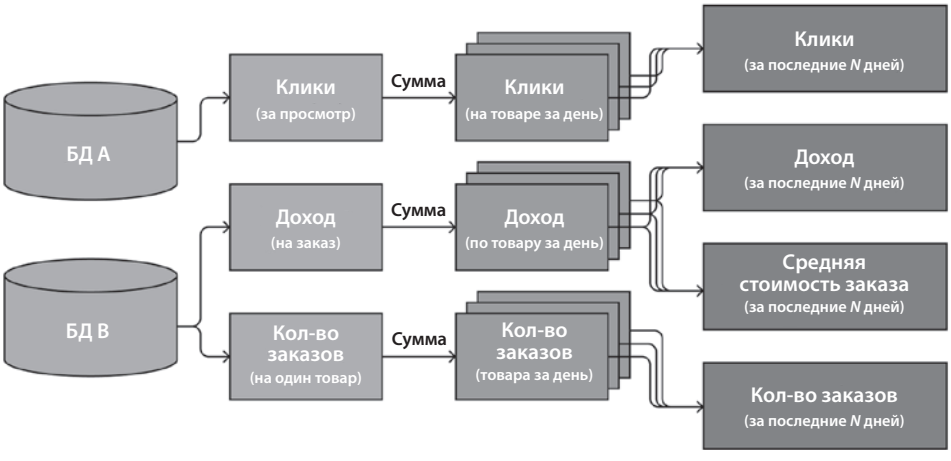


Рис. 11.14. Иерархия агрегации, используемая в хранилище признаков

Версионирование признаков

Стоит взять за правило, что каждое обновление признака должно рассматриваться либо как новый признак (не самое лучшее решение, поскольку ведет к созданию множества схожих признаков после каждой незначительной оптимизации и захламляет код), либо как новая версия уже существующего признака. Почему это важно?

Предположим, инженер реализует новую версию признака, а ваша система воспринимает ее как ту же самую, без различий. Хорошо, если новый расчет даст тот же результат, только, например, быстрее. Но если изменен принцип расчета (пусть даже немного) — или, еще хуже, изменен источник данных для того же признака — это приведет к несогласованности в предварительно рассчитанном признаке. Значения признака до и после обновления могут существенно различаться, и вы не захотите смешивать данные, полученные по старой и по новой методике. Хорошо спроектированное хранилище признаков автоматически перезапишет обновленный признак или, еще лучше, запишет его в новую таблицу с ретроспективным заполнением предыдущих значений на основе доступной истории.

Каждый датасет должен содержать в своей метаинформации не только список признаков, включенных в него для заданного диапазона временных меток, но и версии этих признаков на момент их вычисления.

Это позволяет с легкостью откатиться к старой версии датасета и полностью воспроизвести результаты старой модели, разработанной, например, два месяца назад. Такой подход схож с «замораживанием» версий библиотек в приложении.

Зависимости признаков или иерархия признаков

Не все признаки легко вычисляются из исходных данных, находящихся в хранилище, что может привести к дорогостоящим с вычислительной точки зрения запросам и не позволяет повторно использовать результаты предыдущих вычислений. В связи с этим поговорим о концепции зависимостей признаков, или иерархии признаков, при которой каждый признак зависит от других признаков и/или источников данных.

Паттерн предварительной агрегации, о котором мы говорили ранее, может рассматриваться как прародитель итоговых признаков. На рис. 11.15 мы выделяем признаки первого уровня вместе с их дочерними признаками (признаками второго уровня и т. д.).

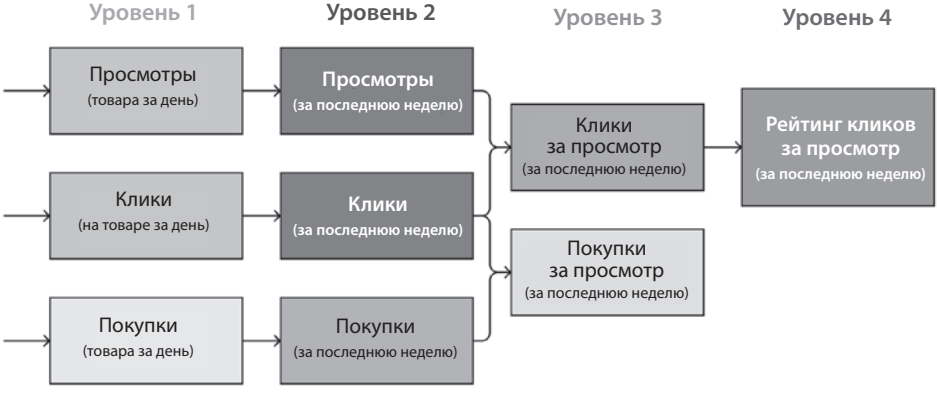


Рис. 11.15. Граф зависимостей признаков

Способ получения каждого признака из его источников данных называется *происхождением* (lineage), которое по сути представляет собой направленный ациклический граф (DAG) (обсуждался в главе 6). Мы отслеживаем происхождение каждого признака, чтобы понимать, в каком порядке выполнять расчеты признаков, нужно ли обновлять дочерние признаки после изменения реализации одного из родительских (что запускает волну обновлений версий), а также, в случае необходимости, выполнять обновление версии признака, устранять повреждения или удалять его.

Отслеживание происхождения также помогает инженерам и аналитикам быстро найти источник каждого признака, упрощая отладку и улучшая понимание причин выбросов или другого неожиданного поведения.

Бандлы признаков

Мы часто применяем одни и те же преобразования и фильтры к тесно связанным столбцам из одних и тех же источников данных (например, цена до скидки, цена

после скидки, цена после применения промокода). Похожие признаки имеют один и тот же ключ и одно и то же происхождение.

Это означает, что нет необходимости работать с ними как с изолированными столбцами и создавать отдельный пайплайн данных для каждого из них. Мы должны реализовывать хранилище признаков таким образом, чтобы оно объединяло вычисления для признаков, полученных из одних и тех же источников. Таким образом, единицей вычислений становится не отдельный признак, а группа:

```
<day, user_id, item_id, f1, f2, f3, f1 / f2, …>
```

Несмотря на объединение вычислительных графов, мы предпочитаем оперировать наборами похожих признаков как единым целым в API (или UI), чтобы добавлять их в датасет одновременно.

11.4.3. Каталог признаков

В UI финальным «секретным ингредиентом» хранилища признаков является каталог признаков. Каталог признаков — это сервис с веб-интерфейсом, в котором ML-инженеры, аналитики и даже ваши нетехнические коллеги могут искать признаки и изучать детали их реализации.

Среди другой информации, которая доступна пользователям, может быть важность признака, распределение значений, категория, владелец, график обновления (ежедневный, почасовой), ключ (user, item, user-item, item-day, category-day), происхождение признака, ML-сервисы, использующие его, и другая метаинформация.

11.5. Дизайн-документ: генерация признаков

Как мы упомянули во введении к этой главе, признаки являются основой способности вашей ML-системы к прогнозированию, и уже только по этой причине они заслуживают отдельного раздела в дизайн-документе. Рассмотрим дизайн-документы для двух наших компаний.

11.5.1. Признаки для Supermegaretail

После построения базовой модели необходимо определить дальнейшие шаги для ее улучшения. Один из основных способов это сделать — использовать признаки, которые помогут модели извлечь полезные закономерности и зависимости из необработанных данных.

ДИЗАЙН-ДОКУМЕНТ: SUPERMEGARETAIL

VIII. ПРИЗНАКИ

Ключевые критерии выбора подходящих признаков:

- *Качество прогнозов*: чем точнее прогнозы, тем лучше.
- *Интерпретируемость и объяснимость*: мы отдаем предпочтение признакам, которые легко описать и объяснить («черный ящик» не является ни прозрачным, ни надежным решением, особенно на начальных этапах проекта).
- *Время вычислений (и вычислительная сложность)*: признаки, которые требуют много времени на вычисление (а также признаки, основанные на нескольких источниках данных или имеющие сложные зависимости), менее предпочтительны, если только они не обеспечивают значительное улучшение качества прогнозов. Это связано с тем, что они замедляют цикл обучения и сокращают количество гипотез, которые мы можем проверить.
- *Риски (и стабильность признаков)*: признаки, требующие внешних/многочисленных источников данных, вспомогательных моделей (или просто плохо спроектированные), а также признаки, основанные на источниках с низким качеством данных, делают пайплайн более уязвимым, чего следует избегать.

Если признак обеспечивает статистически значимое улучшение производительности модели, но нарушает один из других критериев (например, требует два дня на вычисление), лучше не добавлять его в пайплайн.

Основные источники новых признаков:

- добавление внутренних и внешних источников данных (например, мониторинг конкурентов);
- трансформация и комбинация уже существующих признаков.

Ниже приведен список признаков, с которыми мы будем экспериментировать и которые направят наши дальнейшие шаги по улучшению модели после ее начального развертывания:

- Цены конкурентов и их отличие от наших цен (внешние источники).
- Календарь промоакций и скидок.
- Цены (старая цена, цена со скидкой).
- Проникновение (соотношение продаж SKU к продажам категории).
- Атрибуты SKU (бренд, категории различных уровней).
- Коэффициент линейной эластичности.
- Сумма/минимум/максимум/среднее/стандартное отклонение продаж SKU за предыдущие N дней.
- Медиана/квантиль продаж SKU за предыдущие N дней.
- Прогноз погоды (внешние источники).

- Посещаемость магазина (внешние источники).
- Объем продаж магазина.
- Продажи данного SKU год назад.
- Экономические индикаторы (внешние источники).

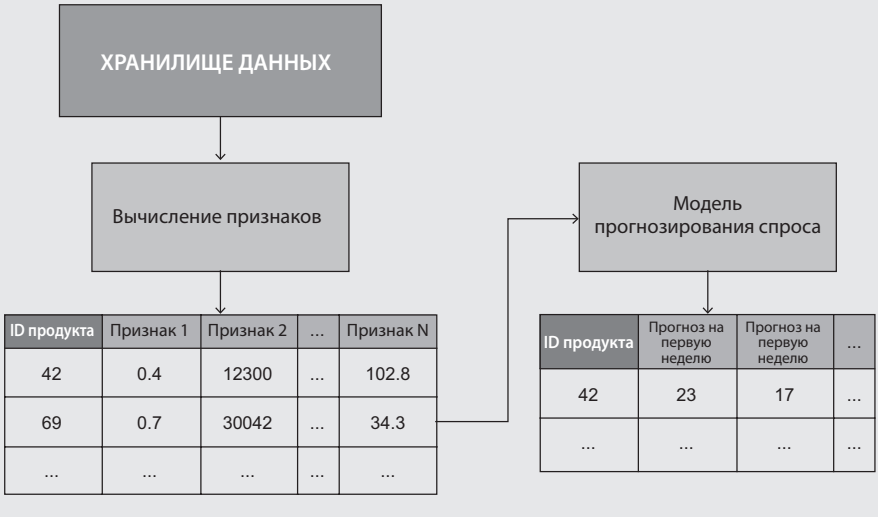
Мы формулируем каждый признак как гипотезу. Например, использование календаря промоакций поможет модели зафиксировать мгновенное увеличение спроса во время маркетинговой активности, что позволит избежать избыточных остатков в этот период.

Мы будем использовать независимые от модели методы (SHAP, LIME, перестановочная важность), а также встроенные методы (коэффициенты линейной модели, количество разделений в градиентном бустинге) для оценки важности признаков. Основная цель — понять вклад каждого признака в вывод модели. Если признак вносит незначительный вклад — мы его исключаем.

Для автоматического отбора признаков на начальных этапах (когда их базовый набор еще не определен) мы используем RFE (рекурсивное исключение признаков).

Мы также включаем тесты признаков в пайплайн обучения до и после обучения модели:

- Проверка диапазонов значений и выбросов (например, 0.0 <= скидка < X).
- Проверка, что корреляция между любой парой признаков меньше X.
- Проверка, что коэффициент признака или количество разбиений > 0.
- Проверка, что время вычисления признака не превышает 6 часов.



Для упрощения вычисления и доступа к признакам мы можем повторно использовать централизованное хранилище признаков, которое собирает данные из различных источников в хранилище данных и, после ряда трансформаций и агрегирования, объединяет их в единую витрину (SKU, магазин, день). Признаки пересчитываются ежедневно, что упрощает эксперименты с новыми признаками, а также отслеживание их версий, зависимостей и другой метаинформации.

11.5.2. Признаки для PhotoStock Inc.

Потенциальный набор признаков для PhotoStock Inc. будет существенно отличаться от набора для Supermegaretail.

ДИЗАЙН-ДОКУМЕНТ: PHOTOSTOCK INC.

VIII. ПРИЗНАКИ

Как уже было указано в разделе с описанием базовой модели, мы планируем начать с системы, основанной исключительно на релевантности контента, измеряя расстояния между запросом и документом (изображением и его описанием):

степень_релевантности = расстояние(запрос, изображение) +
+ расстояние(запрос, описание)

И хотя это может навести на мысль, что генерации признаков не требуется, это не совсем так, по крайней мере по двум причинам:

- Описание изображения и метаданные должны быть каким-то образом преобразованы, чтобы использоваться в качестве входных данных модели. Следовательно, для этого необходимо предложить надежный и расширяемый способ.
- Возможно, на последующих этапах понадобится ввести дополнительные источники сигнала, представляющие пользователей и документы. Примеры таких признаков для документов включают коэффициент кликабельности (CTR), среднюю частоту покупок и время, проведенное на странице документа. Примеры пользовательских признаков могут включать: агрегированные данные их истории кликов, настройки профиля или признаки, рассчитанные с помощью подходов, подобных коллаборативной фильтрации. Это типичный сценарий для поисковых систем. Наконец, мы можем использовать признаки, относящиеся к авторам фотографий, например их средний рейтинг или количество проданных работ, чтобы неявно продвигать ключевых авторов. Однако это требует значительных трудозатрат, поэтому пока что мы не хотим реализовывать эту идею. Тем не менее мы

стремимся спроектировать систему таким образом, чтобы ее можно было легко расширить в будущем.

Учитывая, что мы ожидаем появления новых типов признаков, система должна быть способна эффективно использовать сигналы из различных источников и выдавать единый показатель релевантности, например:

степень_релевантности = функция_расстояния(запрос_на_изображение_расстояние, запрос_на_описание_расстояние, признаки_пользователя, признаки_документа, другие_признаки_которые_мы_добавим_позже)

В базовой модели мы предполагали, что функция расстояния — это простая сумма расстояний. Однако это не единственный вариант. Простым решением является взвешенная сумма, что, по сути, означает обучение небольшой линейной модели поверх рассчитанных расстояний.

1. Кодирование метаданных фотографии

Учитывая, что мы рассматриваем мультимодальный энкодер в стиле CLIP, можно использовать тот факт, что он принимает любой текст в качестве входа. Следовательно, метаданные можно закодировать в текстовой форме и подать на вход энкодеру. Мы предлагаем собрать все значимые атрибуты фотографии и объединить их в одну строку, например, по следующему шаблону:

«Описание: {описание}, теги: {теги}, местоположение: {местоположение}»

Создание такого промта — универсальный подход. Мы уверены, что описание и теги являются важными составляющими метаданных, но, вероятно, можно добавить и другие. Например, здесь предлагается использовать координаты, если они содержатся в EXIF-данных фотографии. Возможно, есть и другие варианты, например создание признаков на основе даты съемки (для отражения времени года), модели камеры и т. д. Также может потребоваться фильтрация тегов, например, путем сокращения их списка до наиболее информативных.

Как обсуждалось ранее, при создании признаков из метаданных существует определенная гибкость. Однако чем больше признаков мы используем, тем выше общая сложность. Даже если добавление фиктивного признака не влияет на производительность модели по метрикам, это может увеличить как время обучения, так и время инференса, поскольку модели на основе трансформеров имеют квадратичную сложность по длине входа. Поэтому следует осторожно подходить к количеству используемых признаков и применять методы отбора признаков для контроля сложности. Учитывая характер признаков, стандартные методы фильтрации применить не удастся, поэтому с учетом их небольшого количества мы предлагаем использовать точный, пусть и медленный метод, например последовательный жадный отбор признаков.

2. Важность признаков

Необходимо иметь доступ к информации о важности признаков по крайней мере в двух сценариях:

- Нам нужно общее представление о модели, чтобы определить приоритеты для дальнейшей работы. Например, если мы видим, что модель в значительной степени опирается на признаки, полученные из метаданных, возможно, стоит инвестировать больше в пайплайн генерации признаков на основе метаданных. В то же время мы, вероятно, хотим, чтобы все компоненты вносили вклад в итоговую оценку, что позволит снизить вероятность переобучения и минимизировать риски эксплуатации слабых мест модели (например, чрезмерной оптимизации тегов у загружаемых фотографий).
- Будут возникать жалобы на качество ранжирования, и нам нужно будет понять причину. Поэтому необходимо иметь возможность объяснить, почему конкретный документ получил именно такую оценку по конкретному запросу. Это может выявить новые возможности для улучшения в будущем, а также обнаружить системные проблемы.

К счастью, для реализации решения, описанного в документе, нам не требуется новый компонент вроде хранилища признаков. Однако такая потребность может возникнуть, если (или когда) мы начнем использовать данные о поведении пользователей в качестве источника признаков. В таком случае будет необходимо заранее вычислять и где-то хранить признаки — тут нам и понадобится хранилище признаков.

Итоги

- Анализ важности признаков помогает достичь интерпретируемости и объяснимости модели, выявляя признаки, которые вносят наибольший вклад в ее прогнозы, и сигнализируя о готовности модели к вводу в эксплуатацию.
- Тщательный отбор признаков позволит усовершенствовать возможности модели в области прогнозирования и в итоге получить точное решение, которое будет легко интерпретируемым, объяснимым, а также обеспечит более быстрое обучение и инференс.
- Не забывайте о генерации признаков даже при работе с мультимодальными данными, такими как изображения или тексты.
- Хранилище признаков позволяет командам централизованно рассчитывать, хранить, агрегировать, тестировать, документировать и отслеживать признаки для своих ML-пайплайнов.
- Наличие хранилища признаков будет особенно полезно при работе с несколькими источниками табличных данных. С другой стороны, их использование не подходит для проблем, решаемых исключительно методами глубокого обучения.

12 Измерение и представление результатов

В этой главе

- ✓ Измерение результатов
- ✓ Польза A/B-тестирования
- ✓ Представление полученных результатов

В предыдущих главах мы рассмотрели основные строительные блоки, формирующие основу для нашей системы машинного обучения, начиная со сбора данных и выбора модели, метрик, функций потерь и стратегии разделения данных на выборки и заканчивая всесторонним анализом ошибок. После интеграции всех этих элементов в пайплайн пришло время вернуться к изначальной цели системы и задуматься о правильной подаче достигнутых результатов.

Отчетность включает в себя оценку производительности ML-системы с учетом ее конечной цели и передачу результатов членам команды и стейкхолдерам. В главе 5 мы ввели два типа метрик: онлайн- и офлайн-метрики, соответствующие двум типам оценивания — онлайн- и офлайн-тестированию. В то время как офлайн-тестирование относительно простое, онлайн-тестирование предполагает проведение экспериментов в реальных условиях. Наиболее эффективный подход представляет собой серию A/B-тестов — важнейшую процедуру при разработке эффективной и корректно работающей модели, которую мы подробно рассматриваем в разделе 12.2. Этот процесс позволяет собирать метрики, которые либо напрямую соответствуют бизнес-целям, либо тесно с ними связаны.

Измерение результатов при офлайн-тестировании (иногда называемом *бэктестингом*) — это попытка оценить эффект, который мы ожидаем на стадии

онлайн-тестирования, будь то напрямую или с использованием прокси-метрик. Однако онлайн-тестирование зачастую представляет собой более сложную задачу. Например, системы рекомендаций сильно зависят от обратной связи, так что обучающие данные зависят от предыдущих прогнозов. Более того, мы не можем точно знать, что произошло бы, если бы мы показали пользователю A элемент X или Y вместо элемента Z, который мы реально показали и зафиксировали это в истории на временной отметке T.

Наконец, по завершении эксперимента мы готовы отчитаться о влиянии модели на интересующие нас бизнес-метрики. Что именно мы должны сообщить? Как представить результаты? Какой вывод мы должны сделать по поводу дальнейших шагов в развитии системы? В этой главе мы ответим на все вопросы, которые могут возникнуть в процессе подготовки и представления результатов.

12.1. Измерение результатов

Перед внедрением ML-системы необходимо четко понимать, какой цели мы хотим достичь, — именно поэтому дизайн-документ должен начинаться с описания цели. Аналогично, перед запуском эксперимента, направленного на проверку влияния изменения на нашу систему (в данной главе основное внимание уделяется A/B-тестированию), мы разрабатываем его и формулируем гипотезу, описывающую ожидаемое влияние на метрики. Здесь нам понадобится офлайн-тестирование, так как мы используем офлайн-оценки в качестве показателя для онлайн-оценки. Его цель — быстро определить, лучше ли новое (модифицированное) решение по сравнению с текущим, и если да, попытаться количественно оценить, насколько оно лучше.

Как уже упоминалось ранее, все метрики иерархически связаны между собой. Существуют прокси-метрики для других прокси-метрик, которые, в свою очередь, связаны с интересующими нас метриками (см. главу 5). Следовательно, офлайн-оценку можно провести несколькими способами.

12.1.1. Эффективность модели

Первый уровень оценки обычно включает базовую проверку метрик ML, которые были представлены в главе 7. Предполагается, что наряду с офлайн-метриками, такими как среднеквадратичная ошибка, средняя абсолютная процентная ошибка или NDCG (нормализованный дисконтированный кумулятивный выигрыш), мы также улучшаем бизнес-метрики (например, среднюю прибыль с пользователя или коэффициент кликабельности — CTR).

Оценка прогнозов — это самый быстрый способ проверить качество модели и протестировать различные ее версии, но, как правило, он менее связан с бизнес-метриками (взаимосвязь между качеством прогнозов и онлайн-метриками зачастую далека от идеала).

Процедура офлайн-оценки надежна и полезна для оценки качества, однако редко существует прямая связь между офлайн- и онлайн-метриками. Понимание того, как улучшение офлайн-метрик A, B и C влияет на онлайн-метрики X, Y и Z, является проблемой регрессии. Чтобы преодолеть разрыв между офлайн- и онлайн-метриками и сделать офлайн-тестирование более надежным, можно собирать историю онлайн-тестов и соответствующих онлайн-метрик, а также связанных с ними офлайн-метрик, чтобы вычислить корреляцию между результатами офлайн- и онлайн-оценки.

12.1.2. Переход к бизнес-метрикам

В некоторых сценариях можно перейти от прогнозов модели к реальным бизнес-метрикам. Для иллюстрации рассмотрим пример прогнозирования продаж авиабилетов. Обычно основными метриками эффективности модели, используемыми в задачах прогнозирования, являются средневзвешенная процентная ошибка (WAPE) и смещение:

$$\text{WAPE} = \frac{\sum(|\,\text{Прогноз} - \text{Реальность}\,|)}{\sum \text{Реальность}};$$

$$\text{Смещение} = \frac{\sum(\text{Прогноз} - \text{Реальность})}{\sum \text{Реальность}}.$$

Предположим, у нас есть две модели, каждая из которых имеет смещение ±10 % при прогнозировании продаж билетов на конкретный рейс в определенное время. Мы прогнозируем, что за определенный период времени будет продано 110 билетов, тогда как на самом деле было продано 100 билетов. Также допустим, что у нас есть фактические цены билетов на каждый день.

Наша цель — избежать овербукинга (упущенная прибыль из-за предложения слишком большого количества билетов со скидками) и минимизировать количество непроданных мест (упущенная выручка). Для простоты примера предположим, что количество проданных билетов отражает реальный спрос пассажиров и что мы корректируем цены в начале каждого дня на основе прогноза.

Фактические продажи билетов за 4 дня: [120, 90, 110, 80]. Прогнозируемые продажи билетов:

- *Модель A* — [90, 90, 90, 90].
- *Модель B* — [110, 110, 110, 110].

Смещения моделей:

- *Модель A* — $\frac{-30+0-20+10}{400} = \frac{-40}{400} = -10\,\%$.

- *Модель B* — $\frac{-10+20+0+30}{400}=\frac{40}{400}=10\,\%$.

Значения WAPE моделей:

- *Модель A* — $\frac{30+0+20+10}{400}=\frac{60}{400}=15\,\%$.
- *Модель B* — $\frac{10+20+0+30}{400}=\frac{60}{400}=15\,\%$.

Обе модели имеют одинаковые значения WAPE и смещения, однако у модели A смещение отрицательное и составляет 10 % (тенденция к занижению прогноза), тогда как у модели B — положительное и также составляет 10 % (тенденция к завышению прогноза).

Цены на билеты на каждый из четырех дней составляют [$200, $220, $230, $210].

Общая выручка (мы выбираем минимальное значение из количества проданных и прогнозируемых билетов на каждый день) составляет:

- Модель A — 90 * $200 + 90 * $220 + 90 * $230 + 80 * $210 = $75 300.
- Модель B — 110 * $200 + 90 * $220 + 110 * $230 + 80 * $210 = $83 900.

Как видите, модель B приносит на $8600 больше выручки всего за 4 дня для одного рейса (при этом затраты на сам рейс остаются неизменными). При умножении этой разницы на сотни рейсов и дней получается значительное увеличение дохода.

Конечно, мы упростили реальную ситуацию, в которой модель формирует прогнозы и влияет на принятие решений (например, сколько билетов выпустить в продажу, как динамически регулировать цены). Здесь весь переход реализован путем умножения на цену и, при необходимости, вычитания себестоимости рейса.

Тем не менее данный пример иллюстрирует, как можно перейти от метрик качества модели, которые могут не отражать практического эффекта от прогнозов, к бизнес-показателям, таким как выручка. Все бизнес-метрики можно представить команде до проведения A/B-тестирования.

12.1.3. Среда моделирования

Когда цена ошибки высока, разумно инвестировать в более сложную, вычислительно затратную, но и более надежную процедуру офлайн-тестирования, — например, смоделировать онлайн-среду для рассматриваемой проблемы и запустить алгоритм в этой среде.

Хорошим примером подобной среды является прогнозирование временных рядов, которое относительно легко моделировать благодаря доступности меток, вспомогательной информации (например, исторических цен или оборота

товара на складе) и четкости в определении результата. Но это, как правило, не относится к онлайн-системам рекомендаций — новостным, рекламным или товарным, где мы работаем с частично размеченными данными и знаем только информацию об объектах контента (просмотры, клики, переходы), которые были показаны конкретному пользователю, тогда как по тем объектам, которые мы не показывали, обратной связи нет. Всегда существует вероятность, что при иной последовательности показа результаты были бы совершенно другими.

Рассмотрим, как можно создать такую среду в случае системы рекомендаций.

Иногда онлайн-системы рекомендаций создаются с помощью подходов «многоруких бандитов» и «контекстных бандитов» (этот подкласс систем рекомендаций обычно называют *«ставки в реальном времени»*, но, по сути, цель остается та же: выбрать наилучший вариант из множества, чтобы максимизировать определенную целевую метрику). В данном случае у нас есть агент — наш алгоритм, который «взаимодействует» со средой. Например, он получает информацию о контексте пользователя (его недавняя активность) и выбирает, какой рекламный баннер показать, чтобы максимизировать CTR, коэффициент конверсии (CR) или эффективную цену за тысячу показов (eCPM). Это типичный пример задачи обучения с подкреплением с выраженным эффектом обратной связи, так как ранее показанные объявления изменяют поведение пользователя в будущем и дают дополнительную информацию о его предпочтениях.

Так называемый алгоритм воспроизведения (replay algorithm) — это простой (хотя и не самый интуитивный) способ построения среды моделирования для рекомендаций в режиме реального времени, подробно описанный в статье «Unbiased Offline Evaluation of Contextual-bandit-based News Article Recommendation Algorithms» Лихон Ли (Lihong Li) и др. (https://arxiv.org/abs/1003.5956). Ниже приводится краткое изложение этой практичной статьи.

1. Предположим, у нас есть исторический датасет, содержащий информацию о контексте пользователя (его признаки), рекомендованных рекламных объявлениях (предположим, что можно рекомендовать по одному элементу за раз) и обратную связь пользователя (как бинарную, так и непрерывную). Также по каждой строке данных нам известны все доступные элементы, которые мы могли бы рекомендовать пользователю в данный момент (эти элементы выступают в роли «рук» в алгоритме контекстных бандитов).
2. Кроме того, у нас есть второй датасет — виртуальный, который изначально пуст.
3. Мы разбиваем наш исторический датасет на батчи (например, по часовым интервалам на каждую итерацию).
4. Для каждой строки в батче прогнозируем, какое рекламное объявление наш алгоритм мог бы порекомендовать на основе контекста пользователя и доступных элементов. Алгоритм обучается на виртуальном датасете, который изначально пуст. Следовательно, мы начинаем со случайной «разведки».

5. Строки, в которых рекомендация, сгенерированная моделью, совпадает с фактически показанным историческим рекламным объявлением, сохраняются в виртуальный датасет (вместе с соответствующей обратной связью).
6. Мы повторно обучаем модель на обновленном виртуальном датасете.
7. Повторяем шаги 4–6 до тех пор, пока не обработаем весь исторический датасет.

Существует также оптимизированная версия шагов 4 и 5: вместо одного элемента (как в реальной среде) мы рекомендуем три или пять элементов для каждой строки, и если один из рекомендованных элементов совпадает с фактическим, то добавляем эту строку в виртуальный датасет. Это позволяет модели обучаться более эффективно и снижает степень стохастичности.

Пример на рис. 12.1–12.5 разбит на несколько частей, иллюстрирующих работу несмещенного оценщика: рис. 12.1 демонстрирует реальные события, а рис. 12.2–12.5 показывают смоделированные события на основе исторических данных.

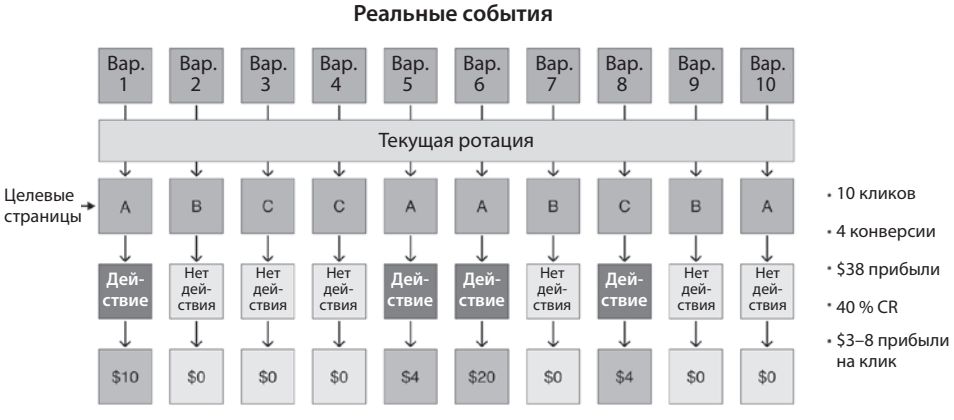


Рис. 12.1. Несмещенный оценщик (офлайн-валидация): реальные события

В смоделированных событиях мы проходим по реальной истории событий и рассматриваем только те случаи, когда выход новой модели (выбранные целевые страницы) совпадает с выходом старой модели. На рис. 12.2 смоделированное поведение пользователей включает пять переходов с баннеров на целевые страницы, две успешные покупки и $6 прогнозируемой прибыли на клик.

На рис. 12.3 пользователи просматривают всего четыре товара, что приводит к наименьшей возможной прибыли ($10) и прибыли на клик ($2.5).

Рисунок 12.4 демонстрирует другой пример смоделированного пользовательского поведения с хорошим коэффициентом конверсии (50 %), но одной из самых низких прибылей на клик (всего $3) из-за низкой стоимости покупок.



Рис. 12.2. Несмещенный оценщик (офлайн-валидация): моделирование событий, пример 1

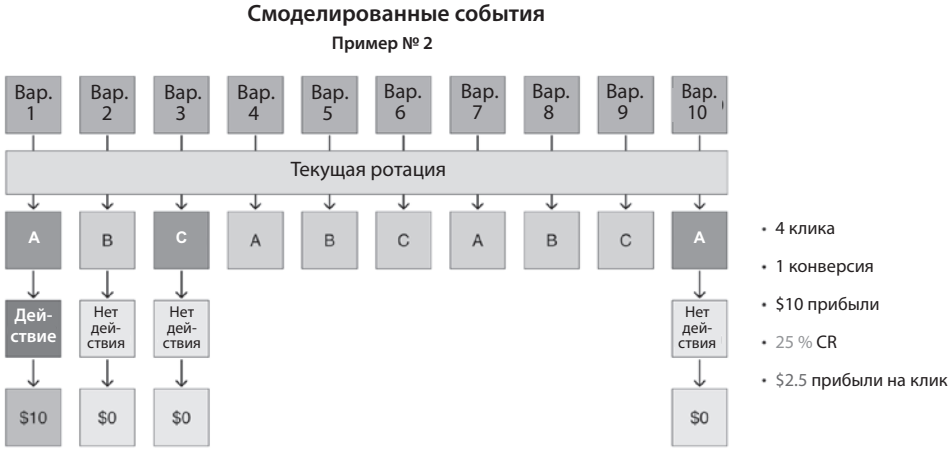


Рис. 12.3. Несмещенный оценщик (офлайн-валидация): моделирование событий, пример 2

Наконец, смоделированная среда на рис. 12.5 показывает наилучший возможный коэффициент конверсии (CR) — 60 % и прибыль на клик — $7, в то время как общая прибыль всего на $3 меньше, чем в реальности (рис. 12.1).

В итоге мы получаем виртуальный датасет, сформированный в результате взаимодействия нашей модели с моделированным потоком данных. На его основе можно рассчитать все метрики:

- CTR = количество кликов / количество показов (общее количество строк)
- CR = количество конверсий / количество кликов
- eCPM = (доход / количество кликов) * 1000

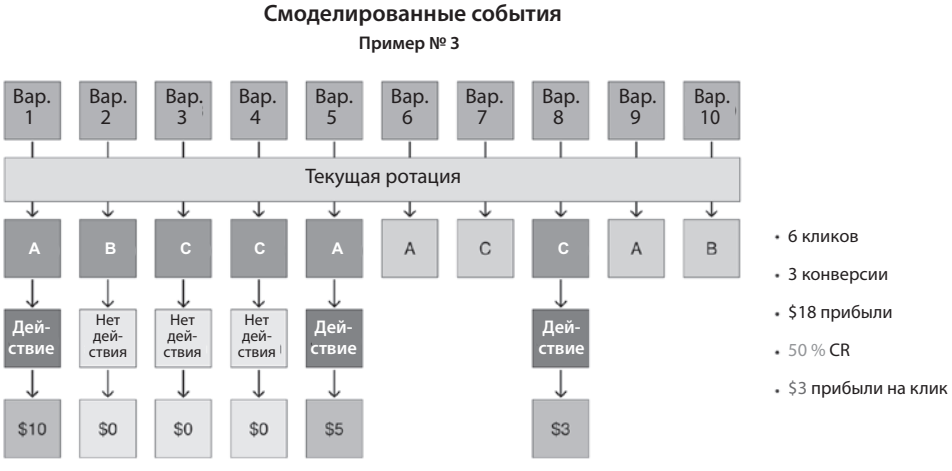


Рис. 12.4. Несмещенный оценщик (офлайн-валидация): моделирование событий, пример 3

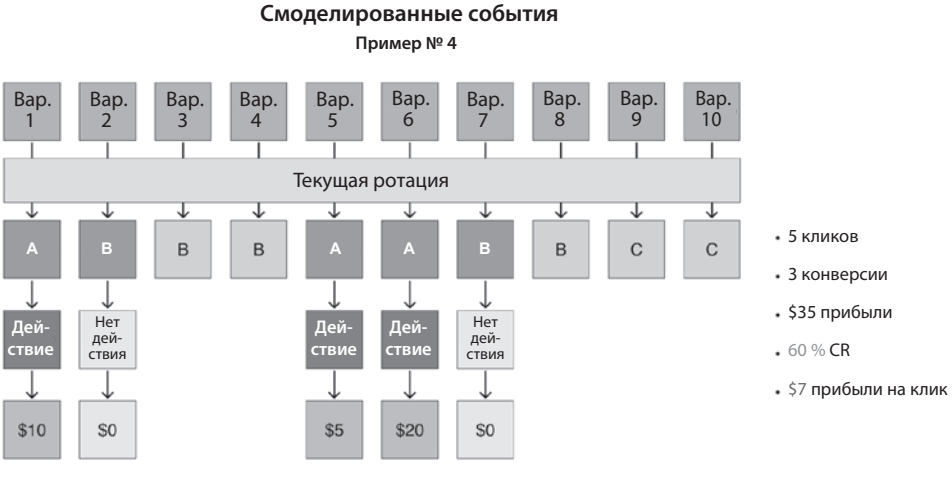


Рис. 12.5. Несмещенный оценщик (офлайн-валидация): моделирование событий, пример 4

Данный метод неидеален, но позволяет оценить модели с точки зрения продуктовых метрик, понять связь между онлайн-метриками и точностью прогнозов, а затем представить эту информацию более широкой аудитории.

12.1.4. Оценка человеком

Иногда последним вариантом, заслуживающим упоминания при оценке результатов до ввода системы в эксплуатацию, является валидация экспертами, или оценка человеком. Если моделирование среды требует значительных вычислительных ресурсов и выполняется медленно, то этот вариант может оказаться еще менее эффективным из-за ограниченной пропускной способности, большей задержки, субъективности и высоких затрат. Тем не менее в ситуациях, где участие человека возможно, это, как правило, наиболее точный и надежный способ тестирования модели, за исключением прямого онлайн-тестирования на реальных данных.

Например, после разработки новой версии поискового движка можно либо попросить группу экспертов оценить релевантность результатов поиска по шкале от 1 до 5 для ряда эталонных запросов, либо предложить им сравнить результаты старой и новой версий и выбрать более релевантный вариант. Второй (сравнительный) подход обычно дает более точную оценку, поскольку устраняет субъективность численных оценок. В случае генеративных LLM наиболее распространенными способами оценки качества генераций являются оценка человеком и гибридные подходы (вспомогательная модель-критик, обученная на данных, размеченных человеком).

И хотя человеческая оценка имеет низкую пропускную способность и высокую стоимость, что может затруднить ее частое применение, ее способность давать высокоточные и надежные оценки (по сравнению с быстрыми, но ограниченными автоматическими методами) остается неоспоримым преимуществом.

12.2. A/B-тестирование

Широко используемое в различных сферах, включая маркетинг, рекламу, дизайн продуктов и многое другое, A/B-тестирование считается «золотым стандартом» причинно-следственного анализа, который позволяет принимать решения на основе данных, а не полагаться исключительно на интуицию. У вас может возникнуть закономерный вопрос: мы обсуждали способы измерения и представления результатов, а теперь внезапно перешли к причинно-следственным выводам — почему? В некотором смысле измерение результатов является частью причинно-следственного анализа, поскольку мы хотим быть уверены, что наши результаты были вызваны именно тем фактором, на который мы повлияли (в нашем случае — ML-системой). Конечно, мы можем развернуть систему в продакшене и измерить все интересующие нас метрики напрямую, но на практике редко так поступаем по следующим причинам:

- Это рискованно. Если что-то работает некорректно, развертывание для всех пользователей может привести к серьезным сбоям. Поэтому систему сначала стоит внедрить только для небольшой доли пользователей.
- Даже если нас не волнует предыдущий пункт (что маловероятно, но все же), может возникнуть ситуация, когда внешние изменения, находящиеся вне зоны

нашего контроля, повлияли на итоговую производительность, но мы ошибочно решили, что причиной стали наши действия, и сделали неверный вывод.

- Эти два пункта вместе подводят нас к необходимости собрать контрольную (статус-кво) и тестовую (наше изменение) группы, что позволяет учитывать оба фактора: с одной стороны, управлять долей охваченной аудитории, а с другой — сравнивать влияние изменений на тестовую группу с результатами контрольной.

Во время проведения A/B-экспериментов мы разделяем объекты (в большинстве случаев пользователей) на две группы:

- контрольная группа, использующая текущую версию системы;
- тестовая группа, использующая новую версию.

Это позволяет изолировать влияние изменений на ключевые метрики, одновременно контролируя внешние факторы, которые могут повлиять на результаты (рис. 12.6).

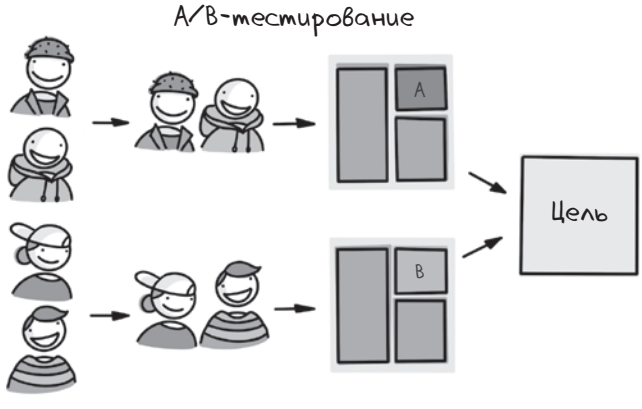


Рис. 12.6. Разделение при A/B-тестировании

12.2.1. Планирование эксперимента

Каждый эксперимент начинается с формулировки гипотезы:

> *Если мы сделаем [действие A], это поможет нам достичь/решить [цели/проблему P], что отразится в виде ожидаемого прироста [X %] в [метриках M] на основе [исследования/оценки R].*

Рассмотрим подробнее переменные, используемые в гипотезе:

- *Действие* — это внедрение нового решения для его сравнения с существующим. Например, цель может состоять в сокращении числа ситуаций нехватки какого-то товара на складе и снижении расхождения между прогнозируемыми и фактическими продажами в компании Supermegaretail.

- *Метрики* дают нам возможность количественно оценить прогресс в достижении цели. Например, метрика «потеря веса» в фитнес-приложении демонстрирует прогресс пользователя в похудении. Подробнее об этом см. в главе 5.
- *Ожидаемый прирост* (uplift) может быть получен из предыдущего опыта, доступных бенчмарков, эмпирических правил или даже желаемого результата (что случается довольно часто!). Для повышения точности обычно привлекаются дополнительные данные из офлайн-тестов.

Перед запуском A/B-теста необходимо определить как минимум три гиперпараметра:

- *Минимально детектируемый эффект* (MDE, minimum detectable effect) — это наименьшая разница между двумя группами, которую можно достоверно зафиксировать при данном дизайне исследования и размере семпла. MDE должен быть меньше или равно ожидаемому приросту, чтобы последний можно было уловить.
- *Ошибка первого рода* (ложноположительный результат) — это вероятность ошибочного вывода о наличии эффекта, когда его на самом деле нет. Стандартный порог данной ошибки составляет 5 % (обозначается как $\alpha = 0.05$).
- *Ошибка второго рода* (ложноотрицательный результат) — это вероятность не зафиксировать эффект, когда он на самом деле есть. Стандартный порог составляет 20 % (обозначается как $\beta = 0.2$).

Вместе с изменениями в данных это влияет на необходимый размер выборки (и, следовательно, на длительность эксперимента, учитывая также сезонные колебания). Все эти факторы объединяются в следующем уравнении:

$$n = \frac{2s^2(Z_\alpha + Z_\beta)^2}{MDE^2}.$$

Здесь n — требуемый размер выборки для каждой группы, Z_α — Z-значение, соответствующее желаемому уровню значимости (для ошибки первого рода), Z_β — Z-значение, соответствующее желаемой мощности теста (для ошибки второго рода), s^2 — оценочная дисперсия метрики, а MDE — минимально детектируемый эффект. Как уже упоминалось ранее, наиболее часто используемой единицей наблюдения (sample unit) является *пользователь*, но это может быть и *сессия*, *транзакция*, *магазин* и т. п.

После формулировки гипотезы и ожиданий мы выбираем *стратегию разделения* (split strategy), которая определяется в основном природой самой системы и тем, как она повлияет на текущие процессы.

Далее мы определяем *статистические критерии* и проводим тесты в режиме моделирования — как A/A (где сравниваются две идентичные версии системы для оценки уровня ошибок первого рода), так и A/B (где моделируется

ожидаемый эффект с добавлением шума заданной величины) — в качестве проверки корректности на исторических данных. Это помогает подтвердить, соблюдаются ли заранее заданные уровни ошибок первого и второго рода. Полное описание данного процесса выходит за рамки нашей книги, но всем желающим ознакомиться с деталями мы рекомендуем книгу «Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing» Рона Кохави (Ron Kohavi) и др.[1] (https://mng.bz/86Qz).

Наконец, у нас есть все необходимое для запуска A/B-теста. Помните, что план эксперимента фиксируется заранее и не должен изменяться по ходу его выполнения. Далее мы кратко рассмотрим основные этапы A/B-тестирования.

12.2.2. Стратегия разделения

Возникает вопрос: как именно разделить данные? Обычный ответ — «по пользователям». Например, у нас есть контрольная группа пользователей, взаимодействующих с существующим поисковым движком, и экспериментальная группа, использующая его новую версию. Однако ситуация усложняется, если разделение по пользователям невозможно (например, отсутствует постоянный ID пользователя) или же полностью нерелевантно для нашего сервиса.

Рассмотрим гипотетические системы ценообразования в различных областях:

- В *офлайн-торговле* физически невозможно показать разные ценники разным покупателям в одном магазине. Поэтому более подходящей единицей разделения будет магазин, а не отдельные покупатели.
- В *онлайн-торговле* технически возможно показывать разные цены разным пользователям, но это может привести к юридическим проблемам, так как во многих странах существуют законы, запрещающие дискриминацию пользователей в электронной торговле.
- В *райдшеринге* пользователи могут быть разделены, поскольку заказы осуществляются через приложение, но нужно учитывать возможные негативные пользовательские сценарии. Если провести разделение по пользователям и два человека решат прокатиться вместе на скутерах, то один может увидеть цену $5 в час, а другой — $6 в час. Это может вызвать недоумение, негативную реакцию и соответствующую обратную связь.
- В случае с динамическим ценообразованием для *кредитов и займов* предлагаемые ставки, как правило, строго персонализированы, например, как в ипотечном кредитовании. Поэтому различные «цены» не вызовут подозрений, даже если пользователи находятся в одной комнате. В таком случае относительно безопасно проводить разделение по пользователям.

[1] Кохави Р., Тан Д., Сюй Я. «Доверительное A/B-тестирование: Практическое руководство по контролируемым экспериментам».

- Идеальным примером для тестирования на уровне пользователей являются *рекламные сети*, поскольку они предоставляют большое количество разнообразных, относительно недорогих точек данных, а индивидуальная цена за клик является обычным делом. Такая конфигурация чаще всего позволяет использовать традиционные подходы к разделению по пользователям.

Если разделение данных по отдельным пользователям невозможно, в качестве атомарных единиц следует использовать более высокоуровневые объекты. Для офлайн-торговли это могут быть целые магазины, для скутеров поездки можно разделить по парковочным зонам, районам, городам или даже регионам. Однако такая стратегия может изменить распределение данных и привести к неравномерному объему выборок или нехватке репрезентативности, что необходимо учитывать при выборе подходящего статистического теста.

Если используется нетипичный ключ для разделения, необходимо стремиться к максимальному сходству между группами (бакетами). Например, если вы вынуждены проводить географическое разделение, выбранные регионы должны быть сопоставимы и иметь схожие экономические показатели (они должны соответствовать друг другу).

Наконец, существует и более продвинутая стратегия разделения, называемая *switchback-тестированием*, которая предполагает деление данных на бакеты «регион — время» и случайное непрерывное переключение между различными моделями (рис. 12.7). Это обеспечивает попадание каждого региона как в контрольную, так и в тестовую группу примерно на одинаковое количество времени. Более подробную информацию см. в статье «Design and Analysis of Switchback Experiments» Явора Божинова (Iavor Bojinov) и др. (https://arxiv.org/abs/2009.00148).



Рис. 12.7. Switchback-разделение: распределение по бакетам «регион — время»

12.2.3. Выбор метрик

При планировании A/B-теста выбор метрик — это крайне важный этап, который может определить успех или провал эксперимента. Для онлайн-тестирования предпочтительны надежные чувствительные интерпретируемые метрики с минимальной задержкой обратной связи.

Каждый эксперимент, как правило, включает три типа метрик:

- *Ключевые метрики* — это основные показатели, на улучшение которых направлен эксперимент. Такие метрики напрямую влияют на достижение бизнес-целей и используются для определения окончательного результата эксперимента. Например, ключевой метрикой для сайта онлайн-магазина может быть коэффициент конверсии (CR), средний чек (AOV, average order value) или общий объем продаж/выручка.
- *Контрольные метрики* — это показатели, которые, как ожидается, остаются неизменными в ходе эксперимента и служат для проверки его корректности. Например, если контрольной метрикой является количество посетителей сайта (или время загрузки страницы) и при этом наблюдается значительное снижение (или увеличение) данной метрики, это может свидетельствовать о проблемах в планировании эксперимента или о внешних факторах, повлиявших на результаты.
- *Вспомогательные метрики* — это показатели, которые предоставляют дополнительную информацию об эксперименте, но не используются для определения окончательного результата. Такие метрики позволяют лучше понять итоги эксперимента и могут быть использованы для выявления потенциальных проблем или, наоборот, перспектив. Например, вспомогательной метрикой для сайта онлайн-магазина может быть количество просмотров товаров или время, проведенное на сайте.

Все три группы метрик играют важную роль в планировании и проведении A/B-тестирования. Ключевые метрики определяют успех эксперимента, контрольные обеспечивают достоверность результатов, а вспомогательные предоставляют дополнительную информацию для их интерпретации и планирования последующих экспериментов.

12.2.4. Статистические критерии

Третьим необходимым элементом любого A/B-эксперимента является статистический тест, который принимает окончательное решение о наличии значимого эффекта.

Вкратце, статистические критерии используются для количественной оценки вероятности получения определенных результатов при заданных допущениях. Например, вероятность получения t-статистики, равной или превышающей 3,

как правило, составляет менее 0.01 (если между группами нет различий и выполнены условия применения *t*-теста).

Наиболее распространенным статистическим тестом, применяемым в A/B-тестировании, является упомянутый выше *t*-тест. У него существует множество модификаций. Например, *t*-тест Уэлча относительно прост в интерпретации и похож на *t*-тест Стьюдента, но может применяться даже в случае неравенства дисперсий двух групп. Статистика для *t*-теста Уэлча рассчитывается следующим образом:

$$t = \frac{\mu_B - \mu_A}{\sqrt{\frac{s_A^2}{n_A} + \frac{s_B^2}{n_B}}}.$$

Нулевая гипотеза (базовое предположение, проверяемое тестом) для *t*-теста Уэлча заключается в том, что между средними значениями двух групп нет различий, а наблюдаемая разница между группами возникла случайно и обусловлена шумом. Альтернативная гипотеза состоит в том, что наблюдаемая разница не вызвана шумом, и различие статистически значимо.

Под статистической значимостью понимается, что вероятность получения конкретного или еще более экстремального значения — при условии, что нулевая гипотеза верна (такая вероятность называется *p-значением*), — меньше *уровня значимости* α (фактически то же самое, что и ошибка первого рода, — как правило, $\alpha = 0.05$). *P*-значение вместе с уровнем значимости представляет собой способ стандартизации результатов любого статистического теста и их отображения относительно единой точки отсчета, а не определения критических значений для каждого конкретного теста.

Существует множество различных статистических тестов, и выбор конкретного теста зависит от контекста эксперимента и природы данных. Мы не будем приводить подробное руководство по выбору наиболее подходящего теста, данную информацию вы тоже можете найти в упоминаемой ранее книге «Trustworthy Online Controlled Experiments: A Practical Guide to A/B Testing» Рона Кохави и др.

T-тест широко применяется благодаря своей надежности, способности обрабатывать разнообразные ситуации, простоте реализации и легкости интерпретации результатов. Тем не менее выбор конкретного теста всегда должен осуществляться с учетом типа данных, особенностей эксперимента и допущений, необходимых для каждого теста.

12.2.5. Моделирование экспериментов

Моделирование позволит проверить, удалось ли нам правильно спланировать эксперимент. Для этого мы воссоздаем весь процесс несколько раз с разными выборками и временными периодами.

Смоделированный A/A-тест включает случайную выборку двух групп, между которыми нет различий, с последующим применением выбранной тестовой статистики. Действия повторяются многократно (скажем, 1000 или 10 000 раз), и ожидается, что *p*-значение будет следовать равномерному распределению (если все проходит правильно). Например, в 5 % случаев моделирования тест отвергнет нулевую гипотезу при $\alpha = 0.05$. Точная доля случаев, когда тест фиксирует различия между двумя группами, является *смоделированной оценкой уровня значимости ошибки первого рода*.

Смоделированный A/B-тест устроен аналогичным образом, но теперь мы добавляем определенный прирост (обычно соответствующий целевому минимально детектируемому эффекту — MDE) ко второй группе B (предполагается, что группы A и B имеют тот же размер, как и в реальном A/B-тесте). Мы снова применяем тестовую статистику и проводим тест 1000–10 000 раз. Затем анализируем распределение полученных *p*-значений и считаем, сколько раз была отвергнута нулевая гипотеза («*p*-значение превысило порог»). Доля случаев, когда выбранный тест отверг нулевую гипотезу, является оценкой чувствительности ($1 - \beta$), то есть вероятностью того, что тест обнаружит различие, если оно действительно существует. Если вычесть чувствительность из 1, получится *уровень значимости ошибки второго рода* (вероятность не обнаружить различие между A и B, когда оно существует). Если рассчитанные уровни ошибок первого и второго рода соответствуют заданным значениям, значит, статистический тест выбран правильно, а размер выборки оценен корректно (рис. 12.8).

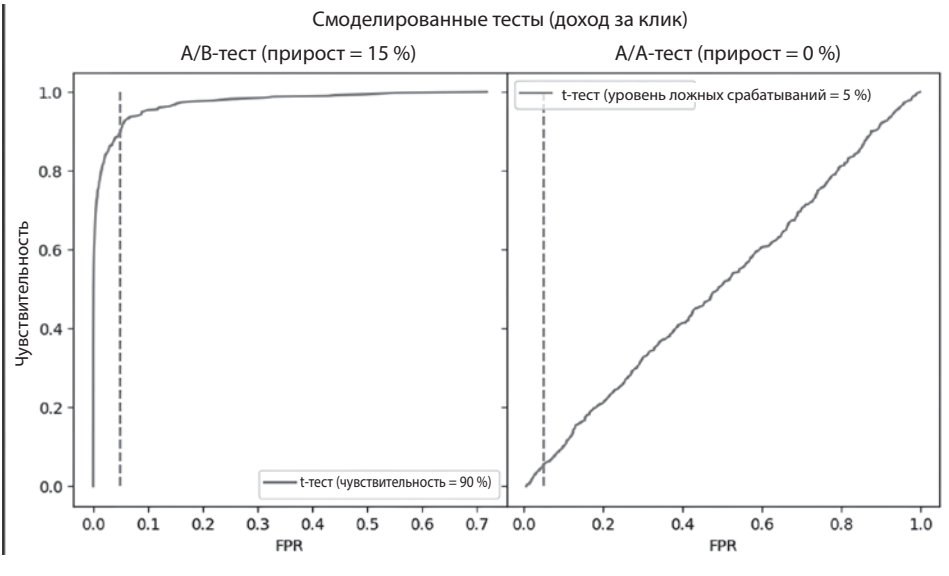


Рис. 12.8. Моделирование служит финальной проверкой того, что все настроено корректно и готово к запуску A/B-теста

12.2.6. Когда проведение A/B-тестирования невозможно

В некоторых ситуациях проведение классических A/B-тестов невозможно или нежелательно, будь то какие-либо юридические ограничения (некоторые отрасли, такие как здравоохранение или финансы, строго регулируются и запрещают проведение экспериментов на клиентах), логистические ограничения (может быть сложно случайным образом разделить клиентскую базу на две группы для тестирования) или иные причины.

Существует несколько способов решения такой проблемы, однако поскольку это тема узкоспециализированная и выходит за рамки данной книги, мы не будем рассматривать ее подробно. Тем не менее для вашего удобства приводим список таких методов, которые вы можете изучить самостоятельно:

- каузальный эффект (casual effect);
- метод разности разностей (difference-in-difference);
- синтетический контроль;
- анализ прерванных временных рядов;
- метод регрессионного разрыва;
- каузальный вывод (causal inference).

12.3. Представление результатов

Мониторинг текущего эксперимента имеет важное значение для обеспечения его корректного проведения и получения достоверных результатов. Если во время эксперимента что-то идет не так, крайне важно оперативно выявить и устранить проблему, чтобы избежать накопления негативного эффекта. Иногда можно исключить конкретных пользователей, элементы или сегменты из эксперимента и продолжить его проведение. Однако такой подход не рекомендуется, если он не был предусмотрен в исходном плане. В иных случаях предпочтительнее полностью остановить эксперимент и начать анализ факторов, приведших к его неудаче.

Байка от Арсения

Тем, кто работает в максимально гибкой среде стартапов, может показаться, что все требования к отчетности касаются только крупных корпораций. Но это не так: даже в небольших компаниях необходимо уделять достаточно внимания корректной отчетности и управлению экспериментами. Я усвоил этот урок на собственном опыте: когда-то я работал в относительно небольшой компании, где было всего три ML-инженера, и мне поручили улучшить базовую производительность модели. После нескольких месяцев исследований и экспериментов мне удалось достичь хороших результатов, которые были высоко оценены моим руководителем и техническим директором компании. Я как раз готовил модель к внедрению в эксплуатацию, когда

от технического директора пришло сообщение: «Привет, завтра заседание совета директоров, я хочу рассказать о твоей работе. Пожалуйста, пришли подробную информацию по X с детализацией до уровня Y».

Я попытался составить отчет в соответствии с запросом, но внезапно, после тщательного анализа, оказалось, что новая модель показывает худшие результаты, чем старая! Ужасное открытие, но я не собирался обманывать директора, придумав несуществующие улучшения. Поскольку это был мой первый крупный проект в компании, выглядело бы это крайне подозрительно. Мы втроем — технический директор, мой менеджер и я — начали разбираться в проблеме.

Только ближе к полуночи я нашел корень зла — во время проведения дополнительных экспериментов я случайно перезаписал результаты лучшей модели результатами неудачных экспериментов и использовал эти данные для отчета к заседанию совета. Поскольку я был единственным человеком, работающим над задачей, я не уделил должного внимания структуре хранения данных, а отсутствие системы отчетности и управления экспериментами (вместо этого — полурандомные фрагменты данных на компьютере разработчика) привели к такой ситуации.

12.3.1. Контрольные и вспомогательные метрики

В разделе 12.2.1 мы упомянули контрольные и вспомогательные метрики, которые необходимо отслеживать во время проведения теста. Они могут послужить ранним индикатором ситуации, когда что-то пошло не так, — еще до того, как ключевые метрики начнут заметно снижаться. Например, важно отслеживать обратную связь от пользователей: если вы замечаете резкое падение вовлеченности или всплеск оттока клиентов, это явный сигнал сбоя. В подобной ситуации можно сделать вывод, что экспериментальная группа негативно реагирует на внедренные изменения. Такая информация может стать основанием для досрочного прекращения теста.

Кроме того, при проведении A/B-тестирования необходимо учитывать объективность и предвзятость в различных сегментах пользователей, что может повлиять как на целевые, так и на вспомогательные метрики. Такой анализ схож с тем, что мы подробно рассматривали в главе 9. Дополнительную информацию можно найти в статье «Fairness through Experimentation: Inequality in A/B Testing as an Approach to Responsible Design» Гийома Сент-Жака (Guillaume Saint-Jacques) и др. (https://arxiv.org/pdf/2002.05819.pdf).

12.3.2. Мониторинг прироста

Наиболее ценной метрикой, которую следует отслеживать во время A/B-эксперимента, является прирост (uplift), то есть относительное различие ключевого показателя между группой A и группой B. Чем дольше продолжается эксперимент, тем выше чувствительность теста и тем меньший эффект (положительный или отрицательный) он способен зафиксировать.

На рис. 12.9 показана воронка, представляющая диапазон эффектов, которые невозможно обнаружить с требуемой статистической значимостью. Если прирост выходит за пределы этой воронки, эффект считается статистически значимым. Обратите внимание: без специального проектирования вы не можете просматривать результаты теста столько раз, сколько захотите, пока не получите желаемый результат. Для дальнейшего изучения этой темы рекомендуем публикации «Peeking Problem — The Fatal Mistake in A/B Testing and Experimentation» Олега Я (Oleg Ya) (https://gopractice.io/data/peeking-problem) и «Sequential A/B Testing, Further Reading Choosing Sequential Testing Framework — Comparisons and Discussions» Мортена Шульцберга (Mårten Schultzberg) и др. (https://mng.bz/NBon).

Такая воронка может быть вычислена на основе MDE (минимально детектируемого эффекта). Здесь необходимо решить то же уравнение, что и ранее, но теперь для различных временных периодов. Однако принять решение можно только после того, как пройдет заранее рассчитанный период времени. Возникает вопрос: в чем тогда смысл ее использования? Чтобы иметь критерий экстренной остановки эксперимента в случае, если результат выходит за пределы воронки и оказывается отрицательным, — наименее желаемый, ожидаемый и вероятный исход. Для любого фреймворка последовательного тестирования, упомянутого ранее (например, mSPRT), решение можно принять, как только результат выходит за пределы воронки или когда заканчивается время эксперимента.

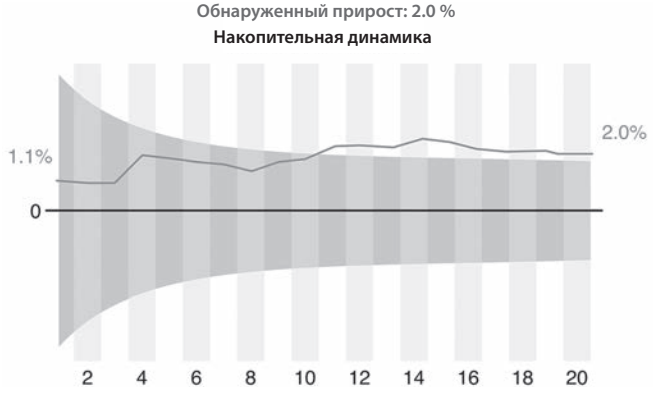


Рис. 12.9. Пример накопительной динамики прироста во времени с определенными доверительными интервалами

12.3.3. Когда завершать эксперимент

Может возникнуть соблазн преждевременно остановить или, наоборот, продлить A/B-тест на основе промежуточных результатов. Однако это может привести к ошибочным выводам из-за изменения статистической мощности и риска ложноположительных или ложноотрицательных результатов, если в план эксперимента не было заложено последовательное тестирование. Так когда же завершать эксперимент, если мы не проводим последовательное тестирование?

В большинстве случаев эксперимент должен проводиться в соответствии с заранее определенным планом, если только не происходит какого-то катастрофического снижения ключевых метрик (существенно отрицательный эффект), которое в случае продолжения эксперимента может привести к значительным финансовым потерям.

С другой стороны, если наблюдается положительный эффект по ключевым метрикам, лучше всего продолжать следовать первоначальному плану и дать эксперименту завершиться в полном объеме. Это позволит убедиться, что положительный эффект сохраняется во времени, а не является следствием каких-то краткосрочных изменений.

Что же делать с крайними случаями, когда одни метрики показывают положительные изменения, другие — отрицательные, а некоторые находятся в пределах MDE? Важно еще на этапе планирования эксперимента определить, как именно вы будете интерпретировать такие смешанные результаты и какие приоритеты задаются между различными ключевыми метриками.

Если вы не можете дождаться финальных результатов, стоит использовать методы последовательного тестирования, позволяющие многократно проверять статистическую значимость на разных этапах эксперимента. Однако такой подход имеет свои допущения и особенности, которые важно учитывать перед его применением.

Решение о завершении или продолжении эксперимента — задача сложная и требует внимательного анализа метрик, потенциальных исходов и их последствий. Не забудьте задокументировать все свои решения в протоколе эксперимента, включая критерии возможной досрочной остановки.

12.3.4. Что указывать в отчете

После завершения эксперимента наступает этап анализа. Здесь необходимо рассчитать требуемые метрики, повторно проверить мониторинг и предоставить доверительные интервалы для каждого измерения.

Не каждый эксперимент оказывается успешным. Согласно нашему опыту, хороший показатель — когда 20 % A/B-тестов демонстрируют статистически значимое различие (напомним, что при обычном уровне ложноположительных результатов в 5 % случаев допустимый порог составляет 15 %). И не забывайте: если тест показал статистическую значимость, это еще не означает положительного результата или существенной величины эффекта.

Предположим, мы пришли к выводу, что A/B-тест прошел успешно и эффект статистически значим. В таком случае у нас есть два варианта:

- *Сообщить о точечном эффекте*. «Эффект значим и составляет X», где X — это рассчитанная разница между значениями метрики в экспериментальной и контрольной группах.

- *Оценить доверительный интервал для эффекта*. Если пессимистичная оценка эффекта равна нижней границе доверительного интервала, то реалистичная — точечной разнице, а оптимистичная — верхней границе доверительного интервала.

Мы представили основное содержание отчета ниже (разумеется, метрики в отчете должны быть понятны целевой аудитории, желательно выражены в терминах полученной или сэкономленной прибыли, увеличения времени сессий пользователей и т. п.). Кроме того, полезно более глубоко проанализировать изменения в метриках, изучить, как они повлияли на разные сегменты пользователей или товаров, и наметить дальнейшие шаги (например, как изменятся агрегированные метрики при масштабировании на 100 % аудитории). В табл. 12.1 приведены примеры полей, которые могут быть включены в отчетную таблицу.

Таблица 12.1. Пример отчетной таблицы

Метрика	Группа A	Группа B	MDE	Прирост	p-значение	Вывод
CR	75.2 %	79.8 %	5 %	6.12 %	0.0472	+4.2–6.9 % (значимо)
AOV	$232.2	$242.8	11 %	4.57 %	0.3704	Значимого эффекта нет
…	…	…	…	…	…	…

После того как прирост зафиксирован, можно провести еще один эксперимент на большем объеме данных или полный переход на новую систему либо полномасштабное внедрение, паузу и обратный A/B-тест. Серия успешных A/B-экспериментов обеспечит вас убедительными, основанными на данных аргументами, которые позволят принять взвешенное решение о переходе к следующему этапу.

12.3.5. Отчетный документ

Подготовка отчетного документа важна как для прозрачного представления результатов A/B-эксперимента, так и для повышения качества будущих экспериментов. Такой документ должен быть составлен вовремя, например после завершения эксперимента, и включать в себя ключевые выводы, инсайты, обнаруженные проблемы и рекомендации. Наличие данного документа в команде обеспечивает общее понимание проекта и способствует непрерывному обучению и совершенствованию системы.

Если эксперимент оказался успешным, отчетный документ должен содержать предложения по аналогичным экспериментам, но уже для других продуктов. В случае неудачи необходимо проанализировать, что следует изменить в будущих тестах, каких ошибок важно избежать, а также разработать новые контрольные метрики, которые позволят на более раннем этапе выявить потенциальные неудачи.

12.4. Дизайн-документ: измерение и отчетность

Так как на этапе планирования невозможно заранее определить, как именно будет происходить отчетность, данный раздел может быть пропущен. Тем не менее мы все равно включили его в дизайн-документ в демонстрационных целях.

12.4.1. Измерение и отчетность для Supermegaretail

Раздел дизайн-документа, посвященный измерению и отчетности, для компании Supermegaretail имеет свои особенности. Одна из них заключается в том, что поскольку мы прогнозируем будущее, оценить качество системы можно только тогда, когда это будущее наступит. Мы хотим понять, какую прибыль удастся получить, но узнать это можно только постфактум. При этом единственный способ оценить будущую эффективность — использование определенных метрик, которые напрямую с прибылью не связаны.

ДИЗАЙН-ДОКУМЕНТ: SUPERMEGARETAIL

IX. ИЗМЕРЕНИЕ И ОТЧЕТНОСТЬ

1. Оценка результатов

В качестве первого шага по улучшению качества прогнозирования в уже развернутой модели прогнозирования продаж мы планируем провести эксперимент по объединению существующих моделей (по одной на каждую категорию) в единую модель. Причина такого решения заключается в том, что так мы сможем закодировать информацию о группе товаров без потери специфичности, получив при этом больше данных. Это улучшит обобщающую способность модели. Даже при отсутствии повышения качества прогнозов поддерживать одну модель гораздо проще, чем несколько.

В качестве офлайн-метрик мы рассмотрели различные квантили, процентные ошибки и смещения. Вместо того чтобы оценивать только общее качество, мы провели анализ ошибок и метрик по отдельным категориям. Офлайн-тесты показали следующие промежуточные результаты:

- Общее качество прогнозов устойчиво повышается по всем метрикам и в большинстве валидационных фолдов при переходе на объединенную модель.
- Категории с небольшим числом товаров показали рост по офлайн-метрикам по сравнению с базовой (мульти) моделью. Объема данных в этих категориях ранее не хватало для обучения значимым паттернам. Крупные категории не продемонстрировали большого прироста при переходе на объединенную модель. Данный результат воспроизводится в разные сезоны и по основным географическим регионам.

Ранее A/B-тесты позволяли оценить, какого прироста можно ожидать в каждой крупной категории на основе офлайн-метрик. Если мы уменьшаем

метрику M для товара P на X %, это приводит к снижению упущенной прибыли (из-за отсутствия товара в наличии) на Y % и уменьшает потери из-за избыточных запасов на Z %. По нашим оценкам, общий прирост выручки в пилотной группе ожидается в пределах от 0.3 до 0.7 %.

2. A/B-тесты

- *Гипотеза*. Гипотеза эксперимента заключается в том, что, согласно улучшению офлайн-метрик, мы ожидаем увеличения выручки как минимум на 0.3 %.
- *Ключевые метрики*. Наиболее подходящая прокси-метрика для выручки, которую мы можем использовать в A/B-эксперименте, — это средний чек. Он идеально коррелирует с выручкой (при условии, что количество чеков не изменилось). Единицей наблюдения для статистического теста будет один чек.
- *Стратегия разделения*. Мы проводим разделение по распределительным центрам и через них получаем два набора магазинов, так как каждый центр обслуживает группу магазинов. Из этих наборов мы выбираем репрезентативные подмножества и используем их как группы A и B.
- *Дополнительные метрики* — контрольные метрики для эксперимента:
  - *Количество чеков в день* — отслеживает, произошло ли значительное падение объема продаж.
  - *Частота обновления модели* — регулярно ли модель собирает новые данные.
  - *Офлайн-метрики модели* — квантильная метрика, смещение, WAPE.
- *Вспомогательные метрики*:
  - *Дневная выручка*.
  - *Дневная прибыль*.
- *Статистический критерий*. Для выявления различий между выборками будет использоваться *t*-критерий Уэлча.
- *Ошибки первого и второго рода*. Уровень значимости установлен на 5 %, вероятность ошибки второго рода — 10 %.
- *Длительность эксперимента*. Согласно нашим расчетам, двух недель будет достаточно для проверки результатов. Однако с учетом того, что цикл пополнения запасов в распределительном центре составляет одну неделю, этот период будет продлен до одного полного месяца.

3. Отчетность по результатам

Необходимо предоставить отчет, включающий следующие разделы:

- *Результаты*: представлены в виде 95 %-ных доверительных интервалов для ключевых и вспомогательных метрик.

- *Графическое представление*: значения конкретной метрики на конкретную дату для всех метрик как в контрольной, так и в тестовой группе, для удобства восприятия.
- *Абсолютные значения*: например, количество магазинов в каждой группе, общее число чеков и общая выручка.
- *Методология*: например, каким образом группы были отобраны как репрезентативные, какие виды моделирования проводились для оценки ошибок первого и второго рода и т. д. (подробности приведены в приложении).
- *Рекомендации/дальнейшие шаги*: что делать дальше на основании полученных результатов.

12.4.2. Измерение и отчетность для компании PhotoStock Inc.

Теперь рассмотрим, как будет проходить данный этап в компании PhotoStock Inc.

ДИЗАЙН-ДОКУМЕНТ: PHOTOSTOCK INC.

IX. ИЗМЕРЕНИЕ И ОТЧЕТНОСТЬ

1. Оценка результатов

Ниже приведен базовый перечень наших офлайн-метрик:

- *NDCG@10 (разметка человеком)*: используем оценки релевантности, выставленные вручную, для каждого изображения на странице результатов поиска (SERP).
- *NDCG@30 (неявная обратная связь)*: используем неявную обратную связь в качестве оценки релевантности:
  - *3* — изображение куплено.
  - *2* — изображение добавлено в «Избранное».
  - *1* — по изображению кликнули.
  - *0* — просмотр без взаимодействия.
- *MRR (разметка человеком)*: средняя позиция первого изображения, отмеченного как релевантное.
- *MRR (клик)*: средняя позиция первого изображения, по которому был произведен клик в SERP.
- *MRR (покупка)*: средняя позиция первого изображения, которое было куплено в SERP.

Мы ретроспективно применяем данные метрики к текущему алгоритму поиска без использования ML, собирая SERP, показанные по каждому запросу. Далее мы измеряем онлайн-метрики (CTR и CR) за разные временные периоды. После этого рассчитываем коэффициент ранговой корреляции Спирмена и выбираем офлайн-метрику, которая наиболее сильно коррелирует с каждой онлайн-метри-

кой. Выбранная офлайн-метрика затем используется в простой регрессионной модели для получения первоначальной оценки MDE в A/B-эксперименте.

Учитывая, что это первый A/B-тест, у нас пока нет сравнительных данных для более точной оценки MDE. Однако для будущих экспериментов мы предлагаем системный подход, при котором мы будем собирать и сопоставлять различия в офлайн- и онлайн-метриках между вариантами A и B в каждом A/B-тесте. Конечная цель — установить приблизительную зависимость: при каждом улучшении офлайн-метрики C на X % наблюдается прирост онлайн-метрики D на Y %.

Дальнейшее улучшение оценки наших онлайн-метрик может включать моделирование поведения пользователя. Это более сложная задача, поэтому мы считаем, что текущий подход уже предоставляет прочную основу для проведения тестирования.

Помимо вышеупомянутых автоматизированных методов офлайн-тестирования, мы также можем включить обратную связь от экспертов. Мы предлагаем две задачи для оценки:

- Задано поисковое слово Q и 20 лучших фотографий, предложенных новой поисковой системой. Эксперты оценивают общую релевантность SERP по шкале от 1 до 5.
- Задано поисковое слово Q и по 20 лучших фотографий от двух различных поисковых систем (одна — текущая версия, другая — новая). Эксперты определяют, какой из SERP является более релевантным.

Вторая задача позволяет провести сравнительную оценку, которая, как правило, дает более надежные результаты, снижая влияние индивидуальных смещений в оценивании.

2. Планирование A/B-эксперимента

Допустим, мы получили приблизительные оценки минимально ожидаемого эффекта по офлайн-метрикам: +5 % для CTR и +20 % для CR. *Гипотеза* нашего эксперимента следующая:

- Если мы заменим текущую поисковую систему на основанное на ML решение, оно будет предоставлять более релевантные результаты поиска, что отразится на увеличении CTR как минимум на 5 % и увеличении CR как минимум на 20 %.

Соответственно, нашими *ключевыми метриками* являются CTR и CR.

Контрольные метрики, которые не должны изменяться в ходе эксперимента:

- *Просмотры на каждую вариацию*: если они различаются, это может свидетельствовать либо о некорректной стратегии разделения, либо о сильном воздействии на пользовательский опыт в тестовой группе.
- *Задержка ответа*: не должна сильно увеличиваться или, по крайней мере, должна соответствовать SLA.

- *Количество запросов в секунду*: если значение низкое, возможно, поисковая система стала недоступной.
- *Количество запросов в день и доля сессий, начатых с поисковой строки*: обе метрики отражают вовлеченность пользователей. Ожидается, что данные значения не будут снижаться.
- *Количество обращений в службу поддержки*: если наблюдается резкий рост, вероятно, что-то работает неправильно.

Вспомогательные метрики, предоставляющие дополнительную информацию, но не участвующие напрямую в принятии решений:

- *Среднее время, проведенное на SERP*: если увеличилось, это может свидетельствовать о снижении качества поиска: пользователю требуется больше времени, запросов и пролистываний страниц, чтобы найти нужное изображение.
- *Общее количество кликов и покупок по каждой вариации*: помогут понять, что именно приводит к росту конверсии (числитель, знаменатель или оба компонента с разными темпами изменения).
- *Общая прибыль и средняя прибыль с покупки*.
- *Средняя позиция изображения, на котором кликнули, в SERP и среднее количество запросов на сессию*: сложные метрики, снижение которых, при определенных предположениях, может указывать на упрощение навигации: пользователю требуется меньше действий и времени для нахождения нужного результата.
- *Процент сессий, завершившихся покупкой*: менее чувствительная версия метрики CR.
- *Стратегия разделения*: новая поисковая система не вносит дополнительных зависимостей в пользовательский опыт, поэтому ничто не мешает разделять пользователей стандартным образом по их ID.
- *Статистический критерий* — учитывая, что мы стремимся повысить CTR и CR (обе являются пропорциональными метриками), мы выбираем Z-критерий для пропорций.

Мы будем использовать традиционные уровни ошибок первого и второго рода: $\alpha = 0.01$ и $\beta = 0.05$:

- *Размер выборки* — проанализируем данные, имеющиеся до начала теста:
  - Сайт PhotoStock Inc. посещают 100 000 пользователей ежедневно.
  - В среднем 1.5 поискового запроса на визит.
  - В среднем 20 просмотренных изображений на каждый запрос.
  - В среднем на 10 % просмотренных изображений кликают.
  - В среднем 1 % изображений, на которых кликали, покупаются.

Следовательно, у нас $100\,000 \times 1.5 \times 20 = 3\,000\,000$ просмотров в день:

- Базовое значение CTR составляет 10 % (число кликов на просмотр).

- Базовое значение CR составляет 10 % × 1 % = 0.1 % (число покупок на просмотр).

Таким образом, ежедневно фиксируется 300 000 кликов и 3000 покупок.

Как обсуждалось ранее, чтобы снизить возможные риски, мы планируем поэтапный запуск. Поэтому в первом A/B-тесте мы возьмем только 10 % пользователей, разделив их поровну: 5 % на группу A и 5 % на группу B. Это означает, что у нас будет 3 000 000 × 2.5 % = 150 000 просмотров в день для каждой группы.

Сколько времени должен длиться эксперимент? Ниже приведена формула оценки необходимого размера выборки на основе требуемого MDE и уровней ошибок:

$$n = \frac{\left(Z_1 - \frac{\alpha}{2} + Z_1\right)^2 [p_A(1-p_A) + p_B(1-p_B)]}{(p_A - p_B)^2}.$$

Здесь p_A и p_B — старые и новые пропорции (в нашем случае CTR или CR), а Z_x — критические значения Z-статистики для заданного уровня значимости.

У нас есть 10 % и 10.5 % в качестве старого и нового значений CTR:

$$n = \frac{(2.57 + 1.65)^2 [0.1(1-0.1) + 0.105(1-0.105)]}{0.0052}.$$

Получаем 131 095 необходимых просмотров, тогда как у нас есть 150 000 просмотров в день на каждую группу. Это означает, что для выявления значимого увеличения CTR (на 5 % и более) требуется всего 1 день сбора данных.

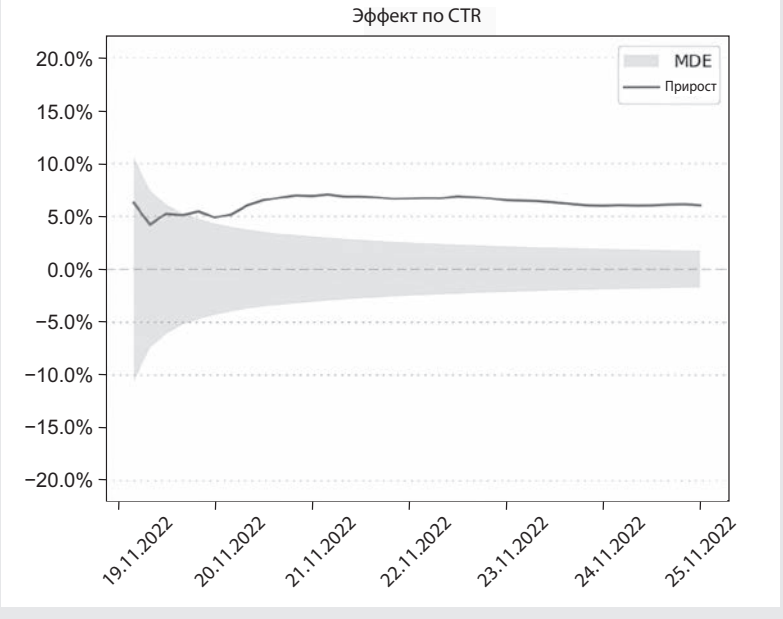
Что касается коэффициента конверсии, то у нас есть 0.1 % для старой версии и 0.12 % для новой версии. Повторное применение формулы приводит к необходимости в 978 693 просмотрах. Таким образом, на сбор данных потребуется 7 дней.

Подводя итог: эксперимент будет длиться неделю, чтобы охватить обе метрики. Такой промежуток времени также включает в себя полный цикл недельной сезонности, если таковая имеется (например, для PhotoStock Inc. может наблюдаться больше пользователей-профессионалов в будние дни и больше любителей — в выходные).

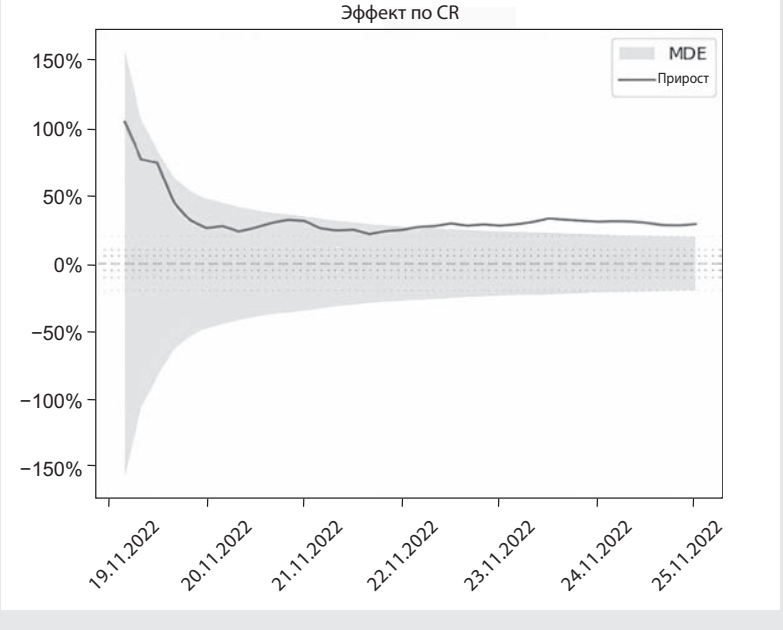
Чтобы убедиться, что все сделано правильно, мы проведем многократные смоделированные A/A- и A/B-тесты, генерируя выборки из биномиального распределения с заданным объемом и требуемыми значениями CTR и CR, чтобы проверить, что фактические уровни ошибок ниже заданных.

3. Шаблон отчета

В ходе эксперимента мы в первую очередь отслеживали динамику MDE. Как и ожидалось, эффект по CTR оказался статистически значим уже на второй день. В то же время статистически значимый эффект по CR был зафиксирован только на четвертый день эксперимента.



Эффект по CTR



Эффект по CR

К концу эксперимента были зафиксированы следующие значения метрик:

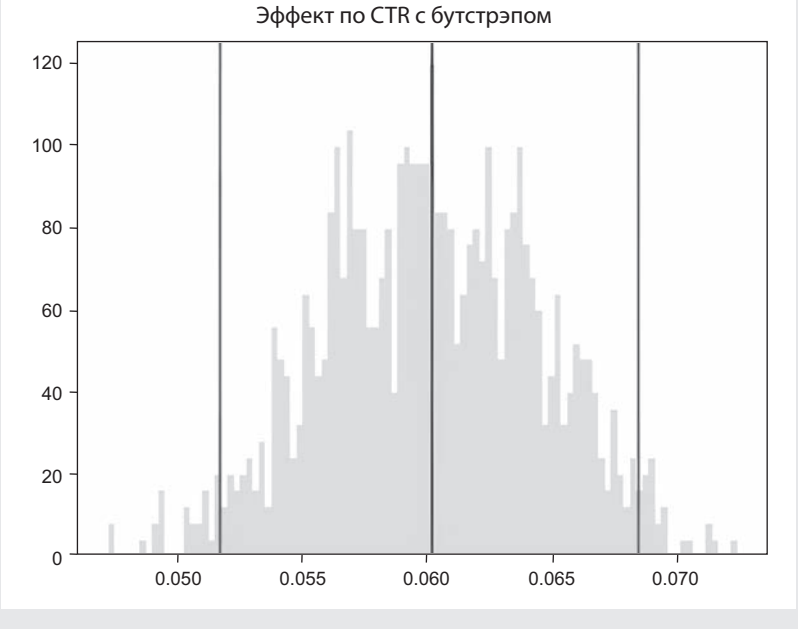
- CTR составил 9.91 % в группе A и 10.51 % в группе B (разница +6 %).
- CR составил 0.10 % в группе A и 0.13 % в группе B (разница +29 %).

Сделаем предварительный вывод:

- Новая поисковая система лишь незначительно увеличила количество кликов на просмотр (+6 %), однако изображения, на которых кликали, оказались существенно более релевантными (+29 % больше конверсий в покупки). Таким образом, A/B-эксперимент можно считать успешным.

РАСШИРЕННЫЙ ОТЧЕТ ОБ ЭФФЕКТЕ

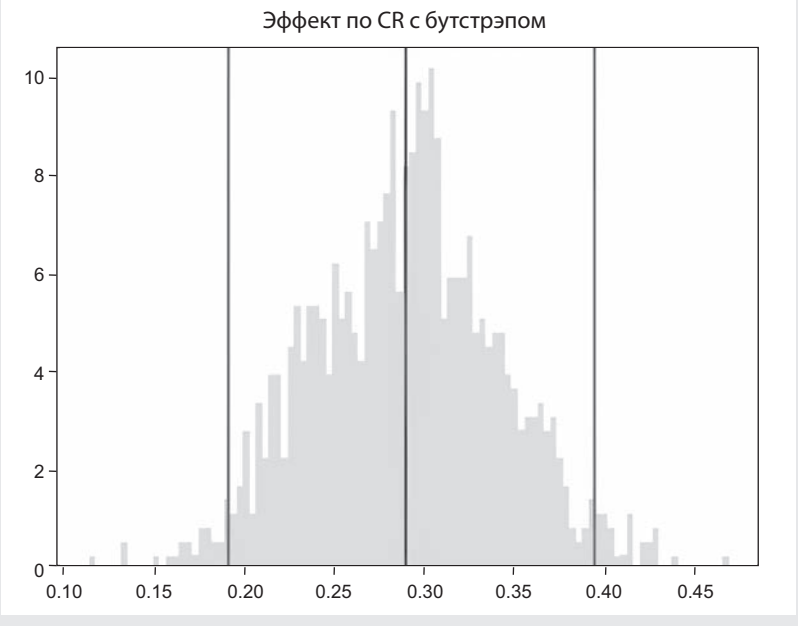
Чтобы перейти от точечной оценки эффекта к распределению, была применена бутстрэп-процедура. Мы рассматриваем нижнюю и верхнюю границы 95 %-ного доверительного интервала как пессимистичную и оптимистичную оценки эффекта соответственно. Оригинальная точечная оценка используется в качестве реалистичной.



Бутстрэп-оценка CTR

Оценки эффекта по CTR следующие:

- Пессимистичная оценка: +5.1 % (новое значение CTR — 10.4 %).
- Реалистичная оценка: +6 % (новое значение CTR — 10.5 %).
- Оптимистичная оценка: +6.8 % (новое значение CTR — 10.6 %).



Бутстрэп-оценка CR

Оценки эффекта по CR следующие:

- Пессимистичная оценка: +19.2 % (новое значение CR — 0.12 %).
- Реалистичная оценка: +29 % (новое значение CR — 0.13 %).
- Оптимистичная оценка: +39.4 % (новое значение CR — 0.14 %).

МАСШТАБИРОВАНИЕ ЭФФЕКТА

Оценим данный прирост в денежном выражении при масштабировании протестированной поисковой системы на базе ML на весь трафик, предположив, что средняя прибыль с одного фото составляет $3, а ежедневное количество визитов — 3 млн.

Базовое значение CR составляло 0.1 %. Умножая его на суточный трафик и прибыль с одного фото, получаем $9000 в день, или $ 3.3 млн в год:

- Пессимистичная оценка дает $10 800 в день (+$1800), или $3.9 млн в год (+$600 000).
- Реалистичная оценка дает $11 700 в день (+$2700), или $4.3 млн в год (+$1 млн).
- Оптимистичная оценка дает $12 600 в день (+$3 600), или $4.6 млн в год (+$1.3 млн).

Данные показатели полностью оправдывают затраченные усилия и время.

Следующим шагом станет проведение двух или трех более коротких A/B-тестов с охватом большей доли пользователей (например, 20 % → 30 % → 50 %), чтобы убедиться, что система безопасна, эффект воспроизводим, а результаты эксперимента достоверны.

Обратите внимание на следующие сопутствующие наблюдения:

- Контрольные метрики не зафиксировали инцидентов во время эксперимента.
- Среднее время поиска немного сократилось, что является положительным сигналом. Это означает, что пользовательский опыт улучшился и пользователи находят нужную информацию быстрее.
- Количество сессий, завершившихся покупкой, увеличилось. Страницы результатов стали более релевантными.
- Средняя прибыль с одного фото осталась неизменной (около $3.0).

После завершения эксперимента мы также ретроспективно оценили, сколько из всех изображений, на которых кликали, находились в топ-1000 рекомендаций новой модели. Оказалось, что показатель Recall@1000 для кликов составил 97 %. Такой результат указывает на потенциал дальнейшего улучшения системы, в частности на возможность перехода к двухэтапному пайплайну за счет добавления этапа retrieval (отбора модели-кандидата).

Итоги

- Используйте офлайн-оценку, чтобы установить ожидаемый эффект от вашей ML-системы до ее развертывания и запуска онлайн-тестирования.
- Существует множество способов установить связь между офлайн- и онлайн-метриками, включая бенчмарки и историю ранее проведенных A/B-тестов с соответствующими офлайн-оценками.
- Чтобы принимать взвешенные решения о том, является ли новая модель лучше старой (или решение на базе ML лучше, чем без ML), используйте A/B-тест (однако существуют и другие способы выявления причинно-следственной связи).
- Для успешного и безопасного проведения A/B-теста необходимо сформулировать правдоподобную гипотезу, выбрать ключевые/контрольные/вспомогательные метрики, оценить ожидаемый прирост и продолжительность эксперимента, а иногда также выполнить смоделированные A/A- и A/B-тесты перед началом реального тестирования.
- При представлении результатов эксперимента стейкхолдерам и команде необходимо указать зафиксированный эффект, ожидаемый эффект при полном развертывании сервиса, эффект, наблюдаемый в разных сегментах, выявленные в ходе эксперимента проблемы и предложения по дальнейшим действиям и улучшениям.

Часть 4

Интеграция и масштабирование

Заключительная часть книги посвящена интеграции и масштабированию. Глава 13 охватывает вопросы интеграции — от проектирования API и релизного цикла до эксплуатации системы и перехода на фоллбэк-механизмы в случае неисправностей. В главе 14 рассматривается мониторинг системы, ее надежность и работоспособность, качество и целостность данных, а также релевантность и качество модели. Глава 15 посвящена сервингу и оптимизации инференса, проблемам, возникающим на этапе обработки запросов, предпочтительным инструментам и фреймворкам, а также оптимизации пайплайнов инференса. Наконец, в главе 16 рассматриваются вопросы владения и сопровождения, а также ответственность как один из ключевых факторов поддержания работоспособности ML-системы. В ней также обсуждаются компромиссы между эффективностью команд и избыточностью, важность правильно организованной документации и обманчивая привлекательность сложности.

13 Интеграция

В этой главе

- ✓ Проектирование API
- ✓ Релизный цикл
- ✓ Эксплуатация системы
- ✓ Переопределения и фоллбэк-механизмы

Как мы уже отмечали ранее, худшее, что можно сделать, — это построить систему, а затем «положить ее на полку» и не использовать. Оба автора сталкивались с подобными ситуациями — опыт не из приятных.

Типичная ошибка новичка: полагать, что интеграция — это единоразовое событие или одна из фаз проекта. Это антипаттерн — вы не можете просто выделить несколько недель «на будущую интеграцию» и начать разрабатывать систему в вакууме. На практике это непрерывный процесс, который начинается с самого начала проекта и заканчивается лишь в момент вывода системы из эксплуатации. Более того, по завершении жизненного цикла системы необходимо провести дезинтеграцию, чтобы убедиться, что ни прямые, ни косвенные пользователи не пострадают от отключения системы. Корректно выполненная интеграция — ключевой фактор успеха: она существенно упрощает получение обратной связи и последующее улучшение системы. Чем лучше различные элементы интегрированы в систему, тем короче цикл обратной связи и быстрее итерации.

В этой главе мы обсудим, как эффективно интегрировать систему, сосредоточившись на технической стороне данного вопроса.

13.1. Проектирование API

Проектирование API — важнейшая часть процесса интеграции. Его можно воспринимать как «договор» между вашей системой и ее пользователями, но этот договор необходимо обязательно внимательно прочесть перед подписанием. Изменение архитектуры API по ходу разработки обойдется очень дорого.

Если у вас уже есть опыт работы в области машинного обучения, то вам может показаться, что этот раздел вам не нужен — вы ведь уже умеете проектировать API и делали это не раз. Но не беспокойтесь, мы не собираемся учить вас общим принципам проектирования API или рассказывать о разнице между REST и RPC. И без нас уже существует множество отличных книг и статей на эту тему (полезную подборку можно найти по ссылке https://news.ycombinator.com/item?id=24383180). Вместо этого мы сосредоточимся на ключевых особенностях и типичных подводных камнях, характерных именно для ML.

Если бы нам нужно было выделить всего два свойства хорошего API, мы бы выбрали *простоту* и *предсказуемость*. Тут нам пригодится цитата Батлера Лэмпсона (Butler Lampson), часто называемая «фундаментальной теоремой разработки ПО»: «Любые проблемы в информатике решаются добавлением еще одного уровня косвенности».

Вариация этой цитаты заключается в том, что любую проблему программирования можно решить с помощью нового уровня абстракции, за исключением проблемы чрезмерного количества абстракций. Таким образом, простота API заключается в искусстве нахождения правильной абстракции, которая не будет «протекать», то есть раскрывать слишком много внутренних деталей реализации.

Благодаря *простоте* можно легко изучать API и использовать его без глубокого понимания внутренних механизмов. Типичная ML-система часто содержит множество хендлеров и параметров, и всегда возникает желание предоставить их все внешнему пользователю. Однако это приводит к чересчур сложным решениям, где для вызова метода требуется указание большого числа параметров и в итоге становится трудно понять их смысл и взаимосвязь. Куда лучше скрыть эту сложность за простым интерфейсом, предлагающим нескольких методов с минимальным количеством параметров. Пользователи будут вам за это только благодарны.

Однако скрывать абсолютно все параметры — тоже не лучшая идея (рис. 13.1). Важно обеспечить возможность настройки поведения системы, особенно в целях отладки. Представьте, что вы устраняете неисправность в системе с десятком параметров в ночную смену дежурства и у вас нет возможности изменить ни один из них. Ситуация малоприятная! В подобных случаях стоит задать адекватные значения по умолчанию и предоставить механизм для их переопределения.

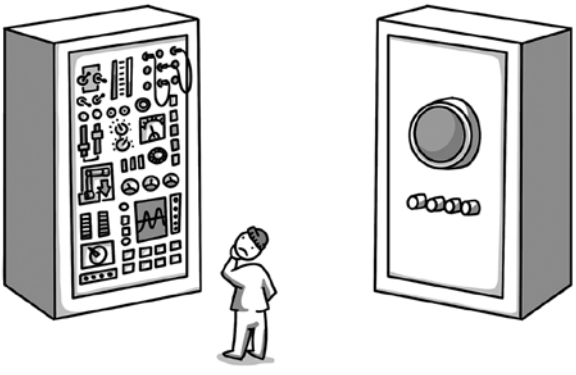


Рис. 13.1. Избыточная vs недостаточная конфигурация

Байка от Арсения

Раньше я работал в компании, которая предоставляла API для внешних разработчиков. Я отвечал за создание нового эндпоинта — интерфейса к системе классификации. На раннем этапе API казался простым — он принимал объект на входе и возвращал метку из предопределенной таксономии на выходе.

Однако точность базовой модели была неидеальной, и один из коллег предложил возвращать не одну метку, а их список. Это была неплохая идея, и я без колебаний ее реализовал. Однако на практике оказалось, что пользователям требовалась только одна метка, даже если она была неверной (их паттерны проектирования не позволяли работать со второй и последующими метками). Но поздно — список меток уже был в API, и удалить его без нарушения совместимости было проблематично. В результате API стал неоправданно сложным, и многие пользователи высказывали недовольство: «Зачем возвращать список меток, если в нем всегда только один элемент?» Слишком поспешное решение по дизайну API привело к неоптимальному результату, что впоследствии было сложно исправить.

Предсказуемость — еще одно важное свойство хорошего API. Мы уже говорили о том, что ML-системы по своей природе склонны к недетерминированности, если только их поведение специально не ограничено (см. главу 10). Однако API должны быть детерминированными и предсказуемыми — необходимо гарантировать, что один и тот же ввод всегда будет давать один и тот же вывод. Конечно, существуют алгоритмы по своей сути недетерминированные (например, генерация текста при семплировании с температурой), но это скорее исключение, подтверждающее правило.

Всегда существует возможность принудительно сделать поведение детерминированным. Один из простых примеров — использование случайного начального значения, передаваемого через параметры (и выбор собственного начального значения, если оно не указано). Кстати, хотя многие ML-библиотеки используют случайные исходные данные из глобального состояния, это невозможно в JAX — новой библиотеке численных вычислений от Google, которая появилась совсем недавно. Ее архитектура предполагает явную передачу случайного начального

состояния именно для обеспечения полной воспроизводимости. Подробнее см. по ссылке https://mng.bz/lrQ2.

Другим источником недетерминированности могут быть входные данные, включая неявные данные, например текущее время (оно тоже должно передаваться через параметры).

Рассмотрим два варианта реализации функции `predict`, использующей время в качестве входных данных и возвращающей условно случайные результаты. В листинге ниже у функции явно указан только один параметр, однако результат зависит от трех параметров, что делает его невоспроизводимым.

Листинг 13.1. Не самый лучший дизайн функции predict

```
def predict(features):
    time = datetime.now()
    return model.predict(features, time, seed=42)
```

В отличие от предыдущего примера, в следующем листинге вызывающий код контролирует все параметры, влияющие на результат.

Листинг 13.2. Более удачный дизайн функции predict

```
def predict(features, time=None, seed=42):
    if time is None:
        time = datetime.now()
    return model.predict(features, time, seed)
```

ПРИМЕЧАНИЕ Хотя в примерах выше используется Python, принципы проектирования более высокоуровневых абстракций остаются неизменными. Представьте, что эта функция — часть HTTP API, где `time` и `seed` — это вновь введенные параметры запроса. В этом случае применимы те же самые принципы.

Отдельный аспект предсказуемости — это *совместимость*. Говоря о совместимости, инженеры обычно имеют в виду либо обратную, либо прямую совместимость. Обратная совместимость означает, что новая версия API совместима со старой (старый код может работать с новой версией API без изменений). Прямая совместимость подразумевает, что старая версия API совместима с новой (новый код может работать со старой версией API без изменений).

В случае ML-систем совместимость также связана с версионированием базовой модели. Распространенной практикой является предоставление механизма запроса конкретной версии модели при инициализации.

Листинг 13.3. Добавление версии модели в API

```
class Model:
    def __init__(self, version):
        self.version = version
        self.model = load_model(version)
```

```
def predict(self, features, time=None, seed=42):
    if time is None:
        time = datetime.now()
    return self.model.predict(features, time, seed)
```

Этот пример сильно упрощен, и для более продвинутой системы вам, скорее всего, потребуется гораздо более сложное решение. Если вы хотите более подробно ознакомиться с данной темой, почитайте о реестре моделей (например, https://neptune.ai/blog/ml-model-registry).

Версионирование — задача непростая. Не стоит обновлять модель без изменения ее версии, это приведет к изменению поведения системы без какого-либо уведомления. Существует множество сценариев, при которых обновление модели считается кардинальным изменением, что обязательно должно отражаться в версии. Более того, это касается не только самой модели, но и любых других компонентов пайплайна — входных и выходных данных (IO), предобработки, постобработки и т. д. Некоторые изменения могут происходить даже непреднамеренно: например, обновление зависимостей может привести к изменению результатов, тем самым нарушив совместимость.

Байка от Арсения

Сколько времени требуется на обновление версии Python? Вопрос кажется простым, но однажды мне пришлось на собственном опыте убедиться, что на самом деле все гораздо сложнее. На практике проблема оказалась куда глубже.

Система, над которой я работал, требовала полной совместимости. И каково же было мое удивление, когда повышение версии Python (при сохранении всех остальных компонентов без изменений) привело к несоответствию в нескольких выходных значениях. После пары итераций бинарного поиска стало ясно, что проблема связана с библиотекой изображений. Оказалось, что одна и та же версия библиотеки, собранная под Python 3.5, вела себя немного иначе, чем под Python 3.6. В результате файлы, читаемые одной версией, оказывались несовместимыми с другой.

Как такое могло произойти? Оказалось, что библиотека использовала низкоуровневую JPEG-библиотеку, реализованную на языке C. При этом разные сборки одной и той же версии Python-библиотеки использовали разные версии этой C-библиотеки, так как каждая из них ссылалась на глобально установленную версию на машине сборки. Определить, какая конкретно версия использовалась в каждом случае, оказалось непросто и потребовало настоящих «археологических раскопок» ПО: пришлось копаться в логах сборки open-source проекта шестилетней давности, искать хоть какие-то зацепки и воссоздавать ту самую сборку.

Разница в поведении была незначительной, но все же достаточной, чтобы нарушить совместимость. Модели были обучены на изображениях, считанных с использованием одной версии библиотеки, и оказались слишком чувствительными к изменениям на входе. Пользователи, скорее всего, не заметили бы разницы, но с технической точки зрения это все равно было кардинальным изменением, чего по условиям пользовательского соглашения быть не должно.

В то время как Арсений столкнулся со сложностями обеспечения совместимости внутри одной системы, случай, произошедший с Валерием, показывает, что проблемы версионирования имеют еще более критическое значение, если речь идет не о внутренней системе, а о взаимодействии с внешним решением.

Байка от Валерия

Однажды мне нужно было реализовать решение KYC (know your customer — «знай своего клиента») для финансовой организации. Часть этого решения заключалась в проверке документов, удостоверяющих личность, которые загружались пользователями, и подтверждение того, что лицо пользователя отсутствует среди зарегистрированных клиентов, — обязательное требование к уникальности пользователя. Размышляя над дилеммой «купить или создать» (см. главу 3), я решил воспользоваться системой распознавания лиц от одного крупного и популярного поставщика.

Система была простая: документы поступали на вход, из них извлекалось лицо, рассчитывался вектор признаков и проверялось его отсутствие в базе данных. Если аналогичных векторов не находилось, пользователь регистрировался, в противном случае кейс направлялся на ручную проверку в службу поддержки. Все работало нормально до тех пор, пока в один не самый удачный день служба поддержки не оказалась перегруженной ложными срабатываниями. В ходе расследования выяснилось, что версия API поставщика была обновлена и новая версия возвращала результаты, несовместимые с предыдущей, что и привело к инциденту.

Подобные сбои особенно опасны из-за своей неявности. Например, если внешний API внезапно изменит имя поля, это приведет к сбою, который, к счастью, будет легко заметить и исправить. Но если меняется версия модели, это может остаться незамеченным, ведь раньше API возвращал вектор с плавающей точкой и теперь формально возвращает то же самое. Такой сбой может быть выявлен либо с помощью грамотно настроенной системы тестирования/мониторинга, либо уже после деградации функции, использующей эти данные (в нашем случае — проверки клиента).

Ничто не способно выявлять такие проблемы лучше, чем правильно подобранный набор тестов, запускаемых в процессе непрерывной интеграции (continuous integration, CI).

13.1.1. Работа с API

Со временем в индустрии сложился ряд подходов к работе с API. Ниже перечислены лишь некоторые из них — те, что мы считаем особенно эффективными. Они не связаны исключительно с ML, но применимы и в данной сфере (рис. 13.2):

- *Проектируйте как минимум два слоя API.* Речь идет о внешнем и внутреннем слоях, при этом первый логически является подсистемой второго, но не обязательно использует тот же протокол. Внешний API предназначен для пользователей или других компонентов, а внутренний используется самим внешним API. Пока внутренний API недоступен пользователям, его можно

изменять без нарушения совместимости. В свою очередь, внешний API, как подсистема внутреннего, должен разрабатываться с учетом совместимости. Такой подход позволяет разделять ответственность и упрощает поддержку внешнего API, делая его проще и стабильнее и при этом оставляя внутренний API гибким и поддающимся изменениям.

- *По возможности старайтесь отделять ML-компоненты от компонентов ввода-вывода в структуре API.* ML-сервис проще в обслуживании, если он не хранит состояние, то есть является идемпотентным. Такое удается реализовать не всегда, но к этому стоит стремиться. Данный подход полезен не только для упрощения сопровождения, но и для масштабируемости: компоненты ввода-вывода и ML можно масштабировать независимо, что важно, учитывая различия в их требованиях (например, ML-компонент, как правило, зависит от ресурсов CPU или GPU). Кроме того, это упрощает процесс улучшения системы: можно развернуть новую версию ML-компонента, не затрагивая компонент ввода/вывода, и использовать обе версии ML одновременно в ходе A/B-тестирования или при поэтапном развертывании.
- *Разработайте клиентскую библиотеку для вашего API, чтобы упростить его использование.* Независимо от того, предназначена ли она для внешних пользователей или ваших коллег, наличие клиентской библиотеки снижает порог входа, облегчает отладку и ускоряет проведение экспериментов. Она также подходит для реализации практик, которые не входят напрямую в API, — например, рекомендованные повторные попытки, тайм-ауты и т. д.
- *Используйте переключатели функций* (также известные как фича-флаги) или другие аналогичные механизмы, применяемые в вашей компании. ML-системы часто работают в условиях повышенного риска, и всегда полезно иметь возможность отключить новую версию модели или перейти к фоллбэк-решению при возникновении каких-либо проблем. Переключатели функций не являются частью API, но фактически служат обходным способом управления поведением системы без ее повторного развертывания или изменения API/поведения клиента.

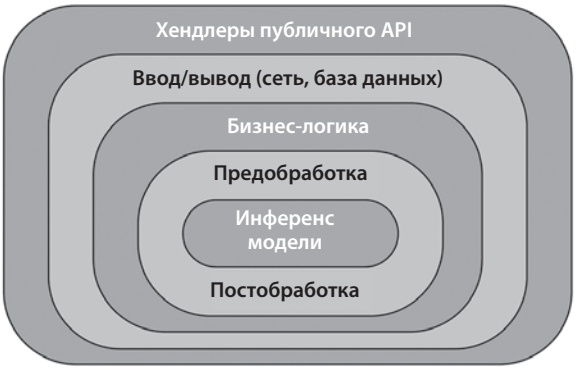


Рис. 13.2. Многослойная структура API проще в сопровождении, тестировании и разработке

13.2. Релизный цикл

Релизный цикл ML-системы обычно схож с релизным циклом обычного ПО. Однако существуют два ключевых отличия:

- ML-системы сложнее тестировать.
- Обучение новой модели (даже при полностью автоматизированном пайплайне) обычно занимает гораздо больше времени, чем компиляция кода и сборка прочих артефактов.

Рассмотрим эти моменты подробнее.

Из-за сложности тестирования одного лишь запуска тестов недостаточно, и регрессии все равно могут возникать. При обновлении обычного ПО, как правило, достаточно просто запустить тесты и убедиться, что все работает как надо. Но в случае с ML-моделями ситуация несколько иная, так как многие улучшения сопровождаются издержками. Когда обновляется ML-модель, даже если у нас есть репрезентативный датасет для тестирования и хорошее покрытие тестами программных компонентов, все равно нет гарантии, что изменения не приведут к негативным последствиям. Например, допустим, финальная тестовая выборка содержит 100 семплов, и новая модель улучшает результаты на трех из них, которые ранее считались ошибочными, но одновременно приводит к двум новым ошибкам на других семплах. Общая эффективность повысилась, но достаточно ли этого улучшения для выпуска новой версии?

Реальная практика изобилует подобными примерами, и это означает, что во многих случаях релизы требуют участия человека. Это можно сравнить с тестированием изменений в пользовательском интерфейсе, когда сотрудник должен убедиться в том, что изменения действительно приносят ощутимую пользу. Оценка компромиссов между улучшениями и регрессиями обычно гораздо сложнее, чем просто проверка корректности анимации интерфейса. Степень участия человека зависит от системы. В одних случаях ответственность лежит на ML-инженере, в других — на продакт-менеджере или приглашенном эксперте, а иногда задача может быть даже делегирована краудсорсинговым платформам, предоставляющим агрегированную обратную связь.

Существуют и более простые случаи, когда с помощью автоматического ML (AutoML) можно выпускать новую версию модели (но не всей системы) без дополнительной проверки. Представьте, что вы разрабатываете продвинутый текстовый редактор с функцией автозаполнения, имитирующей стиль автора (о, как бы нам пригодилась такая функция при написании этой книги!). Такому ПО необходимо запускать пользовательскую модель (скорее всего, поверх крупной базовой модели) для каждого пользователя. В таком случае сбор пользовательских текстов и регулярное обновление модели без участия человека — оптимальный вариант, иначе систему невозможно будет масштабировать. Однако вам придется проводить еще более дотошное тестирование: необходимо

покрыть тестами как можно больше негативных сценариев, чтобы отключать обновления модели при малейшей вероятности сбоя.

Допустим, тестирование — не проблема. Вот вы обнаружили ошибку в коде предобработки данных и уверены, что ее исправление не усугубит ситуацию. Кроме того, вы обладаете достаточным набором инструментов тестирования, которые позволят вам подтвердить свое предположение. Для обычного ПО это означало бы просто исправить ошибку, запустить тесты и развернуть новую версию — и все это в разумные сроки. Но в случае с ML-системами такой подход не сработает, потому что вам потребуется повторно обучить модель. Мы не будем приводить примеры в масштабах Big Tech-компаний, но даже по нашему опыту существуют модели, обучение которых занимает недели, так что выпустить исправление на следующий день просто невозможно. На практике это означает, что релизный цикл ML-систем обычно длиннее, чем у классического ПО, причем разброс может быть очень большим — от нескольких релизов в день до одного в год. Учитывайте это при проектировании своей системы. Длинный релизный цикл предполагает высокую надежность системы и ее тщательное тестирование, тогда как короткий цикл позволяет быть более гибкими и проводить больше экспериментов.

Многокомпонентные системы могут иметь разные релизные циклы для разных компонентов. Возьмем, например, простой поисковый движок, состоящий из четырех компонентов: индекса, содержащего документы для поиска; легкого фильтра для предварительного отбора; ресурсоемкой модели для финального ранжирования; и API для отображения результатов поиска. Документы могут постоянно добавляться в индекс (без необходимости релиза), кодовая база индекса редко изменяется, а фильтр и API — это обычное ПО без ML, которое проще тестировать и собирать, а значит, эти компоненты могут выпускаться чаще. В свою очередь, модель ранжирования обучается и выпускается раз в две недели с дополнительной проверкой. Это позволяет нам быть более гибкими в отношении не ML-компонентов и более стабильными с ML-компонентами.

Существует целый ряд техник, связанных с релизом, включая сине-зеленое развертывание (рис. 13.3) и канареечное развертывание (рис. 13.4). Они немного отличаются, но суть у них одна — одновременно в эксплуатации находятся две или более версии системы. Новые пользователи направляются на новую версию сразу после ее развертывания, в то время как старая версия еще продолжает работать. В сине-зеленом развертывании изменения разделяются (пользователи переключаются либо на синюю, либо на зеленую версию), тогда как в канареечном развертывании переход происходит постепенно и новая версия предоставляется лишь для небольшой доли пользователей. Это позволяет протестировать новую версию в продакшене и легко откатиться в случае сбоев. Такая практика вполне применима и в ML. Другой подход, который также можно использовать для ML-систем, — это декомпозиция. Например, некоторые компоненты, такие как модель, выпускаются с использованием канареечного развертывания, в то время как другие, например API, будут выпускаться более традиционным способом.

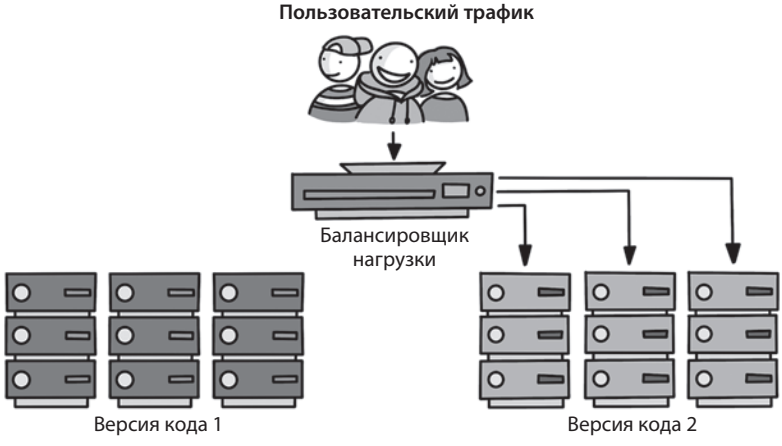


Рис. 13.3. Сине-зеленое развертывание



Рис. 13.4. Канареечное развертывание

Не стоит путать канареечное развертывание с A/B-тестированием, о котором шла речь в главе 12. Хотя с технической точки зрения они выглядят схоже (несколько вариантов системы работают одновременно, и пользовательский трафик распределяется между ними определенным образом), их цели различны. A/B-тесты используются для оценки эффективности различных версий системы, тогда как канареечное развертывание применяется для тестирования новой версии системы перед полным переходом на нее. В A/B-тестировании нас

интересует сравнение эффективности между версиями, и продолжительность теста определяется на основе статистической значимости, в результате чего вполне допустимо оставить как вариант A, так и вариант B. При канареечном развертывании цель — убедиться, что новая версия достаточно надежна для использования всеми пользователями, и как можно скорее полностью перейти на нее.

Хотя в крупных компаниях чаще всего придерживаются более строгой политики и проводят длительные релизные циклы, так происходит не всегда: многие компании стремятся сократить временной разрыв между итерациями с помощью технических и организационных подходов. Это является важной составляющей культуры DevOps, и ML не исключение. Если вам интересна данная тема, рекомендуем ознакомиться с книгой «The Phoenix Project»[1]. Она не о ML-системах и даже не особо техническая (больше похожа на «бизнес-притчу»), но подробно рассказывает о культуре DevOps и о том, как ее можно применять на практике.

Стартапы и зрелые технологические компании, как правило, действуют более гибко и используют короткие релизные циклы, хотя и здесь возможны исключения. Арсений как-то работал в стартапе, где релизы в пятницу поздно вечером были обычным делом (иногда это приводило к сбоям, которые инженерам приходилось устранять уже с бокалом пива в руках). В другом, более зрелом стартапе релизный цикл был довольно гибким: каждый разработчик мог в любой момент развернуть свой компонент, но система предупреждала, если время для этого было неподходящим (например, в пятницу после обеда).

Тем, кто имеет возможность планировать релизы, следует учитывать все зависимости — каким образом система повлияет на другие системы и как другие системы могут повлиять на нее. Крупнейшие сбои чаще всего происходят либо на уровне инфраструктуры, либо между системами или компонентами, за которые отвечает не одна и та же команда. Эффективное взаимодействие с другими командами для предотвращения подобных проблем — обязательный навык любого сеньор-разработчика. К сожалению, осознание важности этого навыка часто приходит через личный опыт (после серьезного сбоя).

Крупнейший сбой Арсения был связан с конфигурацией логгера (не то, о чем сильно заботятся при создании ML-системы). Нагрузка, связанная с ML, обрабатывалась в отдельных потоках, и для отслеживания поведения он переусложнил конфигурацию логгера, чтобы сохранять идентификаторы запросов между потоками. В одной среде все работало нормально, и поэтому данное решение впоследствии было развернуто в другой среде, где у разработчиков было меньше контроля. Проблема проявилась в самый неожиданный момент — сбой произошел только после тысячи определенных запросов, которых раньше просто не было. На выяснение причины и откат ушло много времени, и инцидент стал полезным уроком — необходимо больше проверок в процессе выпуска.

---

[1] «Проект “Феникс”. Роман о том, как DevOps меняет бизнес к лучшему».

13.3. Эксплуатация системы

Недостаточно просто построить систему и интегрировать ее напрямую с другими компонентами, использующими ее вывод. Любая система требует дополнительных связей для корректной эксплуатации — как технического, так и нетехнического характера. Некоторые из них необходимы для облегчения обслуживания и сопровождения (как с инженерной, так и с продуктовой стороны), другие вызваны неявными нефункциональными требованиями к системе (например, юридическими или требованиями конфиденциальности). Приведем несколько примеров.

13.3.1. Технические связи

CI (непрерывная интеграция) обычно является первым настраиваемым элементом всей инфраструктуры. Это позволяет выявлять и устранять проблемы интеграции, способствуя более плавному и быстрому процессу разработки. Типичные задачи CI — запуск тестов (юнит- или интеграционных) и сборка артефактов, которые будут использоваться далее по цепочке (например, для последующего развертывания). Однако на уровне CI могут решаться и другие задачи, такие как тестирование безопасности, проверка производительности, анализ затрат («Потребует ли данный релиз дополнительного масштабирования облачной инфраструктуры?»), развертывание в тестовых средах, проверка соответствия кода стандартам форматирования, сбор и отчетность по ключевым метрикам и многие другие.

Два основных типа данных, не относящихся напрямую к данным системы, но требующих хранения, — это логи и метрики. Как правило, в компании существует общепринятый подход к тому, как логи и метрики хранятся, агрегируются и мониторятся, и достаточно просто ему следовать. Данную тему мы подробнее рассмотрим в следующей главе.

Производительность системы может быть подвержена сбоям, что не должно вас удивлять. Поэтому систему необходимо подключить к платформе оповещений и управления инцидентами, используемой в компании, чтобы ответственный дежурный сотрудник был своевременно проинформирован о потенциальной проблеме и мог как-то на нее отреагировать.

Но что именно он должен сделать? Для ответа на этот вопрос полезно подготовить специальные инструкции, описывающие типовые сценарии отказов и способы их обработки. Также могут понадобиться дополнительные инструменты, облегчающие устранение инцидентов, такие как административная панель для настройки, специализированные панели мониторинга и прочие средства, которые мы рассмотрим в главе 16. Как правило, в компании существует какой-то стандарт, общий как для ML-систем, так и для всех остальных, поэтому, скорее всего, вам нужно будет просто адаптировать уже используемый набор инструментов и не изобретать заново велосипед.

Проектирование системы требует учета ее полного жизненного цикла, а не только идеального сценария работы, и здесь полезно проявить немного дотошности.

13.3.2. Нетехнические связи

Помимо сугубо технических аспектов эксплуатации необходимо учитывать и нетехнические. Они часто связаны с действиями пользователя или соблюдением нормативных требований и далеко не всегда очевидны. Что делать, если пользователь требует удалить все свои персональные данные, что допускается Общим регламентом по защите данных (вспомните главу 11 и представьте, насколько глубоко эти данные могли проникнуть)? Существует ли нормативное требование к интерпретируемости модели и как его соблюсти, не жертвуя ее качеством? Что, если высокопоставленный руководитель или инвестор стартапа столкнулся с ошибкой в системе и возмущен этим? Как отладить поведение системы в трудновоспроизводимых сценариях (например, когда дефект воспроизводится только под учетной записью того самого руководителя)? Все эти вопросы должны быть проработаны еще до выпуска системы, и ответы на них должны быть как минимум отражены на этапе проектирования. Последующее внесение изменений может оказаться слишком дорогостоящим, например, может потребоваться разработка дополнительных компонентов, таких как механизм имитации действий пользователя, административная панель для управления данными или интерпретируемостью модели. На это может уйти много времени, и потребуется участие других команд (например, юридического отдела или отдела соблюдения нормативных требований — для понимания правил либо команды веб-разработки — для создания нужной панели мониторинга).

По нашему опыту, все эти дополнительные связи и действия часто отнимают больше времени, чем разработка самой системы, и чем крупнее компания, тем больше усилий на это требуется. Учитывая текущие тенденции, вряд ли ситуация изменится в лучшую сторону в ближайшем будущем, так как по всему миру вводятся все новые регламенты, касающиеся ML и конфиденциальности данных.

13.4. Переопределения и фоллбэк-решения

Ваша система может выйти из строя по самым разным причинам — например, из-за какой-то внешней зависимости: вы извлекаете данные через сторонний API, и в какой-то момент этот API становится недоступным. Это лишь один из сценариев, когда вам может понадобиться фоллбэк-решение.

Фоллбэк — это запасной план либо альтернативное решение, которое может быть использовано в случае сбоя или недоступности основного решения. Фоллбэк обеспечивает работоспособность системы и возможность принятия решений даже в случае, если основная ML-модель по какой-либо причине не справляется.

Это особенно важно для систем, используемых при выполнении критически важных задач или в отраслях, где даже кратковременный простой может привести к серьезным последствиям. Например, фоллбэк может играть ключевую роль в системе, предназначенной для прогнозирования отказов оборудования на

производстве, обеспечивая непрерывность работы, даже если основная модель испытывает какие-то затруднения.

Еще одной причиной использования фоллбэк-решений могут быть ситуации, когда основная система не может предоставить удовлетворительный результат или достаточно уверенный прогноз либо если результат работы модели выходит за пределы допустимого диапазона.

Существует множество различных подходов к реализации фоллбэка. Один из распространенных вариантов — использование вспомогательной ML-модели, обученной на другом датасете или с использованием другого алгоритма. Это может быть базовая модель, рассмотренная нами в главе 8, или конфигурация с двумя моделями. В таком случае первая модель строится исключительно на стабильных признаках, в то время как вторая применяется для корректировки вывода основной модели с использованием более широкого набора признаков. Такие модели могут работать и совместно — вывод одной модели предпочтительнее другого в зависимости от входных данных или заранее заданного правила. Можно также реализовать мониторинг смещения входных признаков (см. главу 14) для «основной» модели, чтобы отслеживать критические изменения.

Другой вариант — использование основанной на правилах системы, обеспечивающей стабильный и предсказуемый отклик, если модель недоступна или демонстрирует низкую эффективность. Также возможна комбинация этих подходов: например, использование системы, основанной на правилах, для обработки простых случаев, а ML-модели — для более сложных сценариев (однако данный подход повышает общую сложность системы и число потенциальных точек отказа).

Простая константная модель также может выступать в роли фоллбэка. А в некоторых случаях в качестве решения может быть просто вывод явного сообщения об ошибке.

Фоллбэк-решение всегда должно сопровождаться планом активации и переключения между ML-моделью и фоллбэк-системой, которое может быть автоматическим (триггером служит событие мониторинга), ручным или гибридным — в зависимости от конкретного случая.

Особым подвидом фоллбэка является переопределение (override). Это способ вручную переопределить вывод модели в случае, когда она выдает заведомо некорректный результат: например, игнорирование вывода модели и подстановка константы вместо него, если прогноз выходит за допустимые границы или если уровень уверенности модели слишком низкий. Еще одна причина для использования переопределения — особенности релизного цикла. Например, клиент жалуется, что модель не справляется с каким-то специфическим сценарием. В идеале мы должны включить этот сценарий в обучающую выборку, повторно обучить модель, пройти все этапы проверки и развернуть обновление. Но, как мы обсуждали ранее, на это может уйти слишком много времени. Поэтому лучше временно переопределить вывод модели для данного конкретного случая

с помощью системы правил, удовлетворить запрос клиента и впоследствии уже корректно обработать ситуацию в следующем релизе.

Недостаток переопределений заключается в том, что они непрозрачны и могут легко забыться. Поэтому важно иметь механизм их отслеживания и план по их дальнейшей обработке, иначе они превратятся в технический долг. Положительным побочным эффектом наличия большого числа переопределений является возможность их использования для улучшения модели с помощью *многоканального слабого обучения с учителем* (multisource weak supervision) — метода, при котором неразмеченные данные маркируются с помощью «функций разметки». Подобное решение неидеально, но его просто реализовать. Функции разметки формируют зашумленный датасет, который затем используется для обучения модели. Подробнее об этом методе можно прочитать в статьях Александра Ратнера (Alexander Ratner) и др.: «Data Programming: Creating Large Training Sets, Quickly» (https://arxiv.org/abs/1605.07723) и «Training Complex Models with Multi-Task Weak Supervision» (https://arxiv.org/abs/1810.02840). Концепция многоканального слабого обучения с учителем получила широкое распространение благодаря популярной библиотеке Snorkel (https://www.snorkel.org/).

Однажды коллега Арсения реализовал более элегантное решение, позволившее справиться с ограничениями длинного релизного цикла. Он работал над задачей распознавания именованных объектов, и модель не всегда могла идентифицировать объекты, важные для клиентов. Однако повторное обучение модели после обнаружения проблемы было невозможно из-за длительности процесса. Поэтому он реализовал решение, основанное на базе знаний: перед запуском модели входной текст проверялся по базе знаний, и если потенциальный объект там находился, он служил подсказкой для модели. Это позволило исправить проблему без повторного обучения. Добавить пример в базу знаний можно было за минуту, причем это мог сделать и не технический специалист, что обеспечивало оперативное решение проблемы. Данный случай подробно описан в блоге (https://mng.bz/BgG1) и чем-то похож на переопределения, но вместо модификации вывода модели осуществлялось дополнение входных данных.

13.5. Дизайн-документ: интеграция

Подходы к интеграции как для Supermegaretail, так и для PhotoStock Inc. нацелены на создание удобных для пользователя, быстрых и эффективных механизмов прогнозирования запасов или выдачи результатов поиска.

13.5.1. Интеграция для Supermegaretail

Стратегия интеграции Supermegaretail ориентирована на предоставление цельной, динамичной и высокоотзывчивой системы прогнозирования, помогающей управлять складскими запасами.

ДИЗАЙН-ДОКУМЕНТ: SUPERMEGARETAIL

X. ИНТЕГРАЦИЯ

1. Фоллбэк-решения

Фоллбэки имеют решающее значение для поддержания эффективности системы в условиях непредвиденных обстоятельств. Supermegaretail использует многоуровневую фоллбэк-систему:

- *Первичный фоллбэк*: основная модель обучается на выборке наиболее значимых признаков. Она используется, если в этой выборке не зафиксировано смещения признаков или каких-либо проблем.
- *Вторичный фоллбэк*: следующий уровень представляет собой модели временных рядов, такие как SARIMA или Prophet, рассмотренные в разделе 4.4. Данные модели меньше зависят от внешних признаков, что обеспечивает более устойчивые прогнозы при наличии смещения.
- *Третичный фоллбэк*: в крайнем случае прогноз осуществляется на основе данных предыдущей недели с поправками на ожидаемые события и праздники.

Система отслеживает смещения в данных и проблемы с качеством, автоматически переключаясь на соответствующий фоллбэк для обеспечения максимально точных прогнозов.

2. Дизайн API

- *HTTP-хендлер API*: этот компонент отвечает за обработку запросов и ответов, взаимодействуя с пользователями в формате структурированного JSON.
- *API модели*: этот компонент извлекает прогнозы непосредственно из модели.

Формат запроса:

```
GET /predictions?query=<query_string>&parameters=<parameters>&version=
<version>&limit=<limit>&request_id=<request_id>&sku=<sku>&entity_id=
<entity_id>&group=<group_type>
```

Формат ответа:

```
{
  "predictions": [
    {
      "sku": <sku_id>,
      "demand": <demand>,
      "entity": <entity_id>,
      "period": <time_period_for_demand>
    },
    ...
  ]
}
```

3. Релизный цикл

A. Релиз обертки vs релиз модели

В рамках стратегии интеграции релиз обертки и релиз модели рассматриваются как два отдельных процесса. Ниже приведены особенности каждого из них.

Для релиза обертки (инфраструктуры) необходимо учитывать следующее:

- *Частота и сроки*. Релиз обертки проходит, как правило, реже, чем релиз модели. Поскольку модели чувствительны к изменениям спроса, которые могут произойти за одну ночь, важно иметь возможность оперативно обучить модель.
- *Зависимости*. Релизы инфраструктуры зависят в основном от обновлений ПО, сторонних сервисов или системных требований. Любые изменения в этих компонентах могут потребовать нового релиза.
- *Тестирование*. Необходимо провести всестороннее интеграционное тестирование, чтобы убедиться в согласованной работе всех компонентов. Кроме того, критически важно обеспечить обратную совместимость, чтобы не нарушить работу уже существующих сервисов.
- *Внедрение*. Обычно используется стандартная стратегия развертывания ПО. В зависимости от характера изменений применение сине-зеленого развертывания может не потребоваться, особенно если изменения не касаются пользователей или пакетных заданий.
- *Мониторинг*. Основное внимание уделяется состоянию системы, времени безотказной работы, времени отклика и уровню ошибок.

Для релиза модели необходимо учитывать следующее:

- *Частота и сроки*. Релизы модели проходят чаще и зависят от появления новых данных, изменений в закономерностях в данных или от значительных улучшений в методах моделирования.
- *Зависимости*. В основном определяются качеством и объемом новых обучающих данных. Любые смещения в закономерностях в данных или добавление новых источников данных могут стать причиной обновления модели.
- *Тестирование*. Перед развертыванием модель проходит тщательную офлайн-валидацию. После этого она может быть протестирована в теневом режиме, при котором ее прогнозы выполняются параллельно с текущей моделью, но не используются. Это позволяет сравнивать и проверять эффективность новой модели в реальных условиях без риска.
- *Внедрение*. При развертывании новой модели важно не только загрузить файл модели, но и убедиться, что все этапы предварительной обработки данных, генерации признаков и другие компоненты пайплайна соответствуют требованиям модели.
- *Мониторинг*. Основное внимание уделяется метрикам эффективности модели. Также необходимо отслеживать смещение данных. Подробнее об этом см. в главе 14.

B. Взаимодействие релизов обертки и модели

В случаях, когда обновления инфраструктуры влияют на модель (например, изменения в пайплайнах данных), крайне важно согласовать релизы. Любые существенные изменения в архитектуре модели могут потребовать модификации обертки. Рассматривая релизы обертки и модели как отдельные, но согласованные процессы, мы сохраняем стабильность системы при одновременном улучшении ее функциональности.

4. Эксплуатационные аспекты

Обратная связь является неотъемлемой частью непрерывного улучшения системы. Для внутренних пользователей должна быть предусмотрена система обратной связи, включая возможность переопределения. Это не только позволяет улучшить точность прогнозов, но и предоставляет бизнес-пользователям инструмент гибкой адаптации решений в режиме реального времени.

5. Неинженерные аспекты

Стратегия интеграции также учитывает неинженерные аспекты, например:

- *Административные панели*: критически важны для управления системой и получения общей картины происходящего.
- *Интеграция с корпоративными дашбордами*: обеспечивает видимость на уровне всей компании и способствует принятию решений.
- *Дополнительные отчеты*: необходимы для более глубокого анализа и получения дополнительной информации.
- *Переопределения*: важная функция, позволяющая вручную вносить изменения в случае непредвиденных или нестандартных ситуаций.

Кроме того, будут интегрированы и стандартные CI-инструменты, используемые в компании, а также типовой планировщик заданий, что обеспечит согласованность и оптимизацию рабочих процессов.

6. Развертывание

Поскольку основными пользователями системы являются внутренние клиенты, а также из-за частого выполнения пакетных заданий нет необходимости в применении сине-зеленого или канареечного развертывания. Отсутствие трафика от конечных пользователей упрощает стратегию внедрения, устраняя потребность в поэтапном развертывании.

13.5.2. Интеграция для PhotoStock Inc.

Стратегия интеграции для PhotoStock Inc. направлена на обеспечение максимально релевантных результатов поиска вне зависимости от сложности поисковых запросов при сохранении быстрой реакции системы.

ДИЗАЙН-ДОКУМЕНТ: PHOTOSTOCK INC.

X. ИНТЕГРАЦИЯ

1. Дизайн API

Наш поисковый движок должен предоставлять один HTTP API-хендлер, который принимает строку запроса и дополнительные фильтры (например, цена, коллекция, разрешение, автор и т. д.) и возвращает список ID фотографий, соответствующих запросу, отсортированных по релевантности. Помимо параметров, ориентированных на продукт, необходимо также передавать технические параметры, такие как `version`, `limit` и `request_id`.

Данный хендлер будет использоваться только внутри компании, поэтому вопросы аутентификации и авторизации можно не рассматривать, так как он работает в рамках приватной сети. Мы не можем сделать сервис полностью без сохранения состояния (он должен хранить индекс и переопределения), однако вся метаинформация, связанная с запросами, должна обрабатываться бэкенд-сервисом, который уже хранит пользовательские метаданные.

Система будет использовать каскадную фильтрацию для сужения результатов поиска. Сначала будет проводиться фильтрация по дополнительным параметрам, затем поиск ближайших соседей строки запроса в пространстве эмбеддингов, после чего окончательная модель отсортирует результаты по релевантности.

Мы рассматриваем возможность использования Qdrant (https://qdrant.tech/) как быстрой векторной базы данных, способной фильтровать и извлекать кандидатов в масштабе. Однако данная технология ранее не использовалась в компании, поэтому потребуется провести ее полноценное тестирование перед запуском в эсплуатацию. При необходимости можно рассмотреть и другие векторные базы данных.

Формат запроса:

```
GET /search?query=<query_string>&filters=<filters>&version=
                <version>&limit=<limit>
&request_id=<request_id>
```

Формат ответа:

```
{
  "results": [
    {
      "id": <photo_id>,
      "score": <score>
    },
    ...
  ]
}
```

И запрос и ответ предоставляются в формате JSON, так как это стандартный формат, используемый во всех наших внутренних API. Структура запроса и ответа на данный момент проста и интуитивно понятна, однако при необходимости может быть расширена.

API должен быть организован по следующей многослойной схеме:

- HTTP-хендлер API выступает исключительно в роли прокси к основному API и не содержит бизнес-логики, он лишь парсит запрос, передает его основному API, оборачивает ответ в JSON-формат и обрабатывает ошибки.
- API векторной базы данных отвечает за фильтрацию и извлечение кандидатов на основе эмбеддинга строки запроса.
- API модели отвечает за извлечение эмбеддингов из строки и оценку релевантности кандидатов.
- API ранжирования отвечает за сортировку кандидатов по релевантности и применение возможных переопределений.

2. Релизный цикл

Мы предполагаем, что обновления модели будут происходить относительно редко, так как обучение занимает значительное время, а данные, как ожидается, не будут сильно изменяться со временем. Новые версии модели и соответствующих API могут выпускаться с периодичностью от одного до двух месяцев, при этом большинство экстренных обновлений будет связано исключительно с индексом и переопределениями.

Индекс является ядром поискового движка и будет регулярно обновляться (речь идет о данных, а не о ПО). Новые элементы можно добавлять ежедневно (например, с помощью пакетной задачи, выполняемой каждую ночь), а также нужно быть готовыми к обновлению индекса по запросу (например, для удаления запрещенных изображений или добавления изображений по индивидуальному запросу VIP-пользователей).

3. Эксплуатационные аспекты

Внутренние пользователи могут предоставлять полезную обратную связь по результатам поиска, поэтому необходимо обеспечить для них соответствующие инструменты. В качестве начального шага можно добавить кнопку «Пожаловаться на нерелевантный результат» на странице результатов поиска для внутренних пользователей PhotoStock Inc. Она будет отправлять запрос в шлюз данных, благодаря чему в хранилище данных будет сохраняться событие с ID изображения, позицией на странице результатов поиска и строкой запроса. Эти данные затем могут использоваться при переобучении модели, анализе ошибок и для переопределений. В будущем можно рассмотреть возможность предоставления аналогичного функционала и отдельным внешним аутентифицированным пользователям (например, постоянным покупателям).

4. Переопределения и фоллбэк-решения

В качестве фоллбэка мы планируем использовать существующий поисковый движок на базе Elasticsearch. Хотя он уступает новому по уровню релевантности, он все же остается достаточно надежным решением.

Что касается переопределений, мы можем предусмотреть такую возможность для конкретных запросов, а все переопределения будут храниться в отдельной базе данных. Такая необходимость может возникнуть в случае плохой релевантности по популярным или критически важным запросам, которые невозможно оперативно откорректировать с помощью модели. На текущем этапе достаточно использовать простое хранилище типа «ключ — значение», где ключом является регулярное выражение строки запроса, а значением — список ID фотографий, которые будут отображаться в результатах поиска. Хотя такое решение не отличается масштабируемостью, оно вполне подходит для первой версии системы. В дальнейшем можно будет разработать простой интерфейс для управления переопределениями.

Существует и другой тип переопределений, связанный с так называемыми нежелательными изображениями, которые необходимо скрыть из результатов поиска (например, фотографии с элементами насилия или порнографии, по каким-то причинам прошедшие модерацию). Если будет выявлено, что какое-то изображение не должно отображаться в результатах поиска, его можно просто удалить из индекса.

Итоги

- Интеграция — это не разовое действие и не отдельная стадия, а непрерывный процесс, начинающийся с самого старта проекта и заканчивающийся только при выводе системы из эксплуатации.
- При выборе API для вашей системы двумя основными критериями должны быть простота и предсказуемость.
- Эффективные практики проектирования API для ML-систем включают: реализацию как минимум двух слоев API, разделение компонентов ML и ввода-вывода, создание клиентской библиотеки для API, а также внедрение переключателя функций или его аналогов.
- Фоллбэк-решение представляет собой некий план «Б» или альтернативный подход на случай, если основная система выйдет из строя или станет недоступна.

14 Мониторинг и надежность

В этой главе

- ✓ Мониторинг как часть проектирования ML-систем
- ✓ Работоспособность программного обеспечения
- ✓ Качество и целостность данных
- ✓ Качество и релевантность модели

Традиционная разработка ПО основывается на простом принципе: качественно созданный продукт будет демонстрировать высокую стабильность, эффективность и предсказуемость, и эти характеристики со временем не изменятся. В противоположность этому, мир машинного обучения устроен гораздо сложнее, и работа над ML-системой не заканчивается ее релизом. Это имеет практическое объяснение: если в первом случае решение работает строго в рамках заранее определенных алгоритмов, то во втором функциональность основана на вероятностной модели, обученной на ограниченном объеме определенных входных данных.

Это означает, что модель неизбежно будет подвержена деградации с течением времени, а также может демонстрировать непредсказуемое поведение из-за различий между данными, на которых она обучалась, и данными, которые она будет получать в реальных условиях.

Эти риски невозможно полностью устранить, но к ним нужно быть готовыми и уметь ими управлять, чтобы система оставалась эффективной и ценной для бизнеса в долгосрочной перспективе.

В этой главе мы поговорим о таком элементе проектирования ML-систем, как мониторинг, и о проблемах, с которыми может столкнуться ваша модель в процессе ее использования. Мы также разберемся, как отслеживать типичные изменения в поведении системы и что с ними делать.

14.1. Зачем нужен мониторинг

Запуск в продакшен функционирующего решения на базе ML вызывает чувство гордости. Но еще лучше (хотя и встречается реже) — иметь правильно выстроенную схему валидации, тестовое покрытие и отлаженные процессы непрерывной интеграции и развертывания (CI/CD). Но к сожалению, все это еще не гарантирует, что ваша система останется стабильной и не начнет «сходить с ума», то есть выдавать неожиданные, нелогичные, абсолютно неверные и необъяснимые выходные данные, которые могут существенно повлиять на работу системы, — словом, все, чего нельзя ожидать от надежной и стабильной системы (по крайней мере, без соответствующего уведомления и перехода на фоллбэк).

Без должного мониторинга даже самые хорошо обученные и точные модели могут со временем начать деградировать из-за изменений в данных, с которыми они работают. Это особенно заметно в периоды значительных изменений, например во время недавней пандемии COVID-19, когда модели, используемые для задач вроде прогнозирования доступности товаров, оценки кредитных рисков и др., столкнулись с огромными трудностями. С помощью систем мониторинга мы выявляем и устраняем проблемы с качеством данных, чтобы модели продолжали выдавать надежные прогнозы.

На рис. 14.1 показана упрощенная схема типичной ML-системы:

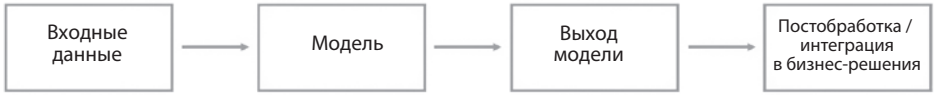


Рис. 14.1. Упрощенная структура стандартной ML-системы

Ожидать детерминированного (например, воспроизводимого) результата мы можем только в том случае, если все этапы фиксированы — абсолютно тот же самый семпл проходит через точно такую же модель (с той же структурой и теми же весами), причем эта модель не является вероятностной (некоторые из самых популярных моделей в мире, например ChatGPT или Midjourney, вероятностные, но можно зафиксировать генератор случайных чисел или случайное начальное значение и тем самым повысить воспроизводимость), и используется идентичная постобработка. Тогда можно рассчитывать на одинаковый результат. Собственно, это одна из ключевых причин для создания корректного пайплайна обучения, способного формировать и поддерживать условия для воспроизводимых итераций, тем самым способствуя улучшению системы и проведению экспериментов.

Тем не менее вероятность того, что все эти условия будут выполнены, крайне мала, поскольку мы всегда имеем дело со множеством входных данных, которые могут изменяться и оказывать влияние на модель. Давайте рассмотрим все четыре компонента с рис. 14.1, чтобы понять, с какими проблемами мы можем столкнуться.

14.1.1. Входные данные

К сожалению, чем сложнее система, тем меньше вероятность того, что входные данные будут повторяемыми (если бы это было так, можно было бы обойтись простым кэшем), а значит, можно предположить, что каждый вход будет уникален. Кроме того, распределение отдельных признаков в данных может со временем изменяться, что влияет на последующие этапы обработки. Также возможно, что в пайплайне данных произойдет какой-то сбой, в результате которого данные окажутся повреждены, или кто-то попытается воздействовать на входные данные, чтобы получить желаемый (иногда злонамеренный) эффект на выходе (такая проблема скорее имеет отношение к планированию бэкдоров, о которых можно подробнее узнать в статье «Planting Undetectable Backdoors in Machine Learning Models» (https://arxiv.org/pdf/2204.06974.pdf). Если ваша система не устойчива к подобным возмущениям, вы можете столкнуться с массой негативных последствий.

14.1.2. Модель

В отличие от входных данных, структура (или архитектура) модели и ее веса могут быть стабильными. Это, конечно, зависит от наличия онлайн-обучения (например, обновления модели на основе входящих данных в режиме реального или квазиреального времени) и запланированных обновлений (например, еженедельное повторное обучение на новом батче данных). После любого обновления, независимо от того, использовалось ли пакетное или онлайн-обучение, модель, по сути, уже не та, что раньше. А значит, при одинаковых входных данных ее поведение может измениться, что, собственно, и является причиной повторного обучения. Тем не менее мы должны быть уверены, что повторное обучение или обновление действительно было необходимо и улучшило модель (см. раздел 13.2).

Кроме того, система по своей природе может быть вероятностной или генеративной. Пример — диалог с ChatGPT (версии июля 2024 года), показанный на рис. 14.2.

На скриншоте изображены три различных (хотя и схожих) ответа ChatGPT. То, что ответы примерно похожи, вполне ожидаемо, но тот факт, что одинаковый ввод не приводит к одинаковому выводу, усложняет оценку работоспособности системы. Какой из ответов считается удовлетворительным, а какой — нет? По состоянию на январь 2023 года наш опыт с ChatGPT показал, что нередко друг за другом можно получить несколько ответов, которые существенно различаются или даже противоречат друг другу, хотя в последующих версиях это частично было исправлено.

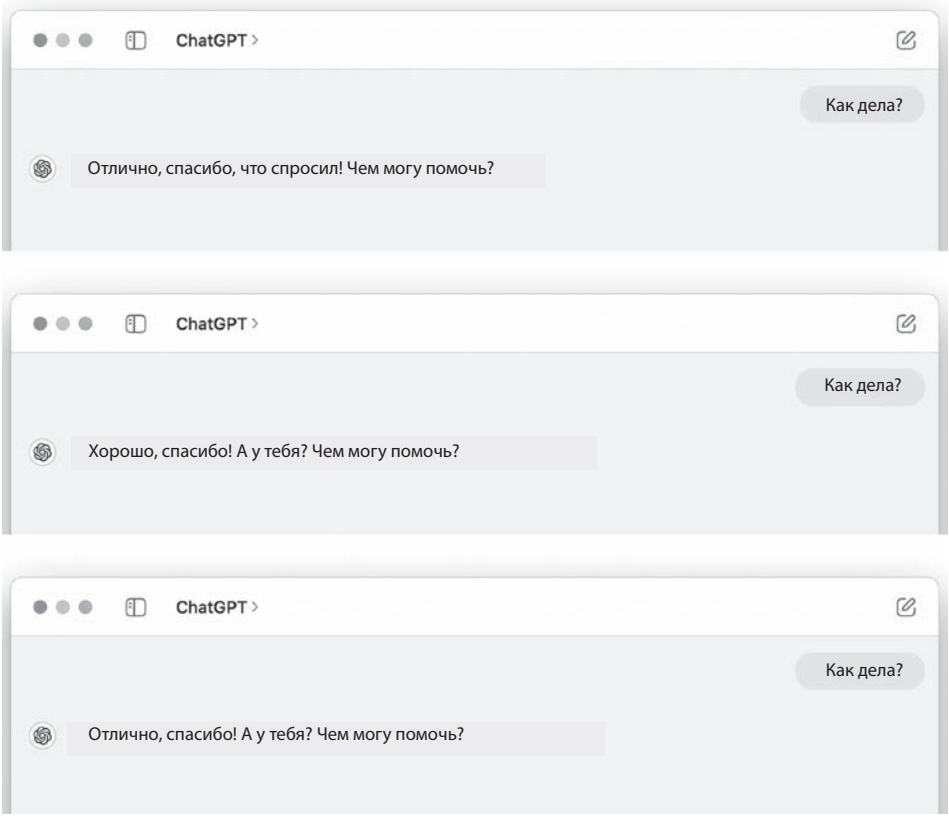


Рис. 14.2. Убеждаемся, что ChatGPT представляет собой недетерминированную модель. Ответы похожи, но не идентичны

14.1.3. Выход модели

Хотя такое и встречается достаточно редко, но иногда выход модели может стать нерелевантным из-за так называемого *дрейфа концепции* (concept drift), например, при изменении базовых зависимостей между входными данными и выходом, в результате чего зависимости, заложенные моделью, устаревают и становятся неадекватными. Данное явление отличается от смещения распределения входных данных: распределение остается прежним, но метки, которые раньше соответствовали этим данным, теперь другие. Например, в соответствии с недавними изменениями в налоговом законодательстве Великобритании, вступившими в силу в 2022 году, самая высокая налоговая ставка теперь применяется к лицам с доходом от 125 000 фунтов в год, тогда как ранее порог составлял 150 000 фунтов. Теперь представьте, что у вас запущена маркетинговая кампания по продвижению налоговых консультационных услуг и модель настроена на выбор вероятных клиентов, при этом модель не должна выходить

за маркетинговый бюджет. Подобное изменение в законодательстве сразу же потребует корректировки модели.

Другой пример — временной лаг, из-за которого восприятие какого-то события со временем меняется. Вспомним *инвестиционный скандал Бернарда Мейдоффа*, вскрывшийся лишь десятилетия спустя. То, что в течение многих лет считалось успешным хедж-фондом, на самом деле оказалось финансовой пирамидой. Другими словами, на протяжении многих лет входные данные маркировались как благонадежные, тогда как в действительности они оказались мошенническими.

14.1.4. Постобработка / Принятие решений

Важно понимать, что между качеством ML-модели и ее бизнес-ценностью может наблюдаться несоответствие. На это могут влиять различные факторы, включая изменения в сфере, в которой используется модель, а также изменения в способах ее применения.

Если модель была разработана для поддержки определенного бизнес-процесса, она может утратить свою эффективность, если сам процесс со временем изменится. Например, модель была настроена на приоритетное обслуживание входящих обращений в службу поддержки клиентов на основе предполагаемой прибыли от этих клиентов, а теперь приоритет отдан обращению, которое можно решить быстрее остальных. В таком случае модель уже не будет выполнять изначальную задачу. Соответственно, если модель используется не так, как было задумано, или вовсе не используется, она не сможет обеспечить ожидаемую бизнес-ценность. Чтобы снизить риски, необходимо постоянно отслеживать эффективность ML-модели и ее вклад в бизнес, а также по мере необходимости вносить определенные корректировки, чтобы модель продолжала соответствовать поставленным бизнес-целям.

В следующих разделах рассмотрим наиболее распространенные случаи возникновения сбоев, сосредоточившись в первую очередь на первых трех этапах. Мы обсудим, как отслеживать и обнаруживать такие случаи, как быть, если что-то изменилось, и как отразить все это в дизайн-документе, чтобы можно было подготовиться к проблемам заранее, а не после того, как случится нечто ужасное. Но прежде чем мы перейдем к потенциальным неисправностям и методам их устранения, давайте рассмотрим основные компоненты мониторинга ML-систем. В своей статье «Monitoring ML Systems in Production. Which Metrics Should You Track?» (https://mng.bz/86B2) Елена Самуйлова выделяет четыре ключевых компонента мониторинга ML-систем, графически представленных на рис. 14.3.

Основой полноценного мониторинга является бэкенд программного обеспечения, к которому затем добавляются уровни контроля качества и целостности данных, качества и релевантности модели, а также ключевых показателей эффективности (KPI) бизнеса. Последний пункт не относится к ML-системе напрямую, но влияние, которое может оказать сбой в системе на KPI, а затем

и на сам бизнес, нельзя недооценивать. Давайте разберем каждый уровень этой пирамиды в обратном порядке.



Рис. 14.3. Основные компоненты мониторинга ML-систем (источник: https://mng.bz/86B2)

14.2. Работоспособность программного обеспечения

Крайне важно рассматривать бэкенд программного обеспечения как фундамент проектирования системы. Сюда относится мониторинг производительности ПО с целью убедиться, что оно работает корректно, эффективно выполняет задачи и быстро реагирует на запросы. Отсутствие контроля за стабильностью и производительностью ПО может нанести серьезный ущерб производительности всей ML-системы. Поскольку в данной книге рассматриваются принципы проектирования ML-систем, а теме надежности традиционного ПО посвящено множество других изданий, мы не будем подробно останавливаться на данном аспекте. Тем не менее он заслуживает внимания, так как любая система надежна лишь настолько, насколько надежен ее самый слабый элемент. Более подробный анализ работоспособности систем можно найти в статье Криса Джонса (Chris Jones) и др. (https://mng.bz/EOAl).

Мониторинг работоспособности ML-системы является важным аспектом ее выхода в продакшен. Необходимо обеспечить работоспособность системы и отслеживать характеристики ее эффективности для соблюдения целевых показателей качества обслуживания (service-level objectives, SLO). Многие подходы, применяемые для мониторинга традиционного ПО, например мониторинг приложений и инфраструктуры, система оповещений и управление инцидентами — могут быть с успехом использованы и в ML-системах. Данные методы помогают поддерживать работоспособность и производительность ML-системы, а также позволяют оперативно реагировать на возникающие проблемы. Используя инструменты и практики, разработанные для традиционного ПО,

можно эффективно отслеживать и управлять состоянием и производительностью ML-системы в продакшене.

В зависимости от архитектуры развертывания ML-системы можно отслеживать разные метрики: метрики использования сервиса, такие как общее количество вызовов модели, количество запросов в секунду и уровень ошибок, а также метрики производительности системы, такие как время безотказной работы, задержка, время холодного старта, уровень ошибок и метрики потребления ресурсов, включая потребление памяти и GPU/CPU. Важно тщательно отобрать несколько ключевых метрик, которые отражают различные аспекты производительности сервиса — такие метрики часто называют *индикаторами уровня обслуживания* (service-level indicators, SLI). Они позволяют выявлять проблемы в системе и оперативно принимать меры для предотвращения сбоев или снижения эффективности.

Важно вести журнал событий и прогнозов с указанием временны́х меток для мониторинга и отладки ML-системы. Если это технически и финансово возможно (в некоторых случаях хранение результатов каждой модели может существенно повлиять на размер прибыли), мы рекомендуем записывать данные о каждом прогнозе, включая входные признаки и выход модели. Это позволит отслеживать эффективность модели и своевременно выявлять возникающие проблемы. Следует логировать каждый прогноз, за исключением случаев, когда этому мешают определенные условия, например требования законодательства о защите персональных данных или использование edge-моделей на устройствах пользователей. В таких ситуациях может потребоваться разработка обходных решений для сбора и разметки части данных.

Помимо журналов прогнозов, важно также вести журналы работы ПО — событий с временными метками, которые содержат информацию о том, что происходит внутри приложения, использующего ML. Такие журналы помогают в диагностике проблем и повышении эффективности модели. Существует множество инструментов, позволяющих централизованно собирать и анализировать журналы, например Prometheus и Grafana (решения с открытым кодом) либо AWS Cloudwatch и Datadog (облачные сервисы).

> **ПРИМЕЧАНИЕ** Вы всегда можете удалить ненужные журналы, но не сможете добавить задним числом те, что не были записаны. Не стоит вести журнал, если он не несет никакой новой информации. Убедитесь, что журналы содержат актуальные значения и ID, а не просто статические сообщения.

Ниже приведены примеры неудачной и удачной записи в журнале:

```
✗ logger.info("Обнаружена вероятность мошенничества")
✓ logger.info(f"Вероятность мошенничества = {score:.3f} для пользователя
  с ID {user_id}")
```

Вам также может пригодиться платформа контроля за работоспособностью моделей от LinkedIn (https://mng.bz/dZEo). А в документе «Service Level Objectives» (https://mng.bz/r1eJ) от Google можно найти полезную терминологию по теме.

Логи и метрики должны сохраняться в системах, анализирующих их и отправляющих оповещения в случае обнаружения отклонений. Для этих целей существует множество программных решений. Примеры self-hosted систем: ELK (Elasticsearch, Logstash и Kibana) — для логов и Prometheus или Victoria Metrics — для метрик. Аналогичные функции предлагают и облачные провайдеры, например такие как AWS Cloudwatch, а также специализированные сервисы вроде Datadog. Как мы отмечали в главе 13, подключение ML-системы к подходящему набору инструментов для логирования и сбора метрик — это обязательный шаг, значение которого часто недооценивается.

14.3. Качество и целостность данных

Мониторинг качества данных необходим для того, чтобы обеспечить точность и надежность данных, используемых для обучения ML-модели и ее прогнозов. Работа с некорректными данными неизбежно приведет к ошибочным прогнозам. Для проверки данных может потребоваться приостановка системы и переход на фоллбэк до восстановления должного качества данных либо оперативное расследование и устранение проблем. Далее мы рассмотрим наиболее распространенные ситуации, которые могут потребовать активации фоллбэк-решения.

14.3.1. Проблемы обработки

ML-системы часто зависят от различных внешних источников данных. Однако такие источники могут стать причиной проблем, влияющих на эффективность вашей системы. Например, данные могут не поступать или приходить в искаженном виде, что может быть вызвано сбоями в пайплайне данных. Представим себе ML-систему, которая персонализирует рекламные предложения для клиентов и использует данные из внутренней клиентской базы, кликстримы и логи колл-центра. Все эти источники объединяются и сохраняются в хранилище данных. Если с одним из источников возникнут проблемы, это может повлиять на работу всей системы.

Один из наших любимых примеров: задача обработки данных обрабатывает ноль строк, но при этом сообщает об успешном выполнении (в пользовательском интерфейсе отображается зеленый квадрат, указывающий на успешные извлечение, передачу и обработку данных). Все довольны — пока через неделю не выясняется, что в одном из источников данных была проблема.

Кроме того, этап обработки данных в пайплайне ML также может быть подвержен сбоям. Сюда относятся ошибки в источнике данных, проблемы с доступом к данным, ошибки в SQL-запросах или других средствах извлечения данных, обновления инфраструктуры, приводящие к изменениям формата данных, и ошибки в коде, ответственном за расчет признаков. В системах пакетного инференса такие ошибки можно обнаружить и устранить путем повторного запуска модели. Однако в высоконагруженных потоковых системах, применяемых, например, в онлайн-торговле или банковской сфере, проблемы на этапе

обработки данных могут привести к гораздо более серьезным последствиям — из-за большого объема данных и необходимости принимать решения в режиме реального времени на основе выходных данных модели.

14.3.2. Повреждение источника данных

Помимо изменений, данные могут быть утеряны из-за проблем с источником данных. Такое может происходить из-за ошибок в коде, аппаратных сбоев или сбоев внешних API. Необходимо тщательно отслеживать состояние пайплайнов данных, чтобы своевременно выявлять подобные проблемы, поскольку они могут привести к безвозвратной потере данных, требующихся для последующего повторного обучения модели.

Иногда такие сбои затрагивают лишь часть данных, что затрудняет их обнаружение. Кроме того, поврежденный источник данных может продолжать отдавать информацию, но она будет некорректной. В подобных случаях крайне важно отслеживать нетипичные значения и закономерности, чтобы оперативно выявлять потенциальные проблемы. Такие сбои могут происходить как постепенно, так и мгновенно. Представим две системы компьютерного зрения для промышленных задач: одна используется на стерильных сборочных линиях, где делаются и анализируются фотографии полуготовых устройств, а другая — на крупных металлургических предприятиях, где отслеживается процесс переработки металлического лома в новые сплавы. В первом случае может внезапно выйти из строя освещение, и поток данных станет темным. Во втором случае на камеру со временем воздействуют пыль и высокая температура, и объектив постепенно портится.

Если выявлена проблема с источником данных, необходимо оценить масштаб ущерба и предпринять соответствующие меры, например обновить данные, заменить источник или временно приостановить работу модели.

Некоторые проблемы с источниками данных, к сожалению, неизбежны. Например, возьмем систему рекомендаций для маркетплейса, где одни продавцы заполняют все характеристики своих товаров, а другие игнорируют необязательные поля. В таком случае разумно заранее предусмотреть подобные ситуации на этапе построения признаков, чтобы модель могла игнорировать пробелы в данных.

Рассмотрим более конкретный пример:

```
def get_average_rating(item):
    # возвращает среднюю оценку по всем отзывам о товаре, от 1 до 5 звезд
    try:
        # штатная ситуация: чтение из БД и возврат значения
        ...
    except Exception:
        return -1
```

Такая реализация — не лучший вариант для обработки отсутствующих или поврежденных данных, если значение –1 далее по пайплайну не заменяется.

Хотя возвращение невозможного значения часто встречается в низкоуровневом программировании, для целей ML более подходящим вариантом было бы возвращение, например, медианного значения по другим товарам.

14.3.3. Каскадные/вышестоящие модели

В более сложных ML-системах может существовать несколько моделей, зависящих друг от друга, где выходные данные одной модели служат входом для другой. Это создает цепочку взаимосвязанных моделей, уязвимую к ошибкам (рис. 14.4).

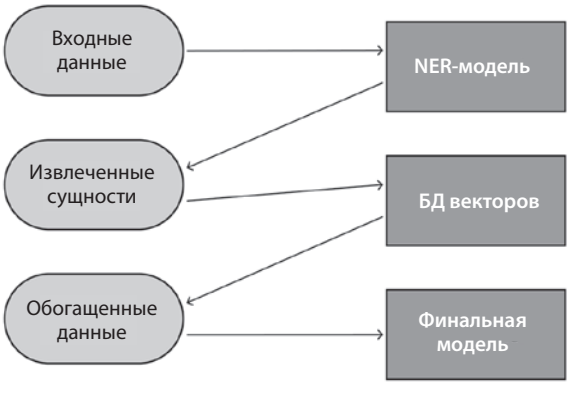


Рис. 14.4. Входные данные обрабатываются цепочкой моделей, даже небольшой дрейф на первой из них может привести к накоплению ошибок на последующих этапах

Арсений однажды работал в компании, где применялась подобная архитектура: сначала модель распознавания именованных сущностей (NER, named entity recognition) извлекала ключевые сущности, затем эти данные обогащались с помощью различных внутренних и внешних источников, и, наконец, результат подавался на вход классификатору. Компоненты системы разрабатывались и обслуживались разными специалистами, что иногда приводило к ситуациям, когда модель извлечения сущностей показывала улучшения по своим собственным метрикам, но при этом ухудшалась эффективность классификаторов, находящихся далее в цепочке. К счастью, после первых подобных инцидентов инженеры внедрили перекрестные проверки между уровнями, и больше нельзя было развертывать новые версии вышестоящих моделей без прохождения проверки на нижестоящих моделях, что позволило выявлять потенциальные ошибки на ранней стадии.

Например, в системе рекомендаций контента или товаров одна модель может прогнозировать популярность товара, а другая — на основе данных прогнозов формировать рекомендации пользователю. Если модель, оценивающая популярность, ошибается, то и вторая модель будет формировать некорректные рекомендации.

Аналогичная проблема может возникнуть в системе навигации для автомобилей, где одна модель прогнозирует ожидаемое время прибытия по маршрутам, а другая

ранжирует эти маршруты и рекомендует оптимальный. Если модель времени прибытия работает неправильно, это приведет к ошибочным рекомендациям по маршрутам, что, в свою очередь, может повлиять на транспортные потоки.

Подобные взаимосвязанные системы уязвимы к сбоям, и ошибка одной модели способна вызвать цепную реакцию с негативными последствиями для всей системы. Поэтому крайне важно тщательно проектировать и постоянно контролировать подобные архитектуры, чтобы обеспечить их корректную работу.

14.3.4. Изменение схемы данных

Изменения формата, типа или структуры данных могут стать серьезной проблемой для ML-системы и привести к потере сигнала для модели, поскольку она не сможет сопоставить новые категории со старыми или обработать новые признаки. Это особенно актуально для систем, использующих сложные признаки, зависящие от типа категории, и любые изменения в категориях могут потребовать от модели повторного обучения способам интерпретации данных.

Например, в системах рекомендаций или прогнозирования спроса в онлайн-торговле изменения в товарном каталоге могут повлиять на способность модели интерпретировать данные. Аналогично, обновления бизнес-систем или внедрение новых источников данных и API могут также вызвать сбои в работе модели, если она не была обучена на новых данных (для дополнительной информации см. главу 6).

Чтобы избежать подобных последствий, крайне важно проектировать модель уже с учетом данных сложностей и информировать бизнес-пользователей о потенциальных рисках, связанных с такими изменениями. Мониторинг качества данных также способен своевременно выявлять и устранять подобные проблемы.

14.3.5. Перекос между обучением и эксплуатацией

Перекос между обучением и эксплуатацией (training-serving skew) — это ситуация, когда ML-модель демонстрирует слабые результаты при работе с реальными данными, поскольку была обучена на искусственно созданном или очищенном датасете, не отражающем всей полноты и сложности реальной задачи. Такое может произойти, если обучающий датасет был неполный и не охватил все разнообразие и сложность ситуаций, встречающихся в действительности (подробнее см. главу 6).

Примером такого расхождения может служить модель, обученная на ограниченном наборе изображений, полученных с помощью краудсорсинга, которая показывает плохие результаты при применении к реальным изображениям разного формата и качества. Аналогично, модель, обученная на высококачественных изображениях, сделанных в лабораторных условиях, может не справляться с реальными изображениями, сделанными при плохом освещении (рис. 14.5).

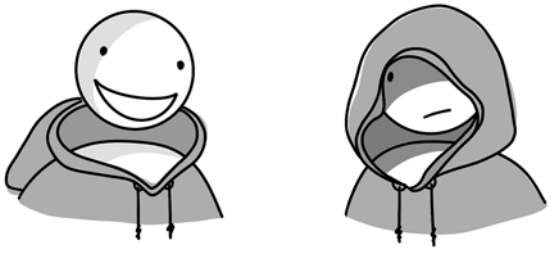


Рис. 14.5. Такое радикальное расхождение между сэндбокс-наборами изображений и реальными фотографиями может привести к значительному снижению производительности модели

Байка

Этой историей с нами поделился один из наших друзей. В одной компании разрабатывалось приложение дополненной реальности на основе компьютерного зрения, использующее данные с камер. Офис компании находился в Санкт-Петербурге — городе, расположенном на побережье Финского залива. Санкт-Петербург известен своей пасмурной погодой — среднегодовое количество солнечных часов составляет всего около 1600. Именно в таких условиях инженеры собирали большую часть данных и тестировали свое решение.

Главные инвесторы компании находились на Кипре — в средиземноморской стране, где количество солнечных часов в год составляет 3400. Когда они протестировали предрелизную версию продукта, то были шокированы — компонент ML работал крайне плохо. После небольшого расследования разработчики поняли, что проблема заключалась в несовпадении распределения данных: сцены на улице, использованные при обучении, были сняты в условиях пасмурной и снежной российской зимы, тогда как инвесторы тестировали продукт под палящим солнцем кипрского лета.

Для устранения перекоса между обучением и эксплуатацией может потребоваться дообучение модели на основе нового датасета или адаптация существующей модели с использованием данных, полученных в ходе неудачного теста. Иногда такая пробная эксплуатация может сама по себе сгенерировать достаточный объем данных для обучения новой модели или адаптации текущей. Хорошую статью по этой теме, опубликованную в Technology Review, написал Уилл Дуглас Хевен (Will Douglas Heaven) — о медицинском ИИ от Google (https://mng.bz/V2ay). Обратите внимание, что данная проблема частично пересекается с темой дрейфа данных, которая будет рассмотрена в следующем разделе.

14.3.6. Как осуществлять мониторинг и реагировать на проблемы

Предупрежден — значит, вооружен, поэтому первым и самым важным шагом в организации мониторинга является понимание потенциальных проблем. В данном разделе мы также постараемся дать некоторые практические рекомендации по работе с обнаруженными ошибками.

Мониторинг качества данных в ML имеет свои особенности, поскольку он включает в себя проверку, соответствуют ли данные, используемые для обучения модели и получения прогнозов, определенным ожиданиям. Однако подобный мониторинг актуален и в других аналитических сценариях, где могут быть использованы уже существующие подходы и инструменты.

Традиционный мониторинг данных часто проводится на макроуровне, например, отслеживается состояние всех активов и потоков данных в хранилище. Однако в ML требуется более детальный подход, сосредоточенный на конкретных входах модели. В некоторых случаях можно полагаться на уже имеющиеся данные мониторинга с предыдущих этапов, но могут потребоваться дополнительные проверки, чтобы проконтролировать этапы трансформации признаков, входные данные модели в режиме реального времени или внешние источники данных.

Для обеспечения качества данных, используемых в ML-модели, необходимо отслеживать различные метрики. Ниже приведены основные типы метрик и проверок:

- *Проверка пропущенных данных* предполагает выявление отсутствующих значений в отдельных признаках, а также оценку общего объема пропущенных данных на входе модели. Обычно допустимо наличие некоторого количества пропущенных значений, но важно следить, чтобы их доля оставалась в приемлемых пределах — как для всего датасета, так и для конкретных признаков. Также стоит учитывать различные формы представления пропущенных значений, такие как «N/A», «NaN» или «undefined», поскольку простые проверки могут их не обнаружить. Для визуализации можно использовать графики и установить порог, при превышении которого модель следует приостановить или задействовать фоллбэк. Также можно определить ключевые факторы на основе важности признаков или значений SHAP (см. главу 11) и убедиться, что именно они не пропущены. Это позволит сформулировать различные политики мониторинга для критичных и вспомогательных признаков.
- *Дублированные данные* — проблема противоположная, но при этом не менее опасная. Дубликаты часто затрагивают лишь часть датасета, из-за чего меняется распределение данных, что может повлиять на работу моделей, использующих эти данные.
- *Проверка схемы валидации* позволяет убедиться, что структура входных данных соответствует ожидаемой. Это помогает выявить ошибочные входы и зафиксировать такие проблемы, как появление новых столбцов или категорий.
- *Ограничения на типы отдельных признаков* позволяют удостовериться, что, например, числовой признак действительно передается как число. Такие проверки помогают выявить ошибки формата входных данных.
- *Проверка количества обращений к модели* позволяет удостовериться, что модель работает корректно. Это особенно важно, если предполагается ее регулярное использование, — тогда можно отследить резкие изменения или отклонения от

нормального паттерна использования. Аналогично, анализ количества ответов модели может помочь выявить сбои в ее работе или проблемы с сервисом. Такая проверка легко реализуется и может предотвратить серьезные сбои.

- *Ограничения на диапазон значений признаков* формируют ожидания относительно «нормальных» значений, включая элементарные проверки (например, «возраст меньше 100») или специфические для предметной области («нормальные условия работы датчика находятся в диапазоне от 10 до 12»). Нарушение подобных ограничений может указывать на проблемы с качеством данных. И если в простых случаях достаточно общих знаний, то в более сложных сферах вам потребуется глубокое понимание предметной области.
- *Статистика признаков* позволяет отслеживать средние значения, диапазоны минимум/максимум, стандартные отклонения, корреляции между признаками, процентильное распределение или результаты специальных статистических тестов. Это помогает выявлять менее очевидные ошибки, когда признак ведет себя аномально, оставаясь в пределах ожидаемого диапазона. Для категориальных признаков можно использовать гистограммы или распределение по классам — такие методы подходят для проверки вручную, но их сложно применить в автоматическом мониторинге.
- При наличии аномальных эффектов можно применять *методы обнаружения аномалий и выбросов* для фиксации «необычных» значений и выявления испорченных входных данных. Это позволяет отслеживать как отдельные выбросы, так и их общее количество.

Для неструктурированных данных, таких как изображения, тексты или аудиозаписи, применимы схожие принципы. Здесь невозможно использовать простые проверки вроде «возраст клиента должен быть в диапазоне от 12 до 100», но можно ввести вспомогательные признаки поверх исходных данных, которые будут использоваться только для мониторинга качества данных. Такие признаки не обязательно должны использоваться самой моделью (то есть, например, модель по-прежнему применяет глубокое обучение к изображениям), но служат для мониторинга качества данных. Это может быть яркость или цветовая температура для изображений, длина (количество символов) для текстов, распределение частот для аудио, и т. п.

В большинстве случаев полезно проводить *валидацию входных и выходных данных отдельно для каждого этапа пайплайна*, чтобы упростить выявление источника проблемы. Это особенно актуально в ситуациях, когда пайплайн сложный и включает несколько этапов, таких как объединение данных из различных источников или применение нескольких трансформаций. Проверяя данные на разных стадиях пайплайна, можно локализовать источник проблемы и быстрее ее устранить. Если же проверять только финальный результат вычислений можно обнаружить, что некоторые признаки некорректны, и тогда будет сложнее понять, где именно произошел сбой, и, возможно, придется заново пройти весь пайплайн.

Выбор метрик для мониторинга зависит от множества факторов, таких как архитектура развертывания модели (например, пакетный режим, работа в режиме реального времени или потоковая обработка), особенности данных и процессов реального мира, значимость конкретного сценария использования и желаемый уровень реактивности. Например, если цена ошибки высока, могут потребоваться более сложные проверки качества данных и онлайн-валидация в формате «пройдено/не пройдено» перед тем, как использовать прогнозы модели. В других случаях может быть достаточно реактивного подхода с отслеживанием на дашборде средних значений определенных признаков или доли пропущенных данных для наблюдения за изменениями.

Одной из проблем мониторинга качества данных в ML является сам процесс его реализации. Обеспечение должного качества данных, поступающих в пайплайны ML, критически важно, однако создание полноценной системы мониторинга может быть трудоемким. Это особенно актуально в случаях, когда необходимо использовать экспертные знания за пределами ML-команды или настроить большое количество проверок — как для необработанных входных данных, так и для преобразованных признаков.

Еще одна сложность — управление большим числом точек контроля в процессе мониторинга качества данных. Важно спроектировать мониторинговую систему таким образом, чтобы она позволяла выявлять проблемы, не перегружая пользователей. Если система мониторинга ежедневно генерирует десятки ложных тревог, в какой-то момент они будут просто игнорироваться, поэтому важно найти баланс между чувствительностью и надежностью.

Наконец, определение первопричины проблемы с качеством данных может быть затруднено, особенно если пайплайн включает множество шагов и преобразований. Мониторинг качества данных тесно связан с отслеживанием происхождения и трассировкой данных, реализация которых требует дополнительных усилий.

Мы настоятельно советуем ознакомиться со статьей Google под названием «Data Validation for Machine Learning» (https://research.google/pubs/pub47967/). Ниже приведено краткое изложение этой статьи с перечнем действий, рекомендуемых для эффективного мониторинга системы:

- Определите *ключевые измерения качества данных*, критически важные для вашего сценария использования ML, такие как полнота, точность, своевременность и согласованность.
- Установите *ограничения* качества данных по каждому измерению и задайте допустимые пределы для каждого из них. Например, можно установить требование, чтобы полнота данных составляла не менее 95 %, то есть не более 5 % пропущенных значений.
- Внедрите *пайплайн валидации данных*, который будет регулярно проверять соответствие данных заданным ограничениям. Пайплайн должен формировать отчет о состоянии данных по каждому измерению.

- Настройте надежную *систему уведомлений и оповещений* на случаи, когда качество данных выходит за пределы допустимого диапазона. Это позволит своевременно предпринимать меры по устранению проблем. Под надежностью понимается баланс между ложными срабатываниями и пропущенными ошибками, зависящий от контекста.
- *Постоянно отслеживайте* и улучшайте качество данных, например обновляйте существующие ограничения или добавляйте новые по мере понимания данных и специфики сценария использования.

Для более подробного изучения данной темы советуем ознакомиться с упомянутой выше статьей Google.

Мы также рекомендуем внедрить фреймворк управления данными, чтобы гарантировать поддержание системы валидации данных и ее соответствие бизнес-целям. Сюда относится определение ролей и обязанностей по управлению качеством данных, а также установление процессов по их улучшению и решению возникающих проблем.

14.4. Качество и релевантность модели

Даже если ПО функционирует корректно, а данные обладают высоким качеством, это еще не гарантирует, что ML-модель будет работать так, как вам нужно. Одна из возможных проблем — деградация модели, когда ее эффективность снижается: либо внезапно, либо постепенно.

Деградация модели (model decay), также известная как *дрейф модели* (model drift) или ее устаревание, — это явление, при котором эффективность модели ухудшается со временем. Это может происходить по различным причинам, включая изменения в данных или в реальных зависимостях, на которых модель была обучена. Скорость деградации модели может сильно варьироваться: некоторые модели могут работать годами без необходимости в обновлении, другие же требуют ежедневного повторного обучения на свежих данных. Один из способов отслеживать деградацию модели — это регулярный мониторинг ключевых метрик эффективности и их сравнение с историческими значениями. Если метрики начинают существенно ухудшаться, это может быть сигналом о деградации модели (рис. 14.6).

Существует два основных типа дрейфа модели (рис. 14.7):

- *Дрейф данных* возникает, когда модель работает с новыми для нее входными данными, например, с данными из новой демографической группы. Исходный датасет оказался недостаточно репрезентативен, чтобы модель могла обобщать.
- *Дрейф концепции* возникает, когда изменяются взаимосвязи в данных, например, при изменении поведения пользователей.

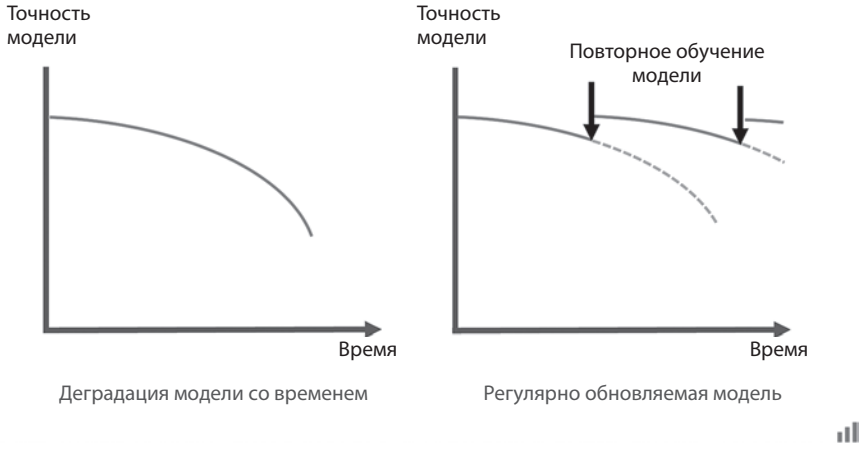


Рис. 14.6. Регулярное повторное обучение помогает справляться с дрейфом модели

Крайне важно постоянно отслеживать дрейф модели и принимать соответствующие меры. Для поддержания точности и эффективности модели можно ее повторно обучать.

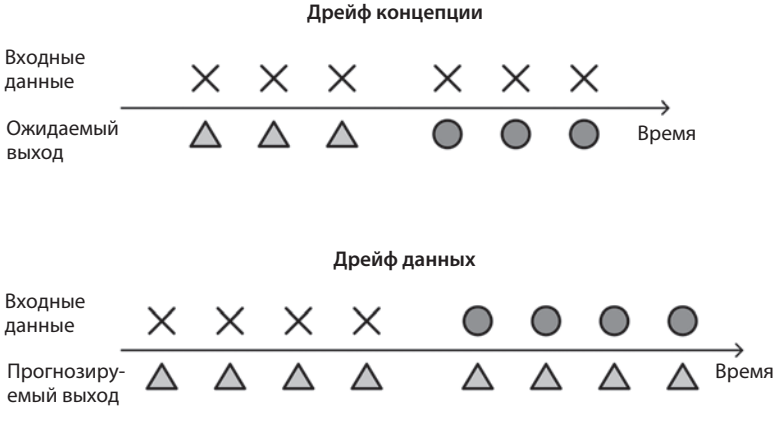


Рис. 14.7. При дрейфе концепции одинаковые входные данные могут приводить к разному выходу, тогда как при дрейфе данных изменяются входные данные, а модель остается неизменной

Возможные причины дрейфа модели:

- Изменения в мире — начало пандемии, войны, кризиса или изменения в законодательстве.
- Целенаправленные изменения в бизнесе, например запуск приложения в новом регионе или для новой категории пользователей.

- Действия со стороны злоумышленников, пытающихся подстроиться под поведение модели.
- Петля обратной связи, при которой сама модель влияет на реальность. Например, система рекомендаций использует текущую популярность товара как признак и рекомендует его, вследствие чего товар становится еще более популярным, — возникает эффект самоподкрепления.
- Деградация модели также может возникнуть при несоответствии между проектным назначением модели и ее фактическим использованием. Например, если модель скоринга лидов изначально была разработана для прогнозирования вероятности конверсии, но пользователи начинают использовать ее для сценарного анализа, подавая различные комбинации входных данных для оценки влияния различных факторов на ее решения, это может привести к снижению качества работы модели. В таких случаях лучше использовать другой аналитический инструмент, более подходящий для нового сценария. Арсений как-то раз допустил подобную ошибку: в компании агрегатора такси его команда разработала модель для оценки времени прибытия водителя к месту назначения. Сама модель работала корректно, но ее попытались применить в немного ином контексте — для оценки общей продолжительности поездки, и результат оказался неудовлетворительным. Изначально задача касалась только коротких поездок (свободный водитель подъезжает к пассажиру), тогда как в новом сценарии участвовали и длительные поездки.

Некоторые специалисты выделяют также дрейф выходных данных, под которым понимается изменение прогнозов, рекомендаций или других ответов ML-модели. Подобные изменения можно выявить путем сравнения «новых» выходных данных со «старыми» с использованием статистических тестов или описательной статистики. Например, если модель, ранее редко рекомендовавшая покупку солнцезащитных очков, начинает включать их в каждый блок рекомендаций, это может свидетельствовать о дрейфе выходных данных.

Дрейф выходных данных является одним из признаков, позволяющих выявить и диагностировать дрейф модели, поскольку он указывает как на снижение качества модели, так и на изменение взаимосвязи между входными и выходными данными, что может относиться и к дрейфу концепции, и к дрейфу данных. Обнаружение и устранение дрейфа выходных данных позволяет гарантировать, что модель продолжит выдавать надежные и точные результаты.

Дрейф модели может привести к увеличению числа ошибок или к неверным прогнозам, а в особо сложных случаях всего за одну ночь модель может оказаться полностью непригодной. Необходимо постоянно отслеживать признаки дрейфа и своевременно принимать меры для сохранения ее точности и эффективности (чтобы получить дополнительную информацию, см. главу 8).

14.4.1. Дрейф данных

Дрейф данных, также известный как *дрейф признаков*, или *ковариационный сдвиг* (covariant shift), — ситуация, при которой входные данные так сильно изменяются, что ML-модель становится нерелевантной. Такое может происходить, когда распределение переменных в датасете значительно отличается от того, на котором модель обучалась. В результате модель начинает плохо справляться с новыми данными, даже если продолжает показывать хорошие результаты на старых данных.

Один из примеров дрейфа данных — когда модель, обученная на прогнозирование вероятности покупки пользователем на онлайн-платформе, применяется к новой группе пользователей, которая была привлечена другой рекламной кампанией. Если новые пользователи пришли, например, из Facebook, а в обучающей выборке было мало подобных примеров, модель может показать плохие результаты для данного сегмента. Аналогично, дрейф данных может возникнуть при использовании модели в новом географическом регионе, для другой демографической группы или при изменении распределения ключевых признаков во времени.

Для устранения дрейфа данных может потребоваться повторное обучение модели на новых данных или построение новой модели специально под новый сегмент. Мониторинг данных и качества работы модели позволяет своевременно выявлять дрейф данных и принимать соответствующие меры до того, как эффективность модели существенно снизится. Так как дрейф данных не является проблемой самих данных, его необходимо устранять на уровне модели.

14.4.2. Дрейф концепции

Дрейф концепции (concept drift) возникает тогда, когда выявленные моделью закономерности больше не соответствуют действительности, даже если распределения входных признаков остаются неизменными. В зависимости от масштаба изменений дрейф концепции может привести как к снижению точности модели, так и к полной потере ее актуальности.

Существует несколько типов дрейфа концепции: *постепенный* (gradual), *внезапный* (sudden) и *периодический* (recurrent).

Постепенный, или *инкрементный*, дрейф возникает при изменении внешних факторов со временем, что постепенно ухудшает работу модели (рис. 14.8). Такой тип дрейфа обычно предсказуем и может быть вызван множеством факторов, включая изменение потребительского поведения, колебания в экономике или износ оборудования, если данные поступают с физических датчиков.

Для устранения постепенного дрейфа концепции может потребоваться повторное обучение модели на новых данных или даже ее полная переработка. Чтобы правильно определить необходимые действия, важно отслеживать качество модели с течением времени, чтобы убедиться, что она по-прежнему делает точные

и надежные прогнозы. Скорость, с которой ухудшается работа модели, или ее «старение», может значительно различаться в зависимости от конкретного приложения и используемых данных.



Рис. 14.8. Постепенный дрейф концепции

Для оценки того, насколько быстро модель «стареет», можно провести тест с использованием устаревших данных и измерить эффективность модели при разной частоте повторного обучения. Это может дать представление о том, как часто следует обновлять модель с учетом новых данных, чтобы сохранить ее точность. Также важно учитывать влияние внешних факторов, таких как изменения на рынке или выпуск новых продуктов, которые могут изменить взаимосвязи между входами и выходами модели и вызвать дрейф концепции.

Мониторинг качества ML-моделей и их регулярное переобучение по мере необходимости — крайне важный аспект поддержания их эффективности.

Внезапный дрейф концепции обычно происходит вследствие резких или значительных внешних изменений, которые трудно не заметить. Подобные изменения могут затронуть любые модели, даже те, которые считаются стабильными. Например, пандемия COVID-19 практически мгновенно изменила паттерны перемещений и покупательского поведения, что привело к неспособности моделей прогнозирования спроса предугадать всплеск интереса к определенным товарам или массовую отмену авиарейсов из-за закрытия границ. Помимо событий, подобных пандемии или обрушению фондовых рынков, внезапный дрейф концепции может быть вызван изменениями ключевых процентных ставок центральными банками, технической модернизацией производственных линий или значительными изменениями интерфейса приложений. Такие изменения могут привести к неспособности модели адаптироваться к новым паттернам и к ее устареванию либо потере релевантности из-за изменения пользовательского пути.

В ML-системах с течением времени возникают определенные повторяющиеся события или паттерны поведения. Например, поведение пользователей может быть иным в праздничные периоды или в определенные дни недели. Такие повторяющиеся изменения, также известные как *периодический*, или *повторяющийся*, *дрейф*, можно спрогнозировать и учесть при проектировании системы (рис. 14.9). Например, создать набор моделей, применяемых при определенных условиях, или встроить в систему учет циклических изменений и особых событий, чтобы компенсировать этот тип дрейфа и предотвратить снижение эффективности модели.

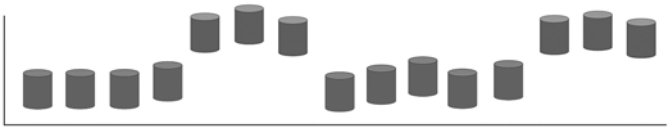


Рис. 14.9. Периодический дрейф концепции

14.4.3. Как осуществлять мониторинг

Мониторинг качества модели служит двум основным целям:

- обеспечить уверенность в надежности модели;
- выдавать предупреждение, если обнаружены сбои в ее работе.

Эффективная система мониторинга должна предоставлять достаточно контекста для оперативного выявления и устранения возникающих проблем с моделью. Можно выделить три основных сценария:

- повторное обучение модели;
- переработка модели;
- использование стратегии резервного копирования.

Дрейф данных более критичен для табличных данных по сравнению с другими модальностями, например изображениями или речью. Если наблюдается задержка между прогнозом и моментом получения эталонных (ground truth) данных, это может быть поводом для мониторинга прокси-метрик, включая дрейф данных и прогнозов. В случае моделей повышенного риска или критически важных систем, вероятно, потребуется более детализированный мониторинг и использование специфических метрик (например, показатели справедливости). Для остальных же моделей будет достаточно провести мониторинг стандартных метрик, исходя из конкретного типа модели.

Существуют сотни различных метрик, которые можно рассчитать для оценки эффективности ML-модели (см. главу 5). Ниже мы рассмотрим несколько категорий, на которые можно условно разделить все метрики.

Метрики качества модели оценивают фактическое качество прогнозов модели. Данные метрики можно вычислить, только если у вас имеются эталонные значения или обратная связь. К ним относятся:

- средняя абсолютная ошибка и среднеквадратичная ошибка для регрессионных моделей;
- доля верных результатов (accuracy), точность (precision) и F1-мера для классификационных моделей;
- точность топ-k и средняя точность для моделей ранжирования.

Качество модели по сегментам означает отслеживание эффективности модели для конкретных подмножеств данных, например выделенных по географическому признаку. Это позволит выявить расхождения в качестве работы модели на отдельных сегментах.

Дрейф прогнозов возникает, когда прогнозы модели существенно изменяются со временем. Для выявления дрейфа прогнозов можно использовать статистические тесты, метрики расстояния между вероятностными распределениями или анализ изменений описательной статистики выходных данных модели. Чтобы ознакомиться с данной темой подробнее, рекомендуем прочитать статью Ольги Филипповой (https://mng.bz/x60d).

Дрейф входных данных относится к изменениям во входных данных, используемых моделью. Он может быть выявлен путем отслеживания изменений описательной статистики отдельных признаков, проведения статистических тестов, применения метрик расстояния для сравнения распределений либо выявления изменений в линейной корреляции между признаками и прогнозами. Мониторинг дрейфа входных данных позволяет определить, способна ли модель работать в незнакомой среде.

Выбросы (outliers) — это необычные частные случаи, при которых модель работает некорректно. Необходимо выявлять подобные случаи и помечать их для дальнейшего анализа экспертами. Обратите внимание, что выбросы отличаются от дрейфа данных, при котором целью является обнаружение смещения распределения в целом. Выбросы можно выявить с помощью статистических методов и метрик расстояния.

Байка от Валерия

Когда я работал в известной криптовалютной компании, я стал активным пользователем ее сервисов, включая, разумеется, возможность покупки криптовалюты за фиатные деньги. И все шло хорошо, пока однажды меня не заблокировала система защиты от мошенничества.

Почему меня заблокировали? Потому что я совершил слишком много транзакций. Почему так получилось? Потому что лимит на транзакции, введенный командой разработки, заставил меня совершить больше транзакций, чем я сделал бы в обычной ситуации. Это лишь один из возможных подходов к обработке выбросов — в данном случае полагаться на один признак (пользователи, совершившие слишком много транзакций), — но он не очень удобен для пользователей и приводит к потере дохода от ошибочно заблокированных клиентов. К счастью, нам удалось улучшить систему, внедрив многоступенчатое обнаружение выбросов и специальный поток обработки (иногда с ручной проверкой).

Справедливость также имеет ключевое значение для системы. В некоторых случаях необходимо убедиться, что модель работает с одинаковой эффективностью

для различных демографических групп. Для оценки смещения модели можно использовать такие метрики, как демографический паритет или равные шансы.

Процесс мониторинга неизбежно сопровождается скрытыми трудностями, в числе которых:

- *Отсутствие готовых решений*. Актуально для проблем, которые до сих пор не имеют универсального решения, поскольку выбор подходящих метрик и методов зависит от конкретного контекста и целей модели. Поэтому крайне важно понимать как саму модель, так и данные, чтобы выбрать правильный подход к мониторингу.
- *Мониторинг без эталонных данных*. Качество прогнозов модели зачастую трудно оценить без доступа к эталонным данным. В подобных случаях можно использовать прокси-метрики, внимательно подбирая пороговые значения и условия срабатывания сигналов тревоги.
- *Вычисление метрик в масштабе*. Расчет сложных метрик в больших объемах может потребовать много ресурсов, особенно при работе с распределенными системами. Необходимо найти быстрые, эффективные и масштабируемые способы вычисления метрик.

14.4.4. Как реагировать

Потребовалась бы отдельная книга, чтобы описать все возможные сценарии и способы устранения дрейфа данных и концепции, поэтому мы рассмотрим лишь некоторые из них.

Дрейф данных

При столкновении с дрейфом данных есть два варианта действий. Первый — отслеживать эффективность модели и фиксировать изменения метрик, таких как точность, средняя ошибка и уровень мошенничества. При снижении эффективности стоит заново оценить модель и определить, насколько она соответствует текущим данным. Второй вариант — добавить в модель дополнительные данные или признаки, чтобы более точно отразить изменившиеся закономерности в данных. В некоторых случаях может потребоваться полное повторное обучение модели на обновленных данных.

Если вам доступны новые метки, а пайплайн обучения уже настроен и запущен, появляется соблазн просто нажать кнопку «обучить заново» (больше информации можно найти в статье Эмели Драл (Emeli Dral) (https://mng.bz/AaYo). Однако, прежде чем это сделать, стоит копнуть глубже.

Проверьте, связано ли изменение с качеством данных или это действительно новое поведение, которое модель пытается зафиксировать. Проблемы с качеством данных могут возникнуть по разным причинам, включая ошибки при вводе, изменения в схеме данных или сбои во входящих потоках данных от других

моделей. Важно иметь отдельные проверки для контроля качества данных и оперативно устранять такие проблемы (см. раздел 14.3).

Если дрейф данных все-таки был обнаружен и подтвержден, попытайтесь выяснить его причину. Это поможет определить действия, необходимые для его устранения и сохранения точности и эффективности модели.

Для начала можно построить распределения признаков, которые подверглись дрейфу, чтобы получить представление о характере изменений. Также можно проконсультироваться с экспертами в данной области, которые прольют свет на реальные причины изменений.

Если был обнаружен еще и дрейф концепции, возможно, изменилась сама взаимосвязь между признаками и выходными значениями модели, даже если распределения признаков остались прежними. Визуализация взаимосвязей также может помочь в понимании проблемы. Например, построение графиков корреляций между определенными признаками и прогнозами модели поможет выявить изменения в них. Дополнительную информацию может дать анализ смещений в попарных корреляциях признаков.

Важно *определить, насколько наблюдаемый дрейф значим и требует реакции*. Для этого можно установить пороговые значения срабатывания детектора дрейфа, адаптированные под конкретный сценарий использования, и получать уведомления при потенциально значимых изменениях.

Тем не менее зачастую такие пороговые значения приходится подбирать методом проб и ошибок, поскольку заранее спрогнозировать, каким образом будет происходить дрейф данных, довольно сложно. Когда срабатывает оповещение о дрейфе, необходимо оценить его характер и масштаб, а также влияние, которое он может оказать на эффективность модели. Для этого можно проконсультироваться с экспертами или провести дополнительный анализ для более глубокого понимания происходящих изменений. На основании данной оценки вы в дальнейшем примете решение о том, стоит ли реагировать на зафиксированный дрейф или нет. В некоторых случаях можно определить причины дрейфа и принять решение (временно) не предпринимать никаких действий.

Например, если предполагается, что в ближайшее время появятся дополнительные метки или данные, можно отложить какие-либо действия до тех пор, пока эта информация не будет учтена. Если же оказалось, что это было ложное срабатывание, можно изменить условия генерации оповещений о дрейфе или используемые статистические тесты, а также игнорировать уведомление, чтобы больше не получать подобных оповещений.

В некоторых случаях модель может продолжать работать эффективно, несмотря на дрейф. Например, если какая-то категория становится более распространенной в прогнозах модели, но это соответствует наблюдаемому дрейфу признаков и ожидаемому поведению, возможно, не стоит предпринимать никаких

дополнительных действий. В подобных ситуациях можно продолжить использование модели без повторного обучения или обновления. Однако, учитывая возможные последствия подобного решения, крайне важно внимательно отслеживать работу модели и убедиться, что она остается эффективной.

Адаптируйте этап предварительной обработки в пайплайне. Предположим, что ваша система использует изображения, получаемые с камеры на заводе. В какой-то момент менеджер завода решает заменить лампы освещения, в результате чего производственная линия становится более освещенной, как и получаемые изображения. Если это влияет на эффективность модели из-за дрейфа данных, одним из решений может быть применение функции затемнения, чтобы имитировать исходное распределение данных.

Повторно обучите модель с использованием обновленных или новых данных. Это зачастую самый прямолинейный подход, который может быть эффективен, если необходимая информация доступна. Повторно обучив модель на новых или обновленных данных, можно улучшить ее эффективность и адаптировать к изменяющимся закономерностям (если пайплайны обработки данных продолжают работать). Если вы следуете практикам, описанным в главе 10, повторное обучение модели не должно представлять для вас никакой сложности.

Вместо того чтобы просто запустить повторное обучение, можно *разработать ML-модель, более устойчивую к дрейфам*, — скажем, с использованием более устойчивых архитектур (например, рассчитанных на онлайн-обучение) или методов адаптации к предментной области, чтобы повысить устойчивость модели к изменениям в распределении данных. Ниже приведены дополнительные способы борьбы с дрейфом данных.

- *Взвешивание семплов*: предполагает присвоение большего веса более новым семплам в обучающей выборке для приоритизации новых закономерностей. Это простой способ справиться с дрейфом данных, однако он может оказаться неэффективным, если дрейф значительный.
- *Создание отдельных моделей для разных сегментов*: если модель плохо работает на отдельных сегментах данных, можно построить отдельную модель специально для них или же использовать несколько моделей, каждая из которых отвечает за определенный сегмент. (По нашему опыту, одна модель почти всегда эффективнее нескольких, поскольку использует больше данных, но тут многое зависит от контекста.)
- *Изменение целевой переменной прогнозирования*: можно повысить эффективность модели, изменив целевую переменную. Например, переход от еженедельного прогнозирования к ежедневному позволит модели лучше учитывать краткосрочные изменения в данных. Кроме того, можно изменить тип используемой модели, например перейти от модели регрессии к классификатору (в некотором смысле это можно рассматривать как более «грубое» переосмысление задачи).

- *Использование знаний экспертов в процессе обучения модели*: можно использовать экспертные признаки или методы активного обучения для управления выбором данных, на которых будет обучаться модель. В конечном счете наиболее эффективный подход будет зависеть от конкретных характеристик ваших данных и требований к приложению.
- *Введение дополнительной регуляризации*: использование техник, которые могут косвенно противодействовать дрейфу. Тут стоит вспомнить об аугментации — распространенной практике регуляризации. Аугментацию можно разрабатывать с учетом потенциальных дрейфов в вашей области — для изображений, например, можно имитировать различные погодные условия.
- *Использование более мощной предварительно обученной модели в качестве инициализатора*: в глубоком обучении кастомная модель дает лучшие результаты, если первоначальная модель была обучена на более крупном и разнообразном датасете и, как следствие, обладает лучшей обобщающей способностью.

Один из подходов к работе с изменениями в распределении данных — *выявление и изоляция сегментов данных с низкой эффективностью*. Это может быть особенно полезно, если изменение не носит универсального характера и затрагивает только определенную часть данных.

Можно начать с анализа признаков, в которых произошли изменения, и выявить потенциальные корреляции с эффективностью модели. Если наблюдается сдвиг в распределении признака (например, местоположения), можно попробовать отфильтровать данные по этому признаку, чтобы проверить, не он ли является причиной снижения эффективности.

После того как вы выявили сегменты данных с низкой эффективностью, можно решать, что с ними делать. Один из вариантов — перенаправить обработку прогнозов для этих сегментов, полагаясь либо на эвристики, либо на ручную корректировку результатов. Также можно выделить данные сегменты и отложить принятие мер до тех пор, пока не будет собрано достаточное количество новых размеченных данных для обновления модели.

Кроме того, обработка выбросов в отдельном рабочем процессе также может помочь снизить количество ошибок при возникновении дрейфа данных. Выбросы — это отдельные точки данных, которые значительно отличаются от остальной выборки. Их отдельная обработка позволяет обеспечить стабильную работу модели в условиях изменений.

Другим способом борьбы с дрейфом данных является *добавление дополнительной бизнес-логики поверх модели* — за счет корректировки прогнозов модели или изменения логики приложения. Такой подход эффективен, однако его сложно обобщить, и при некорректной реализации он может привести к нежелательным последствиям.

Хорошим примером такого подхода является корректировка результатов вручную, особенно актуальная для задач прогнозирования спроса. Бизнес-правила, связанные с определенными товарами, категориями и регионами, могут использоваться для корректировки прогноза модели с учетом акций, маркетинговых кампаний и известных событий. В случае дрейфа данных можно применить новую корректировку к выходу модели, чтобы учесть произошедшие изменения.

Другой пример — установка нового порога принятия решения в проблемах классификации. Выход модели — это часто вероятность, и поэтому можно настроить порог, при котором метка будет присваиваться в соответствии с требуемым уровнем уверенности. При обнаружении дрейфа данных порог можно изменить с учетом новой информации.

Также существует и гибридный подход, сочетающий методы ML и не ML-решения, который подходит для ограниченных данных или если требуется выполнить прогноз в динамически изменяющейся среде, где взаимосвязи между переменными постоянно меняются. В некоторых случаях подход без ML может быть более устойчивым и простым в обслуживании, поскольку он не зависит от закономерностей в данных и может основываться на причинно-следственных связях или экспертных знаниях. Однако подобные решения менее гибкие и хуже адаптируются к изменяющимся условиям по сравнению с ML-моделями. Необходимо тщательно взвесить все плюсы и минусы и выбрать наиболее подходящее решение для конкретной задачи (см. раздел 13.4).

Дрейф концепции

При обнаружении дрейфа концепции существует несколько подходов к повторному обучению модели, включая использование всех доступных данных, присвоение более высоких весов новым данным и исключение старых данных, если новых уже достаточно. В некоторых случаях простого повторного обучения (retraining) модели может быть недостаточно, и может потребоваться ее настройка (fine-tuning), а также тестирование новых признаков, архитектур или источников данных, если новые закономерности слишком сложны для захвата текущей моделью.

Повторное обучение модели может означать либо полное обучение заново в случае небольшой модели, либо тонкую настройку более крупной модели в зависимости от степени выраженности дрейфа и доступного вычислительного бюджета.

Возможно, вам потребуется изменить охват модели или бизнес-процесс (например, сократить горизонт прогнозирования, отложить генерацию прогноза, чтобы собрать больше данных или увеличить частоту запуска модели). При таком подходе важно поддерживать контакт с владельцами бизнес-процессов и другими пользователями системы, чтобы все были готовы к изменениям и могли с ними справиться.

Отдельное внимание следует уделять периодическим дрейфам или сезонности. Они могут быть вызваны множеством факторов, например ростом

покупательской активности в черную пятницу, изменением схемы передвижений в праздничные дни или периодом получения зарплаты (в конце месяца). Постарайтесь учесть подобные периодические изменения или используйте несколько моделей, чтобы их компенсировать. Это позволит избежать снижения эффективности модели, поскольку такие предсказуемые изменения (периодический дрейф) ожидаемы и поддаются учету.

Для учета сезонности в ML-системе необходимо обучить модель распознавать и реагировать на подобные изменения. Например, если модель используется для прогнозирования продаж домашней одежды, она должна уметь распознавать и предполагать рост спроса по выходным. В таком случае модель сможет корректно прогнозировать увеличение покупательской активности в выходные дни. Такое предсказуемое изменение не требует генерации оповещения, поскольку это регулярное событие.

Фреймворк для обнаружения дрейфа

При работе с дрейфом первым шагом является определение возможных действий, после чего следует разработать фреймворк для отслеживания дрейфа, чтобы мониторить потенциальные изменения. Это позволит задать степень изменений, при которой будет срабатывать оповещение, и определить, как на него реагировать.

Для эффективного проектирования такого фреймворка следует учитывать возможные изменения как в модели, так и в реальном процессе, лежащем в основе данных. В зависимости от потребностей и ограничений модели могут применяться различные подходы к обнаружению и обработке дрейфа.

Например, если качество модели можно оценить напрямую, используя своевременно полученные истинные метки, дрейф распределения можно проигнорировать. Вместо этого лучше сосредоточиться на выявлении и устранении проблем, связанных с некорректными входными данными, с помощью проверок валидации данных на основе правил.

С другой стороны, если модель используется в критически важном приложении, где наблюдается задержка с получением эталонных и интерпретируемых признаков, вам потребуется более комплексная система обнаружения дрейфа. Подобная система может включать в себя панель мониторинга дрейфа данных и набор статистических тестов, позволяющих выявлять изменения в распределении данных и корреляциях между признаками. Если известны ключевые признаки модели и их бизнес-ценность, можно назначить им различные веса и сосредоточиться на отслеживании дрейфа именно в данных признаках или же рассматривать совокупный дрейф нескольких слабых признаков, при этом увеличив порог чувствительности к дрейфу, чтобы избежать ложных срабатываний.

В итоге оптимальный подход к обнаружению дрейфа будет зависеть от конкретных требований и ограничений модели и может включать в себя комбинацию различных методов и метрик. Поэтому необходимо учитывать вероятность

ложных срабатываний и последствия отказа модели и разрабатывать систему, способную эффективно обнаруживать и обрабатывать дрейф с минимальными перебоями и простоями.

Бизнес-KPI

Мониторинг бизнес-показателей (KPI) может быть затруднен из-за сложности изолирования влияния ML-системы и проблем при измерении отдельных метрик. В таких случаях важно находить прокси-показатели или интерпретируемые проверки, способные дать представление о работе ML-системы. Также стоит отметить, что отслеживание бизнес-показателей не всегда обеспечивает необходимый контекст в случае деградации модели. В подобных ситуациях может потребоваться исследование и других факторов, таких как сама модель, данные и ПО, чтобы определить первопричину проблем.

Тем не менее существует причина, по которой бизнес-KPI располагаются на вершине пирамиды (см. рис. 14.5). Мониторинг бизнес-метрик и ключевых показателей эффективности имеет решающее значение для понимания бизнес-ценности ML-системы. Отслеживание таких метрик, как выручка и коэффициенты конверсии (например, если модель используется для привлечения клиентов), позволяет определить, выполняет ли система поставленные цели и оказывает ли она положительное влияние на бизнес.

Также важно вовлекать в процесс мониторинга как специалистов по данным, так и бизнес-стейкхолдеров, чтобы убедиться, что ML-система отвечает всем требованиям. Рекомендуем отслеживать как абсолютные, так и относительные значения, чтобы получить более полное представление о влиянии системы.

И самое главное: какими бы технически совершенными ни были выбранные вами метрики для вашей ML-системы, они всего лишь результат постепенной декомпозиции бизнес-KPI. Именно по этой причине эффективность и устойчивость любой системы прямо или косвенно влияют на успех компании-работодателя или вашего собственного бизнеса.

14.5. Дизайн-документ: мониторинг

Методы мониторинга ML-системы могут различаться в зависимости от ее целей, характеристик и архитектуры. Ниже представлены два дизайн-документа, демонстрирующие различные подходы к мониторингу, поскольку проектируемые в обоих кейсах системы имеют принципиальные различия и особенности.

14.5.1. Мониторинг для Supermegaretail

Ранее в этой главе мы рассмотрели особенности мониторинга систем прогнозирования, а теперь перейдем к практической части, посвященной мониторингу модели для Supermegaretail.

ДИЗАЙН-ДОКУМЕНТ: SUPERMEGARETAIL

XI. МОНИТОРИНГ

1. Анализ существующей инфраструктуры

К сожалению, прогнозирование спроса является одним из первых ML-проектов для Supermegaretail, что означает отсутствие полноценной инфраструктуры мониторинга. К счастью, предварительные исследования оказались плодотворными, и мы нашли Evidently AI — библиотеку с открытым исходным кодом на Python (https://github.com/evidentlyai/evidently), предназначенную для мониторинга. Лозунг проекта: «Мы создаем инструменты для оценки, тестирования и мониторинга ML-моделей вместо вас» — отлично соответствует нашим целям до тех пор, пока мы не примем решение о создании собственной платформы (см. раздел 3.2). Согласно описанию, Evidently AI охватывает такие аспекты, как качество модели, дрейф данных, дрейф целевой переменной и качество данных. Следовательно, нам все же потребуется создать некоторые базовые компоненты, чтобы применить данное решение.

2. Логирование

Мы будем хранить логи прогнозов модели в колоночной системе управления базами данных. Для каждого прогноза необходимо сохранять входные признаки, выход модели и временны́е метки. Мы планируем использовать систему с открытым исходным кодом ClickHouse, поскольку она уже применяется в компании для аналогичных задач.

Кроме того, мы будем вести логи базовых статистик: количество запросов в секунду, потребление ресурсов, частота ошибок, задержки на уровнях p90, p99, p999, количество вызовов модели в час, день и неделю, а также средние, медианные, минимальные и максимальные значения прогнозов модели на аналогичных уровнях агрегации. Для этого мы будем использовать связку Kafka + Prometheus + Grafana и хранить данные за последний месяц. Данный стек также будет использоваться для мониторинга и визуализации ML-системы в режиме реального времени (https://mng.bz/ZVOR).

3. Качество данных

В дополнение к базовым операциям извлечения, преобразования и загрузки данных, а также проверкам их качества мы будем отслеживать следующие параметры:

- *Отсутствующие данные*, в процентах от всего датасета, а также отдельно — в процентах от наиболее значимых признаков согласно важности признаков (см. раздел 11.2). Мы будем использовать исторические данные (исключая случаи с нарушенной передачей данных) для расчета z-оценки. Порог срабатывания системы оповещений составит три z-оценки для важных признаков и четыре — для остальных. Кроме того, мы воспользуемся несколькими преднастроенными наборами тестов из библиотеки Evidently AI, в частности пресетом для проверки качества и устойчивости данных.

- *Соответствие схеме*. Все ли признаки присутствуют? Соответствуют ли их типы ожидаемым? Появились ли новые столбцы?
- *Диапазоны и статистики признаков*. Чтобы обеспечить подачу в обучающую модель данных надлежащего качества, мы вручную определим ожидаемые диапазоны для каждого важного признака, ориентируясь на три z-оценки и проводя проверки на недопустимые значения (например, отрицательные значения объема продаж, минимальное значение должно быть ≥ 0).
- *Корреляции*. Для выявления аномалий в данных мы построим корреляционную матрицу признаков и сравним разницу между двумя графиками. Порог срабатывания сигнала — остатки, превышающие $|0.15|$.

4. Качество модели

Нам повезло — истинные метки доступны с минимальной задержкой. Мы ежедневно получаем информацию о продажах через 15 минут после их совершения. Учитывая это, мы будем отслеживать метрики по квантилям 1.5, 25, 50, 75, 95 и 99 — как в исходном виде, так и с весами, соответствующими цене уникального складского идентификатора товара (SKU). Также мы будем мониторить среднеквадратичную ошибку и среднюю абсолютную ошибку для отслеживания среднего и медианного значений. Окончательные пороговые значения будут установлены после получения данных за первые три месяца, первоначальные пороги мы зададим на основе исторических данных и качества модели на валидации.

Кроме того, мы настроим оповещения на отрицательные значения и предельно допустимые значения. Система оповещений будет срабатывать, если новое максимальное значение превышает ранее зафиксированное на 50 %.

Мы настроим мониторинг дрейфа прогнозов, который будет использоваться в качестве раннего сигнала для прогнозов на следующий день, неделю и месяц. Мы протестируем два подхода: индекс стабильности популяции > 0.2 и расстояние Вассерштейна > 0.1. Для расстояния Вассерштейна мы применим коэффициент роста к контрольному датасету. Например, сравнивая апрель 2021 года с апрелем 2022 года и зная, что ожидается общий прирост на 15 %, мы умножаем все значения за 2021 год на 1.15. Мы дополнительно скорректируем данный коэффициент после экспериментов с историческими данными.

5. Дрейф данных

Поскольку реальные данные у нас появляются с очень небольшой задержкой, нет необходимости отслеживать дрейф входных данных как прокси для оценки актуальности модели. Тем не менее это может помочь заранее выявить возможные изменения, прежде чем они повлияют на качество модели. После прочтения статьи под названием «Which Test Is the Best? We Compared 5 Methods to Detect Data Drift on Large Datasets» (https://mng.bz/2g5g) мы решили использовать расстояние Вассерштейна в качестве сигнала при возникновении дрейфа данных. В качестве начального порога используется среднее значение

метрики дрейфа. В дальнейшем мы можем попробовать применить подход, изложенный в статье «Feature Shift Detection: Localizing Which Features Have Shifted via Conditional Distribution Tests» (https://mng.bz/RNQZ).

6. Бизнес-метрики

Актуальные бизнес-метрики остаются теми же, что описаны в разделе 2.1.1: выручка (ожидается рост), уровень запасов (ожидается снижение или без изменений) и маржа (ожидается рост). Данные показатели мы будем отслеживать с помощью серии A/B-тестов, периодически меняя и чередуя контрольные группы.

14.5.2. Мониторинг для PhotoStock Inc.

Набор мероприятий по мониторингу системы для PhotoStock Inc. будет отличаться от предыдущего примера, так как в данном случае речь идет о проектировании модели для «умной» поисковой системы по стоковым фотографиям.

ДИЗАЙН-ДОКУМЕНТ: PHOTOSTOCK INC.

XI. МОНИТОРИНГ

1. Работоспособность ПО

Необходимо убедиться, что ведутся следующие логи:

- Исходный поисковый запрос.
- ID документов-кандидатов.
- Топ-20 документов в итоговом ранжировании.

Кроме того, должны логироваться все негативные сценарии и фоллбэки.

Логи должны быть отслеживаемы по трем ID:

- *user_id* — может быть пустым, так как не все пользователи авторизованы.
- *session_id* — формируется в бэкенде и может содержать цепочку запросов.
- *request_id* — объединяет группу логов, относящихся к одному поисковому запросу.

В качестве хранилища логов будет использоваться постоянный поставщик компании (AWS Cloudwatch).

Необходимо собирать метрики, связанные с задержками, поскольку они оказывают критическое влияние на пользовательский опыт, и некоторые компоненты могут быть относительно медленными. Примерный (неполный) список метрик:

- Время на получение кандидатов.
- Время итогового ранжирования.
- Время применения переопределений.

Метрики также будут отправляться в AWS Cloudwatch.

2. Состояние данных

Учитывая источник данных, мы можем быть абсолютно уверены в качестве изображений (фотографии тщательно проверяются нашей внутренней командой модерации, а также с помощью их ИИ-инструментов), но мы менее уверены в поисковых запросах (поступающих от пользователей) и оценках ранжирования (полученных с помощью краудсорсинга).

Поисковые запросы следует фильтровать, чтобы уменьшить количество полностью некорректных данных (например, случайный набор символов, если по клавиатуре прошла кошка). Однако при этом возможны и «глупые» запросы, которые должны присутствовать в обучающей выборке, так как это реальный пользовательский ввод. Слишком строгая фильтрация может привести к перекосу между обучением и эксплуатацией (training-serving skew).

Метки ранжирования, используемые для обучения модели, являются нашим основным активом. Они должны быть максимально точными, и мы должны инвестировать в их качество совместно с поставщиком платформы для разметки, включая перекрестные проверки и контрольные задания для выявления недобросовестных разметчиков. Дополнительно можно ввести алгоритмическую валидацию размеченных данных с помощью базовых моделей, например, с использованием API GPT для проверки соответствия описания изображения и тегов поисковому запросу либо через мультимодальные модели, такие как LLAVA (https://llava-vl.github.io). В будущем можно рассмотреть вариант использования модели в качестве основного источника меток, а людей — в качестве валидаторов, в зависимости от точности меток, полученных от обеих сторон.

3. Работоспособность модели

Насколько нам известно, задача поиска по изображениям не склонна к дрейфу. Тем не менее могут происходить определенные изменения как в предпочтениях пользователей, так и в самих изображениях, размещенных на сайте: могут появляться новые темы, новые типы изображений и т. д. Мы занимаемся исключительно ранжированием, поэтому сложно представить себе какой-то особо критичный сбой (в худшем случае мы можем откатиться на предыдущую версию поисковой системы без ML).

Следовательно, нет необходимости включать в первую версию системы какой-либо мониторинг состояния модели при условии, что качество модели обеспечено на этапах валидации и тестирования. Однако в текущей поисковой системе уже используются определенные метрики мониторинга: средняя позиция клика, средняя позиция первого клика и т. п. В целях безопасности мы будем сохранять эти метрики. В дальнейшем для подтверждения концепции (proof of concept) можно рассмотреть использование решения с открытым кодом, например Deepchecks (https://deepchecks.com/), если мы обнаружим, что проблема дрейфа для нас действительно актуальна.

Итоги

- Даже самые хорошо обученные и точные модели могут со временем деградировать без должного мониторинга. Это может быть вызвано изменениями во входных данных, архитектуре модели или даже выходных данных модели.
- Основа надежного мониторинга — это ПО-бэкенд, поверх которого последовательно выстраиваются: контроль качества и целостности данных, оценка качества и релевантности модели, а также отслеживание бизнес-метрик.
- Необходимо мониторить работоспособность программной части системы, так как это критически важно для эффективного выполнения задач и оперативного реагирования на запросы. Снижение стабильности и производительности ПО может негативно повлиять на общую эффективность ML-системы.
- Многие практики мониторинга, используемые в традиционном ПО, такие как мониторинг приложений и инфраструктуры, система оповещений и управление инцидентами, могут применяться и к ML-системам.
- Мониторинг качества данных позволяет выявить потенциальные проблемы на этапе обработки данных (от сбоев доступа до изменений формата данных) или порчу источников данных, которая может возникнуть из-за ошибок, аппаратных неисправностей или проблем с внешними API.
- В каскадных моделях необходимо работать с цепочкой взаимосвязанных моделей, причем выходные данные одной модели могут привести к некорректным прогнозам другой, что приведет к сбою всей системы.
- Некоторые модели могут годами работать без обновлений, в то время как другие требуют ежедневного повторного обучения на свежих данных. Необходимо регулярно отслеживать ключевые метрики эффективности и сравнивать их с историческими значениями, чтобы избежать деградации модели или выявить ее еще на ранней стадии.
- Дрейф данных возникает, когда распределение переменных в датасете существенно отличается от того, на котором обучалась модель. Для борьбы с дрейфом данных может потребоваться повторное обучение модели на новых данных или создание новой модели для соответствующего сегмента данных.
- Дрейф концепции происходит, когда закономерности, выученные моделью, больше неактуальны, даже если распределения признаков остались прежними. Это может привести к снижению точности модели или полной ее неработоспособности.
- При обнаружении дрейфа данных есть два варианта действий: либо мониторить эффективность модели и отслеживать изменения таких метрик, как доля верных результатов (accuracy), средняя ошибка или уровень мошенничества (fraud rate), либо включать в модель дополнительные данные или признаки, чтобы точнее отражать изменившиеся закономерности в данных.
- При возникновении дрейфа концепции может потребоваться повторное обучение модели, включая весь доступный датасет, с назначением более высоких весов новым данным или полным исключением старых данных, если новые достаточно репрезентативны.

15 Сервинг и оптимизация инференса

В этой главе

- ✓ Сложности, которые могут возникнуть на этапах сервинга и инференса
- ✓ Полезные инструменты и фреймворки
- ✓ Оптимизация пайплайнов инференса

Запуск модели машинного обучения в эксплуатацию — один из завершающих этапов эффективного жизненного цикла системы. Некоторые специалисты в области ML проявляют недостаточной интерес к данной стадии работы, предпочитая сосредоточиться на разработке и обучении моделей. Однако это неправильно, поскольку модель будет полезна только при условии ее развертывания и эффективного использования в продакшен-среде. В этой главе мы обсудим сложности, связанные с развертыванием и сервингом ML-моделей, а также рассмотрим различные способы оптимизации инференса.

Как мы уже писали в главе 10, пайплайн инференса — это последовательность (в большинстве случаев) либо более сложный ациклический граф шагов, который принимает на вход необработанные данные и на выходе выдает прогнозы. Вместе с самой ML-моделью пайплайн инференса включает такие этапы, как вычисление признаков, предварительная обработка данных, постобработка вывода и др. Предобработка и постобработка — это довольно обобщенные термины, поскольку их реализация может варьироваться в зависимости от конкретных требований системы. Приведем несколько примеров. Типичный пайплайн

компьютерного зрения обычно начинается с изменения размера изображения и его нормализации, типичный пайплайн обработки естественного языка — с токенизации, а типичный пайплайн системы рекомендаций — с извлечения пользовательских признаков из хранилища.

Роль грамотно настроенного инференса также может различаться в зависимости от области применения. Компании, занимающиеся высокочастотным трейдингом или рекламными технологиями, постоянно находятся в поиске лучших специалистов, чтобы усовершенствовать свои системы с крайне низкой задержкой. Разработчики мобильных приложений и решений для интернета вещей (IoT) отдают приоритет эффективному инференсу, стремясь снизить энергопотребление устройства и, следовательно, улучшить пользовательский опыт. Компании, использующие высоконагруженные бэкенды в своих продуктах, заинтересованы в том, чтобы поддерживать нагрузку без чрезмерных затрат на инфраструктуру. В то же время существует много сценариев, когда прогнозы модели не являются делом первостепенной срочности. Например, вам раз в неделю нужно выполнить пакетную задачу для прогноза продаж на следующую неделю, сформировать отчет и отправить его в отдел закупок. Если отчет должен оказаться на корпоративной почте к утру понедельника, не так важно, займет его формирование 10 минут или 3 часа — главное, чтобы оно происходило в воскресенье вечером.

Эта глава в основном сосредоточена на системах, основанных на глубоком обучении. И это неудивительно, ведь тяжеловесные модели требуют больше инженерных усилий перед их развертыванием в продакшен-среде, тогда как обслуживать относительно легкие решения, такие как логистическая регрессия или небольшой ансамбль деревьев, не так уж и сложно, при условии наличия инфраструктуры признаков (описанной в главе 11). В то же время некоторые принципы и методы, рассмотренные здесь, применимы к любой ML-системе.

15.1. Сервинг и инференс: сложности

Как и всегда при проектировании систем, первым шагом является определение требований. Существует несколько ключевых факторов, которые необходимо учитывать, и далее мы кратко рассмотрим каждый из них.

- *Задержка* (latency) определяет наши ожидания относительно того, насколько быстро система должна предоставлять прогнозы. Для приложений, требующих немедленного отклика, задержка измеряется в миллисекундах (в крайних случаях даже 1 мс может быть слишком долго!), тогда как в других сценариях ожидание в течение часов или даже дней вполне допустимо.
- *Пропускная способность* (throughput) указывает на количество задач или объем данных, который система способна обработать за определенный промежуток времени. В контексте ML это означает общее число прогнозов, которое модель может выдать за единицу времени. Оптимизация пропускной способности особенно важна, когда необходимо обработать большие объемы

данных за короткий срок, как, например, при пакетной обработке крупных датасетов.

- *Масштабируемость* (scalability): важно понимать, с каким количеством прогнозов может столкнуться система и как это число может изменяться. Паттерны нагрузки часто носят сезонный характер и могут варьироваться в зависимости от отрасли. В розничной торговле, например, наблюдается всплеск продаж в праздничные сезоны или в дни массовых скидок, такие как черная пятница или киберпонедельник. В рекламных технологиях нагрузка может быть выше из-за увеличения активности в интернете. Некоторые всплески, как указано выше, можно спрогнозировать, но другие могут возникнуть внезапно: вирусная популярность приложения или неожиданный наплыв пользователей от крупного клиента — это то, к чему сложно подготовиться заранее. Система должна обладать достаточной масштабируемостью, чтобы справиться с пиковыми нагрузками без ущерба для пропускной способности и с минимальной задержкой.
- *Целевые платформы* (target platforms): модели работают на серверах только с CPU или с ускорением на GPU, в бессерверных средах, таких как AWS Lambda или Cloudflare Workers, на ПК, мобильных устройствах или устройствах IoT или же прямо в браузере. Каждая из этих платформ наряду с преимуществами имеет собственные ограничения и требования, которые необходимо тщательно изучить до начала создания системы. Если мы разрабатываем мобильное приложение, необходимо учитывать размер модели: она должна быть достаточно компактной, чтобы войти в пакет приложения, и при этом достаточно эффективной, чтобы работать на устройстве пользователя без чрезмерного расхода заряда батареи. Если мы разрабатываем серверную систему, мы получаем бóльшую свободу в выборе аппаратного обеспечения, что предоставляет широкие возможности в плане сложности и размера системы. Если целевая платформа — это специализированное IoT-устройство, возможно, нам не удастся реализовать сложную модель с множеством функций, и в таком случае использование простейших архитектур становится основным техническим требованием.

  Ситуация усложняется, если продукт должен работать на нескольких платформах. В некоторых случаях можно запускать небольшие батчи на устройствах пользователей и отправлять остальное на бэкенд либо проводить донастройку на сервере перед передачей модели на устройство пользователя для инференса. Таким образом, соблюдаются совокупные требования обеих платформ. Кроме того, иногда приходится использовать различные вычислительные единицы даже в рамках одной платформы. Арсений однажды столкнулся с необходимостью ускорить систему, работающую на устройствах с недорогими GPU. Внутри система использовала небольшую модель для обработки нескольких параллельных запросов, что приводило к перегрузке этих GPU. Решением стало внедрение пула сессий на разных типах устройств: каждый раз, когда GPU перегружался, следующий запрос обрабатывался

на CPU, что позволило более равномерно распределить нагрузку между устройствами и соответствовать требованиям по задержке.

- *Стоимость*: ML-системы зачастую являются одними из самых ресурсоемких решений в компании, и с ростом популярности тяжеловесных генеративных моделей вопрос стоимости выходит на первый план. По мере того как затраты на ML-инфраструктуру приобретают все большее значение, бизнесу приходится искать разумные подходы к организации пайплайнов инференса при сохранении финансовой выгоды. Важно заранее понимать стоимость инфраструктуры и то, как она будет масштабироваться при увеличении нагрузки. В некоторых случаях затраты на инфраструктуру могут даже превысить доход, генерируемый системой. В других случаях это не играет роли, например, если инференс выполняется на устройстве пользователя. Для мобильных приложений или IoT-устройств рост пользовательской базы практически не повлияет на стоимость инфраструктуры (хотя полностью исключить ее влияние нельзя: если пользователи загружают модели с вашей сети, каждые новые 1000 пользователей обойдутся в дополнительные центы).
- *Надежность* (reliability) может оказаться под угрозой, если в выборе платформы инференса был сделан упор исключительно на дешевизну. Выбор поставщика самого недорогого аппаратного обеспечения, которое внезапно выходит из строя в разгар сезона пиковых нагрузок, может обернуться бóльшими убытками, чем инвестиции в проверенное, надежное решение. Следует учитывать все возможные сбои: внезапный всплеск нагрузки, ошибку в модели или аппаратный сбой — и главное, подумать о том, как система будет справляться с подобными ситуациями (и будет ли справляться вообще).
- *Гибкость* (flexibility): даже если новая система демонстрирует стабильную производительность и эффективность, нельзя быть уверенными, что будущие требования и идеи не потребуют каких-либо изменений. Поэтому система должна быть достаточно гибкой, чтобы адаптироваться к новым условиям и улучшениям. Это может включать в себя новую модель (возможно, обученную с использованием другого фреймворка), дополнительную предобработку или постобработку, новые признаки, новые API и т. д. Всегда имейте в виду, что система будет развиваться, и она должна быть легко модифицируемой без нарушения существующего функционала.
- *Безопасность и конфиденциальность*: эту тему сложно уложить в рамки одного абзаца; упомянем лишь, что требования безопасности в значительной степени зависят от целевой платформы. Например, если система полностью работает на бэкенде и прогнозы никогда не покидают периметр компании, то особых усилий по обеспечению безопасности, помимо текущих протоколов, не требуется. Однако если вы разрабатываете мобильное приложение, работающее на устройстве пользователя, необходимо уделить большое внимание защите модели от реверс-инжиниринга. В некоторых случаях модель должна быть защищена даже от самих пользователей — ярким примером является взлом крупных языковых моделей (LLM), многократно случавшийся

в 2023 году, когда пользователи пытались обмануть модели, чтобы получить ответы на чувствительные вопросы.

15.2. Компромиссы и паттерны

Перечисленные факторы часто противоречат друг другу или даже взаимоисключают. По этой причине невозможно оптимизировать их все одновременно, поэтому вам придется искать разумные компромиссы. Тем не менее существуют определенные паттерны (или шаблоны), на которые можно опираться при поиске справедливого баланса в различных сценариях.

15.2.1. Компромиссы

Начнем с *задержки* и *пропускной способности*. Эти два параметра являются наиболее частыми объектами оптимизации, и улучшения в одном могут как положительно, так и отрицательно сказаться на другом.

Реальные продакшен-системы часто оптимизируются для достижения максимальной пропускной способности при заданном времени задержки. Время задержки определяется требованиями продукта или пользовательским опытом (ожидают ли пользователи отклик в реальном либо почти реальном времени или допустимы задержки), и при заданном времени задержки задача состоит в том, чтобы максимизировать пропускную способность (или минимизировать количество требуемых серверов при заданной ожидаемой пропускной способности) путем изменения архитектуры модели или схемы инференса (например, с использованием пакетной обработки, о чем будет рассказано далее в разделе 15.3).

Давайте рассмотрим несколько примеров. Представим простую модель глубокого обучения — сверточную сеть вроде ResNet или трансформер, такой как BERT. Если просто уменьшить количество блоков, с высокой вероятностью и задержка, и пропускная способность улучшатся вне зависимости от используемой схемы инференса, так что тут все довольно очевидно (точность модели может снизиться, но в данном примере это не рассматривается). Однако представьте две модели с одинаковым числом блоков и одинаковым количеством параметров в каждом блоке, но с разной архитектурой: модель A запускает блоки параллельно с последующей агрегацией (подобно архитектуре ResNeXt), а модель B выполняет блоки последовательно. Модель B будет иметь бóльшую задержку по сравнению с моделью A из-за последовательного характера своей архитектуры, однако ее можно запускать в нескольких экземплярах параллельно или обрабатывать большие пакеты данных, так что пропускная способность модели A не будет превосходить модель B. Таким образом, параллелизм — это один из внутренних факторов, по-разному влияющих на задержку и пропускную способность. На практике это означает, что количество параметров или блоков не может служить окончательным критерием сравнения моделей, и именно понимание архитектуры в контексте используемой платформы инференса помогает выявить проблемные места (рис. 15.1).

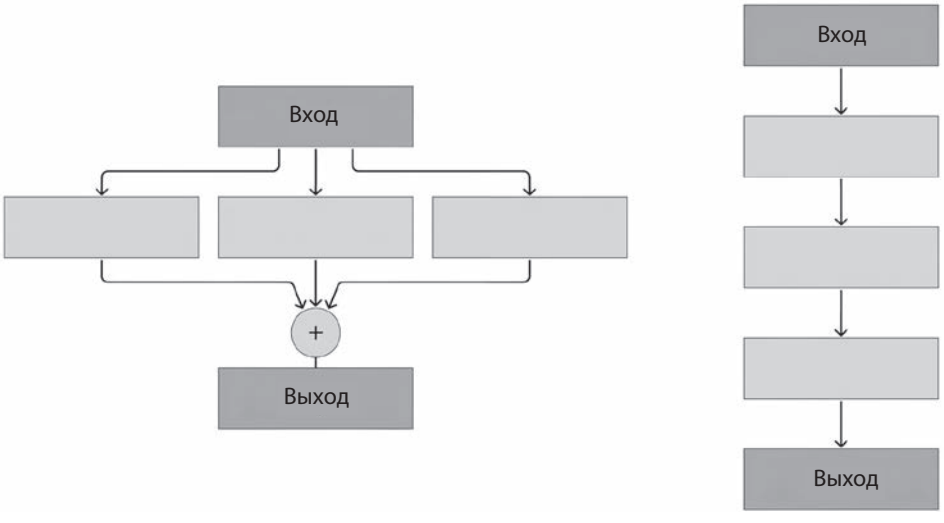


Рис. 15.1. Пример сравнения «широких» и «глубоких» моделей. Несмотря на одинаковое число параметров, глубокая модель имеет больше ограничений с точки зрения параллельных вычислений

Еще один компромисс связан с точностью (accuracy) модели (или другими метриками, специфичными для задач ML) и соответствующими показателями задержки, пропускной способности и стоимости. ML-инженеры нередко хотят устранить недостатки модели, просто увеличив ее размер. Иногда им это удается, но такой подход дорог и плохо масштабируется. В то же время чрезмерное стремление минимизировать затраты, выбирая предельно простую модель, также не является удачным решением. Зависимость между вычислительными затратами и ценностью модели нелинейна, и в большинстве случаев существует оптимальная точка, в которой модель достаточно хороша, а затраты относительно невелики. Крайности по обе стороны спектра, как правило, менее эффективны. Здесь мы можем использовать идею из раздела 9.1 и построить график зависимости точности модели от вычислительных затрат, чтобы найти подходящий баланс под конкретную задачу (рис. 15.2).

Наконец, существует компромисс не столь очевидный, но все равно заслуживающий внимания — это баланс между гибкостью исследований и эффективностью системы при эсплуатации. С одной стороны, удобно иметь модель, которую легко перенести из экспериментальной среды напрямую в продакшен-среду. С другой стороны, четкое разделение среды исследований и продакшен позволяет команде свободно экспериментировать с новыми идеями, не рискуя стабильностью рабочей системы. Однако чем дальше расходятся эти две среды, тем выше риск возникновения дефектов, вызванных различиями в окружении, которые сложно отследить без соответствующих инвестиций в системы мониторинга и интеграционные тесты (см. главу 13 о развертывании и главу 14 о наблюдаемости системы).

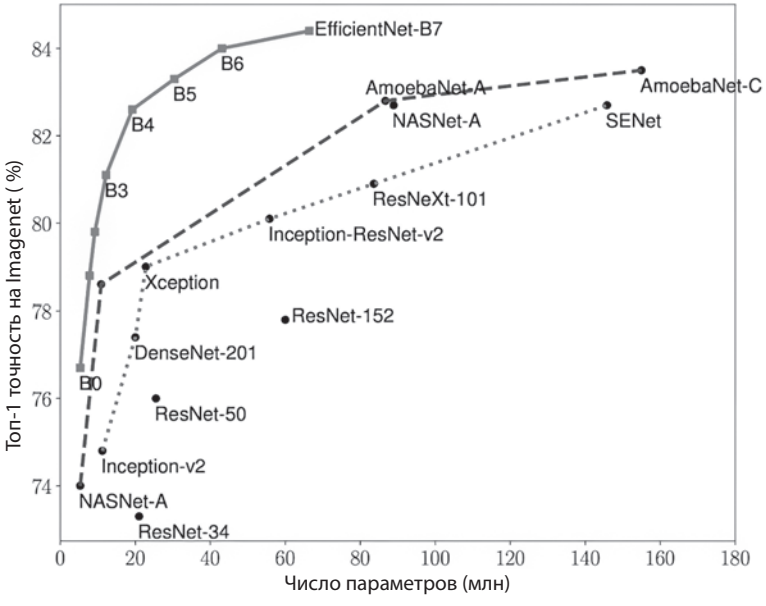


Рис. 15.2. Модели разного размера из одного и того же семейства и соответствующие значения точности (accuracy) на датасете Imagenet. График демонстрирует, что более крупные модели, как правило, обеспечивают более высокую точность, но данный эффект имеет тенденцию к насыщению (источник: https://mng.bz/QVzG)

Существуют определенные инструменты и фреймворки, которые стоит учитывать при выборе одного из этих путей. Поскольку принятое решение будет напрямую влиять на устойчивость всей системы, следующий раздел полностью посвящен этому крайне важному вопросу.

15.2.2. Паттерны

После того как мы определили, какими характеристиками нужно пожертвовать ради оптимизации модели, наступает момент выбора подходящего паттерна реализации. Мы снова ограничимся коротким списком и рассмотрим три разных паттерна. Хотя существует и множество других вариантов, скорее всего, в своей системе вы воспользуетесь одним из приведенных нами.

Первый паттерн, который стоит упомянуть, — это *пакетная обработка* (или *батчинг*), техника, направленная на повышение пропускной способности и требующая предварительного планирования. Если известно, что система будет работать в пакетном режиме, ее следует проектировать с учетом этой особенности. Батчинг — это не просто бинарная характеристика (использовать или не использовать), она имеет множество нюансов. Представим типичный API для веб-страницы: пользователь, на которого мы не можем повлиять, отправляет

данные, а сервер возвращает прогноз. Это не классическая офлайн-пакетная обработка, но здесь возможно применение динамической пакетной обработки: на стороне сервера мы ждем короткий интервал (например, 20 мс или, скажем, до 32 запросов — в зависимости от того, что наступит раньше), собираем все поступившие за это время запросы и выполняем пакетный инференс. Таким образом удается снизить задержку для пользователя, при этом сохраняя отзывчивость системы. Данный подход реализован в таких фреймворках, как, например, Triton Inference Server от Nvidia и TensorFlow Serving. Другой пример пакетной обработки можно найти в языковых моделях: у них размер входных данных динамический, и выполнение пакетного инференса на входах разной длины требует дополнения до размера самого длинного ввода. Однако можно группировать входные данные по размеру и выполнять инференс по батчам для каждой группы отдельно, что снижает издержки на дополнение (ценой уменьшения размера батчей или увеличения времени их накопления). Подробнее об этой технике можно прочитать в блоге Graphcore (https://mng.bz/XVEv).

Второй паттерн, о котором мы хотим рассказать, — это *кэширование*, заслуженно считающееся высшей формой оптимизации (оптимизация инференса будет обсуждаться далее в главе). Использование кэша основано на очевидной идее: никогда не вычисляй одно и то же дважды. Иногда кэш реализуется так же просто, как и в традиционном ПО, — входные данные ассоциируются с ключом, и в хранилище типа «ключ — значение» сохраняются ранее вычисленные результаты, чтобы не запускать дорогостоящие вычисления повторно.

Листинг 15.1. Пример простейшего возможного кэша в оперативной памяти

```
class InMemoryCache:
    def __init__(self):
        self.cache = {}
    def get(self, key: str, or_else: Callable):
        v = self.cache.get(key)
        if v is None:
            v = or_else()
            self.cache[key] = v
        return v
```

На практике распределение входных данных для ML-модели может иметь «длинный хвост», поскольку большинство запросов являются уникальными. Например, по данным Google, 15 % всех поисковых запросов абсолютно уникальны (https://mng.bz/yoeB). Учитывая, что срок жизни кэша обычно значительно короче всей истории Google, доля уникальных (не поддающихся кэшированию) запросов будет высокой в пределах любого разумного временно́го окна. Означает ли это, что кэш бесполезен? Вовсе нет. Во-первых, даже небольшой процент сэкономленных вычислений может дать значительную выгоду с точки зрения затрат. Во-вторых, существует концепция так называемого нечеткого кэширования — вместо точного совпадения ключа можно ослабить условия соответствия. Например, Арсений

однажды работал с системой, где ключ кэша основывался на регулярных выражениях, благодаря чему результат можно было использовать повторно для нескольких похожих запросов, совпадающих с одним и тем же регулярным выражнием. Еще более «агрессивный» подход к кэшированию предполагает использование семантической близости ключей (например, https://github.com/zilliztech/GPTCache применяет такой подход для кэширования запросов к LLM). Как практики, которые ценят надежность, мы рекомендуем хорошо подумать, прежде чем внедрять такие методы кэширования: уровень неточности здесь высок, и, как следствие, кэш становится подвержен ложным попаданиям.

Третий паттерн, который недавно появился в области LLM, — это *маршрутизация между моделями*. Вдохновленный архитектурой Mixture of Experts («смесь экспертов») (https://mng.bz/M1qW), он зарекомендовал себя как эффективный шаг в сторону оптимизации: некоторые запросы сложны (и должны обрабатываться продвинутыми, но ресурсоемкими моделями), другие проще (и могут быть переданы менее сложным, дешевым моделям). Такой подход в основном используется для универсальных LLM, машинного перевода и других задач, в основном относящихся к обработке естественного языка. Пример реализации можно найти в статье «Leeroo Orchestrator: Elevating LLMs Performance Through Model» Алирезы Мохаммадшахи (Alireza Mohammadshahi) и др. (https://arxiv.org/abs/2401.13979v1).

Вариацией данного паттерна является *использование двух моделей* (одна — быстрая, но неточная, другая — медленная, но более точная). При таком подходе первая легкая модель сначала выдает быстрый ответ (с низкой задержкой), который затем заменяется результатом работы более тяжелой и точной модели. В отличие от предыдущего паттерна, он не сокращает общие вычисления, но оптимизирует специфический тип задержки — время до получения первого ответа.

15.3. Инструменты и фреймворки

Процесс инференса в значительной степени сфокусирован на инженерных задачах. К счастью, существует множество доступных инструментов и фреймворков. В этой книге мы не ставим цель дать исчерпывающий обзор всех возможных решений для построения надежного пайплайна инференса, а сосредоточиваемся лишь на некоторых принципах и приводим наиболее популярные решения в качестве примеров.

15.3.1. Выбор фреймворка

Один из распространенных, хотя и неочевидных приемов — это разделение фреймворков для обучения и инференса. Обычно используются такие инструменты, как Pandas, scikit-learn и Keras, поскольку они обеспечивают гибкость и простоту на этапах исследования, прототипирования или обучения. Однако они не являются наилучшим решением для инференса из-за неизбежного

компромисса между гибкостью и эффективностью. Поэтому распространенной практикой является обучение модели в одном фреймворке с последующим преобразованием для инференса в другой. Кроме того, важно максимально разделить фреймворки обучения и инференса, чтобы при необходимости перехода к другому обучающему фреймворку с новыми возможностями это не влияло на пайплайн инференса. Это особенно важно для тех систем, которые должны функционировать и развиваться на протяжении многих лет.

С другой стороны, некоторые ориентированные на исследование фреймворки, такие как Torch, стремятся сократить разрыв между экспериментальным и продакшен-кодом. Функция компиляции, представленная в Torch 2.0, позволяет создавать достаточно оптимизированный пайплайн инференса на основе того же кода, который использовался для обучения и экспериментов. Таким образом, вне зависимости от того, хотите вы использовать универсальный фреймворк или комбинацию двух (или даже более) решений для различных целей, обе парадигмы являются жизнеспособными — в зависимости от выбранного вами подхода (на рис. 15.3 представлены наиболее популярные фреймворки).



Рис. 15.3. Широкий спектр инструментов с ориентацией либо на исследования, либо на продакшен

Достижение баланса между гибкостью исследований и высокой эффективностью в продакшене может потребовать взаимодействия между фреймворками. Популярным выбором для этой цели является ONNX, поддерживаемый многими фреймворками обучения для конвертации своих моделей в формат ONNX. С другой стороны, фреймворки инференса часто работают с форматом ONNX или позволяют конвертировать ONNX-модели в собственный формат, что делает ONNX своеобразным «общим языком» в мире ML.

Здесь стоит упомянуть, что ONNX — это не просто формат представления, а целая экосистема. Она включает в себя среду выполнения, которую можно использовать для инференса, и набор инструментов для преобразования и оптимизации моделей. ONNX Runtime хорошо подходит для различных типов бэкенд-сред и может работать практически на любой платформе с применением специфических оптимизаций для каждой из них. Это делает его оптимальным вариантом

для многоплатформенных систем. По нашему опыту, ONNX Runtime позволяет обеспечить баланс между гибкостью (хотя и меньшей, чем при прямом обслуживании моделей PyTorch или scikit-learn) и производительностью (хотя и уступающей специально подобранному решению, оптимизированному под конкретное сочетание моделей, аппаратного обеспечения и сценариев использования).

Для инференса на серверных CPU популярным движком является OpenVINO от Intel. Для CUDA-инференса часто используется TensorRT от Nvidia. Оба движка оптимизированы под свою целевую аппаратную платформу и тоже доступны как бэкенд-модули для ONNX Runtime, что позволяет использовать их так же, как и сам ONNX Runtime. Еще два заслуживающих упоминания инструмента — это TVM (https://tvm.apache.org/), компилятор моделей глубокого обучения, способный генерировать код для различных типов оборудования, и AITemplate (https://github.com/facebookincubator/AITemplate), который можно использовать даже для запуска моделей на менее распространенных GPU от AMD с помощью программного стека ROCm.

Разработчики, знакомые с развертыванием моделей на iOS, могут знать о CoreML — движке инференса для iOS, использующем собственный формат. Разработчики под Android, как правило, выбирают TensorFlow Lite, хотя благодаря большей фрагментации платформы Android доступно больше опций. Этот список далеко не полный, но он дает представление о широте доступных решений.

Когда мы говорим что-то вроде «движок X работает на устройстве Y», это не совсем точное выражение. Например, даже если предполагается, что весь инференс будет выполняться на GPU, некоторые операции могут передаваться на CPU, поскольку это обеспечивает большую эффективность. GPU превосходно справляются с задачами массово-параллельных вычислений, такими как умножение матриц или свертки. Однако некоторые операции, связанные с управляющим потоком или разреженными данными, лучше обрабатываются на CPU. Например, CoreML динамически разделяет граф исполнения между CPU, GPU и Apple Neural Engine, чтобы максимизировать эффективность.

Различные движки инференса часто поставляются с оптимизаторами, которые можно использовать для повышения производительности модели. Например, ONNX Runtime предоставляет набор оптимизаторов, способных уменьшить размер модели путем удаления неиспользуемых частей графа (например, тех, что использовались только на этапе обучения для вычисления функции потерь) или за счет сокращения количества операций после слияния нескольких операций в одну. Такие оптимизаторы, как правило, являются отдельными инструментами, предназначенными для подготовки модели к инференсу, и не входят в сам движок инференса.

На момент публикации книги данный раздел может уже устареть, поскольку эта область стремительно развивается. Например, когда мы начали писать книгу, лишь немногие работали с большими языковыми моделями (LLM), а теперь они повсеместны. Часто обсуждается, какой движок инференса лучше подходит

для них: VLLM (https://github.com/vllm-project/vllm), TGI (https://mng.bz/aVG7) или GGML (https://github.com/ggerganov/ggml). Инференс LLM, в принципе, представляет собой отдельную тему: в отличие от большинства традиционных моделей, LLM ограничены большим объемом памяти и авторегрессионной парадигмой (нельзя спрогнозировать токен T + 1 до прогноза токена T, что делает параллелизацию практически невозможной без дополнительных ухищрений). Если вы интересуетесь инференсом LLM, рекомендуем ознакомиться с записью в блоге по адресу https://vgel.me/posts/faster-inference/ и с более значительными исследованиями (например, на момент редактирования этой главы нас впечатлила статья «PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU» от Сонг (Song) и др. (https://arxiv.org/abs/2312.12456).

И хотя термины *«движок инференса»* (inference engine) и *«фреймворк инференса»* (inference framework) часто используются как взаимозаменяемые, они не являются полностью идентичными. Движок инференса — это среда выполнения, используемая для инференса, в то время как фреймворк инференса — более общий термин, который может включать в себя движок, набор инструментов для конвертации и оптимизации моделей, а также другие компоненты, помогающие с различными аспектами обслуживания, такими как батчинг, версионирование, реестр моделей, логирование и т. д. Например, ONNX Runtime — это движок инференса, а TorchServe (https://pytorch.org/serve/) — это фреймворк.

Не существует универсального ответа на вопрос, нужен ли вам полнофункциональный фреймворк или достаточно небольшой обертки поверх движка инференса. По нашему опыту, если требования достаточно определены и вы, как правило, работаете со сложной системой, при этом обладая достаточными человеческими ресурсами для ее сопровождения, лучше выбирать фреймворк. С другой стороны, если вы работаете в стартапе и нужно выпустить систему как можно быстрее, но вы знаете, что окончательное понимание требований появится только через несколько месяцев, разумно выбрать более легкий путь — развернуть модель с использованием простого сочетания движка инференса и коммуникационного слоя (например, веб-фреймворка), отложив реализацию более надежного решения до следующей версии.

15.3.2. Бессерверный инференс

Бессерверный инференс — это новая концепция, отличающаяся от традиционной модели, предполагающей использование серверов. Она была популяризирована AWS Lambda и в настоящее время представлена в ряде альтернатив от крупных облачных провайдеров, таких как Google Cloud Functions, Azure Functions и Cloudflare Workers, а также от стартапов, например Banana.dev и Replicate. На момент написания этого раздела ведущие провайдеры в основном предлагают инференс на CPU с ограниченной поддержкой GPU, однако ситуация может измениться в связи с тем, что стартапы продолжают развиваться в этой области.

Более продвинутые продукты крупных облачных провайдеров, такие как задания AWS Sagemaker, также можно рассматривать как бессерверные. Они обладают основными характеристиками бессерверных решений (управляемая инфраструктура, оплата по факту использования, автомасштабирование), но предполагают иной уровень абстракции: медленные функции и более контейнеризованные задачи, работающие в течение продолжительного времени.

Важно понимать, что термин *бессерверный* (serverless) может несколько вводить в заблуждение. Он не означает, что в системе отсутствуют серверы. Скорее это означает, что инженеры не имеют прямого доступа к ним, поскольку облачные провайдеры изолируют серверы, и, таким образом, инженерам не нужно заботиться об управлении ими.

Отношение к бессерверному инференсу, как правило, либо резко положительное, либо резко отрицательное из-за его заметных преимуществ и недостатков. Ниже перечислим некоторые из плюсов и минусов.

- *Отсутствие необходимости управлять инфраструктурой или платить за простаивающие ресурсы* — вы платите только за фактическое использование. И хотя сторонники бессерверных решений постоянно акцентируют на этом внимание, на практике все же может потребоваться некоторое управление инфраструктурой, пусть и на более высоком уровне и менее сложное.
- *Простота масштабирования, особенно при непредсказуемой или скачкообразной нагрузке*, — но это не универсальное решение. Облачные провайдеры могут накладывать ограничения на количество одновременных запросов, что препятствует бесконечному и мгновенному масштабированию. Кроме того, существует проблема «холодного старта», при которой инициализация крупной модели может занимать несколько секунд, что критично для приложений с требованиями к низкой задержке. Некоторые бессерверные провайдеры, такие как Runpod, предоставляют больше контроля в данном аспекте, позволяя задавать минимальное и максимальное количество рабочих процессов и настраивать правила масштабирования. Холодный старт считается серьезной проблемой в бессерверных вычислениях, поэтому провайдеры разрабатывают специальные решения, например такие как SnapStart от AWS (https://mng.bz/gAWV) и Flashboot от Runpod (https://mng.bz/eV9Q). При использовании бессерверного инференса время холодного старта становится важным фактором при выборе движка инференса, поскольку следует стремиться к облегченным артефактам (например, Docker-образам) и минимальному времени загрузки.
- *Экономическая эффективность при низкой нагрузке* — тем не менее для умеренно низкой, но стабильной и предсказуемой нагрузки выделенные серверы могут оказаться более выгодными с точки зрения стоимости, чем бессерверные решения. Соотношение цены и задержки также не совсем понятно. Например, если модель обрабатывает запрос за 100 мс в «теплом» состоянии и за 5000 мс в «холодном» (то есть сразу после перерыва) и цена вычисляется на основе времени обработки, стоимость холодного запроса

будет в 50 раз выше. Оптимизировать подобные сценарии непросто. Частным случаем низкой нагрузки являются различные тестовые среды: удобно избегать затрат, характерных для традиционной архитектуры, и можно платить только за редкие тестовые запросы в своей стейдж-среде.

- *Сложности с локальным тестированием* — общая сложность инфраструктуры разработки, как правило, увеличивается при использовании бессерверного инференса, несмотря на его потенциальную дешевизну. Причем проблема не ограничивается только лишь «я не могу протестировать систему без подключения к интернету». Если бессерверный инференс становится неотъемлемой частью вашей системы, он может принести дополнительные трудности (например, необходимость повторной загрузки тестовых артефактов при незначительных изменениях, обеспечение корректных прав доступа в тестовой среде и т. д.).

Некоторые бессерверные провайдеры, например Replicate, предлагают широкий спектр заранее обученных базовых моделей, доступных к использованию, что особенно полезно при запуске новых проектов, особенно на стадии прототипирования или проведения исследований.

Мы были свидетелями как успешных, так и неудачных случаев применения бессерверного инференса — и в небольших любительских проектах, и в высоконагруженных продакшен-средах. Безусловно, это неплохой подход, но перед его внедрением необходимо тщательно оценить целесообразность и провести анализ соотношения затрат и выгод. Наша основная рекомендация такова: рассматривайте возможность использования бессерверного инференса, если ключевым преимуществом является автомасштабирование (например, при высокой изменчивости количества запросов), а модель сама по себе не слишком велика (хотя даже LLM иногда можно развернуть в бессерверной среде только на CPU (https://mng.bz/pxnz)). Также стоит обратить внимание на бессерверный инференс в случае неопределенной будущей нагрузки — такой подход может подстроиться под различные сценарии на начальных этапах проекта, давая вам время на осмысление архитектуры инференса.

15.4. Оптимизация пайплайнов инференса

Оптимизация пайплайнов инференса — довольно широкая тема, которая формально не относится к проектированию ML-систем как таковому; тем не менее она представляет собой критически важный аспект инженерии ML-систем, заслуживающий отдельного рассмотрения — как минимум для обзора наиболее распространенных подходов и инструментов. В своей основе оптимизация пайплайнов инференса часто сводится к балансу между скоростью и точностью модели и требуемыми ресурсами, что предполагает использование множества оптимизационных техник, в значительной степени зависящих от характеристик и архитектуры модели.

Разумный вопрос, который может здесь возникнуть: «С чего начать оптимизацию?». Мы задавали подобный вопрос множеству ML-инженеров во время собеседований и получали самые разные ответы, в которых упоминались такие понятия, как *обрезка модели*, *квантизация* (см. «Pruning and Quantization for Deep Neural Network Acceleration: A Survey» (https://arxiv.org/abs/2101.09671)) и *дистилляция* (см. «Knowledge Distillation: A Survey» (https://arxiv.org/abs/2006.05525)) с отсылками к соответствующим научным публикациям (мы упомянули лишь пару статей, которые могут стать вашей отправной точкой в изучении данной темы). Приведенные выше методы широко известны в исследовательском сообществе ML, они полезны и часто применимы, но сосредоточены исключительно на оптимизации модели, что не позволяет контролировать всю картину в целом. Самым прагматичным ответом на наш вопрос был такой: «Я бы начал с профилирования».

15.4.1. Начинаем с профилирования

Профилирование — это процесс измерения производительности системы и выявления потенциальных проблем. В отличие от ранее упомянутых техник, профилирование — более общий подход, который может быть применен ко всей системе. Подобно тому как стратегия предшествует тактике, профилирование — отличная отправная точка, позволяющая определить наиболее перспективные направления оптимизации и выбрать подходящие методы.

Вы удивитесь, что некоторые кажущиеся очевидными факторы на самом деле не являются уязвимыми местами вашей модели. Возьмем, к примеру, задержку. Существуют случаи, когда она вовсе не представляет никакой проблемы (особенно если обслуживаемая модель — это не новейшая генеративная система, а что-то более традиционное), и вам лучше обратить внимание на другие моменты, например на предобработку данных или сетевые взаимодействия. Более того, даже если модель действительно является самым медленным звеном пайплайна, это еще не означает, что именно ее следует оптимизировать. Может показаться нелогичным, но мы должны искать не самую медленную часть системы, а наиболее поддающуюся оптимизации. Представим, что полный запуск занимает 200 мс, из которых 120 мс приходится на модель. Но модель уже оптимизирована: она работает на GPU с использованием высокопроизводительного движка, и возможностей для дальнейшего улучшения практически нет. С другой стороны, предобработка данных занимает 80 мс и реализована в виде произвольного Python-кода, который можно оптимизировать множеством способов. В таком случае разумнее начать именно с предобработки данных, а не с модели.

Другой пример из практики Арсения: однажды его попросили уменьшить задержку в системе. Это был относительно простой пайплайн, последовательно запускающий десятки простых моделей. Первая идея была — заменить последовательное выполнение на пакетный инференс. Однако профилирование показало противоположное: инференс сам по себе занимал лишь около 5 % общего времени, тогда как основное время уходило на предобработку данных,

и батчинг здесь бы не помог. Что действительно помогло, это оптимизация этапа предобработки, которая оказалась связана с изначально запланированным последовательным запуском. В результате Арсению удалось ускорить систему на 40 % без изменения основного инференса модели, так как гораздо легче было оптимизировать такие элементы, как управление потоками, операции ввода-вывода, (де)сериализация и каскадное кэширование.

Здесь стоит отметить, что профилирование ML-систем немного отличается от профилирования обычного ПО по таким причинам, как широкое использование GPU, асинхронное выполнение, наличие тонких Python-оберток над высокопроизводительным нативным кодом и т. д. Поскольку весь процесс может включать в себя большее число переменных, при интерпретации результатов профилирования следует соблюдать осторожность — легко запутаться из-за коварной природы таких систем.

Особенно сложными могут оказаться вычисления на GPU. Типичная нагрузка на CPU достаточно проста: данные загружаются в память, и CPU исполняет код — готово. Конечно, существует ряд нюансов, связанных с кэшем CPU, одиночным потоком команд и множественным потоком данных (SIMD) или параллельным исполнением, но в целом все довольно прозрачно. GPU же — это отдельное устройство, как правило, с собственной встроенной памятью, и перед выполнением данные необходимо в эту память скопировать. Сам процесс копирования может вызывать сложности, так как зачастую его сложно измерить. Высокая степень параллелизма вычислений на GPU также приводит к нелинейным эффектам. Простейший пример: если запустить модель на одном изображении, это может занять 100 мс, но запуск на 64 изображениях увеличит время лишь до 200 мс. Это происходит потому, что при обработке одного изображения GPU не используется на полную мощность, и значительное количество времени уходит на копирование данных в память. То же самое происходит на более низком уровне архитектуры модели: уменьшение числа фильтров в сверточном слое не обязательно сокращает задержку, поскольку GPU может быть все так же недозагружен и использовать одно и то же CUDA-ядро. В целом программирование под CUDA и другие фреймворки для общих вычислений на GPU — это отдельная и крайне глубокая тема. Единственный вывод, на котором мы хотим акцентировать внимание: интуиция обычного программиста по поводу того, что работает быстро, а что — медленно, не поможет с вычислениями на GPU.

Таким образом, правильный подход к профилированию требует наличия широкого набора инструментов, начиная с базовых профайлеров типа cProfile из стандартной библиотеки Python, до более продвинутых сторонних утилит, таких как Scalene (https://github.com/plasma-umass/scalene), memray (https://github.com/bloomberg/memray), py-spy (https://github.com/benfred/py-spy), специализированных инструментов для ML-фреймворков (например PyTorch Profiler), и заканчивая низкоуровневыми GPU-профайлерами, например nvprof (https://mng.bz/OmqE). Наконец, при работе с нестандартным оборудованием, таким как тензорные

процессоры или IoT-чипы, может потребоваться использование инструментов, предоставляемых их производителем.

После интерпретации результатов профилирования можно приступать к оптимизации системы, устраняя самые заметные проблемы. Наиболее распространенные подходы по различным уровням:

- *оптимизация, связанная с моделью*: изменение архитектуры, обрезка модели, квантизация, дистилляция, отбор признаков и т. д.;
- *оптимизация сервинга*: пакетная обработка, кэширование, предварительные вычисления и т. д.;
- *оптимизация кода*: более эффективные низкоуровневые алгоритмы, переход к более эффективным парадигмам (например, векторизованные алгоритмы вместо циклов `for`), переписывание кода с использованием более быстрых библиотек/фреймворков (например, numba для численных вычислений) или даже других языков программирования (например, вместо Python — более быстрый оберточный модуль на C++ или на Rust);
- *аппаратная оптимизация*: использование более производительного оборудования, вертикальное/горизонтальное масштабирование (см. главу 13) и т. д.

15.4.2. Минимальная оптимизация — наилучшая оптимизация

Если отвлечься от уровня эксплуатации системы и рассмотреть оптимизацию с точки зрения общего проектирования, можно избежать ряда проблем на этапе сопровождения системы, если на этапе проектирования она была разработана с учетом исходных требований. Если известны строгие ограничения по задержке, необходимо изначально выбирать модель, которая достаточно быстра на целевой платформе. Конечно, можно уменьшить потребление памяти за счет квантизации или обрезания модели либо сократить задержку с помощью дистилляции, но лучше изначально начинать с модели, близкой к целевым требованиям, чем пытаться ускорять ее постфактум, когда система уже готова. По нашему опыту, срочная необходимость в оптимизации чаще всего возникает из-за неверных решений на ранних этапах жизненного цикла системы (например, использования тяжелых моделей в качестве базового варианта) в случае неожиданного успеха (например, стартап создал быстрый прототип, и внезапно возникла необходимость в масштабировании) или в рамках запланированного технического долга («Созданный нами продукт неоптимальный, но быстрый, и если он “выстрелит” и поможет нам выйти на рынок, мы потом все подчистим»).

Выбор уровня оптимизации — это ключевое решение для эффективного инференса. Архитектор-основатель стартапа, которому нужно масштабировать минимально жизнеспособный продукт до обслуживания второго клиента, скорее всего, просто арендовал бы дополнительные вычислительные ресурсы в облаке. С другой стороны, инженер платформы в крупной технологической компании, вероятно, нашел бы решение в «Algorithms for Modern Hardware» (https://en.algorithmica.org/hpc/) и применил бы ряд низкоуровневых оптимизаций в масштабе.

15.5. Дизайн-документ: сервинг и инференс

Отдельный раздел дизайн-документа, посвященный оптимизации инференса, должен описывать предполагаемые действия на этапе сопровождения системы. Рассмотрим общие и различающиеся черты оптимизации инференса для двух разных ML-систем.

15.5.1. Сервинг и инференс для Supermegaretail

Основываясь на ключевых особенностях и требованиях ML-систем, ориентированных на ритейл, решение для Supermegaretail не требует обработки в режиме реального времени, что позволяет модифицировать модель пакетно, но при этом предполагает значительный объем работ.

ДИЗАЙН-ДОКУМЕНТ: SUPERMEGARETAIL

XII. СЕРВИНГ И ИНФЕРЕНС

Основные аспекты сервинга и инференса:

- Эффективная пакетная обработка, поскольку прогнозы будут осуществляться ежедневно, еженедельно и ежемесячно на больших объемах данных.
- Безопасность конфиденциальных данных о запасах и продажах.
- Экономичная архитектура, способная масштабировать пакетные задания.
- Мониторинг качества данных и прогнозов.

1. Архитектура сервинга

Пакетные задания по прогнозированию спроса будут запускаться в контейнерах Docker, управляемых с помощью AWS Batch на машинах EC2. AWS Batch позволит задать требования к ресурсам, динамически масштабировать необходимое количество контейнеров и ставить в очередь крупные задания.

Пакетные задания будут запускаться по расписанию: обрабатывать входные данные из S3, выполнять инференс и сохранять результаты обратно в S3. Простой API на Flask позволит по требованию запускать пакетный инференс.

Вся передача и обработка данных будет происходить внутри защищенной инфраструктуры AWS, изолированной от внешнего доступа. Аутентификация и авторизация будет осуществляться с использованием надлежащих учетных данных.

2. Инфраструктура

Пакетные серверы будут использовать группы автоматического масштабирования для соответствия нагрузке. Для гибких пакетных заданий можно использовать спот-инстансы, чтобы снизить затраты.

На данном этапе не требуется специализированное оборудование или оптимизация, так как приоритетом является пропускная способность пакетной

обработки, а ее характер допускает широкую параллелизацию. Мы воспользуемся возможностями горизонтального масштабирования, предоставляемыми AWS Batch и S3.

3. Мониторинг

Ключевые метрики для отслеживания при выполнении пакетных заданий:

- Уровень успешного выполнения, продолжительность и частота сбоев.
- Количество обработанных строк на задание.
- Использование серверов: CPU, память, пространство на диске.
- Точность прогнозов по сравнению с фактическим спросом.
- Проверка данных и система оповещений.

Такой мониторинг позволит обеспечить эффективность и масштабируемость пакетной обработки, а также высокое качество прогнозов. На основе данных, полученных при эксплуатации, в будущем можно будет оценить необходимость в оптимизации.

15.5.2. Сервинг и инференс для PhotoStock Inc.

Оптимизация поисковой системы включает два основных компонента:

- Обработка пользовательских запросов в режиме реального времени, при этом общее количество пользователей подвержено сезонным колебаниям (например, разница между дневной и ночной активностью) и возможным резким всплескам.
- Индекс, используемый для поиска фотографий, который должен регулярно обновляться. Аналогично Supermegaretail, в данном случае не требуется обработка в режиме реального времени, однако необходимо обрабатывать большие объемы данных.

ДИЗАЙН-ДОКУМЕНТ: PHOTOSTOCK INC.

XII. СЕРВИНГ И ИНФЕРЕНС

Поскольку наша поисковая система основана на векторном сходстве, необходимо учитывать два момента: генерацию векторов для объектов, подлежащих поиску (обновление индекса), и выполнение поиска по пользовательским запросам (запрос к индексу). Два этих аспекта имеют разные требования и ограничения, поэтому они будут спроектированы отдельно.

1. Обновление индекса

Обновление индекса — это пакетный процесс, происходящий один раз в день (см. главу 13). Помимо регулярных обновлений, необходимо поддерживать начальное создание индекса или его полную пересборку в случае обновления

основной модели. Хотя такие случаи встречаются редко, важно иметь возможность запускать данный процесс по требованию.

Обе задачи имеют схожие характеристики:

- Умеренные требования к задержке.
- Жесткие требования к пропускной способности.

Необходимо обеспечить возможность обработки большого количества элементов за разумное время и по разумной стоимости. Для грубых оценок необходимо поддерживать переиндексацию порядка ~10^5 объектов в день в течение нескольких часов. Также нужно обеспечить возможность переиндексации порядка ~10^8 объектов в разумные сроки при обновлении основной модели.

2. Запрос к индексу

Запрос к индексу — это процесс, выполняемый в режиме реального времени и инициируемый каждым пользовательским запросом. Требуется минимизировать задержку запроса, чтобы требования к пропускной способности не были чрезмерно высокими: среднее количество запросов составляет 150 000 в день (см. главу 12), что соответствует примерно двум запросам в секунду. Однако количество запросов распределено неравномерно, поэтому необходимо обеспечивать обработку пиковых нагрузок до ~100 запросов в секунду, а также обеспечивать быструю масштабируемость при всплесках трафика и возвращение к обычной нагрузке.

Мы предлагаем использовать одну и ту же модель, конвертированную в ONNX, и для пакетной обработки, и для инференса в режиме реального времени. Это не является обязательным, но упростит проектирование и сопровождение системы. Однако процесс инференса различается для пакетной обработки и реального времени, поэтому эти два режима должны быть разделены в соответствии с варьирующимися требованиями.

3. Фреймворк и аппаратное обеспечение

С программной точки зрения мы будем использовать Nvidia Triton Inference Server в качестве фреймворка для сервинга. Это высокопроизводительное ПО с открытым исходным кодом для инференса, поддерживающее ONNX и обладающее множеством функций, упрощающих процесс сервинга. Для связи с ним из нашего приложения мы будем использовать HTTP API сервера Triton. Мы будем использовать одну и ту же модель как для пакетной обработки, так и для инференса в режиме реального времени, но для второго случая будет применяться конфигурация, оптимизированная по задержке (например, параметр `max_queue_delay_microseconds` в динамическом пакетировании должен быть менее 10 мс).

Для пакетного инференса мы будем использовать стандартное решение от нашего облачного провайдера — AWS Sagemaker. Это управляемый сервис, который позволяет запускать пакетный инференс на настраиваемых инстансах, при необходимости легко масштабируется и интегрирован с другими ис-

пользуемыми нами сервисами AWS. Мы также рассматриваем возможность использования спот-инстансов в фоновом режиме для снижения стоимости пакетного инференса. Само пакетное задание будет представлять собой простой скрипт поверх инференса в режиме реального времени с добавлением слоя ввода-вывода, считывающего данные из очереди и записывающего результаты как в S3, так и в базу данных.

Для инференса в режиме реального времени желательно иметь бессерверное решение, способное масштабироваться до нуля при отсутствии запросов. Однако, учитывая высокую нагрузку и требования к низкой задержке, этого сложно достичь с использованием основных провайдеров, таких как AWS Lambda, поэтому мы выберем более традиционный подход с пулом серверов за балансировщиком нагрузки. Для серверов мы будем использовать экземпляры AWS EC2, а для балансировки — AWS Application Load Balancer. Мы можем использовать спот-инстансы в фоновом режиме для снижения стоимости инференса в режиме реального времени, поскольку каждый воркер будет без состояния (stateless), и его можно легко заменить новым в случае завершения AWS-инстанса. Необходимо предусмотреть, чтобы в системе было гарантированное количество доступных воркеров, и предусмотреть возможность дополнительного масштабирования при необходимости.

Точная конфигурация оборудования как для пакетной обработки, так и для задач в режиме реального времени будет определяться в рамках последующих экспериментов. Очевидно, что в обоих случаях необходимо использовать GPU-инстансы, однако конкретный тип GPU и прочие ресурсы пока не определены. Мы не ожидаем высокой загрузки CPU, поскольку предварительная обработка относительно проста, но нужно убедиться в наличии достаточного количества CPU и избежать их замедления, для чего необходим тщательный мониторинг использования ресурсов (CPU/RAM/GPU). Точное количество инстансов под балансировщиком также будет уточняться в ходе экспериментов.

4. Вспомогательная инфраструктура

Мы начнем сервинг модели с точностью float32 по умолчанию, но в дальнейшем проведем эксперименты с пониженной точностью (например, float16) для снижения затрат на сервинг. Кроме того, позже может быть осуществлена оптимизация задержки модели, хотя на настоящий момент мы не ожидаем проблем в этой области, так как модель CLIP достаточно проста.

Поскольку одни запросы встречаются чаще других, мы можем использовать кэш для снижения нагрузки на серверы инференса. Для этого мы можем применить AWS ElastiCache — управляемый сервис, поддерживающий Redis и Memcached. Мы реализуем простой кэш «ключ — значение» с коротким временем жизни (конкретное значение определяется на основе анализа данных). Кэширование актуально для инференса в режиме реального времени, но не для пакетной обработки, отвечающей за обновление кэша в случае изменения ключа.

Необходимо убедиться, что система масштабируема и способна справляться с пиковыми нагрузками. Данный элемент архитектуры следует доработать совместно с командой технического обеспечения сайта (SRE) и экспертами AWS. На начальном этапе мы хотим удостовериться, что автоскейлинг включен для инференса в режиме реального времени, а все соответствующие метрики (например, потребление ресурсов, количество запросов, количество активных инстансов) отслеживаются и для них настроена система оповещений.

Дополнительные меры безопасности для сервинга не требуются, поскольку он не взаимодействует с интернетом и доступен только из нашего приложения. Необходимо предусмотреть, чтобы доступ к сервингу был разрешен исключительно нашему приложению и только к тем ресурсам, которые действительно необходимы (так, например, сервинг не должен иметь доступа к базе данных).

Итоги

- Хотя может возникнуть соблазн ограничиться только разработкой и обучением системы, оптимизация инференса — это не менее важный этап, который обеспечит стабильную работу системы на этапе ее эксплуатации.
- Существуют ключевые факторы, влияющие на то, как вы будете проектировать процесс оптимизации инференса вашей системы. Среди них: задержка, пропускная способность, масштабируемость, целевые платформы, стоимость, надежность, гибкость, безопасность и конфиденциальность.
- Перечисленные факторы могут противоречить друг другу или даже быть взаимоисключающими. Следовательно, невозможно оптимизировать все одновременно, и в любом случае придется идти на компромиссы, чтобы настроить модель наилучшим образом.
- Не забывайте обучать модель в одном фреймворке, а затем преобразовывать ее для инференса в другом. Такой шаг особенно важен, поскольку не затрагивает сам пайплайн инференса, когда в дальнейшем возникнет необходимость смены фреймворка ради новых возможностей.
- Одна из основных целей — достичь баланса между гибкостью при исследовании и высокой эффективностью в продакшене. Однако следует помнить, что для этого потребуется промежуточный слой между фреймворками.
- Самый важный момент заключается в том, что лучшая оптимизация — это минимальная оптимизация. Некоторые проблемы, возникающие на этапе сопровождения, можно предотвратить, если система была корректно спроектирована в соответствии с изначальными требованиями.

16 Владение и сопровождение

В этой главе

- ✓ Важность надлежащего сопровождения системы
- ✓ Ответственность как ключевой фактор построения и сопровождения надежной ML-системы
- ✓ Понятие «фактора автобуса» и компромисс между эффективностью команд и избыточностью
- ✓ Значение правильно организованной документации
- ✓ Обманчивая привлекательность сложности

> Компетентный программист полностью осознает строго ограниченные размеры своего черепа, поэтому подходит к задачам программирования со всей возможной скромностью и избегает заумных трюков, как чумы.
>
> *Эдсгер Дейкстра*

На протяжении предыдущих 15 глав мы старались выстроить нашу книгу таким образом, чтобы после ее прочтения у вас сформировался подробный чек-лист, к которому вы смогли бы обращаться на любом этапе проектирования вашей системы машинного обучения. Но на деле следовать всем приведенным в книге и, казалось бы, очевидным рекомендациям бывает гораздо сложнее, чем кажется.

Создание ML-системы с нуля или же использование уже существующего решения — это настолько сложный и всеобъемлющий процесс, что в нем неизбежны ошибки, промахи и отклонения от принципов ради достижения бизнес-результатов. Да и мы сами, конечно, не идеальны и порой можем игнорировать собственные же советы.

На практике система может не следовать принципам, изложенным в этой книге. Причины для этого могут быть самые разные: что-то могли упустить, у компании могли измениться приоритеты и ограничения, система действительно нуждается в обновлении, поскольку ее изначальные предположения больше не актуальны, и т. д.

Но зачастую все куда прозаичнее: люди, спроектировавшие и создавшие какую-то систему, со временем меняют работу. Более того, они могут покинуть компанию даже до завершения разработки системы — такова реальность. Учитывая это, владение отдельными компонентами системы и их сопровождение — это не то, о чем можно подумать когда-нибудь потом, это основа всей архитектуры, которую необходимо закладывать с самого начала. В этом процессе, однако, есть риск излишней привязанности к своему «детищу», чего следует избегать. Система не должна восприниматься как часть ее автора, и порой необходимо подавить свое эго ради наилучшего решения.

Если свести все вышесказанное к трем пунктам, то в любой момент времени необходимо понимать:

- Каковы зоны ответственности каждого участника проекта? Есть ли пробелы в распределении ответственности/владения?
- Сколько человек обладают узкоспециализированными знаниями? Достаточно ли у нас избыточности?
- Где хранится документация и какие компоненты системы задокументированы? Сможет ли кто-либо вне ответственной команды запустить систему?

В некоторой степени дизайн-документ помогает ответить на эти вопросы (хотя и частично, и не всегда напрямую). А это означает только одно: необходимо сосредоточить свое внимание на этих моментах, чтобы избежать печальных последствий в будущем. В конечном счете самое важное — это создать и поддерживать продукт, который будет соответствовать вашим требованиям и приносить ценность.

В этой главе мы рассмотрим основные принципы, обеспечивающие долговечность системы и ее устойчивость к изменениям, вызванным человеческим фактором.

16.1. Ответственность

В предыдущих главах мы упоминали важность привлечения стейкхолдеров к процессу сбора информации, чтобы те изложили свое понимание проекта еще на подготовительном этапе разработки ML-системы.

По мере развития проекта, его обогащения новыми данными и перехода от простой базовой версии к серии тесно взаимосвязанных сложных моделей в него будут постепенно вовлекаться новые участники. Некоторые из них присоединятся к вам лишь на короткое время и покинут проект, как только их участие станет неактуальным (например, разметчики данных необходимы на этапе подготовки датасета, но уже не нужны на этапе развертывания), другие будут постоянными контрибьюторами до самого релиза системы, и лишь немногие со временем окажутся настоящим «ядром» команды — вашими ближайшими коллегами по проекту. Такие специалисты (или представители сторонних подрядчиков, если речь идет о внешних участниках) станут ответственными за конечный результат и стабильность вашей системы. И вы должны регулярно поддерживать с ними контакт — по крайней мере, знать их лично.

Понимание зон ответственности и закрепленных за ними людей имеет решающее значение для успешного запуска и будущей стабильной работы любого проекта или системы. Это может показаться абсурдным, но еще важнее, чтобы сами члены команды, на которых возложена ответственность за конкретные компоненты, это осознавали. Вы будете удивлены, узнав, сколько раз человек А был уверен, что за определенную часть проекта отвечает человек В, в то время как человек В был уверен в обратном. Классический пример такой ситуации изображен на рис. 16.1.



Рис. 16.1. Явный и неявный подходы к вовлечению участников в проект

Лучше проговорить то, что и так всем известно, чем надеяться, что все понимают друг друга без слов. Исходя из нашего личного опыта, такой подход применим уже при наличии трех человек в проекте — не говоря уже о больших командах. Одна из наших любимых цитат на эту тему принадлежит лауреату Нобелевской премии по физике Ричарду Фейнману. В своей книге «Вы, конечно, шутите, мистер Фейнман!» он писал[1]:

[1] Цитируется по переводу с английского Н. А. Зубченко, О. Л. Тиходеевой, М. Шифмана. — *Примеч. ред.*

Одно из самых интересных событий во время работы над проектом в Принстоне — встречи с великими людьми. До этого я никогда не встречал великих людей в большом количестве. Существовал консультативный комитет, который должен был способствовать нашему продвижению и помочь в конце концов решить, каким способом разделить уран. В этом комитете были такие люди, как Комптон и Толмен, и Смит, и Ури, и Раби и в довершение всего Оппенгеймер. Я принимал участие в заседаниях, поскольку понимал теорию того, как идет процесс разделения изотопов, так что многие задавали мне вопросы, и мы обо всем этом беседовали. Обычно при таких обсуждениях кто-нибудь делал какое-то утверждение. Тогда Комптон, например, выдвигал противоположную точку зрения. Как правило, он говорил, что то-то и то-то будет протекать так-то и так-то, и был совершенно прав. Кто-то еще заявлял: ну что ж, может и так, но существует и другая возможность, поэтому нужно рассмотреть такой-то вариант.

В результате все сидевшие за круглым столом были не согласны друг с другом. Я удивлялся и огорчался, что Комптон не повторяет свои утверждения и не настаивает на них. Наконец, Толмен, который был председателем, изрекал: «Итак, выслушав все аргументы, я полагаю, следует принять, что аргумент Комптона — наилучший, а теперь нам пора двигаться дальше».

Меня поражало, что комитет способен обсуждать такое множество идей, причем каждый представляет какую-то свою грань, и в то же время помнит, что сказали другие. В итоге принимается решение о том, чья идея наилучшая, — все обсуждение суммируется без повторения каждого пункта по три раза. Это были действительно великие люди.

Если у вас нет команды гениев, работающих над проектом национального масштаба, не стоит полагаться на всеобщее понимание — лучше сказать (или написать!) все трижды, поделиться этим со всеми и разместить там, где это невозможно не заметить. Все протоколы встреч, фоллоу-апы, обновления и синхронизации, по сути, делаются именно с этой целью.

Тем не менее мы видели немало примеров, когда данные принципы использовались неправильно: слишком много времени тратилось на бессмысленные обсуждения, а сама концепция встреч превращалась в настоящий кошмар. Все ритуалы, артефакты или как бы вы их ни называли не требуют много времени и не должны выполняться просто потому, что так написано в учебниках, — они служат конкретной цели: поддержанию фокуса. Не использовать их, как и использовать их чрезмерно, — оба подхода могут в долгосрочной перспективе серьезно повлиять на эффективность и устойчивость системы. Мы считаем, что ведение документации сильно недооценивается и ее важность крайне занижена.

Области ответственности должны быть документально зафиксированы для явного и недвусмысленного понимания всеми членами команды. Уровень формальности может варьироваться — от формата стартапов «Алиса полностью

отвечает за работу системы X» до сложной многостраничной иерархии. Сбалансированным подходом считается использование простой, но эффективной матрицы RACI, в которой участники делятся на четыре группы:

- *Исполнители (Responsible)* — лица, непосредственно выполняющие задачу, например ML-инженеры.
- *Ответственный за результат (Accountable)* — лицо, которое в итоге несет ответственность за выполнение задачи, как правило, менеджер или руководитель команды. Рекомендуется, чтобы за каждый компонент отвечал один человек, обладающий полномочиями одобрять или отклонять работу, например руководитель проекта или сеньор-разработчик.
- *Консультанты (Consulted)* — те, чье мнение учитывается, обычно это эксперты в какой-то предметной области, стейкхолдеры, имеющие особый интерес к задаче или представители юридического отдела.
- *Информируемые (Informed)* — лица, которых необходимо держать в курсе прогресса или принятых решений, но которые не участвуют в выполнении задачи напрямую, например фронтенд-разработчики.

Хорошо это или плохо, но всегда остается вероятность пробелов в распределении ответственности, поэтому целесообразно заранее описать явный механизм эскалации (обращения к вышестоящему менеджменту) — что делать, если ситуация не соответствует матрице или другому документу, связанному с распределением ответственности.

И последняя, пожалуй, наименее любимая часть системы ответственности — ротация дежурств. Быть на дежурстве означает, что конкретный человек обязан реагировать на критические инциденты в течение своей смены. Он должен быть готов быстро действовать в случае чрезвычайной ситуации, обычно в течение 30–60 минут. Ротация подразумевает, что эта обязанность, как правило, распределяется между всеми членами команды по очереди. Например, один человек может дежурить в течение недели, а затем передает свои обязанности коллеге на следующую неделю (рис. 16.2).

График дежурств обычно возникает по мере роста компании. Те, кто начинал карьеру или работал в небольших компаниях, помнят, что ротация там не требовалась, так как все постоянно находились на связи, а инциденты устранял тот, кто был в этот момент свободен. Однако в какой-то момент такой подход становится неуправляемым: ответственность разных членов команды размыта, и в итоге инциденты как бы решают все и одновременно никто. Как только вы начинаете замечать подобную ситуацию, пора утвердить официальный график дежурств. В дальнейшем может возникнуть необходимость в отдельной команде дежурных инженеров с многоуровневой структурой (например, L1/L2/L3 и т. д.).

Но даже если у вас уже есть график дежурств, могут быть и другие причины перегрузки команды. Арсений как-то работал в еще только начавшей свое развитие



Рис. 16.2. Ротация дежурств — необходимая мера, чтобы не допустить выгорания команды

компании, где сталкивался с инцидентами, которые не удавалось решить просто потому, что он не понимал, какие именно системы затронуты. Это доставляло множество проблем, пока инженеры, разрабатывавшие и обслуживавшие эти системы, наконец не написали качественные инструкции с рекомендациями по типовым проблемам. Технический долг был частично устранен, и дежурства перестали быть кошмаром, превратившись в обычную, хотя и не самую приятную обязанность. Когда инциденты уровня L1 перешли в ведение специальной команды, затраты времени на дежурства со стороны ML-команды значительно сократились.

Разумеется, решать какую-то проблему в субботу в 3:00 ночи — не особо приятно, и это придает дополнительную мотивацию создать надежную и хорошо отслеживаемую систему, способную минимизировать сбои, и заранее составить расписание дежурств, чтобы все знали, кто и когда находится на связи. Любой инженер, не понимающий ценности качественного логирования и наблюдаемости (observability), должен находиться на дежурстве до тех пор, пока не изменит свое мнение.

Мы убеждены, что человек, отвечающий за проектирование и внедрение системы, также несет ответственность за ее поддержку и сопровождение. Он лучше всех знает детали и может спрогнозировать крайние случаи и предложить кратчайшие пути для их устранения. Это не означает, что он обязан быть единственным дежурным, но он должен подготовить систему к дежурствам и снабдить дежурную команду всеми необходимыми инструментами и документацией.

К таким инструментам относятся:

- логи, метрики, оповещения и дашборды;
- конфигурация системы и сопутствующей инфраструктуры;
- исходный код системы и документация.

Доступ к продакшен-данным — вопрос уже более деликатный. Это не всегда требуется для устранения проблемы, и мы сталкивались со случаями, когда политика конфиденциальности ограничивала такой доступ. Однако чаще всего полезно его иметь для анализа инцидента и выявления первопричины. В противном случае логи и метрики должны быть достаточно подробными, чтобы заменить этот доступ.

Обязательно должен существовать ранбук с комментариями о том, как устранять наиболее распространенные проблемы, а также список людей, к которым можно обратиться при необходимости (например, может произойти сбой на стороне поставщика, и дежурному инженеру нужно будет связаться с техническим директором, который поддерживает контакт с этим поставщиком). Многие проблемы цикличны и являются отражением либо технического долга, либо характерного паттерна использования. Например, имеется крупный клиент с нерегулярными пиковыми нагрузками, из-за чего система не справляется с объемом запросов. Для такого сценария дежурному инженеру потребуется четкая инструкция по запуску дополнительных экземпляров системы и по обратному масштабированию, когда нагрузка вернется к норме.

Возникающие при эксплуатации проблемы порождают два следствия: решение и обучение. Чтобы избежать повторения одних и тех же ошибок, необходимо организовать процесс извлечения уроков из сбоев. Это можно реализовать через ретроспективы и постмортемы. Мы рекомендуем придерживаться принципа соразмерной реакции — набора действий после инцидента, сопоставимых с его масштабом и учитывающих вероятность его повторения. Некоторые инциденты требуют всего лишь 10-минутного обсуждения и добавления нового абзаца в ранбук. Арсений однажды спровоцировал настолько крупный сбой, что технический директор инициировал кампанию под названием «Гонка за надежными релизами», в которую были вовлечены все инженерные команды, и надежность всех систем, а не только сломанной Арсением ML-системы, значительно повысилась.

16.2. Фактор автобуса

Фактор автобуса — это мера риска срыва проекта из-за потери одного из участников команды. Термин *«фактор автобуса»* происходит из гипотетической ситуации, при которой участника команды сбивает автобус, в результате чего он внезапно и неожиданно исключается из проекта.

Теорема CAP утверждает, что любое распределенное хранилище данных может одновременно обеспечить лишь два из следующих трех свойств:

- *Согласованность (Consistency)* — каждый запрос на чтение получает самые актуальные данные либо ошибку.
- *Доступность (Availability)* — каждый запрос получает корректный ответ, но нет гарантии, что это самые свежие данные.
- *Устойчивость к разделению (Partition tolerance)* — система продолжает работать, даже если нарушено взаимодействие между ее узлами.

Когда возникает отказ из-за разделения сети, необходимо сделать выбор: либо отменить операцию ради поддержания согласованности ценой снижения доступности, либо выполнить операцию, сохранив доступность, но, возможно, нарушив согласованность.

Мы рассматриваем структуру команды как нечто очень близкое к данному принципу, но с другими критериями: эффективность и избыточность. В computer science под избыточностью (redundancy) понимается наличие дополнительных или дублирующих ресурсов, поддерживающих основную систему. Это запасные или резервные ресурсы, которые могут быть задействованы в случае отказа основной системы. Такие ресурсы считаются избыточными, поскольку не используются при нормальной работе.

> **ПРИМЕЧАНИЕ** Инженеры часто склонны решать организационные проблемы техническими методами (другими словами, относиться к людям как к ПО) просто потому, что в этом они сильны. Иногда такой подход вполне оправдан, так как написать код может быть проще, чем наладить взаимодействие с людьми и прийти к единому пониманию. Однако сеньор-разработчик не должен быть подобен молотку, видящему везде только гвозди, — он должен выбирать правильный инструмент для каждой задачи, и иногда этот инструмент вовсе не технический.

Как проблема избыточности/эффективности влияет на структуру команды? Как показывает практика, схема «компания–команда–проект» обычно развивается от высокой эффективности к высокой избыточности. Однако чрезмерная эффективность, как и чрезмерная избыточность, имеет свои недостатки.

16.2.1. Чем опасна чрезмерная эффективность?

Если вы чрезмерно эффективны и решаете множество задач малым числом людей, то находитесь в зоне риска. Каждый человек в команде становится незаменимым не только из-за своего опыта, но и просто потому, что работает на пределе. Если с кем-то из членов команды что-то случится, компания (или проект) окажется в бедственном положении. Единственный способ справиться с возникшими проблемами без потери эффективности — это акты самопожертвования, которые не масштабируются и считаются антипаттерном (https://mng.bz/YVyA) (рис. 16.3).



Рис. 16.3. Обратная сторона сверхэффективности — высокая уязвимость к внешним факторам

16.2.2. Чем опасна чрезмерная избыточность?

В отличие от эффективности, основное преимущество избыточности — это дополнительная мощность, обеспечивающая надежность, устойчивость, возможности для роста и запас прочности на случай кризисов. Однако избыток резервов ведет к небрежности, снижает доверие между членами команды и отпугивает сильных специалистов, так как создает ощущение бессмысленности и отсутствия значимости работы.

Оба приведенных случая являются крайностями, которых следует избегать. Как только вы замечаете, что команда приближается к пределу своей нагрузки, стоит задуматься о ее расширении. Такое решение принесет пользу в долгосрочной перспективе, несмотря на дополнительные затраты в настоящем.

То же самое и с другой стороной спектра — мощностей должно быть достаточно, чтобы не работать на износ, но и быть готовыми к возможным всплескам нагрузки.

16.2.3. Когда и как учитывать фактор автобуса

Необходимо сохранять баланс между эффективностью и избыточностью, а первым шагом к управлению чем-либо является измерение. Одной из простых и известных метрик является фактор автобуса.

Фактор автобуса рассчитывается как количество членов команды, потеря которых приведет к невозможности продолжения проекта. Например, если фактор автобуса проекта равен 1, то при потере одного (конкретного) человека проект окажется в критической ситуации.

Очевидно, фактор автобуса 1 — крайне нежелателен, а фактор в 10 000 может оказаться чрезмерным (впрочем, такое возможно, например, в службе поддержки клиентов, где требуется множество сотрудников с одинаковыми функциями). Конкретное значение зависит от множества переменных: значимости проекта, ограничений бюджета, сроков, ожидаемой текучести кадров и степени уверенности/тревожности владельца проекта. Однако как только у вас есть список ответственных за различные части проекта или системы, вы можете рассчитать фактор автобуса, включив в расчет области ответственности и людей, за них отвечающих. Следующий шаг — выяснить, сколько других сотрудников достаточно хорошо знают эту область. Как правило, человек, ответственный за что-то, знает, кто еще способен (в той или иной степени) заменить его. Исходя из этой информации, можно оценить потенциальные риски и принять решения по найму, перераспределению или коллаборации, чтобы обеспечить обмен знаниями и снизить уязвимость проекта.

Конечно, нельзя четко разделить людей на тех, кто «знает, как работает система», и тех, кто «не знает». Обычно знание системы имеет свои степени, и даже если невозможно держать в курсе всех инженеров, можно распространить знания хотя бы частично, что может быть реализовано, например, через ревью архитектуры,

кода, документации, парное программирование и открытые часы, во время которых ключевые сотрудники делятся своим опытом.

Так можно достичь баланса между двумя критериями. Однако на практике при принятии решений приходится учитывать и другие параметры (например, не только численность команды, но и соотношение опытных и неопытных специалистов, а также их знакомство с конкретными технологиями).

Тем, кто заинтересован в простой, но практичной модели анализа компромиссов между затратами, эффективностью системы и человеческими ресурсами, рекомендуем ознакомиться с моделью отказов Расмуссена, хорошо описанной в статье по ссылке https://mng.bz/GNqO. В отличие от предыдущего текста, сосредоточенного больше на структуре команды, эта статья отражает подход SRE (Site Reliability Engineering — техническое обеспечение надежности сайта) к системным компромиссам: несмотря на различия между людьми и оборудованием, при поиске оптимального баланса прослеживаются схожие закономерности.

16.3. Документация

Мы уже не раз упоминали важность документации. К сожалению, ее значение часто сильно недооценивается, и она не получает должного внимания. Ирония состоит в том, что слово «документация» часто соседствует с такими словами, как «потом», «не сейчас» и «не срочно». А когда она действительно становится нужна, бывает уже либо слишком поздно ее составлять, либо она оказывается безнадежно устаревшей.

Важность документации невозможно переоценить. Она служит средством передачи знаний и онбординга новых членов команды, помогая сгладить кривую обучения для новичков и обеспечивая устойчивость системы. Полноценная документация позволяет человеку за пределами команды понять и запустить систему.

Байка от Валерия

Я работал в разных компаниях: в офлайн- и онлайн-торговле продуктами питания, на маркетплейсах, в социальных сетях и финтехе. Одним из достижений, которым я особенно горжусь, является то, что — независимо от размера компании, рынка и стадии развития — построенные мной системы продолжали успешно работать и после моего ухода. Это удавалось благодаря плавной кривой обучения новых сотрудников, подробной документации, тесной интеграции с другими отделами и четкого понимания, кто за что отвечает.

Однажды я получил сообщение от бывшего коллеги из одной Big Tech компании с вопросом о том, как воспроизвести конкретный компонент системы (языковую модель, способную генерировать отзывы о продаваемых товарах). Мой ответ был кратким: вся документация по данному проекту находится под определенным тегом, и действительно — она там была! (Честно говоря, ему даже не нужно было меня об этом спрашивать.) Она была не только общедоступна, но и позволяла воспроизве-

сти весь процесс развертывания одной командой или, при необходимости, изучить репозиторий кода и датасеты.

А вот совершенно иной пример. Однажды я работал в компании, где нам нужно было воссоздать ончейн-анализ блокчейна биткоина, который был выполнен 2 года назад, но часть источников изменилась и потребовала замены/обновления. К сожалению, практически никакой документации не было, и то, что должно было занять максимум 2 недели, растянулось почти на 3 месяца (ончейн-анализ — это не самая простая задача, например, вы можете отправить 0.1 BTC, но это будет выглядеть как отправка 1 и получение 0.9, что сильно отличается от реальной ситуации). Вскоре после завершения проекта бóльшая часть команды была уволена, как и 30 % всего штата компании. Это было уже второе сокращение за 6 месяцев. В сумме численность компании сократилась почти на 60 %. Трудно сказать, была ли главной причиной именно нехватка документации, но я уверен, что повторное изобретение велосипеда (в данном случае — повторное прохождение одного и того же этапа) вряд ли является самым эффективным способом использования ресурсов. Именно поэтому кэширование так широко применяется в computer science. К сожалению, сокращение произошло, когда проект уже завершился, но документация еще не была написана.

Документация также способствует повышению надежности за счет снижения негативных последствий от ухода каких-то членов команды (фактор автобуса) или при приеме новых сотрудников. Более того, она облегчает сотрудничество между разными отделами и стейкхолдерами, формируя общее понимание системы (ответственность) и позволяя каждому инженеру быть более автономным в своей работе, а значит, деятельным и продуктивным.

И хотя составление документации требует дополнительного времени и усилий на начальном этапе, в долгосрочной перспективе это только сэкономит ресурсы. Документация снижает зависимость от неформальных знаний, предотвращает дублирование работы и минимизирует риск дорогостоящих ошибок и задержек, вызванных отсутствием информации. Документирование процессов, процедур, конфигураций и лучших практик позволяет команде работать более эффективно и создает основу для постоянного совершенствования. Документация должна с самого начала быть одним из приоритетов, а не второстепенной задачей. Признавая ее важность и выделяя необходимые ресурсы, команды могут создавать устойчивые системы, способные адаптироваться к изменениям и продолжать приносить пользу.

В некотором смысле вся наша книга посвящена тому, почему следует и как писать определенный тип документации, фиксирующий важную информацию о проектировании системы, архитектуре, деталях реализации и эксплуатации. Однако дизайн-документ — это весьма специфичная форма документации, которая не стремится охватить все аспекты системы, а скорее служит обзором, помогающим наладить связь между командой разработчиков и стейкхолдерами. Тем не менее всегда есть место и для других видов документации, включая:

- *Руководства пользователя (мануалы и гайды)*, объясняющие, как пользоваться системой, и предоставляющие пошаговые инструкции для типичных задач, таких как подготовка данных, обучение модели и получение прогнозов.
- *Поваренные книги (Cookbook)*, предлагающие простые способы достижения целей с использованием системы, а также содержащие фрагменты кода, примеры данных и конкретные шаги к определенным результатам.
- *Низкоуровневые технические спецификации и документацию по API* для разработчиков, описывающие технические детали ML-системы и содержащие рекомендации по взаимодействию с системой на уровне программного кода с использованием функций и API.
- *Руководства по установке и конфигурации*, предоставляющие инструкции по настройке сред ML, а также объясняющие зависимости, этапы установки и параметры конфигурации ПО.
- *Часто задаваемые вопросы (FAQ) и руководства по устранению неполадок*, рассматривающие типовые вопросы по использованию инструментов ML и предлагающие решения или обходные пути для распространенных проблем, возникающих на этапах обучения, тестирования или развертывания.
- *Примечания к релизам*, описывающие изменения, внесенные в новых версиях ПО, объясняющие новые функции, исправления ошибок, улучшения эффективности или другие известные проблемы и информирующие пользователей о влиянии новых релизов на рабочий процесс.
- *Отчеты об анализе предыдущих инцидентов*, в которых анализируются и документируются причины прошлых проблем или сбоев, фиксируются извлеченные уроки и меры по предотвращению аналогичных инцидентов в будущем, что повышает надежность системы и доверие пользователей к ней.

С самого начала выделяя время и усилия на создание и обновление документации, команды могут снизить риски, связанные с дефицитом знаний и зависимостью от конкретных людей. В нашей книге мы рассматривали два дизайн-документа, которые, как мы надеемся, дали вам подробное представление об обеих системах.

16.4. Сложность

> Сделай настолько просто, насколько это возможно, но не проще.
>
> *Приписывается Альберту Эйнштейну*

Согласно второму закону термодинамики, энтропия изолированной системы, предоставленной самой себе, не может уменьшаться со временем. Первоначально термин *«энтропия»* использовался в физике, но позднее был адаптирован

и к computer science с почти тем же значением: энтропия количественно выражает уровень неопределенности или случайности.

Говоря метафорически, ПО и ML-системы подчиняются тому же самому закону: с течением времени энтропия может только возрастать. В какой-то момент система становится слишком сложной для управления, и именно поэтому старые системы часто декомпозируют на наборы более мелких — вспомните фундаментальную теорему разработки ПО из раздела 13.1. Это делается для того, чтобы ими могли управлять команды достаточно умных людей, а не только исключительные гении. Добавление нового уровня абстракции «прячет» энтропию, но на самом деле не снижает ее уровень в системе.

Существует множество выражений, отражающих одну и ту же идею: подход YAGNI («You Aren't Gonna Need It» — «Тебе это не понадобится») из экстремального программирования, принцип KISS («Keep It Simple, Stupid» — «Делай проще, тупица»), зародившийся в ВМС США в середине XX века, и даже «бритва Оккама», появившаяся в Средневековье. Все эти идеи преследуют одну цель — ограничение сложности.

В то же время разработка ПО нередко склоняется в противоположную сторону, что выражается в акронимах вроде DRY («Don't Repeat Yourself» — «Не повторяйся»). На первый взгляд это явное противоречие: следование принципу DRY предполагает введение дополнительных слоев абстракции, а это как раз и является (одним из, а может и единственным) источником сложности.

Представьте, что вам нужно написать простой фрагмент кода, вычисляющий известную метрику по нескольким датасетам. Стандартный код будет содержать три функции: одна читает данные, вторая рассчитывает метрику, а третья управляет выполнением — запускает цикл из обеих функций и сохраняет результат.

Теперь представим две крайности: с одной стороны, слишком простой код, с другой — слишком сложный, при этом оба кода выполняют одну и ту же задачу. Первый вариант мог бы быть написан человеком, не знакомым с циклами и функциями, он просто следовал пошаговой инструкции: считать файл A, вычислить метрику для A, сохранить метрику для A, считать файл B, и т. д. Во втором случае код может содержать метапрограммирование и другие сложные конструкции. Несмотря на их различия, по сути, оба подхода ухудшают читаемость кода.

Это довольно банальный пример, который можно найти в любом учебнике по разработке ПО, посвященном написанию чистого кода. Однако тот же принцип применим и в более широком масштабе. Представим следующую задачу: ваша компания хочет помочь команде поддержки клиентов расставить приоритеты, чтобы в первую очередь обрабатывать самые срочные обращения от наиболее недовольных клиентов.

Типичное решение подобной проблемы сегодня — использовать LLM с настраиваемым промтом для классификации сообщений по нескольким уровням

срочности. До прихода LLM оптимальным подходом было бы построение простой модели (например, логистической регрессии на базе мешка слов), которая обеспечивала бы приемлемую точность классификации для большинства срочных обращений. При этом требовались бы дополнительные инженерные усилия: построение пайплайнов данных для получения обучающей выборки и ее разметки при участии команды поддержки, интеграция модели с CRM-системой для распространения меток и их визуализации в пользовательском интерфейсе и т. д.

Чрезмерно простое решение в данном случае — это построение базовой модели на основе нескольких условий `if` и регулярных выражений. Такое решение действительно будет какое-то время работать, но довольно скоро ваши коллеги будут так же недовольны, как и клиент, дважды оплативший товар из-за сбоя системы, потребовавшего отката транзакции, при этом ваша модель почему-то классифицировала тикет как «низкий приоритет».

С другой стороны, чрезмерно сложное решение может включать множество компонентов, объединенных одним словом — «*нерелевантные*». Можно использовать сложные модели обработки естественного языка, обученные с нуля, дополнить их исследованиями для реализации few-shot сценариев, а затем довести до совершенства с помощью слоя калибровки. Сложность может быть не только в области ML, но и в разработке ПО: вместо пары простых API-интерфейсов разработчик может применить все известные ему модные технологии, сделав систему столь же отказоустойчивой, как прошивка космического корабля, и столь же масштабируемой, как поисковый движок Google, игнорируя тот факт, что система будет обрабатывать лишь несколько десятков сообщений в день.

От сложности страдают все виды систем, включая непрограммные. Но дополнительный уровень сложности в ПО связан с его постоянной эволюцией. При строительстве дома вы вряд ли будете достраивать новые комнаты через месяц после завершения крыши. Однако добавление новых функций в программную систему — это распространенная практика. Сложность систем, ориентированных на данные, возрастает из-за постоянных проблем, связанных с дрейфом данных. ML-системы устроены еще сложнее — некоторые модели по своей природе являются нелинейными черными ящиками, что не облегчает задачу.

В завершение книги мы рекомендуем вам еще раз вернуться к главам 2 и 3, посвященным пониманию проблемы и предварительным исследованиям. Некоторая сложность неизбежна, но значительная часть избыточной сложности возникает из-за упущений именно на начальных этапах: плохое понимание проблемы, поверхностные исследования, постановка нерелевантных целей и игнорирование очевидных рисков. Все это может привести к цепочке неудачных решений и росту сложности.

Если нужно переправиться через реку, достаточно надежной весельной лодки. Можно даже оснастить ее мотором, чтобы ускорить переправу, но никто

не будет использовать ее для перевозки тысяч людей через океан (рис. 16.4). А те, кто начинают собирать круизный лайнер, чтобы проплыть всего пару миль, никогда не достигнут своей цели. Многие ML-системы терпят неудачу именно по этой причине: их разработчики либо пытаются развивать примитивную систему, добавляя к ней все новые «свистелки» и «пищалки», чтобы достичь чрезмерно амбициозных целей, либо начинают строить космический корабль без веских на то оснований — и их увольняют еще до завершения первой фазы проекта.

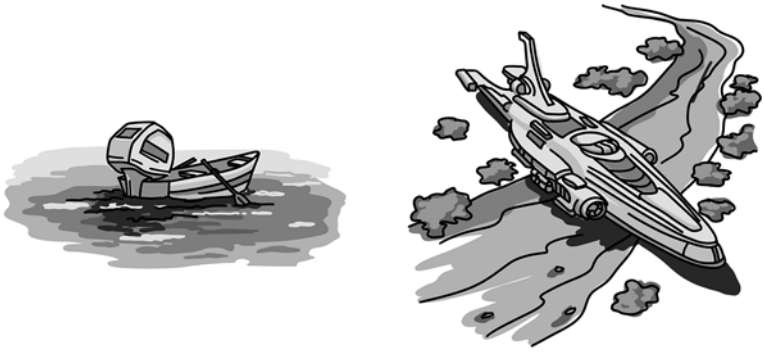


Рис. 16.4. Как базовая модель в итоге не сможет обеспечить нужный уровень эффективности и не достигнет целей проекта, так и чрезмерно сложное решение приведет к неэффективной трате ценных ресурсов бизнеса

Байка от Арсения

Я не уложился в срок завершения этой главы, потому что был слишком загружен на основной работе, в том числе из-за одного своего «умного» решения, принятого в прошлом. Три года назад мне нужно было протестировать часть системы, которая должна была запускать модель в двух средах: локальной и облачной. Тест был достаточно простым — запустить один и тот же код в обеих средах и убедиться, что результаты совпадают. Однако я хотел сэкономить несколько миллисекунд и заставил оба вызова выполняться параллельно в отдельных потоках. Это было «умно» и так эффективно! Прошли годы, кодовая база и инфраструктура изменились, добавились новые функции, и одна из этих инфраструктурных правок повлияла на то, как все должно работать в облаке. Можно было ожидать, что тест начнет сбоить. Однако в Python исключения в потоках неявные, и сбой теста в такой ситуации приводит к почти нерелевантному сообщению. В довершение всего тесты продолжали выполняться параллельно, что еще сильнее скрывало настоящую проблему. Излишняя сложность привела к тому, что я потратил целый день работы на выявление этой проблемы.

Эта история опускает некоторые инженерные детали, например нетипичную конфигурацию CI, но даже в упрощенном виде она демонстрирует, что не было одной-единственной причины потери времени — скорее это был своеобразный снежный

ком. Как отмечается в популярном списке «How Complex Systems Fail» (https://how.complexsystems.fail/): «Установление единственной причины сбоя изначально ошибочно». Не было единственного неверного решения — была комбинация неоптимальных выборов. И комбинаторика тут не на вашей стороне: даже если большинство решений, касающихся системы, оптимальны, огромное количество комбинаций в сложной системе приведет к росту числа инцидентов. Все, что вы можете сделать, — сократить это число и уменьшить последствия каждого отдельного сбоя.

16.5. Сопровождение и владение: Supermegaretail и PhotoStock Inc.

На протяжении всей книги мы завершали главы примерами соответствующих разделов дизайн-документов для компаний Supermegaretail и PhotoStock Inc. В данной главе это нецелесообразно, поскольку потребовало бы придумать пару десятков вымышленных имен и фамилий. Поэтому мы ограничимся лишь общими соображениями о том, как могло бы выглядеть сопровождение системы в данных кейсах.

Supermegaretail — это крупная корпорация с тысячами сотрудников, высоким уровнем бюрократизации и сложными рабочими процессами, при этом ключевые бизнес-единицы преимущественно не являются техническими. Поэтому важнейшую роль в успехе проекта играет координация всех этапов со всеми стейкхолдерами на каждом уровне иерархии. Основой эффективной эксплуатации системы после релиза будет подробная RACI-матрица, включающая сотрудников компании, участвующих в проекте, а также перечни согласований от команды поставки, руководства и бизнес-команд.

PhotoStock Inc., напротив, относительно небольшая, полностью технологическая компания, и специфика пользовательских поисковых движков, работающих с миллионами изображений, предполагает непрерывную нагрузку на систему с регулярными пиковыми нагрузками. Поисковый движок одновременно зависит как от ML, так и от инфраструктуры. Как было отмечено ранее, проект требует тесного взаимодействия между командами. Учитывая, что в системе четыре основных компонента (хендлеры API, индекс, модели ранжирования, внутренние инструменты), необходимо обеспечить следующее:

- За каждый компонент отвечает сеньор-разработчик.
- Сеньор-разработчики осведомлены о работе других компонентов.

Учитывая это, оптимальным решением будут две ступени дежурства:

- Основной дежурный инженер отвечает за первую реакцию на инцидент.
- Основной дежурный инженер может эскалировать конкретные проблемы инженерам, ответственным за соответствующие компоненты.

Как мы говорили в главе 4, дизайн-документ — это живой организм, и раздел о сопровождении заполняется в последнюю очередь не потому, что только к этому моменту формируется полное представление о ML-системе, но и потому, что к этому моменту уже установлены связи с представителями других команд и отделов и становится понятно, какие именно специалисты войдут в рабочую группу проекта.

Итоги

- Человек, ответственный за проектирование и внедрение системы, также несет ответственность за ее сопровождение и поддержку.
- Необходимо точно знать зоны ответственности и людей, назначенных на соответствующие роли. Еще важнее — напоминать этим людям об их ролях, а не надеяться на то, что все и так все понимают.
- Чтобы обеспечить своевременное реагирование на критические инциденты, обязательно составьте график дежурств.
- Всегда ищите компромисс между эффективностью и избыточностью, чтобы избежать выгорания сотрудников, но при этом быть способными справиться с пиковыми нагрузками.
- Все элементы документации на систему должны быть актуальны и доступны для всех заинтересованных сторон. Сначала это потребует дополнительных затрат времени и усилий, но в долгосрочной перспективе документация сэкономит ваши ресурсы.
- Начинайте с малого, улучшайте по одному-два элемента, завоевывайте доверие, а затем переходите к более масштабным изменениям. Вполне вероятно, что вы получите больше поддержки от коллег и соберете больше информации о системе, что поможет принимать более обоснованные решения.
- Старайтесь избегать чрезмерной сложности решений. Избыточная сложность может быть результатом личных амбиций или свидетельством слабого понимания проблемы, поверхностного исследования, постановки нерелевантных целей и игнорирования очевидных рисков на начальных этапах.

КРОК

СОЗДАЕМ НАСТОЯЩЕЕ,
ИНТЕГРИРУЕМ БУДУЩЕЕ

croc.ru

КРОК — технологический партнер с комплексной экспертизой в области построения и развития инфраструктуры, внедрения информационных систем, разработки программных решений и сервисной поддержки.

Центры компетенций КРОК фокусируются на ключевых отраслевых кластерах — промышленность, финансовый сектор, розничные продажи, муниципальное управление, спорт и культура.

Ежегодно сотни проектов КРОК становятся системообразующими для экономики и социально-культурной сферы.
